第一部分 硬件电路设计

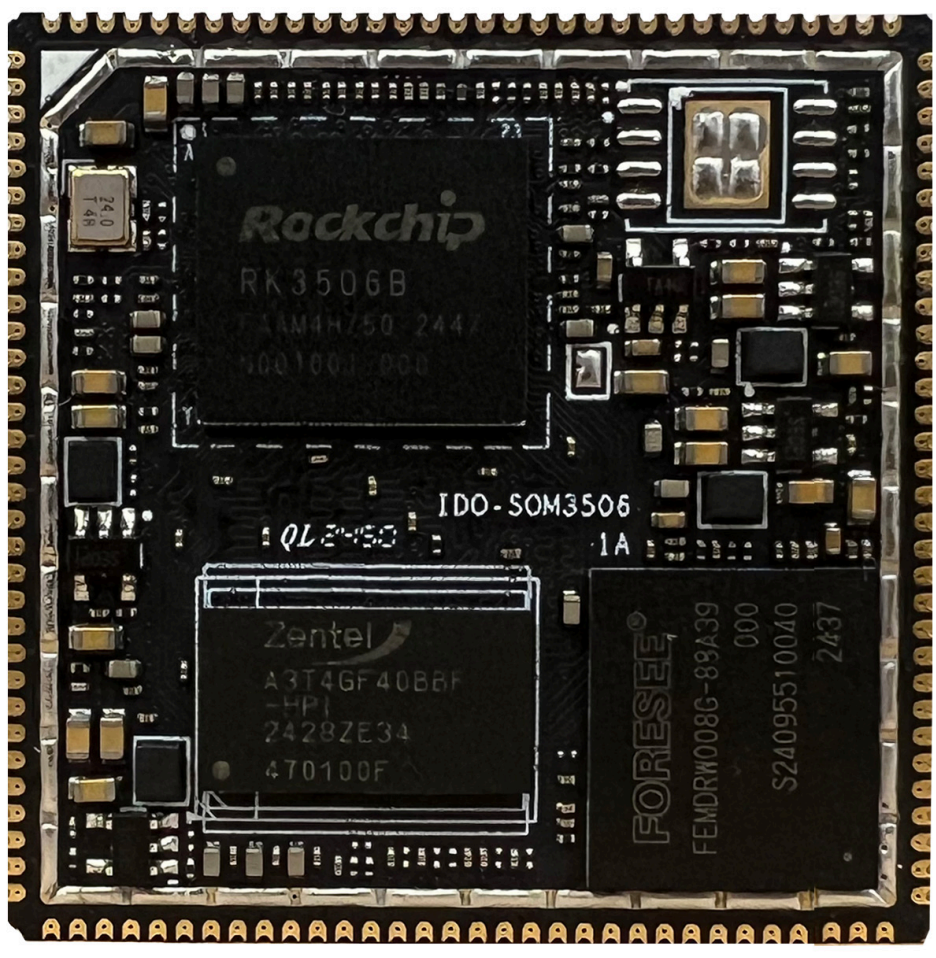
第01章 硬件电路设计

1.1 RK3506参考板

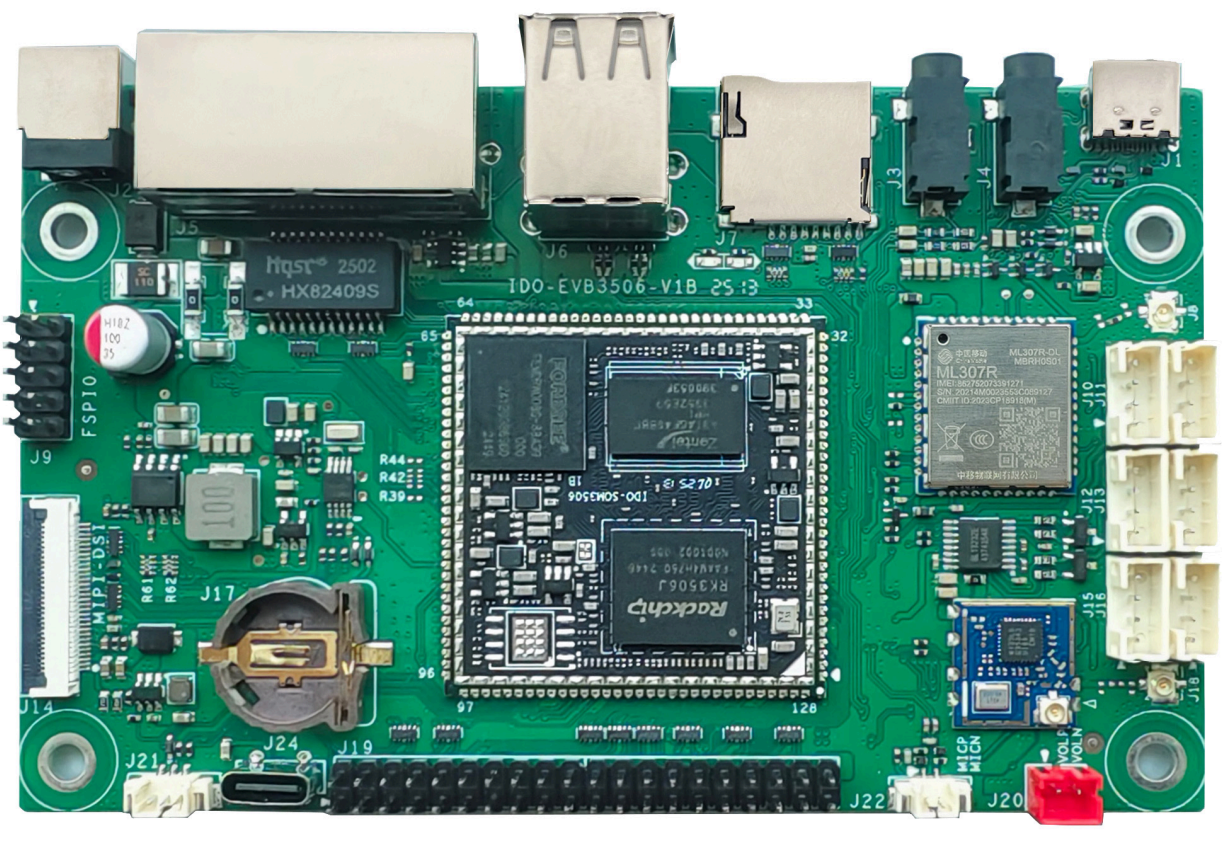
1. 触觉智能开发板资料

```
触觉智能RK3506核心板&配套评估 开发板

SOM3506-S1核心板
官网链接：http://www.industio.cn/product-item-121.html

EVB3506-V1开发板
官网链接：http://www.industio.cn/product-item-122.html

网盘链接：
EVB3506-Linux https://pan.baidu.com/s/1DNLKNJoM6OFy4OTWt7rK9w?pwd=1234
```

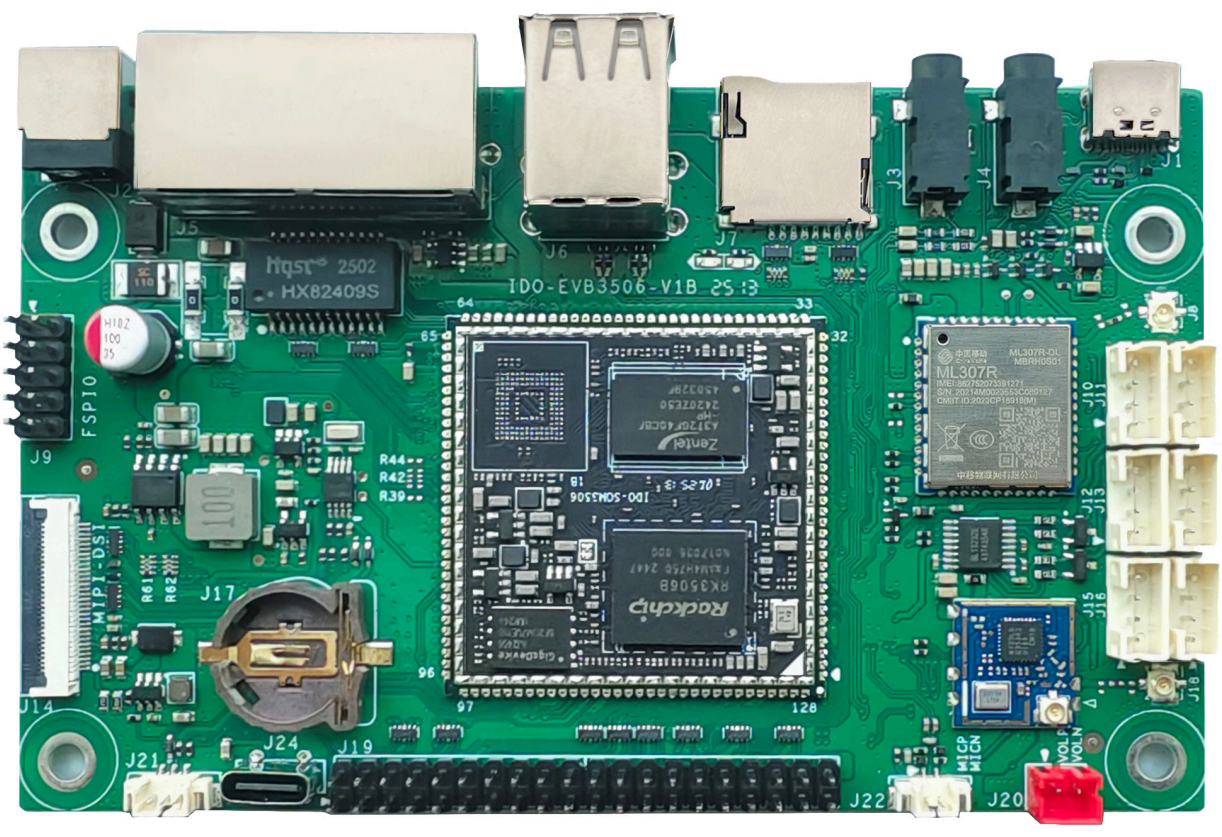
2. RK3506核心板



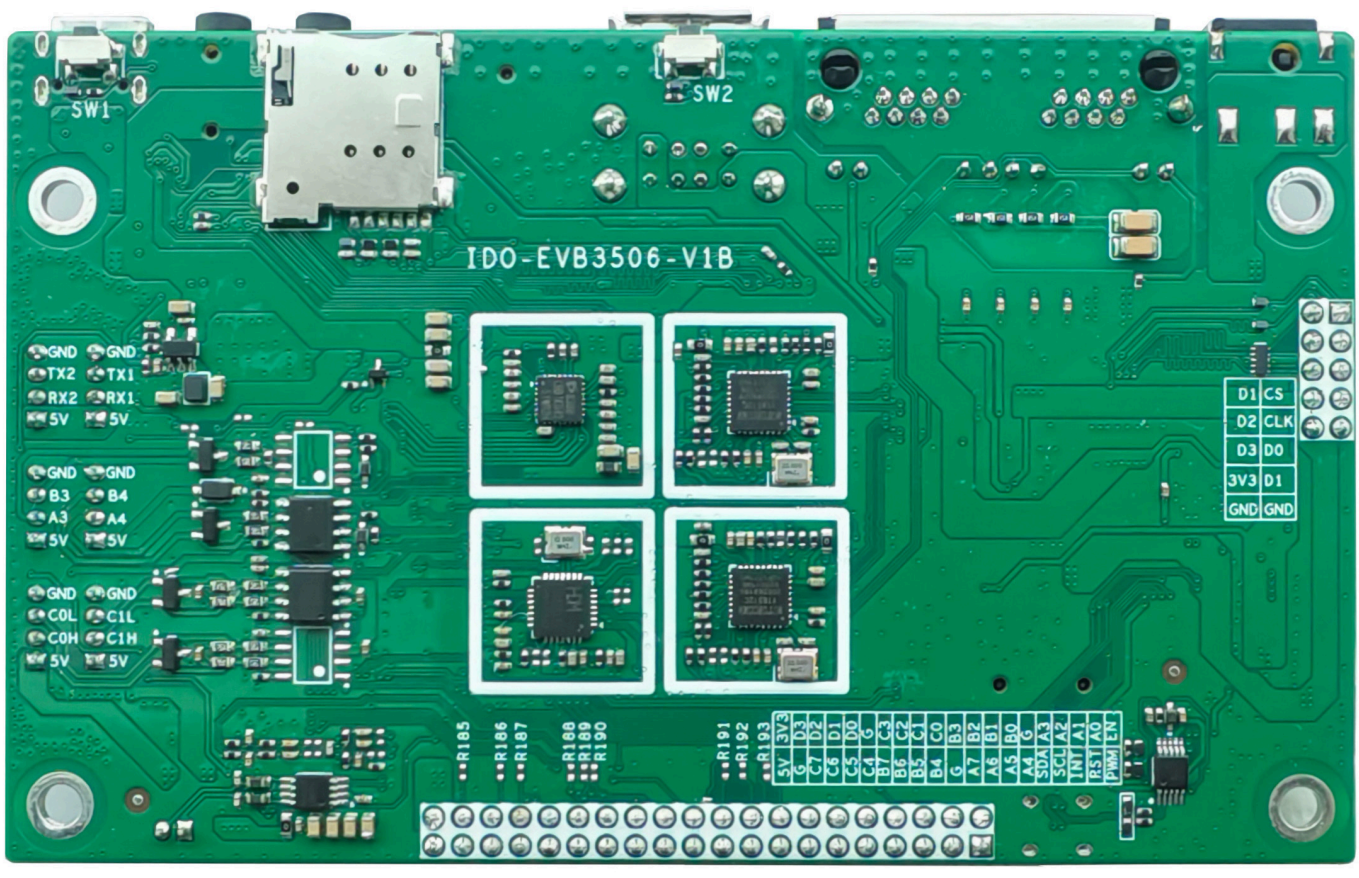
3. EMMC 版本主板



4. NAND版本主板

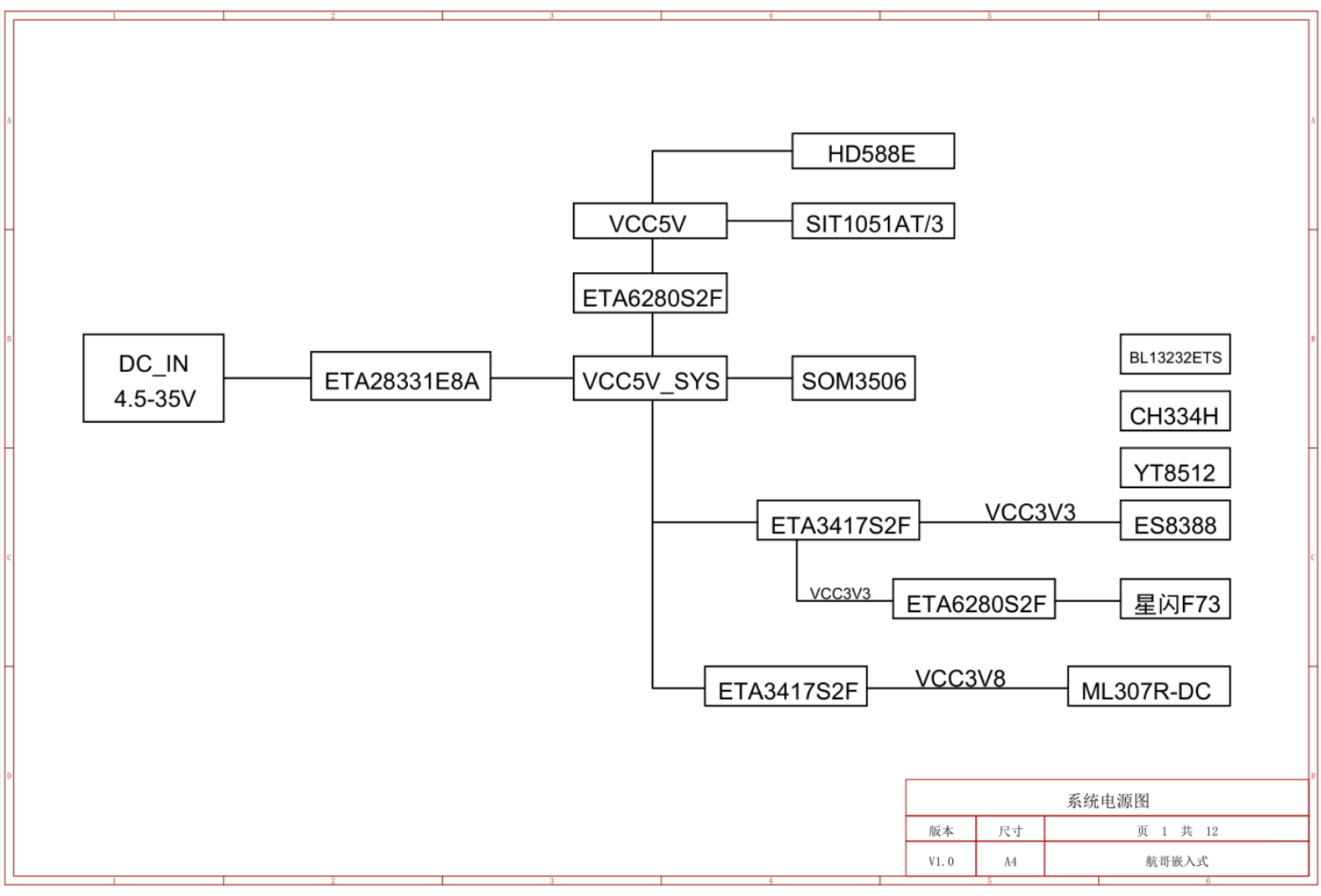


5. 主板背面图



1.2 原理图绘制

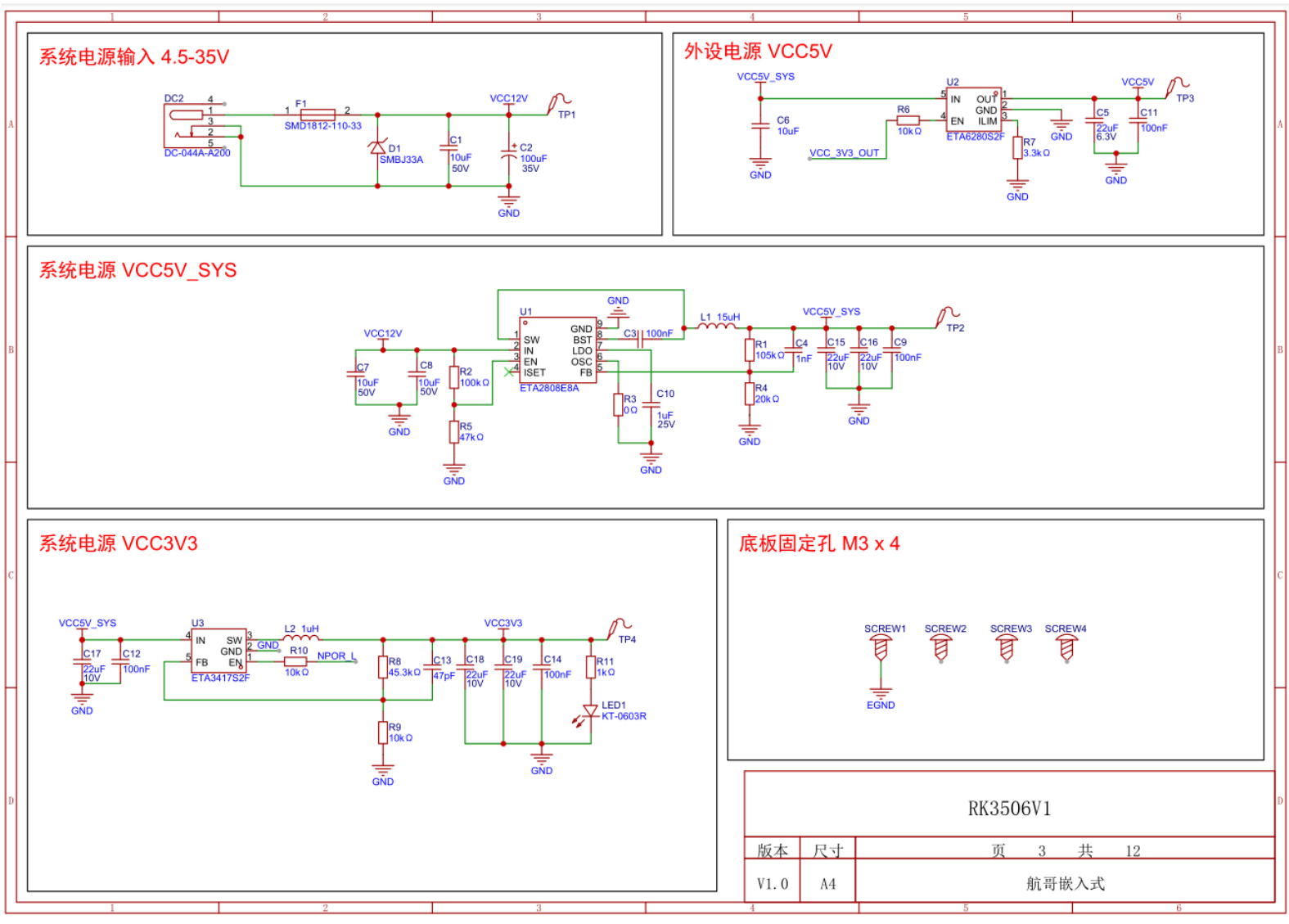
1. 系统电源图



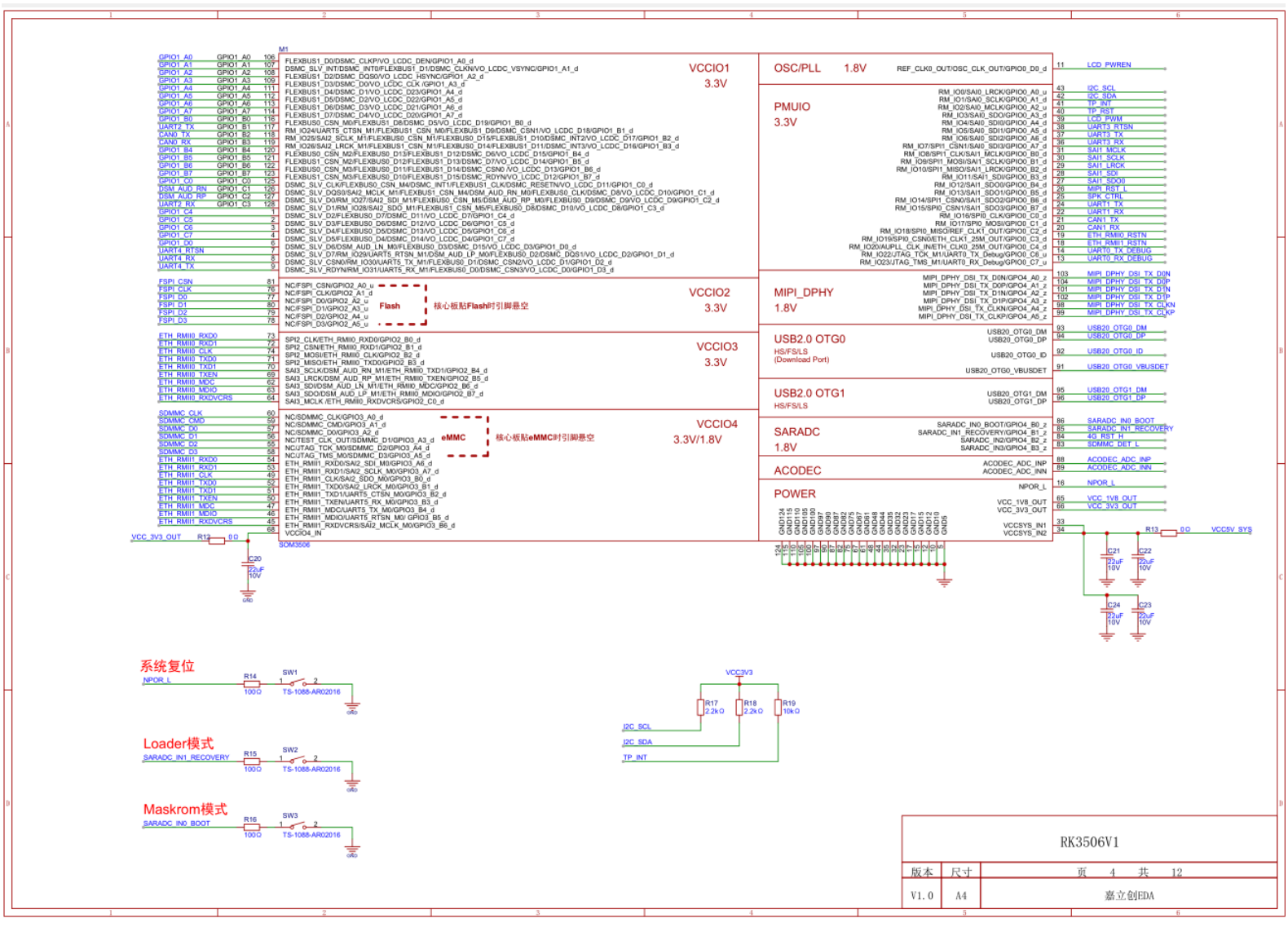
2. 系统功能图

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

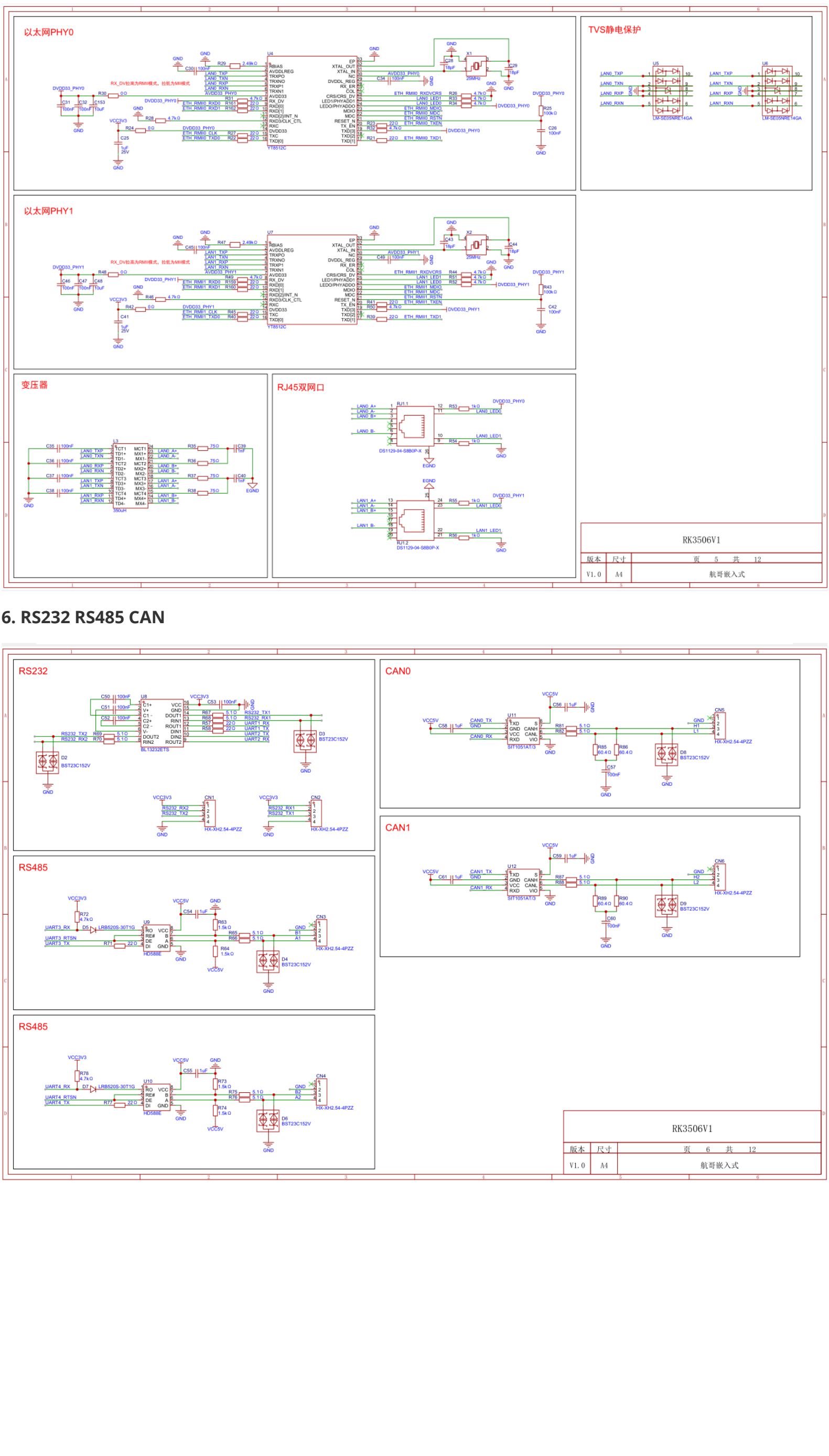
3. 系统电源



4. 核心板

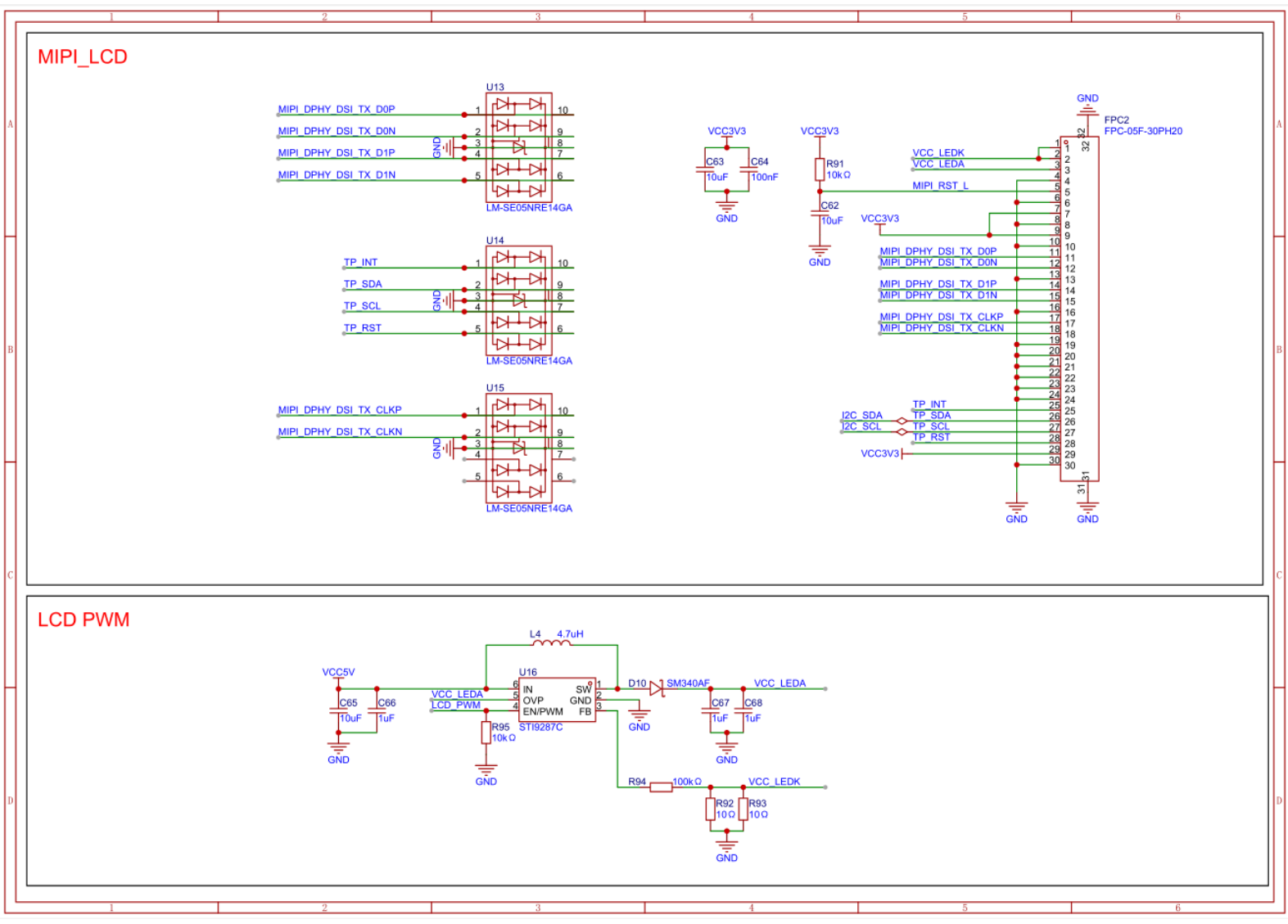


5. 以太网

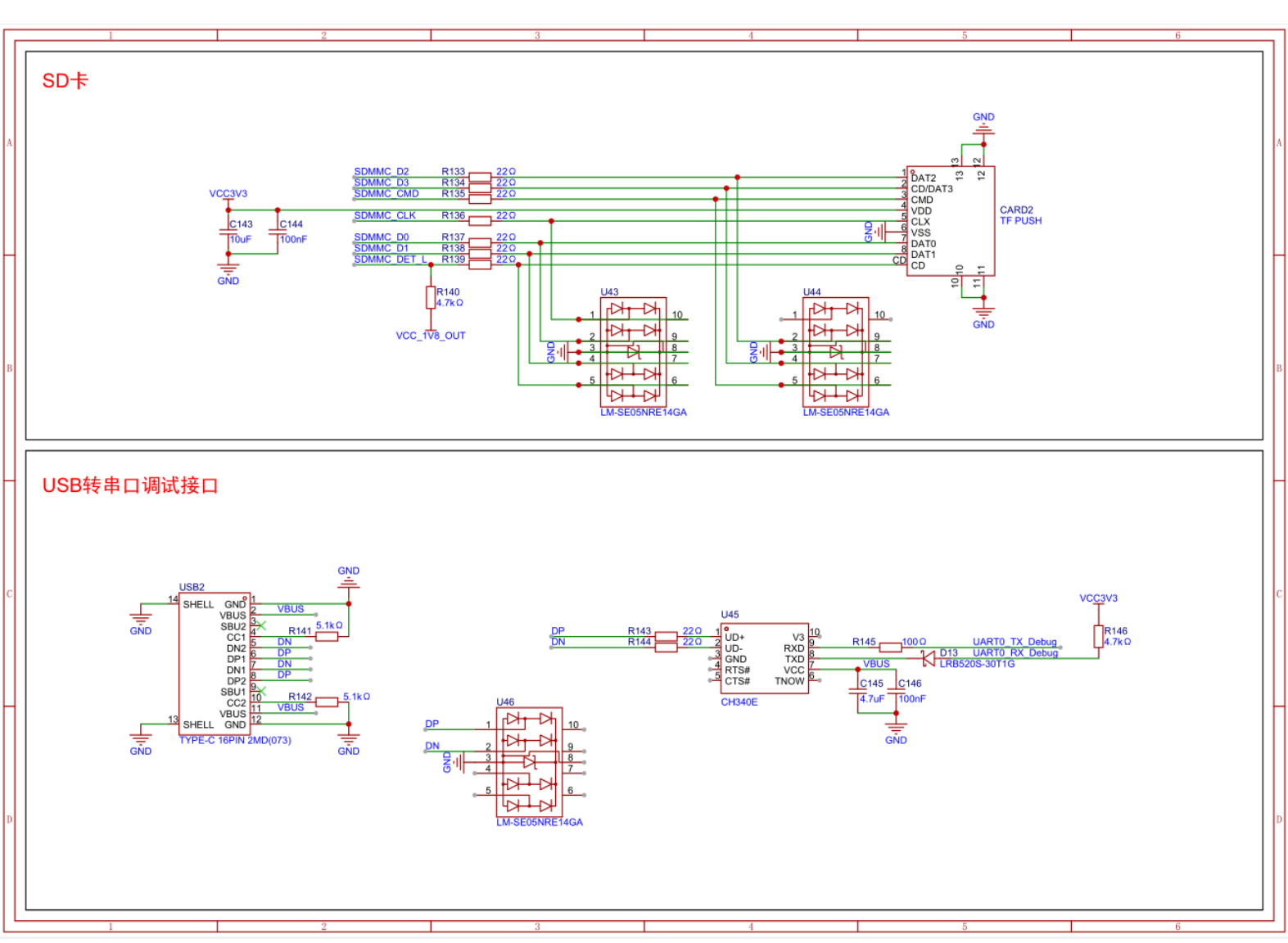


6. RS232 RS485 CAN

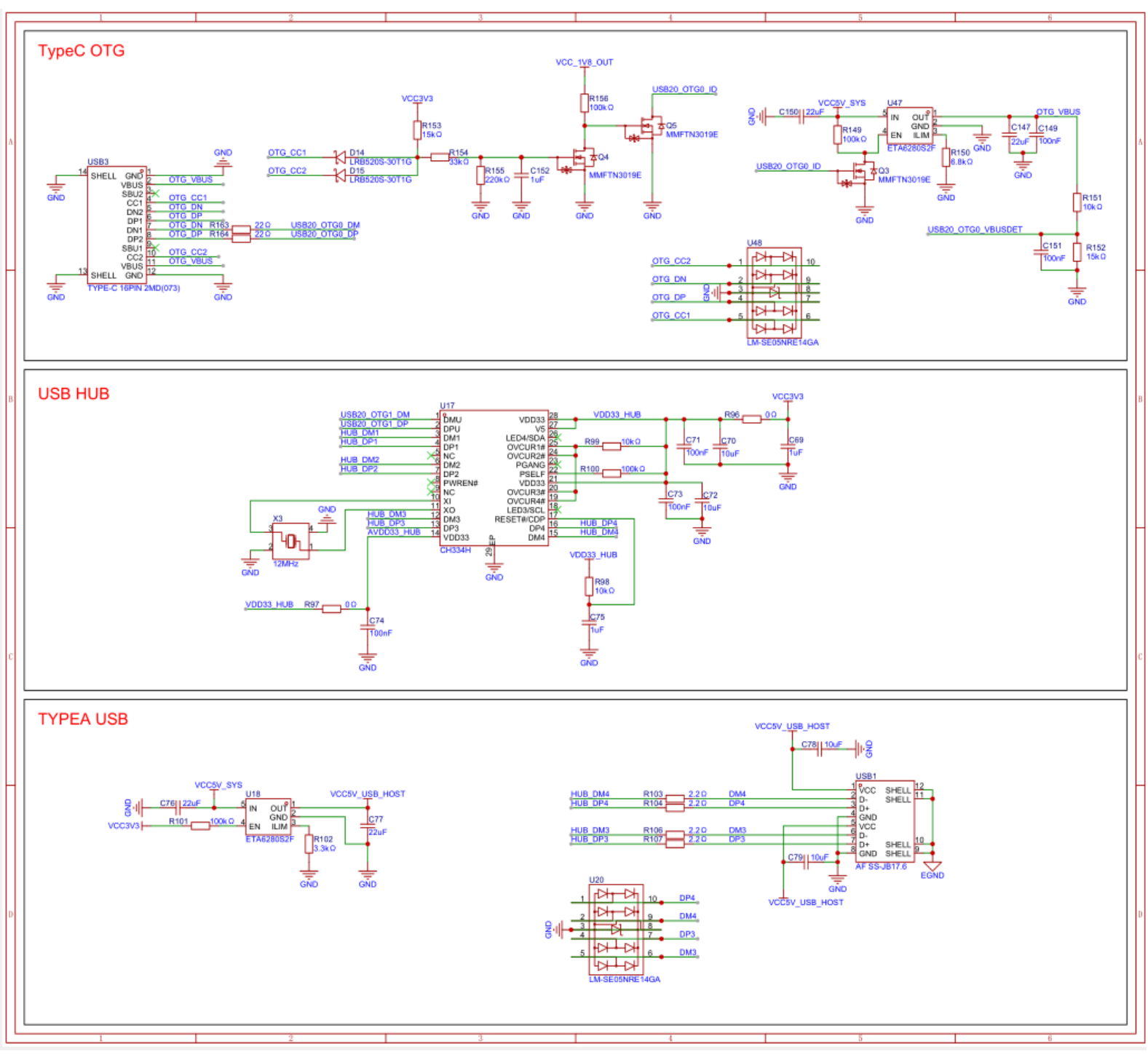
7. MIPI LCD



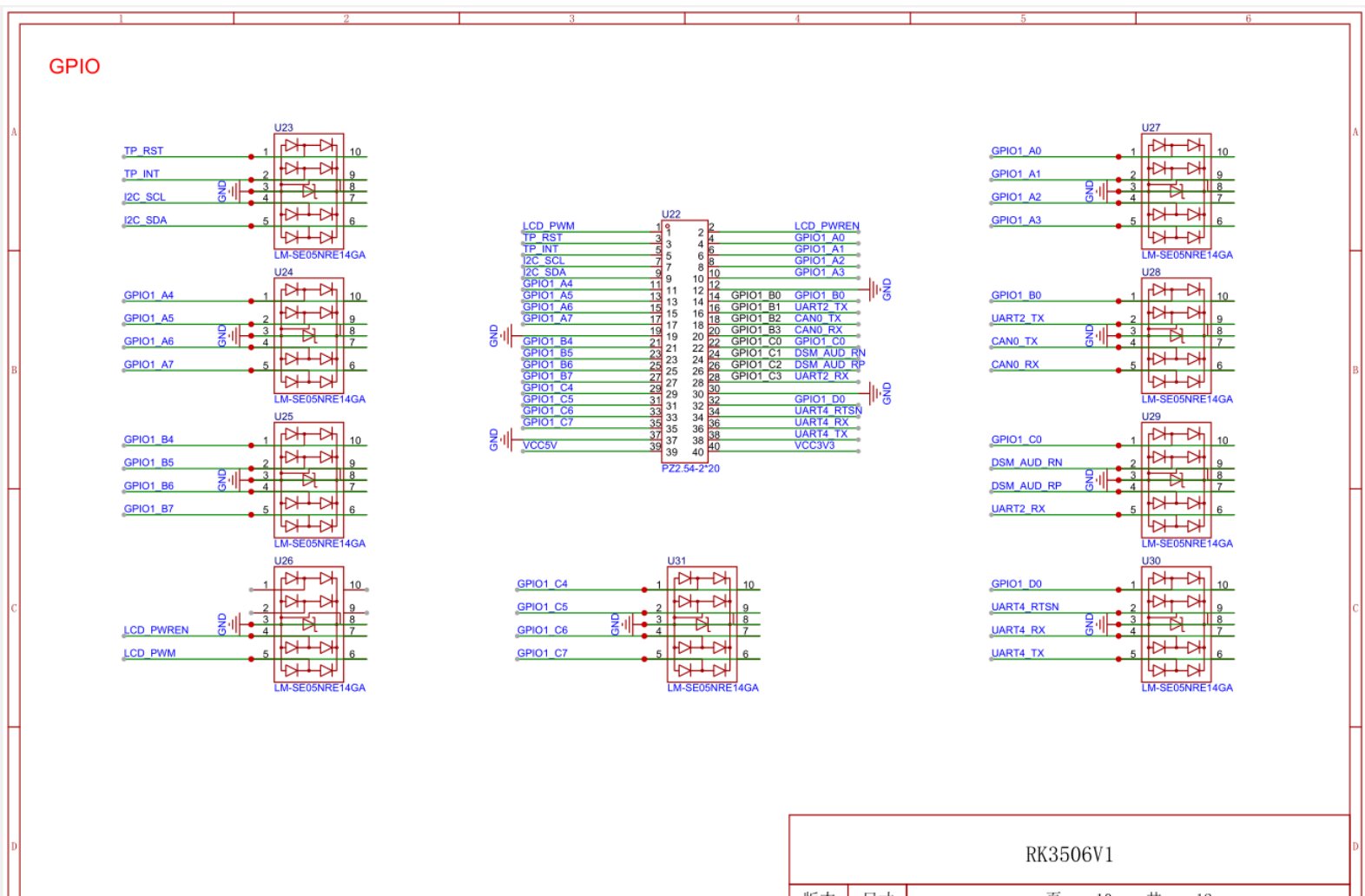
8. SD卡与调试接口



9. USB 接口

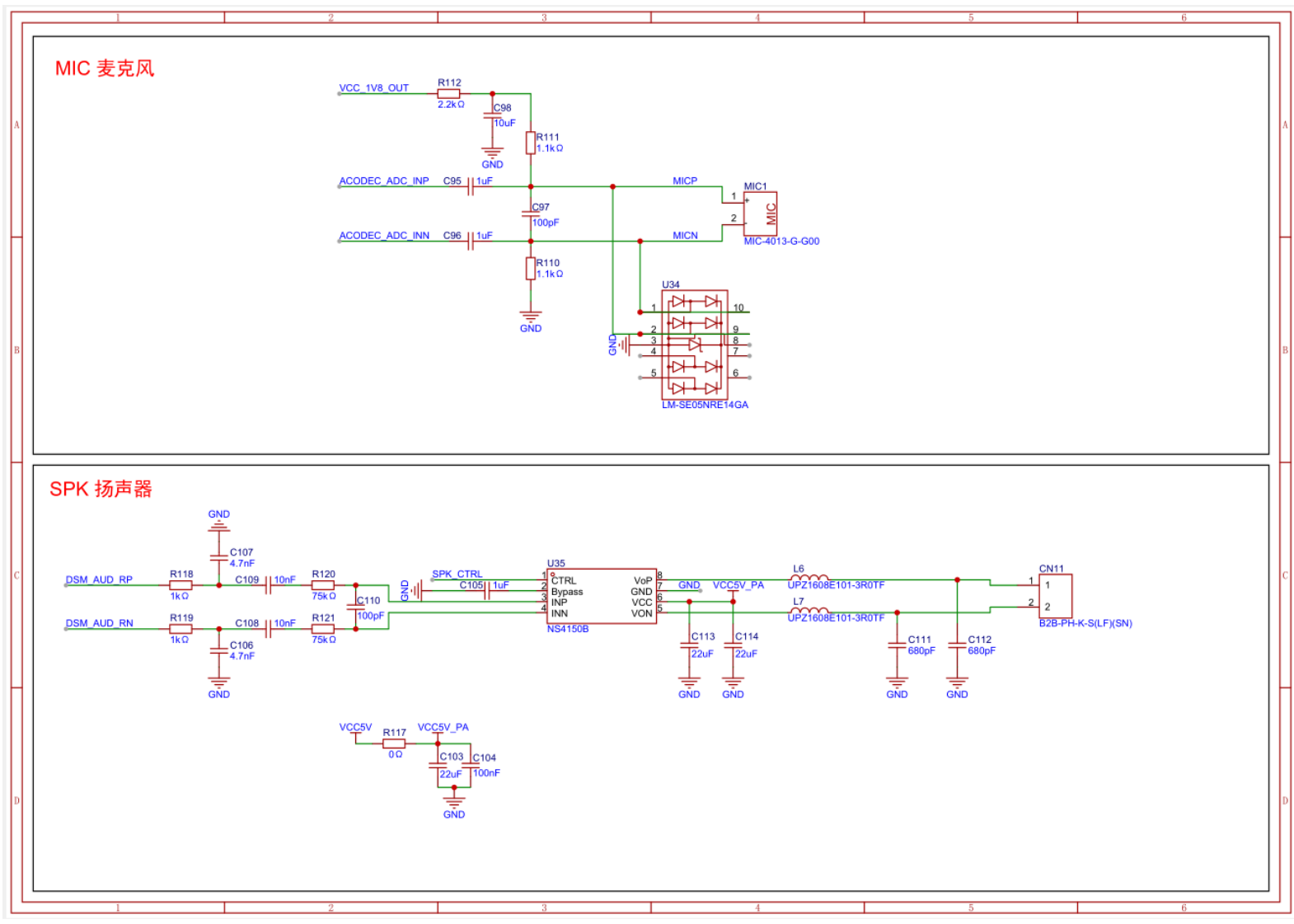


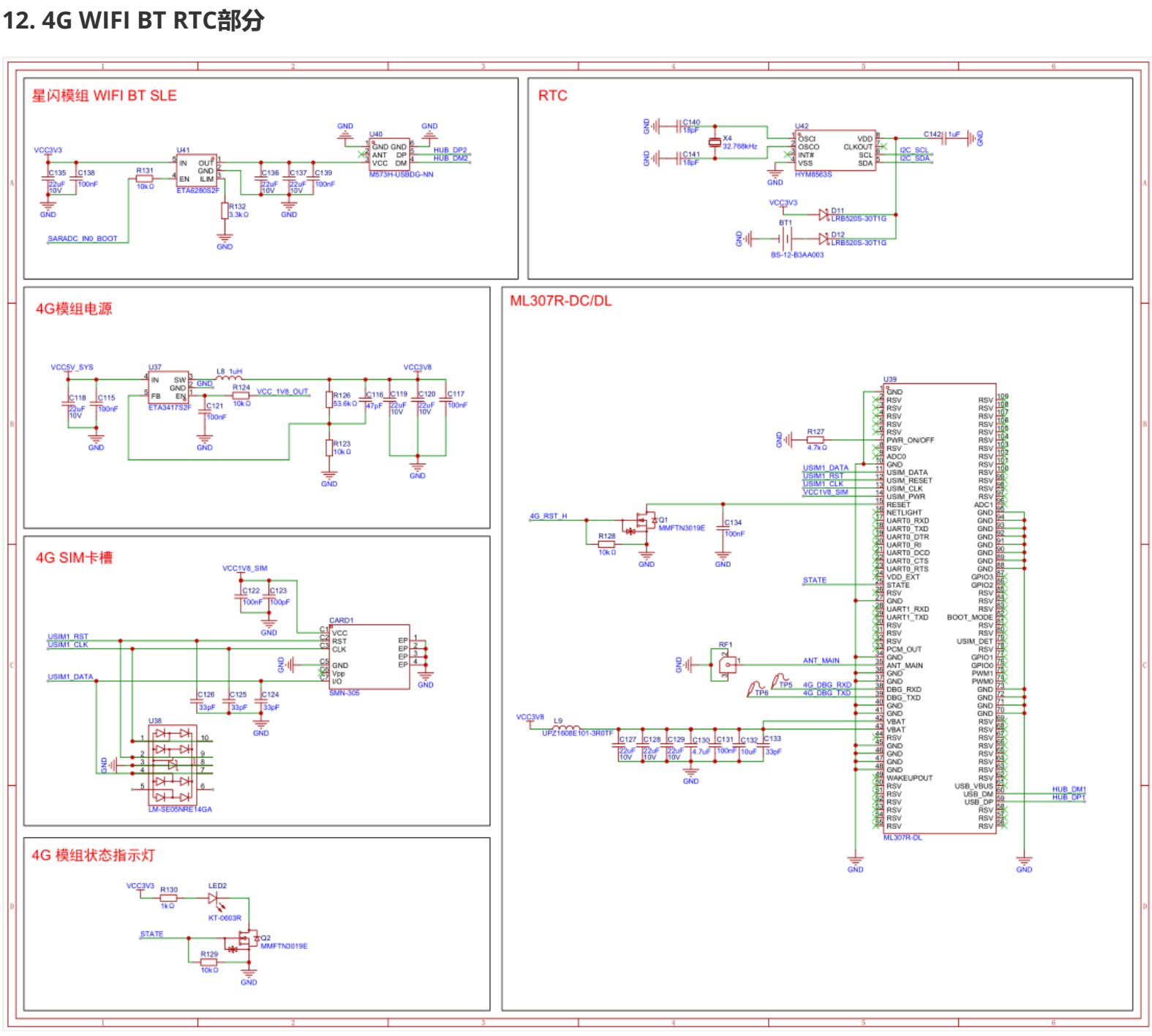
10. GPIO部分



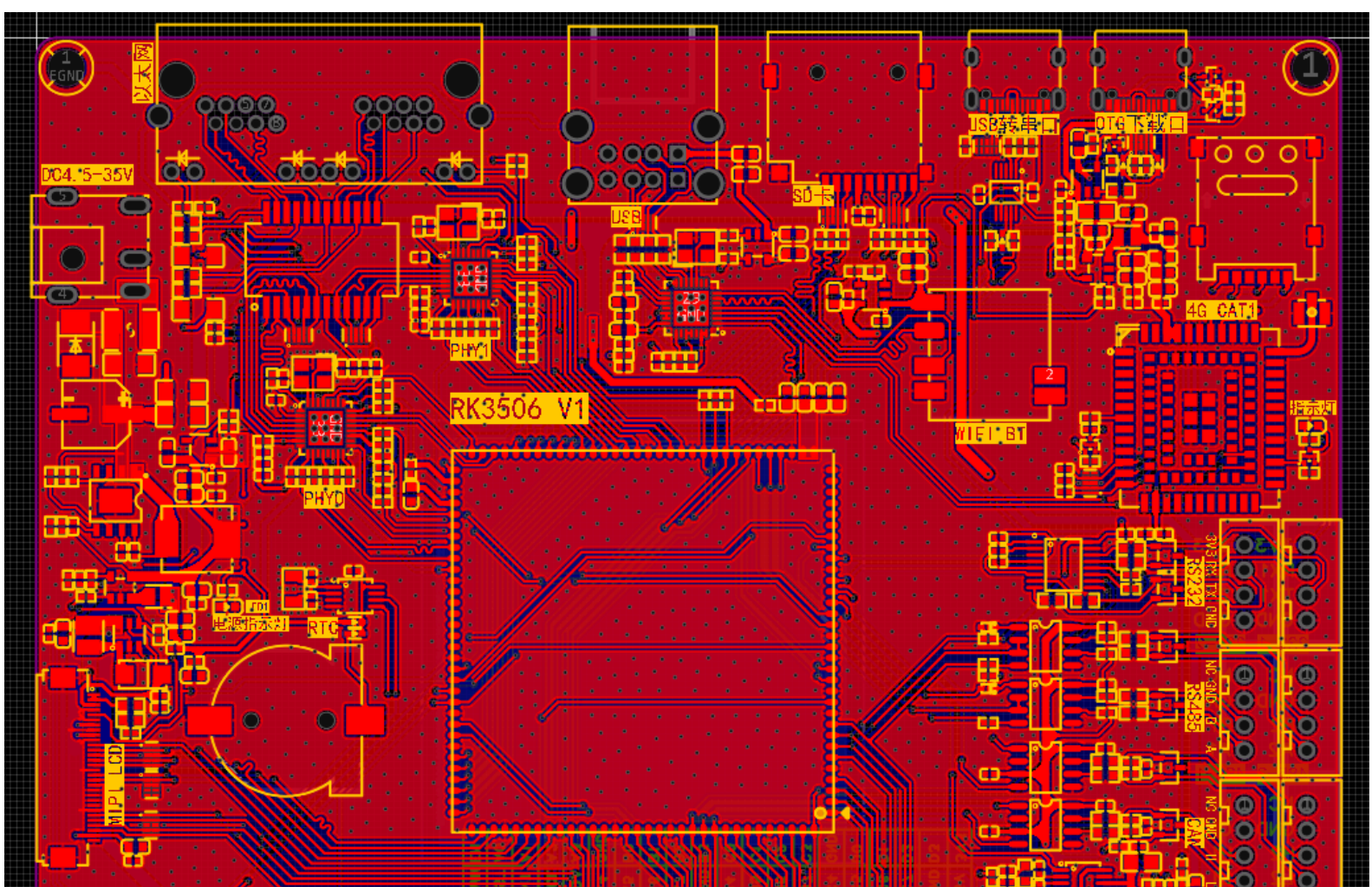
航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

11. 音频部分





1.3 PCB绘制

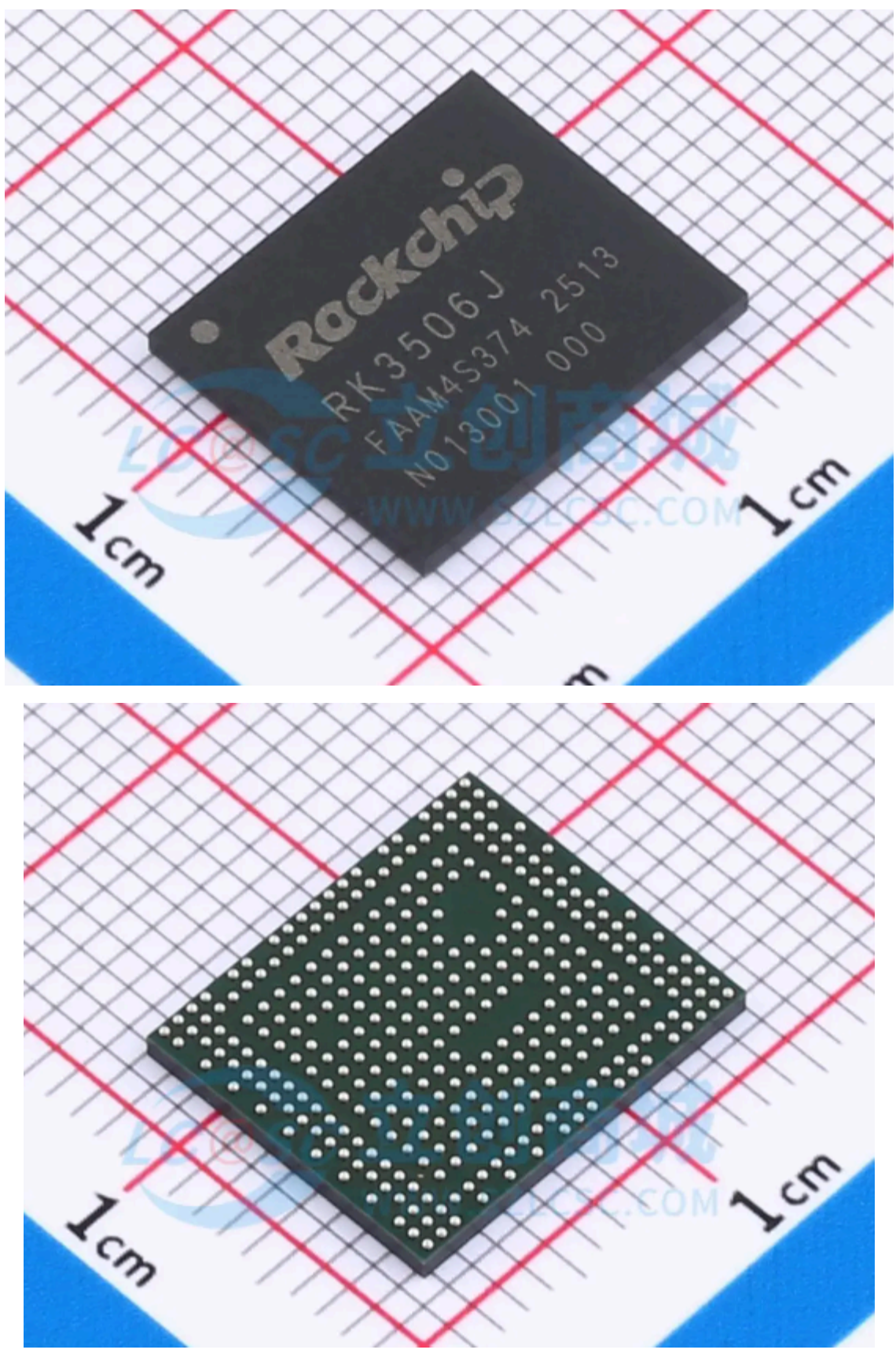


第二部分 Linux系统移植

第02章 RK3506微处理器

2.1 RK3506处理器介绍

1. RK3506处理器介绍



2. 产品概述

RK3506是瑞芯微电子（Rockchip）在2024年第四季度推出的一款**入门级、低成本、低功耗**的嵌入式处理器，专为工业控制和商业应用设计。芯片采用**22nm先进制程工艺**，产品生命周期长达**10~15年**，供货稳定可靠。

核心亮点

RK3506的核心优势可概括为：**异构架构、低功耗高可靠、接口丰富、实时性强、秒级启动和生态完善。**

异构多核架构

RK3506采用 **3×Cortex-A7 + 1×Cortex-M0** 的多核异构设计，兼顾强大的处理能力与高实时性。这种灵活的架构能够实现 **非对称多处理（AMP）**，可将不同核心分配给不同的操作系统，例如：

- **2×A7核 (Linux) + 1×A7核 (RTOS) + M0核 (裸机)**
- **3×A7核 (RTOS) + M0核 (裸机)**

通过标准的**RPMsg**机制进行核间通信，一个芯片即可高效地同时处理复杂的界面显示和高实时性的控制任务。

低功耗与高可靠性

该芯片是**为工业而生**的低功耗典范，满载功耗不足0.7W，常温下温升小于17度，无需散热片即可稳定运行。其工业级型号**RK3506J**支持 **-40℃ 至 85℃** 的工业宽温，能从容应对各类严苛的工业环境。

丰富的工业接口

RK3506虽为入门级芯片，但集成了极其丰富的工业现场所需接口。

接口类型	规格与特性
以太网	2路支持IEEE1588协议的百兆以太网，满足工业精准时间同步需求。
工业总线	**2路CAN-FD**（5Mbps高速率）、总计**6路UART**，轻松应对各类工业设备通信。
高速并行总线	**DSMC总线**，可高效扩展FPGA，写延时<75ns，读延时<260ns。
显示接口	**MIPI-DSI**和Parallel DSI，支持双通道MIPI输出，最大分辨率为1280×1280@60fps。
音频接口	支持**PDM**数字麦克风、**SAI**、**SPDIF**以及多路**ADC**，适用于各种语音应用。

此外，还包含USB 2.0 OTG、SPI、I2C、PWM等通用外设接口。

高实时性与低延迟

凭借AMP架构及对**Preempt-RT**、**Xenomai**等实时Linux内核的支持，系统可实现**微秒级**的实时响应。

- **系统中断响应延迟**：可小于**5微秒**。
- **EtherCAT总线控制抖动**：在进行EtherCAT主站协议栈优化后，1毫秒控制周期的抖动延迟仅为**约90微秒**（<10%），能够支持高达**8轴的总线控制**，为高精度运动控制提供了坚实基础。

2D图形加速与"秒级"启动

芯片内置**2D硬件引擎**，能极大减轻CPU在图像显示时的负担。它原生支持轻量级UI框架**LVGL**，借助2D加速，GUI运行流畅度显著提升。结合深度优化的系统固件，从上电到LVGL界面完全显示，启动时间可以缩短到惊人的**2秒**左右。

完善的软件生态

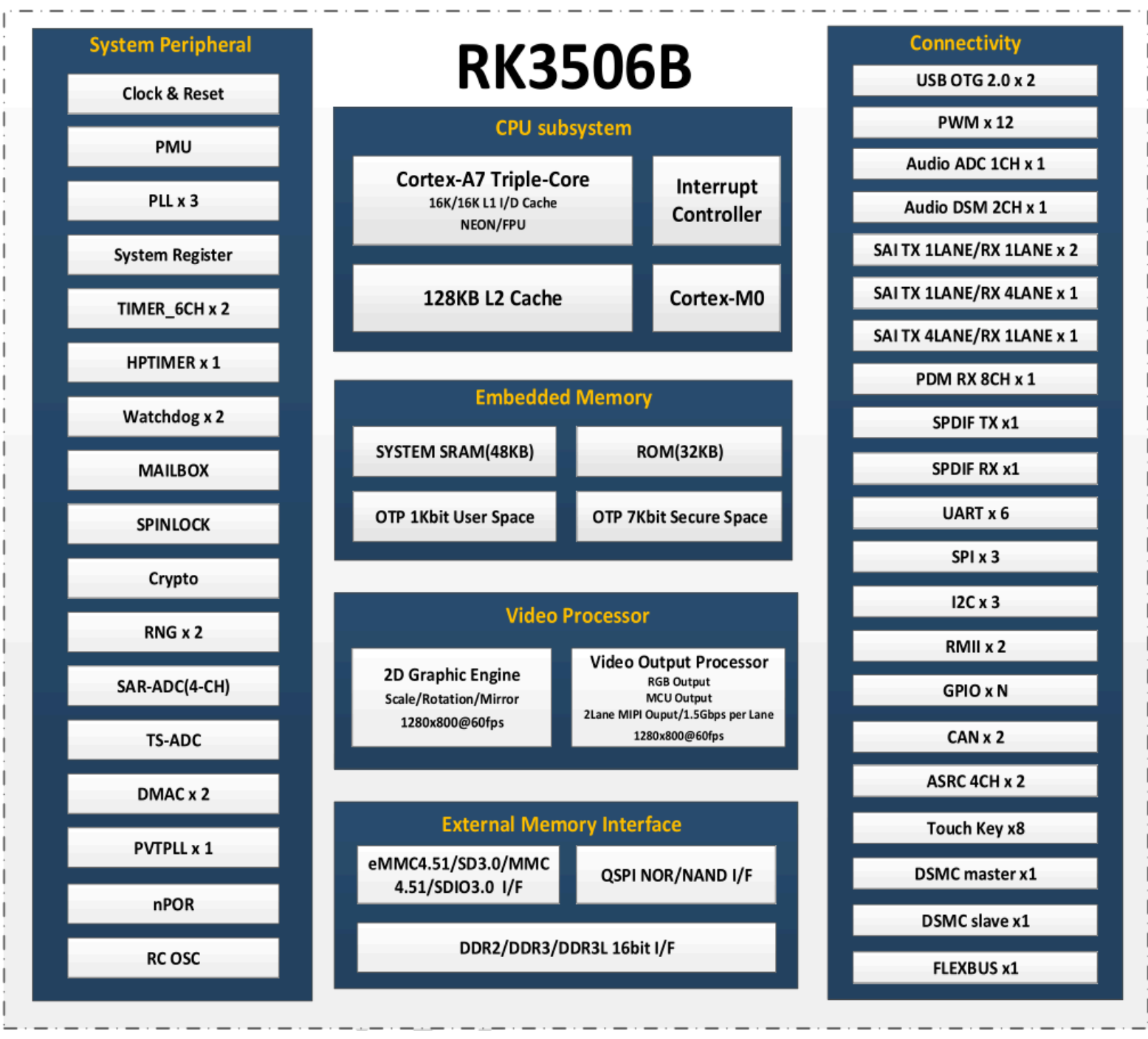
- **多系统支持**：支持Linux (Kernel 6.1)、Buildroot、Yocto、Ubuntu等多种主流系统，并支持RT-Thread等国产RTOS。

典型应用场景

RK3506凭借其特性，精准定位于**轻量级、高性价比**的工业与商业应用。

应用领域	典型场景	核心优势
工业人机界面 (HMI)	工业设备操作面板、生产数据显示屏等	图形加速，LVGL流畅体验，极速开机
工业控制与自动化	**PLC（可编程逻辑控制器）**、远程I/O、伺服驱动器	强大的AMP实时方案，支持EtherCAT等多轴总线控制
电力与能源设备	**FTU（馈线终端单元）**、集中器、充电桩控制器	AMP双系统架构，微秒级实时响应；0.7W超低功耗，宽温适应户外环境
工业通信与网关	**IoT网关**、边缘计算节点、协议转换器	双网口、双CAN、多路串口，网络互联能力强大
智能语音与音频	**IP话机**、语音模组、智能音响	丰富的PDM、SAI等音频接口
商业与家用设备	智能家电显控屏、**手持POS终端**、**扫地机器人**	低功耗、高性价比，支持快速启动

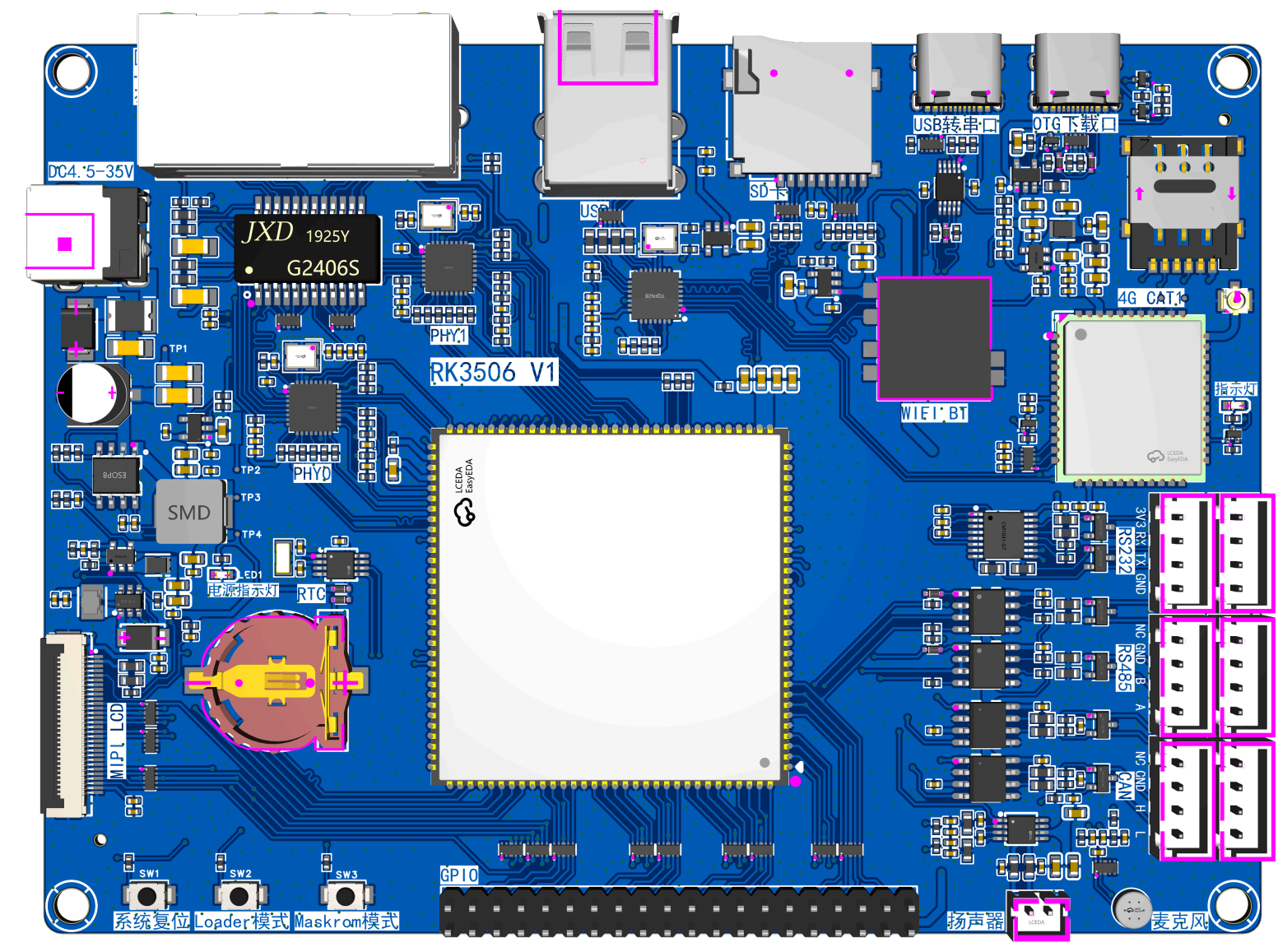
功能架构图

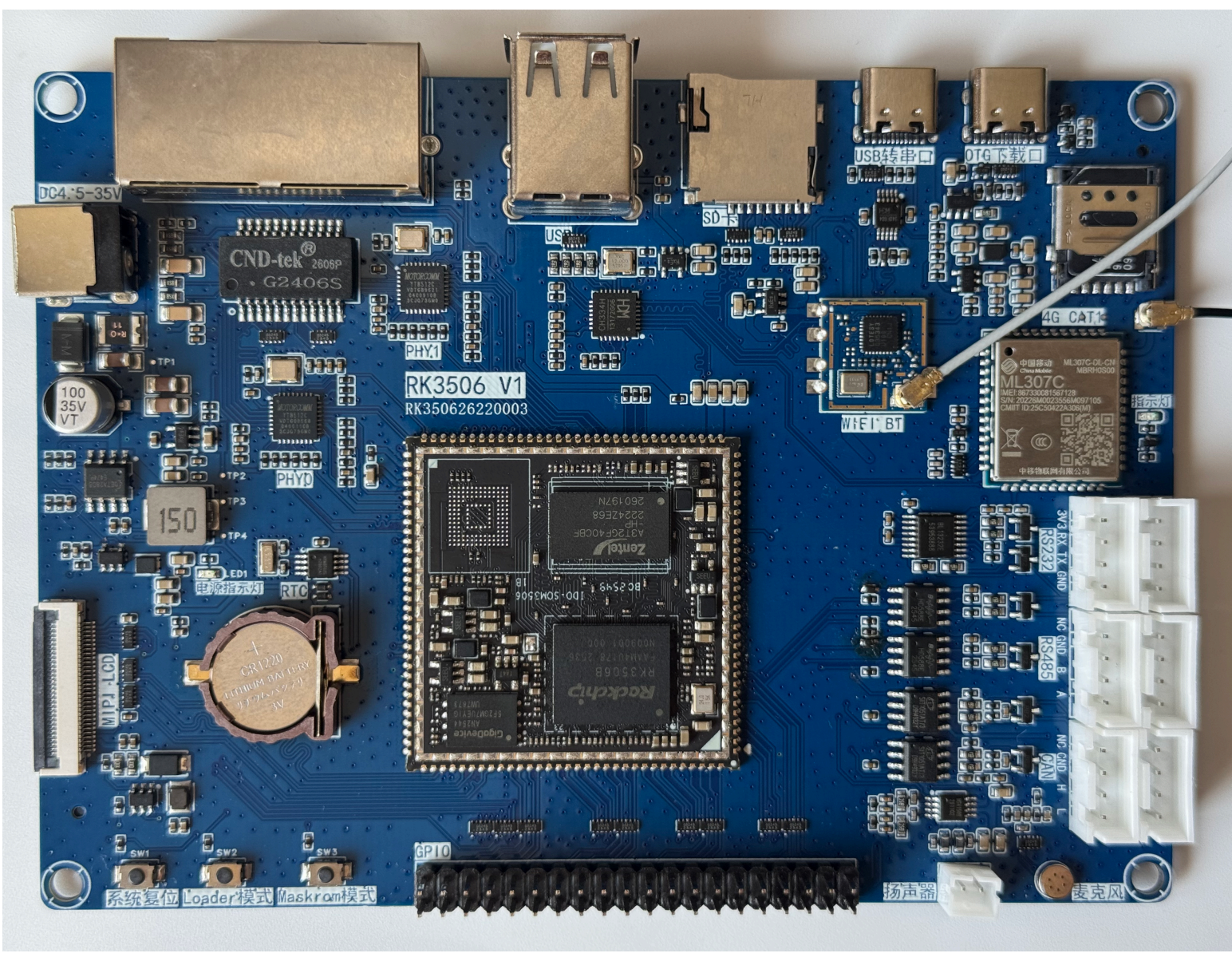


第03章 RK3506 开发平台介绍

3.1 RK3506硬件平台介绍

1. RK3506开发平台





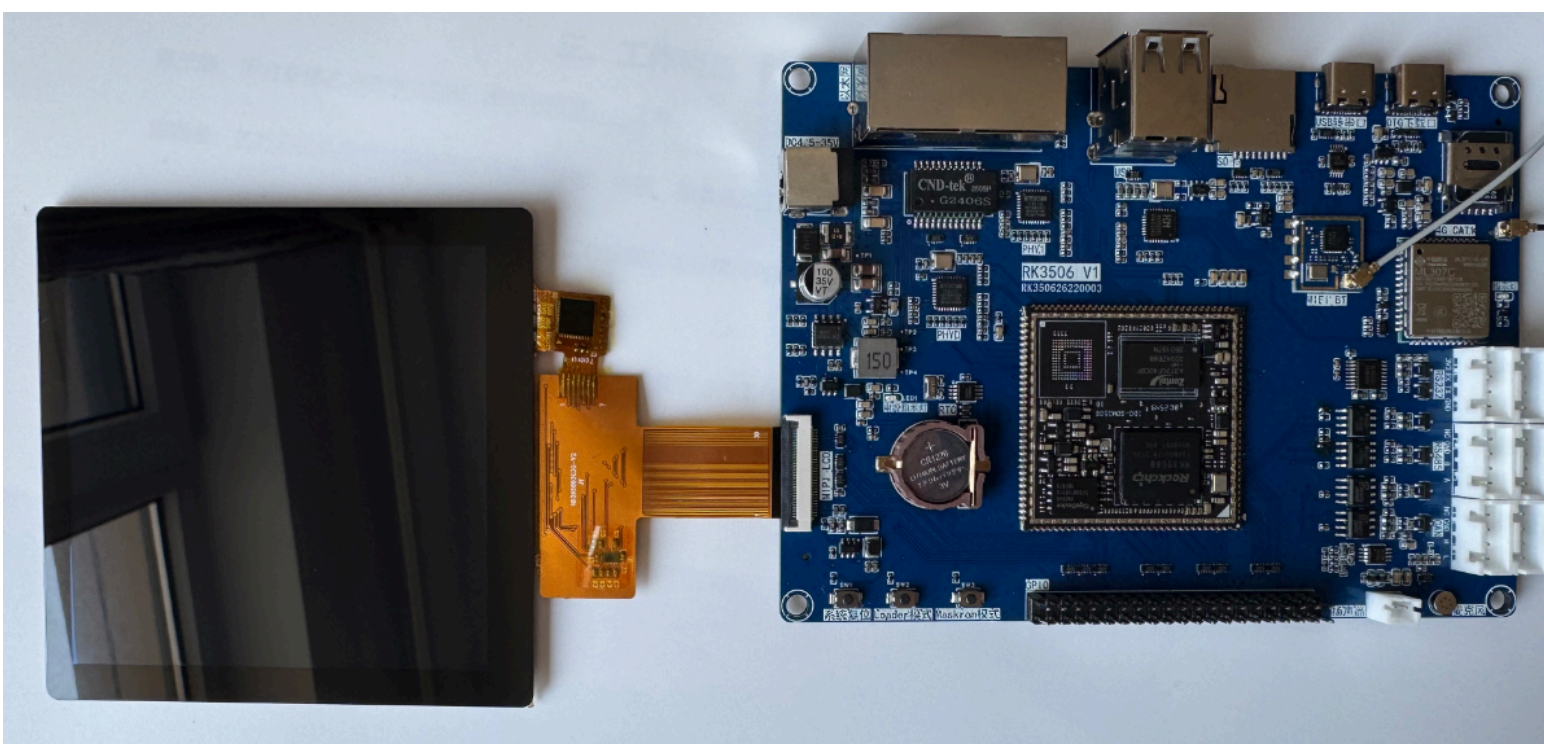
2. IMX6UL 开发板原理图

原理图如下：`RK3506开发板V1\02-原理图\SCH_RK3506V1_2026-05-21.pdf`

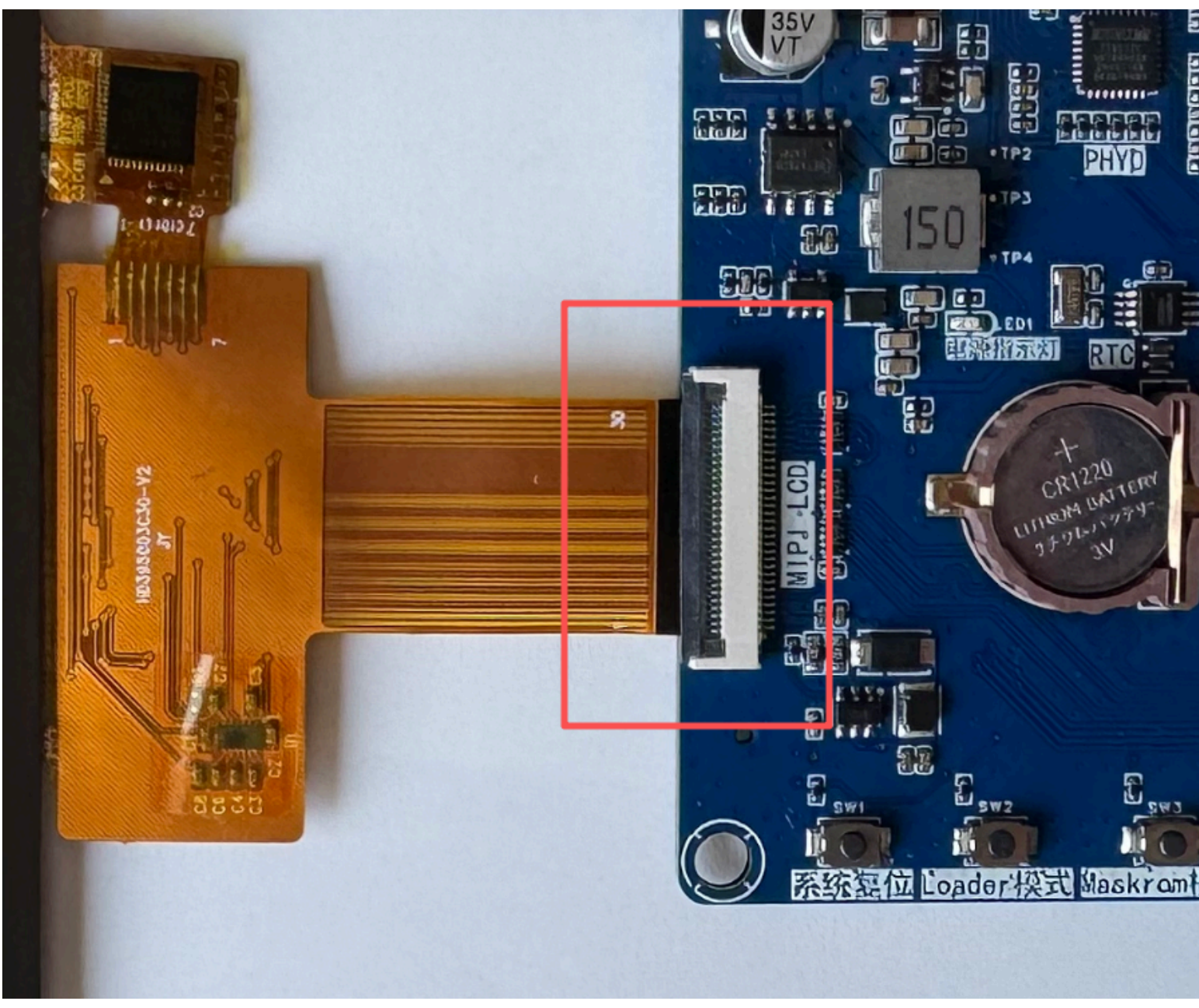
3.2 RK3506开发平台安装步骤

1. 液晶屏安装步骤

屏幕排线安装，蓝色的面向上，如下图

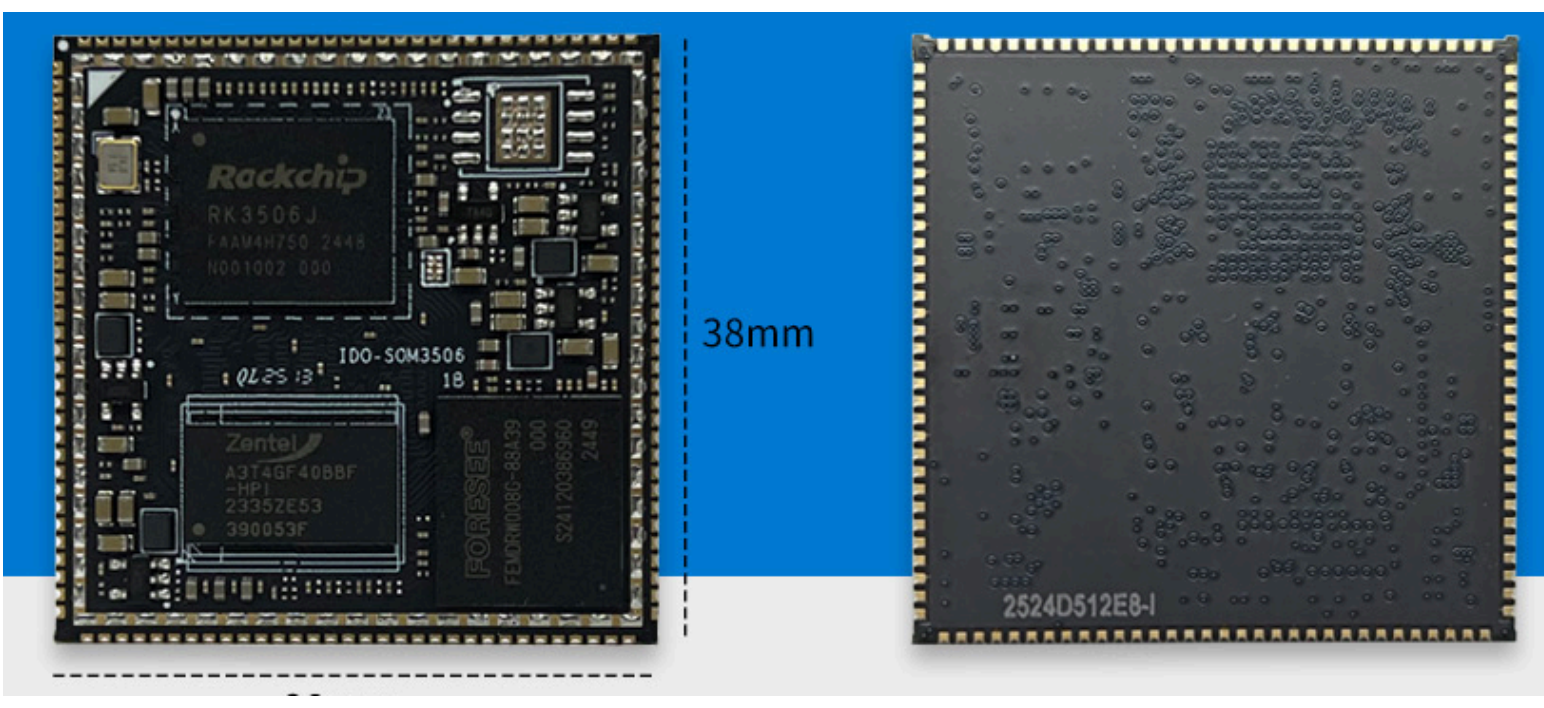


黑色的面向上，液晶屏和主板链接如下图：



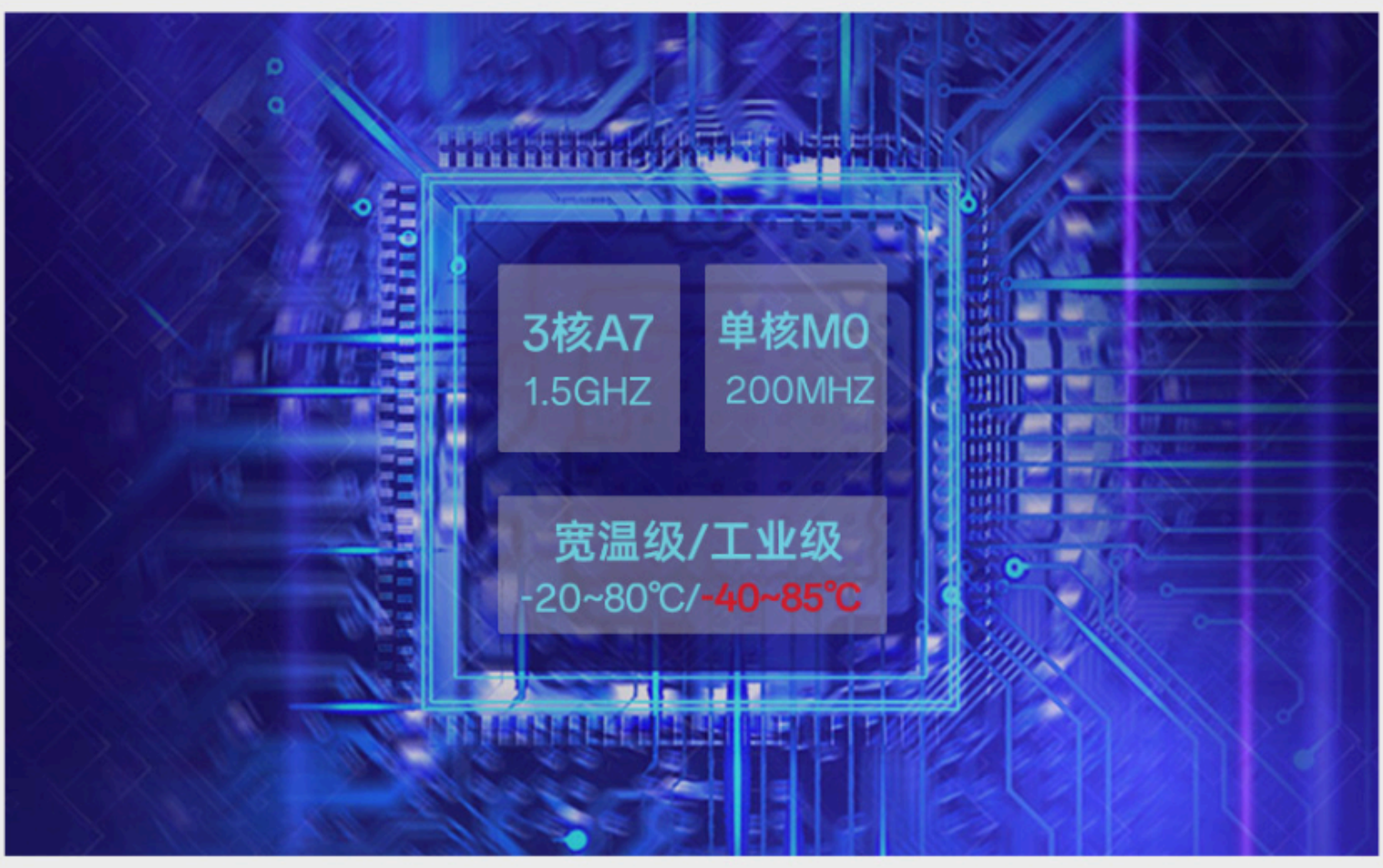
3.3 RK3506开发平台核心板

1. 产品介绍



产品概述

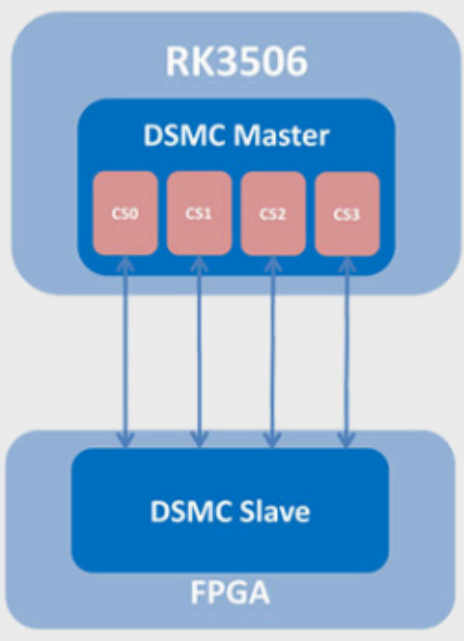
基于瑞芯微全新工业SoC芯片平台RK3506B/RK3506J；集成3核Cortex-A7＋Cortex-M0多核异构处理器，主频最高1.5GHz；支持-40～85°C工业级宽温，轻松面对各种环境。



全功能接口覆盖

内置丰富工业互联接口，支持6路串口、2路CAN FD、2路网口、2路USB2.0 OTG，以及常用视频、音频等，充分满足各种智能化开发需求，助力创新和多样化的应用！

RK3506 Localbus(DSMC)方案示例：



技术指标

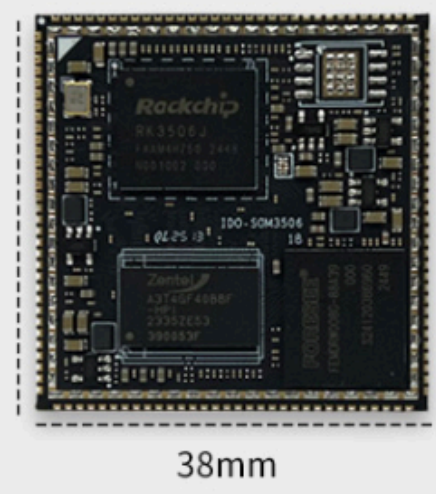
测试指标	测试条件	访问方式	写	读
延迟	DSMC接口延迟	CPU	74ns	258ns
吞吐	CPU频率: 1.608 G DSMC总线频率: 266 M DSMC接口频率: 147 M 接口位宽: x8 理论带宽294MB/s	CPU	274MB/s	158MB/s
		DMA	292MB/s	170MB/s
吞吐	CPU频率: 1.608 G DSMC总线频率: 266 M DSMC接口频率: 147 M 接口位宽: x16 理论带宽588MB/s	CPU	447MB/s	383MB/s
		DMA	449MB/s	418MB/s

产品特色

超小尺寸

38mm × 38mm超小尺寸

集成DDR3/3L、PMIC、NAND/eMMC（2选1）



邮票孔封装（抗震效果更佳）

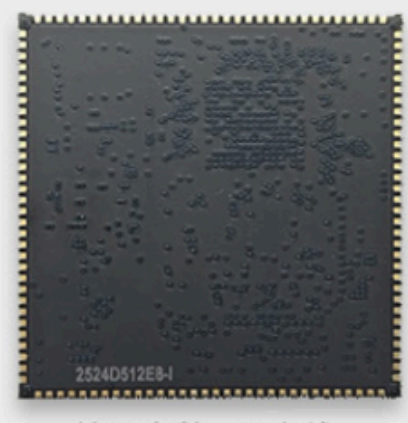
独特的叠层设计，PCB背面完整纯平无走线

核心板支持100%全国产

PCB规格

128Pin全引脚拓展，间距1.1mm

板厚1.4mm，8层板沉金工艺，高Tg材质



纯平完整，无走线

品质甄选

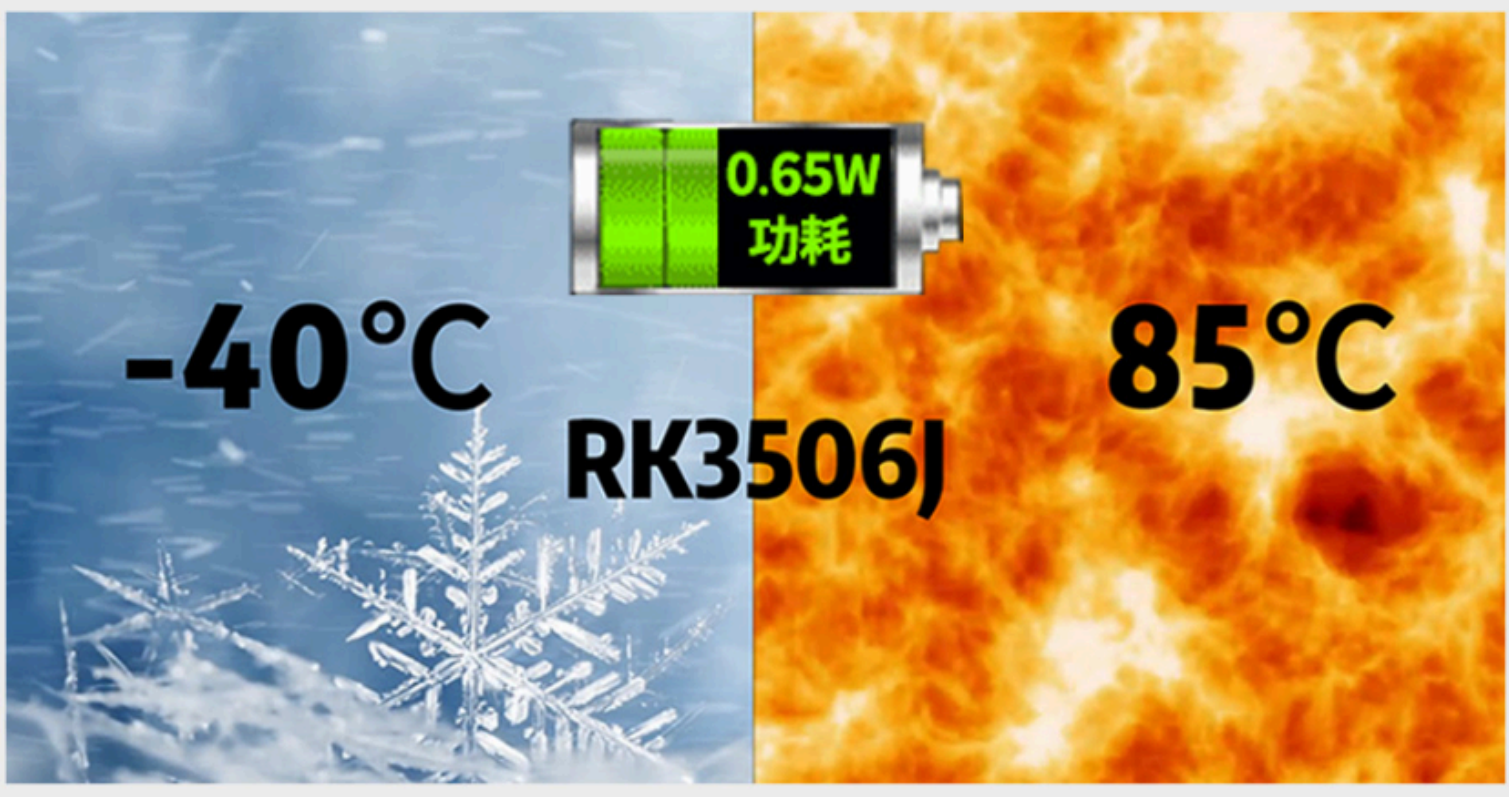
支持CE/FCC/RoHS等认证，优异稳定的EMC性能

10年以上供应周期，长期稳定供货

为工业而生

超低功耗

实测功耗低于0.65W，无散热片运行通过-40～85°C高温环境测试。



IDO-SOM3506-S1核心板进行了严格的电源完整性和信号完整性仿真设计，通过各项电磁兼容、温度冲击、高温高湿老化、长时间存储压力等测试，稳定可靠，批量供货。

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

CE/FCC/RoHS
认证

7×24小时
满负荷测试

10000次开关机
老化测试

高低温湿度
全面测试

雷击/浪涌
测试

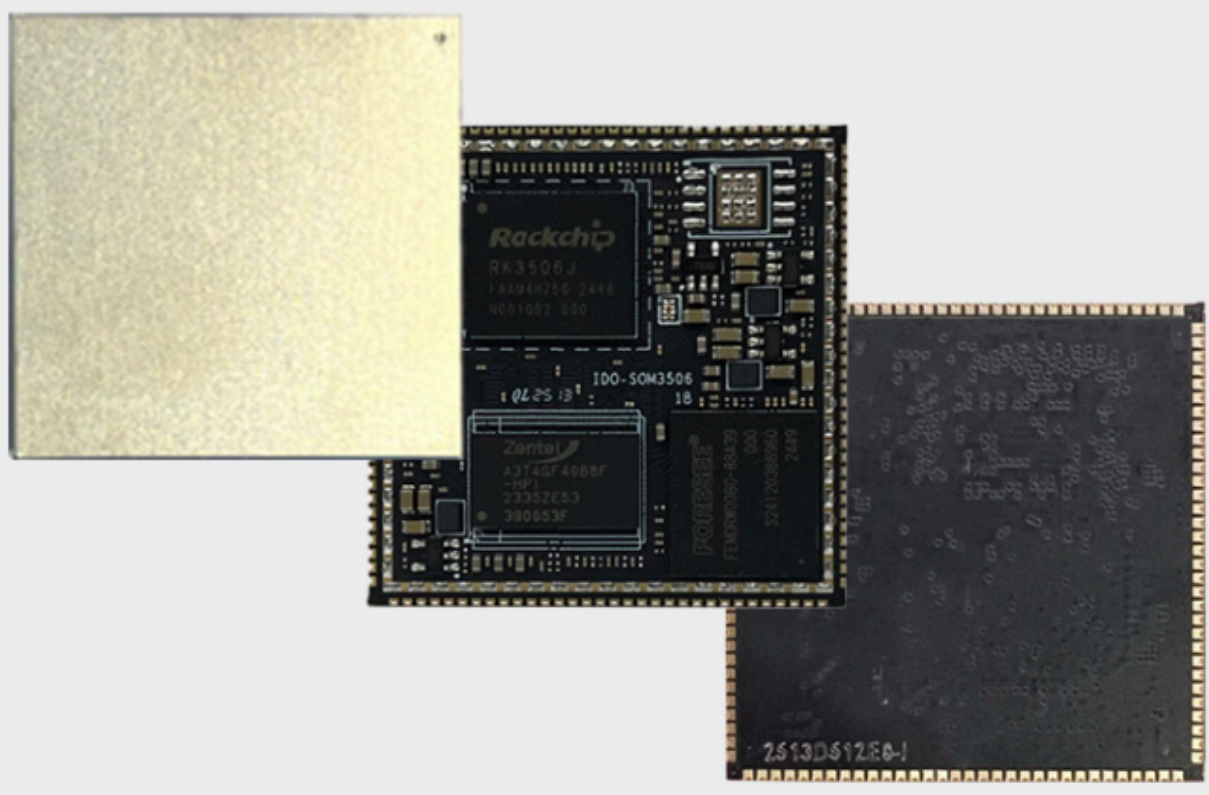
电源稳定性
测试

电磁兼容性
测试

电快速脉冲群
抗扰测试

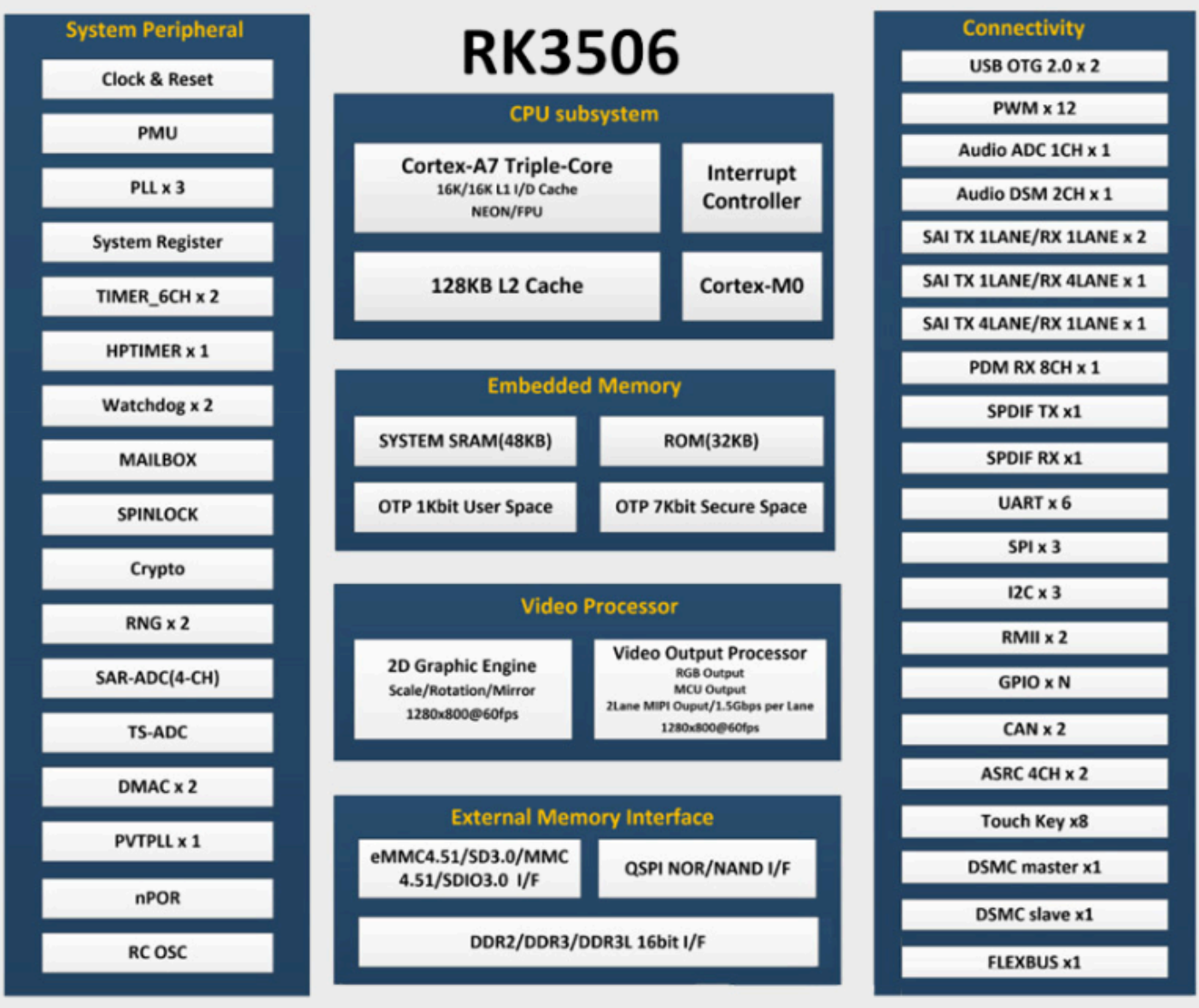
防护升级+抗干扰

可选配屏蔽框+屏蔽罩，全面升级防护与抗干扰，高达8KV抗接触静电!

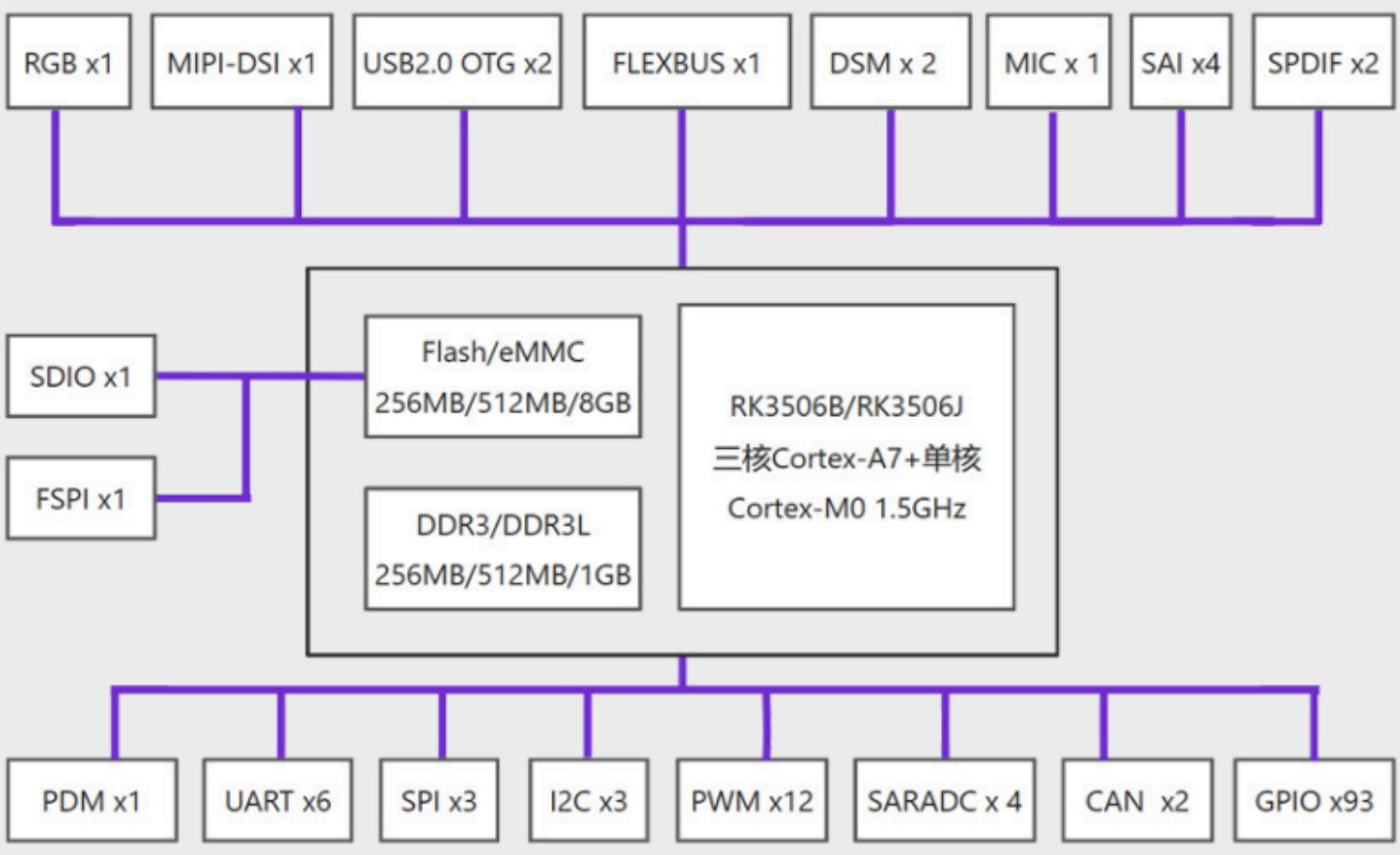


SoC与核心板框图

瑞芯微RK3506 SoC框图如下图所示:



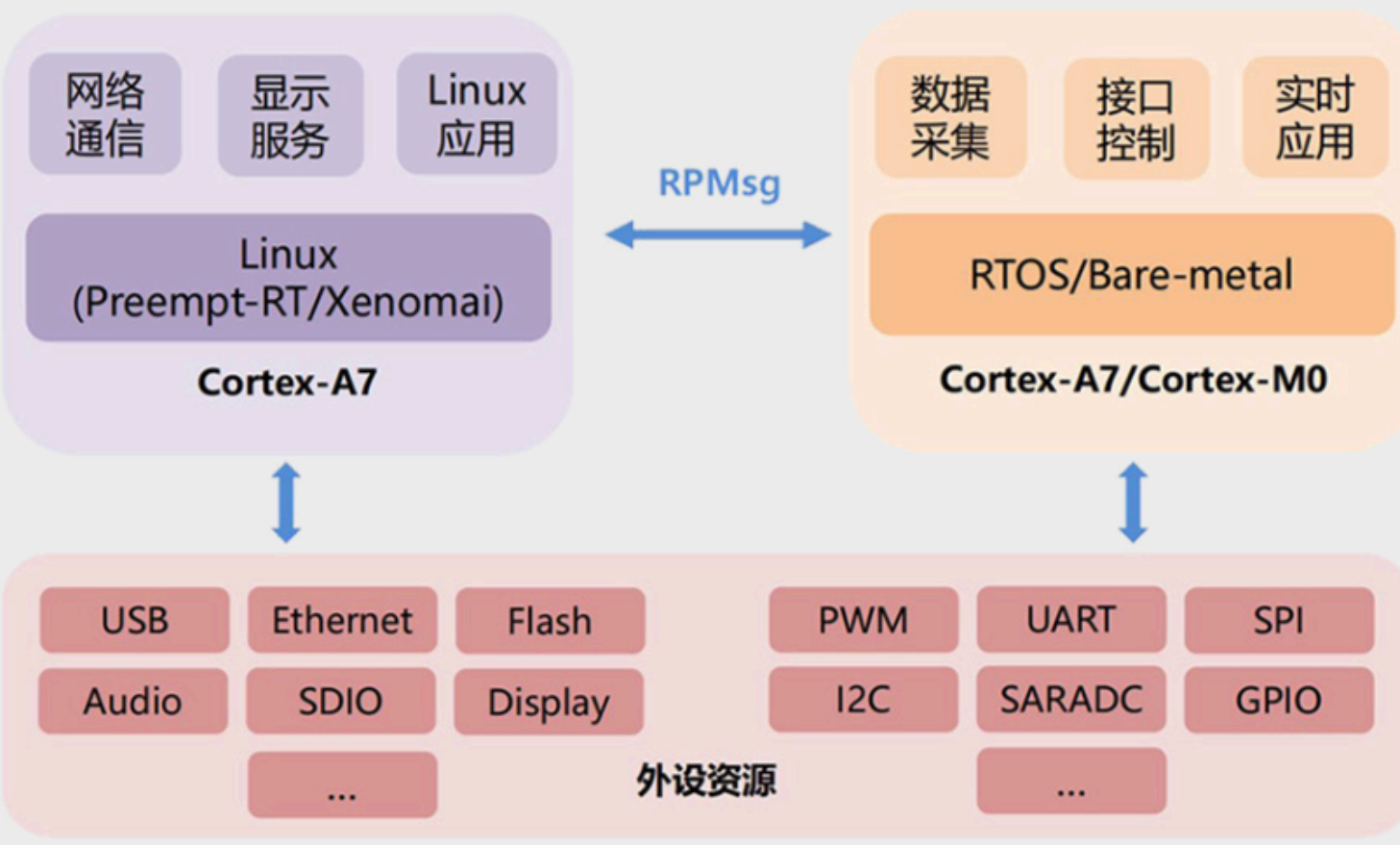
仅需设计外围电路即可快速实现项目的稳定量产。通过矩阵IO，引出SoC全功能，设计简便，量产模块逻辑框图:



基本参数

处理器

3核Cortex-A7 + Cortex-M0多核异构，主频最高1.5Ghz，22nm制程工艺
可支持AMP多系统方案，能够同时运行RTOS和Linux系统



同时在AMP与多核异构系统配合下，实现A核微秒级回应，M0百纳秒级回应，全面提升了系统性能表现和使用体验，流畅体验一触即发！（原厂实验室数据 仅供参考）

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

事件	硬件上电 SPL加载	引导程序SPL 加载完成	kernel启动	UI应用显示
阶段耗时	0ms	386ms	495ms	1519ms
总耗时统计	0ms	386ms	881ms	2400ms

内存与存储

内存：256MB ~ 1GB 高速DDR3

存储：128MB ~ 512MB的NAND Flash或8GB ~ 128GB eMMC

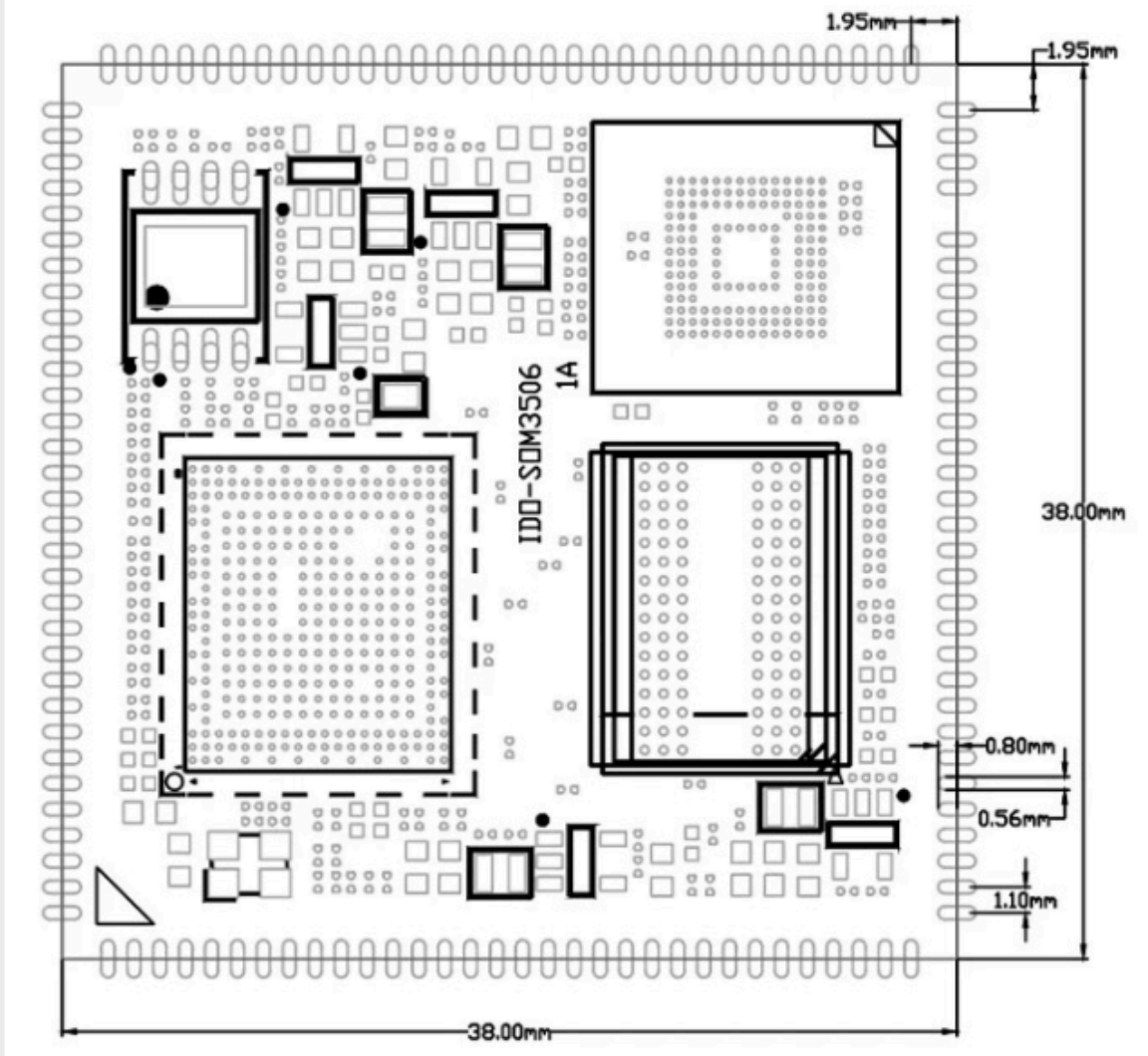
显示输出

内置2D硬件引擎和显示输出引擎，以最小化CPU负载轻松满足图像与视频显示，为用户带来清晰、顺滑、无卡顿的视觉体验。

视频接口：1×MIPI DSI，高达1280×1280@60FPS输出
1×RGB，高达1280×1280@60FPS输出

机械尺寸与电气特性

IDO-SOM3506-S1核心板尺寸如下图所示：



电源特性表（功耗实测低于0.65W）

电源输入	最小电压	标称值	最大电压	峰值电流	待机电流
VCC5V0_SYS	4.5V	5.0V	5.4V	120mA	15mA

系统与软件参数

系统支持

支持多种实时与常规系统，及开源鸿蒙OpenHarmony L1、中移物联OneOS，并支持裸机开发与RTOS SMP。

软件参数

操作系统	图形界面 开发工具	软件开发套件
Buildroot Ubuntu (Linux6.1.84)	Qt-5.11 LVGL-8.4	RK3506 Linux6.1 SDK Release V1.1.0

驱动支持

eMMC/NAND | DDR3 | USB OTG | SDIO/FSPI | RTC | KEY

MIPI DSI/RGB | LCD/TP | HP OUT/MIC IN | Ethernet/4G Cellular

WiFi/BT/SLE | Watchdog | DSMC | FlexBus | CAN FD | UART

第04章 RK3506固件烧写

4.1 系统启动方式

RK3506的系统启动方式设计得相当灵活，可以通过硬件配置和不同模式来适应各种开发与产品需求。

RK3506支持多种启动介质，但在硬件设计上，系统引导的优先级是固定的，无法通过软件更改，其顺序由高到低为：

- **FSPI** (SPI Flash)
- **SDMMC** (包括eMMC和SD Card)
- **USB/SPI2APB** (USB OTG接口和SPI2APB)

一个需要注意的硬件限制是：eMMC和SD卡接口是复用的。因此，**除非板上贴了SPI Flash，否则无法同时使用eMMC和SD卡作为启动盘，只能二选一。**

RK3506提供了三种主要的启动模式，用于产品的正常运行和开发调试。各个模式的功能和进入方式，我整理在了下面：

启动模式	功能描述	硬件操作
Normal 模式	正常启动流程，按顺序加载Bootloader、Kernel、Rootfs，最终进入系统。	**核心板引脚 `SARADC_IN1_RECOVER` 保持高电平**（不按下Recovery键）。

启动模式	功能描述	硬件操作
Loader 模式	用于**固件烧录和更新**。Bootloader会停止引导，等待主机的`RKDevTool`等工具发送命令。	**上电前或上电瞬间，将`SARADC_IN1_RECOVER`引脚拉低**（如按住Recovery键不放）。
MaskRom 模式	**系统救砖的最后手段**。当Bootloader损坏，Loader模式无法进入时，此模式强制芯片从固化在CPU内部的代码启动，以修复设备。	**将核心板`SARADC_IN0_BOOT`引脚接地后上电。**

- **Loader模式启动了U-Boot后，停止引导内核的模式， 可以理解为：uboot启动后的命令模式**
- **MaskRom模式是USB启动模式，是U-Boot运行之前的一种模式，可以对系统进行刷机**

4.2 ADB工具使用

ADB通常指Android Debug Bridge，Android调试桥，用于与设备通信。

ADB工具包请从以下路径获取: `RK3506开发板V1\01-工具软件\ADB工具\adb_win` ，ADB工具具体使用方法参考压缩包下的readme.txt。

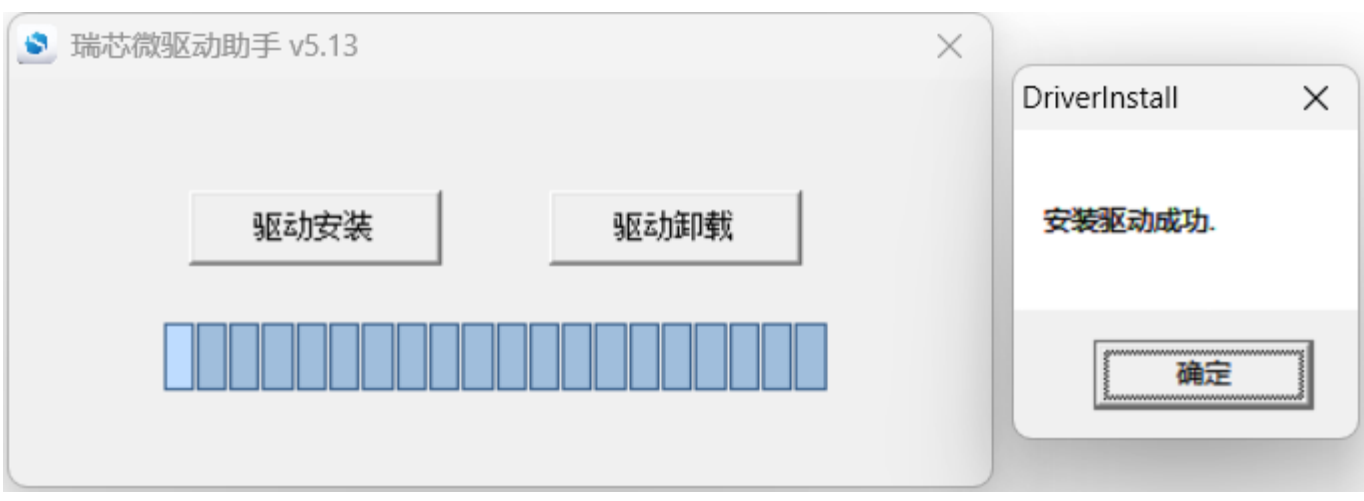
```
# 运行cmd.exe然后再执行adb相关命令如
# adb shell
```

ADB驱动安装

双击运行 `RK3506开发板V1\01-工具软件\RK驱动\DriverAssitant_v5.13\DriverInstall.exe`

弹窗点击 【驱动安装】按钮，如下图所示:

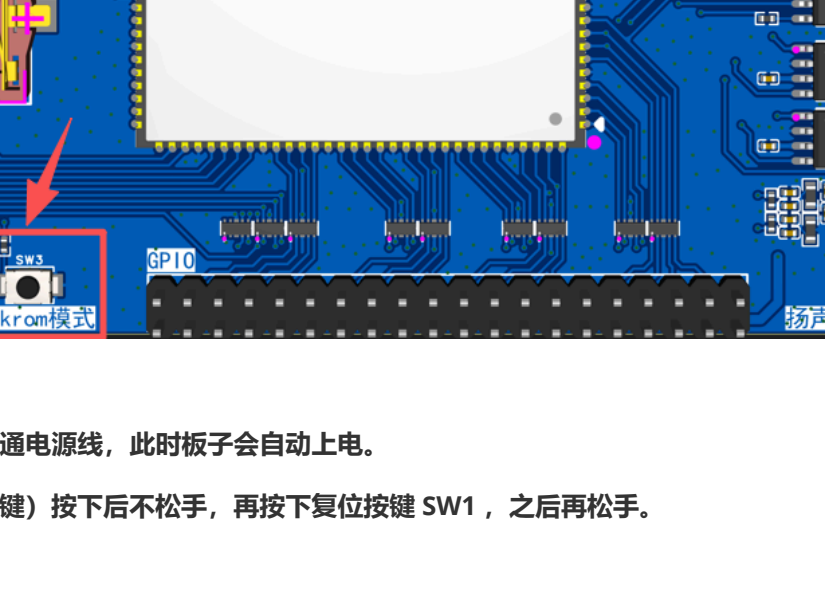
弹窗提示安装完成，点击 【确定】按钮，驱动安装完成如下图所示:



4.3 MaskRom 模式下载固件

MaskRom模式是USB启动模式，是U-Boot运行之前的一种模式，可以对系统进行刷机。

- MaskRom启动原理图

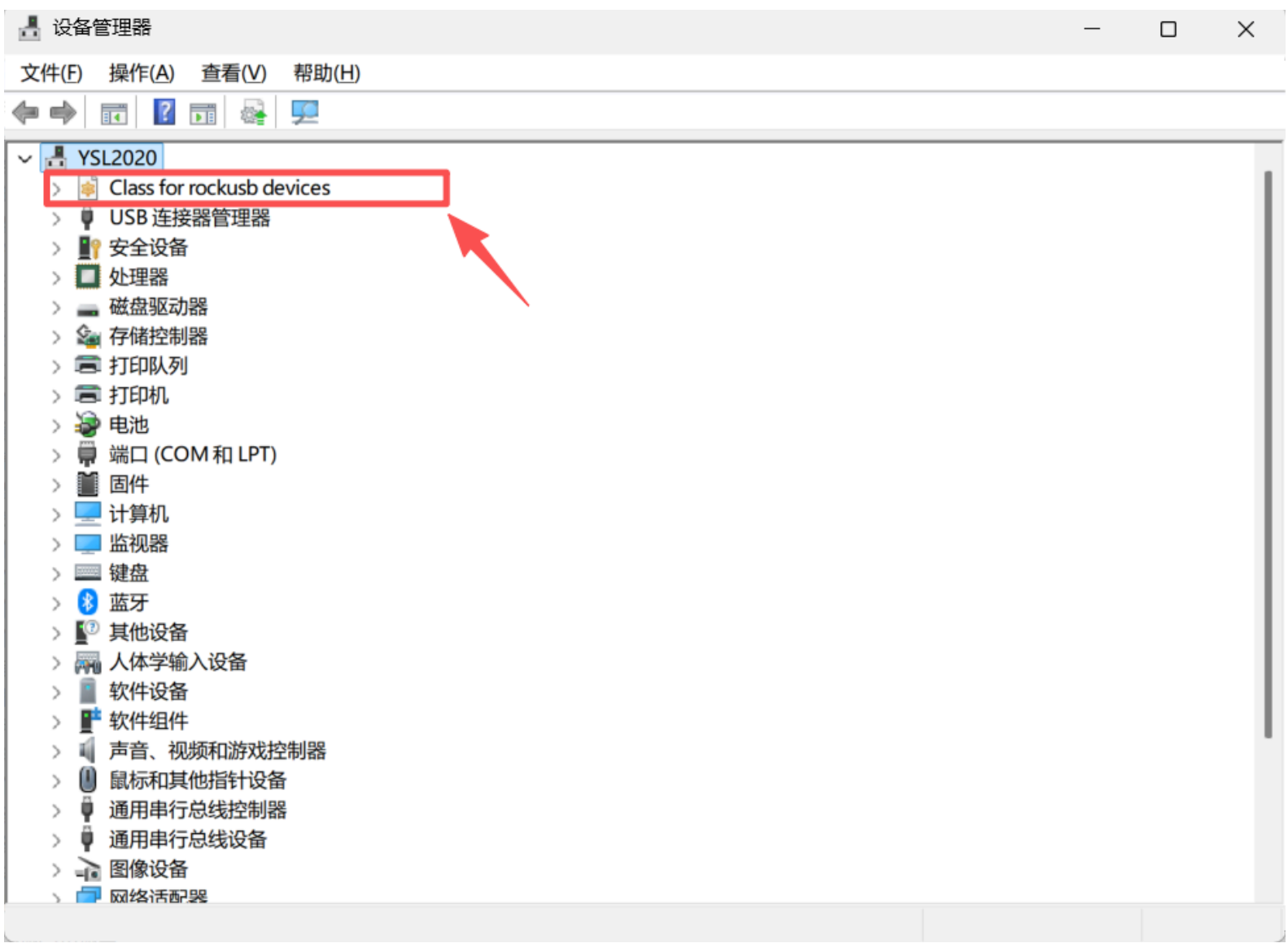


将核心板 `SARADC_IN0_BOOT` 引脚接地后上电。将核心板 SW3 引脚接地后，按下复位按键 SW1 。

插上 OTG下载口的USB线后并接通电源线，此时板子会自动上电。

将核心板 SW3 按键（Maskrom键）按下后不松手，再按下复位按键 SW1 ，之后再松手。

打开设备管理器如下图:



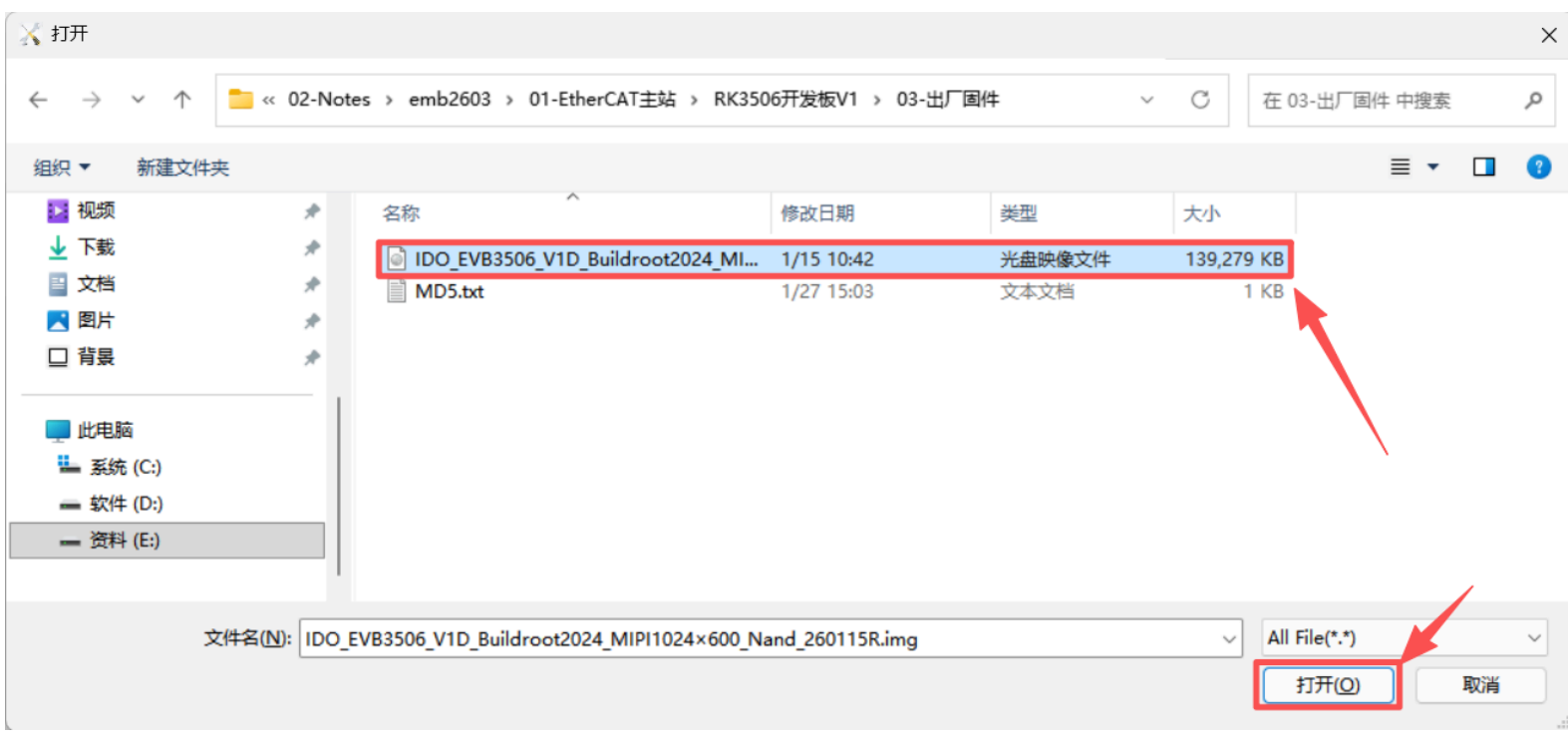
打开烧录软件 `RK3506开发板V1\01-工具软件\RK烧录工具\RKDevTool_v3.32_for_window`，如下图所示:

打开软件后如下图:

如果没有出现 MASKROM 设备， 表示启动方式有问题， 重复以上的步骤即可。

点击 [升级固件] -> [固件] 选择要烧录的固件，如下图所示 :

这里选择要下载的固件： `RK3506开发板V1\03-出厂固件\IDO_EVB3506_V1D_Buildroot2024_MIPI720x720_Nand_260115R.img`



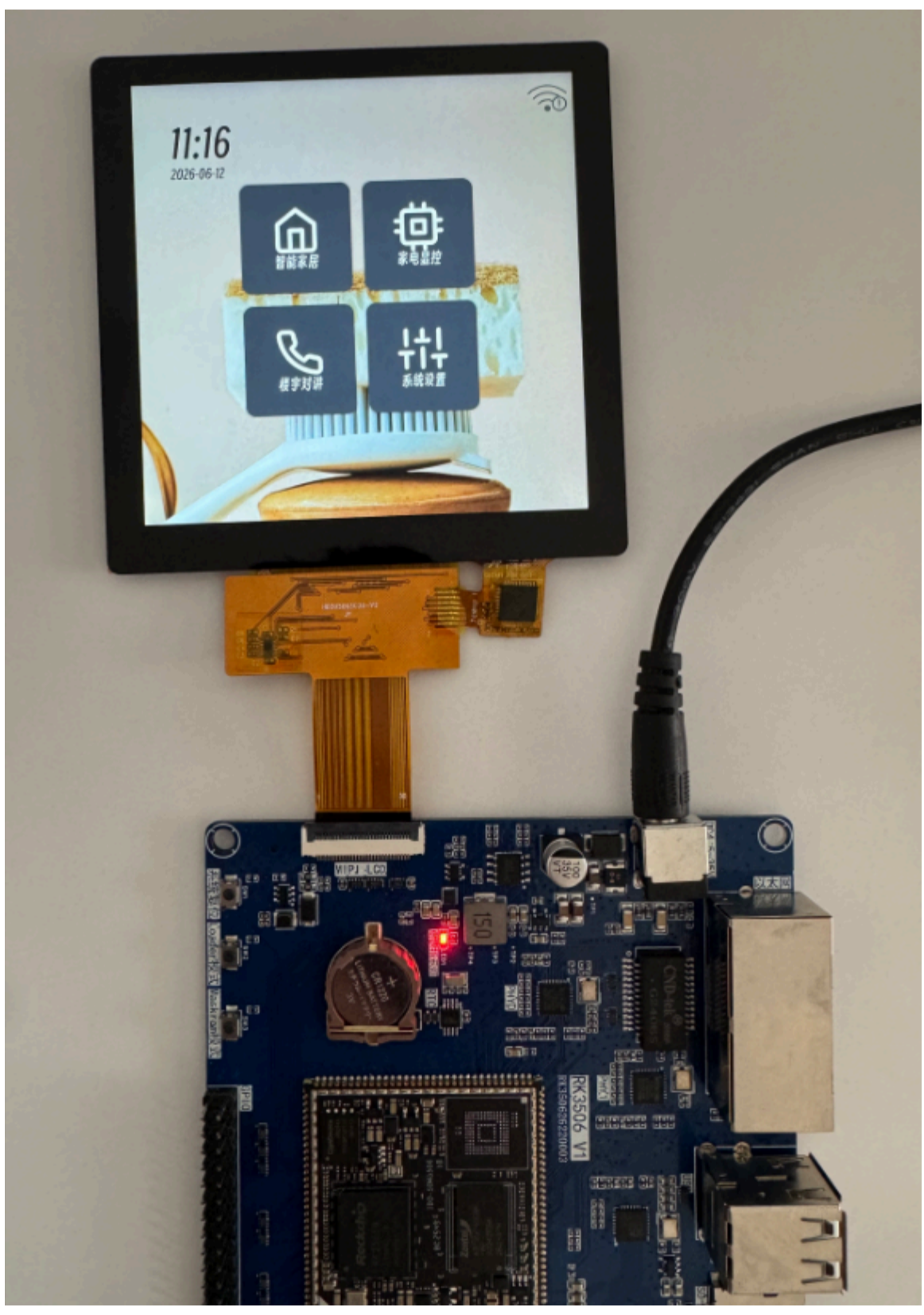
点击升级固件

下载固件。。。

4分钟左右下载固件完成

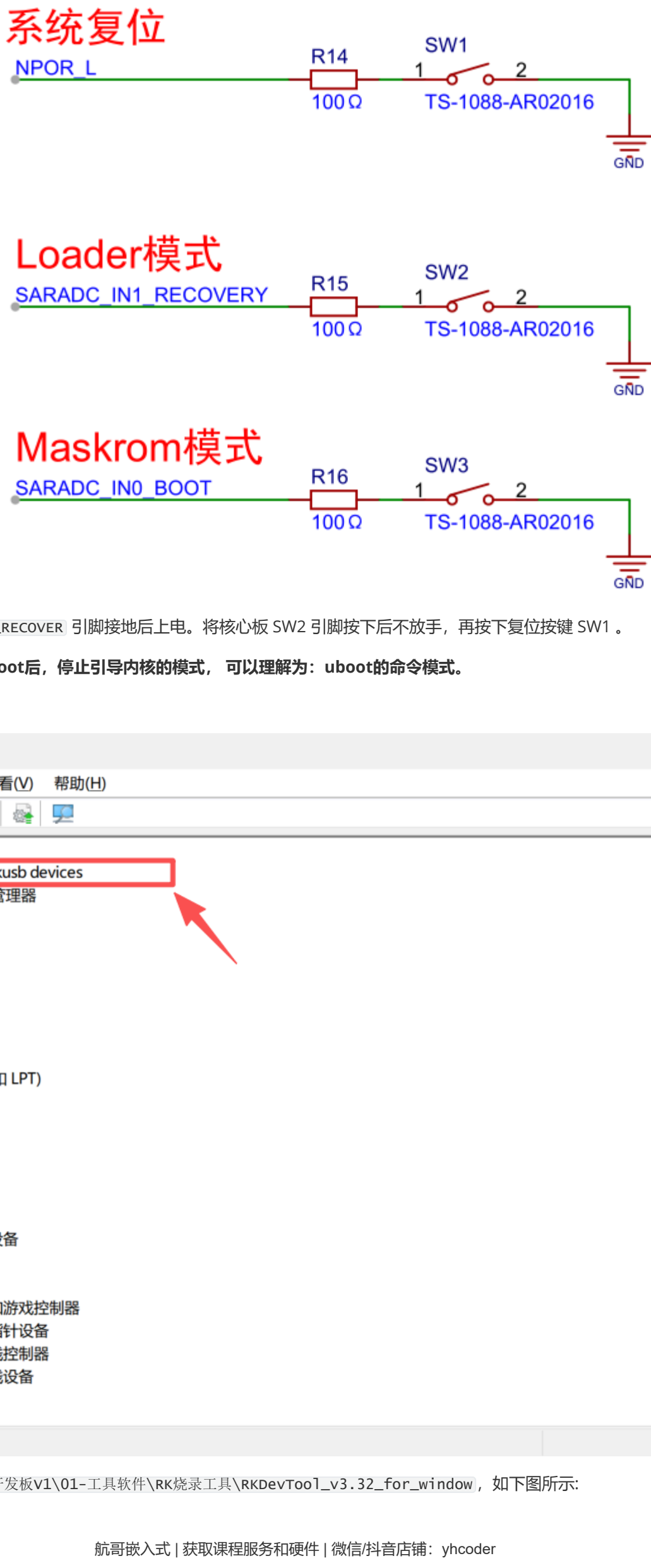
主板完成烧录后等待系统启动完成并且软件底部会显示出“发现一个ADB设备”，代表烧录成功，如下图所示:

液晶屏显示如下内容，表示固件更新完成:



至此固件更新完成。

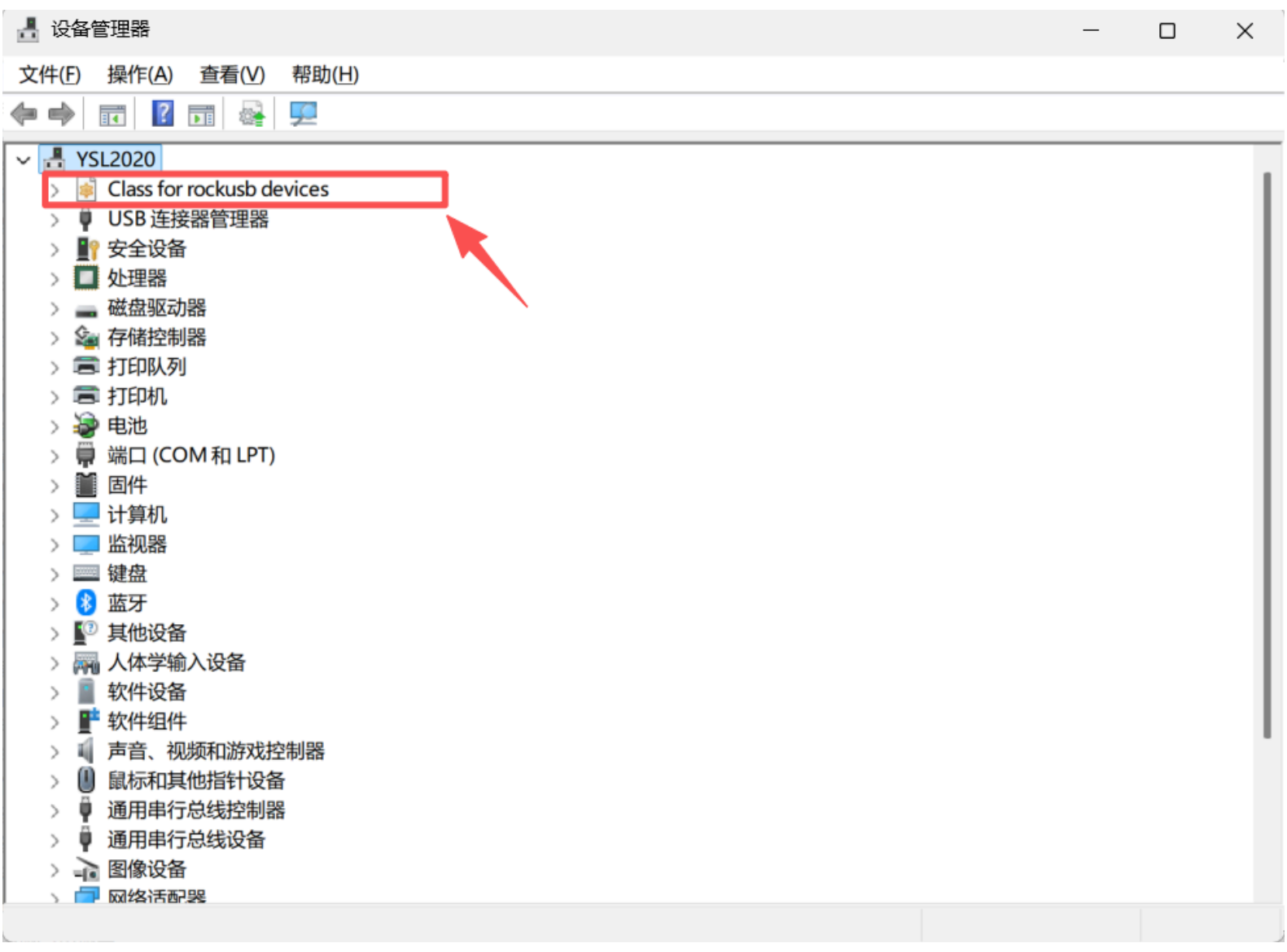
4.4 Loader 模式下载固件



将核心板 `SARADC_IN1_RECOVER` 引脚接地后上电。将核心板 SW2 引脚按下后不放手，再按下复位按键 SW1 。

Loader模式启动了U-Boot后，停止引导内核的模式， 可以理解为：uboot的命令模式。

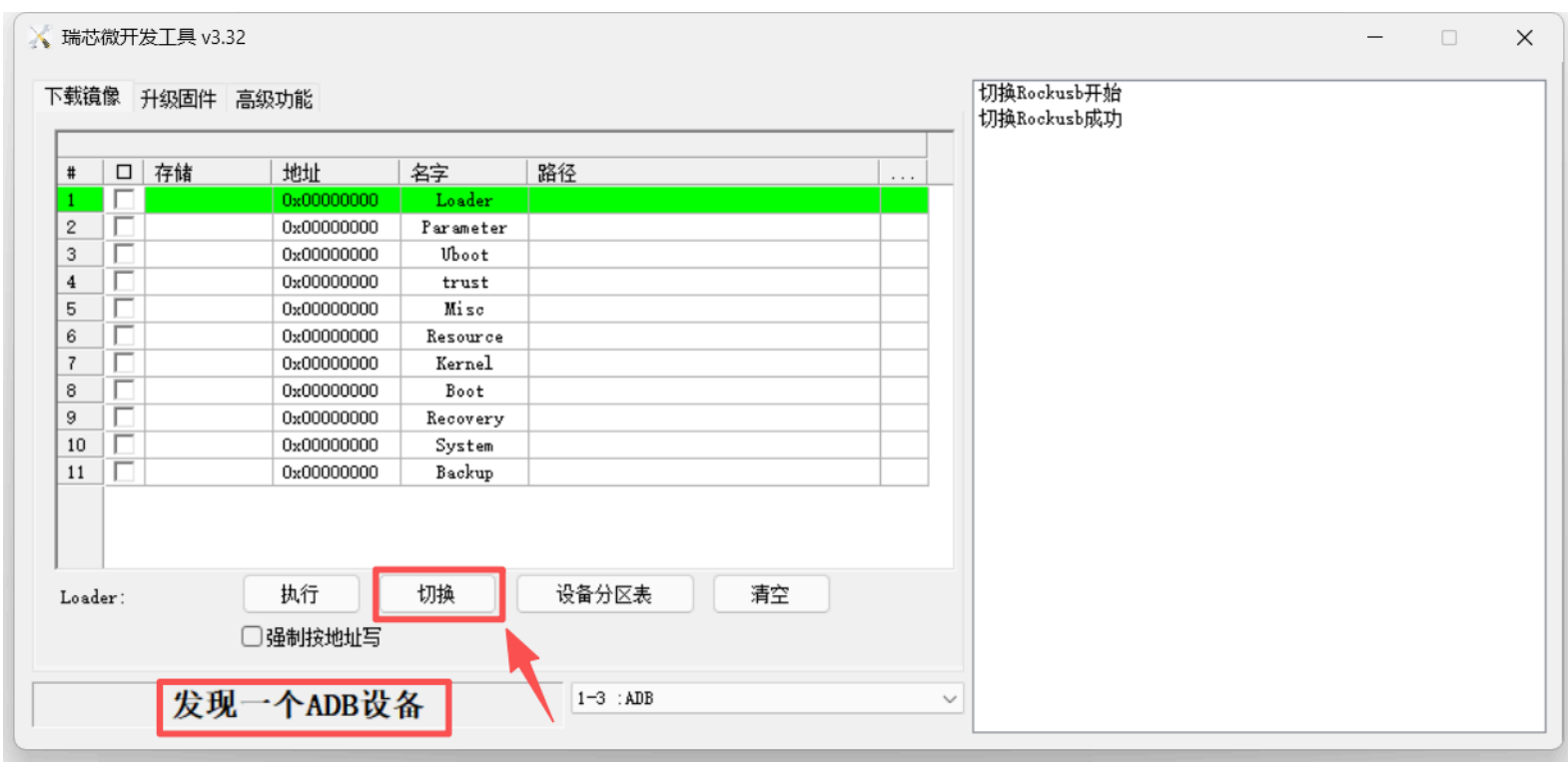
设备管理器如下:



打开烧录软件 `RK3506开发板V1\01-工具软件\RK烧录工具\RKDevTool_v3.32_for_window`，如下图所示:

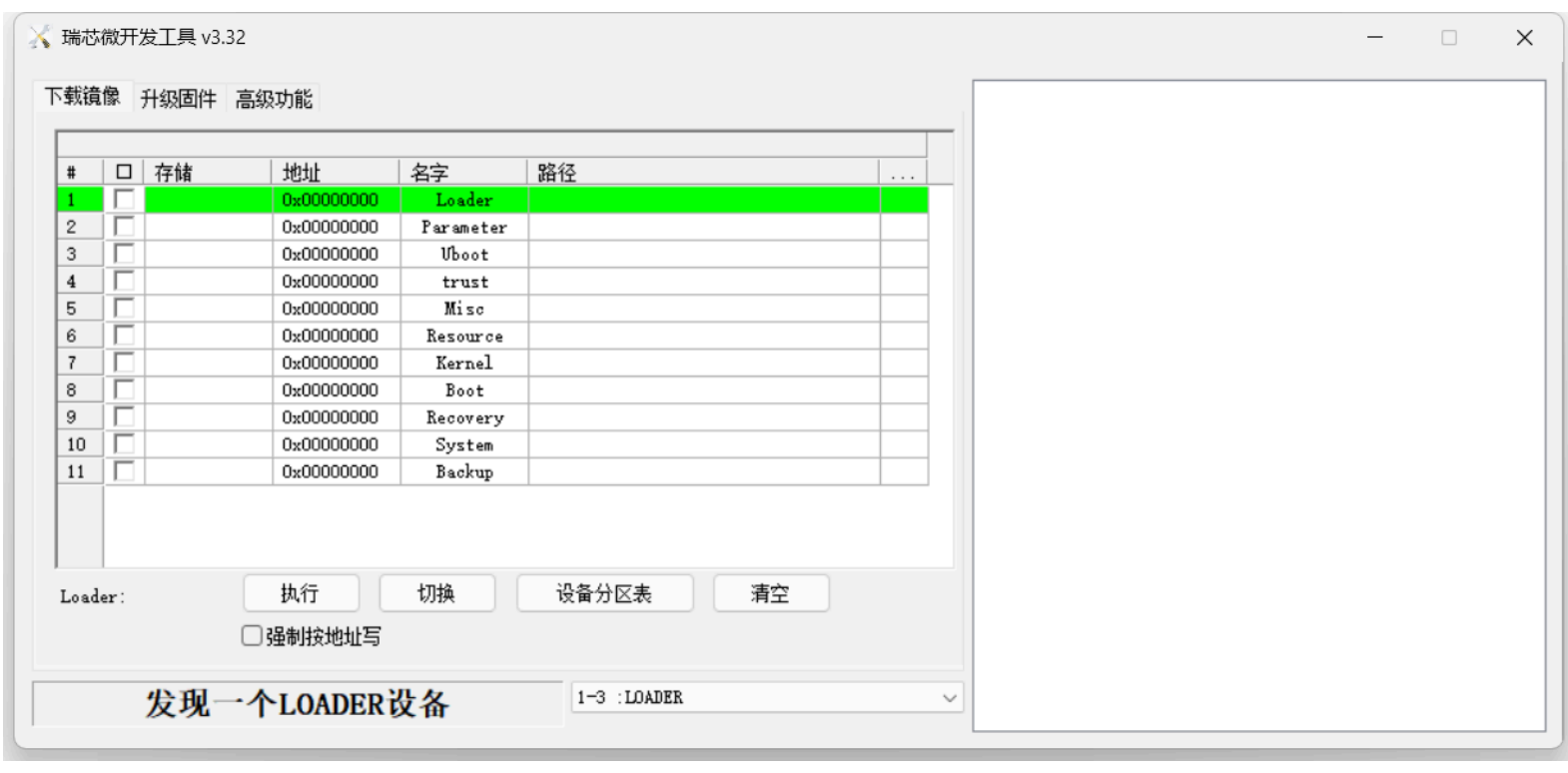
打开软件后如下图：

如果没有出现 LOADER 设备， 而是出现的是一个ADB设备， 说明设备没有正确启动， 我们可以使用切换功能进行切换。



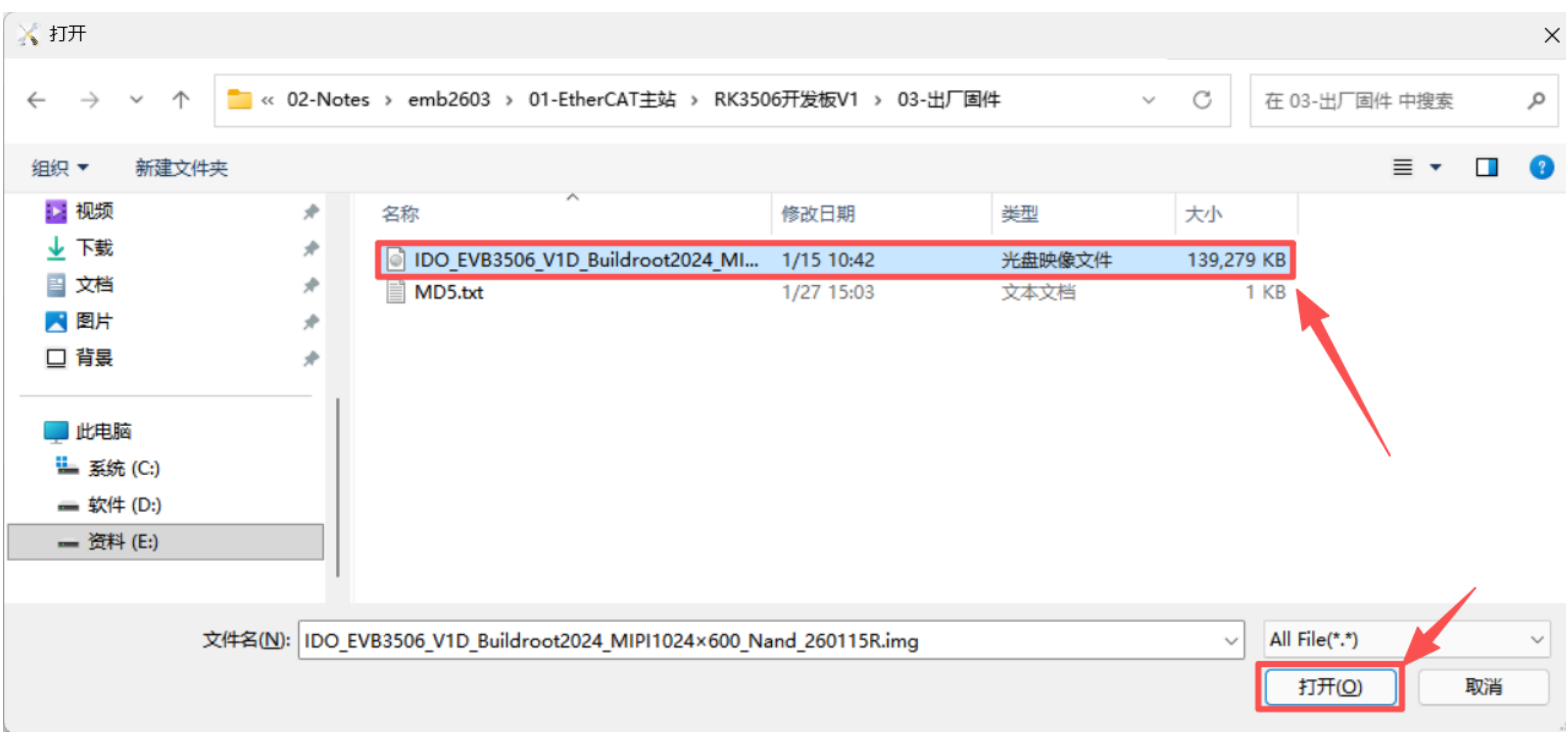
注意注意注意， 只有ADB 设备可以切换为 LOADER设备， 别的模式都不可以。

等待系统复位后，会自动变成 LOADER 设备， 切换成功如下图：



点击 [升级固件] -> [固件] 选择要烧录的固件，如下图所示：

这里选择要下载的固件： `RK3506开发板V1\03-出厂固件\IDO_EVB3506_V1D_Buildroot2024_MIPI1024×600_Nand_260115R.img`



点击升级固件，下载完成后右边会有下载成功提示。

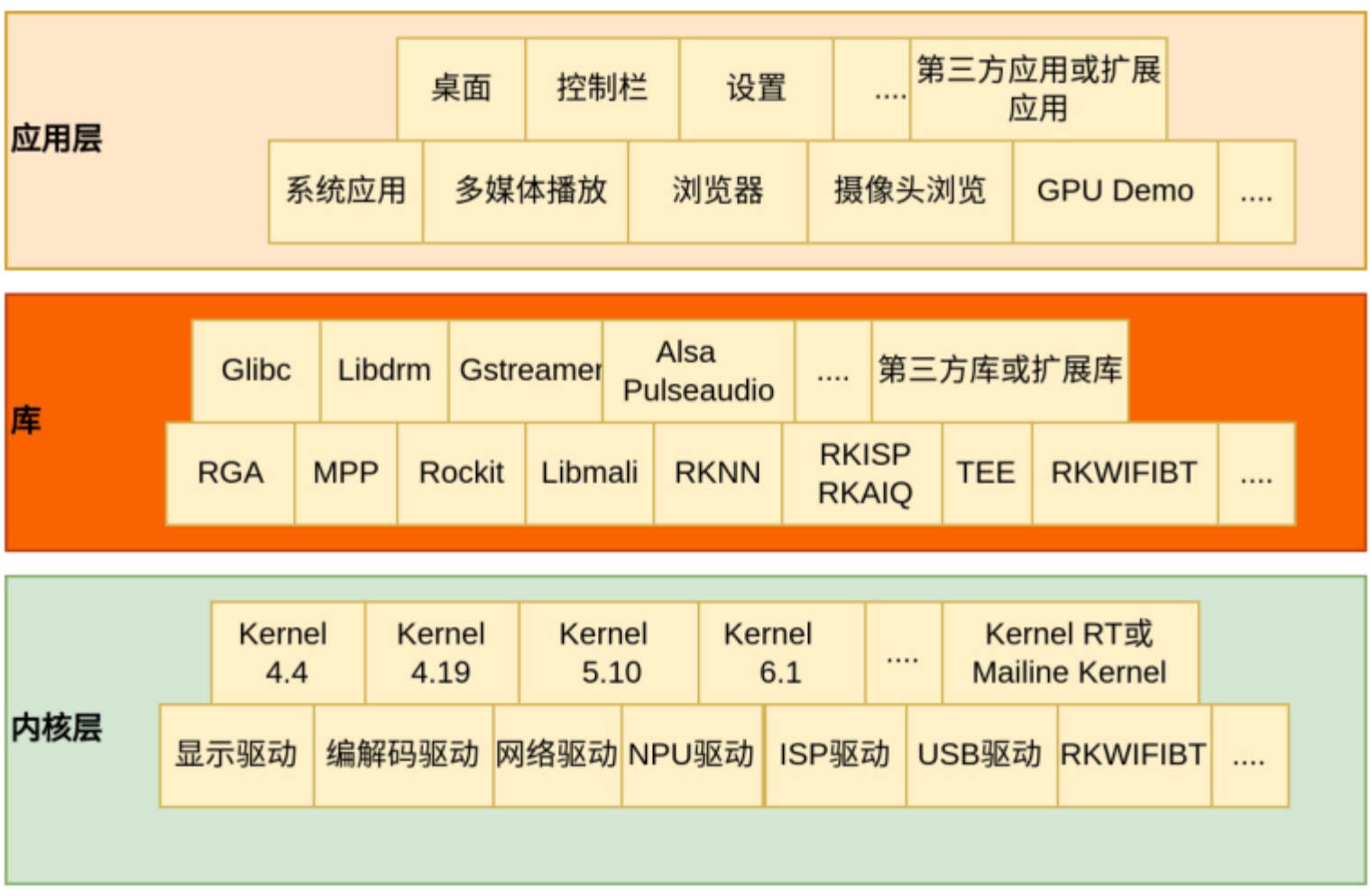
主板完成烧录后等待系统启动完成并且软件底部会显示出“发现一个ADB设备”，代表烧录成功，如下图所示:

液晶屏显示如下内容，表示固件更新完成：

至此固件更新完成。

第05章 SDK软件架构

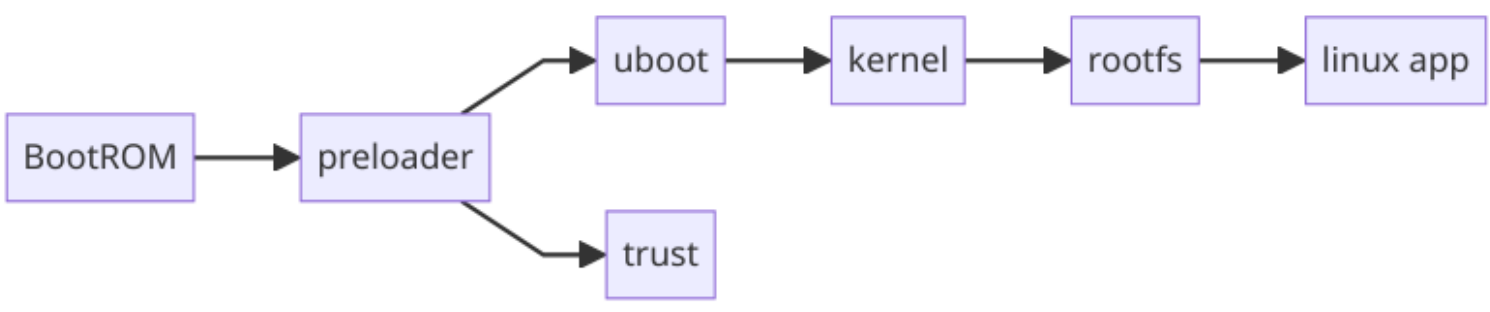
5.1 SDK软件架构



SDK软件框如图所示，从下至上分为启动层、内核层、库、应用层四个层次。各层次内容如下:

- **启动层**主要提供系统启动，如BootROM、U-Boot、ATF等相关支持。
- **内核层**主要提供LinuxKernel的标准实现，Linux也是一个开放的操作系统，Rockchip平台的Linux核心为标准的Linux4.4/4.19/5.10内核，提供安全性，内存管理，进程管理，网络协议栈等基础支持；主要是通过Linux内核管理设备硬件资源，如CPU调度、缓存、内存、I/O等。
- **库层**对应一般嵌入式系统，相当于中间件层次。包含了各种系统基础库，及第三方开源程序库支持，对应用层提供API接口，系统定制者和应用开发者可以基于Libraries层的API开发新的应用。
- **应用层**主要是实现具体的产品功能及交互逻辑，需要一些系统基础库及第三方程序库支持，开发者可以开发实现自己的应用程序，提供系统各种能力给到最终用户。

SDK 系统启动流



说明： AP和MCU内部都有集成一个BOOTROM，系统上电时先会运行BOOTROM代码，然后BOOTROM代码会探测外设存储器并加载Loader代码。

Pre Loader 目前有3 个：miniloader (开源)，uboot spl 和 loader。

5.2 安装环境搭建

安装必要的库

```
linux@ubuntu:~/emb/15-rk3506$ sudo apt install -y git gnupg flex bison gperf build-essential zip curl libc6-dev libncurses5-dev:i386 x11proto-core-dev libx11-dev:i386 libreadline-dev:i386 g++-multilib tofrodos markdown libxml2-utils xsltproc zlib1g-dev:i386 gawk texinfo gettext build-essential gcc libncurses5-dev bison flex zlib1g-dev gettext libssl-dev autoconf libtool linux-libc-dev:i386 wget patch dos2unix tree u-boot-tools libelf-dev libncurses5 lz4 python-is-python3 libgmp-dev libmpc-dev expect expect-dev python2
```

5.3 获取SDK资源包

1. 解压源码

下载路径: `RK3506开发板V1\04-源代码\rk3506_linux6.1_v1.2.0.tar.gz`

复制 rk3506_linux6.1_v1.2.0.tar.gz 源码到 window的共享盘目录内 `E:\YShengli\10-linuxshare` 。

剪切源码到指定的目录内

```
linux@ubuntu:~$ cd emb/
linux@ubuntu:~/emb$ mv /mnt/hgfs/linuxshare/rk3506_linux6.1_v1.2.0.tar.gz .
linux@ubuntu:~/emb$ tar xvf rk3506_linux6.1_v1.2.0.tar.gz -C ./
rk3506_linux6.1_v1.2.0/
rk3506_linux6.1_v1.2.0/.git/
rk3506_linux6.1_v1.2.0/.git/hooks/
rk3506_linux6.1_v1.2.0/.git/hooks/pre-receive.sample
rk3506_linux6.1_v1.2.0/.git/hooks/update.sample
rk3506_linux6.1_v1.2.0/.git/hooks/post-update.sample
......
rk3506_linux6.1_v1.2.0/.git/logs/refs/remotes/
rk3506_linux6.1_v1.2.0/.git/logs/refs/remotes/origin/
rk3506_linux6.1_v1.2.0/.git/logs/refs/remotes/origin/HEAD
rk3506_linux6.1_v1.2.0/.git/logs/refs/heads/
rk3506_linux6.1_v1.2.0/.git/logs/refs/heads/ido-evb3506-preempt-rt
rk3506_linux6.1_v1.2.0/.git/logs/refs/heads/ido-evb3506
rk3506_linux6.1_v1.2.0/.git/logs/refs/heads/ido-evb3506-xenomai-rt
rk3506_linux6.1_v1.2.0/.git/packed-refs
rk3506_linux6.1_v1.2.0/.git/branches/
rk3506_linux6.1_v1.2.0/.git/config
linux@ubuntu:~/emb$ mv rk3506_linux6.1_v1.2.0 15-rk3506
```

```
linux@ubuntu:~/emb$ cd 15-rk3506/
linux@ubuntu:~/emb/15-rk3506$ ls
linux@ubuntu:~/emb/15-rk3506$ git branch -a    #查看分支(默认分支为ido-evb3506)
* ido-evb3506
  ido-evb3506-preempt-rt
  ido-evb3506-xenomai-rt
  remotes/origin/HEAD -> origin/master
  remotes/origin/ido-evb3506
  remotes/origin/ido-evb3506-preempt-rt
  remotes/origin/ido-evb3506-xenomai-rt
  remotes/origin/master
linux@ubuntu:~/emb/15-rk3506$
linux@ubuntu:~/emb/15-rk3506$ git reset --hard
正在更新文件: 100% (198820/198820), 完成.
HEAD 现在位于 43f0c25a8 Release ido-evb3506 v1.0
linux@ubuntu:~/emb/15-rk3506$
```

2. 名字解释

这两个分支名称清楚地表明，它们是为 `ido-evb3506` 这一硬件平台提供的两种不同实时（RT）操作系统方案。它们的核心功能如下：

- `remotes/origin/ido-evb3506-preempt-rt`
- **实时方案：PREEMPT_RT**
- **功能说明**：
  - 该分支包含了将标准 Linux 内核通过 **PREEMPT_RT 补丁** 改造为完全可抢占的实时内核的代码与配置。
  - 内核中几乎所有的临界区都被可抢占的锁机制（如 rt_mutex）替代，使得即使是内核态任务也能被高优先级任务抢占。
  - 编程模型依然完全遵循 **标准 Linux/POSIX 接口**，应用程序无需改动或只需少量调整即可获得确定性延时。
  - 适用于绝大多数对实时性有要求、但希望沿用成熟 Linux 生态和开发习惯的场景（如工业控制、数据采集）。

- `remotes/origin/ido-evb3506-xenomai-rt`
- **实时方案：Xenomai**（通常搭配 Cobalt 内核）
- **功能说明**：
  - 该分支集成了 **Xenomai 实时框架**，通常采用双内核架构，在 Linux 内核旁增加一个微内核（实时核）来调度实时任务。
  - 所有实时任务可以直接运行在 Xenomai 的实时核上，中断和调度延时极低，抖动极小，可实现 **硬实时** 性能。
  - 应用程序需要使用 Xenomai 提供的 **实时 API**（如原生 Alchemy API、POSIX 皮肤等），与普通 Linux 程序有所区别。
  - 适合对实时确定性要求极为严苛的场景（如电机精密控制、高频同步、硬实时通讯）。
- `remotes/origin/ido-evb3506`
- **方案定位：标准 Linux 内核（无实时补丁）**
- **功能说明**：
  该分支是针对 `ido-evb3506` 硬件平台的**基础 BSP（板级支持包）分支**。它包含了使 Linux 能在这个特定板卡上正常运行的所需代码（如设备驱动、设备树、配置文件等），但**没有集成任何专门的实时扩展**。
  - **既没有打 PREEMPT_RT 补丁**，因此内核不是完全可抢占的，中断和调度延时相比实时内核要大且存在不可控抖动。
  - **也没有集成 Xenomai 双核架构**，因此不能提供硬实时 API，只能使用标准 Linux 的调度策略（如 `SCHED_FIFO` / `SCHED_RR`），实时性较弱。
  - 使用标准 POSIX 接口，所有 Linux 应用程序无需修改即可直接运行。
- **适用场景**：
  - 对实时性没有严苛要求的一般嵌入式应用（如显示交互、数据记录、网络通信、轻量控制等）。
  - 作为对比基准，评估开启 PREEMPT_RT 或 Xenomai 后的实时性能提升。
  - 当作构建其他衍生分支（如前两个实时分支）的**基线代码**，开发者可能在此分支基础上合入实时补丁。

3. SDK目录介绍

```
linux@ubuntu:~/rk3506$ tree -L 1   # 显示目录
.
├── app
├── buildroot
├── build.sh -> device/rockchip/common/scripts/build.sh
├── device
├── docs
├── envsetup.sh -> buildroot/scripts/envsetup.sh
├── external
├── hal
├── kernel -> kernel-6.1
├── kernel-6.1
├── Makefile -> device/rockchip/common/Makefile
├── oebuild.log
├── prebuilts
├── rkbin
├── rkflash.sh -> device/rockchip/common/scripts/rkflash.sh
├── rootfs
├── rtos
├── tools
├── u-boot
└── yocto

15 directories, 5 files
linux@ubuntu:~/rk3506$
```

目录解释

```
# app: 存放上层应用APP，主要是一些应用Demo。
# buildroot:基于 Buildroot(2024)开发的根文件系统。
# device/rockchip:存放芯片板级配置以及SDK编译和打包固件的脚本和文件等。
# docs:存放通用开发指导文档、芯片平台相关文档、Linux系统开发相关文档、其他参考文档等。
# external:存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
# kernel:存放Kernel开发的代码。
# output:存放每次生成的固件信息、编译信息、XML、主机环境等。
# prebuilts:存放交叉编译工具链。
# rkbin:存放Rockchip 相关二进制和工具。
# rockdev:存放编译输出固件，实际软链接到 output/firmware。
# tools:存放Linux 和Window 操作系统下常用工具。
# u-boot:存放基于 v2017.09 版本进行开发的U-Boot 代码。
# rtos:存放rtos相关源码。
# yocto:存放yocto相关编译脚本和代码。
```

4. 配置文件介绍

在 device/rockchip/rk3506 目录下，有不同板型的配置文件(xxxx_defconfig)，用于管理SDK每个环节的编译配置。以 device/rockchip/rk3506/rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig 做介绍:

```
RK_ROOTFS_UBI=y                         # UBI文件系统支持 ， 是nandflash的文件系统
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506b rk3506_tb"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="ido-evb3506-v1a-flash-mipi-720x720"  # 板子设备树的名字
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS_NAME="thunderboot.its"                   #快速启动配置
RK_RECOVERY_FIT_ITS_NAME="thunderboot4recovery.its"      #recovery 固件编译打包使用的 its的配置
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_PARAMETER="parameter-flash.txt"                       #打包使用的分区表
RK_USE_FIT_IMG=y
```

5. 分区信息

parameter-flash.txt 文件中包含了固件的分区信息，以parameter-flash.txt 为例:

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3506
MACHINE_ID:007
MANUFACTURER: RK3506
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3506
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0
CMDLINE:mtdparts=:0x00003000@0x00002000(uboot),0x00000800@0x00005000(misc),0x00000200@0x00005800(vnvm),0x000
09000@0x00005A00(recovery),0x00005000@0x0000EA00(boot),0x00050000@0x00013A00(rootfs),0x00008000@0x00063A00(o
em),-@0x0006BA00(userdata:grow)
```

命令格式解析

`mtdparts=:<size>@<offset>(<name>),<size>@<offset>(<name>),...`

- `mtdparts=`：指定 MTD 分区表的命令行参数。

- **冒号** `:` 前没有设备名，表示该分区表适用于默认或唯一的 MTD 设备（如 `mtd0`）。
- 每个分区由 **大小**、**@**、**起始地址** 和 **括号内的名称** 组成。
- 数字均为 **十六进制**，单位是 **字节**。
- 大小若为 `-`，表示占用从起始地址到设备末尾的所有剩余空间。
- `userdata:grow` 表示 `userdata` 分区在系统启动后可以动态扩展以利用全部剩余空间（常见于 Android/嵌入式系统，使最后分区可调整大小）。

分区表详细说明

分区名	起始地址 (hex)	起始地址 (dec)	大小 (hex)	大小 (dec)	典型用途
uboot	0x00002000	8,192 (8KB)	0x00003000	12,288 (12KB)	U-Boot 引导加载程序
misc	0x00005000	20,480 (20KB)	0x00000800	2,048 (2KB)	存放系统杂项配置（如恢复模式标记、bootloader 消息）
vnvm	0x00005800	22,528 (22KB)	0x00000200	512 (0.5KB)	非易失性存储器（Vendor NVM），可能保存设备关键校准数据（如 MAC 地址、序列号等）
recovery	0x00005A00	23,040 (22.5KB)	0x00009000	36,864 (36KB)	恢复模式内核/根文件系统（用于系统恢复）
boot	0x0000EA00	59,904 (58.5KB)	0x00005000	20,480 (20KB)	内核镜像或设备树（boot 分区）
rootfs	0x00013A00	80,384 (78.5KB)	0x00050000	327,680 (320KB)	主根文件系统
oem	0x00063A00	408,064 (398.5KB)	0x00008000	32,768 (32KB)	OEM 定制数据或应用
userdata	0x0006BA00	440,832 (430.5KB)	`-` (剩余所有)	取决于 Flash 总大小	用户数据分区，`grow` 表示可扩展至全部剩余空间

分区功能简述

- uboot 分区：供 uboot 编译出来的 uboot.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用。
- rootfs 分区：供 buildroot、debian 或 yocto 编出来的 rootfs.img。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录下

5.4 编译系统

1. build.sh 命令说明

```
linux@ubuntu:~/rk3506$ ./build.sh help
Log colors: message notice warning error fatal

Usage: build.sh [OPTIONS]
Available options:
chip[:<chip>[:<config>]]                choose chip
defconfig[:<config>]                    choose defconfig
 *_defconfig                            switch to specified defconfig
    available defconfigs:
    rockchip_ido_evb3506_v1_emmc_mipi_1024x600_defconfig
    rockchip_ido_evb3506_v1_emmc_mipi_1280x800_defconfig
    rockchip_ido_evb3506_v1_emmc_mipi_720x720_amp_defconfig
    rockchip_ido_evb3506_v1_emmc_mipi_720x720_defconfig
    rockchip_ido_evb3506_v1_emmc_rgb_1024x600_defconfig
    rockchip_ido_evb3506_v1_flash_mipi_1024x600_defconfig
    rockchip_ido_evb3506_v1_flash_mipi_1280x800_defconfig
    rockchip_ido_evb3506_v1_flash_mipi_720x720_amp_defconfig
    rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
    rockchip_ido_evb3506_v1_flash_rgb_1024x600_defconfig
 olddefconfig                           resolve any unresolved symbols in .config
 savedefconfig                          save current config to defconfig
 menuconfig                             interactive curses-based configurator
config                                  modify SDK defconfig
print-parts                             print partitions
list-parts                              alias of print-parts
mod-parts                               interactive partition table modify
edit-parts                              edit raw partitions
new-parts:<offset>:<name>:<size>...     re-create partitions
insert-part:<idx>:<name>[:<size>]       insert partition
del-part:(<idx>|<name>)                 delete partition
move-part:(<idx>|<name>):<idx>          move partition
rename-part:(<idx>|<name>):<name>       rename partition
resize-part:(<idx>|<name>):<size>       resize partition
misc                                    pack misc image
kernel-6.1[:dry-run]                    build kernel 6.1
kernel[:dry-run]                        build kernel
recovery-kernel[:dry-run]               build kernel for recovery
kernel-modules[:<dst dir>:dry-run]      build kernel modules
modules[:<dst dir>:dry-run]             alias of kernel-modules
linux-headers[:<arch>:dry-run]          build linux-headers
kernel-config[:dry-run]                 modify kernel defconfig
kconfig[:dry-run]                       alias of kernel-config
kernel-make[:<arg1>:<arg2>]             run kernel make
kmake[:<arg1>:<arg2>]                   alias of kernel-make
wifibt[:<dst dir>[:<chip>]]             build Wifi/BT
amp                                     build and pack amp system
rtos                                    build RT-Thread
hal                                     build HAL
hal_mcu                                 build HAL MCU
buildroot-config[:<config>]             modify buildroot defconfig
bconfig[:<config>]                      alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]          run buildroot make
bmake[:<arg1>:<arg2>]                   alias of buildroot-make
buildroot-sdk                           build the buildroot SDK tarball
bsdk                                    alias of buildroot-sdk
rootfs[:<rootfs type>]                  build default rootfs
buildroot                               build buildroot rootfs
yocto                                   build yocto rootfs
debian                                  build debian rootfs
ubuntu                                  build ubuntu rootfs
recovery                                build recovery
security-createkeys                     create keys for security
security-misc                           build misc with system encryption key
security-ramboot[:system_image]         build security ramboot
security-system[:system_image]          build security system
loader[:dry-run]                        build loader (u-boot)
uboot[:dry-run]                         build u-boot
u-boot[:dry-run]                        alias of uboot
extra-parts                             pack extra partition images
firmware                                pack and check firmwares
edit-package-file                       edit package-file
edit-ota-package-file                   edit package-file for OTA
updateimg                               build update image
ota-updateimg                           build update image for OTA
all                                     build images
release[:<subdir>[:<name>]]             release images and build info
all-release[:<subdir>[:<name>]]         build and release images
shell                                   setup a shell for developing
buildroot-shell                         setup a shell for buildroot developing
bshell                                  alias of buildroot-shell
yocto-shell                             setup a shell for yocto developing
yshell                                  alias of yocto-shell
cleanall                                cleanup all
clean-extra-parts                       cleanup extra-parts
clean-config                            cleanup config
clean-rootfs                            cleanup rootfs
clean-firmware                          cleanup firmware
clean-loader                            cleanup loader
clean-security                          cleanup security
clean-amp                               cleanup amp
clean-kernel                            cleanup kernel
clean-updateimg                         cleanup updateimg
clean-misc                              cleanup misc
clean-recovery                          cleanup recovery
post-rootfs <rootfs dir>                trigger post-rootfs hook scripts
help                                    display this information

Default option is 'all'.
linux@ubuntu:~/emb/15-rk3506$
```

1. 选择项目配置

```
linux@ubuntu:~/rk3506$ ./build.sh lunch
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-12_15-29-23
Pick a defconfig:

1. rockchip_ido_evb3506_v1_emmc_mipi_1024x600_defconfig
2. rockchip_ido_evb3506_v1_emmc_mipi_1280x800_defconfig
3. rockchip_ido_evb3506_v1_emmc_mipi_720x720_amp_defconfig
4. rockchip_ido_evb3506_v1_emmc_mipi_720x720_defconfig
5. rockchip_ido_evb3506_v1_emmc_rgb_1024x600_defconfig
6. rockchip_ido_evb3506_v1_flash_mipi_1024x600_defconfig
7. rockchip_ido_evb3506_v1_flash_mipi_1280x800_defconfig
8. rockchip_ido_evb3506_v1_flash_mipi_720x720_amp_defconfig
9. rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
10. rockchip_ido_evb3506_v1_flash_rgb_1024x600_defconfig
Which would you like? [1]: 9
Switching to defconfig:
/home/linux/rk3506/device/rockchip/.chip/rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
mkdir -p /home/linux/rk3506/output/kconf/lxdialog
make CC="gcc" HOSTCC="gcc" \
    obj=/home/linux/rk3506/output/kconf -C /home/linux/rk3506/device/rockchip/common/kconfig -f Makefile.br
conf
make[1]: Entering directory '/home/linux/rk3506/device/rockchip/common/kconfig'
gcc -I/usr/include/ncursesw -DCURSES_LOC="<curses.h>"  -DNCURSES_WIDECHAR=1 -DLOCALE  -
I/home/linux/rk3506/output/kconf -DCONFIG_=\"\"  -MM *.c > /home/linux/rk3506/output/kconf/.depend
2>/dev/null || :
gcc -I/usr/include/ncursesw -DCURSES_LOC="<curses.h>"  -DNCURSES_WIDECHAR=1 -DLOCALE  -
I/home/linux/rk3506/output/kconf -DCONFIG_=\"\"   -c conf.c -o /home/linux/rk3506/output/kconf/conf.o
gcc -I/usr/include/ncursesw -DCURSES_LOC="<curses.h>"  -DNCURSES_WIDECHAR=1 -DLOCALE  -
I/home/linux/rk3506/output/kconf -DCONFIG_=\"\"  -I. -c /home/linux/rk3506/output/kconf/zconf.tab.c -o
/home/linux/rk3506/output/kconf/zconf.tab.o
gcc -I/usr/include/ncursesw -DCURSES_LOC="<curses.h>"  -DNCURSES_WIDECHAR=1 -DLOCALE  -
I/home/linux/rk3506/output/kconf -DCONFIG_=\"\"   /home/linux/rk3506/output/kconf/conf.o
/home/linux/rk3506/output/kconf/zconf.tab.o  -o /home/linux/rk3506/output/kconf/conf
rm /home/linux/rk3506/output/kconf/zconf.tab.c
make[1]: Leaving directory '/home/linux/rk3506/device/rockchip/common/kconfig'
#
# configuration written to /home/linux/rk3506/output/.config
#
Using current kernel version(6.1)
linux@ubuntu:~/rk3506$
```

解析语句： Log colors: message notice warning error fatal

```
Log colors: message notice warning error fatal
#它只是在告诉你：在之后的编译过程中，不同类型的日志会以不同颜色显示，级别包括
# message（普通信息）
# notice（注意）
# warning（警告）
# error（错误）
# fatal（致命错误）
```

2. 编译全部系统

时间需要1小时左右，根据电脑情况而定。

```
linux@ubuntu:~/emb/15-rk3506$ ./build.sh
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-23_17-58-12
Using last kernel version(6.1)

......
=========================================
          Start building U-Boot
=========================================

Your python2 is missing for U-Boot
Please install it:
wget https://www.python.org/ftp/python/2.7.18/Python-2.7.18.tgz
tar xf Python-2.7.18.tgz
cd Python-2.7.18
sudo apt-get install libsqlite3-dev
./configure --enable-optimizations
sudo make install -j8

ERROR: Running /home/linux/emb/15-rk3506/device/rockchip/common/scripts/mk-loader.sh - build_uboot failed!
ERROR: exit code 1 from line 10:
    "$RK_SCRIPTS_DIR/check-loader.sh"
ERROR: call stack:
    mk-loader.sh: build_uboot(10)
    mk-loader.sh: build_hook(118)
    mk-loader.sh: main(132)
ERROR: Running /home/linux/emb/15-rk3506/device/rockchip/common/build-hooks/99-all.sh - build_all failed!
ERROR: exit code 1 from line 19:
    "$RK_SCRIPTS_DIR/mk-loader.sh"
ERROR: call stack:
    99-all.sh: build_all(19)
    99-all.sh: build_hook(136)
    build-helper: try_func(63)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
ERROR: Running /home/linux/emb/15-rk3506/device/rockchip/common/build-hooks/99-all.sh - try_func build_hook
all failed!
ERROR: exit code 1 from line 67:
    build_hook
ERROR: call stack:
    build-helper: try_func(67)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
linux@ubuntu:~/emb/15-rk3506$
```

编译缺少**U-Boot 构建依赖 Python 2，但你的系统没有安装它**，我们重新安装Python2 即可。

```
linux@ubuntu:~/emb/15-rk3506$ sudo apt install -y python2
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件：
  libpython2-stdlib libpython2.7-minimal libpython2.7-stdlib python2-minimal python2.7 python2.7-minimal
建议安装：
  python2-doc python-tk python2.7-doc binfmt-support
下列【新】软件包将被安装：
  libpython2-stdlib libpython2.7-minimal libpython2.7-stdlib python2 python2-minimal python2.7 python2.7-
minimal
升级了 0 个软件包，新安装了 7 个软件包，要卸载 0 个软件包，有 215 个软件包未被升级。
需要下载 4,012 kB 的归档。
解压缩后会消耗 16.2 MB 的额外空间。
获取:1 http://mirrors.aliyun.com/ubuntu jammy-updates/universe amd64 libpython2.7-minimal amd64 2.7.18-
13ubuntu1.5 [347 kB]
......
正在处理用于 mailcap (3.70+nmu1ubuntu1) 的触发器 ...
linux@ubuntu:~/emb/15-rk3506$
linux@ubuntu:~/emb/15-rk3506$ python2 --version   # 应输出 Python 2.7.18
Python 2.7.18
linux@ubuntu:~/emb/15-rk3506$
```

错误分析：

```
2026-06-12T15:45:48 >>> host-make 4.3 Downloading
Done in 8s  (error code: 2)
Failed to build rockchip_rk3506:
# 2026-06-12 15:45:44
2026-06-12T15:45:44 make: Entering directory '/home/linux/rk3506/buildroot'
2026-06-12T15:45:48 /usr/bin/make -j1  O=/home/linux/rk3506/buildroot/output/rockchip_rk3506
HOSTCC="/usr/bin/gcc" HOSTCXX="/usr/bin/g++" syncconfig
2026-06-12T15:45:48 GEN     /home/linux/rk3506/buildroot/output/rockchip_rk3506/Makefile
2026-06-12T15:45:48 >>> host-make 4.3 Downloading
2026-06-12T15:45:48 wget --passive-ftp -nd -t 3 --no-check-certificate -O
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/build/.make-4.3.tar.lz.5ZJ7KT/output'
'https://sources.buildroot.net/make/make-4.3.tar.lz'
2026-06-12T15:45:48 --2026-06-12 15:45:48--  https://sources.buildroot.net/make/make-4.3.tar.lz
2026-06-12T15:45:48 Resolving sources.buildroot.net (sources.buildroot.net)... 104.26.1.37, 104.26.0.37,
172.67.72.56, ...
2026-06-12T15:45:49 Connecting to sources.buildroot.net (sources.buildroot.net)|104.26.1.37|:443...
connected.
2026-06-12T15:45:50 HTTP request sent, awaiting response... 404 Not Found
2026-06-12T15:45:50 2026-06-12 15:45:50 ERROR 404: Not Found.
2026-06-12T15:45:50
```

```
2026-06-12T15:45:50 wget --passive-ftp -nd -t 3 --no-check-certificate -O
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/build/.make-4.3.tar.lz.dh19uZ/output'
'https://sources.buildroot.net/make-4.3.tar.lz'
2026-06-12T15:45:50 --2026-06-12 15:45:50--  https://sources.buildroot.net/make-4.3.tar.lz
2026-06-12T15:45:50 Resolving sources.buildroot.net (sources.buildroot.net)... 172.67.72.56, 104.26.0.37,
104.26.1.37, ...
2026-06-12T15:45:51 Connecting to sources.buildroot.net (sources.buildroot.net)|172.67.72.56|:443...
connected.
2026-06-12T15:45:51 HTTP request sent, awaiting response... 404 Not Found
2026-06-12T15:45:51 2026-06-12 15:45:51 ERROR 404: Not Found.
2026-06-12T15:45:51
2026-06-12T15:45:51 wget --passive-ftp -nd -t 3 --no-check-certificate -O
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/build/.make-4.3.tar.lz.k8O9wd/output'
'https://mirrors.ustc.edu.cn/gnu/make/make-4.3.tar.lz'
2026-06-12T15:45:51 --2026-06-12 15:45:51--  https://mirrors.ustc.edu.cn/gnu/make/make-4.3.tar.lz
2026-06-12T15:45:51 Resolving mirrors.ustc.edu.cn (mirrors.ustc.edu.cn)... 202.141.160.110,
2001:da8:d800:95::110
2026-06-12T15:45:51 Connecting to mirrors.ustc.edu.cn (mirrors.ustc.edu.cn)|202.141.160.110|:443...
connected.
2026-06-12T15:45:52 HTTP request sent, awaiting response... 403 Forbidden
2026-06-12T15:45:52 2026-06-12 15:45:52 ERROR 403: Forbidden.
2026-06-12T15:45:52
2026-06-12T15:45:52 make[1]: *** [package/pkg-generic.mk:196:
/home/linux/rk3506/buildroot/output/rockchip_rk3506/build/host-make-4.3/.stamp_downloaded] Error 1
2026-06-12T15:45:52 make: *** [Makefile:112: _all] Error 2
2026-06-12T15:45:52 make: Leaving directory '/home/linux/rk3506/buildroot'
2026-06-12T15:45:52 make: Leaving directory '/home/linux/rk3506/buildroot'
Please check details in /home/linux/rk3506/output/sessions/2026-06-12_15-45-30/br-rockchip_rk3506_2026-06-
12_15-45-44.log
ERROR: Running /home/linux/rk3506/device/rockchip/common/scripts/mk-rootfs.sh - build_buildroot failed!
ERROR: exit code 1 from line 16:
    "$RK_SCRIPTS_DIR/mk-buildroot.sh" $RK_BUILDROOT_CFG "$IMAGE_DIR"
ERROR: call stack:
    mk-rootfs.sh: build_buildroot(16)
    mk-rootfs.sh: build_hook(365)
    mk-rootfs.sh: main(421)
ERROR: Running /home/linux/rk3506/device/rockchip/common/build-hooks/99-all.sh - build_all failed!
ERROR: exit code 1 from line 21:
    "$RK_SCRIPTS_DIR/mk-rootfs.sh"
ERROR: call stack:
    99-all.sh: build_all(21)
    99-all.sh: build_hook(136)
    build-helper: try_func(63)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
ERROR: Running /home/linux/rk3506/device/rockchip/common/build-hooks/99-all.sh - try_func build_hook all
failed!
ERROR: exit code 1 from line 67:
    build_hook
ERROR: call stack:
    build-helper: try_func(67)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
linux@ubuntu:~/rk3506$
```

解决办法:

```
https://mirrors.ustc.edu.cn/gnu/make/make-4.3.tar.lz
```

复制网址到浏览器内， 手动下载后， 复制文件到 `buildroot/dl/make/` 目录内。

```
linux@ubuntu:~/rk3506$ ls buildroot/dl/make/
make-4.3.tar.lz
linux@ubuntu:~/rk3506$
```

再次编译内核， 出现问题以此类推。

```
linux@ubuntu:~/emb/15-rk3506$ ./build.sh
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-23_17-58-12
Using last kernel version(6.1)
......
uboot.img(/home/linux/emb/15-rk3506/u-boot/uboot.img): 4.0M
userdata.img(/home/linux/emb/15-rk3506/output/extra-parts/userdata.img): 2.4M
Making update image...
=========================================
          Start packing update image
=========================================
Generating package-file for update:
# NAME  PATH
package-file    package-file
parameter   parameter.txt
bootloader  MiniLoaderAll.bin
uboot   uboot.img
misc    misc.img
recovery    recovery.img
boot    boot.img
rootfs  rootfs.img
oem oem.img
userdata    userdata.img
Packing /home/linux/emb/15-rk3506/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
------ PACKAGE ------
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xd1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x1ec,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x449c0,userspace=0x8a
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x46800,size=0x400000,userspace=0x800,flash_address=0x00002000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x446800,size=0xc000,userspace=0x18,flash_address=0x00005000
Add file: ./recovery.img
recovery,Add file: ./recovery.img
done,offset=0x452800,size=0xf09000,userspace=0x1e12,flash_address=0x00005a00
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x135b800,size=0x405a00,userspace=0x80c,flash_address=0x0000ea00
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x1761800,size=0x6620000,userspace=0xcc40,flash_address=0x00013a00
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x7d81800,size=0x660000,userspace=0xcc0,flash_address=0x00063a00
Add file: ./userdata.img
userdata,Add file: ./userdata.img
done,offset=0x83e1800,size=0x260000,userspace=0x4c0,flash_address=0x0006ba00
Add CRC...
Make firmware OK!
------ OK ------
********rkImageMaker ver 2.29********
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/linux/emb/15-rk3506/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
linux@ubuntu:~/emb/15-rk3506$
```

- 生成文件目录

```
linux@ubuntu:~/rk3506$ ls rockdev
boot.img  MiniLoaderAll.bin  misc.img  oem.img  parameter.txt  recovery.img  rootfs.img  uboot.img
update.img  userdata.img
linux@ubuntu:~/emb/15-rk3506$
linux@ubuntu:~/emb/15-rk3506$ du -mh  output/update/Image/update.img
135M    output/update/Image/update.img
linux@ubuntu:~/emb/15-rk3506$
```

3. 编译uboot系统

```
linux@ubuntu:~/rk3506$ ./build.sh  uboot
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-24_17-08-15
```

```
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/emb/15-rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
......
 COPY    spl/u-boot-spl.dtb
  CC      spl/arch/arm/mach-rockchip/spl.o
  CC      spl/lib/display_options.o
  LD      spl/common/spl/built-in.o
  LD      spl/arch/arm/mach-rockchip/built-in.o
  LD      spl/lib/built-in.o
  LD      spl/u-boot-spl
  OBJCOPY spl/u-boot-spl-nodtb.bin
  CAT     spl/u-boot-spl-dtb.bin
  COPY    spl/u-boot-spl.bin
  CFGCHK  u-boot.cfg
pack u-boot.itb okay! Input: /home/linux/emb/15-rk3506/rkbin/RKTRUST/RK3506TOS.ini

FIT description: FIT Image with ATF/OP-TEE/U-Boot/MCU
Created:         Sun May 24 17:08:19 2026
 Image 0 (uboot)
  Description:  U-Boot
  Created:      Sun May 24 17:08:19 2026
  Type:         Standalone Program
  Compression:  uncompressed
  Data Size:    679880 Bytes = 663.95 KiB = 0.65 MiB
  Architecture: ARM
  Load Address: 0x00200000
  Entry Point:  unavailable
  Hash algo:    sha256
  Hash value:   0dd0732f85f7950ce4f8f6b2b3b1f033ba7a4c36f092064aa50b3b9930c4ace2
 Image 1 (optee)
  Description:  OP-TEE
  Created:      Sun May 24 17:08:19 2026
  Type:         Firmware
  Compression:  uncompressed
  Data Size:    124360 Bytes = 121.45 KiB = 0.12 MiB
  Architecture: ARM
  Load Address: 0x00001000
  Hash algo:    sha256
  Hash value:   e2a712a5330df72daa4e5c8900e98aa85e0880d0cc0e551f1fb18eb27488da2d
 Image 2 (fdt)
  Description:  U-Boot dtb
  Created:      Sun May 24 17:08:19 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    6385 Bytes = 6.24 KiB = 0.01 MiB
  Architecture: ARM
  Hash algo:    sha256
  Hash value:   959172d4d90b22346788fa35c2d66e909013413709074806c726d567bcf14954
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  rk3506-evb-tb
  Kernel:       unavailable
  Firmware:     optee
  FDT:          fdt
  Loadables:    uboot
********boot_merger ver 1.35********
Info:Pack loader ok.
creating new idblock from loader...
idblock binary saving at rk3506_idblock_v1.06.111.img
pack loader(SPL) okay! Input: /home/linux/emb/15-rk3506/rkbin/RKBOOT/RK3506BMINIALL.ini

/home/linux/emb/15-rk3506/u-boot
pack loader with new: spl/u-boot-spl.bin

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3506_idblock_v1.06.111.img (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/linux/emb/15-rk3506/rkbin/RKTRUST/RK3506TOS.ini

Platform RK3506 is build OK, with new .config(make   rk3506_tb.config -j32)
/home/linux/emb/15-rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-
gnueabihf/bin/arm-none-linux-gnueabihf-
Sun May 24 17:08:20 CST 2026
+ cd ..
Running 70-loader.sh - build_uboot succeeded.
linux@ubuntu:~/emb/15-rk3506$
```

固件编译后生成路径:rockdev/uboot.img

```
linux@ubuntu:~/emb/15-rk3506$ ls rockdev
boot.img  MiniLoaderAll.bin  misc.img  oem.img  parameter.txt  recovery.img  rootfs.img  uboot.img
update.img  userdata.img
linux@ubuntu:~/emb/15-rk3506$
```

4. 编译kernel系统

```
linux@ubuntu:~/emb/15-rk3506$ ./build.sh kernel
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-24_17-18-08
Using last kernel version(6.1)

.......
  SYNC    include/config/auto.conf.cmd
  CALL    scripts/checksyscalls.sh
  Kernel: arch/arm/boot/Image is ready
  Kernel: arch/arm/boot/Image.gz is ready
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  CC      arch/arm/boot/compressed/fdt_rw.o
  CC      arch/arm/boot/compressed/fdt_ro.o
  CC      arch/arm/boot/compressed/fdt_wip.o
  CC      arch/arm/boot/compressed/fdt.o
  CC      arch/arm/boot/compressed/fdt_check_mem_start.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
  Image:  resource.img (with ido-evb3506-v1a-flash-mipi-1024x600.dtb logo.bmp logo_kernel.bmp) is ready
  Image:  boot.img (with Image  resource.img) is ready
  Image:  zboot.img (with zImage  resource.img) is ready
make: Leaving directory '/home/linux/emb/15-rk3506/kernel-6.1'
+ /home/linux/emb/15-rk3506/device/rockchip/common/scripts/mk-fitimage.sh kernel/zboot.img
/home/linux/emb/15-rk3506/device/rockchip/.chip/thunderboot.its kernel/arch/arm/boot/zImage
kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-1024x600.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm
Created:         Sun May 24 17:18:15 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Sun May 24 17:18:15 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44465 Bytes = 43.42 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   05119a208c3b2eedff02c2652de2c8a5fd35f2f32408a120ac5d7daa563ea585
 Image 1 (kernel)
  Description:  unavailable
  Created:      Sun May 24 17:18:15 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4169496 Bytes = 4071.77 KiB = 3.98 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   e1080e62b3c1d10a597894296b4e61d81f4c95f6509b05bc752184c8463c0f5a
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and no ramdisk
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/linux/emb/15-rk3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-1024x600.dts
Running 10-kernel.sh - build_kernel succeeded.
```

```
linux@ubuntu:~/rk3506$
```

固件编译后生成路径:rockdev/boot.img

```
linux@ubuntu:~/rk3506$ ls rockdev
boot.img  MiniLoaderAll.bin  misc.img  oem.img  parameter.txt  recovery.img  rootfs.img  uboot.img
update.img  userdata.img
linux@ubuntu:~/rk3506$
```

5. 编译buildroot系统

```
linux@ubuntu:~/rk3506$ ./build.sh  buildroot
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-24_17-22-15
Using last kernel version(6.1)
Using rootfs system(buildroot) from cmdline

。。。。。。

Running 95-extra-parts.sh for /home/linux/emb/15-rk3506/buildroot/output/rockchip_rk3506/target (buildroot
init=busybox)...
Preparing extra partitions...
Running 97-ldcache.sh for /home/linux/emb/15-rk3506/buildroot/output/rockchip_rk3506/target (buildroot
init=busybox)...
Keep original /etc/ld.so.cache
Running 99-owner.sh for /home/linux/emb/15-rk3506/buildroot/output/rockchip_rk3506/target (buildroot
init=busybox)...
Running 30-rootfs.sh - build_buildroot /home/linux/emb/15-rk3506/output/buildroot/images succeeded.
Running 30-rootfs.sh - build_rootfs buildroot succeeded.
linux@ubuntu:~/rk3506$
```

固件编译后生成路径:rockdev/rootfs.img

```
linux@ubuntu:~/rk3506$ ls rockdev
boot.img  MiniLoaderAll.bin  misc.img  oem.img  parameter.txt  recovery.img  rootfs.img  uboot.img
update.img  userdata.img
linux@ubuntu:~/rk3506$ ls -l rockdev/
总计 0
lrwxrwxrwx 1 linux linux 26  6月 14 17:13 boot.img -> ../../kernel-6.1/zboot.img
lrwxrwxrwx 1 linux linux 44  6月 14 17:09 MiniLoaderAll.bin -> ../../u-boot/rk3506_spl_loader_v1.06.111.bin
lrwxrwxrwx 1 linux linux 11  6月 12 15:48 misc.img -> ../misc.img
lrwxrwxrwx 1 linux linux 22  6月 12 17:31 oem.img -> ../extra-parts/oem.img
lrwxrwxrwx 1 linux linux 55  6月 12 17:31 parameter.txt -> ../../device/rockchip/.chips/rk3506/parameter-
flash.txt
lrwxrwxrwx 1 linux linux 23  6月 12 17:31 recovery.img -> ../recovery/ramboot.img
lrwxrwxrwx 1 linux linux 56  6月 14 17:20 rootfs.img ->
../../buildroot/output/rockchip_rk3506/images/rootfs.ubi
lrwxrwxrwx 1 linux linux 22  6月 14 17:09 uboot.img -> ../../u-boot/uboot.img
lrwxrwxrwx 1 linux linux 26  6月 12 17:31 update.img -> ../update/Image/update.img
lrwxrwxrwx 1 linux linux 27  6月 12 17:31 userdata.img -> ../extra-parts/userdata.img
linux@ubuntu:~/rk3506$
```

6. 编译recovery

`./build.sh recovery` 是 Rockchip Linux SDK 构建系统中的一个分步编译命令，它的作用是**独立编译用于系统更新与恢复的精简版 Linux 系统（Recovery）**。

```
linux@ubuntu:~/emb/15-rk3506$ ./build.sh  recovery
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-24_17-44-47
Using last kernel version(6.1)

。。。。。。

  SYNC    include/config/auto.conf.cmd
  CALL    scripts/checksyscalls.sh
  Kernel: arch/arm/boot/Image is ready
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  CC      arch/arm/boot/compressed/fdt_rw.o
  CC      arch/arm/boot/compressed/fdt_ro.o
  CC      arch/arm/boot/compressed/fdt_wip.o
  CC      arch/arm/boot/compressed/fdt.o
  CC      arch/arm/boot/compressed/fdt_check_mem_start.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
make: Leaving directory '/home/linux/emb/15-rk3506/kernel-6.1'
+ make -C /home/linux/emb/15-rk3506/kernel/ -j17 CROSS_COMPILE=/home/linux/emb/15-
rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-
gnueabihf- ARCH=arm ido-evb3506-v1a-flash-mipi-1024x600.dtb
make: Entering directory '/home/linux/emb/15-rk3506/kernel-6.1'
make: Leaving directory '/home/linux/emb/15-rk3506/kernel-6.1'
+ ln -rsf arch/arm/boot/zImage /home/linux/emb/15-rk3506/output/recovery-kernel.img
+ ln -rsf arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-1024x600.dtb /home/linux/emb/15-
rk3506/output/recovery-kernel.dtb
+ scripts/resource_tool arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-1024x600.dtb logo.bmp logo_kernel.bmp
Pack to resource.img successed!
+ ln -rsf resource.img /home/linux/emb/15-rk3506/output/recovery-resource.img
Running mk-kernel.sh - build_recovery-kernel succeeded.
Packing /home/linux/emb/15-rk3506/output/recovery/ramboot.img...
FIT description: U-Boot FIT source file for arm
Created:         Sun May 24 17:45:12 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Sun May 24 17:45:12 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44465 Bytes = 43.42 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   05119a208c3b2eedff02c2652de2c8a5fd35f2f32408a120ac5d7daa563ea585
 Image 1 (kernel)
  Description:  unavailable
  Created:      Sun May 24 17:45:12 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4169496 Bytes = 4071.77 KiB = 3.98 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   e1080e62b3c1d10a597894296b4e61d81f4c95f6509b05bc752184c8463c0f5a
 Image 2 (ramdisk)
  Description:  unavailable
  Created:      Sun May 24 17:45:12 2026
  Type:         RAMDisk Image
  Compression:  uncompressed
  Data Size:    11465481 Bytes = 11196.76 KiB = 10.93 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01800000
  Entry Point:  unavailable
  Hash algo:    sha256
  Hash value:   850c2d05a4e76d02cad03794bff024050050f32264c057cd55b11a6a4d6477c8
 Image 3 (resource)
  Description:  unavailable
  Created:      Sun May 24 17:45:12 2026
  Type:         Multi-File Image
  Compression:  uncompressed
  Data Size:    82432 Bytes = 80.50 KiB = 0.08 MiB
  Hash algo:    sha256
  Hash value:   8a942d85bd0419be8344f51c19a7ba2fc1b52175408d1933f7308e2fd490e26b
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and ramdisk
  Kernel:       kernel
  Init Ramdisk: ramdisk
  FDT:          fdt
Running 40-recovery.sh - build_recovery succeeded.
linux@ubuntu:~/emb/15-rk3506$
```

固件编译后生成路径:rockdev/recovery.img

```
linux@ubuntu:~/rk3506$ ls -l rockdev/
总计 0
lrwxrwxrwx 1 linux linux 26  6月 14 17:13 boot.img -> ../../kernel-6.1/zboot.img
lrwxrwxrwx 1 linux linux 44  6月 14 17:09 MiniLoaderAll.bin -> ../../u-boot/rk3506_spl_loader_v1.06.111.bin
lrwxrwxrwx 1 linux linux 11  6月 12 15:48 misc.img -> ../misc.img
```

```
lrwxrwxrwx 1 linux linux 22  6月 12 17:31 oem.img -> ../extra-parts/oem.img
lrwxrwxrwx 1 linux linux 55  6月 12 17:31 parameter.txt -> ../../device/rockchip/.chips/rk3506/parameter-
flash.txt
lrwxrwxrwx 1 linux linux 23  6月 14 17:22 recovery.img -> ../recovery/ramboot.img
lrwxrwxrwx 1 linux linux 56  6月 14 17:20 rootfs.img ->
../../buildroot/output/rockchip_rk3506/images/rootfs.ubi
lrwxrwxrwx 1 linux linux 22  6月 14 17:09 uboot.img -> ../../u-boot/uboot.img
lrwxrwxrwx 1 linux linux 26  6月 12 17:31 update.img -> ../update/Image/update.img
lrwxrwxrwx 1 linux linux 27  6月 12 17:31 userdata.img -> ../extra-parts/userdata.img
linux@ubuntu:~/rk3506$
```

7. 固件打包

```
linux@ubuntu:~/rk3506$ ./build.sh updateimg
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-24_17-50-19
Using last kernel version(6.1)

。。。。。。

Making update image...
=========================================
          Start packing update image
=========================================
Generating package-file for update:
# NAME  PATH
package-file    package-file
parameter   parameter.txt
bootloader  MiniLoaderAll.bin
uboot   uboot.img
misc    misc.img
recovery    recovery.img
boot    boot.img
rootfs  rootfs.img
oem oem.img
userdata    userdata.img
Packing /home/linux/emb/15-rk3506/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
------ PACKAGE ------
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xd1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x1ec,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x449c0,userspace=0x8a
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x46800,size=0x400000,userspace=0x800,flash_address=0x00002000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x446800,size=0xc000,userspace=0x18,flash_address=0x00005000
Add file: ./recovery.img
recovery,Add file: ./recovery.img
done,offset=0x452800,size=0xf09000,userspace=0x1e12,flash_address=0x00005a00
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x135b800,size=0x405a00,userspace=0x80c,flash_address=0x0000ea00
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x1761800,size=0x6620000,userspace=0xcc40,flash_address=0x00013a00
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x7d81800,size=0x660000,userspace=0xcc0,flash_address=0x00063a00
Add file: ./userdata.img
userdata,Add file: ./userdata.img
done,offset=0x83e1800,size=0x260000,userspace=0x4c0,flash_address=0x0006ba00
Add CRC...
Make firmware OK!
------ OK ------
********rkImageMaker ver 2.29********
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running 90-updateimg.sh - build_updateimg succeeded.
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ ls -l rockdev/
总计 0
lrwxrwxrwx 1 linux linux 26  6月 14 17:13 boot.img -> ../../kernel-6.1/zboot.img
lrwxrwxrwx 1 linux linux 44  6月 14 17:09 MiniLoaderAll.bin -> ../../u-boot/rk3506_spl_loader_v1.06.111.bin
lrwxrwxrwx 1 linux linux 11  6月 12 15:48 misc.img -> ../misc.img
lrwxrwxrwx 1 linux linux 22  6月 12 17:31 oem.img -> ../extra-parts/oem.img
lrwxrwxrwx 1 linux linux 55  6月 12 17:31 parameter.txt -> ../../device/rockchip/.chips/rk3506/parameter-
flash.txt
lrwxrwxrwx 1 linux linux 23  6月 14 17:22 recovery.img -> ../recovery/ramboot.img
lrwxrwxrwx 1 linux linux 56  6月 14 17:20 rootfs.img ->
../../buildroot/output/rockchip_rk3506/images/rootfs.ubi
lrwxrwxrwx 1 linux linux 22  6月 14 17:09 uboot.img -> ../../u-boot/uboot.img
lrwxrwxrwx 1 linux linux 26  6月 14 17:24 update.img -> ../update/Image/update.img
lrwxrwxrwx 1 linux linux 27  6月 12 17:31 userdata.img -> ../extra-parts/userdata.img
linux@ubuntu:~/rk3506$
```

8. 分别烧录固件

增加自动复制脚本

执行命令：

```
linux@ubuntu:~/rk3506$ vi build.sh
linux@ubuntu:~/rk3506$
```

- build.sh 在文件的末尾追加如下内容

```
if [ -e rockdev ] ;then
    cp -v rockdev/* /mnt/hgfs/linuxshare/
fi
```

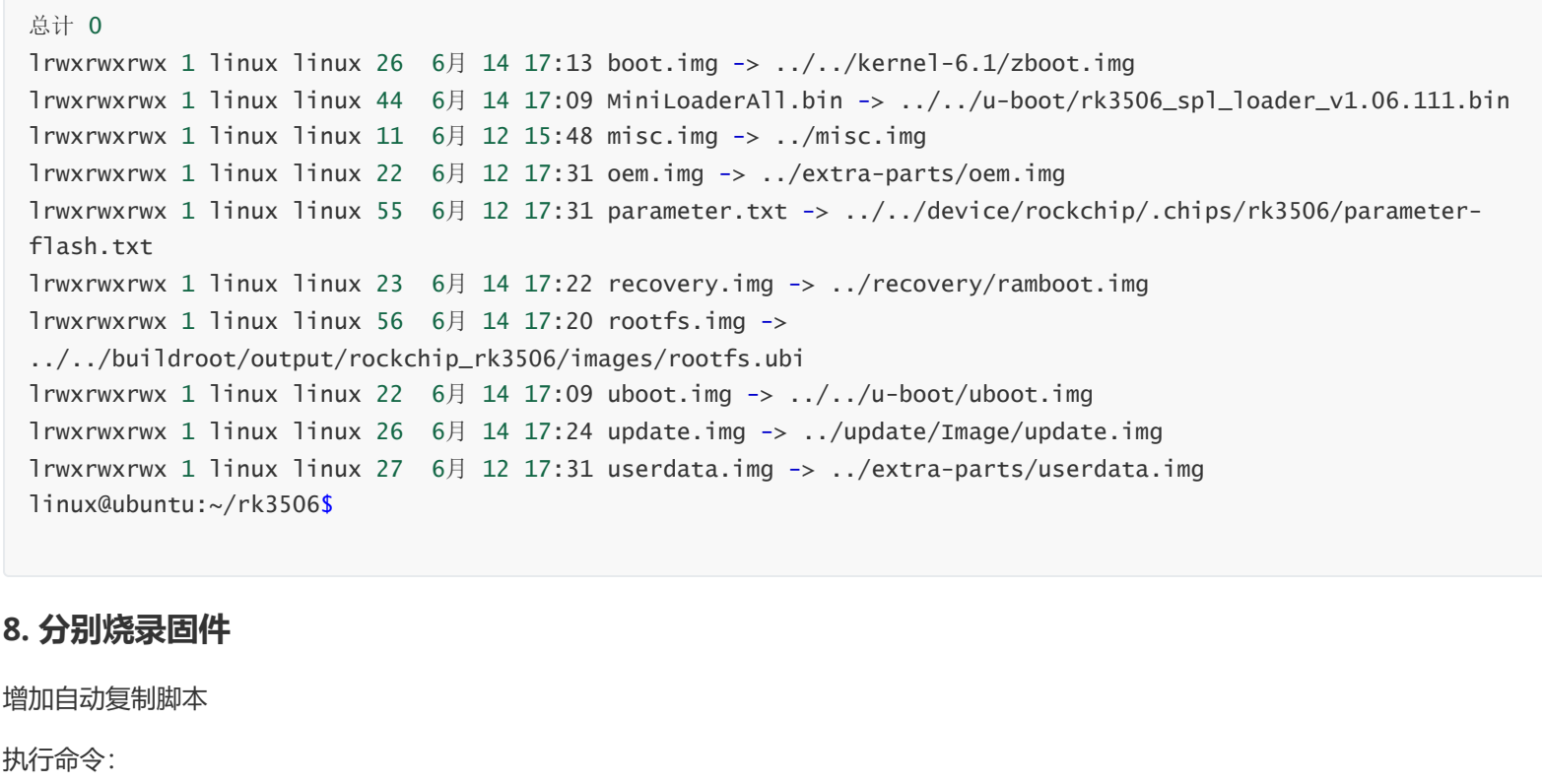
设置共享目录， 这个时候固件会自动复制到你指定的目录内。

打开瑞芯微开发工具

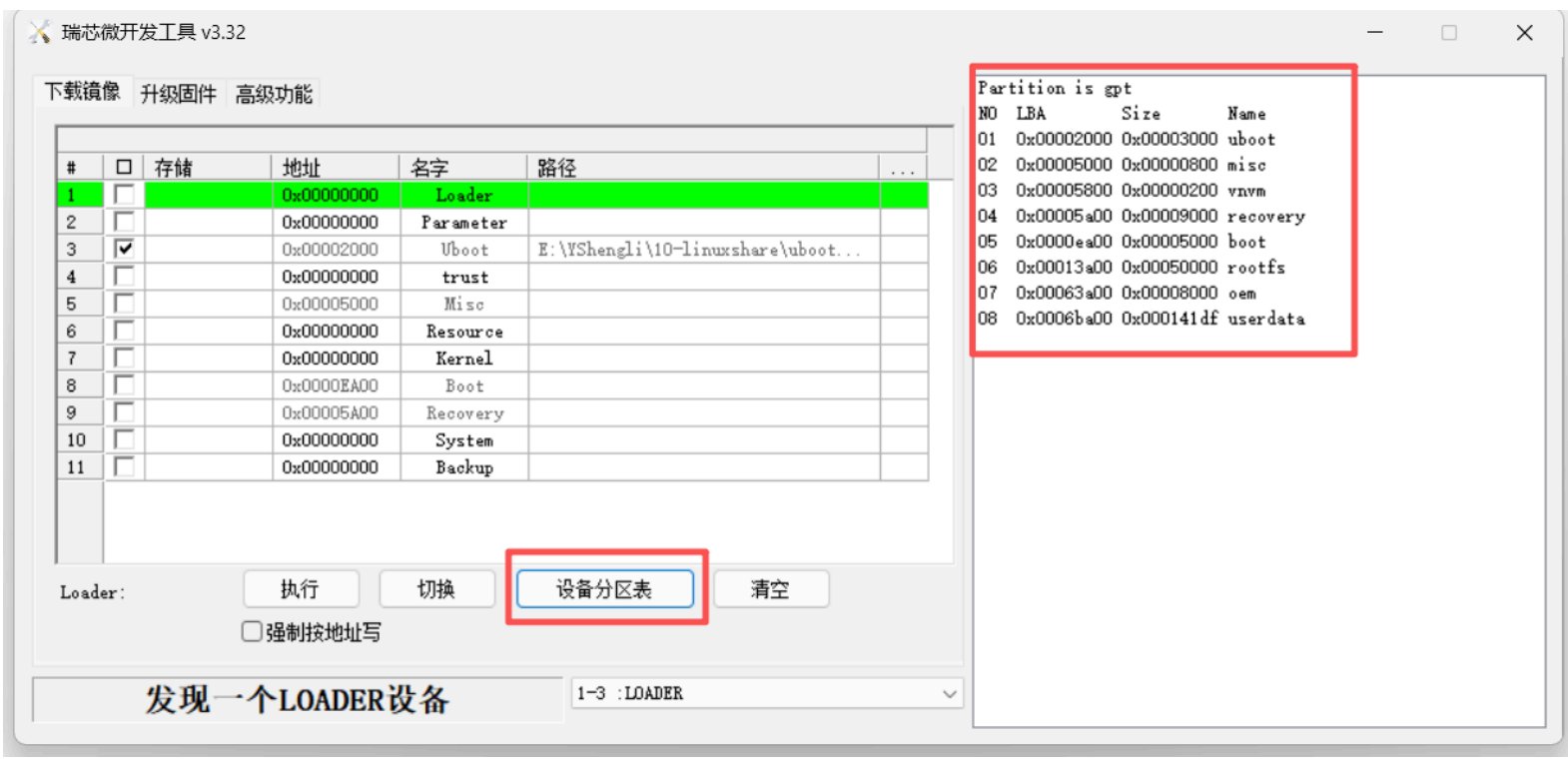
1.在对应的分区前选择对号

2.然后点击右侧去选择对应的镜像

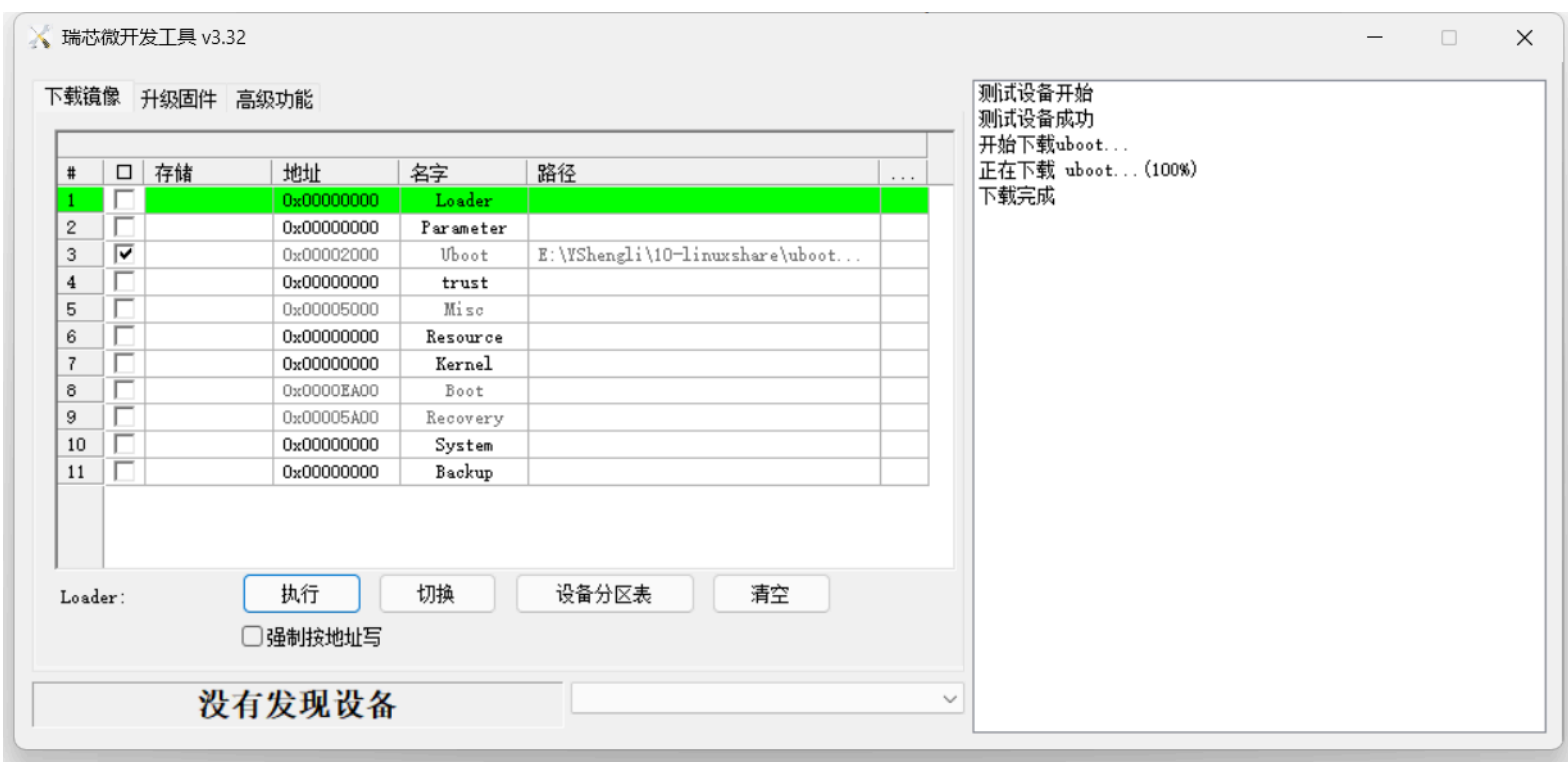
3.点击执行即可烧录



读取设备分区表



下载固件



第06章 Bootloader开发

6.1 Bootloader

1. 什么是Bootloader

- Bootloader是硬件启动的引导程序，是运行操作系统的前提；
- 在操作系统内核或用户应用程序运行之前运行的一小段代码。对软硬件进行相应的初始化和设定，为最终运行操作系统做准备；
- 在嵌入式系统中，整个系统的启动加载任务通常由Bootloader来完成。
- 等价于PC机的BIOS

2. Bootloader的特点

- Bootloader不属于操作系统，一般采用汇编语言和C语言开发。需要针对特定的硬件平台编写。
- 在移植系统时，首先为开发板移植Bootloader。
- Bootloader不但依赖于CPU的体系结构，而且依赖于嵌入式系统板级设备的配置。

3. Bootloader的操作模式

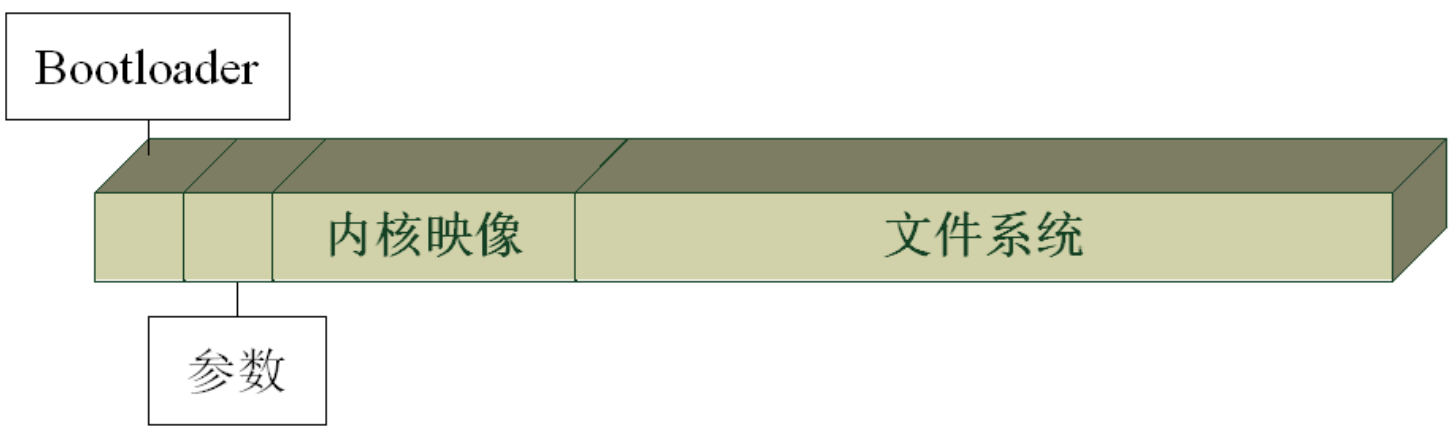
- 自启动模式：在这种模式下，Bootloader从目标机上的固态存储设备上将操作系统加载到内存中运行，整个过程并没有用户的介入。
- 交互模式：在这种模式下，目标机上的Bootloader将通过USB或网络等方式从开发主机（Host）上下载内核映像和根文件系统映像等到内存中。然后把固件写到目标机上的存储媒质中，或者直接进行系统的引导。也可以通过串口接收用户的命令。

4. Bootloader的基本功能

- 初始化相关硬件
- 把bootloader自搬运到内存中，并执行
- 执行用户的命令
  - 访问环境变量
  - 通过网络/串口通信
  - 读写RAM/FLASH（nand 、emmc、sd卡）
- 加载并执行内核

5. Bootloader的启动

- 系统加电或复位后，所有CPU都会从某个地址开始执行代码，嵌入式系统的开发板通常把板上ROM或FLASH映射到这个地址。
- 把Bootloader程序存储在相应的FLASH位置。系统加电后，CPU将首先执行它。



6. Bootloader启动的第一阶段（汇编阶段）

- 初始化基本的硬件；
- 把bootloader自搬运到内存中；
- 设置堆栈指针并将*bss*段清零。为后续执行C代码做准备；
- 跳转到第二阶段代码中；

7. Bootloader的第二阶段（c语言阶段）

- 初始化本阶段要使用到的硬件；
- 读取环境变量；
- 如果是**自启动模式**，从Flash或通过网络加载内核并执行；
- 如果是**交互模式**，接收到用户的命令后执行；

8. 常用bootloader介绍

Bootloader	Monitor	描述	X86	ARM	PowerPC
LILO	否	Linux磁盘引导程序	是	否	否
GRUB	否	GNU的LILO替代程序	是	否	否
Loading	否	从DOS引导linux	是	否	否
ROLO	否	从ROM引导linux而不需要BOIS	是	否	否
Etherboot	否	通过以太网卡启动linux系统的固件	是	否	否
LinuxBOIS	否	完全替代BUIS的linux引导程序	是	否	否
BLOB	是	LART等硬件平台的引导程序	否	是	否
U-boot	是	通用引导程序	是	是	是
RedBoot	是	基于eCos的引导程序	是	是	是
Vivi	是	Mizi公司针对于三星的arm cpu设计的引导程序			

U-Boot是ARM bootloader标准

- armboot 加入到 ppcboot 形成了 u-boot
- 支持arm920t, arm926, cortexa8,cortexa7
- http://www.denx.de/wiki/U-Boot/WebHome

Blob

- blob是LART工程使用的bootloader
- 移植到多个ARM平台上

Redboot

- RedBoot（也称为ROM Monitor）是一种用于嵌入式系统的开源引导加载程序（bootloader）

6.2 U-boot

1. U-boot介绍

u-boot(Universal Boot Loader)是德国DENX小组开发的用于多种嵌入式CPU的bootloader程序。遵循GPL条款。

从FADSROM、8xxROM 、PPCBOOT、Armboot逐步发展演化而来；

当前版本号：参考Makefile。

```
http://www.denx.de/wiki/U-Boot/WebHome
```

U-boot的特点

- 代码结构清晰、易于移植（见目录结构）
- 支持多种处理器体系结构（见arch目录）
- 支持众多开发板（目前官方包中有200多种，见board目录）
- 命令丰富、有监控功能
- 支持网络协议、USB、SD等多种协议和设备
- 支持文件系统
- 更新较活跃，使用者多，有助于解决问题

u-boot的下载地址

```
https://ftp.denx.de/pub/u-boot/
```

2. 瑞芯微U-Boot介绍

v2017(next-dev)是RK从U-Boot 官方的v2017.09正式版本中切出来进行开发的版本，目前已经支持RK所有主流在售芯片。支持的功能主要有:

- 支持RKAndroid固件启动;
- 支持Android AOSP 固件启动;
- 支持LinuxDistro固件启动;
- 支持 Rockchip miniloader 和 SPL/TPL两种 Pre-loader 引导;
- 支持LVDS、EDP、MIPI、HDMI、CVBS、RGB等显示设备;
- 支持eMMC、Nand Flash、SPI Nand flash、SPI NOR flash、SD卡、U盘等存储设备启动;
- 支持FAT、EXT2、EXT4文件系统;
- 支持GPT、RK parameter分区表;
- 支持开机LOGO、充电动画、低电管理、电源管理;
- 支持 I2C、 PMIC、 CHARGE、 FUEL GUAGE、 USB, GPIO, PWM, GMAC、 eMMC、 NAND,Interrupt 等;
- 支持Vendor storage保存用户的数据和配置;
- 支持 RockUSB 和Google Fastboot 两种 USB gadget烧写 eMMC;
- 支持Mass storage、ethernet、HID等USB设备;
- 支持通过硬件状态动态选择kernel DTB;

RK的U-Boot一共有两个版本，v2014旧版本和v2017新版本，内部名称分别为rkdevelop和next-dev。
用户有两个方式确认当前U-Boot是否为v2017版本。

```
linux@ubuntu:~/rk3506$ vi u-boot/Makefile

#
# SPDX-License-Identifier:  GPL-2.0+
#

VERSION = 2017
PATCHLEVEL = 09
SUBLEVEL =
EXTRAVERSION =
NAME =

linux@ubuntu:~/rk3506$
```

3. rkbin工具包

这是一个工具包仓库，用于存放RK不开源的bin、脚本、打包工具。U-Boot编译时会从该仓库索引相关文件，打包生成loader、trust、uboot固件。rkbin和U-Boot工程必须保持同级目录关系。

- **仓库内容**：主要包含三类东西：
  - **不开源的二进制文件**：比如 DDR 初始化固件、Arm Trusted Firmware（ATF/BL31）、OP-TEE（BL32）等。
  - **打包脚本与配置**：如 `ddrbin_tool.py`，以及各种 `.ini` 配置文件，告诉工具如何把这些二进制文件拼成完整的启动镜像。
  - **打包工具源码/可执行文件**：例如 `loaderimage`、`trust_merger` 等。
- **与 U-Boot 的协作流程**：U-Boot 编译时，并不会把 rkbin 里的二进制重新编译一遍，而是根据芯片型号，从 rkbin 中“选取”对应的文件，并用里面的工具和脚本打包生成最终的固件，通常包括：
  - **loader**（如 `rkxxx_loader_v1.xx.bin`）：包含 DDR 初始化、USB 下载等功能的 Loader 固件。
  - **trust**（如 `trust.img`）：包含 ATF、OP-TEE 等安全启动链的镜像。
  - **uboot**（如 `uboot.img`）：U-Boot 本身被打包成的可启动镜像。
- **严格的目录关系**：这也是你强调的重点——**rkbin 必须和 U-Boot 源码放在同一级目录下。**U-Boot 的 Makefile 和打包脚本会硬编码 `../rkbin` 这样的相对路径去引用里面的文件。如果目录结构不对，编译会直接报错。

```
linux@ubuntu:~/rk3506$ ls rkbin/bin/rk35/
rk3506b_ddr_750MHz_rt_v1.06.bin                 rk3562_ddr_1392MHz_v1.07.bin
rk3568_ramboot_null1.bin
rk3506b_ddr_750MHz_v1.06.bin                    rk3562_ddr_1560MHz_v1.07.bin
rk3568_ramboot_v1.08.bin
rk3506_ddr_750MHz_rt_v1.06.bin                  rk3562_ddr_920MHz_v1.07.bin
rk356x_spl_nand_v1.14.bin
rk3506_ddr_750MHz_v1.06.bin                     rk3562_mcu_v1.00.bin
rk356x_spl_v1.14.bin
rk3506_spl_v1.11.bin                            rk3562_spl_v1.07.bin
rk356x_usbplug_nand_v1.04.bin
rk3506_tee_ta_v1.10.bin                         rk3562_usbplug_v1.04.bin
rk356x_usbplug_v1.17.bin
rk3506_tee_ta_v2.40.bin                         rk3566_ddr_1056MHz_D3_LP3_eyescan_v1.23.bin
rk3576_bl31_v1.17.elf
rk3506_tee_v2.10.bin                            rk3566_ddr_1056MHz_D4_LP4_4x_eyescan_v1.23.bin
rk3576_bl32_v1.05.bin
rk3506_tee_v2.40.bin                            rk3566_ddr_1056MHz_ultra_v1.20.bin
rk3576_boost_v1.03.bin
rk3506_usbplug_v1.02.bin                        rk3566_ddr_1056MHz_v1.23.bin
rk3576_ddr_lp4_2112MHz_lp5_2736MHz_eyescan_v1.09.bin
rk3506_usbplug_v1.03.bin                        rk3566_ddr_528MHz_ultra_v1.10.bin
rk3576_ddr_lp4_2112MHz_lp5_2736MHz_ultra_v1.09.bin
rk3528_bl31_v1.19.elf                           rk3566_ddr_780MHz_ultra_v1.10.bin
rk3576_ddr_lp4_2112MHz_lp5_2736MHz_v1.09.bin
rk3528_bl32_v1.06.bin                           rk3566_ddr_920MHz_ultra_v1.10.bin
rk3576_spl_fspi1_v1.00.bin
rk3528_ddr_1056MHz_2L_PCB_eyescan_v1.10.bin     rk3566_ddr_920MHz_v1.23.bin
rk3576_spl_v1.07.bin
rk3528_ddr_1056MHz_2L_PCB_v1.11.bin             rk3568_bl31_cpu3_v1.01.elf
rk3576_usbplug_fspi1_v1.03.bin
rk3528_ddr_1056MHz_4BIT_PCB_eyescan_v1.10.bin   rk3568_bl31_l3_part_ecc_v1.00.elf
rk3576_usbplug_v1.04.bin
rk3528_ddr_1056MHz_4BIT_PCB_v1.11.bin           rk3568_bl31_rt_v1.03.elf
rk3583_ddr_lp4_1848MHz_lp5_2112MHz_v1.18.bin
rk3528_ddr_1056MHz_eyescan_v1.10.bin            rk3568_bl31_ultra_v2.17.elf
rk3588_bl31_v1.49.elf
rk3528_ddr_1056MHz_v1.11.bin                    rk3568_bl31_v1.45.elf
rk3588_bl32_v1.19.bin
rk3528_mcu_v1.00.bin                            rk3568_bl32_v2.15.bin
rk3588_ddr_lp4_2112MHz_lp5_2400MHz_v1.18.bin
rk3528_spl_v1.06.bin                            rk3568_ddr_1056MHz_D3_LP3_eyescan_v1.23.bin
rk3588_ddr_lp4_2112MHz_lp5_2736MHz_eyescan_v1.11.bin
rk3528_usbplug_v1.04.bin                        rk3568_ddr_1056MHz_v1.23.bin
rk3588_pcie_v2.20.bin
rk3562_bl31_v1.21.elf                           rk3568_ddr_1332MHz_v1.23.bin
rk3588_ramboot_null0.bin
rk3562_bl32_v1.07.bin                           rk3568_ddr_1560MHz_D4_LP4_4x_eyescan_v1.23.bin
rk3588_ramboot_null1.bin
rk3562_ddr_1056MHz_ultra_v1.06.bin              rk3568_ddr_1560MHz_v1.23.bin
rk3588_ramboot_v1.06.bin
rk3562_ddr_1056MHz_v1.07.bin                    rk3568_ddr_920MHz_v1.23.bin
rk3588_spl_v1.13.bin
rk3562_ddr_1332MHz_D3_LP3_eyescan_v1.07.bin     rk3568_miniloader_spinand_v1.15.bin
rk3588_usbplug_v1.11.bin
rk3562_ddr_1332MHz_D4_LP4_4x_eyescan_v1.07.bin  rk3568_pcie_v2.10.bin
rk3562_ddr_1332MHz_v1.07.bin                    rk3568_ramboot_null0.bin
linux@ubuntu:~/rk3506$

linux@ubuntu:~/rk3506$ ls rkbin/RKBOOT
PX30MINIALL.ini           RK3128MINIALL.ini                RK3528MINIALL.ini
RV1103BMINIALL_TB_EMMC.ini
PX30MINIALL_SLC.ini       RK3128MINIALL_SLC.ini            RK3562MINIALL_AMP_CPU3.ini
RV1103BMINIALL_UART0M2_SPI_NOR_SPI_NAND.ini
```

```
PX30MINIALL_WO_FTL.ini    RK3128XMINIALL.ini                RK3562MINIALL.ini
RV1103BMINIALL_UART0M2_SPI_NOR_TB.ini
PX3SEMINIALL.ini          RK3188MINIALL.ini                 RK3562MINIALL_ULTRA.ini
RV1106MINIALL_EMMC_TB.ini
PX3SEMINIALL_SLC.ini      RK322XATMINIALL.ini               RK3566MINIALL.ini
RV1106MINIALL_EMMC_TB_NOMCU.ini
PX5KERNEL4.4MINIALL.ini   RK322XHMINIALL.ini                RK3566MINIALL_NAND.ini
RV1106MINIALL.ini
PX5MINIALL.ini            RK322XMINIALL.ini                 RK3566MINIALL_ULTRA.ini
RV1106MINIALL_SPI_NAND_TB.ini
RK1806MINIALL.ini         RK3288.ini                        RK3568MINIALL.ini
RV1106MINIALL_SPI_NAND_TB_NOMCU.ini
RK1808MINIALL.ini         RK3288MINIALL.ini                 RK3568MINIALL_NAND.ini
RV1106MINIALL_SPI_NOR_TB_GC2093.ini
RK1808MINIALL_WO_FTL.ini  RK3308MINIALL.ini                 RK3568MINIALL_PCIE_EP.ini
RV1106MINIALL_SPI_NOR_TB.ini
RK302A.ini                RK3308MINIALL_SPI_NAND.ini        RK3568MINIALL_RAMBOOT.ini
RV1106MINIALL_SPI_NOR_TB_NOMCU.ini
RK302AMINIALL.ini         RK3308MINIALL_SPI_NAND_UART4.ini  RK3568MINIALL_SPI_NAND.ini
RV1106MINIALL_SPI_NOR_WAKEUP.ini
RK302AMINI.ini            RK3308MINIALL_SPI_NOR.ini         RK3576MINIALL_FSPI1.ini
RV1106MINIALL_TB_DUAL_CAMERA_SC230AI_SC230AI.ini
RK3032MINIALL.ini         RK3308MINIALL_SPI_NOR_UART4.ini   RK3576MINIALL.ini
RV1106MINIALL_TB_DUAL_CAMERA_SC301IOT_SC230AI.ini
RK3032MINIALL_SLC.ini     RK3308MINIALL_UART4.ini           RK3576MINIALL_IPC.ini
RV1106MINIALL_TB_DUAL_CAMERA_SC3338_SC3338.ini
RK3036_ECHOMINIALL.ini    RK3308MINIALL_WO_FTL.ini          RK3576MINIALL_ULTRA.ini
RV1106MINIALL_TB_GC3003.ini
RK3036.ini                RK3326AARCH32MINIALL.ini          RK3583MINIALL.ini
RV1106MINIALL_TB_SC200AI.ini
RK3036MINIALL.ini         RK3326AARCH32MINIALL_SLC.ini      RK3588MINIALL.ini
RV1106MINIALL_TB_SC230AI.ini
RK3036MINIALL_SLC.ini     RK3326MINIALL.ini                 RK3588MINIALL_IPC.ini
RV1106MINIALL_TB_SC3336_SC3338.ini
RK30B.ini                 RK3326MINIALL_SLC.ini             RK3588MINIALL_PCIE_EP.ini
RV1106MINIALL_TB_SC4336.ini
RK30BMINIALL.ini          RK3328MINIALL.ini                 RK3588MINIALL_RAMBOOT.ini
RV1106MINIALL_TB_SC450AI.ini
RK30BMINI.ini             RK3366MINIALL.ini                 RKCAYMAN.ini
RV110XMINIALL.ini
RK30.ini                  RK3368BOXMINIALL.ini              RKCROWN.ini
RV1126MINIALL_EMMC_TB.ini
RK30MINIALL.ini           RK3368HMINIALL.ini                RKNANO.ini
RV1126MINIALL_FTL.ini
RK30MINI.ini              RK3368.ini                        RKNPULIONMINIALL.ini
RV1126MINIALL.ini
RK310B.ini                RK3368MINIALL.ini                 RKPANDA.ini
RV1126MINIALL_IPC.ini
RK310BMINIALL.ini         RK3399MINIALL.ini                 RKSMART.ini
RV1126MINIALL_LP3_EMMC_TB.ini
RK310BMINI.ini            RK3399MINIALL_SPINOR.ini          RV1103BMINIALL_AUTO_MERGE.ini
RV1126MINIALL_LP4_EMMC_TB.ini
RK3126.ini                RK3399PROMINIALL.ini              RV1103BMINIALL_FASTBOOT_EMMC.ini
RV1126MINIALL_RAMBOOT.ini
RK3126MINIALL.ini         RK3506BMINIALL.ini                RV1103BMINIALL_FASTBOOT_SPI_NOR_SPI_NAND.ini
RV1126MINIALL_SPI_NOR_TB.ini
RK3126MINIALL_SLC.ini     RK3506BMINIALL_RT.ini             RV1103BMINIALL.ini
RV1126MINIALL_SPI_NOR_TINY.ini
RK3126MINIALL_WO_FTL.ini  RK3506MINIALL.ini                 RV1103BMINIALL_SPI_NAND_W25N01XX_TB.ini
SDBOOT.ini
RK3128.ini                RK3506MINIALL_RT.ini              RV1103BMINIALL_SPI_NOR_SPI_NAND_TB.ini
linux@ubuntu:~/rk3506$
```

系统启动后，会首先执行.

4. GCC 编译工具

GCC编译器使用gcc-arm-10.3，放置于prebuilts目录之内。prebuilts和U-Boot保持同级目录关系。如下：

```
linux@ubuntu:~/rk3506$ prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-gcc -v
使用内建 specs。
COLLECT_GCC=prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-gcc
COLLECT_LTO_WRAPPER=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/../libexec/gcc/arm-none-linux-gnueabihf/10.3.1/lto-wrapper
目标：arm-none-linux-gnueabihf
配置为：/data/jenkins/workspace/GNU-toolchain/arm-10/src/gcc/configure --target=arm-none-linux-gnueabihf --prefix= --with-sysroot=/arm-none-linux-gnueabihf/libc --with-build-sysroot=/data/jenkins/workspace/GNU-toolchain/arm-10/build-arm-none-linux-gnueabihf/install//arm-none-linux-gnueabihf/libc --with-bugurl=https://bugs.linaro.org/ --enable-gnu-indirect-function --enable-shared --disable-libssp --disable-libmudflap --enable-checking=release --enable-languages=c,c++,fortran --with-gmp=/data/jenkins/workspace/GNU-toolchain/arm-10/build-arm-none-linux-gnueabihf/host-tools --with-mpfr=/data/jenkins/workspace/GNU-toolchain/arm-10/build-arm-none-linux-gnueabihf/host-tools --with-mpc=/data/jenkins/workspace/GNU-toolchain/arm-10/build-arm-none-linux-gnueabihf/host-tools --with-isl=/data/jenkins/workspace/GNU-toolchain/arm-10/build-arm-none-linux-gnueabihf/host-tools --with-arch=armv7-a --with-fpu=neon --with-float=hard --with-mode=thumb --with-arch=armv7-a --with-pkgversion='GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)'
线程模型：posix
Supported LTO compression algorithms: zlib
gcc 版本 10.3.1 20210621 (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29))
linux@ubuntu:~/rk3506$
```

5. U-Boot中快捷键

RK平台提供串口组合键触发一些事件用于调试、烧写(如果无法触发，请多尝试几次;启用secure-boot时无效)。开机时长按:

- ctrl+c:进入U-Boot命令行模式;
- ctrl+d:进入loader烧写模式;
- ctrl+b:进入maskrom烧写模式;
- ctrl+f:进入fastboot模式;
- ctrl+m:打印 bidram/system 信息;
- ctrl+i:使能内核initcall_debug;
- ctrl+p:打印cmdline信息;
- ctrl+s:"Starting kernel..."之后进入U-Boot 命令行;

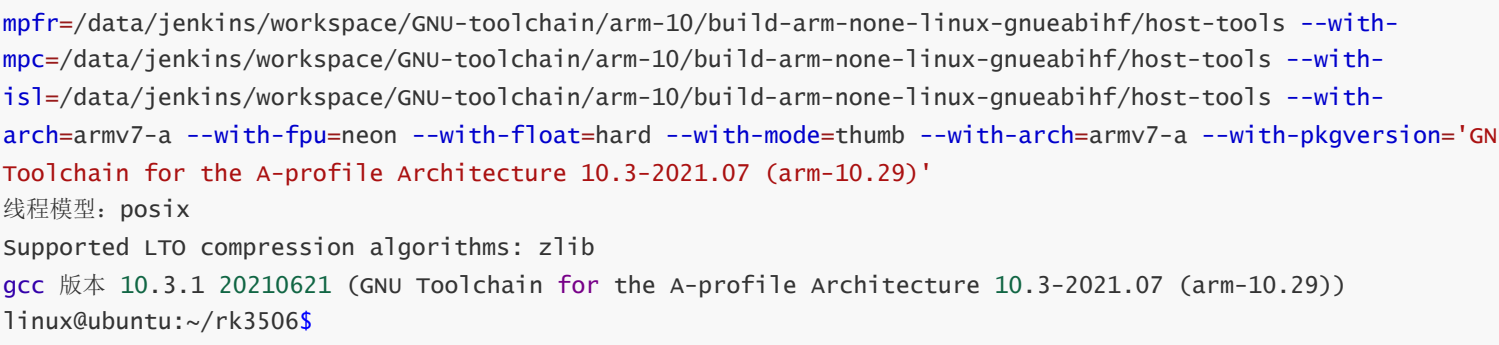
6.3 RK U-Boot启动流程

1. RK BootLoader 简介

RK Boot Loader 需完成：

- 芯片底层硬件初始化（时钟、DDR、电源、引脚复用）
- 多级安全校验与信任链构建
- 加载内核、设备树、initramfs 并传递启动参数
- 提供救援模式、固件升级、硬件调试能力

RK3506 启动链路：



说明： AP和MCU内部都有集成一个BOOTROM，系统上电时先会运行BOOTROM代码，然后BOOTROM代码会探测外设存储器并加载Loader代码。

Pre Loader 目前有3 个：miniloader (开源)，uboot spl 和 loader。

上电复位 → BootROM（BL1） → SPL/TPL（BL2） → 安全固件（tee.bin） → U-Boot（BL3） → Linux Kernel → Rootfs

2. 第一阶段 BootROM（BL1） 芯片出厂固化的灵魂

功能与特性

- 位置：芯片内部OTP/ROM（不可修改）
- 入口：CPU 复位向量0xFFFF0000
- 核心任务：
  - 最小系统初始化（PLL、时钟、UART、SRAM）
  - 启动介质检测（SD→eMMC→SPI→NAND→USB）
  - 加载SPL（idbloader.img）到内部 SRAM（128KB）
  - 安全启动：校验 SPL 签名，验签失败进入 Maskrom 模式

RK3506 启动介质偏移规则

启动介质	SPL（idbloader.img）偏移	说明
SD 卡 /eMMC	第**64 扇区**（32KB）	前 64 扇区保留分区表
SPI Flash	0x000000	直接从首地址存放
NAND Flash	Block 0	坏块管理起始

3. 第二阶段 SPL/TPL（BL2）

对应的文件：MiniLoaderAll.bin

实现功能：

- DDR 初始化与 U-Boot 加载器
- SPL（Secondary Program Loader）：加载 U-Boot、校验安全固件
- TPL（Triple Program Loader）：DDR 初始化专用（体积 < 32KB）
- idbloader.img：RK3506 镜像格式 = TPL + SPL（合并打包）

4. uboot编译文件解析

```
linux@ubuntu:~/rk3506$ ./build.sh  uboot
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-05-27_13-58-40
Using last kernel version(6.1)

......

make[2]: “arch/arm/dts/rk3506-evb-tb.dtb”已是最新。
  MKIMAGE u-boot.img
  COPY    u-boot.dtb
  MKIMAGE u-boot-dtb.img
  CAT     u-boot-dtb.bin
  COPY    u-boot.bin
  ALIGN   u-boot.bin
  COPY    spl/u-boot-spl.dtb
  CC      spl/common/spl/spl.o
  CC      spl/arch/arm/mach-rockchip/spl.o
  CC      spl/lib/display_options.o
  LD      spl/common/spl/built-in.o
  LD      spl/arch/arm/mach-rockchip/built-in.o
  LD      spl/lib/built-in.o
  LD      spl/u-boot-spl
  OBJCOPY spl/u-boot-spl-nodtb.bin
  CAT     spl/u-boot-spl-dtb.bin
  COPY    spl/u-boot-spl.bin
  CFGCHK  u-boot.cfg
pack u-boot.itb okay! Input: /home/linux/emb/15-rk3506/rkbin/RKTRUST/RK3506TOS.ini

.......

********boot_merger ver 1.35********
Info:Pack loader ok.
creating new idblock from loader...
idblock binary saving at rk3506_idblock_v1.06.111.img
pack loader(SPL) okay! Input: /home/linux/emb/15-rk3506/rkbin/RKBOOT/RK3506BMINIALL.ini

/home/linux/emb/15-rk3506/u-boot
pack loader with new: spl/u-boot-spl.bin
Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3506_idblock_v1.06.111.img (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/linux/emb/15-rk3506/rkbin/RKTRUST/RK3506TOS.ini
```

- rk3506_idblock_v1.06.111.img： 根据RK3506BMINIALL.ini 配置文件，生成新的 rk3506_idblock_v1.06.111.img 文件
- rk3506_spl_loader_v1.06.111.bin

```
# rk3506_idblock_v1.06.111.img 和rk3506_spl_loader_v1.06.111.bin 是同一个东西
linux@ubuntu:~/emb/15-rk3506$ ls -l rockdev/MiniLoaderAll.bin
lrwxrwxrwx 1 linux linux 44  5月 27 16:48 rockdev/MiniLoaderAll.bin -> ../../u-
boot/rk3506_spl_loader_v1.06.111.bin
linux@ubuntu:~/emb/15-rk3506$
# MiniLoaderAll.bin名字遵循了瑞芯微烧录工具的传统叫法。
# rk3506_idblock_v1.06.111.img / rk3506_spl_loader_v1.06.111.bin(MiniLoaderAll.bin) 是板子上电后执行的第一级软件
```

- uboot.img：`uboot.img` 是 **第二级启动引导器**

```
uboot.img  = u-boot.bin(U-Boot) +  tee.bin(OP-TEE) + u-boot.dtb(fdt)
BootROM (硬件固件)
   |
   └──> MiniLoaderAll.bin (rk3506_idblock_v1.06.111.img)
          ├── DDR 初始化固件
          └── SPL (u-boot-spl.bin)
                |
                └──> uboot.img (FIT Image)
                       ├── U-Boot (uboot)
                       ├── OP-TEE (optee)
                       └── Device Tree (fdt)
                             |
                             └──> boot.img (zImage + ramdisk)
                                    └──> 挂载 rootfs.img → 进入系统
```

5. 固件组成分析

```
Packed files:
MiniLoaderAll.bin(/home/linux/emb/15-rk3506/u-boot/rk3506_spl_loader_v1.06.111.bin): 275K
boot.img(/home/linux/emb/15-rk3506/kernel-6.1/zboot.img): 4.1M
misc.img(/home/linux/emb/15-rk3506/output/misc.img): 48K
oem.img(/home/linux/emb/15-rk3506/output/extra-parts/oem.img): 6.4M
parameter.txt(/home/linux/emb/15-rk3506/device/rockchip/.chips/rk3506/parameter-flash.txt): 480
recovery.img(/home/linux/emb/15-rk3506/output/recovery/ramboot.img): 16M
rootfs.img(/home/linux/emb/15-rk3506/buildroot/output/rockchip_rk3506/images/rootfs.ubi): 103M
uboot.img(/home/linux/emb/15-rk3506/u-boot/uboot.img): 4.0M
userdata.img(/home/linux/emb/15-rk3506/output/extra-parts/userdata.img): 2.4M
Making update image...
============================================
          Start packing update image
============================================
Generating package-file for update:
# NAME  PATH
package-file    package-file
parameter   parameter.txt
bootloader  MiniLoaderAll.bin
uboot   uboot.img
misc    misc.img
recovery    recovery.img
boot    boot.img
rootfs  rootfs.img
oem oem.img
userdata    userdata.img
Packing /home/linux/emb/15-rk3506/output/firmware/update.img for update...
Android Firmware Package Tool v2.29
------ PACKAGE ------
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xd1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x1ec,userspace=0x1,flash_address=0x00000000
Add file: ./MiniLoaderAll.bin
bootloader,Add file: ./MiniLoaderAll.bin done,offset=0x1800,size=0x449c0,userspace=0x8a
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0x46800,size=0x400000,userspace=0x800,flash_address=0x00002000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x446800,size=0xc000,userspace=0x18,flash_address=0x00005000
Add file: ./recovery.img
recovery,Add file: ./recovery.img
done,offset=0x452800,size=0xf09000,userspace=0x1e12,flash_address=0x00005a00
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x135b800,size=0x405a00,userspace=0x80c,flash_address=0x0000ea00
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x1761800,size=0x6620000,userspace=0xcc40,flash_address=0x00013a00
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x7d81800,size=0x660000,userspace=0xcc0,flash_address=0x00063a00
Add file: ./userdata.img
userdata,Add file: ./userdata.img
done,offset=0x83e1800,size=0x260000,userspace=0x4c0,flash_address=0x0006ba00
Add CRC...
Make firmware OK!
------ OK ------
********rkImageMaker ver 2.29********
Generating new image, please wait...
Writing head info...
```

```
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/linux/emb/15-rk3506/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/oem.img' -> '/mnt/hgfs/linuxshare/oem.img'
'rockdev/parameter.txt' -> '/mnt/hgfs/linuxshare/parameter.txt'
'rockdev/recovery.img' -> '/mnt/hgfs/linuxshare/recovery.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
'rockdev/update.img' -> '/mnt/hgfs/linuxshare/update.img'
'rockdev/userdata.img' -> '/mnt/hgfs/linuxshare/userdata.img'
linux@ubuntu:~/emb/15-rk3506$
```

- 最终产物列表及作用

文件名	性质	作用
MiniLoaderAll.bin	一级加载器	等同于 `idblock`（DDR 初始化 + SPL）的最终烧录形态，由 BootROM 直接加载
uboot.img	二级加载器 (FIT)	包含 U-Boot、OP-TEE、设备树，由 SPL 加载，最终引导内核
boot.img	内核启动镜像	包含 Linux 内核 (zImage) 与 ramdisk (cpio.gz)
recovery.img	恢复模式镜像	类似 boot.img，进入恢复系统
rootfs.img	根文件系统 (UBIFS)	只读的根文件系统，包含 Buildroot 用户态程序
oem.img	OEM 分区镜像	存放厂商定制数据，挂载到 `/oem`
userdata.img	用户数据分区	存放用户数据，挂载到 `/userdata`
misc.img	杂项分区	用于 Android 风格的 bootloader 通信（如命令、恢复标识）
parameter.txt	分区表	定义各分区的名称、大小、文件系统类型等
update.img	完整固件包	将所有上述镜像打包成一个 **统一固件文件**，供量产烧录使用

6.4 uboot启动过程分析

1. uboot整体过程分析

```
start.s // vi u-boot/arch/arm/cpu/armv7/start.S
        // 汇编环境
        => IRQ/FIQ/lowlevel/vbar/errata/cp15/gic // ARM架构相关的lowlevel初始化
        => _main
            => stack // 准备好C环境需要的栈
            // 【第一阶段】C环境初始化，发起一系列的函数调用
            => board_init_f: init_sequence_f[]
                initf_malloc
                arch_cpu_init           // 【SoC的lowlevel初始化】
                serial_init             // 串口初始化
                dram_init               // 【获取ddr容量信息】
                reserve_mmu             // 从ddr末尾开始往低地址reserve内存
                reserve_video
                reserve_uboot
                reserve_malloc
                reserve_global_data
                reserve_fdt
                reserve_stacks
                dram_init_banksize
                sysmem_init
                setup_reloc             // 确定U-Boot自身要reloc的地址
            // 汇编环境
            => relocate_code            // 汇编实现U-Boot代码的relocation
            // 【第二阶段】C环境初始化，发起一系列的函数调用
            => board_init_r: init_sequence_r[]
            initr_caches                // 使能MMU和I/Dcache
                initr_malloc
                bidram_initr
                sysmem_initr
                initr_of_live               // 初始化of_live
                initr_dm                    // 初始化dm框架
                board_init                  // 【平台初始化，最核心部分】
                    board_debug_uart_init   // 串口iomux、clk配置
                    init_kernel_dtb         // 【切到kernel dtb】!
                    clks_probe              // 初始化系统频率
                    regulators_enable_boot_on // 初始化系统电源
                    io_domain_init          // io-domain初始化
                    set_armclk_rate         // __weak，ARM提频(平台有需求才实现)
                    dvfs_init               // 宽温芯片的调频调压
                    rk_board_init           // __weak，由各个具体平台进行实现
                console_init_r
                board_late_init             // 【平台late初始化】
                    rockchip_set_ethaddr    // 设置mac地址
                    rockchip_set_serialno   // 设置serialno
                    setup_boot_mode         // 解析"reboot xxx"命令、
                                            // 识别按键和loader烧写模式、recovery
                    charge_display          // U-Boot充电
                    rockchip_show_logo      // 显示开机logo
                    soc_clk_dump            // 打印clk tree
                    rk_board_late_init      // __weak，由各个具体平台进行实现
                run_main_loop               // 【进入命令行模式，或执行启动命令】
```

2. 汇编启动入口（start.S）

在 `arch/arm/cpu/armv7/start.S`（以 ARMv7 为例）中：

```
.globl _start
_start:
    b   reset
    ...
reset:
    /* 初始化硬件 */
    bl  cpu_init_cp15
    bl  cpu_init_crit

    /* 设置栈指针 */
    ldr sp, =CONFIG_SYS_INIT_SP_ADDR


    /* 跳转到 C 代码 */
    bl  _main
```

3. _main 函数（汇编到 C 的桥梁）

在 `arch/arm/lib/crt0.S` 或 `arch/arm/lib/crt0_64.S` 中：

assembly

```
.globl _main
_main:
    /* 设置初始环境 */
    bl  board_init_f
    ...
    /* 重定位 U-Boot */
    bl  relocate_code
    ...
    /* 设置最终环境 */
    bl  board_init_r
```

4. board_init_r 跳转到 main_loop

在 `common/board_r.c` 中：

```
void board_init_r(gd_t *new_gd, ulong dest_addr)
{
    /* 初始化各个模块 */
    initcall_run_list(init_sequence_r);

    /* main_loop() 不会返回 */
    main_loop();
}
```

完整的调用链如下：

```
_start (start.S)
    ↓
reset (start.S)
    ↓
_main (crt0.S)
    ↓
board_init_f (C)
    ↓
relocate_code (汇编)
    ↓
board_init_r (C)
    ↓
main_loop (C)  ← 最终进入主循环
```

5. main_loop 函数分析

uboot的最终目的是引导内核，在此之前uboot需要完成一系列初始化操作，包括设置时钟、初始化DDR、Flash、串口、网卡等等，这时uboot有两条路走

- 交互模式：通过按键，触发uboot进入命令行模式，等待处理命令。
- 自启动模式：自动引导内核。

具体选择走哪个路，怎么走。main_loop将告诉我们。

uboot经过初始化后最终进入main_loop函数，并陷入死循环，不再返回

common/main.c

```c
/* We come here after U-Boot is initialised and ready to process commands */
void main_loop(void)
{
    const char *s;

    bootstage_mark_name(BOOTSTAGE_ID_MAIN_LOOP, "main_loop");

    if (IS_ENABLED(CONFIG_VERSION_VARIABLE))
        env_set("ver", version_string);  /* set version variable */

    cli_init();

    run_preboot_environment_command();

    if (IS_ENABLED(CONFIG_UPDATE_TFTP))
        update_tftp(0UL, NULL, NULL);

    s = bootdelay_process();    // 读取环境变量 bootdelay 的值
    if (cli_process_fdt(&s))
        cli_secure_boot_cmd(s);

    autoboot_command(s);   // s 是bootcmd的值,

    cli_loop();
    panic("No CLI available");
}
```

- uboot打印信息

```
U-Boot 2017.09 (Jun 14 2026 - 17:37:20 +0800)

Model: Rockchip RK3506 EVB Board
MPIDR: 0xf00
PreSerial: 0, raw, 0xff0a0000
DRAM:  256 MiB
Sysmem: init
Relocation Offset: 0ebee000
Relocation fdt: 0dbfadb0 - 0dbfede8
CR: M/C/I
Using default environment

Hotkey: ctrl+c
DM: v2
SPI Nand ID c8 92 c8
Bootdev(atags): mtd 1
PartType: EFI
boot mode: None
Failed to load DTB, ret=-19
No valid DTB, ret=-22
Failed to get kernel dtb, ret=-22
Model: Rockchip RK3506 EVB Board
rockchip_set_serialno: could not find efuse/otp device
CLK: (arm clk use pvtpll, rate = 1200M)
  gpll 1500000 KHz
  v0pll 1179648 KHz
  v1pll 903168 KHz
  clk_gpll_div 187500 KHz
  clk_gpll_div_100m 93750 KHz
  clk_v0pll_div 147456 KHz
  clk_v1pll_div 129024 KHz
  aclk_bus_root 187500 KHz
  hclk_bus_root 147456 KHz
  pclk_bus_root 93750 KHz
  aclk_hsperi_root 147456 KHz
  hclk_ksperi_root 93750 KHz
Net:   No ethernet found.
Hit key to stop autoboot('CTRL+C'):  0
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
```

6.5 RK3506快速启动方案

1. Thunderboot模式

默认的标准启动流程会执行U-Boot。不过，为了满足工业场景对启动速度的极致要求，RK3506也支持和推广一种可以“跳过”U-Boot的快速启动方案。

这两种方式的区别和特点，可以参考下表：

启动模式	标准启动流程 (Normal Mode)	快速启动方案 (Thunderboot)
启动流程	**BootROM → SPL → U-Boot → Kernel → Rootfs**	BootROM → SPL → **直接引导Kernel** → Rootfs
核心特点	功能完整，启动流程标准，为常规设备设计。	极致速度，为特定场景（如HMI、工业控制）优化，启动时间可达**2秒左右**。
适用场景	大多数通用嵌入式设备和默认开发板。	对启动时间有严苛要求的设备，如**家电显控、手持终端**等。

2. 启动流程解析

RK3506的完整启动过程分为几个阶段，上图的两个模式在最后一步有所不同：

1. **BootROM**：芯片上电后，首先执行固化在内部ROM里的代码，它会根据硬件配置，从不同的存储器（如eMMC、SPI Flash等）中查找并加载下一阶段的引导程序。
2. **SPL (Secondary Program Loader) / BL2**：这是BootROM加载的第一个程序，也是标准启动流程和快速启动方案的“分岔点”。
3. **U-Boot (标准模式)**：在标准启动流程中，SPL会继续加载完整的U-Boot。U-Boot功能强大，能初始化多种硬件，并提供交互式命令行，是开发者进行调试和控制的主要环境。
4. **直接引导Kernel (Thunderboot模式)**：在Thunderboot模式下，SPL会跳过完整的U-Boot，**直接将内核加载到内存并跳转执行**，从而省去了U-Boot的初始化和等待时间，实现快速启动。

3. 如何实现“极速启动”

如果想在自己的项目里实现这种快速启动，可以参考下面两种常见方式：

- **在U-Boot中开启Thunderboot**
  这是主流方案。在编译U-Boot时，通过配置文件开启 `CONFIG_THUNDER_BOOT` 宏。这样，SPL就会按照预设规则，从指定位置直接加载内核镜像并跳转。
- **SPL直接作为引导器**
  在一些对实时性要求极高的RTOS（实时操作系统）或裸机程序中，也可以直接让SPL加载应用程序，从而完全省去U-Boot这一环节。

第07章 Kernel开发

7.1 Linux内核介绍

1. 什么是linux内核

- 从技术上说 linux 是一个内核， 直观的理解就是（zImage和*.dtb）
- 内核指的是一个提供硬件抽象层、磁盘及文件系统控制、多任务等功能的系统软件。一个内核不是一套完整的操作系统。
- 通常我们使用的 linux 系统是一个集 linux 内核、工具集、各种库、桌面管理器、应用程序等一体的一个发布包 (发行版)

2. Linux内核的特性

- 可以移植，支持的硬件平台广泛
- 有MMU和没有MMU的处理器均支持
- 32 bit 处理器 (arch/ subdirectories)
    - arm, h8300, i386, m68k, m32r, mips, ppc
- 64 处理器:
    - ia64, mips64, ppc64, x86_64
- 高可扩展性
    - 可剪裁、可扩展，可以运行在大型主机，也可以运行在个人计算机上，同样可以运行在嵌入式系统
- 高可靠性、稳定性
    - 稳定性是linux鲜明特点，安装了linux系统的主机，连续运行一年不宕机是很平常的事情
- 超强的网络功能
- 真正的多任务，多用户系统
    - 多个用户可以同时登录到系统同时工作
- 耗费的资源相对较少
- 模块化设计
    - 模块可以动态加载，卸载，可以减少系统体积，同时可以用来解决冲突问题，模块调试
- 遵循GPL开源许可协议的，开放系统
- 编程更加简单，资源丰富

3. Linux内核版本

Linux 内核的版本号采用的是主版本号.次版本号.修订号的格式。

Linux 内核的版本号经常进行更新和发布，每个新版本都会引入新的功能、修复漏洞和改进性能。以下是一些常见的 Linux 内核版本：

- Linux 2.0.x：这是较早期的 Linux 内核版本，已经过时。
- Linux 2.2.x：这个版本引入了许多新的功能和改进，例如对 SMP（对称多处理）的支持。
- Linux 2.4.x：这是一个重要的稳定版本，引入了许多新的功能和改进，如对 USB 和 ACPI 的支持。
- Linux 2.6.x：这个版本在内部做了很大的重构，并引入了许多新的特性，如 CFS（完全公平调度器）和 KVM（内核虚拟机）。
- Linux 3.x.x：这个版本引入了一些新的特性和改进，如 Btrfs 文件系统和更好的电源管理支持。
- Linux 4.x.x：这个版本引入了更多的新功能和改进，如对 ARM64 架构的支持和更好的网络性能。
- Linux 5.x.x：这个版本引入了更多的新功能和改进，如对 RISC-V 架构的支持和即时预取优化。
- Linux 6.x.x：这个版本引入了更多的新功能和改进。

如果你想确定正在使用的 Linux 内核版本，可以在终端中使用命令 `uname -r` 或者查看 `/proc/version` 文件来获取。

4. Linux内核子系统

Linux 内核是一个复杂的软件系统，包含许多不同的子系统，每个子系统都负责管理不同的硬件资源、处理不同的系统功能或提供不同的服务。以下是一些常见的 Linux 内核子系统：

1. 进程管理子系统：这个子系统负责管理和调度进程并提供系统调用接口，包括进程调度器、进程通信机制、内存管理等。
2. 文件系统子系统：这个子系统负责管理文件系统、磁盘和文件，包括虚拟文件系统、磁盘缓存、文件系统驱动程序等。
3. 网络子系统：这个子系统负责管理网络协议栈和网络接口，包括 TCP/IP 协议栈、网络设备驱动程序、网络过滤和防火墙规则等。
4. 设备驱动程序子系统：这个子系统负责管理和控制硬件设备，包括设备驱动程序、设备树、设备文件和 I/O 系统等。
5. 电源管理子系统：这个子系统负责管理电源和节能功能，包括 ACPI（高级配置和电源管理接口）、 CPU 睡眠模式、设备断电等。
6. 调试子系统：这个子系统负责提供调试和性能分析工具，包括 kdump、systemtap 和 perf 工具等。

5. Linux内核代码结构

Linux 6.10 内核是一个非常复杂的软件系统，其代码结构包含了数百个不同的目录和文件，每个都承担着不同的作用。以下是 Linux 5.10 内核代码结构的一些主要部分：

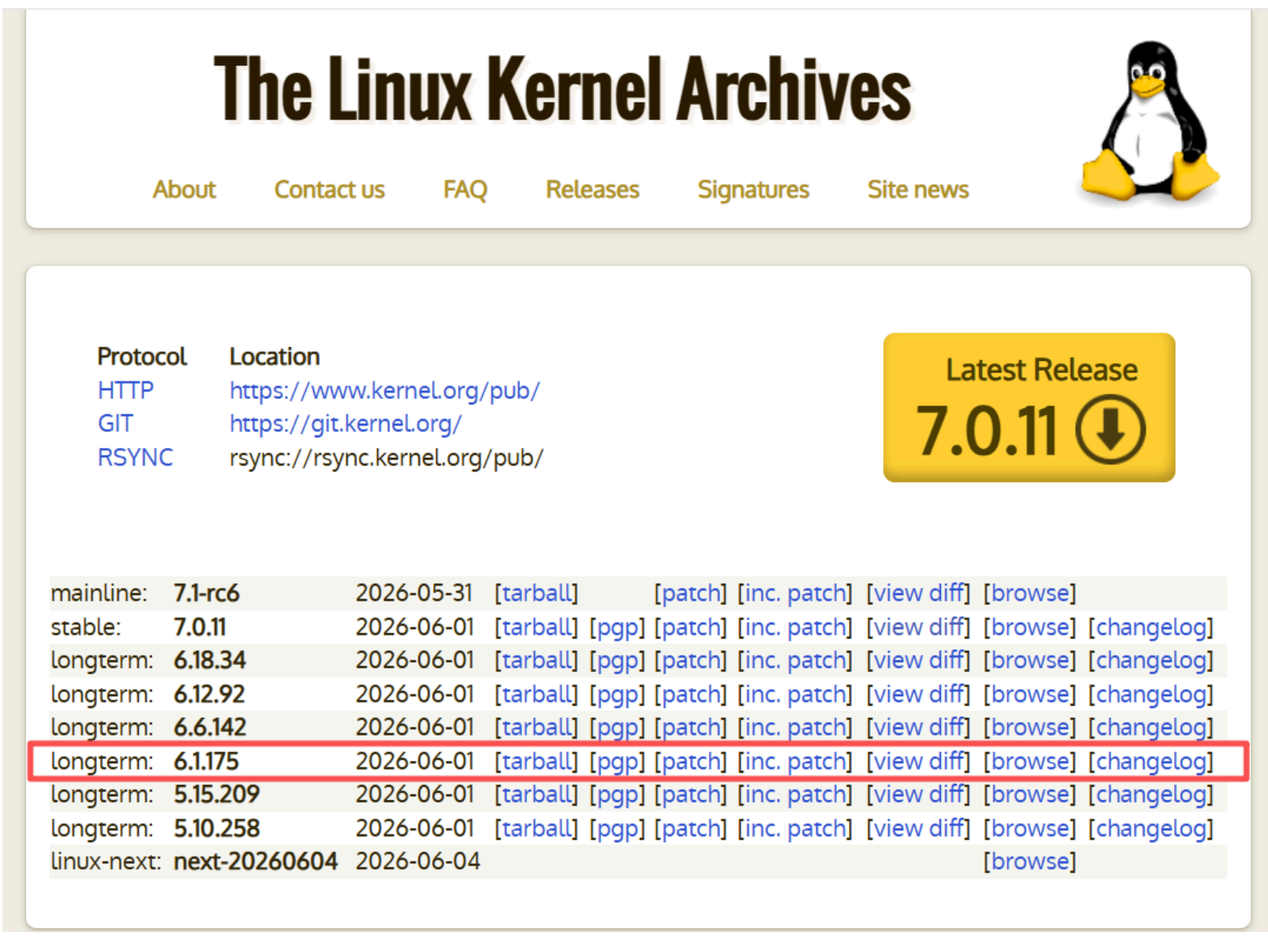
1. arch 目录：这个目录包含了针对不同硬件架构的代码，如 x86、ARM、MIPS 等。每个子目录都包含了与特定硬件架构相关的代码、头文件和 Makefile。
2. block 目录：这个目录包含了块设备驱动程序的代码，如硬盘、USB 存储和闪存驱动器等。每个子目录都包含了一个或多个具体设备的驱动程序。
3. crypto 目录：这个目录包含了加密和解密相关的代码，如 AES、SHA 和 RSA 等算法的实现。
4. drivers 目录：这个目录包含了大量的设备驱动程序代码，如网络设备驱动程序、声卡驱动程序、显卡驱动程序等。
5. fs 目录：这个目录包含了文件系统相关的代码，如 ext4、NTFS 和 FAT 文件系统的实现。
6. include 目录：这个目录包含了内核头文件，其中包括了所有的系统调用接口、数据结构定义和宏定义等。
7. kernel 目录：这个目录包含了内核的核心代码，如进程调度器、内存管理和系统调用实现等。
8. lib 目录：这个目录包含了一些常用的库函数代码，如字符串处理、数学运算和时间管理等。
9. mm 目录：这个目录包含了内存管理相关的代码，如虚拟内存管理、物理内存管理和页面换入换出等。
10. net 目录：这个目录包含了网络协议栈的代码，如 TCP/IP 协议栈、网络设备驱动程序和网络过滤规则等。

7.2 Linux内核配置与编译

1. 主线Linux内核

Linux6.10 内核下载地址

```
http://kernel.org
```



下载tarball文件压缩包

2. 瑞芯微定制Linux内核

瑞芯微RK3506的Linux 6.1内核是其新一代长期支持版本，延续了对双核Cortex-A7平台的深度优化。

- 该版本重点升级了显示与多媒体框架：DRM驱动全面支持Atomic模式，MIPI DSI/RGB显示接口兼容性更强，V4L2框架下的MIPI CSI摄像头驱动也获得增强。
- 内置的0.5 TOPS NPU通过更新版RKNPU驱动实现更高能效的AI推理。
- 基础外设驱动（GMAC、USB 2.0、SDIO、I2C等）保持完备，Device Tree结构进一步模块化，便于引脚复用配置。
- 协处理器Cortex-M0仍通过RPMsg进行核间通信，确保低功耗实时响应。
- 系统可配合Buildroot或Yocto构建精简镜像，广泛适配智能家居、工业HMI等物联网边缘计算场景。

3. 配置内核

- 进入图形化配置界面

```
#进入图形化配置界面
./build.sh kernel-config
```

例如：

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-22_14-48-01
Using last kernel version(6.1)

============================================
          Final configs
============================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubifs:defaults:rk3506_oem:auto:@auto:userdata:auto:ubifs:defaults:norma
l:auto:@@@
...
RK_YOCTO=y
RK_YOCTO_DISPLAY_PLATFORM=wayland
RK_YOCTO_MACHINE=rockchip-rk3506-evb
RK_YOCTO_USBMOUNT=y

Toolchain for kernel:
/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-
none-linux-gnueabihf-

============================================
          Start building kernel-config
============================================
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-
arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm rk3506_defconfig
rk3506-display.config
make: Entering directory '/home/linux/rk3506/kernel-6.1'
arch/arm/configs/rk3506_defconfig:374:warning: override: reassigning to symbol USB_NET_DRIVERS
arch/arm/configs/rk3506_defconfig:379:warning: override: reassigning to symbol NET_CORE
arch/arm/configs/rk3506_defconfig:391:warning: symbol value 'm' invalid for USB_CONFIGFS_F_PRINTER
arch/arm/configs/rk3506_defconfig:402:warning: override: reassigning to symbol VT
#
# configuration written to .config
#
Using .config as base
Merging ./arch/arm/configs/rk3506-display.config
Value of CONFIG_BACKLIGHT_CLASS_DEVICE is redefined by fragment ./arch/arm/configs/rk3506-display.config:
Previous value: # CONFIG_BACKLIGHT_CLASS_DEVICE is not set
New value: CONFIG_BACKLIGHT_CLASS_DEVICE=y

Value of CONFIG_DRM is redefined by fragment ./arch/arm/configs/rk3506-display.config:
Previous value: # CONFIG_DRM is not set
New value: CONFIG_DRM=y

Value of CONFIG_I2C_MUX is redefined by fragment ./arch/arm/configs/rk3506-display.config:
Previous value: # CONFIG_I2C_MUX is not set
New value: CONFIG_I2C_MUX=y

Value of CONFIG_INPUT_TOUCHSCREEN is redefined by fragment ./arch/arm/configs/rk3506-display.config:
Previous value: # CONFIG_INPUT_TOUCHSCREEN is not set
New value: CONFIG_INPUT_TOUCHSCREEN=y

Value of CONFIG_MEDIA_SUPPORT is redefined by fragment ./arch/arm/configs/rk3506-display.config:
Previous value: # CONFIG_MEDIA_SUPPORT is not set
New value: CONFIG_MEDIA_SUPPORT=y

#
# merged configuration written to .config (needs make)
#
#
# configuration written to .config
#
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-
arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm menuconfig
make: Entering directory '/home/linux/rk3506/kernel-6.1'
  UPD     scripts/kconfig/mconf-cfg
  HOSTCC  scripts/kconfig/mconf.o
  HOSTCC  scripts/kconfig/lxdialog/checklist.o
  HOSTCC  scripts/kconfig/lxdialog/inputbox.o
  HOSTCC  scripts/kconfig/lxdialog/menubox.o
  HOSTCC  scripts/kconfig/lxdialog/textbox.o
  HOSTCC  scripts/kconfig/lxdialog/util.o
  HOSTCC  scripts/kconfig/lxdialog/yesno.o
  HOSTLD  scripts/kconfig/mconf


*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-
arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm savedefconfig
make: Entering directory '/home/linux/rk3506/kernel-6.1'
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ mv kernel/defconfig kernel/arch/arm/configs/rk3506_defconfig
Running 10-kernel.sh - kernel-config succeeded.
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/oem.img' -> '/mnt/hgfs/linuxshare/oem.img'
'rockdev/parameter.txt' -> '/mnt/hgfs/linuxshare/parameter.txt'
'rockdev/recovery.img' -> '/mnt/hgfs/linuxshare/recovery.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
'rockdev/update.img' -> '/mnt/hgfs/linuxshare/update.img'
'rockdev/userdata.img' -> '/mnt/hgfs/linuxshare/userdata.img'
linux@ubuntu:~/rk3506$
```

4. 编译内核

```
./build.sh  kernel
```

单独编译 Linux 内核的脚本命令，它会完成从配置检查、源码编译到镜像打包的全流程。

例如：

```
linux@ubuntu:~/rk3506$ ./build.sh kernel
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-22_14-49-39
Using last kernel version(6.1)

============================================
          Final configs
============================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubifs:defaults:rk3506_oem:auto:@auto:userdata:auto:ubifs:defaults:norma
l:auto:@@@
RK_FLASH_SIZE=2048
RK_KERNEL=y
RK_KERNEL_ARCH=arm
RK_KERNEL_ARM32=y
RK_KERNEL_CFG=rk3506_defconfig
RK_KERNEL_CFG_FRAGMENTS=rk3506-display.config
RK_KERNEL_DTS=kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
RK_KERNEL_IMG=kernel/arch/arm/boot/zImage
RK_KERNEL_VERSION=6.1
RK_KERNEL_VERSION_RAW=6.1
RK_LOADER=y
RK_MISC=y
```

```
RK_MISC_BLANK=y
RK_NETWORK_CHECK=y
RK_OVERLAY=y
RK_OWNER=linux
RK_OWNER_UID=1000
RK_PARAMETER=parameter-flash.txt
RK_RECOVERY=y
RK_RECOVERY_CFG=rockchip_rk3506_recovery
RK_RECOVERY_CPIO_GZ=y
RK_RECOVERY_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot4recovery.its
RK_RECOVERY_INITRD_TYPE=cpio.gz
RK_ROOTFS=y
RK_ROOTFS_ASYNC_COMMIT=y
RK_ROOTFS_DEBUG_INFO=y
RK_ROOTFS_FSTRIM=y
RK_ROOTFS_FSTRIM_INTERVAL=3600
RK_ROOTFS_GENERATE_LOGS=y
RK_ROOTFS_INPUT_EVENT_DAEMON=y
RK_ROOTFS_INSTALL_MODULES=y
RK_ROOTFS_IRQBALANCE=y
RK_ROOTFS_LD_CACHE=y
RK_ROOTFS_OVERLAY=y
RK_ROOTFS_PREFER_PREBUILT_TOOLS=y
RK_ROOTFS_STRIP_MODULES=y
RK_ROOTFS_SYSTEM=buildroot
RK_ROOTFS_TYPE=ubi
RK_ROOTFS_UBI=y
RK_ROOTFS_UDEV_RULES=y
RK_SAVE_TRACKED=y
RK_SESSION=2026-06-22_14-49-39
RK_UBI=y
RK_UBI_BLOCK_SIZE=0x20000
RK_UBI_PAGE_SIZE=0x800
RK_UBOOT_ARCH=arm
RK_UBOOT_ARM32=y
RK_UBOOT_CFG=rk3506
RK_UBOOT_CFG_FRAGMENTS=rk3506b rk3506_tb
RK_UBOOT_SPL=y
RK_UPDATE=y
RK_USB_ADBD=y
RK_USB_ADBD_SHELL=/bin/bash
RK_USB_ADBD_TCP_PORT=5555
RK_USB_FW_VER=0x0310
RK_USB_GADGET=y
RK_USB_MANUFACTURER=Rockchip
RK_USB_PID=0
RK_USB_PRODUCT=rk3xxx
RK_USB_VID=0x2207
RK_USE_FIT_IMG=y
RK_WIFIBT=y
RK_WIFIBT_CHIP=AP6256
RK_WIFIBT_MODULES=AP6256
RK_WIFIBT_ON_BOARD=y
RK_YOCTO=y
RK_YOCTO_DISPLAY_PLATFORM=wayland
RK_YOCTO_MACHINE=rockchip-rk3506-evb
RK_YOCTO_USBMOUNT=y

Toolchain for kernel:
/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-

==========================================
          Start building kernel
==========================================
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm rk3506_defconfig rk3506-display.config
make: Entering directory '/home/linux/rk3506/kernel-6.1'
#
# No change to .config
#
Using .config as base
Merging ./arch/arm/configs/rk3506-display.config
#
# merged configuration written to .config (needs make)
#
#
# configuration written to .config
#
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm ido-evb3506-v1a-flash-mipi-720x720.img
make: Entering directory '/home/linux/rk3506/kernel-6.1'
  SYNC    include/config/auto.conf.cmd
  CALL    scripts/checksyscalls.sh
  Kernel: arch/arm/boot/Image is ready
  Kernel: arch/arm/boot/Image.gz is ready
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  CC      arch/arm/boot/compressed/fdt_rw.o
  CC      arch/arm/boot/compressed/fdt_ro.o
  CC      arch/arm/boot/compressed/fdt_wip.o
  CC      arch/arm/boot/compressed/fdt.o
  CC      arch/arm/boot/compressed/fdt_check_mem_start.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
  Image:  resource.img (with ido-evb3506-v1a-flash-mipi-720x720.dtb logo.bmp logo_kernel.bmp) is ready
  Image:  boot.img (with Image  resource.img) is ready
  Image:  zboot.img (with zImage  resource.img) is ready
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ /home/linux/rk3506/device/rockchip/common/scripts/mk-fitimage.sh kernel/zboot.img /home/linux/rk3506/device/rockchip/.chip/thunderboot.its kernel/arch/arm/boot/zImage kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm
Created:         Mon Jun 22 14:50:02 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Mon Jun 22 14:50:02 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44569 Bytes = 43.52 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   b75a4ab0958f483139b03b081242c439b9e4b8e0ea8338d58331f56337a1e290
 Image 1 (kernel)
  Description:  unavailable
  Created:      Mon Jun 22 14:50:02 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4169496 Bytes = 4071.77 KiB = 3.98 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   0df37779b875e5acd717e0c8c756e69b131862cd246f2d116ec6b2cea4153cef
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and no ramdisk
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/linux/rk3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
Running 10-kernel.sh - build_kernel succeeded.
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/oem.img' -> '/mnt/hgfs/linuxshare/oem.img'
'rockdev/parameter.txt' -> '/mnt/hgfs/linuxshare/parameter.txt'
'rockdev/recovery.img' -> '/mnt/hgfs/linuxshare/recovery.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
'rockdev/update.img' -> '/mnt/hgfs/linuxshare/update.img'
'rockdev/userdata.img' -> '/mnt/hgfs/linuxshare/userdata.img'
linux@ubuntu:~/rk3506$
```

最终产物位置

- **内核镜像、设备树**：`kernel/arch/arm/boot/` 下
- **可直接烧录的 boot.img**：`rockdev/Image/boot.img`（或 `rockdev/` 根目录）
- **模块**：若自动安装，通常在 rootfs 的 `lib/modules/` 下

```
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
  Image:  resource.img (with ido-evb3506-v1a-flash-mipi-720x720.dtb logo.bmp logo_kernel.bmp) is ready
  Image:  boot.img (with Image  resource.img) is ready
  Image:  zboot.img (with zImage  resource.img) is ready
make: Leaving directory '/home/linux/emb/15-rk3506/kernel-6.1'
+ /home/linux/emb/15-rk3506/device/rockchip/common/scripts/mk-fitimage.sh kernel/zboot.img
/home/linux/emb/15-rk3506/device/rockchip/.chip/thunderboot.its kernel/arch/arm/boot/zImage
kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm
Created:         Fri Jun  5 20:45:05 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Fri Jun  5 20:45:05 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44569 Bytes = 43.52 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   b75a4ab0958f483139b03b081242c439b9e4b8e0ea8338d58331f56337a1e290
 Image 1 (kernel)
  Description:  unavailable
  Created:      Fri Jun  5 20:45:05 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4169496 Bytes = 4071.77 KiB = 3.98 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   699c858f4c1456c5c029c07a26c66c726cb2ddfabec460502ec81654d06496be
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and no ramdisk
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/linux/emb/15-rk3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
Running 10-kernel.sh - build_kernel succeeded.
linux@ubuntu:~/emb/15-rk3506$
```

5. 清除内核

```
#清除内核配置文件（.config）及配置阶段生成的中间文件，但通常不会清除编译产物（如内核镜像、模块等）。
./build.sh clean-config                              cleanup config
#只清理中间文件：仅删除内核源码目录下的 *.o、*.ko .config 等编译中间件。
./build.sh clean-kernel                             cleanup kernel

# 手动进入内核目录，清空内核所有配置文件
cd kernel && make distclean && cd ..

# 手动进入内核目录，清空内核所有配置文件但是保留 .config 配置文件
cd kernel && make clean && cd ..

#清理整个SDK编译输出
#会清除 uboot、kernel、rootfs 等所有模块的产物
./build.sh cleanall                                  cleanup all
```

- 运行结果

```
linux@ubuntu:~/emb/15-rk3506$ ./build.sh clean-kernel
Log colors: message notice warning error fatal

Log saved at /home/linux/emb/15-rk3506/output/sessions/2026-06-05_21-10-52
Using last kernel version(6.1)
make: Entering directory '/home/linux/emb/15-rk3506/kernel-6.1'
  CLEAN   certs
  CLEAN   drivers/scsi
  CLEAN   drivers/tty/vt
  CLEAN   init
  CLEAN   lib
  CLEAN   net/wireless
  CLEAN   usr
  CLEAN   .
  CLEAN   modules.builtin modules.builtin.modinfo .vmlinux.export.c
  CLEAN   scripts/basic
  CLEAN   scripts/dtc
  CLEAN   scripts/gcc-plugins
  CLEAN   scripts/kconfig
  CLEAN   scripts/mod
  CLEAN   scripts
  CLEAN   include/config include/generated .config .config.old .version Module.symvers
make: Leaving directory '/home/linux/emb/15-rk3506/kernel-6.1'
Running build.sh - clean - kernel succeeded.
linux@ubuntu:~/emb/15-rk3506$
```

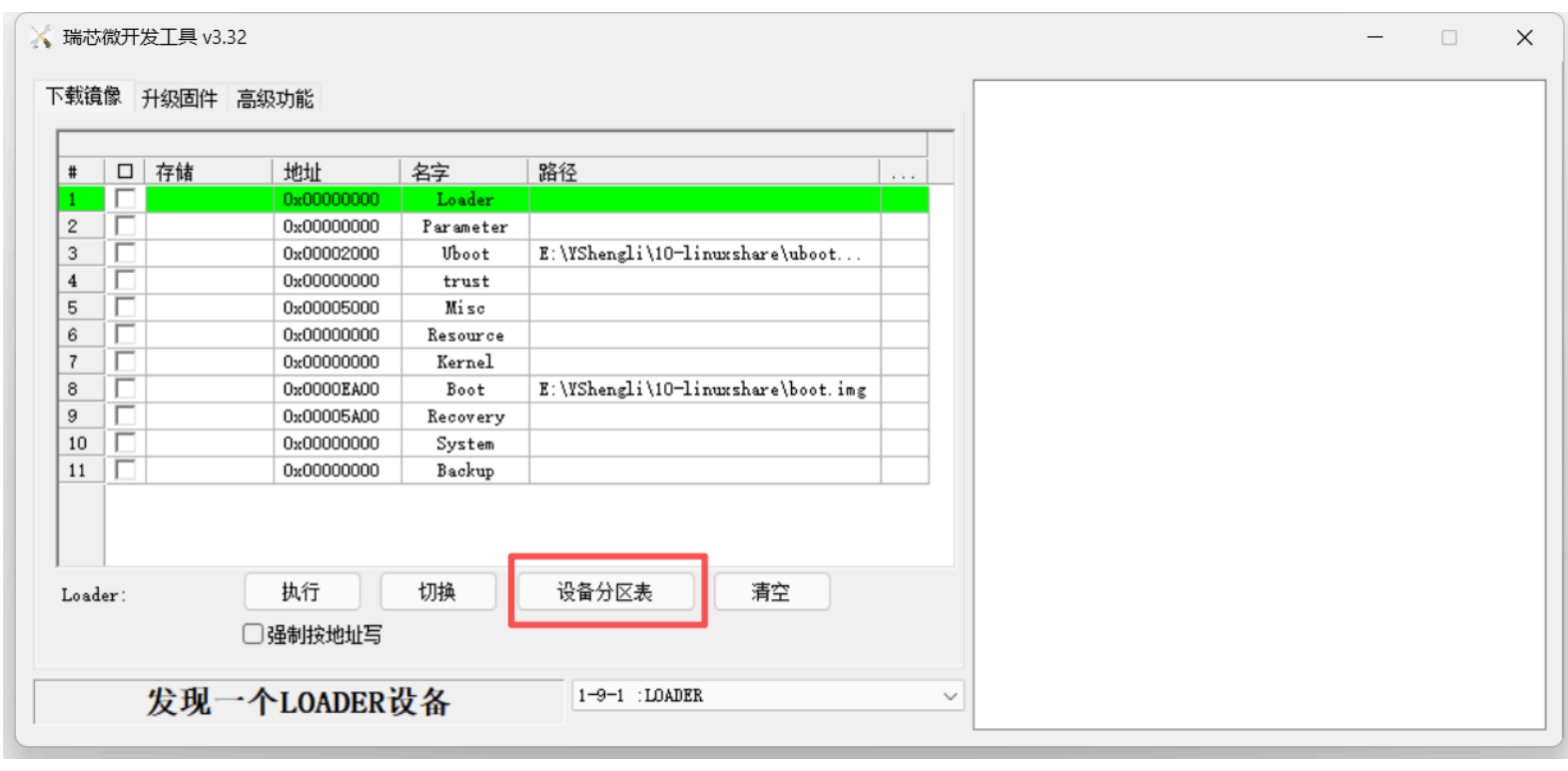
- build.sh 在文件的末尾追加如下内容

```
if [ -e rockdev ] ;then
    cp -v rockdev/* /mnt/hgfs/linuxshare/
fi
```

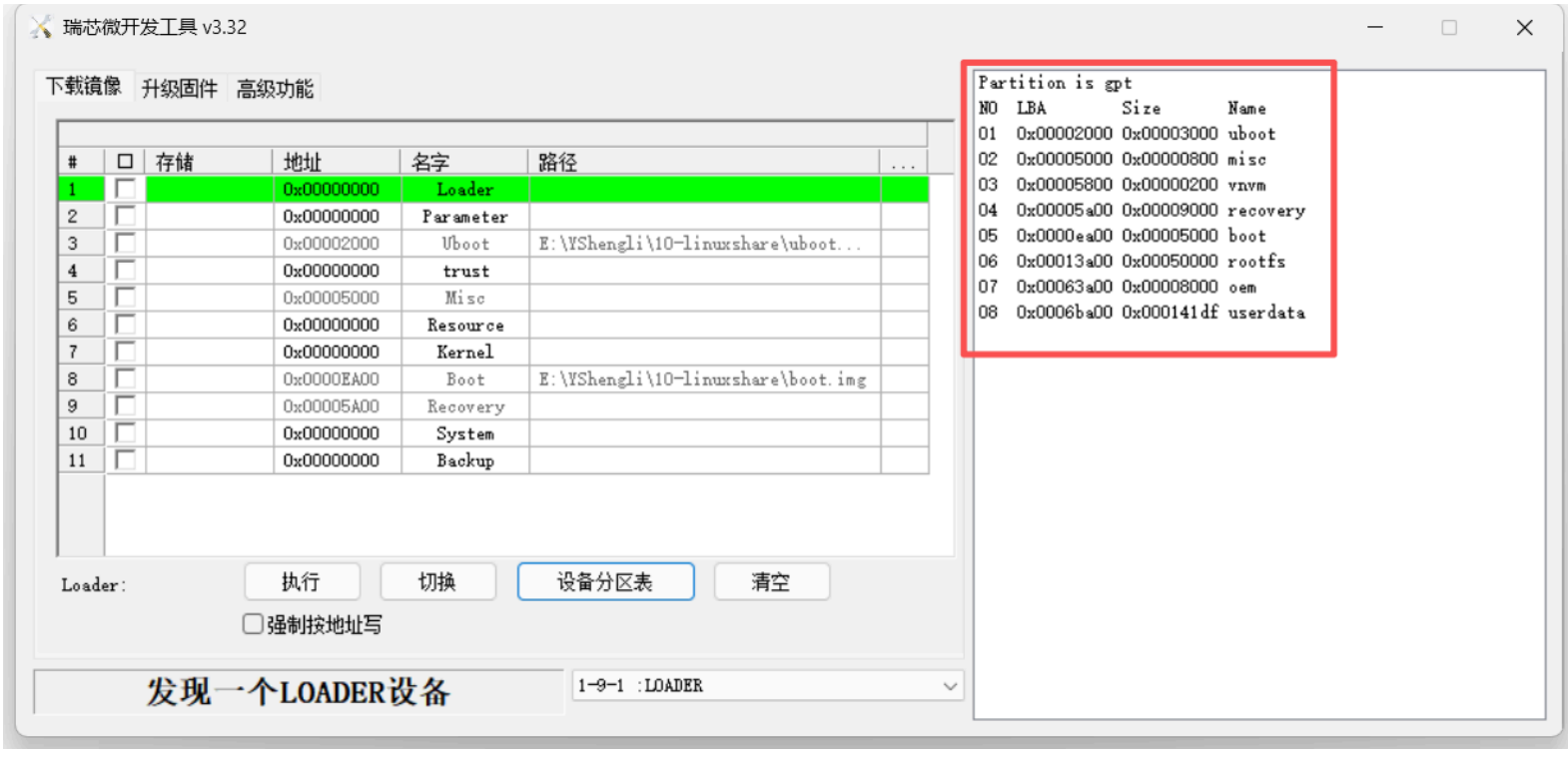
- 设置共享目录， 这个时候固件会自动复制到你制定的目录内。

7. 下载内核到开发内

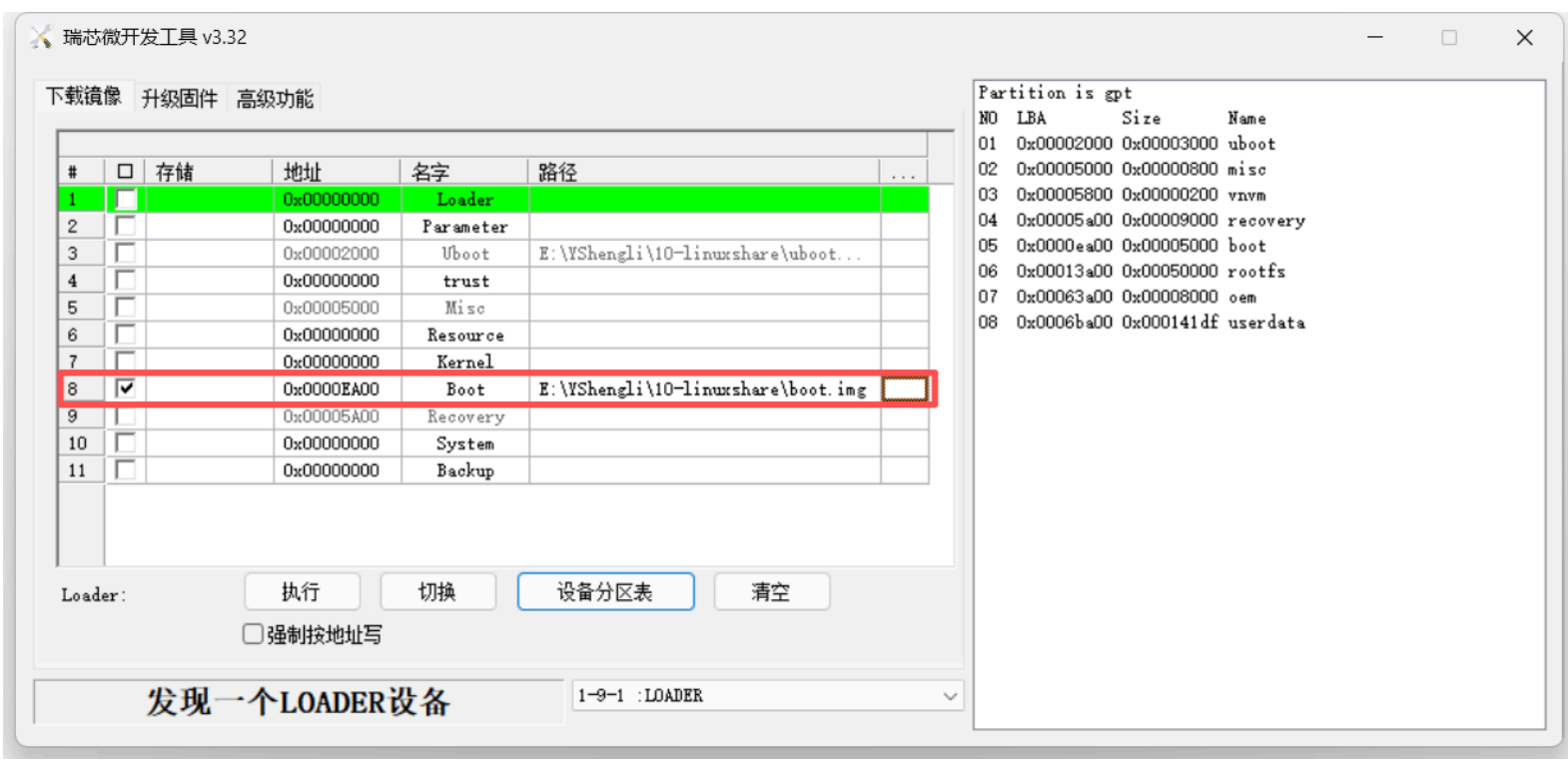
- 读取设备分区



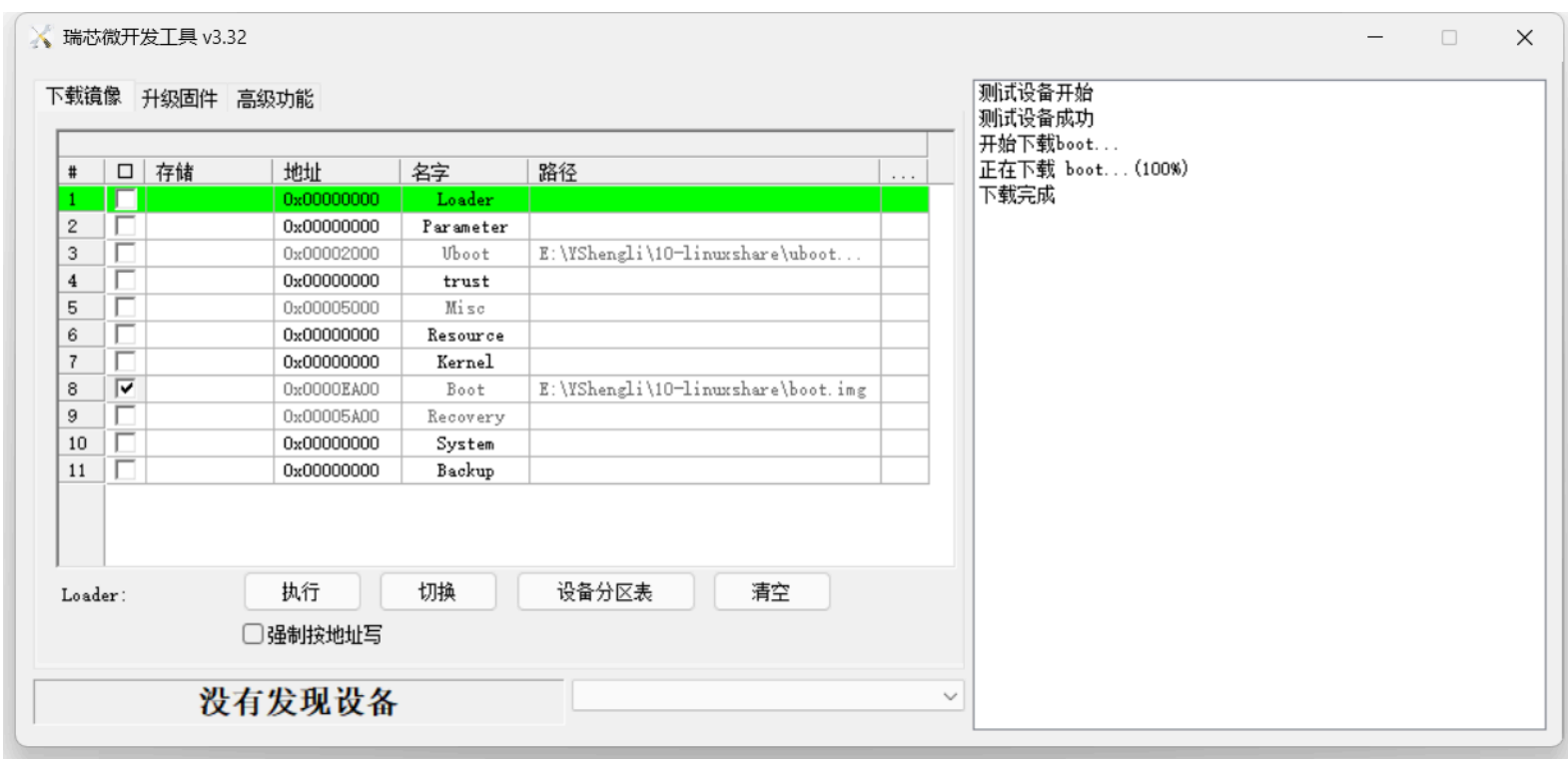
- 设备分区信息



- 选择要下载的内核



- 下载内核固件



7.3 linux内核启动流程

SDK 系统启动流

说明： AP和MCU内部都有集成一个BOOTROM，系统上电时先会运行BOOTROM代码，然后BOOTROM代码会探测外设存储器并加载Loader代码。

上电复位 → BootROM（BL1） → SPL/TPL（BL2） → 安全固件（tee.bin） → U-Boot（BL3） → Linux Kernel → Rootfs

---

1. 第1阶段：芯片内部 BootROM（一级引导BL1 ）

作用：芯片上电后，固化在内部 ROM 中的代码最先执行。

对应文件 ：芯片内部固件好的， 出厂带的。

关键动作：

- 初始化最基本的时钟源、电压与复位状态。
- 根据硬件启动引脚（Boot Strapping Pins）的电平组合，确定启动设备类型（如 SPI NOR Flash、eMMC、SD 卡等）。
- 从选定的存储设备固定偏移处读取二级程序加载器（通常为 U-Boot SPL），加载到内部 SRAM 并执行。

---

2. 第2阶段：U-Boot SPL（二级引导BL2）

作用：弥补 BootROM 的功能限制，完成内存初始化与下一级加载。

对应文件： `MiniLoaderAll.bin`

```
linux@ubuntu:~/rk3506$ ls rockdev/MiniLoaderAll.bin  -l
lrwxrwxrwx 1 linux linux 44  6月  6 15:56 rockdev/MiniLoaderAll.bin -> ../../u-
boot/rk3506_spl_loader_v1.06.111.bin
linux@ubuntu:~/rk3506$
```

关键动作：

- **初始化 DDR 控制器**：根据 PCB 设计配置时序，使外部 DDR 内存可用。
- **切换存储访问**：利用 DDR 初始化更高级的存储驱动（如 SPI Flash 的 Dual/Quad IO 模式），以获得更高读取速度。
- **加载 ATF (BL31)**：从存储设备中加载 ARM Trusted Firmware 的 BL31 镜像到 DDR 指定地址。
- **加载 U-Boot：将完整 U-Boot 镜像（uboot.img）加载到 DDR。
- **准备跳转**：设置 ATF 的入口参数（如 BL31 基址、U-Boot 基址等），然后跳转到 ATF BL31 入口。

---

3. 第3阶段：ATF BL31（三级引导）

作用：建立安全监控环境，提供 PSCI 等标准接口，管理安全与非安全世界切换。

对应文件： `tee.bin`

```
linux@ubuntu:~/rk3506$ ls u-boot/tee.bin  -l
-rw-rw-r-- 1 linux linux 124360  6月  6 15:56 u-boot/tee.bin
linux@ubuntu:~/rk3506$
```

关键动作：

- 初始化 ARM TrustZone 安全扩展，配置安全监控模式调用（SMC）处理程序。
- 注册电源管理操作（PSCI），为后续 CPU 热插拔、挂起/唤醒提供接口。
- 初始化 GIC (通用中断控制器) 的安全侧（若需要）。
- 完成安全环境设置后，切回非安全 EL2（Hyp 模式）或 EL1，将控制权交给 U-Boot。

---

4. 第4阶段：U-Boot（四级引导）

作用：初始化复杂外设、准备内核启动环境、执行启动脚本。

对应文件： `u-boot.bin`

```
linux@ubuntu:~/rk3506$ ls u-boot/u-boot.bin -l
-rw-rw-r-- 1 linux linux 686272  6月  6 15:56 u-boot/u-boot.bin
linux@ubuntu:~/rk3506$
```

关键动作：

- **重定位与初始化**：将自身代码从加载地址重定位到 DDR 高地址，设置堆栈，初始化控制台串口。
- **外设初始化**：根据需要驱动存储控制器（eMMC、SD 卡）、网络芯片（10/100M GMAC）、USB 等，以便加载内核与文件系统。
- **环境变量解析**：从存储设备读取 U-Boot 环境变量，确定启动命令（如 `bootcmd`）。
- **加载内核**：从启动分区（如 boot 分区或 ext4 分区）加载 Linux 6.1 内核镜像（zImage）和设备树 blob (DTB) 到 DDR 指定地址。
- **准备 FDT**：RK3506 使用扁平设备树，U-Boot 可修改或补充设备树节点（如 memory 节点、命令行参数等），最终将设备树地址

传递给内核。

- **跳转内核**：设置寄存器 `r2` 为设备树地址，然后调用 `kernel_entry`（zImage 入口），CPU 异常级别多为 EL1。

---

5. 第5阶段：Linux 6.1 内核启动

这是整个启动的核心，可分五个子阶段详述：

5.1 自解压与架构入口（arch/arm/boot/compressed/head.S -> arch/arm/kernel/head.S)

- **自解压阶段**：
  - zImage 压缩内核入口 `start` 首先验证设备树地址是否有效，设置临时页表，然后调用 `decompress_kernel` 解压内核到物理内存。
  - 解压后跳转到解压后的 `zreladdr`，即内核正式入口 `stext`（`arch/arm/kernel/head.S`）。
- **head.S 初始化**：
  - 检查处理器 ID、设备树合法性（`__lookup_processor_type`）。
  - 调用 `__create_page_tables` 建立初始页表，映射内核代码段和1MB的初始内存。
  - 使能 MMU 并跳转到 `__mmap_switched`，此处完成数据段复制、BSS 清空，然后调用 `start_kernel`（位于 `init/main.c`）。

5.2 平台早期初始化（setup_arch）

- **setup_arch()** 是 ARM 平台特定的初始化入口，它解析设备树中的以下关键信息：
  - `model` 和 `compatible` 确认平台为 “rockchip,rk3506”。
  - `memory` 节点获取物理内存大小和基址，初始化 meminfo。
  - 设置机器描述符 `machine_desc`，包含机器 ID、设备树根匹配等。
- **CPU 与调试**：初始化早期控制台（earlycon），解析 `chosen` 节点中的 `stdout-path` 使能早期打印（如 `earlycon=uart8250,mmio32,0xff130000`）。
- **中断初始化前置**：分配中断描述符表，初始化 IRQ 堆栈。

5.3 核心子系统初始化

按顺序在 `start_kernel()` 中调用：

- **setup_command_line()**：保存 bootloader 传递的命令行。
- **mm_init()**：初始化内存管理，包括伙伴系统、slab 分配器、高端内存映射等。
- **sched_init()**：初始化进程调度器。
- **irq_init()**：初始化中断框架，随后调用 `init_IRQ()`，通过设备树找到 GIC（ARM Generic Interrupt Controller）驱动（如 `gic_of_init`），注册中断控制器。RK3506 采用 GIC-400。
- **time_init()**：初始化系统定时器，通常使用 ARM 通用定时器（arch timer），通过设备树 `arm,armv7-timer` 节点配置，提供 `clocksource` 和 `clockevents`。
- **init_timers()** 和 **hrtimers_init()**：高精度定时器初始化。
- **console_init()**：正式注册串口控制台，替代 earlycon。
- **kthreadd、workqueue** 等内核线程创建。

5.4 设备驱动模型初始化与设备树展开

- **driver_init()**：创建 bus、class、platform 等基础结构。
- **do_initcalls()**：按顺序调用各级 initcall（**initcall0 到** initcall7），内核将调用所有内置模块和驱动。关键顺序：
  - 早期时钟和电源驱动通过 `CLK_OF_DECLARE` 和 `IOMMU_OF_DECLARE` 提前初始化，确保平台时钟树和电源域可用。
  - Rockchip 的时钟控制器驱动（`drivers/clk/rockchip/clk-rk3506.c`）注册固定时钟、PLL 和分频器。
  - 电源管理 IC (PMIC) 驱动通过 I2C 初始化，配置各路电压。
  - Pin controller (`pinctrl-rk3506.c`) 解析引脚复用，使能必要功能。
- **平台设备注册**：设备树中所有可寻址设备（如 UART、I2C、SPI）转换为 `platform_device`，绑定对应驱动。
- **协处理器启动**：RK3506 的特色是 Cortex-M0 协处理器。内核通过 `remoteproc` 框架加载 M0 固件，并利用 RPMsg 建立核间通信。相关驱动（`drivers/remoteproc/rk3506_remoteproc.c`）在 `initcall` 阶段加载，解析设备树中的 `m0_rproc` 节点，加载 ELF 固件并启动 M0。
- **存储与网络驱动**：eMMC/SD（`dw_mmc-rockchip`）、GMAC（`stmmac` 或 `dwmac-rk`）等也在这一阶段初始化，但块设备探测稍后。

5.5 挂载根文件系统与启动 init

- `prepare_namespace()` 或者直接通过 `mount_root()` 挂载根文件系统。根文件系统类型和设备由内核命令行 `root=` 指定（如 `root=/dev/mmcblk0p2`）或通过 initramfs。
- 若使用 initramfs（Buildroot 常内置），根文件系统已在内核镜像中，可直接执行 `/init`。
- 内核调用 `run_init_process()` 尝试运行一系列可能的 init 程序：`/sbin/init`、`/etc/init`、`/bin/init`。RK3506 SDK 的根文件系统（Buildroot/Yocto）通常提供 Busybox init 或 systemd。
- 进入用户空间后，`init` 进程依据 `/etc/inittab` 或 systemd 单元启动系统服务，如：
  - 挂载文件系统（/proc、/sys 等）
  - 启动网络服务
  - 启动 AI 推理相关服务（与 NPU 驱动交互）
  - 加载 M0 协处理器通信服务（rpmsg 字符设备等）

---

6. 第6阶段：多核启动（SMP）

- 内核启动主 CPU 后，通过 SMP 框架启动第二个 Cortex-A7 核。
- `smp_init()` 调用架构的 `smp_ops`（`arch/arm/mach-rockchip/platsmp.c`），通过 GIC 发送 SGI 唤醒从核，从核执行 `secondary_startup` 进入内核初始化，最终参与调度。

第08章 Rootfs开发

8.1 根文件系统目录结构

1. 什么是根文件系统

根文件系统（root filesystem）是存放运行、维护系统所必须的各种工具软件、库文件、脚本、配置文件和其他特殊文件的地方，也可以安装各种软件包。

2. 文件系统层次结构标准

- 文件系统层次结构标准 【Filesystem Hierarchy Standard (FHS)】
- Linux和UNIX的文件系统是一个以“/ “为根的阶层式的树状文件结构，/ 因此被称为根目录。
- 在早期的UNIX系统中，各个厂家各自定义了自己的UNIX系统的文件系统构成，比较混乱。为了避免在Linux也产生同样的问题，在Linux面世不久，就开始了对Linux文件系统进行标准化的活动。
- FHS标准定义了如下的内容。
  - 文件系统中每个区域的用途
  - 所需要的最小构成的文件和目录
- 提高软件的可移植性

3. 根文件系统主要目录结构

目录	内容
/bin	可执行程序
/dev	块、字符设备节点文件
/etc	主要配置文件和初始化执行文件
/lib	基本的库文件, 如：C 库,和内核模块（modules）
/mnt	挂载点，临时挂载文件系统用
/opt	附加的软件包
/proc	虚拟文件系统，用于内核和进程通讯
/sbin	基本的系统管理程序
/tmp	临时文件目录
/usr	更多的用户程序，包括Xserver
/sys	虚拟文件系统 sysfs 挂载点
/var	可变信息储存，如log等

8.2 构建根文件系统

1. BusyBox 介绍

BusyBox 是C语言写的一个开源程序。

BusyBox 是一个集成了大量标准 UNIX 工具的精简工具集，旨在为嵌入式系统和小型 Linux 环境提供命令行工具。

BusyBox 将数十个常用的 UNIX 工具整合到一个单一的可执行文件中，包括诸如 ls、cat、grep、mount 等常见命令，从而使得系统管理员可以在资源受限的环境中使用这些工具。

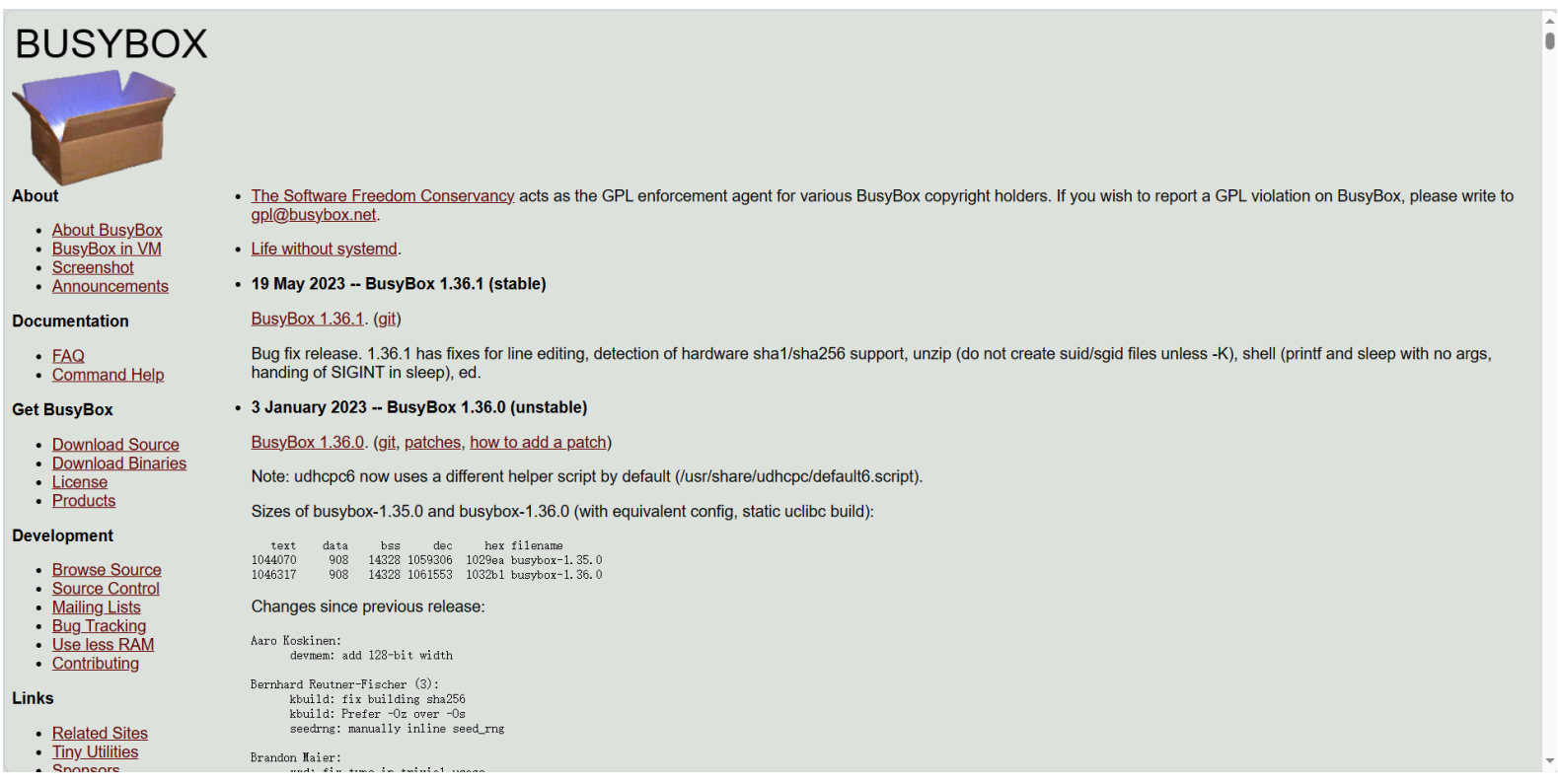
```
linux@ubuntu:~/buildroot/output/target/bin$ ls -l
总用量 732
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 arch -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 ash -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 base32 -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 base64 -> busybox
-rwxr-xr-x 1 linux linux 748516 12月 27 03:21 busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 cat -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 chattr -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 chgrp -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 chmod -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 chown -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 cp -> busybox
lrwxrwxrwx 1 linux linux      7 12月 27 03:04 cpio -> busybox
```

BusyBox 的主要特点包括：

1. 精简：BusyBox 的设计目标是尽可能地减小可执行文件的大小，使其能够在资源有限的系统中运行。因此，BusyBox 中的每个工具都被实现为一个单独的函数，可以根据需要进行编译和链接。
2. 集成：BusyBox 集成了大量常用的 UNIX 工具，使得用户无需安装大量单独的工具，就可以完成各种常见的任务。
3. 兼容性：BusyBox 的命令行工具大多与对应的标准 UNIX 工具具有相同的命令行接口和行为，因此用户可以在 BusyBox 中使用熟悉的命令和选项。
4. 自包含：BusyBox 在一个单独的可执行文件中包含了所有的命令行工具，这使得它非常适合于嵌入式系统和启动过程中的临时文件系统。

由于 BusyBox 的精简和高效特性，它经常被用于嵌入式 Linux 系统、路由器、手机等资源受限的环境中，以提供命令行工具的功能。另外，由于其开源性质，BusyBox 也被广泛应用于各种自定义的 Linux 发行版中。

BusyBox 可以在其官网下载到，官网地址为： https://busybox.net/，如图所示：



2. Buildroot 介绍

Buildroot 是一个用于嵌入式 Linux 系统构建的开源工具集。它能够自动化地构建整个嵌入式 Linux 系统的文件系统、内核镜像和 bootloader 等。

Buildroot 的目标是简化嵌入式系统的构建过程，提供一个灵活且易于定制的框架，以满足不同嵌入式设备的需求。

Buildroot 提供了一种从源代码构建整个 Linux 系统的方法，它能够根据用户的配置生成交叉编译工具链、根文件系统中的软件包、内核及其配置等。用户可以通过配置 Buildroot 来选择需要的软件包，并进行定制化的设置，以满足特定嵌入式设备的需求。

Buildroot 的主要特点包括：

1. 简化：Buildroot 提供了一个简单而直观的配置界面，使用户可以轻松选择和配置需要的软件包和功能。它自动处理软件包之间的依赖关系，并生成合适的构建顺序。
2. 自定义：Buildroot 允许用户对整个系统进行高度定制，包括选择交叉编译工具链、配置内核选项、添加和配置软件包等。用户可以根据嵌入式设备的需求进行灵活的定制。
3. 快速构建：Buildroot 可以高效地构建整个嵌入式 Linux 系统，通过并行构建和缓存机制，可以大大加快构建时间。
4. 支持广泛的架构和硬件平台：Buildroot 支持多种处理器架构和硬件平台，包括 ARM、x86、MIPS 等，并提供了对各种开发板和芯片组的支持。
5. 社区活跃：Buildroot 是一个活跃的开源项目，拥有庞大的用户社区和开发者团队。用户可以通过文档、邮件列表和 IRC 聊天室等途径获取支持和交流经验。

3. Yocto 介绍

1. **Yocto项目概述**
   - Yocto项目是一个开源的协作项目，它提供了一套灵活的工具和流程，用于创建定制的Linux操作系统。其主要目的是让开发者能够根据自己的需求，从源代码开始构建出高度定制化的嵌入式Linux系统。
   - 例如，在工业自动化设备、智能家居设备、汽车电子系统等众多嵌入式设备中，都可以利用Yocto项目来构建适合这些设备运行的操作系统。
2. **构建系统（Build System）**
   - Yocto项目的核心是它强大的构建系统。它使用一种叫做BitBake的任务调度器来构建软件包和整个系统镜像。BitBake会读取配置文件（通常是`.bb`和`.bbappend`文件），这些文件包含了软件包的构建指令、依赖关系和其他相关信息。
   - 例如，一个设备需要特定版本的Linux内核、特定的驱动程序和一些用户空间的应用程序。通过Yocto项目的构建系统，可以在配置文件中指定这些组件的版本、来源（如从源代码仓库获取还是从二进制仓库获取）以及它们之间的构建顺序和依赖关系。
3. **层（Layers）概念**
   - Yocto采用层的概念来组织构建过程。一个层基本上是一组相关的配方文件（recipes）和配置文件。通常有一个基础层（如`openembedded - core`），它提供了基本的构建工具和通用的软件包定义。
   - 然后，还有许多其他的层可以添加到构建系统中，用于支持特定的硬件平台（如针对某一款ARM处理器的硬件适配层）或者特定的软件功能（如添加一个多媒体处理软件栈的层）。这些层可以方便地进行组合和修改，以满足不同项目的需求。
4. **可定制性和灵活性**
   - Yocto项目允许开发者高度定制操作系统。可以选择不同的Linux内核版本，裁剪不必要的软件组件，添加自己开发的软件包等。例如，对于一个资源受限的物联网传感器设备，开发者可以通过Yocto构建一个只包含必要的网络协议栈、传感器驱动和简单数据采集应用程序的小型Linux系统，从而最大程度地减少系统资源的占用。
5. **社区和支持**
   - 作为一个开源项目，Yocto拥有一个活跃的社区。开发者可以在社区中分享经验、寻求帮助、提交补丁和改进建议。社区也会定期发布更新和新的功能特性，以适应不断变化的嵌入式系统开发需求。同时，许多芯片制造商和硬件平台供应商也会提供与Yocto项目兼容的硬件开发板支持和文档，以帮助开发者更好地将Yocto构建的操作系统应用到他们的产品中。

4. BusyBox 和 Buildroot对比

Yocto学习曲线陡峭，不建议初学者学习。

BusyBox 和 Buildroot 都是用于嵌入式 Linux 系统开发的工具，但它们的作用和功能略有不同。

BusyBox 是一个精简的工具集，旨在为嵌入式系统和小型 Linux 环境提供命令行工具。它集成了大量常用的 UNIX 工具，使得系统管理员可以在资源受限的环境中使用这些工具。BusyBox 的主要作用是提供命令行工具的功能，以及一些基本的系统管理工具，如文件系统操作、网络配置等。

Buildroot 则是一个用于嵌入式 Linux 系统构建的工具集，它能够自动化地构建整个嵌入式 Linux 系统的文件系统、内核镜像和 bootloader 等。Buildroot 支持从源代码构建整个 Linux 系统，包括交叉编译工具链、根文件系统中的软件包、内核及其配置等。通过配置 Buildroot，用户可以定制化地构建适合特定嵌入式设备的 Linux 系统。

因此，BusyBox 更专注于提供命令行工具的功能，而 Buildroot 则更专注于整个嵌入式 Linux 系统的构建。在实际应用中，它们通常会一起使用，例如在 Buildroot 构建的嵌入式系统中，可以使用 BusyBox 来提供命令行工具的支持。

buildroot 同样集成了busybox，可以理解为busybox的升级版。

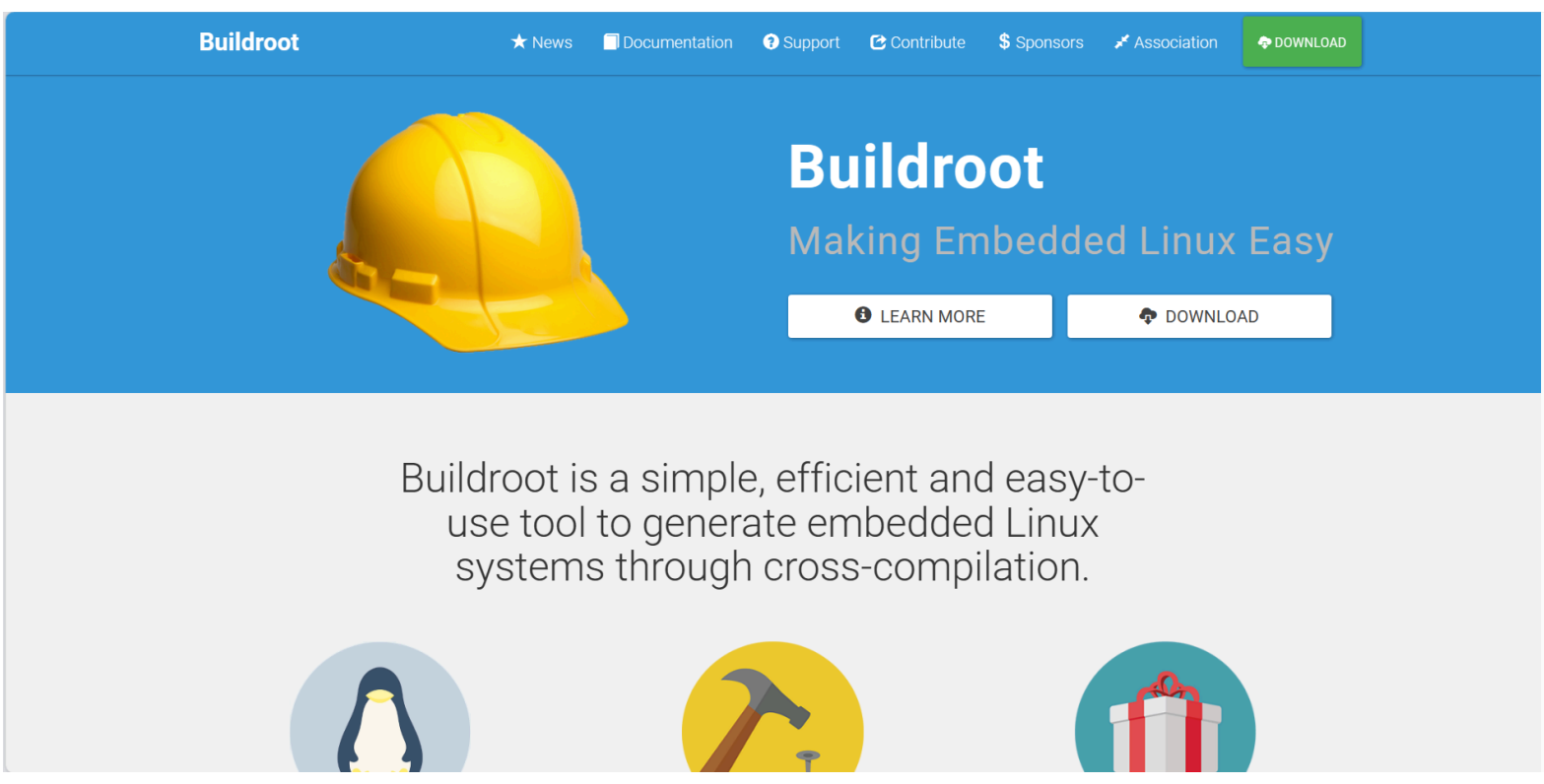
8.3 buildroot 制作根文件系统

1. buildroot 介绍

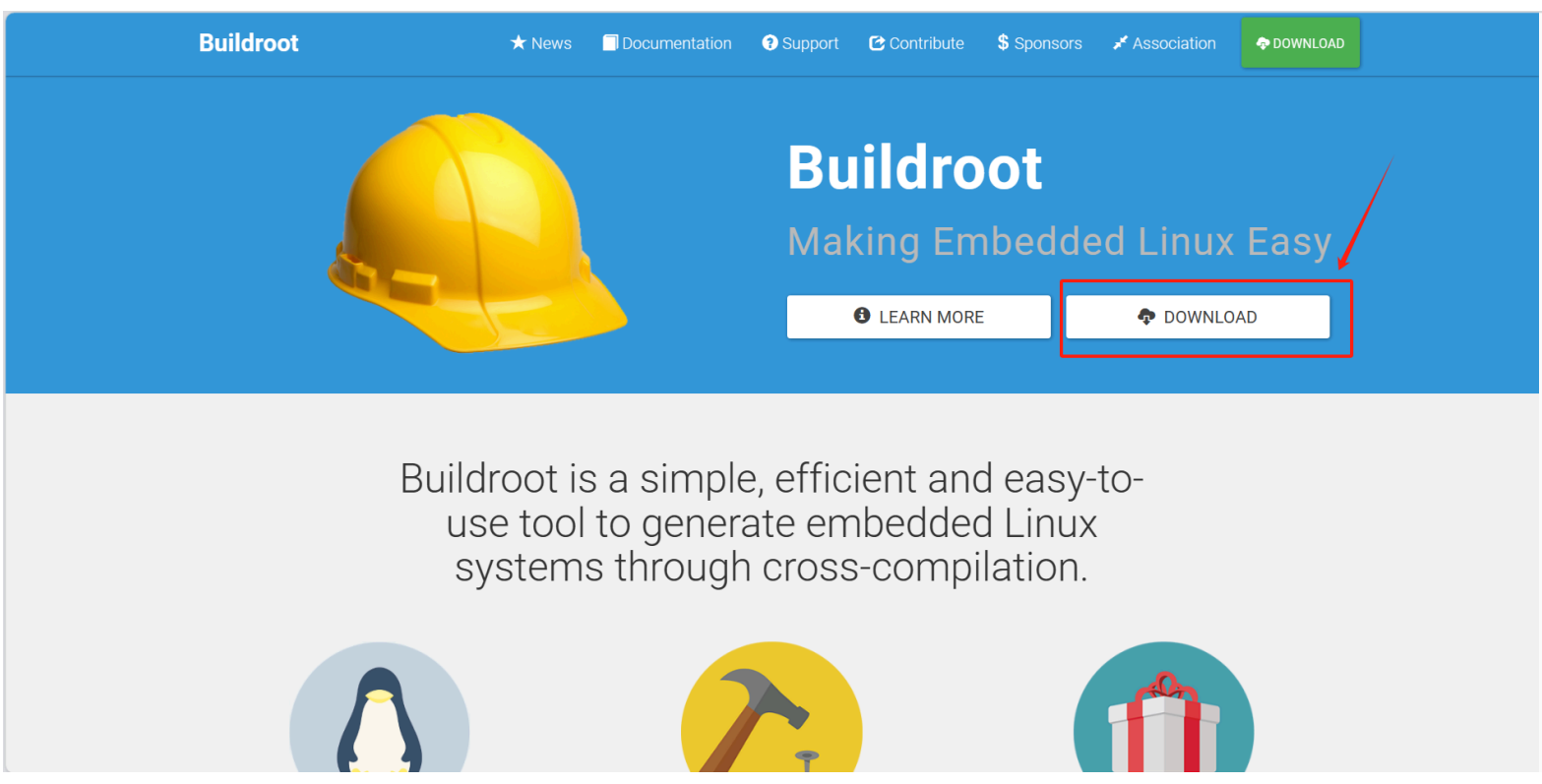
Buildroot 是 Linux 平台上一个构建嵌入式 Linux 系统的框架。整个 Buildroot 是由 Makefile 脚本和 Kconfig配置文件构成的。你可以和编译 Linux 内核一样，通过 buildroot 配置，menuconfig 修改，编译出一个完整的可以直接烧写到机器上运行的 Linux 系统软件(包含 boot、kernel、rootfs 以及 rootfs 中的各种库和应用程序)。

2. 获取 Buildroot

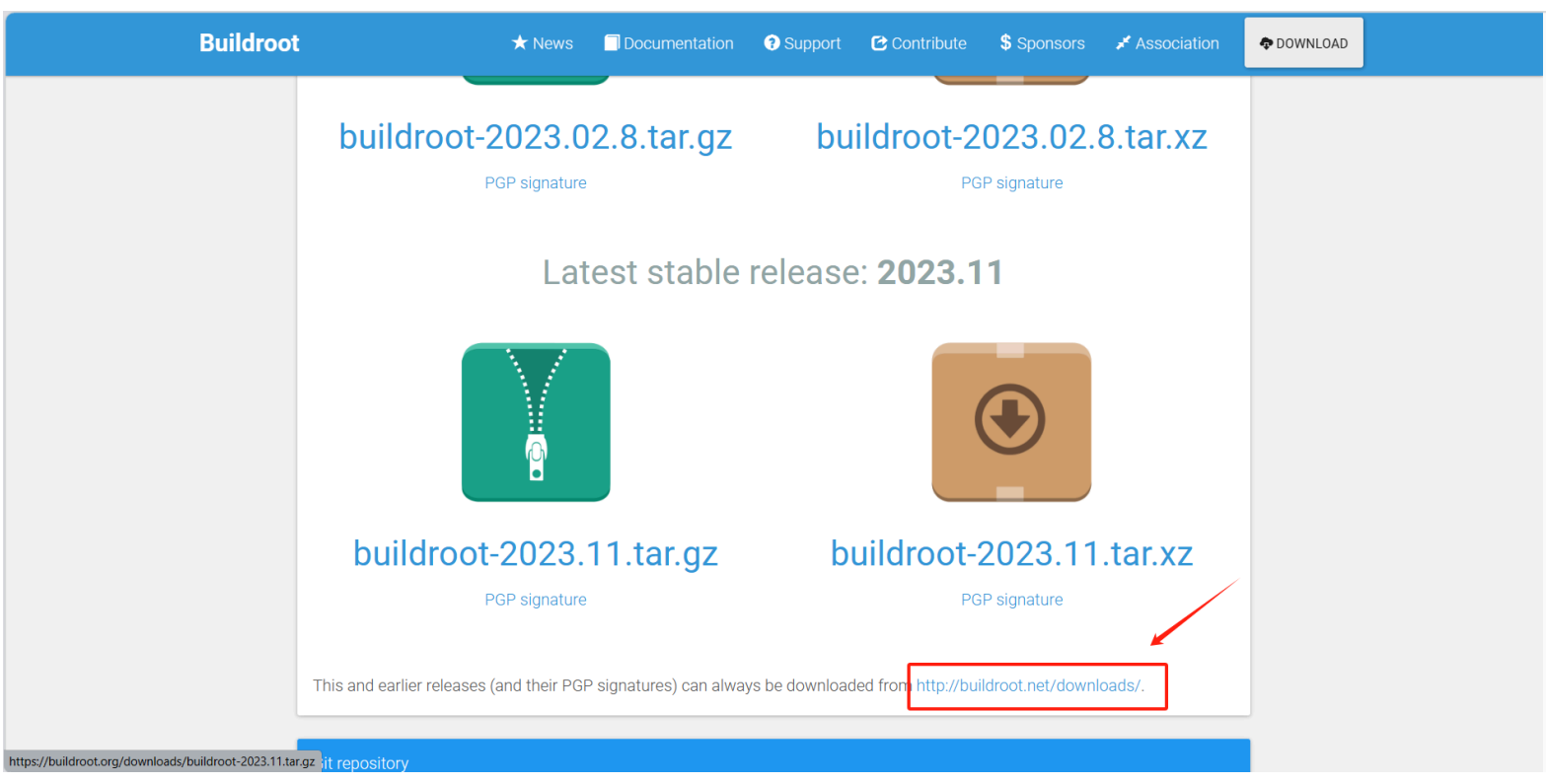
buildroot 官网地址为 https://buildroot.org/，我们可以通过浏览器查看。



点击DOWNLOAD



选择早期版本下载



瑞芯微SDK使用的版本， 我们查看

```
linux@ubuntu:~/rk3506$ vi buildroot/Makefile

# Set and export the version string
export BR2_VERSION := 2024.02
# Actual time the release is cut (for reproducible builds)
BR2_VERSION_EPOCH = 1709640000
```

8.4 `./build.sh` 命令汇总

1. 必知常用

命令	作用
`./build.sh all`	**一键全部编译**（uboot + kernel + rootfs + recovery + firmware + update.img）
`./build.sh kernel`	编译内核（zImage + dtb + boot.img）
`./build.sh rootfs` 或 `buildroot`	编译 Buildroot rootfs
`./build.sh recovery`	编译 recovery
`./build.sh updateimg`	打包固件 update.img
`./build.sh firmware`	生成 rockdev 目录下的所有固件文件

2. 硬件相关

命令	作用
`./build.sh loader` 或 `uboot`	编译 U-Boot（含 SPL + idblock）
`./build.sh wifibt`	编译 WiFi/BT 固件

3. Buildroot 专用

命令	作用
`./build.sh buildroot-config` 或 `bconfig`	Buildroot menuconfig 配置
`./build.sh buildroot-make` 或 `bmake`	只 make Buildroot（不进 SDK 编译）

4. 其他

命令	作用
`./build.sh cleanall`	清理全部
`./build.sh clean-kernel`	只清理内核
`./build.sh clean-rootfs`	只清理 rootfs
`./build.sh amp`	编译 AMP（MCU 协处理器）固件
`./build.sh post-rootfs <dir>`	rootfs 后处理钩子

3. 配置文件系统 buildroot

```
./build.sh buildroot-config
```

Kconfig配置

```
    Target options  --->
    Toolchain  --->
    Build options  --->
    System configuration  --->
    Kernel  --->
    Target packages  --->
    Filesystem images  --->
    Bootloaders  --->
    Host utilities  --->
    Legacy config options  --->
```

退出时保存配置

4. 编译buildroot

编译系统

```
./build.sh buildroot
```

- 运行结果

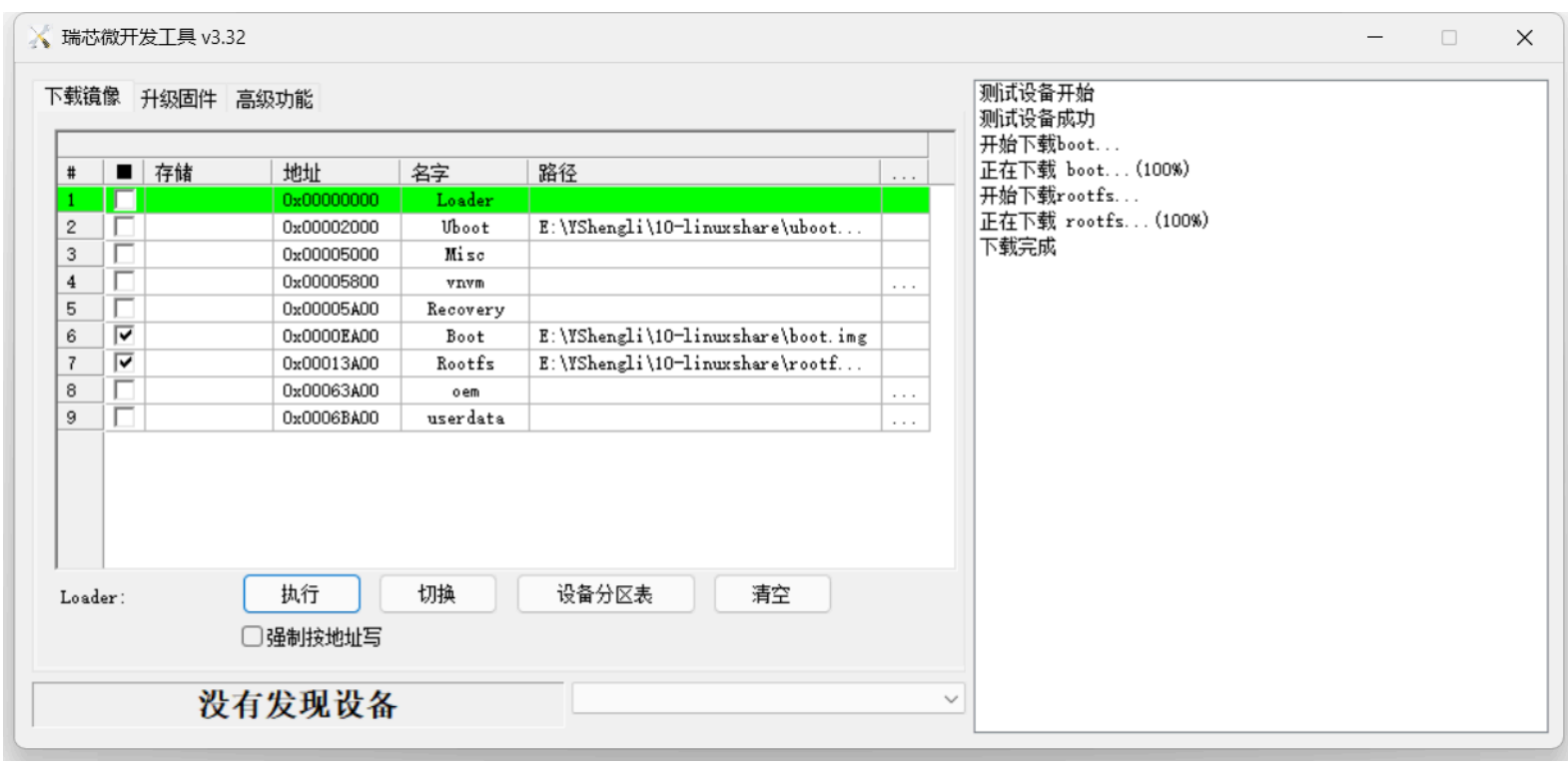
编译时间大约1小时左右，和电脑配置有关系。

```
linux@ubuntu:~/rk3506$ ./build.sh buildroot
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-08_16-48-08
Using last kernel version(6.1)
Using rootfs system(buildroot) from cmdline

============================================
          Final configs
============================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/emb/15-rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubifs:defaults:rk3506_oem:auto:@auto:userdata:auto:ubifs:defaults:norma
l:auto:@@@
RK_FLASH_SIZE=2048
RK_KERNEL=y
RK_KERNEL_ARCH=arm
RK_KERNEL_ARM32=y
RK_KERNEL_CFG=rk3506_defconfig
RK_KERNEL_CFG_FRAGMENTS=rk3506-display.config
RK_KERNEL_DTS=kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
RK_KERNEL_IMG=kernel/arch/arm/boot/zImage
RK_KERNEL_VERSION=6.1
RK_KERNEL_VERSION_RAW=6.1
RK_LOADER=y
RK_MISC=y
RK_MISC_BLANK=y
============================================
......
2026-06-08T16:49:31 >>> Running build.sh - post-rootfs succeeded.
2026-06-08T16:49:31 >>>   Executing post-build script board/rockchip/rk3506/post-build-fast-display.sh
2026-06-08T16:49:31 >>>   Generating root filesystems common tables
2026-06-08T16:49:31 >>>   Generating filesystem image rootfs.cpio
2026-06-08T16:49:48 >>>   Generating filesystem image rootfs.ext2
2026-06-08T16:49:50 >>>   Generating filesystem image rootfs.squashfs
2026-06-08T16:49:53 >>>   Generating filesystem image rootfs.tar
2026-06-08T16:49:56 >>>   Generating filesystem image rootfs.ubifs
2026-06-08T16:50:03 >>>   Generating filesystem image rootfs.ubi
Done in 55s
Log saved on /home/linux/emb/15-rk3506/output/sessions/2026-06-08_16-49-07/br-rockchip_rk3506_2026-06-08_16-
49-10.log
Generated images:
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.cpio
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.cpio.gz
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.ext2
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.ext4
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.squashfs
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.tar
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.ubi
/home/linux/emb/15-rk3506/output/buildroot/images/rootfs.ubifs
# 2026-06-08 16:49:19
# run hook: /home/linux/emb/15-rk3506/buildroot/output/rockchip_rk3506/target
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/oem.img' -> '/mnt/hgfs/linuxshare/oem.img'
'rockdev/parameter.txt' -> '/mnt/hgfs/linuxshare/parameter.txt'
'rockdev/recovery.img' -> '/mnt/hgfs/linuxshare/recovery.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
'rockdev/update.img' -> '/mnt/hgfs/linuxshare/update.img'
'rockdev/userdata.img' -> '/mnt/hgfs/linuxshare/userdata.img'
```

8.5 测试根文件系统

1. 下载文件系统

需要rootfs.img，并选择制定的分区，可以同时下载boot和rootfs， 因为这两个平时改动的比较多。



3. 运行结果

更新内核后，启动系统，如下：

```
[   15.817214] [HCC] get device status:1,result:1,time:0
[   15.817231] [HCC] BLE Mac Addr: 14:0a:02:9a:**:**
[   15.817260] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:2,device_msg:1
[   15.817289] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:4,device_msg:0
[   15.817301] [HCC] Get bsle device action msg, type=[0]
[   15.817310] [HCC] ble_enable_reply_cb is NULL, return!
[   15.817393] [HCC] begin to register hci device
[   15.817411] [HCC] bt_register_hci_dev enter
[   15.818025] [HCC] hcc bt rx proc ble closed
[   15.818636] [HCC] bt_register_hci_dev exit
[   15.819786] [HCC] enter hci_bt_open
[   15.819826] plat_soc:W]bsle_open_close_cmd, service[1], type[1].
[   15.820185] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:2,device_msg:1
[   15.820233] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:4,device_msg:0
[   15.820249] [HCC] Get bsle device action msg, type=[0]
[   15.820261] [HCC]  ble btc open finish
[   15.838514] [HCC]  ble open time:19
[   15.838560] [HCC] hci_bt_setup
[   15.850039] Bluetooth: MGMT ver 1.22
[   15.889498] Bluetooth: hci0: remove_uuid cache: 0
[   15.889604] Bluetooth: hci0: hci_update_class_sync power: 0
[   15.889738] Bluetooth: hci0: hci_update_eir_sync power: 0|07f8edc4
[   15.889773] Bluetooth: hci0: err 0
[   15.895458] Bluetooth: hci0: hci_update_class_sync power: 0
[   15.895580] Bluetooth: hci0: hci_update_eir_sync power: 0|07f8edc4
[   15.895606] Bluetooth: hci0: err 0
[   15.896040] Bluetooth: hci0: hci_update_class_sync power: 0
[   15.896149] Bluetooth: hci0: hci_update_eir_sync power: 0|07f8edc4
[   15.896177] Bluetooth: hci0: err 0
[   15.897268] Bluetooth: hci0: hci_update_class_sync power: 0
[   15.897375] Bluetooth: hci0: hci_update_eir_sync power: 0|07f8edc4
[   15.897397] Bluetooth: hci0: err 0
[   15.931118] Bluetooth: hci0: hci_update_class_sync power: 1
[   15.931168] Bluetooth: hci0: hci_update_eir_sync power: 1|07f8edc4
[   15.931204] Bluetooth: hci0: err 0
[   15.931582] Bluetooth: hci0: hci_update_class_sync power: 1
[   15.931623] Bluetooth: hci0: hci_update_eir_sync power: 1|07f8edc4
[   15.931643] Bluetooth: hci0: err 0
[   15.934627] Bluetooth: hci0: hci_update_class_sync power: 1
[   15.934679] Bluetooth: hci0: hci_update_eir_sync power: 1|07f8edc4
[   15.934728] Bluetooth: hci0: err 0
[   15.935224] Bluetooth: hci0: hci_update_eir_sync power: 1|07f8edc4
The user's boot script is executed here
fstrim: Starting fstrim process (1503)...
Starting input-event-daemon: done
root@rk3506-buildroot:/# ls
bin               lib32             rockchip-test     udisk
busybox.fragment  linuxrc           root              userdata
data              media             run               usr
dev               mnt               sbin              var
etc               oem               sdcard
info              opt               sys
lib               proc              tmp
root@rk3506-buildroot:/#
```

第三部分 Linux驱动实战

第09章 Linux GPIO子系统驱动

在学习了设备树之后，如果还按照裸板开发的方式配置寄存器来控制 IO 的方式太过于原始，Linux 内核提供了 pinctrl 子系统和 gpio 子系统用于GPIO驱动，当然 pinctrl 子系统负责的就不仅仅是GPIO的驱动了而是所有 pin 脚的配置。

pinctrl 子系统是随着设备树的加入而加入的，依赖于设备树。GPIO 子系统在之前的内核中也是存在的，但是 pinctrl 子系统的加入 GPIO 子系统也是有很大的改变，之前的 GPIO 子系统需要芯片厂商提供的 mach文件，而加入设备树后，GPIO 子系统使用设备树来实现。

9.1 pinctrl 子系统

1. pinctrl 子系统简介

Linux的pinctrl子系统是一个用于配置和管理硬件引脚的组件。它提供了统一的接口和机制，使用户能够对硬件引脚进行配置和操作。

主要功能包括：

1. 引脚配置：可以通过pinctrl子系统将硬件引脚设置为不同的工作模式。将引脚设置为输入或输出模式，上拉、下拉或推挽等。
2. 引脚映射：允许将硬件引脚映射到相应的设备。
3. 中断控制：支持引脚的中断功能。可以配置引脚的中断触发方式，如上升沿、下降沿、高电平或低电平，并注册中断处理函数。
4. 引脚状态查询：提供了接口来查询硬件引脚的状态。可以查询引脚的输入输出状态、电平状态和中断状态等信息。

2. 配置 pinctrl

下面我们来看一下 pinctrl 子系统在RK3506的设备树中是如何实现并使用的。

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    leds: leds {
        status = "okay";
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&leds_gpio>;
        work_4g_rst: work-4g-rst {
            gpios = <&gpio4 RK_PB2 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

        wifi_en: wifi-en {
            gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
            default-state = "off";
        };

    };

    led {
        leds_gpio: leds-gpio {
            rockchip,pins = <4 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
```

节点解释。

```
    leds: leds {
        status = "okay";       //  表示该节点可用
        compatible = "gpio-leds";    // 兼容驱动的，  驱动需要匹配的名字
        pinctrl-names = "default";
        pinctrl-0 = <&leds_gpio>;
        work_4g_rst: work-4g-rst {
            gpios = <&gpio4 RK_PB2 GPIO_ACTIVE_HIGH>;    // on 时输出高电平 ，  off 时输出低电平
            default-state = "on";    // 默认输出高电平 ，  驱动加载时会立即将该引脚设置为 高电平（on），4G模块复位
        };

        wifi_en: wifi-en {
            gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;   // 高电平有效 on 时输出高电平 ，off 时输出低电平
            default-state = "off";  // 默认输出低电平 ，wifi 模组关闭电源
            //  即 WiFi 模块默认关闭。后续可由用户空间通过 /sys/class/leds/wifi-en/brightness
            //  写入 1 或 0 来使能或关闭模块。
        };

    };

    led {
        leds_gpio: leds-gpio {
            // 这里是 不启用上啦电阻，也是为了占用这个引脚
            rockchip,pins = <4 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>;

            // 这里少写了，  但是并不影响程序的正常执行，  因为 LED 驱动（gpio-leds）通过 gpios 属性
            // 向内核申请 GPIO，内核 GPIO 子系统只要能成功申请到该 GPIO，就能控制它。
            rockchip,pins = <4 RK_PB0 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
```

9.2 GPIO 子系统

1. GPIO 子系统介绍

GPIO 子系统是 Linux 内核提供的一套框架，用于对通用输入输出（GPIO）进行管理和控制。

GPIO 子系统通过文件系统接口将 GPIO 引脚映射为用户空间的文件，从而允许应用程序通过读写文件的方式来控制和读取 GPIO 状态。

主要组件包括：

1. GPIO 控制器：负责控制一组 GPIO 引脚的硬件模块，每个控制器都有唯一的标识符，如 GPIO 编号或物理地址。
2. GPIO 字符设备驱动：将 GPIO 控制器映射到用户空间文件系统的中间层，提供了一组文件操作接口，用于配置和操作 GPIO 引脚。
3. GPIO 虚拟文件系统：用户空间与内核空间之间的接口，通过文件系统的方式将 GPIO 引脚映射为用户空间的文件。

使用 Linux GPIO 子系统，开发者可以方便地在用户空间中对 GPIO 进行控制，实现各种应用场景，如控制外部设备、采集传感器数据等。

2. GPIO 子系统函数接口

gpio_request

- 函数原型

```
#include<linux/gpio.h>

int gpio_request(unsigned gpio, const char *label);
```

- 函数功能
  - gpio_request 函数用于申请一个GPIO管脚，在使用一个 GPIO 之前一定要使用 gpio_request 进行申请。
- 函数参数
  - gpio：要申请的 gpio 标号，使用 of_get_named_gpio 函数从设备树获取指定GPIO属性信息，此函数会返回这个 GPIO 的标号。
  - label：给 gpio 设置个名字。
- 函数返回值
  - 成功返回：0
  - 失败返回：负的错误码

gpio_free

- 函数原型

```
#include<linux/gpio.h>

void gpio_free(unsigned gpio);
```

- 函数功能
  - 如果不使用某个 GPIO 了，那么就可以调用 gpio_free 函数进行释放。
- 函数参数
  - gpio：要释放的 gpio 标号。
- 函数返回值
  - 无

gpio_direction_output

- 函数原型

```
#include<linux/gpio.h>

int gpio_direction_output(unsigned gpio, int value);
```

- 函数功能
  - 设置某个 GPIO 为输出，并且设置默认输出值。
- 函数参数
  - gpio：要设置为输出的 GPIO 标号。
  - value：GPIO 默认输出值。
- 函数返回值
  - 成功返回：0
  - 失败返回：负的错误码

gpio_direction_input

- 函数原型

```
#include<linux/gpio.h>

int gpio_direction_input(unsigned gpio);
```

- 函数功能
  - 设置某个 GPIO 为输入。
- 函数参数
  - gpio：要设置为输入的 GPIO 标号。
- 函数返回值
  - 成功返回：0
  - 失败返回：负的错误码

gpio_get_value

- 函数原型

```
#include<linux/gpio.h>

#define gpio_get_value   __gpio_get_value

int __gpio_get_value(unsigned gpio);
```

- 函数功能
  - 获取某个 GPIO 的值(0 或 1)，此函数是个宏。
- 函数参数
  - gpio：要获取的 GPIO 标号。
- 函数返回值
  - 成功返回：0
  - 失败返回：负的错误码

gpio_set_value

- 函数原型

```
#include<linux/gpio.h>

#define gpio_set_value __gpio_set_value

void __gpio_set_value(unsigned gpio, int value);
```

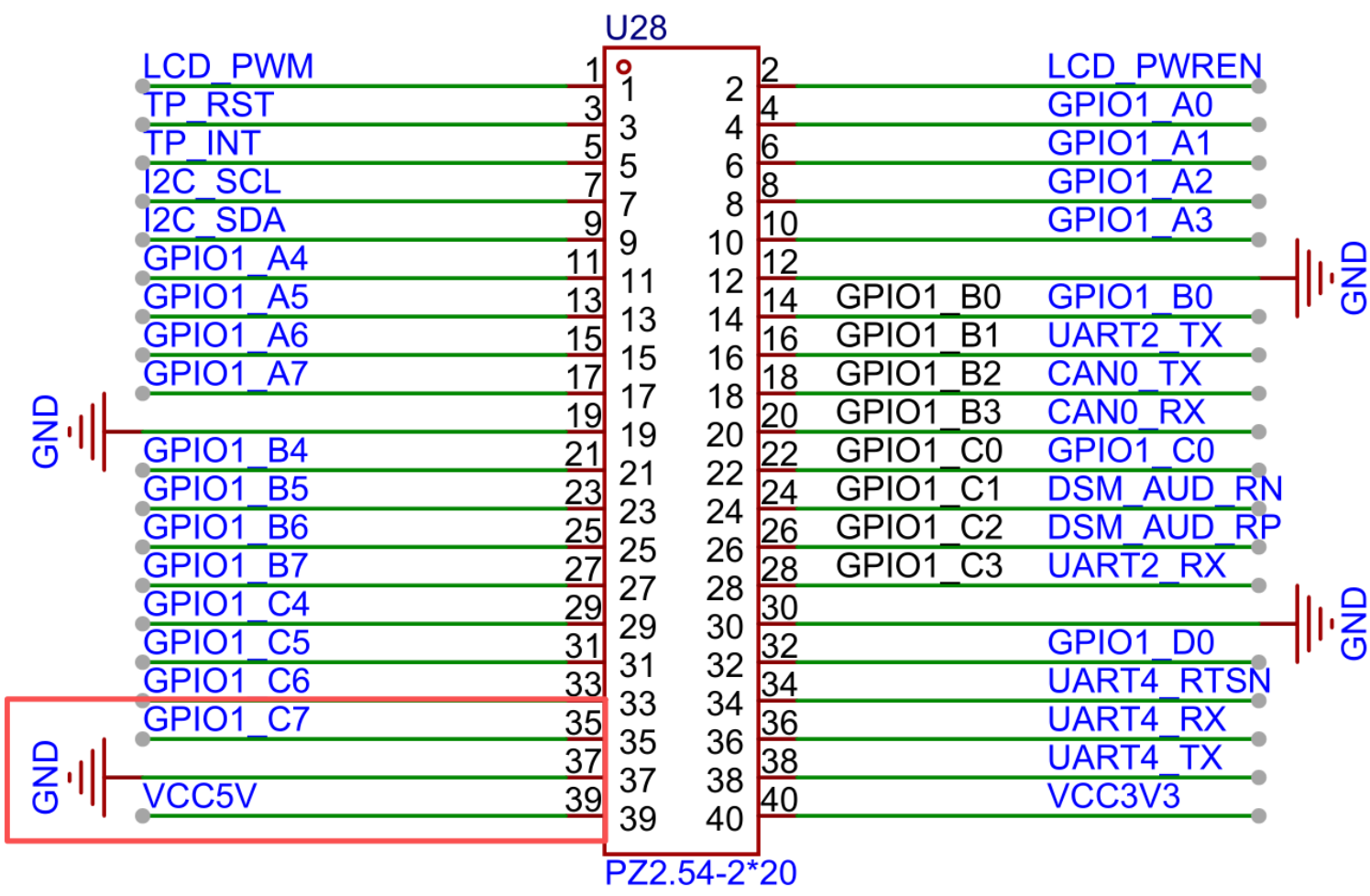
- 函数功能
  - 此函数用于设置某个 GPIO 的值，此函数是个宏。
- 函数参数
  - gpio：要设置的 GPIO 标号。
  - value：要设置的值。
- 函数返回值
  - 无

9.3 GPIO驱动

1. GPIO1_C7 硬件原理图分析

由于我们这个板子的GPIO没有外接硬件， 因此没法直接控制硬件， 我们使用扩展IO来展示现象。

我们控制GPIO1_C7 引脚，使用逻辑分析仪抓取电平状态。



2. 添加设备树节点

kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    leds: leds {
        status = "okay";
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&leds_gpio>;
        work_4g_rst: work-4g-rst {
            gpios = <&gpio4 RK_PB2 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

        wifi_en: wifi-en {
            gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
            default-state = "off";
        };

        gpio1_c7: gpio1-c7 {
            gpios = <&gpio1 RK_PC7 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

    };


    led {
        leds_gpio: leds-gpio {
            rockchip,pins =
                <4 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>,
                <4 RK_PB0 RK_FUNC_GPIO &pcfg_pull_none>,
                <1 RK_PC7 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
```

打开文档 Documentation/devicetree/bindings/leds/leds-gpio.txt，此文档详细的讲解了 Linux 自带驱动对应的设备树节点该如何编写，我们在编写设备节点的时候要注意以下几点：

- leds：创建一个节点表示 LED 设备，比如 leds，如果板子上有多个 LED 的话每个 LED 都作为leds 的子节点。
- compatible：leds 节点的 compatible 属性值一定要为“gpio-leds”。
- pinctrl-names：属性用于指定一个或多个 pinctrl-0、pinctrl-1 等属性的名称，这里只指定了一种情景。
- pinctrl-0 ： 这个就是`default`情景要使用的引脚。
- label : 设置 label 属性，此属性为可选，每个子节点都有一个 label 属性，label 属性一般表示 LED 灯的名字。
- gpios : 每个子节点必须要设置 gpios 属性值，表示此 LED 所使用的 GPIO 引脚。
- default-state ： 设置“default-state”属性值，可以设置为 on、off 或 keep。
  - 为 on 的时候 LED 灯默认打开。
  - 为 off 的话 LED 灯默认关闭。
  - 为 keep 的话 LED 灯保持当前模式。
- 可以设置 “ linux,default-trigger ” 属性值，也就是设置LED灯的默认功能 ， 比如：
  - backlight：LED 灯作为背光。
  - default-on：LED 灯打开。
  - heartbeat：LED 灯作为心跳指示灯，可以作为系统运行提示灯。
  - ide-disk：LED 灯作为硬盘活动指示灯。
  - timer：LED 灯周期性闪烁，由定时器驱动，闪烁频率可以修改。

3. 配置内核LED驱动

在 Linux 内核中集成了 LED 的驱动，如果要使用这个驱动就需要先修改 Linux 内核的配置文件，选中LED 驱动，这样在内核编译的时候，就会把驱动编译到内核中去。

- 启用内核LED驱动

```
Device Drivers  --->
    [*] LED Support  --->
         <*>   LED Support for GPIO connected LEDs
```

配置好 Linux 内核后，打开.config 文件，就会看到`CONFIG_LEDS_GPIO=y`这样的信息。

Linux 内核会根据CONFIG_LEDS_GPIO的值来判断是否编译LED驱动，如果 `CONFIG_LEDS_GPIO =y`，则将其编译进内核中。

```
#
# LED drivers
#
# CONFIG_LEDS_AN30259A is not set
# CONFIG_LEDS_BCM6328 is not set
# CONFIG_LEDS_BCM6358 is not set
# CONFIG_LEDS_CR0014114 is not set
# CONFIG_LEDS_LM3530 is not set
# CONFIG_LEDS_LM3532 is not set
# CONFIG_LEDS_LM3642 is not set
# CONFIG_LEDS_LM3692X is not set
# CONFIG_LEDS_PCA9532 is not set
CONFIG_LEDS_GPIO=y
# CONFIG_LEDS_LP3944 is not set
# CONFIG_LEDS_LP3952 is not set
```

- 操作过程

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-25_16-20-26
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubifs:defaults:rk3506_oem:auto:@auto:userdata:auto:ubifs:defaults:normal:auto:@@@
RK_FLASH_SIZE=2048
RK_KERNEL=y
RK_KERNEL_ARCH=arm
RK_KERNEL_ARM32=y
RK_KERNEL_CFG=rk3506_defconfig
RK_KERNEL_CFG_FRAGMENTS=rk3506-display.config
RK_KERNEL_DTS=kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
RK_KERNEL_IMG=kernel/arch/arm/boot/zImage
RK_KERNEL_VERSION=6.1
RK_KERNEL_VERSION_RAW=6.1
RK_LOADER=y
RK_MISC=y
RK_MISC_BLANK=y
RK_NETWORK_CHECK=y
RK_OVERLAY=y
RK_OWNER=linux
RK_OWNER_UID=1000
RK_PARAMETER=parameter-flash.txt
RK_RECOVERY=y
RK_RECOVERY_CFG=rockchip_rk3506_recovery
RK_RECOVERY_CPIO_GZ=y
RK_RECOVERY_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot4recovery.its
RK_RECOVERY_INITRD_TYPE=cpio.gz
RK_ROOTFS=y
RK_ROOTFS_ASYNC_COMMIT=y
RK_ROOTFS_DEBUG_INFO=y
RK_ROOTFS_FSTRIM=y
RK_ROOTFS_FSTRIM_INTERVAL=3600
RK_ROOTFS_GENERATE_LOGS=y
RK_ROOTFS_INPUT_EVENT_DAEMON=y
RK_ROOTFS_INSTALL_MODULES=y
RK_ROOTFS_IRQBALANCE=y
RK_ROOTFS_LD_CACHE=y
RK_ROOTFS_OVERLAY=y
RK_ROOTFS_PREFER_PREBUILT_TOOLS=y
RK_ROOTFS_STRIP_MODULES=y
RK_ROOTFS_SYSTEM=buildroot
RK_ROOTFS_TYPE=ubi
RK_ROOTFS_UBI=y
RK_ROOTFS_UDEV_RULES=y
RK_SAVE_TRACKED=y
RK_SESSION=2026-06-25_16-20-26
RK_UBI=y
RK_UBI_BLOCK_SIZE=0x20000
RK_UBI_PAGE_SIZE=0x800
RK_UBOOT_ARCH=arm
RK_UBOOT_ARM32=y
RK_UBOOT_CFG=rk3506
RK_UBOOT_CFG_FRAGMENTS=rk3506b rk3506_tb
RK_UBOOT_SPL=y
RK_UPDATE=y
RK_USB_ADBD=y
RK_USB_ADBD_SHELL=/bin/bash
RK_USB_ADBD_TCP_PORT=5555
RK_USB_FW_VER=0x0310
RK_USB_GADGET=y
RK_USB_MANUFACTURER=Rockchip
RK_USB_PID=0
RK_USB_PRODUCT=rk3xxx
RK_USB_VID=0x2207
RK_USE_FIT_IMG=y
RK_WIFIBT=y
RK_WIFIBT_CHIP=AP6256
RK_WIFIBT_MODULES=AP6256
RK_WIFIBT_ON_BOARD=y
RK_YOCTO=y
RK_YOCTO_DISPLAY_PLATFORM=wayland
RK_YOCTO_MACHINE=rockchip-rk3506-evb
RK_YOCTO_USBMOUNT=y

Toolchain for kernel:
/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-

=========================================
          Start building kernel-config
=========================================
```

```
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm rk3506_defconfig rk3506-display.config
make: Entering directory '/home/linux/rk3506/kernel-6.1'
#
# No change to .config
#
Using .config as base
Merging ./arch/arm/configs/rk3506-display.config
#
# merged configuration written to .config (needs make)
#
#
# configuration written to .config
#
...
linux@ubuntu:~/rk3506$ ./build.sh kernel     # 编译内核
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-06-25_16-22-20
Using last kernel version(6.1)

=========================================
          Final configs
=========================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
RK_CHIP=rk3506
RK_CHIP_ARM32=y
RK_CHIP_FAMILY=rk3506
RK_DEFCONFIG=rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ubifs:defaults:rk3506_oem:auto:@auto:userdata:auto:ubifs:defaults:normal:auto:@@@
RK_FLASH_SIZE=2048
RK_KERNEL=y
RK_KERNEL_ARCH=arm
RK_KERNEL_ARM32=y
RK_KERNEL_CFG=rk3506_defconfig
RK_KERNEL_CFG_FRAGMENTS=rk3506-display.config
RK_KERNEL_DTS=kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
RK_KERNEL_IMG=kernel/arch/arm/boot/zImage
RK_KERNEL_VERSION=6.1
RK_KERNEL_VERSION_RAW=6.1
RK_LOADER=y
RK_MISC=y
RK_MISC_BLANK=y
RK_NETWORK_CHECK=y
RK_OVERLAY=y
RK_OWNER=linux
RK_OWNER_UID=1000
RK_PARAMETER=parameter-flash.txt
RK_RECOVERY=y
RK_RECOVERY_CFG=rockchip_rk3506_recovery
RK_RECOVERY_CPIO_GZ=y
RK_RECOVERY_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot4recovery.its
RK_RECOVERY_INITRD_TYPE=cpio.gz
RK_ROOTFS=y
RK_ROOTFS_ASYNC_COMMIT=y
RK_ROOTFS_DEBUG_INFO=y
RK_ROOTFS_FSTRIM=y
RK_ROOTFS_FSTRIM_INTERVAL=3600
RK_ROOTFS_GENERATE_LOGS=y
RK_ROOTFS_INPUT_EVENT_DAEMON=y
RK_ROOTFS_INSTALL_MODULES=y
RK_ROOTFS_IRQBALANCE=y
RK_ROOTFS_LD_CACHE=y
RK_ROOTFS_OVERLAY=y
RK_ROOTFS_PREFER_PREBUILT_TOOLS=y
RK_ROOTFS_STRIP_MODULES=y
RK_ROOTFS_SYSTEM=buildroot
RK_ROOTFS_TYPE=ubi
RK_ROOTFS_UBI=y
RK_ROOTFS_UDEV_RULES=y
RK_SAVE_TRACKED=y
RK_SESSION=2026-06-25_16-22-20
RK_UBI=y
RK_UBI_BLOCK_SIZE=0x20000
RK_UBI_PAGE_SIZE=0x800
RK_UBOOT_ARCH=arm
RK_UBOOT_ARM32=y
RK_UBOOT_CFG=rk3506
RK_UBOOT_CFG_FRAGMENTS=rk3506b rk3506_tb
RK_UBOOT_SPL=y
RK_UPDATE=y
RK_USB_ADBD=y
RK_USB_ADBD_SHELL=/bin/bash
RK_USB_ADBD_TCP_PORT=5555
RK_USB_FW_VER=0x0310
RK_USB_GADGET=y
RK_USB_MANUFACTURER=Rockchip
RK_USB_PID=0
RK_USB_PRODUCT=rk3xxx
RK_USB_VID=0x2207
RK_USE_FIT_IMG=y
RK_WIFIBT=y
RK_WIFIBT_CHIP=AP6256
RK_WIFIBT_MODULES=AP6256
RK_WIFIBT_ON_BOARD=y
RK_YOCTO=y
RK_YOCTO_DISPLAY_PLATFORM=wayland
RK_YOCTO_MACHINE=rockchip-rk3506-evb
RK_YOCTO_USBMOUNT=y

Toolchain for kernel:
/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-

=========================================
          Start building kernel
=========================================
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm rk3506_defconfig rk3506-display.config
make: Entering directory '/home/linux/rk3506/kernel-6.1'
#
# No change to .config
#
Using .config as base
Merging ./arch/arm/configs/rk3506-display.config
#
# merged configuration written to .config (needs make)
#
#
# configuration written to .config
#
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ make -C /home/linux/rk3506/kernel/ -j9 CROSS_COMPILE=/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf- ARCH=arm ido-evb3506-v1a-flash-mipi-720x720.img
make: Entering directory '/home/linux/rk3506/kernel-6.1'
  SYNC    include/config/auto.conf.cmd
  DTC     arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dtb
arch/arm/boot/dts/rk3506.dtsi:192.5-37: Warning (graph_endpoint): /vop@ff600000/port/endpoint@0:remote-endpoint: graph phandle is not valid
  CALL    scripts/checksyscalls.sh
  Kernel: arch/arm/boot/Image is ready
  Kernel: arch/arm/boot/Image.gz is ready
  CC      arch/arm/boot/compressed/misc.o
  CC      arch/arm/boot/compressed/decompress.o
  CC      arch/arm/boot/compressed/string.o
  CC      arch/arm/boot/compressed/fdt_rw.o
  CC      arch/arm/boot/compressed/fdt_ro.o
  CC      arch/arm/boot/compressed/fdt_wip.o
  CC      arch/arm/boot/compressed/fdt.o
  CC      arch/arm/boot/compressed/fdt_check_mem_start.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready
  Image:  resource.img (with ido-evb3506-v1a-flash-mipi-720x720.dtb logo.bmp logo_kernel.bmp) is ready
  Image:  boot.img (with Image  resource.img) is ready
  Image:  zboot.img (with zImage  resource.img) is ready
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
```

```
+ /home/linux/rk3506/device/rockchip/common/scripts/mk-fitimage.sh kernel/zboot.img
/home/linux/rk3506/device/rockchip/.chip/thunderboot.its kernel/arch/arm/boot/zImage
kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm
Created:         Thu Jun 25 16:22:43 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Thu Jun 25 16:22:43 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44661 Bytes = 43.61 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   e7ecb601ad13530cc2804745c223510c313aa5f2864490f25d46d569cdfe4033
 Image 1 (kernel)
  Description:  unavailable
  Created:      Thu Jun 25 16:22:43 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4169496 Bytes = 4071.77 KiB = 3.98 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   b6ef981e85c0a240092ddcacc27dfb776e47aa26e2117bb0831d5c40a1e93e7c
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and no ramdisk
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/linux/rk3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
Running 10-kernel.sh - build_kernel succeeded.
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/oem.img' -> '/mnt/hgfs/linuxshare/oem.img'
'rockdev/parameter.txt' -> '/mnt/hgfs/linuxshare/parameter.txt'
'rockdev/recovery.img' -> '/mnt/hgfs/linuxshare/recovery.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
'rockdev/update.img' -> '/mnt/hgfs/linuxshare/update.img'
'rockdev/userdata.img' -> '/mnt/hgfs/linuxshare/userdata.img'
linux@ubuntu:~/rk3506$
```

4. LED驱动测试

更新内核和设备树到开发板内，并使用nandflash启动系统。

LED控制命令

- 点亮gpio1-c7标签的LED

```
echo 1 > /sys/class/leds/gpio1-c7/brightness
```

- 熄灭gpio1-c7标签的LED

```
echo 0 > /sys/class/leds/gpio1-c7/brightness
```

如果能正常的打开和关闭 gpio1-c7 ， 说明 gpio1-c7 驱动工作正常。

- 运行结果

```
root@rk3506-buildroot:/# ls /sys/class/leds/
gpio1-c7     wifi-en      work-4g-rst
root@rk3506-buildroot:/# echo  1 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/# echo  0 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/# echo  1 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/# echo  1 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/# echo  0 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/# echo  1 >/sys/class/leds/gpio1-c7/brightness
root@rk3506-buildroot:/#
```

5. 硬件现象

- 输出低电平

```
echo 0 > /sys/class/leds/gpio1-c7/brightness
```

- 输出高电平

```
echo  1 >/sys/class/leds/gpio1-c7/brightness
```

6. LED驱动分析

LED驱动文件为`drivers/leds/leds-gpio.c`，首先打开`/drivers/leds/Makefile`这个文件，内容如下：

```
obj-$(CONFIG_LEDS_GPIO)         += leds-gpio.o
```

我们已经配置了`CONFIG_LEDS_GPIO=y`，所以当内核编译时，就会选择编译 led-gpio.c文件。

/drivers/leds/leds-gpio.c

```
// SPDX-License-Identifier: GPL-2.0-only
/*
 * LEDs driver for GPIOs
 *
 * Copyright (C) 2007 8D Technologies inc.
 * Raphael Assenat <raph@8d.com>
 * Copyright (C) 2008 Freescale Semiconductor, Inc.
 */
#include <linux/err.h>
#include <linux/gpio.h>
#include <linux/gpio/consumer.h>
#include <linux/kernel.h>
#include <linux/leds.h>
#include <linux/module.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/property.h>
#include <linux/slab.h>

struct gpio_led_data {
    struct led_classdev cdev;
    struct gpio_desc *gpiod;
    u8 can_sleep;
    u8 blinking;
    gpio_blink_set_t platform_gpio_blink_set;
};

static inline struct gpio_led_data *
            cdev_to_gpio_led_data(struct led_classdev *led_cdev)
{
    return container_of(led_cdev, struct gpio_led_data, cdev);
}
```

```c
static void gpio_led_set(struct led_classdev *led_cdev,
    enum led_brightness value)
{
    struct gpio_led_data *led_dat = cdev_to_gpio_led_data(led_cdev);
    int level;

    if (value == LED_OFF)
        level = 0;
    else
        level = 1;

    if (led_dat->blinking) {
        led_dat->platform_gpio_blink_set(led_dat->gpiod, level,
                        NULL, NULL);
        led_dat->blinking = 0;
    } else {
        if (led_dat->can_sleep)
            gpiod_set_value_cansleep(led_dat->gpiod, level);
        else
            gpiod_set_value(led_dat->gpiod, level);
    }
}

static int gpio_led_set_blocking(struct led_classdev *led_cdev,
    enum led_brightness value)
{
    gpio_led_set(led_cdev, value);
    return 0;
}

static int gpio_blink_set(struct led_classdev *led_cdev,
    unsigned long *delay_on, unsigned long *delay_off)
{
    struct gpio_led_data *led_dat = cdev_to_gpio_led_data(led_cdev);

    led_dat->blinking = 1;
    return led_dat->platform_gpio_blink_set(led_dat->gpiod, GPIO_LED_BLINK,
                        delay_on, delay_off);
}
```

```c
static int create_gpio_led(const struct gpio_led *template,
    struct gpio_led_data *led_dat, struct device *parent,
    struct fwnode_handle *fwnode, gpio_blink_set_t blink_set)
{
    struct led_init_data init_data = {};
    int ret, state;

    led_dat->cdev.default_trigger = template->default_trigger;
    led_dat->can_sleep = gpiod_cansleep(led_dat->gpiod);
    if (!led_dat->can_sleep)
        led_dat->cdev.brightness_set = gpio_led_set;
    else
        led_dat->cdev.brightness_set_blocking = gpio_led_set_blocking;
    led_dat->blinking = 0;
    if (blink_set) {
        led_dat->platform_gpio_blink_set = blink_set;
        led_dat->cdev.blink_set = gpio_blink_set;
    }
    if (template->default_state == LEDS_GPIO_DEFSTATE_KEEP) {
        state = gpiod_get_value_cansleep(led_dat->gpiod);
        if (state < 0)
            return state;
    } else {
        state = (template->default_state == LEDS_GPIO_DEFSTATE_ON);
    }
    led_dat->cdev.brightness = state ? LED_FULL : LED_OFF;
    if (!template->retain_state_suspended)
        led_dat->cdev.flags |= LED_CORE_SUSPENDRESUME;
    if (template->panic_indicator)
        led_dat->cdev.flags |= LED_PANIC_INDICATOR;
    if (template->retain_state_shutdown)
        led_dat->cdev.flags |= LED_RETAIN_AT_SHUTDOWN;

    ret = gpiod_direction_output(led_dat->gpiod, state);
    if (ret < 0)
        return ret;

    if (template->name) {
        led_dat->cdev.name = template->name;
        ret = devm_led_classdev_register(parent, &led_dat->cdev);
    } else {
        init_data.fwnode = fwnode;
        ret = devm_led_classdev_register_ext(parent, &led_dat->cdev,
                            &init_data);
    }

    return ret;
}

struct gpio_leds_priv {
    int num_leds;
    struct gpio_led_data leds[];
};

static inline int sizeof_gpio_leds_priv(int num_leds)
{
    return sizeof(struct gpio_leds_priv) +
        (sizeof(struct gpio_led_data) * num_leds);
}

static struct gpio_leds_priv *gpio_leds_create(struct platform_device *pdev)
{
    struct device *dev = &pdev->dev;
    struct fwnode_handle *child;
    struct gpio_leds_priv *priv;
    int count, ret;

    count = device_get_child_node_count(dev);
    if (!count)
        return ERR_PTR(-ENODEV);

    priv = devm_kzalloc(dev, sizeof_gpio_leds_priv(count), GFP_KERNEL);
    if (!priv)
        return ERR_PTR(-ENOMEM);

    device_for_each_child_node(dev, child) {
        struct gpio_led_data *led_dat = &priv->leds[priv->num_leds];
        struct gpio_led led = {};
        const char *state = NULL;

        led.gpiod = devm_fwnode_get_gpiod_from_child(dev, NULL, child,
                                GPIOD_ASIS,
                                led.name);
        if (IS_ERR(led.gpiod)) {
            fwnode_handle_put(child);
            return ERR_CAST(led.gpiod);
        }

        led_dat->gpiod = led.gpiod;

        fwnode_property_read_string(child, "linux,default-trigger",
                        &led.default_trigger);

        if (!fwnode_property_read_string(child, "default-state",
                        &state)) {
            if (!strcmp(state, "keep"))
                led.default_state = LEDS_GPIO_DEFSTATE_KEEP;
            else if (!strcmp(state, "on"))
                led.default_state = LEDS_GPIO_DEFSTATE_ON;
            else
                led.default_state = LEDS_GPIO_DEFSTATE_OFF;
        }

        if (fwnode_property_present(child, "retain-state-suspended"))
            led.retain_state_suspended = 1;
        if (fwnode_property_present(child, "retain-state-shutdown"))
            led.retain_state_shutdown = 1;
        if (fwnode_property_present(child, "panic-indicator"))
            led.panic_indicator = 1;

        ret = create_gpio_led(&led, led_dat, dev, child, NULL);
        if (ret < 0) {
            fwnode_handle_put(child);
            return ERR_PTR(ret);
        }
        priv->num_leds++;
    }
```

```
    return priv;
}

static const struct of_device_id of_gpio_leds_match[] = {
    { .compatible = "gpio-leds", },
    {},
};

MODULE_DEVICE_TABLE(of, of_gpio_leds_match);

static struct gpio_desc *gpio_led_get_gpiod(struct device *dev, int idx,
                        const struct gpio_led *template)
{
    struct gpio_desc *gpiod;
    unsigned long flags = GPIOF_OUT_INIT_LOW;
    int ret;

    /*
     * This means the LED does not come from the device tree
     * or ACPI, so let's try just getting it by index from the
     * device, this will hit the board file, if any and get
     * the GPIO from there.
     */
    gpiod = devm_gpiod_get_index(dev, NULL, idx, flags);
    if (!IS_ERR(gpiod)) {
        gpiod_set_consumer_name(gpiod, template->name);
        return gpiod;
    }
    if (PTR_ERR(gpiod) != -ENOENT)
        return gpiod;

    /*
     * This is the legacy code path for platform code that
     * still uses GPIO numbers. Ultimately we would like to get
     * rid of this block completely.
     */

    /* skip leds that aren't available */
    if (!gpio_is_valid(template->gpio))
        return ERR_PTR(-ENOENT);

    if (template->active_low)
        flags |= GPIOF_ACTIVE_LOW;

    ret = devm_gpio_request_one(dev, template->gpio, flags,
                    template->name);
    if (ret < 0)
        return ERR_PTR(ret);

    gpiod = gpio_to_desc(template->gpio);
    if (!gpiod)
        return ERR_PTR(-EINVAL);

    return gpiod;
}

static int gpio_led_probe(struct platform_device *pdev)
{
    struct gpio_led_platform_data *pdata = dev_get_platdata(&pdev->dev);
    struct gpio_leds_priv *priv;
    int i, ret = 0;

    if (pdata && pdata->num_leds) {
        priv = devm_kzalloc(&pdev->dev,
                sizeof_gpio_leds_priv(pdata->num_leds),
                    GFP_KERNEL);
        if (!priv)
            return -ENOMEM;

        priv->num_leds = pdata->num_leds;
        for (i = 0; i < priv->num_leds; i++) {
            const struct gpio_led *template = &pdata->leds[i];
            struct gpio_led_data *led_dat = &priv->leds[i];

            if (template->gpiod)
                led_dat->gpiod = template->gpiod;
            else
                led_dat->gpiod =
                    gpio_led_get_gpiod(&pdev->dev,
                                i, template);
            if (IS_ERR(led_dat->gpiod)) {
                dev_info(&pdev->dev, "Skipping unavailable LED gpio %d (%s)\n",
                     template->gpio, template->name);
                continue;
            }

            ret = create_gpio_led(template, led_dat,
                          &pdev->dev, NULL,
                          pdata->gpio_blink_set);
            if (ret < 0)
                return ret;
        }
    } else {
        priv = gpio_leds_create(pdev);
        if (IS_ERR(priv))
            return PTR_ERR(priv);
    }

    platform_set_drvdata(pdev, priv);

    return 0;
}

static void gpio_led_shutdown(struct platform_device *pdev)
{
    struct gpio_leds_priv *priv = platform_get_drvdata(pdev);
    int i;

    for (i = 0; i < priv->num_leds; i++) {
        struct gpio_led_data *led = &priv->leds[i];

        if (!(led->cdev.flags & LED_RETAIN_AT_SHUTDOWN))
            gpio_led_set(&led->cdev, LED_OFF);
    }
}

static struct platform_driver gpio_led_driver = {
    .probe      = gpio_led_probe,
    .shutdown   = gpio_led_shutdown,
    .driver     = {
        .name   = "leds-gpio",
        .of_match_table = of_gpio_leds_match,
    },
};

module_platform_driver(gpio_led_driver);

MODULE_AUTHOR("Raphael Assenat <raph@8d.com>, Trent Piepho <tpiepho@freescale.com>");
MODULE_DESCRIPTION("GPIO LED driver");
MODULE_LICENSE("GPL");
```

第10章 Linux input子系统驱动

Linux INPUT 子系统是用于处理输入设备（如键盘、鼠标、触摸屏等）的核心子系统。它负责从硬件读取输入事件，并将其值传递给用户空间应用程序。

10.1 input 子系统

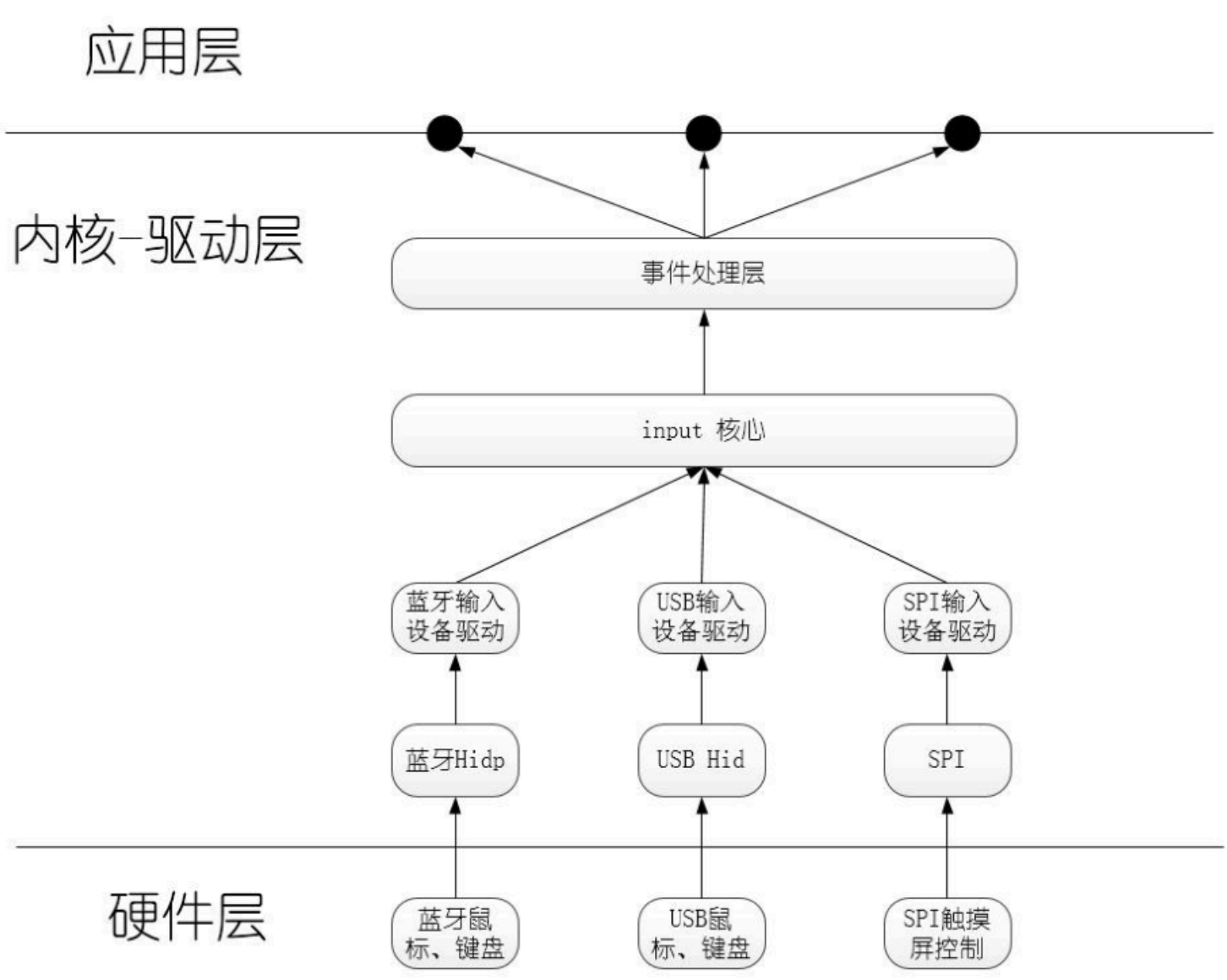
1. input 子系统简介

input 子系统就是管理输入的子系统，和 pinctrl 和 gpio 子系统一样，都是 Linux 内核针对某一类设备而创建的框架。 input 子系统处理输入事务，任何输入设备的驱动程序都可以通过 input子系统提供的接口注册到内核，利用子系统提供的功能来与用户空间交互。

输入设备一般包括键盘，鼠标，触摸屏等，在内核中都是以输入设备出现的。

input 子系统是分层结构的，总共分为三层： 硬件驱动层，子系统核心层，事件处理层。

- 硬件驱动层负责操作具体的硬件设备，这层的代码是针对具体的驱动程序的，需要驱动程序的作者来编写。
- 子系统核心层是链接其他两个层之间的纽带与桥梁，向下提供驱动层的接口，向上提供事件处理层的接口。
- 事件处理层负责与用户程序打交道，将硬件驱动层传来的事件报告给用户程序。



各层之间通信的基本单位就是事件，任何一个输入设备的动作都可以抽象成一种事件，如键盘的按下，触摸屏的按下，鼠标的移动等。

事件有三种属性：类型（type），编码(code)，值(value)，input 子系统支持的所有事件都定义在 input.h 中。

事件传送的方向是 : 硬件驱动层 --> 子系统核心 --> 事件处理层 --> 用户空间。

2. input 核心层

input 核心层文件是 input.c，路径:drivers/input/input.c。

- 设备注册

```
static int __init input_init(void)
{
    int err;

    err = class_register(&input_class);             // 注册设备类，名称为 input
    if (err) {
        pr_err("unable to register input_dev class\n");
        return err;
    }

    err = input_proc_init();
    if (err)
        goto fail1;

    err = register_chrdev_region(MKDEV(INPUT_MAJOR, 0),
                     INPUT_MAX_CHAR_DEVICES, "input");
    if (err) {
        pr_err("unable to register char major %d", INPUT_MAJOR);
        goto fail2;
    }

    return 0;

 fail2: input_proc_exit();
 fail1: class_unregister(&input_class);
    return err;
}

static void __exit input_exit(void)
{
    input_proc_exit();
    unregister_chrdev_region(MKDEV(INPUT_MAJOR, 0),
                 INPUT_MAX_CHAR_DEVICES);
    class_unregister(&input_class);
}

subsys_initcall(input_init);
module_exit(input_exit);
```

- subsys_initcall 模块初始化

`subsys_initcall` 和 `module_init` 都是 Linux 内核中用于初始化的宏定义，但它们的使用场景不同。

`subsys_initcall` 宏定义用于在内核启动时静态初始化子系统，它会在内核初始化阶段执行。

`module_init` 宏定义则用于在模块加载时动态初始化。

它们的区别主要在于初始化的时机。如果你想让一个函数在内核启动时被调用并初始化某个子系统，就应该使用 `subsys_initcall`；如果你想让一个函数在模块加载时被调用并初始化某个模块，就应该使用 `module_init`。

- class_register 注册一个类

注册了一个 input 类，在系统启动后会在/sys/class 目录下生成一个 input 类的子目录

- register_chrdev_region

注册了一个字符设备，所以 input 子系统本质上也是字符设备驱动，主设备号为INPUT_MAJOR。

INPUT_MAJOR 定义在include/uapi/linux/major.h

```
#define INPUT_MAJOR     13
```

所以 input 子系统的所有设备主设备号都为 13，在使用 input 子系统处理输入设备的时候就不需要去注册字符设备了，我们只需要向系统注册一个 input_device 即可。

input_dev 结构体

input_dev 结构体是 input 设备基本的设备结构，每个 input 驱动程序中都必须分配初始化这样一个结构。

结构体定义在 include/linux/input.h 文件中，定义如下：

```
struct input_dev {
    const char *name;
    const char *phys;
    const char *uniq;
    struct input_id id;

    unsigned long propbit[BITS_TO_LONGS(INPUT_PROP_CNT)];

    unsigned long evbit[BITS_TO_LONGS(EV_CNT)];
    unsigned long keybit[BITS_TO_LONGS(KEY_CNT)];
    unsigned long relbit[BITS_TO_LONGS(REL_CNT)];
    unsigned long absbit[BITS_TO_LONGS(ABS_CNT)];
    unsigned long mscbit[BITS_TO_LONGS(MSC_CNT)];
    unsigned long ledbit[BITS_TO_LONGS(LED_CNT)];
    unsigned long sndbit[BITS_TO_LONGS(SND_CNT)];
    unsigned long ffbit[BITS_TO_LONGS(FF_CNT)];
    unsigned long swbit[BITS_TO_LONGS(SW_CNT)];

    int (*open)(struct input_dev *dev);
    void (*close)(struct input_dev *dev);
    int (*flush)(struct input_dev *dev, struct file *file);
    int (*event)(struct input_dev *dev, unsigned int type, unsigned int code, int value);

    bool devres_managed;

    ktime_t timestamp[INPUT_CLK_MAX];
};
#define to_input_dev(d) container_of(d, struct input_dev, dev)
```

- evbit： 表示输入事件类型，可选的事件类型定义在 include/uapi/linux/input-event-codes.h 文件中

```
/*
 * Event types
 */

#define EV_SYN          0x00        /* 同步事件 */
#define EV_KEY          0x01        /* 按键事件 */
#define EV_REL          0x02        /* 相对坐标事件 */
#define EV_ABS          0x03        /* 绝对坐标事件 */
#define EV_MSC          0x04        /* 杂项(其他)事件 */
#define EV_SW           0x05        /* 开关事件 */
#define EV_LED          0x11        /* LED */
#define EV_SND          0x12        /* sound(声音) */
```

```
#define EV_REP          0x14        /* 重复事件 */
#define EV_FF           0x15        /* 压力事件 */
#define EV_PWR          0x16        /* 电源事件 */
#define EV_FF_STATUS    0x17        /* 压力状态事件 */
#define EV_MAX          0x1f
#define EV_CNT          (EV_MAX+1)
```

- keybit： 是按键事件使用的键值，Linux 内核定义了很多按键值，这些按键值定义在 include/uapi/linux/input.h 文件中

```
/*
 * Keys and buttons
 *
 * Most of the keys/buttons are modeled after USB HUT 1.12
 * (see http://www.usb.org/developers/hidpage).
 * Abbreviations in the comments:
 * AC - Application Control
 * AL - Application Launch Button
 * SC - System Control
 */

#define KEY_RESERVED        0
#define KEY_ESC         1
#define KEY_1           2
#define KEY_2           3
#define KEY_3           4
#define KEY_4           5
#define KEY_5           6
#define KEY_6           7
#define KEY_7           8
#define KEY_8           9
#define KEY_9           10
#define KEY_0           11
#define KEY_MINUS       12
#define KEY_EQUAL       13
#define KEY_BACKSPACE       14
#define KEY_TAB         15
#define KEY_Q           16
#define KEY_W           17
#define KEY_E           18
#define KEY_R           19
#define KEY_T           20
#define KEY_Y           21
#define KEY_U           22
#define KEY_I           23
#define KEY_O           24
#define KEY_P           25
#define KEY_LEFTBRACE       26
#define KEY_RIGHTBRACE      27
#define KEY_ENTER       28
#define KEY_LEFTCTRL        29
#define KEY_A           30
#define KEY_S           31
#define KEY_D           32
#define KEY_F           33
#define KEY_G           34
#define KEY_H           35
#define KEY_J           36
#define KEY_K           37
#define KEY_L           38
#define KEY_SEMICOLON       39
#define KEY_APOSTROPHE      40
#define KEY_GRAVE       41
#define KEY_LEFTSHIFT       42
#define KEY_BACKSLASH       43
#define KEY_Z           44
#define KEY_X           45
#define KEY_C           46
#define KEY_V           47
#define KEY_B           48
#define KEY_N           49
#define KEY_M           50
#define KEY_COMMA       51
#define KEY_DOT         52
#define KEY_SLASH       53
#define KEY_RIGHTSHIFT      54
#define KEY_KPASTERISK      55
#define KEY_LEFTALT     56
#define KEY_SPACE       57
#define KEY_CAPSLOCK        58
#define KEY_F1          59
#define KEY_F2          60
#define KEY_F3          61
#define KEY_F4          62
#define KEY_F5          63
#define KEY_F6          64
#define KEY_F7          65
#define KEY_F8          66
#define KEY_F9          67
#define KEY_F10         68
#define KEY_NUMLOCK     69
#define KEY_SCROLLLOCK      70
#define KEY_KP7         71
#define KEY_KP8         72
#define KEY_KP9         73
#define KEY_KPMINUS     74
#define KEY_KP4         75
#define KEY_KP5         76
#define KEY_KP6         77
#define KEY_KPPLUS      78
#define KEY_KP1         79
#define KEY_KP2         80
#define KEY_KP3         81
#define KEY_KP0         82
#define KEY_KPDOT       83

#define KEY_ZENKAKUHANKAKU  85
#define KEY_102ND       86
#define KEY_F11         87
#define KEY_F12         88
#define KEY_RO          89
#define KEY_KATAKANA        90
#define KEY_HIRAGANA        91
#define KEY_HENKAN      92
#define KEY_KATAKANAHIRAGANA    93
#define KEY_MUHENKAN        94
#define KEY_KPJPCOMMA       95
#define KEY_KPENTER     96
#define KEY_RIGHTCTRL       97
#define KEY_KPSLASH     98
#define KEY_SYSRQ       99
#define KEY_RIGHTALT        100
#define KEY_LINEFEED        101
#define KEY_HOME        102
#define KEY_UP          103
#define KEY_PAGEUP      104
#define KEY_LEFT        105
#define KEY_RIGHT       106
#define KEY_END         107
#define KEY_DOWN        108
#define KEY_PAGEDOWN        109
#define KEY_INSERT      110
#define KEY_DELETE      111
#define KEY_MACRO       112
#define KEY_MUTE        113
#define KEY_VOLUMEDOWN      114
#define KEY_VOLUMEUP        115
#define KEY_POWER       116 /* SC System Power Down */
#define KEY_KPEQUAL     117
#define KEY_KPPLUSMINUS     118
#define KEY_PAUSE       119
#define KEY_SCALE       120 /* AL Compiz Scale (Expose) */

#define KEY_KPCOMMA     121
#define KEY_HANGEUL     122
#define KEY_HANGUEL     KEY_HANGEUL
#define KEY_HANJA       123
#define KEY_YEN         124
#define KEY_LEFTMETA        125
#define KEY_RIGHTMETA       126
#define KEY_COMPOSE     127

#define KEY_STOP        128 /* AC Stop */
#define KEY_AGAIN       129
#define KEY_PROPS       130 /* AC Properties */
#define KEY_UNDO        131 /* AC Undo */
#define KEY_FRONT       132
#define KEY_COPY        133 /* AC Copy */
#define KEY_OPEN        134 /* AC Open */
```

```
#define KEY_PASTE       135 /* AC Paste */
#define KEY_FIND        136 /* AC Search */
#define KEY_CUT         137 /* AC Cut */
#define KEY_HELP        138 /* AL Integrated Help Center */
#define KEY_MENU        139 /* Menu (show menu) */
#define KEY_CALC        140 /* AL Calculator */
#define KEY_SETUP       141
#define KEY_SLEEP       142 /* SC System Sleep */
#define KEY_WAKEUP      143 /* System Wake Up */
#define KEY_FILE        144 /* AL Local Machine Browser */
#define KEY_SENDFILE        145
#define KEY_DELETEFILE      146
#define KEY_XFER        147
#define KEY_PROG1       148
#define KEY_PROG2       149
#define KEY_WWW         150 /* AL Internet Browser */
#define KEY_MSDOS       151
#define KEY_COFFEE      152 /* AL Terminal Lock/Screensaver */
#define KEY_SCREENLOCK      KEY_COFFEE
#define KEY_ROTATE_DISPLAY  153 /* Display orientation for e.g. tablets */
#define KEY_DIRECTION       KEY_ROTATE_DISPLAY
#define KEY_CYCLEWINDOWS    154
#define KEY_MAIL        155
#define KEY_BOOKMARKS       156 /* AC Bookmarks */
#define KEY_COMPUTER        157
#define KEY_BACK        158 /* AC Back */
#define KEY_FORWARD     159 /* AC Forward */
#define KEY_CLOSECD     160
#define KEY_EJECTCD     161
#define KEY_EJECTCLOSECD    162
#define KEY_NEXTSONG        163
#define KEY_PLAYPAUSE       164
#define KEY_PREVIOUSSONG    165
#define KEY_STOPCD      166
#define KEY_RECORD      167
#define KEY_REWIND      168
#define KEY_PHONE       169 /* Media Select Telephone */
#define KEY_ISO         170
#define KEY_CONFIG      171 /* AL Consumer Control Configuration */
#define KEY_HOMEPAGE        172 /* AC Home */
#define KEY_REFRESH     173 /* AC Refresh */
#define KEY_EXIT        174 /* AC Exit */
#define KEY_MOVE        175
#define KEY_EDIT        176
#define KEY_SCROLLUP        177
#define KEY_SCROLLDOWN      178
#define KEY_KPLEFTPAREN     179
#define KEY_KPRIGHTPAREN    180
#define KEY_NEW         181 /* AC New */
#define KEY_REDO        182 /* AC Redo/Repeat */
```

当我们编写 input 设备驱动时需要先创建一个 input_dev 结构体变量，但是不用我们手动创建，input子系统提供了两个函数用于创建和注销 input_dev 结构体变量。

```
struct input_dev *input_allocate_device(void) ; //申请 input_dev 结构体
void input_free_device(struct input_dev *dev) ; //注销 input_dev 结构体
```

上报输入事件

在 input 设备驱动中申请、注册完成 input_dev 结构体后，还不能正常使用 input 子系统，需要驱动获取到具体的输入值（事件），然后将输入事件上报给 Linux 内核。

比如，在按键产生后将按键值上报给 Linux 内核，Linux 内核获取到具体的按键值后，才会上报输入值（事件）。

不同的事件上报的函数不同，我们分别来看一下有哪些常用的 API 函数。

input_event 函数

- 函数原型

```
#include<linux/gpio.h>

void input_event(struct input_dev *dev,unsigned int type,unsigned int code,int value);
```

- 函数功能
  - 用于上报指定的事件以及对应的值。
- 函数参数
  - dev：需要上报的 input_dev。
  - type: 上报的事件类型，比如 EV_KEY。
  - code：事件码，也就是我们注册的按键值，比如 KEY_0、KEY_1 等等。
  - value：事件值，比如 1 表示按键按下，0 表示按键松开。
- 函数返回值
  - 无

input_event 结构体

Linux 内核使用 input_event 这个结构体来表示所有的输入事件，input_envent 结构体定义include/uapi/linux/input.h 文件中。

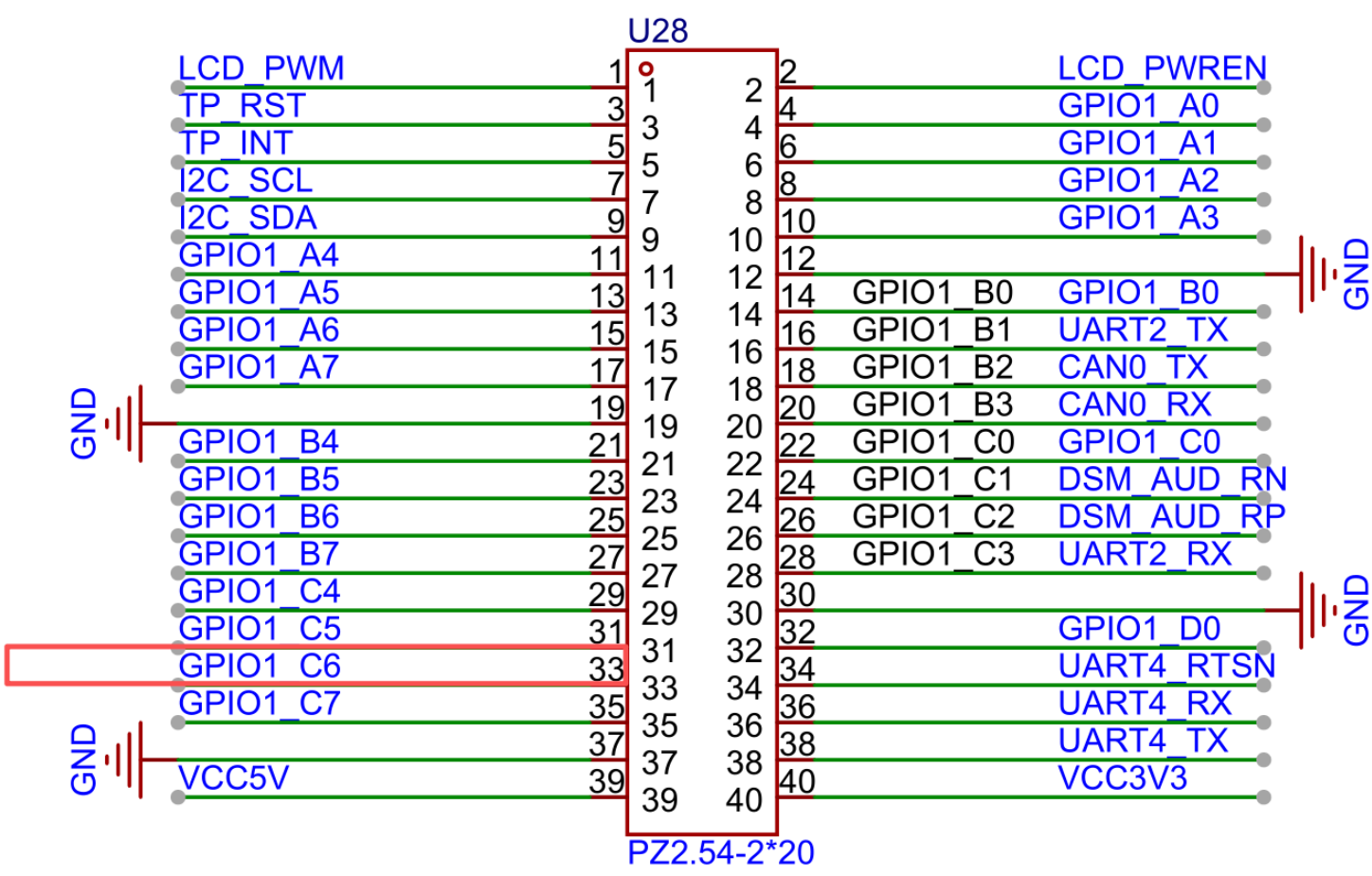
```
struct input_event {
    struct timeval time;
    __u16 type;
    __u16 code;
    __s32 value;
};
```

- time：时间，也就是此事件发生的时间。
- type：事件类型，比如 EV_KEY，表示此次事件为按键事件。
- code：事件码，比如在 EV_KEY 事件中 code 就表示具体的按键码，如：KEY_0、KEY_1 等等这些按键。
- value：表示按键有没有被按下，如果为 1 的话说明按键按下，如果为 0 的话说明按键没有被按下或者按键松开了。

input_envent 这个结构体非常重要，因为所有的输入设备最终都是按照 input_event 结构体呈现给用户的。

10.2 按键驱动

1. 按键硬件原理图分析



板子没有单独设计用户按键， 需要使用扩展IO来模拟按键的操作

2. 添加设备树节点

要使用 Linux 内核中自带的按键驱动程序很简单，只需要在设备树中添加指定的设备节点就可以了，具体可以参考 Documentation/devicetree/bindings/input/gpio-keys.txt 文件。

kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
gpio_keys: gpio-keys {
    status = "disabled";                    // ← 整个节点禁用!
    compatible = "gpio-keys";
    //pinctrl-0 = <&key_wake_up>;

    wake_up: wake-up {
        label = "Wake-up";
        //gpios = <&gpio0 RK_PC1 GPIO_ACTIVE_LOW>;  // ← GPIO 也被注释了
        linux,code = <KEY_WAKEUP>;          // 键值 143
        wakeup-source;                       // 可唤醒系统
    };
};
```

修改的设备树如下:

```
    gpio_keys: gpio-keys {
        status = "okay";                            // disabled → okay
        compatible = "gpio-keys";
        pinctrl-names = "default";
        pinctrl-0 = <&key_gpio>;                    // 新增 pinctrl 引用
        ok:ok {
            label = "OK Key";
            gpios = <&gpio1 RK_PC6 GPIO_ACTIVE_LOW>;
            linux,code = <KEY_ENTER>;
        };
    };

    gpio-keys {
        key_gpio: key-gpio {
            rockchip,pins = <1 RK_PC6 RK_FUNC_GPIO &pcfg_pull_up>;  // 上拉
        };
    };
```

- 删除wake_up 节点内容。

```
    wake_up: wake-up {
        label = "Wake-up";
        gpios = <&gpioX RK_PXX GPIO_ACTIVE_LOW>;
        linux,code = <KEY_WAKEUP>;
        wakeup-source;
    };
```

在 Linux 内核中也集成了按键的驱动程序，要使用的话，需要在内核中进行配置，配置如下:

```
Device Drivers  --->
      Input device support  --->
            [*]   Keyboards  --->
                 <*>   GPIO Buttons
```

保存退出后，在`.config`文件下会出现`CONFIG_KEYBOARD_GPIO=y`。

```
#
# Input Device Drivers
#
CONFIG_INPUT_KEYBOARD=y
# CONFIG_KEYBOARD_ADC is not set
# CONFIG_KEYBOARD_ADP5588 is not set
# CONFIG_KEYBOARD_ADP5589 is not set
CONFIG_KEYBOARD_ATKBD=y
# CONFIG_KEYBOARD_QT1050 is not set
# CONFIG_KEYBOARD_QT1070 is not set
# CONFIG_KEYBOARD_QT2160 is not set
# CONFIG_KEYBOARD_DLINK_DIR685 is not set
# CONFIG_KEYBOARD_LKKBD is not set
CONFIG_KEYBOARD_GPIO=y
# CONFIG_KEYBOARD_GPIO_POLLED is not set
# CONFIG_KEYBOARD_TCA6416 is not set
# CONFIG_KEYBOARD_TCA8418 is not set
# CONFIG_KEYBOARD_MATRIX is not set
# CONFIG_KEYBOARD_LM8323 is not set
# CONFIG_KEYBOARD_LM8333 is not set
# CONFIG_KEYBOARD_MAX7359 is not set
# CONFIG_KEYBOARD_MCS is not set
```

Linux 内核中自带的按键驱动程序路径`drivers/input/keyboard/gpio_keys.c`，按键驱动程序基于 platform框架，使用 input 子系统实现功能。

```
static const struct of_device_id gpio_keys_of_match[] = {
    { .compatible = "gpio-keys", },
    { },
};

static struct platform_driver gpio_keys_device_driver = {
    .probe      = gpio_keys_probe,
    .shutdown   = gpio_keys_shutdown,
    .driver     = {
        .name   = "gpio-keys",
        .pm = &gpio_keys_pm_ops,
        .of_match_table = gpio_keys_of_match,
        .dev_groups = gpio_keys_groups,
    }
};

static int __init gpio_keys_init(void)
{
    return platform_driver_register(&gpio_keys_device_driver);
}

static void __exit gpio_keys_exit(void)
{
    platform_driver_unregister(&gpio_keys_device_driver);
}

late_initcall(gpio_keys_init);
module_exit(gpio_keys_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Phil Blundell <pb@handhelds.org>");
MODULE_DESCRIPTION("Keyboard driver for GPIOs");
MODULE_ALIAS("platform:gpio-keys");
```

编译内核与设备树。

3. buildroot添加evtest工具

evtest是一个Linux命令行工具，用于测试和调试输入设备（如键盘、鼠标、游戏手柄等）的事件接口。它可以显示输入设备的详细信息，并实时显示设备发送的事件数据。

- buildroot添加evtest工具

输入一下命令:

```
./build.sh buildroot-config
```

配置菜单:

```
Target packages  --->
     Hardware handling  --->
            [*] evtest
```

编译文件系统，然后把内核、设备树、文件系统更新到开发板内，使用nandflash启动系统。

4. 按键驱动测试

- 查看系统的输入设备

```
evtest
```

- 测试指定的输入设备

```
evtest /dev/input/eventX
```

- 运行结果

```
root@rk3506-buildroot:/# evtest
No device specified, trying to scan all of /dev/input/event*
Available devices:
/dev/input/event0:  adc-keys
```

```
/dev/input/event1:  gpio-keys
/dev/input/event2:  bt-powerkey
Select the device event number [0-2]: 1
Input driver version is 1.0.1
Input device ID: bus 0x19 vendor 0x1 product 0x1 version 0x100
Input device name: "gpio-keys"
Supported events:
  Event type 0 (EV_SYN)
  Event type 1 (EV_KEY)
    Event code 28 (KEY_ENTER)
Properties:
Testing ... (interrupt to exit)
Event: time 946705454.928301, type 1 (EV_KEY), code 28 (KEY_ENTER), value 1
Event: time 946705454.928301, -------------- SYN_REPORT ------------
Event: time 946705456.272257, type 1 (EV_KEY), code 28 (KEY_ENTER), value 0
Event: time 946705456.272257, -------------- SYN_REPORT ------------
Event: time 946705456.339991, type 1 (EV_KEY), code 28 (KEY_ENTER), value 1
Event: time 946705456.339991, -------------- SYN_REPORT ------------
Event: time 946705456.718654, type 1 (EV_KEY), code 28 (KEY_ENTER), value 0
Event: time 946705456.718654, -------------- SYN_REPORT ------------
^C
root@rk3506-buildroot:/#
```

第11章 Linux IIO(ADC)子系统驱动

工业场合中有大量的模拟量和数字量之间的转换，也就是常说的 ADC 和 DAC，生活中常见的加速度计、光传感器、陀螺仪、气压计、磁力计等，这些传感器本质上都是 ADC。

这些传感器的内部通常存在一个 ADC，然后对外提供 IIC 或者 SPI 接口， SoC可以通过 IIC 或者 SPI 接口来获取到传感器内部的 ADC 数值，从而得到想要测量的结果。

Linux 内核为了管理这些日益增多的 ADC 类传感器，特地推出了 IIO 子系统，更加快捷方便的开发这类传感器的驱动。

11.1 IIO 子系统

1. IIO子系统简介

IIO（Industrial I/O）子系统是 Linux 内核中用于处理工业输入输出设备的框架。它提供了一个通用的 API 和数据结构，用于管理和控制各种类型的模数转换器（ADC）、数字模数转换器（DAC）、压力传感器、温度传感器等工业设备。

IIO 驱动程序通常由以下几个组件构成：

1. 平台驱动程序（Platform Driver）：负责与硬件平台进行交互，初始化 IIO 设备，并注册 I/O 接口。
2. IIO 框架（Industrial I/O Framework）：提供了一组通用的 API 和数据结构，用于管理和控制 IIO 设备。IIO 驱动程序需要实现 IIO 框架定义的接口函数，以便与其他 IIO 设备和应用程序进行交互。
3. 设备树绑定（Device Tree Binding）：在设备树中描述 IIO 设备的硬件配置和属性信息。
4. 用户空间接口：IIO 驱动程序通常通过 sysfs 或者 IIO 设备节点（例如 `/dev/iio:deviceX`）提供用户空间访问接口。用户可以通过读取和写入这些接口，实现对 IIO 设备的配置和数据采集。

2. IIO框架接口

iio_dev 结构体

IIO 子系统使用结构体 iio_dev 来描述一个具体 IIO 设备，此设备结构体定义在 include/linux/iio/iio.h 文件中，其主要内容如下：

```
struct iio_dev {
    int             id;
    struct module           *driver_module;

    int             modes;
    int             currentmode;
    struct device           dev;

    struct iio_event_interface  *event_interface;

    struct iio_buffer       *buffer;
    struct list_head        buffer_list;
    int             scan_bytes;
    struct mutex            mlock;

    const unsigned long     *available_scan_masks;
    unsigned            masklength;
    const unsigned long     *active_scan_mask;
    bool                scan_timestamp;
    unsigned            scan_index_timestamp;
    struct iio_trigger      *trig;
    bool                trig_readonly;
    struct iio_poll_func        *pollfunc;
    struct iio_poll_func        *pollfunc_event;

    struct iio_chan_spec const  *channels;
    int             num_channels;

    struct list_head        channel_attr_list;
    struct attribute_group      chan_attr_group;
    const char          *name;
    const struct iio_info       *info;
    clockid_t           clock_id;
    struct mutex            info_exist_lock;
    const struct iio_buffer_setup_ops   *setup_ops;
    struct cdev         chrdev;
#define IIO_MAX_GROUPS 6
    const struct attribute_group    *groups[IIO_MAX_GROUPS + 1];
    int             groupcounter;

    unsigned long           flags;
#if defined(CONFIG_DEBUG_FS)
    struct dentry           *debugfs_dentry;
    unsigned            cached_reg_addr;
#endif
};
```

- modes 为设备支持的模式，可选择的模式如下：
  - INDIO_DIRECT_MODE ：提供 sysfs 接口
  - INDIO_BUFFER_TRIGGERED ：支持硬件缓冲触发
  - INDIO_BUFFER_SOFTWARE ：支持软件缓冲触发
  - INDIO_BUFFER_HARDWARE ：支持硬件缓冲区
  - INDIO_EVENT_TRIGGERED ：支持事件触发
  - INDIO_HARDWARE_TRIGGERED ：支持硬件触发
  - INDIO_ALL_BUFFER_MODES ：支持所有缓冲模式
  - INDIO_ALL_TRIGGERED_MODES：支持所有触发模式
- currentmode 为当前模式。
- buffer 为缓冲区。
- channels 为 IIO 设备通道，为 iio_chan_spec 结构体类型，下文会详细讲解 IIO通道。
- num_channels 为 IIO 设备的通道数。
- name 为 IIO 设备名字。
- info 为 iio_info 结构体类型，这个结构体里面有很多函数，需要驱动开发人员编写，非常重要！从用户空间读取 IIO 设备内部数据，最终调用的就是 iio_info 里面的函数。

11.2 RK3506 ADC 介绍

在瑞芯微 **RK3506** 芯片中，通常集成了以下几类模数转换器（ADC），你可以根据项目需求选用：

1. TSADC（温度传感器 ADC）

- **用途**：专门用于芯片内部结温（CPU、GPU、NPU 等）监测。
- **通道数**：1 路，分别对应 CPU 传感器（通道 0）
- **测量范围**：-40°C ~ 125°C
- **ADC 精度**：10-bit
- **采样率**：可配置，通常 100Hz~1KHz
- **高温关断（TSHUT）**：硬件自动响应，可配置关断温度、模式（CRU 复位或 GPIO 输出）和极性
- **中断**：支持高温报警中断，用于软件及时降频或关机

2. SAR-ADC（逐次逼近型通用 ADC）

- **用途**：面向外部模拟信号采集，如电池电压检测、按键分压（ADC 按键）、通用传感器输入等。
- **基本规格**：4 通道、单端输入、10 位分辨率，参考电压为电源和地（无需外部参考），输入电压范围典型值为 0V ~ 1.8V。
- **工作模式**：
  - **单次模式**：每次软件访问触发一次转换。
  - **连续模式**：控制器自动循环采样所有通道，直到软件主动停止。

- **中断功能**：可设置高低阈值，并支持当输入值**高于高阈值**、**低于低阈值**或**介于高低阈值之间**时触发中断。

3. 音频 ADC（集成音频编解码器）

- **用途**：如果你使用的 RK3506 型号内部集成了音频 Codec（如用于麦克风输入的 AMIC/DMIC），则会有音频专用 ADC。
- **类型**：一般为 Σ-Δ（Sigma-Delta）ADC，高精度、低速、专用于语音或音频信号。
- **数量**：取决于内部 Codec 规格（例如立体声 ADC 包含左/右两个通道）。

11.3 TSADC 设备树

由于我们没有外接SAR-ADC设备，因此我们无法使用这个adc，我们就使用TSADC进行adc的实验。

1. TSADC介绍

- **用途**：专门用于芯片内部结温（CPU、GPU、NPU 等）监测。
- **通道数**：1路，分别对应 CPU 传感器（通道 0）
- 重要前提：RK3506 只有 1 路温度传感器
- **测量范围**：-40°C ~ 125°C
- **ADC 精度**：10-bit
- **采样率**：可配置，通常 100Hz~1KHz
- **高温关断（TSHUT）**：硬件自动响应，可配置关断温度、模式（CRU 复位或 GPIO 输出）和极性
- **中断**：支持高温报警中断，用于软件及时降频或关机

2. TSADC设备树分析

TSADC（Temperature Sensor ADC）的设备树由 **4 层** 组成，形成 `rk3502.dtsi` → `rk3506.dtsi` → `ido-evb3506-v1a-nand.dtsi` → `ido-evb3506-v1a-flash-mipi-720x720.dts` 的引用链。

- tsadc节点 `kernel/arch/arm/boot/dts/rk3502.dtsi:1244-1262`

```
tsadc: tsadc@ff650000 {
    compatible = "rockchip,rk3506-tsadc";
    reg = <0xff650000 0x400>;              // 寄存器基址 + 1KB 空间
    interrupts = <GIC_SPI 58 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru CLK_TSADC>, <&cru PCLK_TSADC>, <&cru CLK_TSADC_TSEN>;
    clock-names = "tsadc", "apb_pclk", "tsen";   // 3 路时钟
    assigned-clocks = <&cru CLK_TSADC>, <&cru CLK_TSADC_TSEN>;
    assigned-clock-rates = <1000000>, <12000000>; // tsadc 1MHz, tsen 12MHz
    resets = <&cru SRST_TSADC>, <&cru SRST_P_TSADC>;
    reset-names = "tsadc", "tsadc-apb";          // 2 路复位
    #thermal-sensor-cells = <1>;                  // 1 个 sensor
    rockchip,grf = <&grf>;                        // GRF 寄存器引用
    rockchip,hw-tshut-temp = <120000>;            // 硬件关断温度 120°C
    rockchip,hw-tshut-mode = <0>;                 // 关断模式 0=CRU
    rockchip,hw-tshut-polarity = <0>;             // 关断极性 0=低有效
    nvmem-cells = <&cpu_tsadc_trim_l>, <&cpu_tsadc_trim_h>;
    nvmem-cell-names = "trim_l", "trim_h";        // 校准值
    status = "disabled";
};
```

- 热管理区引用 `kernel/arch/arm/boot/dts/rk3502.dtsi:483-498`

```
thermal_zones: thermal-zones {
    soc_thermal: soc-thermal {
        polling-delay-passive = <20>;     // 被动降温轮询间隔 20ms
        polling-delay = <1000>;           // 普通轮询间隔 1s
        thermal-sensors = <&tsadc 0>;     // 引用 tsadc 通道 0
        trips {
            soc_crit: soc-crit {
                temperature = <115000>;   // 115°C 临界温度
                hysteresis = <2000>;      // 2°C 回滞
                type = "critical";        // 临界 trip: 触发系统关机
            };
        };
    };
};
```

- eFuse 校准数据 `kernel/arch/arm/boot/dts/rk3502.dtsi:1192-1198`

```
cpu_tsadc_trim_l: cpu-tsadc-trim-l@20 {
    reg = <0x20 0x1>;             // OTP 偏移 0x20, 1 字节
};
cpu_tsadc_trim_h: cpu-tsadc-trim-h@21 {
    reg = <0x21 0x1>;             // OTP 偏移 0x21, 1 字节
    bits = <0 2>;                 // 仅低 2 bit 有效
};
```

- Pinctrl (管脚复用) `kernel/arch/arm/boot/dts/rk3506-pinctrl-rmio.dtsi`

RK3506 的 TSADC 在硬件层面有一个 **tshut（Thermal Shut-down）输出信号。**当芯片温度超过硬件阈值时，TSADC 控制器可以自动：

模式 0（CRU）：直接给芯片内部的 CRU（时钟复位单元）发信号，触发整个 SoC 复位

模式 1（OTP/GPIO）：通过外部 GPIO 管脚输出电平信号（如拉低某个 GPIO），**通知 PMIC 切断系统电源**

这个 GPIO 管脚不是固定死的，可通过 **RMIO（Reconfigurable Multiplex I/O）系统** 灵活分配到不同物理管脚上。

TSADC 的 `tshut` 硬件保护信号可复用到 GPIO0/GPIO1 的 **大多数管脚**（RMIO 多路复用功能号 92）

```
rm_io21_wdt_tsadc_shut: rm-io21-wdt-tsadc-shut {
    rockchip,pins = <0 RK_PC5 92 &pcfg_pull_none>;  // GPIO0_PC5, 功能 92
};
```

当前 evb3506 板卡缺少 pinctrl 配置，且默认使用 **CRU 模式**：过热时 TSADC 只会通知 CRU 做芯片复位，**不会通过 GPIO 信号控制外部 PMIC。**

- 板级使能 `kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi:581-583`

```
&tsadc {
    status = "okay";
};
```

11.4 TSADC驱动

1. 驱动源码

驱动文件: `kernel/drivers/thermal/rockchip_thermal.c`

```
// SPDX-License-Identifier: GPL-2.0-only
/*
 * Copyright (c) 2014-2016, Fuzhou Rockchip Electronics Co., Ltd
 * Caesar Wang <wxt@rock-chips.com>
 */

#include <linux/clk.h>
#include <linux/delay.h>
#include <linux/interrupt.h>
#include <linux/io.h>
#include <linux/module.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_irq.h>
#include <linux/panic_notifier.h>
#include <linux/platform_device.h>
#include <linux/regmap.h>
#include <linux/reset.h>
#include <linux/rockchip/cpu.h>
#include <linux/thermal.h>
#include <linux/mfd/syscon.h>
#include <linux/pinctrl/consumer.h>
#include <linux/nvmem-consumer.h>

/*
 * If the temperature over a period of time High,
 * the resulting TSHUT gave CRU module,let it reset the entire chip,
 * or via GPIO give PMIC.
 */
enum tshut_mode {
    TSHUT_MODE_CRU = 0,
    TSHUT_MODE_OTP,
};

/*
 * The system Temperature Sensors tshut(tshut) polarity
 * the bit 8 is tshut polarity.
 * 0: low active, 1: high active
 */
enum tshut_polarity {
    TSHUT_LOW_ACTIVE = 0,
    TSHUT_HIGH_ACTIVE,
};
```

```
/*
 * The system has two Temperature Sensors.
 * sensor0 is for CPU, and sensor1 is for GPU.
 */
enum sensor_id {
    SENSOR_CPU = 0,
    SENSOR_GPU,
};

/*
 * The conversion table has the adc value and temperature.
 * ADC_DECREMENT: the adc value is of diminishing.(e.g. rk3288_code_table)
 * ADC_INCREMENT: the adc value is incremental.(e.g. rk3368_code_table)
 */
enum adc_sort_mode {
    ADC_DECREMENT = 0,
    ADC_INCREMENT,
};

#include "thermal_hwmon.h"

/**
 * The max sensors is seven in rockchip SoCs.
 */
#define SOC_MAX_SENSORS 7

/**
 * struct chip_tsadc_table - hold information about chip-specific differences
 * @id: conversion table
 * @length: size of conversion table
 * @data_mask: mask to apply on data inputs
 * @kNum: linear parameter k
 * @bNum: linear parameter b
 * @mode: sort mode of this adc variant (incrementing or decrementing)
 */
struct chip_tsadc_table {
    const struct tsadc_table *id;
    unsigned int length;
    u32 data_mask;
    /* Tsadc is linear, using linear parameters */
    int kNum;
    int bNum;
    enum adc_sort_mode mode;
};

/**
 * struct rockchip_tsadc_chip - hold the private data of tsadc chip
 * @chn_id: array of sensor ids of chip corresponding to the channel
 * @chn_num: the channel number of tsadc chip
 * @conversion_time: the conversion time of tsadc
 * @trim_slope: use to conversion trim code to trim temp
 * @tshut_temp: the hardware-controlled shutdown temperature value
 * @tshut_mode: the hardware-controlled shutdown mode (0:CRU 1:GPIO)
 * @tshut_polarity: the hardware-controlled active polarity (0:LOW 1:HIGH)
 * @initialize: SoC special initialize tsadc controller method
 * @irq_ack: clear the interrupt
 * @control: enable/disable method for the tsadc controller
 * @get_temp: get the temperature
 * @set_alarm_temp: set the high temperature interrupt
 * @set_tshut_temp: set the hardware-controlled shutdown temperature
 * @set_tshut_mode: set the hardware-controlled shutdown mode
 * @get_trim_code: get the trim code by otp value
 * @set_clk_rate: set clock rate
 * @table: the chip-specific conversion table
 */
struct rockchip_tsadc_chip {
    /* The sensor id of chip correspond to the ADC channel */
    int chn_id[SOC_MAX_SENSORS];
    int chn_num;

    /* The sensor electrical characteristics */
    int conversion_time;

    /* Use to conversion trim code to trim temp */
    int trim_slope;

    /* The hardware-controlled tshut property */
    int tshut_temp;
    enum tshut_mode tshut_mode;
    enum tshut_polarity tshut_polarity;

    /* Chip-wide methods */
    void (*initialize)(struct regmap *grf,
               void __iomem *reg, enum tshut_polarity p);
    void (*suspend)(struct regmap *grf, void __iomem *reg);
    void (*irq_ack)(void __iomem *reg);
    void (*control)(void __iomem *reg, bool on);

    /* Per-sensor methods */
    int (*get_temp)(const struct chip_tsadc_table *table,
            int chn, void __iomem *reg, int *temp);
    int (*set_alarm_temp)(const struct chip_tsadc_table *table,
                  int chn, void __iomem *reg, int temp);
    int (*set_tshut_temp)(const struct chip_tsadc_table *table,
                  int chn, void __iomem *reg, int temp);
    void (*set_tshut_mode)(struct regmap *grf, int chn,
                   void __iomem *reg, enum tshut_mode m);
    int (*get_trim_code)(const struct chip_tsadc_table *table,
                 int code, int trim_base, int trim_base_frac);
    int (*set_clk_rate)(struct platform_device *pdev);

    /* Per-table methods */
    struct chip_tsadc_table table;
};


static const struct of_device_id of_rockchip_thermal_match[] = {
#ifdef CONFIG_CPU_PX30
    {   .compatible = "rockchip,px30-tsadc",
        .data = (void *)&px30_tsadc_data,
    },
    {   .compatible = "rockchip,px30s-tsadc",
        .data = (void *)&px30s_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RV1106
    {
        .compatible = "rockchip,rv1106-tsadc",
        .data = (void *)&rv1106_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RV1108
    {
        .compatible = "rockchip,rv1108-tsadc",
        .data = (void *)&rv1108_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RV1126
    {
        .compatible = "rockchip,rv1126-tsadc",
        .data = (void *)&rv1126_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK1808
    {
        .compatible = "rockchip,rk1808-tsadc",
        .data = (void *)&rk1808_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK322X
    {
        .compatible = "rockchip,rk3228-tsadc",
        .data = (void *)&rk3228_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3288
    {
        .compatible = "rockchip,rk3288-tsadc",
        .data = (void *)&rk3288_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3308
    {
        .compatible = "rockchip,rk3308-tsadc",
        .data = (void *)&rk3308_tsadc_data,
    },
    {
        .compatible = "rockchip,rk3308bs-tsadc",
```

```
        .data = (void *)&rk3308bs_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3328
    {
        .compatible = "rockchip,rk3328-tsadc",
        .data = (void *)&rk3328_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3366
    {
        .compatible = "rockchip,rk3366-tsadc",
        .data = (void *)&rk3366_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3368
    {
        .compatible = "rockchip,rk3368-tsadc",
        .data = (void *)&rk3368_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3399
    {
        .compatible = "rockchip,rk3399-tsadc",
        .data = (void *)&rk3399_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3506
    {
        .compatible = "rockchip,rk3506-tsadc",
        .data = (void *)&rk3506_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3528
    {
        .compatible = "rockchip,rk3528-tsadc",
        .data = (void *)&rk3528_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3562
    {
        .compatible = "rockchip,rk3562-tsadc",
        .data = (void *)&rk3562_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3568
    {
        .compatible = "rockchip,rk3568-tsadc",
        .data = (void *)&rk3568_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3576
    {
        .compatible = "rockchip,rk3576-tsadc",
        .data = (void *)&rk3576_tsadc_data,
    },
#endif
#ifdef CONFIG_CPU_RK3588
    {
        .compatible = "rockchip,rk3588-tsadc",
        .data = (void *)&rk3588_tsadc_data,
    },
#endif
    { /* end */ },
};

static struct platform_driver rockchip_thermal_driver = {
    .driver = {
        .name = "rockchip-thermal",
        .pm = &rockchip_thermal_pm_ops,
        .of_match_table = of_rockchip_thermal_match,
    },
    .probe = rockchip_thermal_probe,
    .remove = rockchip_thermal_remove,
    .shutdown = rockchip_thermal_shutdown,
};

static int __init rockchip_thermal_driver_init(void)
{
    return platform_driver_register(&rockchip_thermal_driver);
}
rootfs_initcall(rockchip_thermal_driver_init);

static void __exit rockchip_thermal_driver_exit(void)
{
    platform_driver_unregister(&rockchip_thermal_driver);
}
module_exit(rockchip_thermal_driver_exit);

MODULE_DESCRIPTION("ROCKCHIP THERMAL Driver");
MODULE_AUTHOR("Rockchip, Inc.");
MODULE_LICENSE("GPL v2");
MODULE_ALIAS("platform:rockchip-thermal");
```

2. 内核配置ADC

使能内核里自带的 TSADC 驱动，打开 Linux 内核配置界面，配置ADC的驱动路径如下：

配置内核：

```
./build.sh kernel-config
```

内核配置选项

```
Device Drivers
    [*] Thermal drivers  --->
        <*>   Rockchip thermal driver
```

编译内核与设备树，更新内核与设备树到开发板，并用nandflash启动系统。

3. ADC驱动测试

重要前提：RK3506 只有 1 路温度传感器，RK3506硬件只有 1 个 ADC 通道（CPU 传感器），没有独立的 GPU 温度传感器。

```
# 查看 thermal zone 名称
cat /sys/class/thermal/thermal_zone0/type
# 输出: soc-thermal

# 读取 SoC 温度（毫摄氏度，除以 1000 即 °C）
cat /sys/class/thermal/thermal_zone0/temp
# 输出示例: 45000  → 表示 45.0°C

# 查看可用 trip 点
cat /sys/class/thermal/thermal_zone0/trip_point_0_temp
# 输出示例: 115000 → 115°C（临界关机温度）

cat /sys/class/thermal/thermal_zone0/trip_point_0_type
# 输出: critical

# 查看温度变化统计
cat /sys/class/thermal/thermal_zone0/policy
```

- 运行结果

```
root@rk3506-buildroot:/# ls
bin               lib32             rockchip-test     udisk
busybox.fragment  linuxrc           root              userdata
data              media             run               usr
dev               mnt               sbin              var
etc               oem               sdcard
info              opt               sys
lib               proc              tmp
root@rk3506-buildroot:/# ls /sys/class/thermal/thermal_zone0/type
/sys/class/thermal/thermal_zone0/type
root@rk3506-buildroot:/# cat /sys/class/thermal/thermal_zone0/type
soc-thermal
root@rk3506-buildroot:/# cat /sys/class/thermal/thermal_zone0/temp
41719
root@rk3506-buildroot:/# cat /sys/class/thermal/thermal_zone0/trip_point_0_temp
115000
root@rk3506-buildroot:/# cat /sys/class/thermal/thermal_zone0/trip_point_0_type
critical
root@rk3506-buildroot:/# cat /sys/class/thermal/thermal_zone0/policy
```

```
step_wise
root@rk3506-buildroot:/#
```

- `/sys/class/thermal/thermal_zone0/trip_point_0_type` 解释

```
# trip_point_0_type = critical
# 一句话总结
# 当 SoC 温度达到 `trip_point_0_temp`（115°C）时，系统将立即触发紧急关机/复位，不做任何优雅退出。
```

- Linux Thermal 框架的 Trip 类型

Linux 定义了以下几种 trip point 类型：

类型	枚举值	触发动作
active	`THERMAL_TRIP_ACTIVE`	主动降温：启动风扇、降低频率等 **冷却设备**
passive	`THERMAL_TRIP_PASSIVE`	被动降温：CPU 降频（cpufreq cooling）
hot	`THERMAL_TRIP_HOT`	热警告：通知用户空间，不强制动作
critical	`THERMAL_TRIP_CRITICAL`	**临界温度：立即关机/复位，无任何缓冲**

- `sys/class/thermal/thermal_zone0/policy` 解释

`policy` 文件用于**查看和切换当前 thermal zone 使用的调速策略（thermal governor）**。它决定当温度触发 trip point 时，系统采取什么策略来降温。

```
# 读取当前策略
cat /sys/class/thermal/thermal_zone0/policy
# 输出: step_wise

# 查看所有可用策略
cat /sys/class/thermal/thermal_zone0/available_policies
# 输出: step_wise user_space

# 切换策略
echo user_space > /sys/class/thermal/thermal_zone0/policy
```

- 可用的 thermal governor

Governor	描述	行为
`step_wise` (默认)	阶梯式降温	温度每超一个 trip，逐步提高冷却设备等级（如逐级降频）
`user_space`	用户空间通知	**不做任何自动降温**，只通过 uevent 通知用户态程序处理
`fair_share`	公平分配	在多个冷却设备间按权重公平分配降温量
`bang_bang`	双态开关	低于某阈值关冷却，高于某阈值立即开到最大
`power_allocator`	功率分配器	基于 PID 控制的智能功耗分配（IPA）

第12章 Linux IIC子系统驱动

12.1 Linux I2C子系统

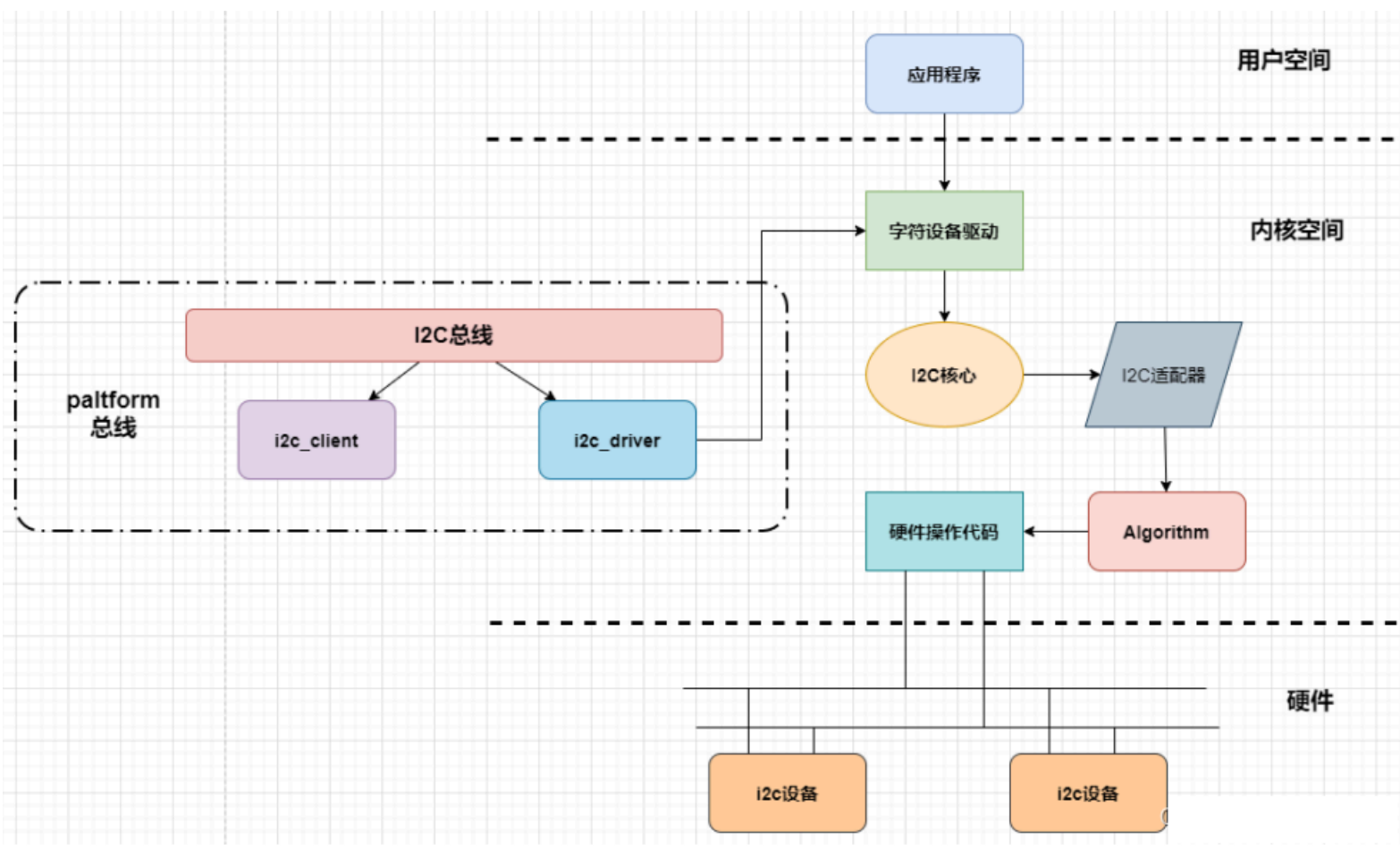
1. I2C 子系统介绍

在 Linux 中，I2C（Inter-Integrated Circuit）子系统是用于管理 I2C 总线的子系统。I2C 总线是一种串行通信协议，可以连接多个设备，并通过两根总线线路来传输数据。

在 Linux 内核中 I2C 的体系结构分为 3 个部分：

- I2C 总线驱动： I2C 总线驱动是对 I2C 硬件的软件化抽象，I2C 总线驱动就是 SOC 的 I2C 控制器驱动，也叫做 I2C 适配器驱动。
- I2C 设备驱动： I2C 设备驱动是对 I2C 硬件设备端的实现， 设备一般挂载在I2C适配器上， 通过 I2C 适配器与CPU交换数据。
- I2C 核心： I2C 核心提供了 I2C 总线驱动和设备驱动的注册、 注销方法。

2. I2C驱动架构图



i2c 总线包括 i2c 设备 (i2c_client) 和 i2c 驱动 (i2c_driver)，当我们向 linux 中注册设备或驱动的时候，按照 i2c 总线匹配规则进行配对，配对成功，则可以通过 i2c_driver 中 `probe` 函数创建具体的设备文件，通过设备文件可以读写设备。

在现代 linux 中，i2c 设备不再需要手动创建，而是使用设备树机制，设备树节点是与 paltform 总线相配合使用的。

3. I2C总线驱动

I2C 总线和 platform 总线类似， 区别在于platform 总线是虚拟的一条总线， 而 I2C 总线是实际存在的。 对于使用 I2C 通信的设备， 在驱动中直接使用 I2C 总线即可。

I2C 总线驱动的重点是 I2C 适配器驱动， 主要涉及到两个结构体： i2c_adapter 和 i2c_algorithm。

i2c_adapter结构体

i2c 适配器对应一个 i2c 控制器，是用于标识物理 i2c 总线以及访问它所需的访问算法的结构。在 Linux 内核中用 i2c_adapter 结构体来表示 I2C 控制器。

i2c_adapter 结构体定义在 include/linux/i2c.h 文件中。

```
struct i2c_adapter {
    struct module *owner;
    unsigned int class; /* classes to allow probing for */
    const struct i2c_algorithm *algo; /* 总线访问算法 */
    void *algo_data;

    /* data fields that are valid for all devices */
    struct rt_mutex bus_lock;

    int timeout; /* in jiffies */
    int retries;
    struct device dev; /* the adapter device */

    int nr;
    char name[48];
    struct completion dev_released;

    struct mutex userspace_clients_lock;
    struct list_head userspace_clients;

    struct i2c_bus_recovery_info *bus_recovery_info;
    const struct i2c_adapter_quirks *quirks;
};
```

i2c_algorithm结构体

i2c_algorithm 结构体用于指定访问总线的算法，在这里就是用于指定外部访问 i2c 总线的接口，体现到代码就是一些接口函数。

从下面代码不难看出 i2c_algorithm 结构体实际提供了一些函数指针，这些函数就是外部访问 i2c 总线的接口。

i2c_algorithm 是 I2C 适配器与 IIC 设备进行通信的方法，i2c_algorithm 结构体定义在 include/linux/i2c.h 文件中。

```
struct i2c_algorithm
{
    ......
    int (*master_xfer)(struct i2c_adapter *adap,struct i2c_msg *msgs,int num);
    int (*smbus_xfer) (struct i2c_adapter *adap, u16 addr,
    unsigned short flags, char read_write,
    u8 command, int size, union i2c_smbus_data *data);

    /* To determine what the adapter supports */
    u32 (*functionality) (struct i2c_adapter *);
    ......
};
```

I2C总线驱动工作介绍

I2C 总线驱动（I2C 适配器驱动）的主要工作就是初始化 i2c_adapter 结构体变量，然后设置 i2c_algorithm 中的 master_xfer 函数。最后通过 i2c_add_numbered_adapter或 i2c_add_adapter 这两个函数向系统注册i2c_adapter。

```
int i2c_add_adapter(struct i2c_adapter *adapter)
int i2c_add_numbered_adapter(struct i2c_adapter *adap)
```

一般 SOC 的 I2C 总线驱动都是由半导体厂商编写的，所以大多数只要专注于 I2C 设备驱动即可。

4. I2C设备驱动

在I2C设备驱动中主要有两个重要的结构体： i2c_client 和 i2c_driver。 i2c_client 是描述设备信息的，i2c_driver 描述驱动内容。

i2c_client结构体

i2c_client表示 i2c 从设备，当驱动和设备匹配成功后，每检测到一个 I2C 设备就会给这个 I2C 设备分配一个i2c_client，这个i2c_client存储着这个设备的所有信息，如芯片地址。

```
struct i2c_client {
    unsigned short flags; /* 标志 */
    unsigned short addr; /* 芯片地址， 7 位，存在低 7 位*/
    ......
    char name[I2C_NAME_SIZE]; /* 名字 */
    struct i2c_adapter *adapter; /* 对应的 I2C 适配器 */

    struct device dev; /* 设备结构体 */

    int irq; /* 中断 */

    struct list_head detected;
    ......
};
```

i2c_driver结构体

i2c 设备驱动程序，当 I2C 设备和驱动匹配成功以后 probe 函数就会执行device_driver 驱动结构体：

```
struct i2c_driver {
    unsigned int class;

    /* Notifies the driver that a new bus has appeared. You should
    * avoid using this, it will be removed in a near future.
    */
    int (*attach_adapter)(struct i2c_adapter *) __deprecated;

    /* Standard driver model interfaces */
    /*i2c 设备和 i2c 驱动匹配后，回调该函数指针。*/
    int (*probe)(struct i2c_client *, const struct i2c_device_id *);
    int (*remove)(struct i2c_client *);

    /* driver model interfaces that don't relate to enumeration */
    void (*shutdown)(struct i2c_client *);

    /* Alert callback, for example for the SMBus alert protocol.
    * The format and meaning of the data value depends on the
    * protocol.For the SMBus alert protocol, there is a single bit
    * of data passed as the alert response's low bit ("eventflag"). */
    void (*alert)(struct i2c_client *, unsigned int data);
    /* a ioctl like command that can be used to perform specific
    * functions with the device.
    */
    int (*command)(struct i2c_client *client, unsigned int cmd,void *arg);

    struct device_driver driver;
    const struct i2c_device_id *id_table;

    /* Device detection callback for automatic device creation */
    int (*detect)(struct i2c_client *, struct i2c_board_info *);
    const unsigned short *address_list;
    struct list_head clients;
};
```

5. I2C 核心

在 I2C 核心层完成设备和驱动的匹配过程。设备和驱动的匹配过程是在 I2C 总线完成的， I2C 总线的数据结构为i2c_bus_type：

```
struct bus_type i2c_bus_type = {
    .name = "i2c",
    .match = i2c_device_match,
    .probe = i2c_device_probe,
    .remove = i2c_device_remove,
    .shutdown = i2c_device_shutdown,
};
```

match 成员变量是 I2C 总线上设备和驱动匹配函数，即i2c_device_match函数。

```
static int i2c_device_match(struct device *dev, struct device_driver *drv)
{
    struct i2c_client *client = i2c_verify_client(dev);
    struct i2c_driver *driver;

    if (!client)
    return 0;

    /* Attempt an OF style match */
    if (of_driver_match_device(dev, drv))
    return 1;

    /* Then ACPI style match */
    if (acpi_driver_match_device(dev, drv))
    return 1;

    driver = to_i2c_driver(drv);
    /* match on an id table if there is one */
    if (driver->id_table)
    return i2c_match_id(driver->id_table, client) != NULL;

    return 0;
}
```

of_driver_match_device 函数用于完成设备树设备和驱动匹配。比较 I2C 设备节点的 compatible 属性和 of_device_id 中的 compatible 属性是否相等，如果相等的话就表示 I2C设备和驱动匹配。

12.2 RTC 驱动

1. HYM8563S芯片介绍

HYM8563S 是一款低功耗 CMOS 实时时钟/日历芯片，与 PCF8563 完全兼容，采用 SOP-8 封装。它通过 I²C 总线提供秒、分、时、星期、日、月、年和世纪信息，内置报警、定时器和可编程时钟输出，广泛应用于需要精确时间保持的嵌入式设备。

---

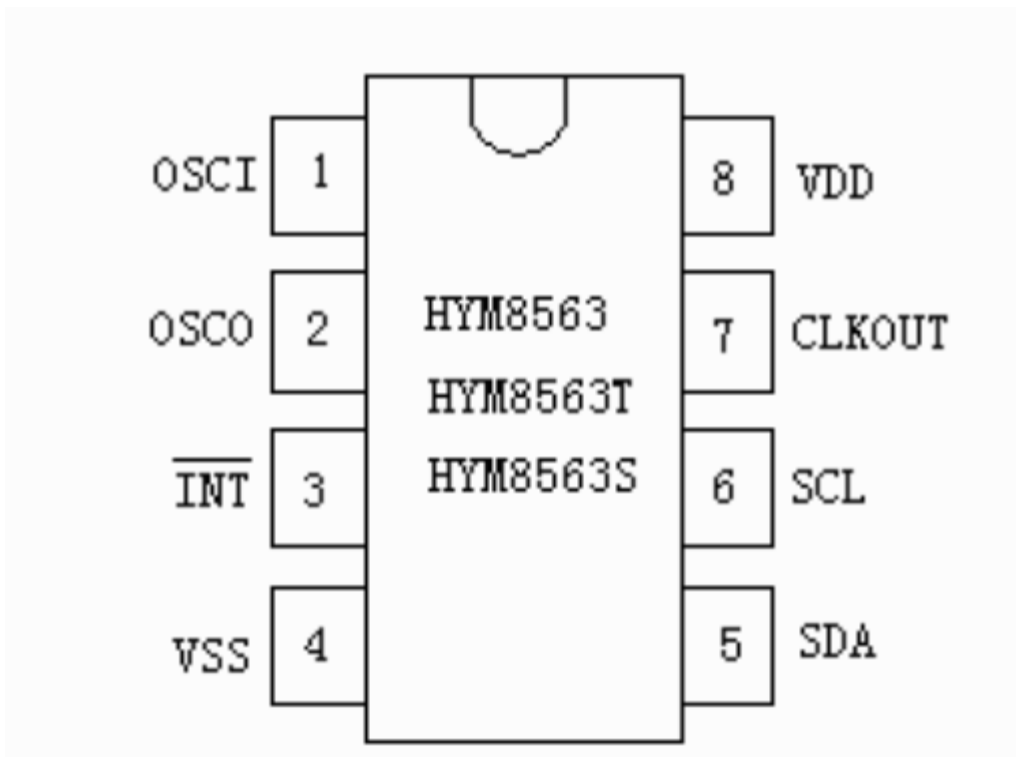
1. 芯片概述

该芯片专为超低功耗场景设计，工作电压范围 1.0V ~ 5.5V，典型后备功耗低于 0.25μA。

2. 核心特性

- **高兼容性与易用性**
  与 PCF8563 引脚及功能完全兼容，可直接替换；标准 SOP-8 封装，生产焊接方便。
- **超低功耗**
  后备电池供电时典型功耗 < 0.25μA，适合电池长期维持时钟。
- **高度集成**
  内置负载电容，无需外接电容；内置电源自动切换，VDD 和电池直连即可。
- **丰富的功能输出**
  - 可编程时钟输出 (CLKOUT)：可配置输出 32.768kHz、1024Hz、32Hz 或 1Hz。
  - 中断输出 (INT)：低电平有效，可由报警或倒计时定时器触发。
- **通信接口**
  标准 I²C 总线，最高支持 400kHz 快速模式，具有唯一从机地址。

3. 引脚功能说明 (SOP-8 封装)



引脚号	引脚名	功能描述
1	OSCI	振荡器输入，接 32.768kHz 晶振
2	OSCO	振荡器输出，接 32.768kHz 晶振
3	INT	中断输出（低电平有效，开漏），需外接上拉电阻
4	VSS	地
5	SDA	I²C 数据线（开漏），需接上拉电阻
6	SCL	I²C 时钟线（开漏），需接上拉电阻
7	CLKOUT	可编程时钟输出
8	VDD	电源正极，主电源与后备电池均连接于此

4. 内部功能框图

- **时间与日期寄存器**：BCD 码存储年、月、日、星期、时、分、秒，自动闰年补偿。
- **报警寄存器**：可设置时、分、日、星期报警条件，匹配时通过 INT 通知。
- **倒计时定时器**：8 位倒计时计数器（255→0），时钟源和初值可编程，归零触发中断。
- **控制/状态寄存器**：控制芯片运行模式、使能报警/定时器，并反映各功能状态标志。

5. 从机地址如下：

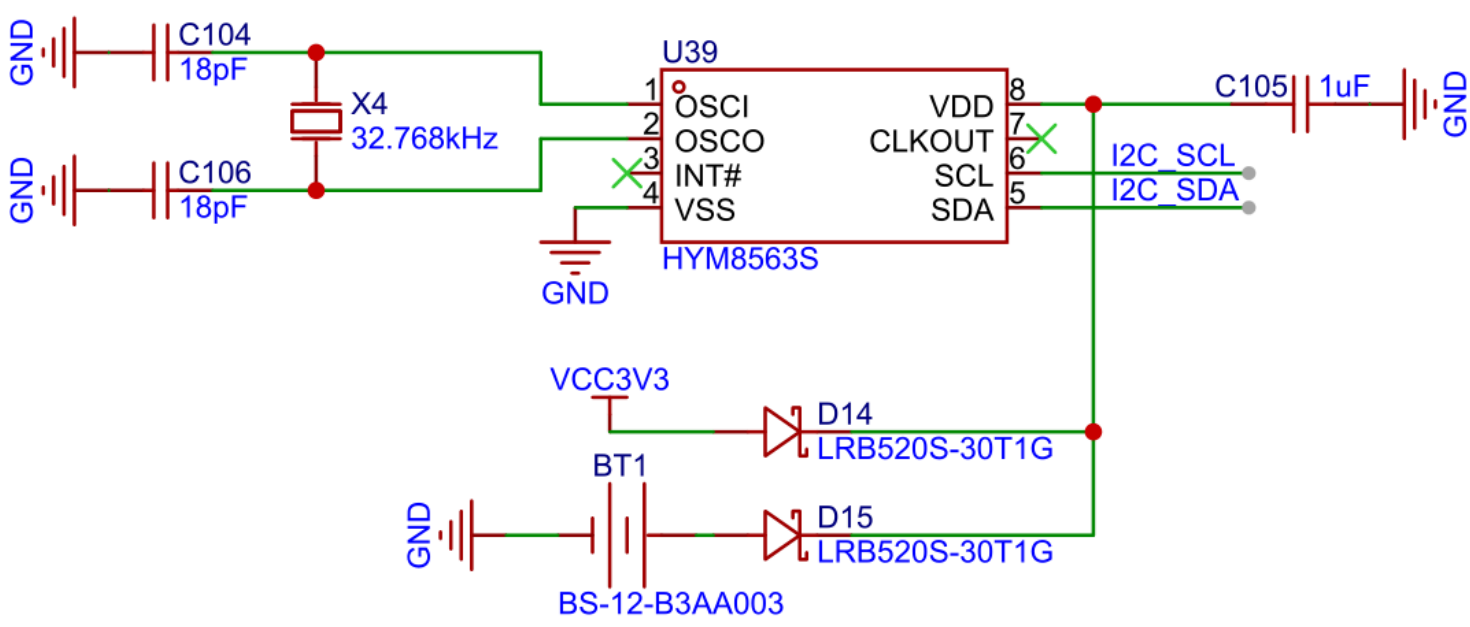
- **7 位地址**：`0x51` (二进制：`1010001`)
- **8 位写入地址** (左移 1 位 + R/W=0)：`0xA2`
- **8 位读取地址** (左移 1 位 + R/W=1)：`0xA3`

注意：实际编程时，I²C 库函数若要求传入 7 位地址，直接写 `0x51` 即可，底层会自动处理读写位；若要求传入 8 位格式，则需区分 `0xA2`（写）和 `0xA3`（读）。

6. 主要应用领域

- **智能三表**：电表、水表、气表，为分时计量和数据冻结提供时间戳。
- **安防与工控**：监控录像时间标记、PLC 系统日志和定时控制。
- **消费电子**：机顶盒、数码相框、智能家电等需要掉电保持时钟的设备。
- **医疗电子**：血压计、血糖仪等便携设备，记录测量时间。

2. 硬件原理图



3. HYM8563S设备树

当前 RK3506 项目采用**外部 I2C RTC 方案** — HYM8563（浩宇微电子），挂载在 I2C0 总线上，I2C 地址为 `0x51`。

```
rk3502.dtsi                          → 定义 i2c0 控制器基址 0xff040000
  └── rk3506.dtsi                    → 继承,无额外 RTC
        └── ido-evb3506-v1a-nand.dtsi    → 定义 hym8563 节点 (NAND版)
              └── ido-evb3506-v1a-flash-mipi-720x720.dts  → 用户打开的文件,继承 nand.dtsi
```

- kernel/arch/arm/boot/dts/rk3502.dtsi

```
    i2c0: i2c@ff040000 {
        compatible = "rockchip,rk3506-i2c", "rockchip,rk3399-i2c";
        reg = <0xff040000 0x1000>;
        interrupts = <GIC_SPI 40 IRQ_TYPE_LEVEL_HIGH>;
        #address-cells = <1>;
        #size-cells = <0>;
        clocks = <&cru CLK_I2C0>, <&cru PCLK_I2C0>;
        clock-names = "i2c", "pclk";
        status = "disabled";
    };
```

- `ido-evb3506-v1a-nand.dtsi:354-366` 中定义如下：

```
&i2c0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io0_i2c0_scl
                 &rm_io1_i2c0_sda>;

    hym8563: hym8563@51 {
        status = "okay";
        compatible = "haoyu,hym8563";       // 匹配驱动: rtc-hym8563.c
        reg = <0x51>;                        // I2C 从设备地址
        #clock-cells = <0>;                  // 时钟输出 provider (单输出)
        clock-frequency = <32768>;           // 输出 32.768kHz 时钟
        clock-output-names = "hym8563";      // 时钟名称
        pinctrl-names = "default";
        //pinctrl-0 = <&hym8563_int>;                 //  中断引脚被注释
        //interrupt-parent = <&gpio0>;                //  中断不可用
        //interrupts = <RK_PA0 IRQ_TYPE_LEVEL_LOW>;  // GPIO0_A0, 低电平
        //wakeup-source;                              //  唤醒功能被注释
    };

    es8388: es8388@11 {
        status = "okay";
```

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

```
        #sound-dai-cells = <0>;
        compatible = "everest,es8388", "everest,es8323";
        reg = <0x11>;
        clocks = <&mclkout_sai1>;
        clock-names = "mclk";
        assigned-clocks = <&mclkout_sai1>;
        assigned-clock-rates = <12288000>;
        pinctrl-names = "default";
        pinctrl-0 = <&rm_io8_sai1_mclk>;
    };
};
```

删除 es8388 节点，我们板子没有使用声卡芯片

```
    es8388: es8388@11 {
        status = "okay";
        #sound-dai-cells = <0>;
        compatible = "everest,es8388", "everest,es8323";
        reg = <0x11>;
        clocks = <&mclkout_sai1>;
        clock-names = "mclk";
        assigned-clocks = <&mclkout_sai1>;
        assigned-clock-rates = <12288000>;
        pinctrl-names = "default";
        pinctrl-0 = <&rm_io8_sai1_mclk>;
    };


    es8388_sound: es8388-sound {
        status = "okay";
        compatible = "rockchip,multicodecs-card";
        rockchip,card-name = "rockchip-es8388";
        spk-con-gpio = <&gpio0 RK_PB6 GPIO_ACTIVE_HIGH>;
        rockchip,pre-power-on-delay-ms = <30>;
        rockchip,post-power-down-delay-ms = <40>;
        rockchip,format = "i2s";
        rockchip,mclk-fs = <256>;
        rockchip,cpu = <&sai1>;
        rockchip,codec = <&es8388>;
        rockchip,audio-routing =
            "Headphone", "LOUT1",
            "Headphone", "ROUT1",
            "Speaker", "LOUT2",
            "Speaker", "ROUT2",
            "Headphone", "Headphone Power",
            "Headphone", "Headphone Power",
            "Speaker", "Speaker Power",
            "Speaker", "Speaker Power",
            "RINPUT1", "Main Mic",
            "RINPUT2", "Main Mic",
            "LINPUT1", "Headset Mic",
            "LINPUT2", "Headset Mic";
        pinctrl-names = "default";
        pinctrl-0 = <&spk_ctrl>;
    };
```

4. HYM8563S驱动

HYM8563S (RTC) 驱动文件是：

- kernel/drivers/rtc/rtc-hym8563.c

```
// SPDX-License-Identifier: GPL-2.0-only
/*
 * Haoyu HYM8563 RTC driver
 *
 * Copyright (C) 2013 MundoReader S.L.
 * Author: Heiko Stuebner <heiko@sntech.de>
 *
 * based on rtc-HYM8563
 * Copyright (C) 2010 Rockchip Electronics Co., Ltd.
 */

#include <linux/module.h>
#include <linux/clk-provider.h>
#include <linux/i2c.h>
#include <linux/bcd.h>
#include <linux/rtc.h>

#define HYM8563_CTL1        0x00
#define HYM8563_CTL1_TEST   BIT(7)
#define HYM8563_CTL1_STOP   BIT(5)
#define HYM8563_CTL1_TESTC  BIT(3)

#define HYM8563_CTL2        0x01
#define HYM8563_CTL2_TI_TP  BIT(4)
#define HYM8563_CTL2_AF     BIT(3)
#define HYM8563_CTL2_TF     BIT(2)
#define HYM8563_CTL2_AIE    BIT(1)
#define HYM8563_CTL2_TIE    BIT(0)

#define HYM8563_SEC     0x02
#define HYM8563_SEC_VL      BIT(7)
#define HYM8563_SEC_MASK    0x7f

#define HYM8563_MIN     0x03
#define HYM8563_MIN_MASK    0x7f

#define HYM8563_HOUR        0x04
#define HYM8563_HOUR_MASK   0x3f

#define HYM8563_DAY     0x05
#define HYM8563_DAY_MASK    0x3f

#define HYM8563_WEEKDAY     0x06
#define HYM8563_WEEKDAY_MASK    0x07

#define HYM8563_MONTH       0x07
#define HYM8563_MONTH_CENTURY   BIT(7)
#define HYM8563_MONTH_MASK  0x1f

#define HYM8563_YEAR        0x08

#define HYM8563_ALM_MIN     0x09
#define HYM8563_ALM_HOUR    0x0a
#define HYM8563_ALM_DAY     0x0b
#define HYM8563_ALM_WEEK    0x0c

/* Each alarm check can be disabled by setting this bit in the register */
#define HYM8563_ALM_BIT_DISABLE BIT(7)

#define HYM8563_CLKOUT      0x0d
#define HYM8563_CLKOUT_ENABLE   BIT(7)
#define HYM8563_CLKOUT_32768    0
#define HYM8563_CLKOUT_1024 1
#define HYM8563_CLKOUT_32   2
#define HYM8563_CLKOUT_1    3
#define HYM8563_CLKOUT_MASK 3

#define HYM8563_TMR_CTL     0x0e
#define HYM8563_TMR_CTL_ENABLE  BIT(7)
#define HYM8563_TMR_CTL_4096    0
#define HYM8563_TMR_CTL_64  1
#define HYM8563_TMR_CTL_1   2
#define HYM8563_TMR_CTL_1_60    3
#define HYM8563_TMR_CTL_MASK    3

#define HYM8563_TMR_CNT     0x0f
#define HYM8563_TMR_MAXCNT  0xff
#define HYM8563_TMR_CFG     (HYM8563_TMR_CTL_ENABLE | HYM8563_TMR_CTL_1)


static int hym8563_probe(struct i2c_client *client)
{
    struct hym8563 *hym8563;
    int ret;
    /*
     * hym8563 initial time(2021_1_1_12:00:00),
     * avoid hym8563 read time error
     */
    struct rtc_time tm_read, tm = {
        .tm_wday = 0,
        .tm_year = 121,
        .tm_mon = 0,
        .tm_mday = 1,
        .tm_hour = 12,
        .tm_min = 0,
        .tm_sec = 0,
```

```
};

hym8563 = devm_kzalloc(&client->dev, sizeof(*hym8563), GFP_KERNEL);
if (!hym8563)
    return -ENOMEM;

hym8563->rtc = devm_rtc_allocate_device(&client->dev);
if (IS_ERR(hym8563->rtc))
    return PTR_ERR(hym8563->rtc);

hym8563->client = client;
i2c_set_clientdata(client, hym8563);

ret = hym8563_init_device(client);
if (ret) {
    dev_err(&client->dev, "could not init device, %d\n", ret);
    return ret;
}

if (client->irq > 0) {
    ret = devm_request_threaded_irq(&client->dev, client->irq,
                    NULL, hym8563_irq,
                    IRQF_TRIGGER_LOW | IRQF_ONESHOT,
                    client->name, hym8563);
    if (ret < 0) {
        dev_err(&client->dev, "irq %d request failed, %d\n",
            client->irq, ret);
        return ret;
    }
}

if (client->irq > 0 ||
    device_property_read_bool(&client->dev, "wakeup-source")) {
    device_init_wakeup(&client->dev, true);
}

/* check state of calendar information */
ret = i2c_smbus_read_byte_data(client, HYM8563_SEC);
if (ret < 0)
    return ret;

    dev_info(&client->dev, "rtc information is %s\n",
        (ret & HYM8563_SEC_VL) ? "invalid" : "valid");

    hym8563_rtc_read_time(&client->dev, &tm_read);
    if ((ret & HYM8563_SEC_VL) || (tm_read.tm_year < 70) || (tm_read.tm_year > 200) ||
        (tm_read.tm_mon == -1) || (rtc_valid_tm(&tm_read) != 0))
        hym8563_rtc_set_time(&client->dev, &tm);

    hym8563->rtc->ops = &hym8563_rtc_ops;
    clear_bit(RTC_FEATURE_UPDATE_INTERRUPT, hym8563->rtc->features);

#ifdef CONFIG_COMMON_CLK
    hym8563_clkout_register_clk(hym8563);
#endif

    return devm_rtc_register_device(hym8563->rtc);
}

static const struct i2c_device_id hym8563_id[] = {
    { "hym8563", 0 },
    {},
};
MODULE_DEVICE_TABLE(i2c, hym8563_id);

static const struct of_device_id hym8563_dt_idtable[] = {
    { .compatible = "haoyu,hym8563" },
    {},
};
MODULE_DEVICE_TABLE(of, hym8563_dt_idtable);

static struct i2c_driver hym8563_driver = {
    .driver     = {
        .name   = "rtc-hym8563",
        .pm = &hym8563_pm_ops,
        .of_match_table = hym8563_dt_idtable,
    },
    .probe_new  = hym8563_probe,
    .id_table   = hym8563_id,
};

module_i2c_driver(hym8563_driver);

MODULE_AUTHOR("Heiko Stuebner <heiko@sntech.de>");
MODULE_DESCRIPTION("HYM8563 RTC driver");
MODULE_LICENSE("GPL");
```

- 配置驱动

```
  Device Drivers  --->
      [*] Real Time Clock  --->
              <*>   Haoyu Microelectronics HYM8563
```

编译内核并更新内核与设备到板子内。

5. HYM8563S测试

i2cdetect是一个用于检测和显示I2C总线上连接的设备地址的命令行实用程序。

i2cdetect命令通过发送I2C总线上的地址探测命令，来扫描整个总线以查找已连接的设备。它可以确定I2C总线上哪些设备正在与系统通信，并显示每个设备的地址。

以下是使用i2cdetect命令的基本语法：

```
i2cdetect [-y] [-r] [bus]
```

参数说明：

- `-y`：自动应答模式，无需用户确认。
- `-r`：以逆序方式显示设备地址。
- `bus`：可选参数，指定要扫描的I2C总线号，默认为1。

使用示例：

```
i2cdetect -l     // 显示总线上所有的控制器
i2cdetect -y 0   // 显示i2c-0 总线上所有的设备地址
i2cdetect -y 1   // 显示i2c-1 总线上所有的设备地址
```

该命令将扫描I2C总线0上的所有设备，并显示每个设备的地址。

- buildroot文件系统配置i2cdetect

```
linux@ubuntu:~/rk3506$ ./build.sh buildroot-config
Target packages  --->
    Hardware handling  --->
        [ ] i2c-tools              # 这个选项我们没有使用， 因为busybox已经默认配置了， 我们这里就不配置了
```

- busybox配置命令如下：

```
linux@ubuntu:~/rk3506$ ./build.sh bshell    # 进入 Buildroot shell
linux@ubuntu:~/rk3506/buildroot (rockchip_rk3506)$ make busybox-menuconfig   # 此时环境变量已自动设置好
linux@ubuntu:~/rk3506/buildroot (rockchip_rk3506)$ exit  # 退出 Buildroot shell

Miscellaneous Utilities  --->
    [*] i2cget (5.5 kb)
    [*] i2cset (6.7 kb)
    [*] i2cdump (7.1 kb)
    [*] i2cdetect (7.1 kb)
    [*] i2ctransfer (4.0 kb)
```

- 运行结果
- 显示`UU`表示设备正在被占用，解除占用后会显示地址。

```
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# i2cdetect -l
i2c-0   i2c         rk3x-i2c                        I2C adapter
root@rk3506-buildroot:/# i2cdetect -y 0
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- UU -- -- -- -- -- -- -- -- -- -- -- -- -- --
```

```
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
root@rk3506-buildroot:/#
# 0x51 就是RTC设备， HYM8563S
```

5. HYM8563S设置系统时间

- hwclock 设置系统时间

```
# === 读取 RTC 硬件时间 ===
hwclock -r                    # 读取 RTC 时间
hwclock --show                # 同上
cat /sys/class/rtc/rtc0/date  # 查看日期
cat /sys/class/rtc/rtc0/time  # 查看时间

# === 设置 RTC 硬件时间 ===
date -s "2026-06-27 15:30:00"                # 设置系统时间
hwclock -w                                   # 将系统时间写入 RTC

# === 同步操作 ===
hwclock -s                    # 从 RTC 同步到系统时间 (hwclock to system)
hwclock -w                    # 从系统时间同步到 RTC (system to hwclock)
```

- 运行结果

```
root@rk3506-buildroot:/# hwclock -r
Sat Jun 27 14:31:11 2026  0.000000 seconds
root@rk3506-buildroot:/# hwclock --show
Sat Jun 27 14:31:20 2026  0.000000 seconds
root@rk3506-buildroot:/# cat /sys/class/rtc/rtc0/date
2026-06-27
root@rk3506-buildroot:/# cat /sys/class/rtc/rtc0/time
06:31:39
root@rk3506-buildroot:/# date -s "2026-06-27 15:30:00"
Sat Jun 27 15:30:00 CST 2026
root@rk3506-buildroot:/# hwclock -w
root@rk3506-buildroot:/# date
Sat Jun 27 15:30:24 CST 2026
root@rk3506-buildroot:/# hwclock -r
Sat Jun 27 15:30:42 2026  0.000000 seconds
root@rk3506-buildroot:/#
```

- ntp服务器时间

NTP（Network Time Protocol）服务原理

NTP 是一种用于在计算机网络中同步时钟的协议，由 David Mills 于 1985 年设计。它使计算机系统能够将本地时钟与参考时间源（如原子钟、GPS 接收器或互联网时间服务器）同步，精度通常可达毫秒级别（局域网内可达微秒级）。

- Buildroot 配置 ntp服务

```
Target packages  --->
     Networking applications  --->
            [*]   ntp
            [*]   ntpd
            [*]   ntpdate
```

生成的启动脚本如下：

```
linux@ubuntu:~/rk3506$ ls buildroot/output/rockchip_rk3506/target/etc/init.d/
pre_init        S01seedrng          S10udev         S40bluetoothd  S50sshd           S99appinit
rcK             S01syslogd          S11alsa-utils   S40network     S50usbdevice.sh   S99-auto-reboot
rcS             S02klogd            S13irqbalance   S41dhcpcd      S89ML37init       S99fstrim
rcS.early       S02sysctl           S30dbus         S49ntp         S89WIFI_EN        S99input-event-daemon
S00mountall.sh  S03modules_init.sh  S35iptables     S50mosquitto   S90rkwifibt
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ cat buildroot/output/rockchip_rk3506/target/etc/init.d/S49ntp
#!/bin/sh
#
# Starts Network Time Protocol daemon
#

DAEMON="ntpd"
PIDFILE="/var/run/$DAEMON.pid"

NTPD_ARGS="-u ntp:ntp -g"

# shellcheck source=/dev/null
[ -r "/etc/default/$DAEMON" ] && . "/etc/default/$DAEMON"

start() {
        printf 'Starting %s: ' "$DAEMON"
        # shellcheck disable=SC2086 # we need the word splitting
        start-stop-daemon -S -q -p "$PIDFILE" -x "/usr/sbin/$DAEMON" \
                -- $NTPD_ARGS -p "$PIDFILE"
        status=$?
        if [ "$status" -eq 0 ]; then
                echo "OK"
        else
                echo "FAIL"
        fi
        return "$status"
}

stop() {
        printf 'Stopping %s: ' "$DAEMON"
        start-stop-daemon -K -q -p "$PIDFILE"
        status=$?
        if [ "$status" -eq 0 ]; then
                rm -f "$PIDFILE"
                echo "OK"
        else
                echo "FAIL"
        fi
        return "$status"
}

restart() {
        stop
        sleep 1
        start
}

case "$1" in
        start|stop|restart)
                "$1";;
        reload)
                # Restart, since there is no true "reload" feature.
                restart;;
        *)
                echo "Usage: $0 {start|stop|restart|reload}"
                exit 1
esac
linux@ubuntu:~/rk3506$
```

使用这个脚本可以自动更新系统的时间。

12.3 GT911触摸屏驱动

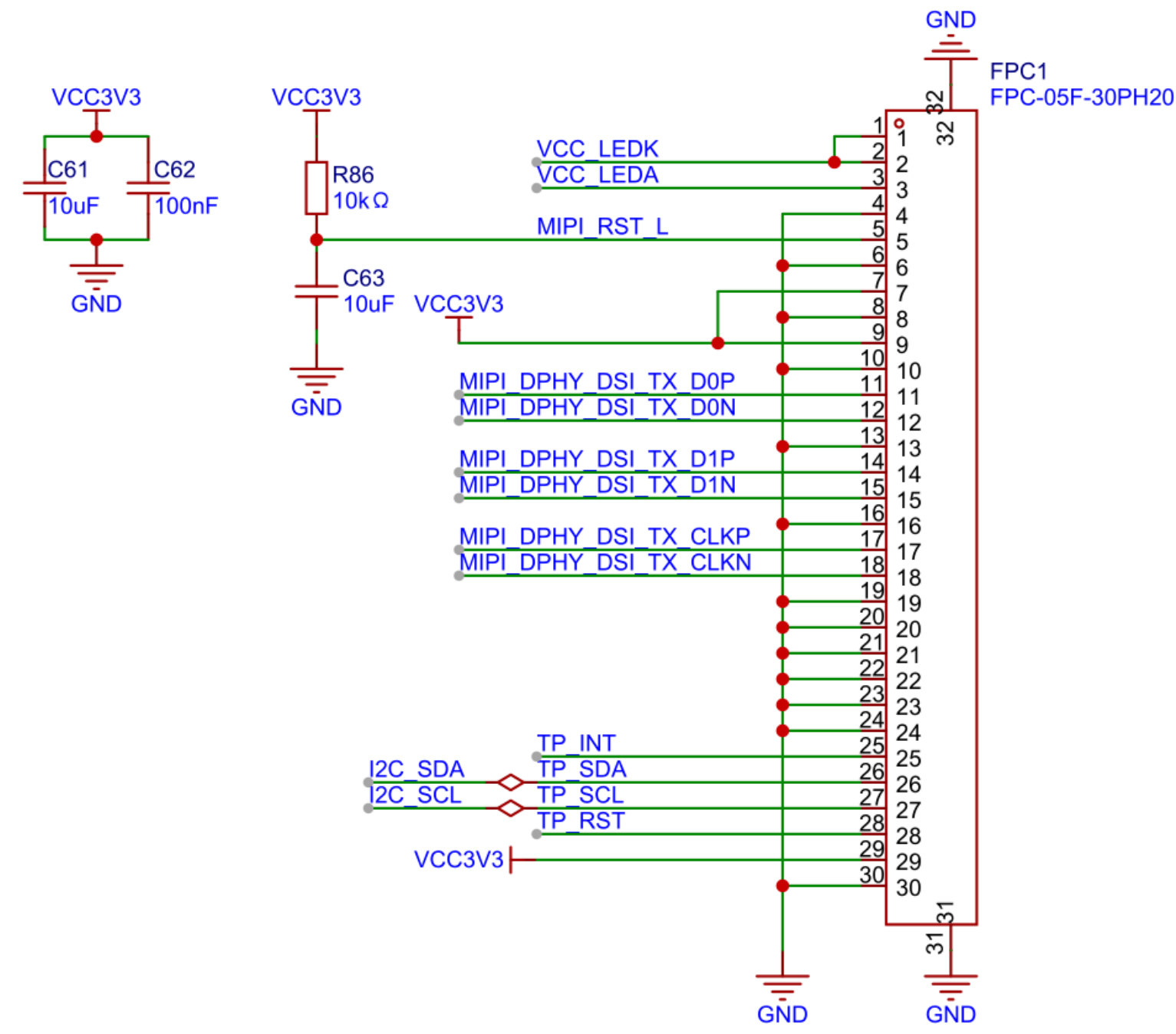
1. GT911芯片介绍

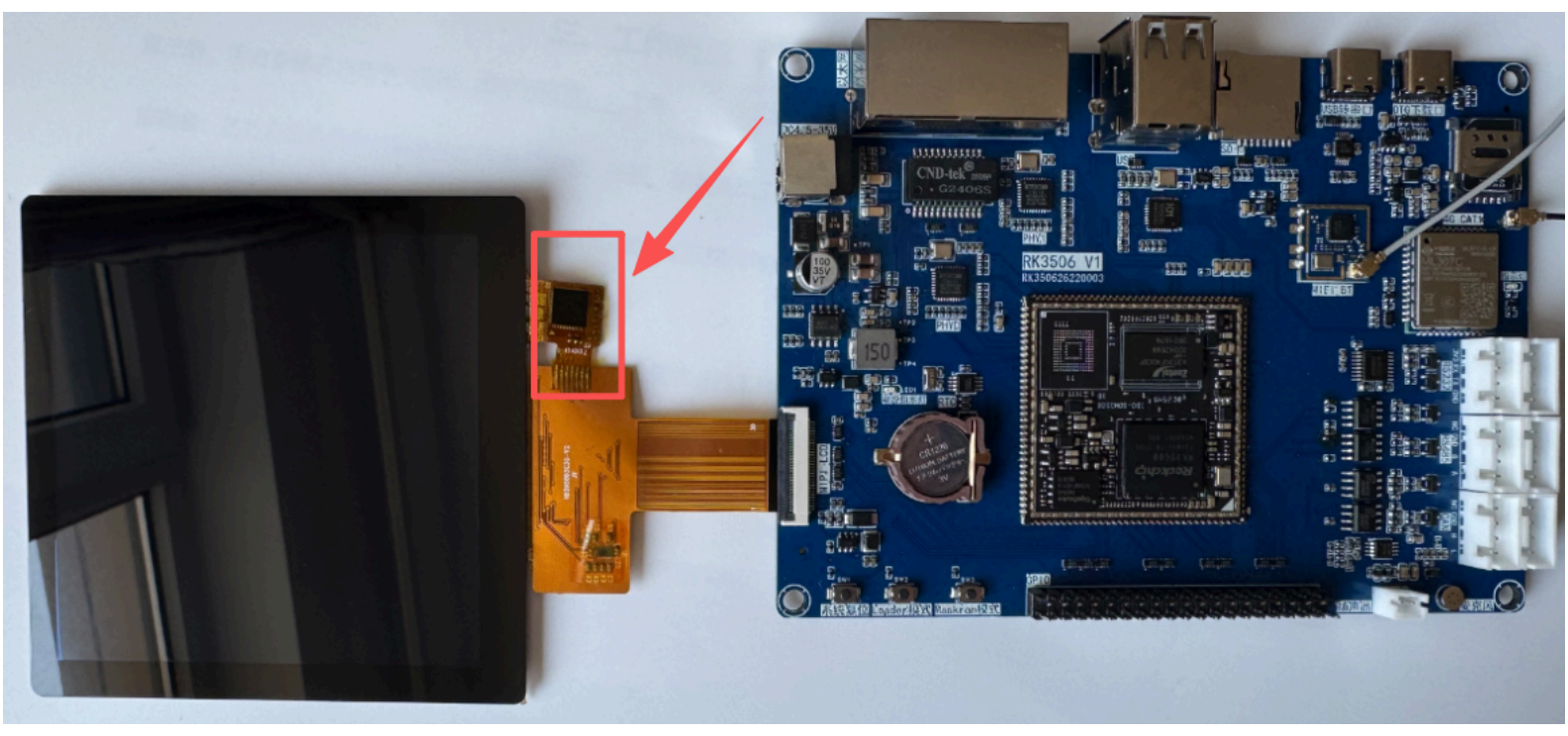
GT911 是汇顶科技（Goodix）推出的一款电容式触摸屏控制器芯片，通过检测电容变化来识别手指触摸位置，支持多点触控手势，广泛应用于嵌入式设备、工业HMI、智能家电及各类带 LCD 屏幕的开发板。

- 主要特性：
  - 最多支持 5 点触摸，可上报每个触点的坐标、面积和压力。

    - 主机接口为 I²C（最高 400 kHz），另配有 INT（中断，低有效）和 RST（复位）引脚。
    - I/O 电压 1.8 V 至 3.3 V，模拟供电 2.8 V 至 3.3 V，可直接连接 3.3 V 主控。
- 典型引脚定义（常见 6 脚排线）：
    - VCC：电源（3.3 V）
    - GND：地
    - SCL：I²C 时钟
    - SDA：I²C 数据
    - INT：触摸中断输出（有触摸数据时拉低）
    - RST：复位输入（用于上电复位或主机控制复位）
- 与主机通信：
    - 常见 7 位 I²C 地址为 0x14。

2. GT911硬件原理图





3. GT911 设备树

kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts

```
&i2c0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io0_i2c0_scl
             &rm_io1_i2c0_sda>;

    gt911@14 {
        compatible = "goodix,gt9xx";
        reg = <0x14>;
        pinctrl-0 = <&touch_gpio>;
        touch-gpio = <&gpio0 RK_PA2 IRQ_TYPE_LEVEL_LOW>;
        reset-gpio = <&gpio0 RK_PA3 GPIO_ACTIVE_HIGH>;
        max-x = <720>;
        max-y = <720>;
        tp-size = <9110>;
        status = "okay";
    };
};
```

4. GT911 驱动

- 修改驱动代码

kernel/drivers/input/touchscreen/gt9xx/gt9xx.c

```
/* drivers/input/touchscreen/gt9xx.c
 *
 * 2010 - 2013 Goodix Technology.
 *
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation; either version 2 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be a reference
 * to you, when you are integrating the GOODiX's CTP IC into your system,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the GNU
 * General Public License for more details.
 *
 * Version: 2.2
 * Authors: andrew@goodix.com, meta@goodix.com
 * Release Date: 2014/01/14
 * Revision record:
 *      V1.0:
 *          first Release. By Andrew, 2012/08/31
 *      V1.2:
 *          modify gtp_reset_guitar,slot report,tracking_id & 0x0F. By Andrew, 2012/10/15
 *      V1.4:
 *          modify gt9xx_update.c. By Andrew, 2012/12/12
 *      V1.6:
 *          1. new heartbeat/esd_protect mechanism(add external watchdog)
 *          2. doze mode, sliding wakeup
 *          3. 3 more cfg_group(GT9 Sensor_ID: 0~5)
 *          3. config length verification
 *          4. names & comments
 *                  By Meta, 2013/03/11
 *      V1.8:
 *          1. pen/stylus identification
 *          2. read double check & fixed config support
 *          3. new esd & slide wakeup optimization
 *                  By Meta, 2013/06/08
 *      V2.0:
 *          1. compatible with GT9XXF
 *          2. send config after resume
 *                  By Meta, 2013/08/06
 *      V2.2:
 *          1. gt9xx_config for debug
 *          2. gesture wakeup
 *          3. pen separate input device, active-pen button support
 *          4. coordinates & keys optimization
 *                  By Meta, 2014/01/14
 */

#include <linux/irq.h>
#include "gt9xx.h"
#include "gt9xx_cfg.h"
```

```
#include <linux/regulator/consumer.h>

#if GTP_ICS_SLOT_REPORT
    #include <linux/input/mt.h>
#endif

static u8 m89or101 = TRUE;
static u8 bgt911 = FALSE;
static u8 bgt9110 = FALSE;
static u8 bgt9111 = FALSE;
static u8 bgt970 = FALSE;
static u8 bgt910 = FALSE;
static u8 gtp_change_x2y = TRUE;
static u8 gtp_x_reverse = FALSE;
static u8 gtp_y_reverse = TRUE;

static const char *goodix_ts_name = "goodix-ts";
static struct workqueue_struct *goodix_wq;
struct i2c_client * gtp_i2c_connect_client = NULL;
static u8 config[GTP_CONFIG_MAX_LENGTH + GTP_ADDR_LENGTH]
                = {GTP_REG_CONFIG_DATA >> 8, GTP_REG_CONFIG_DATA & 0xff};

#if GTP_HAVE_TOUCH_KEY
    static const u16 touch_key_array[] = GTP_KEY_TAB;
    #define GTP_MAX_KEY_NUM  (sizeof(touch_key_array)/sizeof(touch_key_array[0]))

#if GTP_DEBUG_ON
    static const int  key_codes[] = {KEY_HOME, KEY_BACK, KEY_MENU, KEY_SEARCH};
    static const char *key_names[] = {"Key_Home", "Key_Back", "Key_Menu", "Key_Search"};
#endif

#endif

static const struct i2c_device_id goodix_ts_id[] = {
    { GTP_I2C_NAME, 0 },
    { }
};

static struct of_device_id goodix_ts_dt_ids[] = {
    { .compatible = "goodix,gt9xx" },
    { }
};

static struct i2c_driver goodix_ts_driver = {
    .probe      = goodix_ts_probe,
    .remove     = goodix_ts_remove,
    .id_table   = goodix_ts_id,
    .driver = {
        .name     = GTP_I2C_NAME,
     .of_match_table = of_match_ptr(goodix_ts_dt_ids),
    },
};

/*******************************************************
Function:
    Driver Install function.
Input:
    None.
Output:
    Executive Outcomes. 0---succeed.
********************************************************/
static int goodix_ts_init(void)
{
    s32 ret;

    GTP_DEBUG_FUNC();
    GTP_INFO("GTP driver installing...");
    goodix_wq = create_singlethread_workqueue("goodix_wq");
    if (!goodix_wq)
    {
        GTP_ERROR("Creat workqueue failed.");
        return -ENOMEM;
    }
#if GTP_ESD_PROTECT
    INIT_DELAYED_WORK(&gtp_esd_check_work, gtp_esd_check_func);
    gtp_esd_check_workqueue = create_workqueue("gtp_esd_check");
#endif
    ret = i2c_add_driver(&goodix_ts_driver);
    return ret;
}

/*******************************************************
Function:
    Driver uninstall function.
Input:
    None.
Output:
    Executive Outcomes. 0---succeed.
********************************************************/
static void goodix_ts_exit(void)
{
    GTP_DEBUG_FUNC();
    GTP_INFO("GTP driver exited.");
    i2c_del_driver(&goodix_ts_driver);
    if (goodix_wq)
    {
        destroy_workqueue(goodix_wq);
    }
}
//late_initcall(goodix_ts_init);
module_init(goodix_ts_init);

module_exit(goodix_ts_exit);

MODULE_DESCRIPTION("GTP Series Driver");
MODULE_LICENSE("GPL");
MODULE_IMPORT_NS(VFS_internal_I_am_really_a_filesystem_and_am_NOT_a_driver);
```

- 配置内核，选择GT911 驱动

```
linux@ubuntu:~/rk3506$ ./build.sh  kernel-config
Device Drivers  --->
    Input device support  --->
        [*]   Touchscreens  --->
            <*>   Goodix gt9xx support for rockchip platform
```

- 编译内核与设备树

```
linux@ubuntu:~/rk3506$ ./build.sh  kernel
```

更新设备树与内核到开发板内，并启动系统。

5. GT911驱动测试

接上屏幕如下图：

i2cdetect是一个用于检测和显示I2C总线上连接的设备地址的命令行实用程序。

i2cdetect命令通过发送I2C总线上的地址探测命令，来扫描整个总线以查找已连接的设备。它可以确定I2C总线上哪些设备正在与系统通信，并显示每个设备的地址。

以下是使用i2cdetect命令的基本语法：

```
i2cdetect [-y] [-r] [bus]
```

参数说明：

- `-y`：自动应答模式，无需用户确认。
- `-r`：以逆序方式显示设备地址。
- `bus`：可选参数，指定要扫描的I2C总线号，默认为1。

使用示例：

```
i2cdetect -l     // 显示总线上所有的控制器
i2cdetect -y 0   // 显示i2c-0 总线上所有的设备地址
```

该命令将扫描I2C总线0上的所有设备，并显示每个设备的地址。

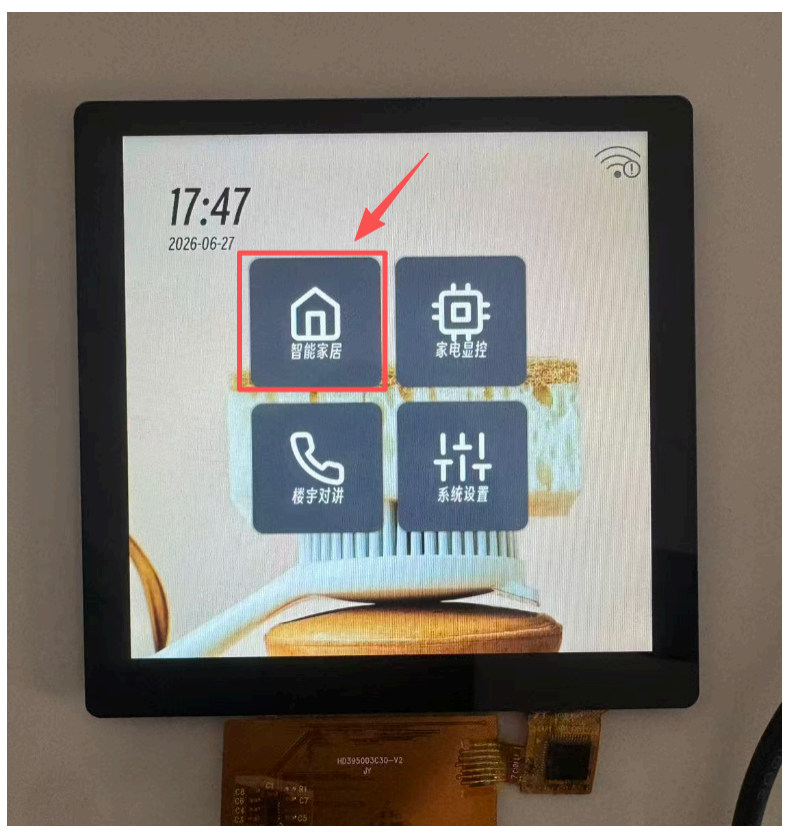
- 运行结果

```
root@rk3506-buildroot:/# i2cdetect -l
i2c-0   i2c         rk3x-i2c                        I2C adapter
root@rk3506-buildroot:/# i2cdetect -y 0
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- UU -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- UU -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
root@rk3506-buildroot:/#
```

- 测试触摸屏

校准触摸屏，buildroot中必须是已经启动Tslib的功能才会有这个命令，需要重新编译文件系统。

点击触摸屏菜单的**智能家居**菜单可以进入黑屏模式，这个是固件的默认功能。



然后执行命令就可以进入触摸屏测试模式。

```
ts_calibrate
```

- 执行结果

```
root@rk3506-buildroot:/# ts_calibrate
xres = 720, yres = 720
Took 7 samples...
Top left : X =   38 Y =   37
Took 8 samples...
Top right : X =  707 Y =   41
Took 10 samples...
Bot right : X =  716 Y =  702
Took 9 samples...
Bot left : X =   12 Y =  706
Took 9 samples...
Center : X =  358 Y =  365
25.264332 0.902385 0.011567
14.878186 0.000032 0.932226
Calibration constants: 1655723 59138 758 975056 2 61094 65536
```

- 测试触摸屏工作是否正常

```
ts_test_mt
```

- 结果如下

上图中有三个按钮“Drag”、“Draw”和“Quit”，这三个按钮的功能如下：

- Drag：拖拽按钮，默认就是此功能，大家可以看到屏幕中间有一个十字光标，我们可以通过触摸屏幕来拖拽此光标。
- Draw：绘制按钮，按下此按钮我们就可以在屏幕上进行简单的绘制。
- Quit：退出按钮，退出 ts_test_mt 测试软件。

至此触摸屏配置完成。

第13章 Linux 串口驱动

13.1 TTY理解

1. TTY概念

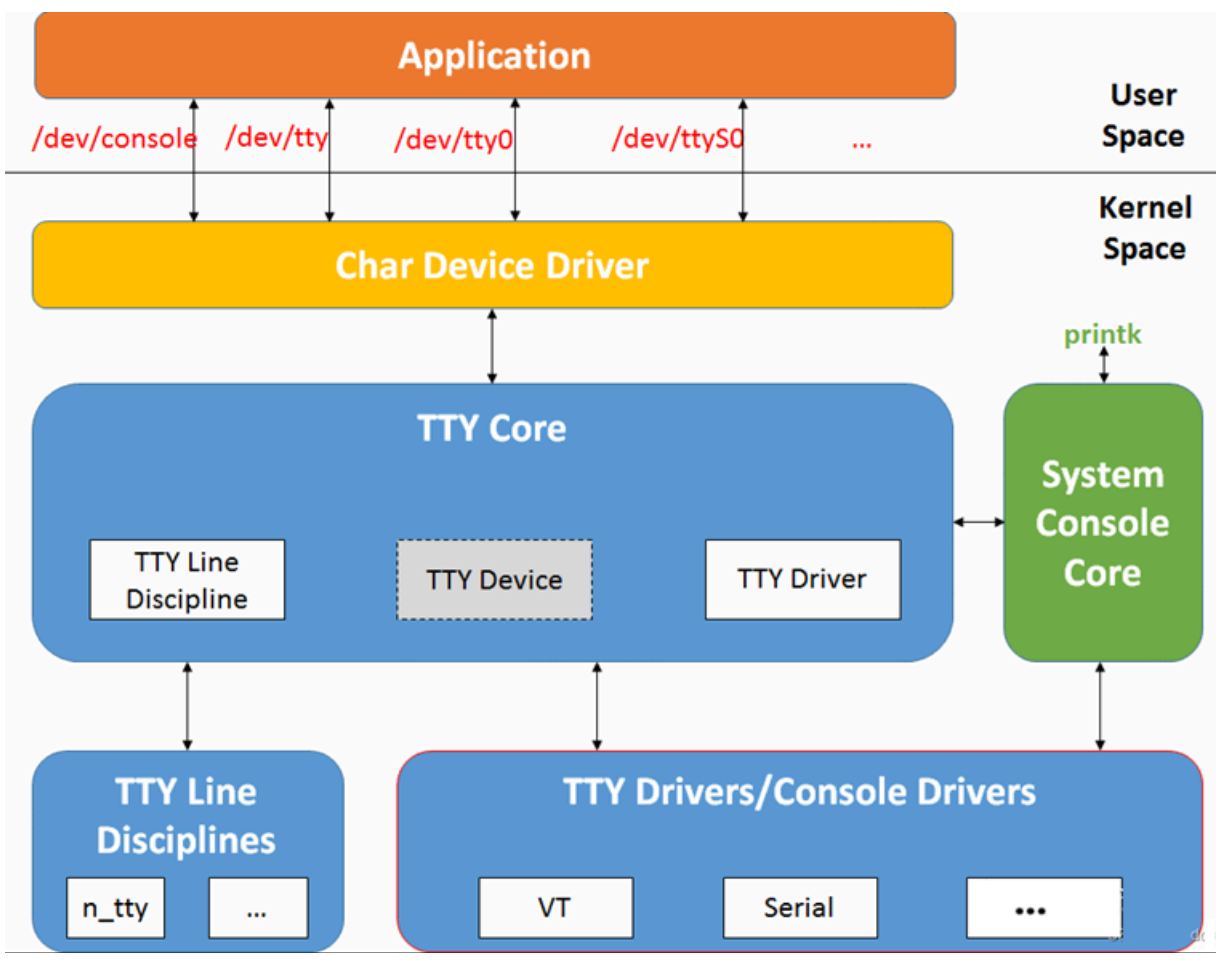
TTY/UART 子系统是 Linux 内核中最古老也最复杂的子系统之一，它抽象了串行终端设备，其中 UART 作为物理串口，是嵌入式、服务器调试和工业通信的基础。

终端一词囊括了很多：

- /dev/ttySx 串口设备终端，对应windows的COMx
- /dev/pts: 当你ssh登陆机器看后台时，就会发现诸如pts/1 、pts/2
- /dev/tty: 控制终端(Control)，就是当前tty设备，所以你echo到这里都会显示到你输入echo的地方。

2. TTY 架构层次

TTY 子系统采用严格的分层设计，从上到下依次为：用户空间接口、TTY 核心层、线路规程层和 TTY 设备驱动层。



- **用户层**：在用户空间，我们操作`TTY`的方法就是通过`open`，`read`，`write`读写设备文件，如：`/dev/tty`、`/dev/console`等
- **字符设备驱动**：以字符设备驱动的形式，向应用程序提供统一接口`open，read，write`，以便输入输出数据。
- `TTY Core`：该核心层主要负责控制tty设备数据流与格式，使TTY Driver一致处理进出硬件的数据，简单理解就是标准化。
- `System Console Core`：其主要有两个功能：
  - 向系统提供控制台终端（Console Terminal），以便让用户登录进行交互操作。
  - 提供printk功能，以便kernel代码进行日志输出。
- `TTY Line Discipline`：线路规程，以特殊的方式对数据进行二次加工，通常表现为协议的转换，如：`PPP`和`Bluetooth`，以便可以虚拟的插入到任何`tty`设备。
- `TTY Driver`：该层主要用于实现各类终端的驱动，用以控制实际硬件设备，用于收发数据。其有3种不通类型的驱动：**控制台，串口，**`pty`。其中，控制台和`pty`驱动已经被编写进内核。

3. 驱动的目录结构

```
linux@ubuntu:~/rk3506$ tree kernel/drivers/tty/
kernel/drivers/tty/
├── hvc                             #虚拟控制台
│   ├── hvc_console.c
│   ├── hvc_console.h
│   ├── hvc_dcc.c
├── ipwireless                      #无线终端驱动
│   ├── built-in.a
│   ├── hardware.c
│   ├── hardware.h
│   ├── setup_protocol.h
│   ├── tty.c
│   └── tty.h
├── n_tty.c                         #tty line discipline 线路规程相关文件
├── n_tty.o
├── pty.c                           # pty驱动
├── pty.o
├── rpmsg_tty.c
├── serial
│   ├── 21285.c
│   ├── 8250
│   │   ├── 8250_core.c               # 8250 串口核心层
│   │   ├── 8250_dma.c                # 8250 dma驱动
│   │   ├── 8250_dw.c                 # 8250 device driver抽象
│   │   ├── 8250_port.c               # 8250 串口port抽象
│   ├── dz.c
│   ├── dz.h
│   ├── Kconfig
├── synclink_gt.c
└── vt                              # 虚拟终端驱动
    ├── vt.c
    ├── vt_ioctl.c
    ├── vt_ioctl.o
    └── vt.o
```

4. TTY在Linux下的分布

`TTY`是所有终端的统称，对于不同的终端，不同的驱动程序，那么我们怎么知道当前系统中，哪个驱动被加载，哪个终端设备存在呢?

- 查看所有终端

可以查看`/proc/tty/drivers`文件，这个文件可以包含当前存在的，不同`tty`驱动的列表，驱动的名字，驱动的主编号，次编号范围，`tty`驱动的类型。

```
linux@ubuntu:~/rk3506$ cat /proc/tty/drivers
/dev/tty             /dev/tty        5       0 system:/dev/tty
/dev/console         /dev/console    5       1 system:console
/dev/ptmx            /dev/ptmx       5       2 system
/dev/vc/0            /dev/vc/0       4       0 system:vtmaster
dbc_serial           /dev/ttyDBC   242 0-63 serial
ttyprintk            /dev/ttyprintk    5       3 console
max310x              /dev/ttyMAX   204 209-224 serial
serial               /dev/ttyS       4 64-111 serial
pty_slave            /dev/pts      136 0-1048575 pty:slave
pty_master           /dev/ptm      128 0-1048575 pty:master
unknown              /dev/tty        4 1-63 console
linux@ubuntu:~/rk3506$
```

5. 设备模型的角度

所有当前注册的以及在内核中出现的`tty`设备，有他们自己的子目录`/sys/class/tty`下面。

在他们`tty`设备的子目录下面的文件夹中，有一个`dev`文件，包含了分配给`tty`设备的主次编号。 如果驱动程序告诉内核与tty设备相关联的物理设备和驱动程序的位置，它就会创建回这些位置的符号链接。

```
linux@ubuntu:~/rk3506$ tree /sys/class/tty/
/sys/class/tty/
├── console -> ../../devices/virtual/tty/console
├── ptmx -> ../../devices/virtual/tty/ptmx
├── tty -> ../../devices/virtual/tty/tty
├── tty0 -> ../../devices/virtual/tty/tty0
├── tty1 -> ../../devices/virtual/tty/tty1
├── tty10 -> ../../devices/virtual/tty/tty10
├── tty11 -> ../../devices/virtual/tty/tty11
├── tty62 -> ../../devices/virtual/tty/tty62
├── tty63 -> ../../devices/virtual/tty/tty63
├── tty7 -> ../../devices/virtual/tty/tty7
├── tty8 -> ../../devices/virtual/tty/tty8
├── tty9 -> ../../devices/virtual/tty/tty9
├── ttyprintk -> ../../devices/virtual/tty/ttyprintk
├── ttyS0 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.0/tty/ttyS0
├── ttyS1 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.1/tty/ttyS1
├── ttyS10 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.10/tty/ttyS10
├── ttyS11 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.11/tty/ttyS11
├── ttyS30 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.30/tty/ttyS30
├── ttyS31 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.31/tty/ttyS31
├── ttyS4 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.4/tty/ttyS4
├── ttyS5 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.5/tty/ttyS5
├── ttyS6 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.6/tty/ttyS6
├── ttyS7 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.7/tty/ttyS7
├── ttyS8 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.8/tty/ttyS8
└── ttyS9 -> ../../devices/platform/serial8250/serial8250:0/serial8250:0.9/tty/ttyS9
linux@ubuntu:~/rk3506$
```

6. 字符设备的角度

每个`tty`设备都有一个`struct cdev`，以便用户空间可以访问。

```
linux@ubuntu:~/rk3506$ ls -l /dev/
总计 0
crw-r--r--   1 root  root     10, 235  6月 26 21:02 autofs
drwxr-xr-x   2 root  root         600  6月 26 21:02 block
drwxr-xr-x   2 root  root          80  6月 26 21:02 bsg
crw-------   1 root  root     10, 234  6月 26 21:02 btrfs-control
lrwxrwxrwx   1 root  root           3  6月 26 21:02 cdrom -> sr0
drwxr-xr-x   2 root  root        3920  6月 28 12:56 char
crw--w----   1 root  tty       5,   1  6月 26 21:02 console
lrwxrwxrwx   1 root  root          11  6月 26 21:02 core -> /proc/kcore
crw--w----   1 root  tty       4,   4  6月 26 21:02 tty4
crw--w----   1 root  tty       4,  40  6月 26 21:02 tty40
crw--w----   1 root  tty       4,  41  6月 26 21:02 tty41
crw--w----   1 root  tty       4,  42  6月 26 21:02 tty42
crw--w----   1 root  tty       4,  43  6月 26 21:02 tty43
crw--w----   1 root  tty       4,  44  6月 26 21:02 tty44
crw--w----   1 root  tty       4,  45  6月 26 21:02 tty45
crw--w----   1 root  tty       4,  46  6月 26 21:02 tty46
crw--w----   1 root  tty       4,  47  6月 26 21:02 tty47
crw--w----   1 root  tty       4,  48  6月 26 21:02 tty48
crw--w----   1 root  tty       4,  49  6月 26 21:02 tty49
crw--w----   1 root  tty       4,   5  6月 26 21:02 tty5
crw--w----   1 root  tty       4,  50  6月 26 21:02 tty50
crw--w----   1 root  tty       4,  51  6月 26 21:02 tty51
crw--w----   1 root  tty       4,  52  6月 26 21:02 tty52
crw--w----   1 root  tty       4,  53  6月 26 21:02 tty53
crw--w----   1 root  tty       4,  54  6月 26 21:02 tty54
crw--w----   1 root  tty       4,  55  6月 26 21:02 tty55
crw--w----   1 root  tty       4,  56  6月 26 21:02 tty56
crw--w----   1 root  tty       4,  57  6月 26 21:02 tty57
crw--w----   1 root  tty       4,  58  6月 26 21:02 tty58
crw--w----   1 root  tty       4,  59  6月 26 21:02 tty59
crw--w----   1 root  tty       4,   6  6月 26 21:02 tty6
crw--w----   1 root  tty       4,  60  6月 26 21:02 tty60
crw--w----   1 root  tty       4,  61  6月 26 21:02 tty61
crw--w----   1 root  tty       4,  62  6月 26 21:02 tty62
crw--w----   1 root  tty       4,  63  6月 26 21:02 tty63
crw--w----   1 root  tty       4,   7  6月 26 21:02 tty7
crw--w----   1 root  tty       4,   8  6月 26 21:02 tty8
crw--w----   1 root  tty       4,   9  6月 26 21:02 tty9
crw-------   1 root  root      5,   3  6月 26 21:02 ttyprintk
crw-rw----   1 root  dialout   4,  64  6月 26 21:02 ttyS0
```

```
crw-rw----   1 root  dialout   4,  65  6月 26 21:02 ttyS1
crw-rw-rw-   1 root  root     10, 262  6月 26 21:02 vsock
crw-rw-rw-   1 root  root      1,   5  6月 26 21:02 zero
crw-------   1 root  root     10, 249  6月 26 21:02 zfs
linux@ubuntu:~/rk3506$
```

13.2 RS232电平标准

1. RS-232 介绍

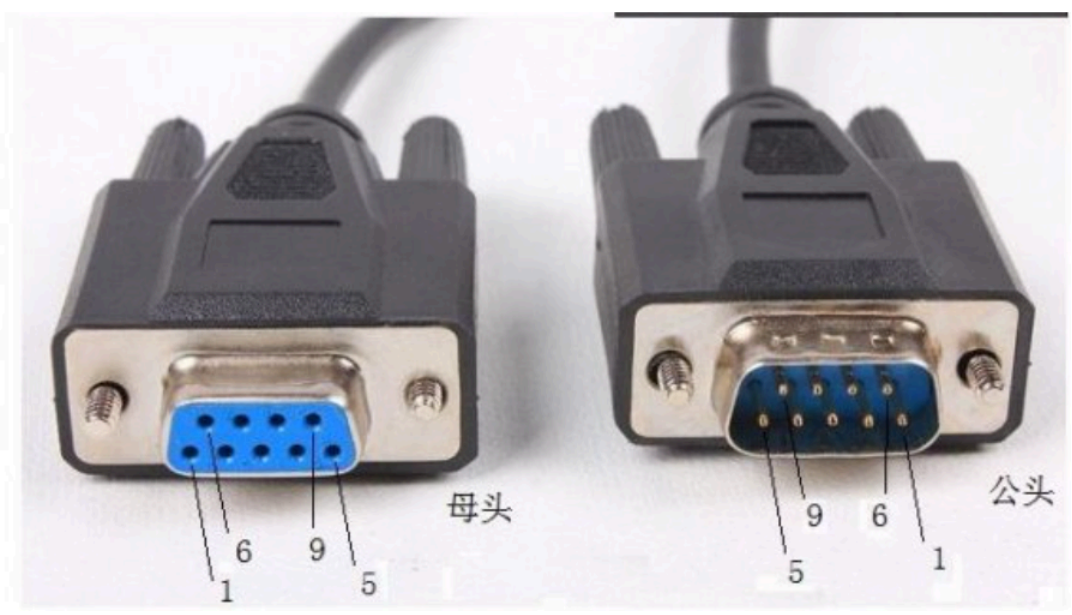
RS-232 的全称是 **Recommended Standard 232**，由美国电子工业协会（EIA）于 1960 年代制定。它定义了**数据终端设备（DTE，如计算机）与数据通信设备（DCE，如调制解调器）之间**的串行、异步通信规范。

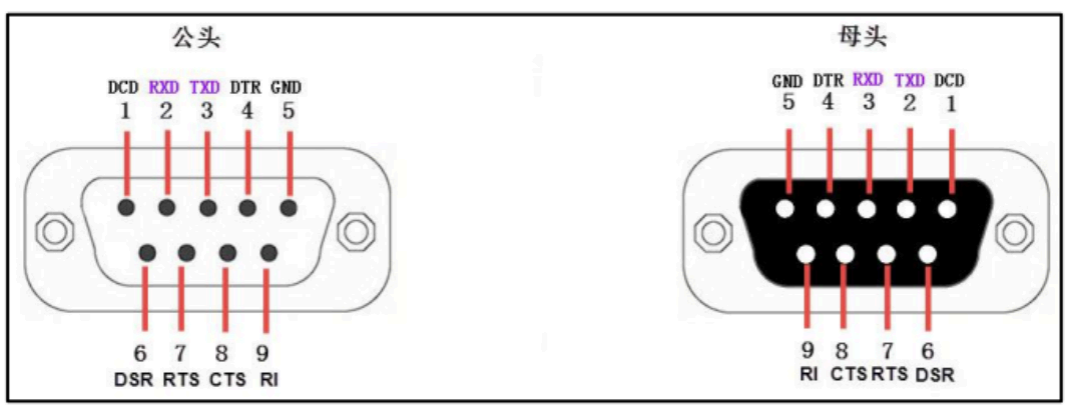
2. 主要特点

- 点对点通信：它是一种以**单端信号、点对点**方式进行数据传输的接口标准。
- **逻辑电平（负逻辑）**
  - 这是 RS-232 最需要留意的部分，因为它和我们常见的 TTL 逻辑电平不同。
  - 逻辑 `1`（Mark）：电压在 **-3V ~ -15V** 之间（通常 -12V）
  - 逻辑 `0`（Space）：电压在 **+3V ~ +15V** 之间（通常 +12V）
  - **注意**：-3V 到 +3V 之间为无效状态，这提供了较好的噪声容限。
- **与 TTL/UART 的区别**
  微控制器上的 UART 接口是 TTL 电平（0V 表示 `0`，3.3V/5V 表示 `1`），**不能直接相连**，必须通过 MAX232、SP3232 等电平转换芯片进行转换。
- 典型速率：
  - 常见波特率有9600、19200、38400、115200等

3. RS232接口与引脚定义

RS232通常使用**DB9**（9针）或**DB25**（25针）连接器，现代设备大多采用DB9。





32通常使用DB9（9针）或DB25（25针）连接器，现代设备大多采用DB9。

引脚	名称	方向	说明
1	DCD	输入	数据载波检测（Modem用）
2	RXD	输入	接收数据
3	TXD	输出	发送数据
4	DTR	输出	数据终端就绪
5	GND	-	信号地
6	DSR	输入	数据设备就绪
7	RTS	输出	请求发送
8	CTS	输入	清除发送
9	RI	输入	振铃指示（Modem用）

常用引脚：TXD（发送）、RXD（接收）、GND（地）是最基本的3线连接方式。

4. RS232接线方式

- TXD（PC1）→ RXD（PC2）
- RXD（PC1）→ TXD（PC2）
- GND（PC1）→ GND（PC2）

简化版（3线制）：仅连接TXD、RXD、GND即可实现基本通信。

5. RS232通信参数设置

串口通信需要双方设备配置相同的参数：

- 波特率（Baud Rate）：如9600、115200
- 数据位（Data Bits）：5、6、7、8（常用8位）
- 停止位（Stop Bits）：1、1.5、2（常用1位）
- 校验位（Parity）：None（无）、Odd（奇）、Even（偶）
- 示例配置：9600,8,N,1（波特率9600，8数据位，无校验，1停止位）

6. RS232 vs RS485 vs TTL

特性	RS232	RS485	TTL（UART）
通信方式	点对点	多点差分	点对点
电平标准	±12V	差分±2V	0V/3.3V或5V
传输距离	≤15m	≤1200m	≤1m
抗干扰	较弱	强	弱
典型应用	PC串口、旧设备	工业总线（Modbus）	单片机通信

注意：RS232不能直接与TTL（3.3V/5V）设备连接，需使用**MAX232**等电平转换芯片。

7. 现代应用

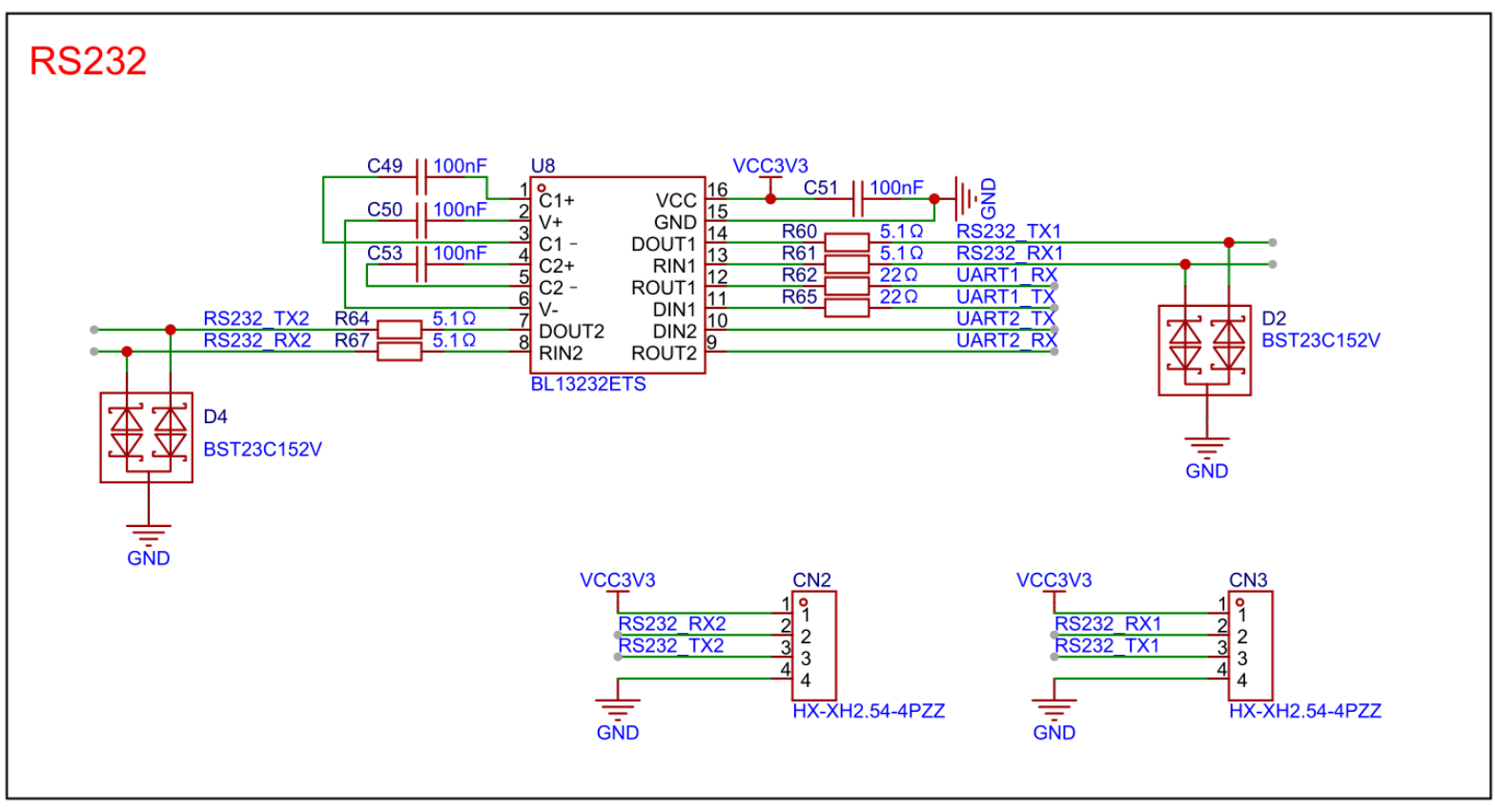
尽管在个人电脑上已被 USB 完全取代，RS-232 仍然活在这些地方：

- **工业自动化**：连接 PLC、变频器、数控机床等，因其稳定可靠、实时性好。
- **嵌入式调试**：开发板的串口控制台，工程师通过 RS-232 查看启动日志、输入指令。

现在的电脑基本不配原生串口，通常使用 **USB 转 RS-232 线缆**（内置 PL2303、FT232、CH340 等芯片）来模拟出虚拟串口。

13.3 RS232(UART1~2)驱动

1. RS2323原理图



RK3506 总共集成了 **6 个 UART 控制器。**但这不代表你最终能同时用满 6 个，因为芯片引脚大多为功能复用（IOMUX）：

串口编号	常用默认功能	说明
UART0	**调试串口（Debug UART）**	**系统默认控制台**，使用CH340做的串口，用来调试日志
UART1	通用串口	RS232第1路
UART2	通用串口	RS232第2路
UART3	通用串口	RS485第1路
UART4	通用串口	RS485第2路
UART5	未被启用	和网卡引脚冲突没有使用

2. RS232设备树

- kernel/arch/arm/boot/dts/rk3502.dtsi

```
    uart0: serial@ff0a0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0a0000 0x100>;
        interrupts = <GIC_SPI 34 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 4 0xff2880a8 0x03000100 0x0 0x0>,
               <&dmac0 5 0xff2880a8 0x0c000400 0x0 0x0>;
        clocks = <&cru SCLK_UART0>, <&cru PCLK_UART0>;
        clock-names = "baudclk", "apb_pclk";
        pinctrl-names = "default";
        pinctrl-0 = <&uart0_xfer_pins>;
        status = "disabled";
    };

    uart1: serial@ff0b0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0b0000 0x100>;
        interrupts = <GIC_SPI 35 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 6 0xff2880a8 0x30001000 0x0 0x0>,
               <&dmac0 7 0xff2880a8 0xc0004000 0x0 0x0>;
        clocks = <&cru SCLK_UART1>, <&cru PCLK_UART1>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };

    uart2: serial@ff0c0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0c0000 0x100>;
        interrupts = <GIC_SPI 36 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 8 0xff2880ac 0x00030001 0x0 0x0>,
               <&dmac0 9 0xff2880ac 0x000c0004 0x0 0x0>;
        clocks = <&cru SCLK_UART2>, <&cru PCLK_UART2>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };

    uart3: serial@ff0d0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0d0000 0x100>;
        interrupts = <GIC_SPI 37 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 10 0xff2880ac 0x00300010 0x0 0x0>,
               <&dmac0 11 0xff2880ac 0x00c00040 0x0 0x0>;
        clocks = <&cru SCLK_UART3>, <&cru PCLK_UART3>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };

    uart4: serial@ff0e0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0e0000 0x100>;
        interrupts = <GIC_SPI 38 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac1 12 0x0 0x0 0x0 0x0>, <&dmac1 13 0x0 0x0 0x0 0x0>;
        clocks = <&cru SCLK_UART4>, <&cru PCLK_UART4>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };
```

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
//232
&uart1 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io15_uart1_tx &rm_io16_uart1_rx>;
};

//232 && 和rgb冲突
&uart2 {
    status = "disabled";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io28_uart2_rx &rm_io24_uart2_tx>;
};
```

修改设备树，启用uart2的功能

```
//232
&uart1 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io15_uart1_tx &rm_io16_uart1_rx>;
};

//232 (RGB并口板需显式禁用, MIPI板无冲突)
&uart2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io28_uart2_rx &rm_io24_uart2_tx>;
};
```

3. UART驱动分析

RK3506 使用 **Synopsys DesignWare APB UART (DW_apb_uart)** IP 核，由标准 `8250_dw` 驱动管理，没有独立的 Rockchip 串口驱动文件。

- kernel/drivers/tty/serial/8250

```
kernel/drivers/tty/serial/8250/
8250_dw.c     ← 主驱动 (probe/irq/termios)
8250_dwlib.c  ← DW 扩展库 (分频/RS485/FIFO)
8250_dma.c    ← DMA 支持 (PL330 循环DMA)
8250_port.c   ← 8250 核心 (startup/shutdown)
```

```
linux@ubuntu:~/rk3506$ tree kernel/drivers/tty/serial/8250/
kernel/drivers/tty/serial/8250/
├── 8250_accent.c
├── 8250_acorn.c
├── 8250_alpha.c
├── 8250_aspeed_vuart.c
├── 8250_bcm2835aux.c
├── 8250_bcm7271.c
├── 8250_boca.c
├── 8250_core.c
├── 8250_core.o
├── 8250_dma.c                  #  DMA 支持 (PL330 循环DMA)
├── 8250_dma.o
├── 8250_dw.c                   # 主驱动 (probe/irq/termios)
├── 8250_dwlib.c
├── 8250_dwlib.h
├── 8250_dwlib.o
├── 8250_dw.o
├── 8250_pnp.c
├── 8250_port.c                #  8250 核心 (startup/shutdown)
├── 8250_port.o
linux@ubuntu:~/rk3506$
```

- kernel/drivers/tty/serial/8250/8250_dw.c

```
// SPDX-License-Identifier: GPL-2.0+
/*
 * Synopsys DesignWare 8250 driver.
 *
 * Copyright 2011 Picochip, Jamie Iles.
 * Copyright 2013 Intel Corporation
 *
 * The Synopsys DesignWare 8250 has an extra feature whereby it detects if the
 * LCR is written whilst busy.  If it is, then a busy detect interrupt is
 * raised, the LCR needs to be rewritten and the uart status register read.
 */
#include <linux/acpi.h>
#include <linux/clk.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/io.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/notifier.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/property.h>
#include <linux/reset.h>
#include <linux/slab.h>
#include <linux/workqueue.h>

#include <asm/byteorder.h>

#include <linux/serial_8250.h>
#include <linux/serial_reg.h>

#ifdef MODULE
#include "8250_dwlib.c"
#else
#include "8250_dwlib.h"
#endif

/* Offsets for the DesignWare specific registers */
#define DW_UART_USR     0x1f /* UART Status Register */
#define DW_UART_DMASA   0xa8 /* DMA Software Ack */
#define DW_UART_RFL     0x21 /* UART Receive Fifo Level Register */

#define OCTEON_UART_USR 0x27 /* UART Status Register */

#define RZN1_UART_TDMACR 0x10c /* DMA Control Register Transmit Mode */
#define RZN1_UART_RDMACR 0x110 /* DMA Control Register Receive Mode */

/* DesignWare specific register fields */
#define DW_UART_MCR_SIRE                BIT(6)

/* Renesas specific register fields */
#define RZN1_UART_xDMACR_DMA_EN         BIT(0)
#define RZN1_UART_xDMACR_1_WORD_BURST   (0 << 1)
#define RZN1_UART_xDMACR_4_WORD_BURST   (1 << 1)
#define RZN1_UART_xDMACR_8_WORD_BURST   (2 << 1)
#define RZN1_UART_xDMACR_BLK_SZ(x)      ((x) << 3)
static const struct of_device_id dw8250_of_match[] = {
        { .compatible = "snps,dw-apb-uart", .data = &dw8250_dw_apb },
#ifndef CONFIG_ROCKCHIP_MINI_KERNEL
        { .compatible = "cavium,octeon-3860-uart", .data = &dw8250_octeon_3860_data },
        { .compatible = "marvell,armada-38x-uart", .data = &dw8250_armada_38x_data },
        { .compatible = "renesas,rzn1-uart", .data = &dw8250_renesas_rzn1_data },
        { .compatible = "starfive,jh7100-uart", .data = &dw8250_starfive_jh7100_data },
#endif
        { /* Sentinel */ }
};
MODULE_DEVICE_TABLE(of, dw8250_of_match);

static const struct acpi_device_id dw8250_acpi_match[] = {
        { "80860F0A", (kernel_ulong_t)&dw8250_dw_apb },
        { "8086228A", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMD0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0022", (kernel_ulong_t)&dw8250_dw_apb },
        { "APMC0D08", (kernel_ulong_t)&dw8250_dw_apb},
        { "BRCM2032", (kernel_ulong_t)&dw8250_dw_apb },
        { "HISI0031", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C4", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C5", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3434", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3435", (kernel_ulong_t)&dw8250_dw_apb },
        { "INTC10EE", (kernel_ulong_t)&dw8250_dw_apb },
        { },
};
MODULE_DEVICE_TABLE(acpi, dw8250_acpi_match);

static struct platform_driver dw8250_platform_driver = {
        .driver = {
                .name           = "dw-apb-uart",
                .pm             = pm_ptr(&dw8250_pm_ops),
                .of_match_table = dw8250_of_match,
                .acpi_match_table = dw8250_acpi_match,
        },
        .probe                  = dw8250_probe,
        .remove                 = dw8250_remove,
};

module_platform_driver(dw8250_platform_driver);

MODULE_AUTHOR("Jamie Iles");
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Synopsys DesignWare 8250 serial port driver");
MODULE_ALIAS("platform:dw-apb-uart");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    Character devices  --->
        Serial drivers  --->
              <*> 8250/16550 and compatible serial support
              [*]   Support for variants of the 16550A serial port
              [*]   Console on 8250/16550 and compatible serial port
              (6)   Maximum number of 8250/16550 serial ports
              (6)   Number of 8250/16550 serial ports to register at runtime
              <*> Support for Synopsys DesignWare 8250 quirks
linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

编译内核更新设备树与内核到开发板内。

4. RS232功能测试

- 查看串口设备

```
root@rk3506-buildroot:/# ls /dev/ttyS*
/dev/ttyS1  /dev/ttyS2  /dev/ttyS3
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ls -l /dev/ttyS*
crw-rw----    1 root     dialout     4,  65 Jun 28 16:46 /dev/ttyS1              # RS232的UART1
crw-rw----    1 root     dialout     4,  66 Jun 28 16:46 /dev/ttyS2              # RS232的UART2
crw-rw----    1 root     dialout     4,  67 Jun 28 16:46 /dev/ttyS3
root@rk3506-buildroot:/#
```

microcom 是 BusyBox 内置的极简串口工具，支持波特率设置、文件发送与十六进制模式，适合嵌入式调试。

- 启动 microcom

```
microcom -s 115200 /dev/ttyS1
```

说明：波特率 115200，8N1 是默认帧格式，通常不用改。

- 执行自环测试

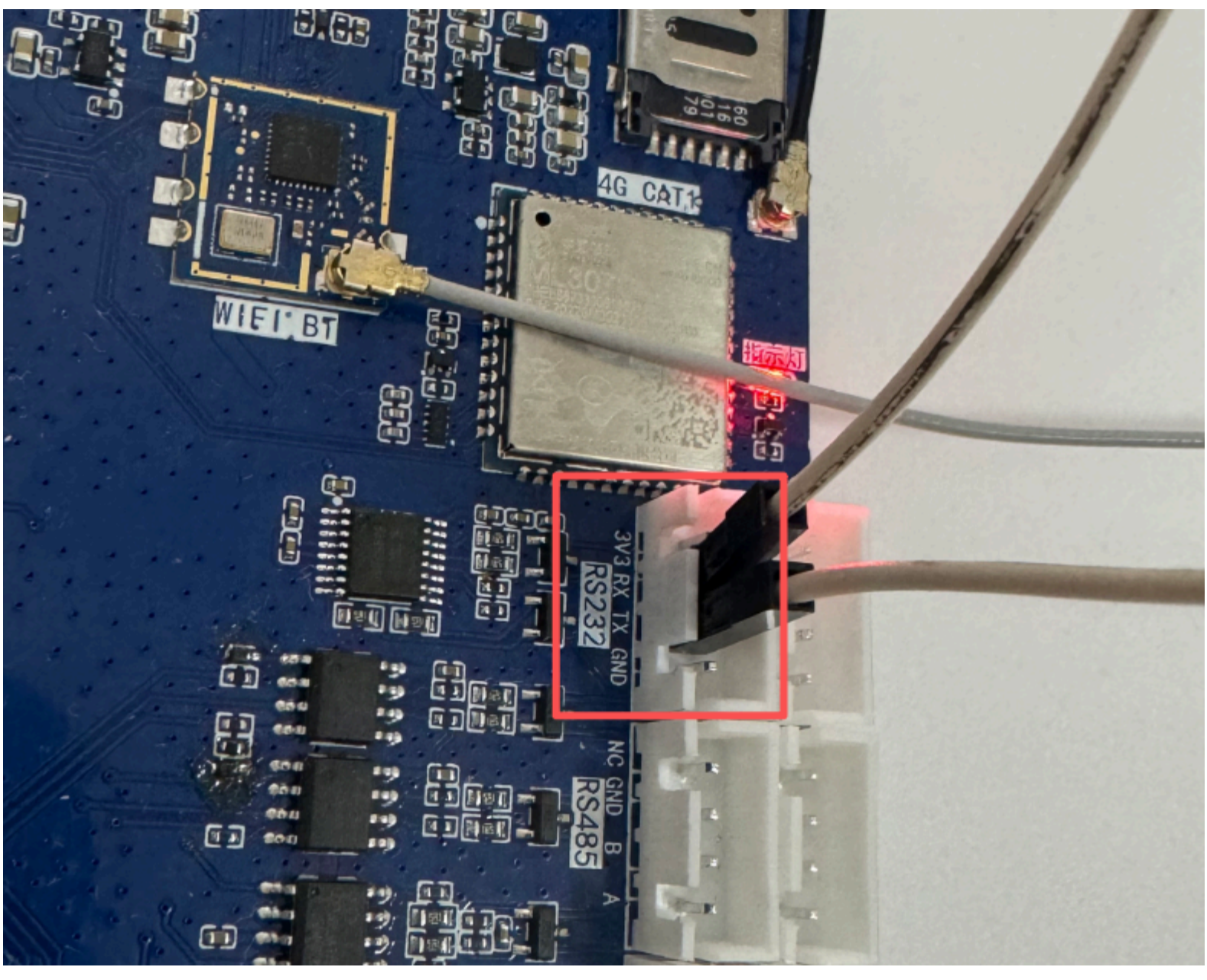
启动后，键盘输入 `abc123`。

运行结果：

```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS1
# 输入字符什么都不显示，这是测试数据发送没有收到的现象
```

- **正常情况**：由于本地回显已关，你按下的字符不会立即出现；当环回数据返回时，屏幕上会显示 `abc123`。
- **故障情况**：如果没有任何字符出现，说明通路有问题（检查跳线、设备节点、波特率）。

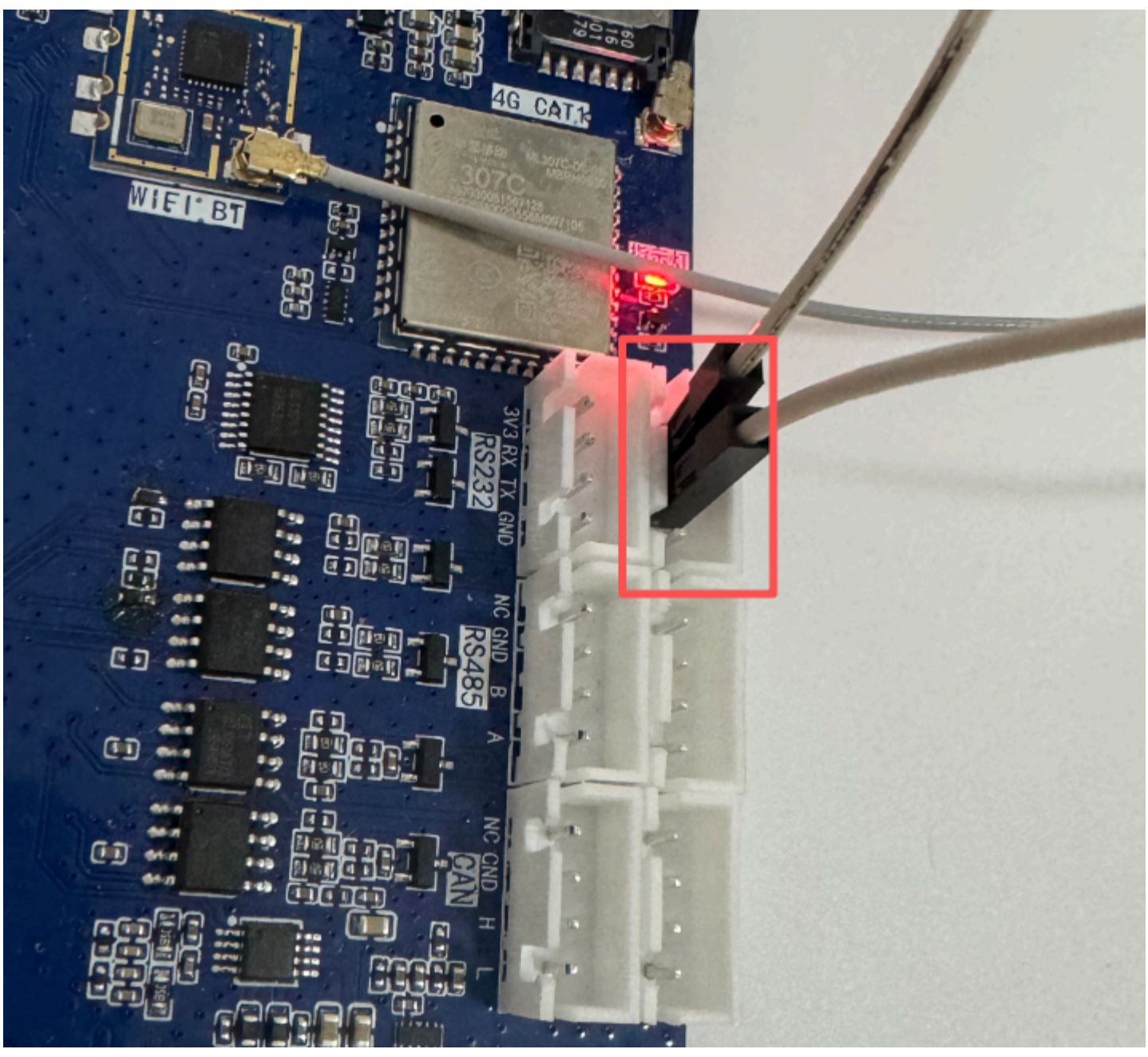
短接UART1的TX和RX， 如下图：



- 接线后再次输入adc123

```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS1
[  739.802657] of_dma_request_slave_channel: dma-names property of node '/serial@ff0c0000' missing or empty
[  739.802714] dw-apb-uart ff0c0000.serial: failed to request DMA, use interrupt mode
abc123
root@rk3506-buildroot:/#   # Ctrl + X 退出命令
```

- 短接UART2的TX和RX



- 测试UART2功能

```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS2
[  739.802657] of_dma_request_slave_channel: dma-names property of node '/serial@ff0c0000' missing or empty
[  739.802714] dw-apb-uart ff0c0000.serial: failed to request DMA, use interrupt mode
abc123
root@rk3506-buildroot:/#   # Ctrl + X 退出命令
```

- 退出 microcom

默认快捷键是 `Ctrl + X`（按住 Ctrl 再按 X）。终端会直接回到命令行。

13.5 RS485电平标准

1. RS485总线协议介绍

串口(UART)的电平协议: TTL ， 电压范围 `0-3.3V`

RS232的电平协议: 232，电压范围 `+-25V` ， 串口的高电压版本 。

RS485的电平协议: 485，电压范围 `+-6V` ， 半双工差分信号传输。

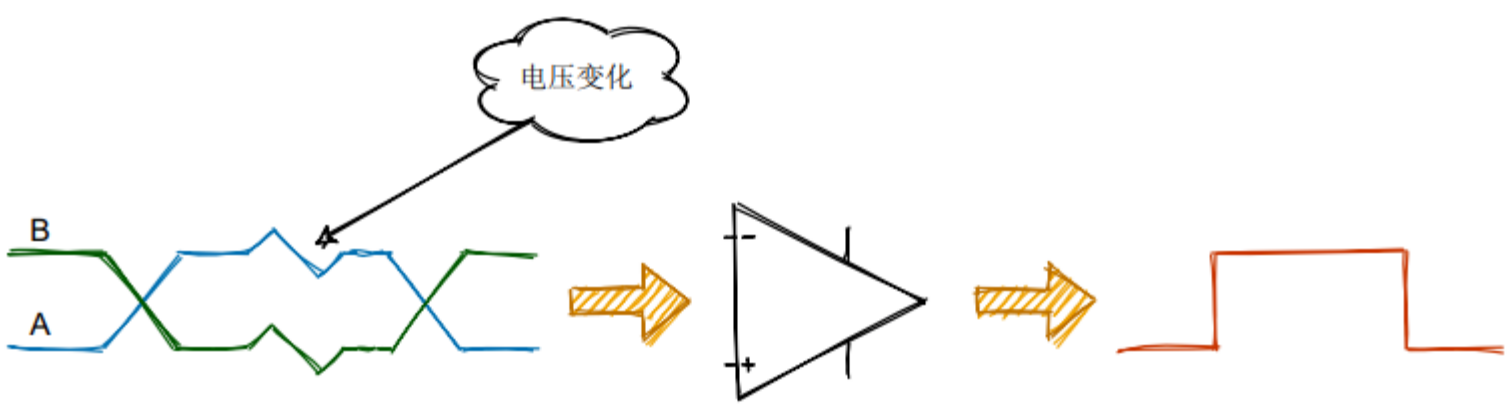
RS422的电平协议: 422，电压范围 `+-6V` , 全双工差分信号传输，2个485总线。

RS-485和RS-232一样，都是串行通信标准，现在的标准名称是TIA485/EIA-485-A，但是人们会习惯称为RS-485标准，RS-485常用在工业、自动化、汽车和建筑物管理等领域。

RS-485总线弥补了RS-232通信距离短，速率低的缺点，RS-485的速率可高达10Mbit/s，理论通讯距离可达1200米；

2. 差分传输

长距离布线会有信号衰减，而且引入噪声和干扰的可能性更大，在线缆A和B上的表现就是电压幅度的变化，但是，采用差分线的好处就是，差值相减就会忽略掉干扰依旧能输出正常的信号，把这种差分接收器忽略两条信号线上相同电压的能力称为共模抑制。

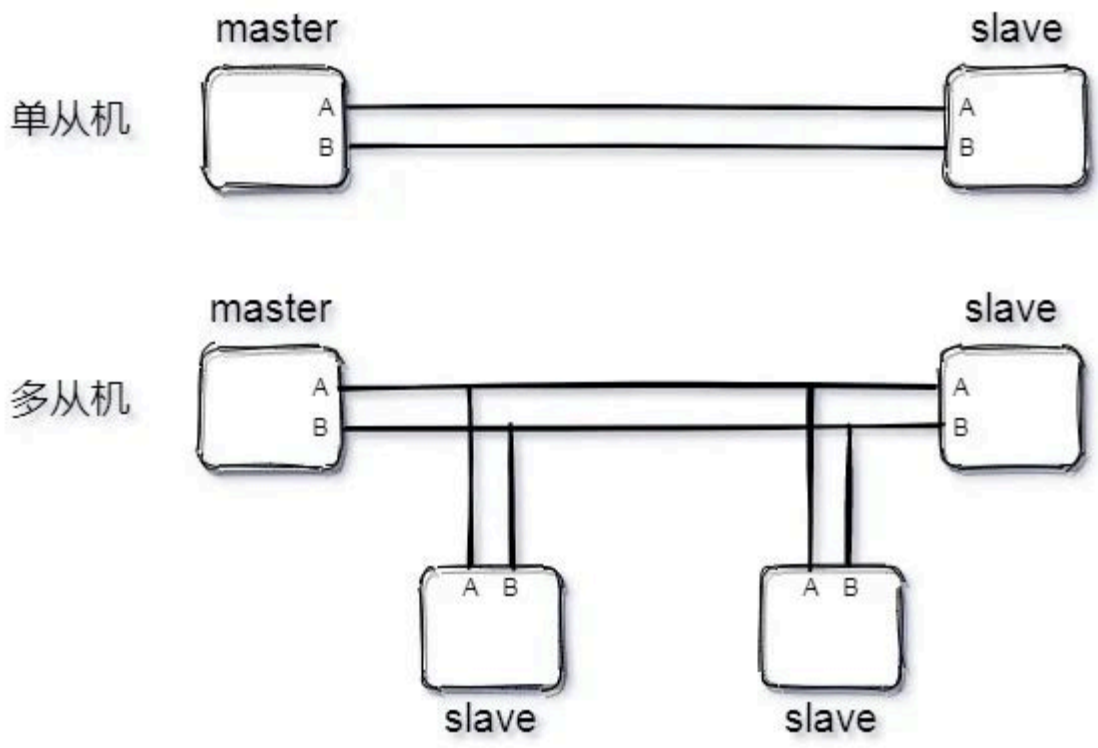


标准规定了，逻辑1：+2V to +6V；逻辑0：-6V to -2V。

3. 组网结构

RS485有两线制和四线制两种接线，四线制只能实现点对点的通信方式，现很少采用，多采用的是两线制接线方式，这种接线方式为总线拓扑结构，在同一总线上最多可以挂接32个节点。

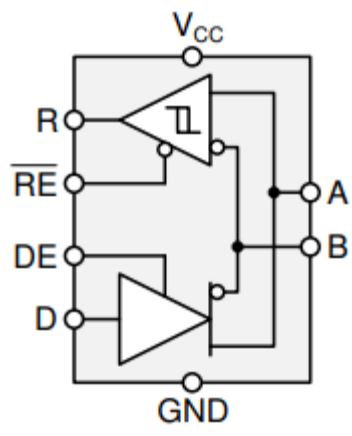
RS-485总线同I2C，也是主从模式，支持点对点单从机模式，也支持多从机模式，不支持多主机模式。



4. RS-485收发器

RS-485是差分传输，如果用单片机控制RS-485接口的设备，需要用到收发器，这一点和CAN总线是类似的，如下是一个MCU控制一个RS-485的图示。

收发器内部是一个接收器（上半部分）加一个发送器（下半部分），下面简单说说收发器的原理，便于理解MCU是如何和485设备通信的。



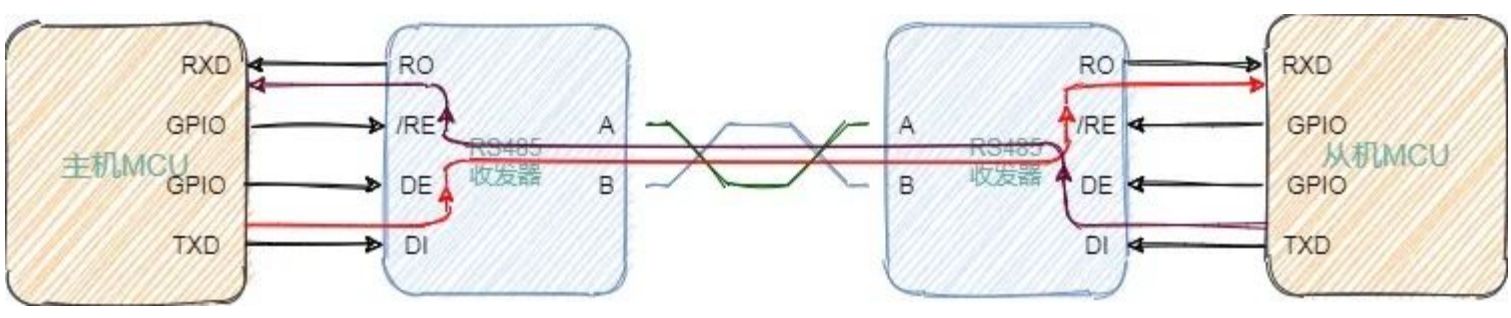
RS-485收发器内部结，其中：

- A和B为总线；
- R为接收器输入；
- RE为接收器使能信号；
- DE为发送器使能信号；
- D为发送器输出；

对于使能信号，字母上面加一横的为低电平有效（如上图RE），不加的为高电平有效（如DE）。

5. RS-485数据传输链路

上面讲到的RS-485收发器的工作原理，下面简单描述RS-485的数据链路。



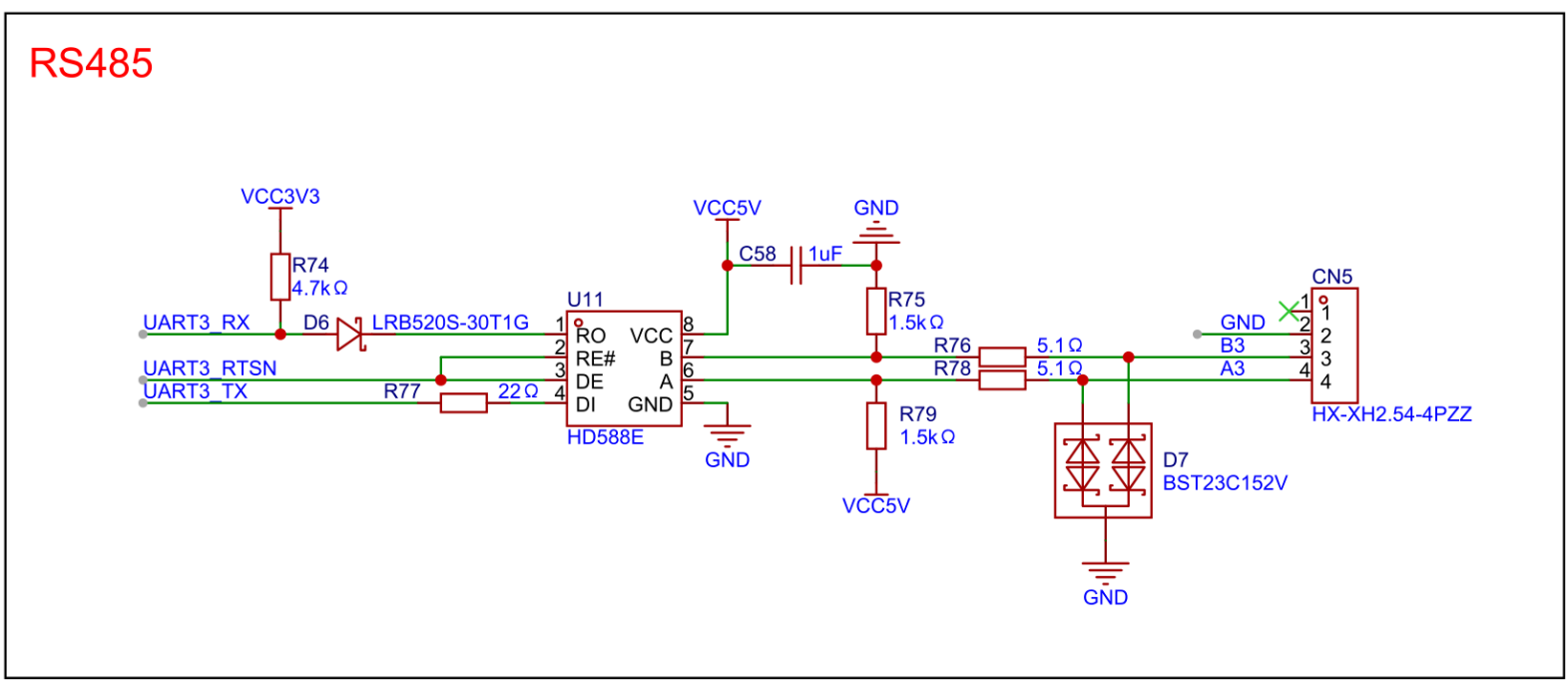
6. RS-485和CAN的区别

虽说RS-485没有标准的数据协议格式，但和CAN总线在很多地方是有相似的，比如A&B和CANH&CANL都是差分信号，通信都需要收发器，都需要120欧姆的匹配电阻等等。

总线特性	CAN总线	RS-485总线
硬件成本	稍高	低廉
总线利用率	优先级自动仲裁，利用率高	采用轮询，利用率低
数据传输率	高	低
错误检测机制	控制器带校验机制，保证底层数据传输正确	只有物理层规范，无数据链路层规定
单节点故障影响	总线无影响	总线瘫痪
开发成本	软件开发灵活，时间成本低	开发难度较大
系统成本	较低	高

13.6 RS485(UART3~4)驱动

1. RS485原理图



RK3506 总共集成了 **6 个 UART 控制器。**但这不代表你最终能同时用满 6 个，因为芯片引脚大多为功能复用（IOMUX）：

串口编号	常用默认功能	说明
UART0	**调试串口（Debug UART）**	**系统默认控制台**，使用CH340做的串口，用来调试日志
UART1	通用串口	RS232第1路
UART2	通用串口	RS232第2路
UART3	通用串口	RS485第1路
UART4	通用串口	RS485第2路

串口编号	常用默认功能	说明
UART5	未被启用	和网卡引脚冲突没有使用

2. RS485设备树

- kernel/arch/arm/boot/dts/rk3502.dtsi

```
    uart3: serial@ff0d0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0d0000 0x100>;
        interrupts = <GIC_SPI 37 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 10 0xff2880ac 0x00300010 0x0 0x0>,
               <&dmac0 11 0xff2880ac 0x00c00040 0x0 0x0>;
        clocks = <&cru SCLK_UART3>, <&cru PCLK_UART3>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };

    uart4: serial@ff0e0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0e0000 0x100>;
        interrupts = <GIC_SPI 38 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac1 12 0x0 0x0 0x0 0x0>, <&dmac1 13 0x0 0x0 0x0 0x0>;
        clocks = <&cru SCLK_UART4>, <&cru PCLK_UART4>;
        clock-names = "baudclk", "apb_pclk";
        status = "disabled";
    };
```

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
//485
&uart3 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io6_uart3_tx &rm_io7_uart3_rx &rm_io5_uart3_rtsn>;
    linux,rs485-enabled-at-boot-time;   // 启动时直接进入RS485模式
};

//485 && 和rgb冲突
&uart4 {
    status = "disabled";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io30_uart4_rx &rm_io31_uart4_tx &rm_io29_uart4_rtsn>;
    linux,rs485-enabled-at-boot-time;   // 启动时直接进入RS485模式
};
```

修改设备树，启用uart4的功能

```
//485
&uart3 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io6_uart3_tx &rm_io7_uart3_rx &rm_io5_uart3_rtsn>;
    linux,rs485-enabled-at-boot-time;
};

//485 && 和rgb冲突 (MIPI板可用, RGB板需禁用)
&uart4 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io30_uart4_rx &rm_io31_uart4_tx &rm_io29_uart4_rtsn>;
    linux,rs485-enabled-at-boot-time;
};
```

3. UART驱动分析

RK3506 使用 **Synopsys DesignWare APB UART (DW_apb_uart)** IP 核，由标准 `8250_dw` 驱动管理，没有独立的 Rockchip 串口驱动文件。

- kernel/drivers/tty/serial/8250

```
kernel/drivers/tty/serial/8250/
8250_dw.c     ← 主驱动 (probe/irq/termios)
8250_dwlib.c  ← DW 扩展库 (分频/RS485/FIFO)
8250_dma.c    ← DMA 支持 (PL330 循环DMA)
8250_port.c   ← 8250 核心 (startup/shutdown)
linux@ubuntu:~/rk3506$ tree kernel/drivers/tty/serial/8250/
kernel/drivers/tty/serial/8250/
├── 8250_accent.c
├── 8250_acorn.c
├── 8250_alpha.c
├── 8250_aspeed_vuart.c
├── 8250_bcm2835aux.c
├── 8250_bcm7271.c
├── 8250_boca.c
├── 8250_core.c
├── 8250_core.o
├── 8250_dma.c                  #  DMA 支持 (PL330 循环DMA)
├── 8250_dma.o
├── 8250_dw.c                   # 主驱动 (probe/irq/termios)
├── 8250_dwlib.c
├── 8250_dwlib.h
├── 8250_dwlib.o
├── 8250_dw.o
├── 8250_pnp.c
├── 8250_port.c                #  8250 核心 (startup/shutdown)
├── 8250_port.o
linux@ubuntu:~/rk3506$
```

- kernel/drivers/tty/serial/8250/8250_dw.c

```
// SPDX-License-Identifier: GPL-2.0+
/*
 * Synopsys DesignWare 8250 driver.
 *
 * Copyright 2011 Picochip, Jamie Iles.
 * Copyright 2013 Intel Corporation
 *
 * The Synopsys DesignWare 8250 has an extra feature whereby it detects if the
 * LCR is written whilst busy.  If it is, then a busy detect interrupt is
 * raised, the LCR needs to be rewritten and the uart status register read.
 */
#include <linux/acpi.h>
#include <linux/clk.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/io.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/notifier.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/property.h>
#include <linux/reset.h>
#include <linux/slab.h>
#include <linux/workqueue.h>

#include <asm/byteorder.h>

#include <linux/serial_8250.h>
#include <linux/serial_reg.h>

#ifdef MODULE
#include "8250_dwlib.c"
#else
#include "8250_dwlib.h"
#endif

/* Offsets for the DesignWare specific registers */
#define DW_UART_USR     0x1f /* UART Status Register */
#define DW_UART_DMASA   0xa8 /* DMA Software Ack */
#define DW_UART_RFL     0x21 /* UART Receive Fifo Level Register */

#define OCTEON_UART_USR 0x27 /* UART Status Register */

#define RZN1_UART_TDMACR 0x10c /* DMA Control Register Transmit Mode */
#define RZN1_UART_RDMACR 0x110 /* DMA Control Register Receive Mode */
```

```
/* DesignWare specific register fields */
#define DW_UART_MCR_SIRE                BIT(6)

/* Renesas specific register fields */
#define RZN1_UART_xDMACR_DMA_EN         BIT(0)
#define RZN1_UART_xDMACR_1_WORD_BURST   (0 << 1)
#define RZN1_UART_xDMACR_4_WORD_BURST   (1 << 1)
#define RZN1_UART_xDMACR_8_WORD_BURST   (2 << 1)
#define RZN1_UART_xDMACR_BLK_SZ(x)      ((x) << 3)
static const struct of_device_id dw8250_of_match[] = {
        { .compatible = "snps,dw-apb-uart", .data = &dw8250_dw_apb },
#ifndef CONFIG_ROCKCHIP_MINI_KERNEL
        { .compatible = "cavium,octeon-3860-uart", .data = &dw8250_octeon_3860_data },
        { .compatible = "marvell,armada-38x-uart", .data = &dw8250_armada_38x_data },
        { .compatible = "renesas,rzn1-uart", .data = &dw8250_renesas_rzn1_data },
        { .compatible = "starfive,jh7100-uart", .data = &dw8250_starfive_jh7100_data },
#endif
        { /* Sentinel */ }
};
MODULE_DEVICE_TABLE(of, dw8250_of_match);

static const struct acpi_device_id dw8250_acpi_match[] = {
        { "80860F0A", (kernel_ulong_t)&dw8250_dw_apb },
        { "8086228A", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMD0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0022", (kernel_ulong_t)&dw8250_dw_apb },
        { "APMC0D08", (kernel_ulong_t)&dw8250_dw_apb},
        { "BRCM2032", (kernel_ulong_t)&dw8250_dw_apb },
        { "HISI0031", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C4", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C5", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3434", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3435", (kernel_ulong_t)&dw8250_dw_apb },
        { "INTC10EE", (kernel_ulong_t)&dw8250_dw_apb },
        { },
};
MODULE_DEVICE_TABLE(acpi, dw8250_acpi_match);
```

```
static struct platform_driver dw8250_platform_driver = {
        .driver = {
                .name           = "dw-apb-uart",
                .pm             = pm_ptr(&dw8250_pm_ops),
                .of_match_table = dw8250_of_match,
                .acpi_match_table = dw8250_acpi_match,
        },
        .probe                  = dw8250_probe,
        .remove                 = dw8250_remove,
};

module_platform_driver(dw8250_platform_driver);

MODULE_AUTHOR("Jamie Iles");
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Synopsys DesignWare 8250 serial port driver");
MODULE_ALIAS("platform:dw-apb-uart");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    Character devices  --->
        Serial drivers  --->
              <*> 8250/16550 and compatible serial support
              [*]   Support for variants of the 16550A serial port
              [*]   Console on 8250/16550 and compatible serial port
              (6)   Maximum number of 8250/16550 serial ports
              (6)   Number of 8250/16550 serial ports to register at runtime
              <*> Support for Synopsys DesignWare 8250 quirks
linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

4. RS485自动发送接收

```
linux,rs485-enabled-at-boot-time // 启动时直接进入RS485模式

    // 驱动的probe函数
probe: serial8250_register_8250_port()
        -> uart_get_rs485_mode()              [serial_core.c:3446]
        //设置 port->rs485.flags |= SER_RS485_ENABLED

configure: uart_configure_port()           [serial_core.c:2551]
        -> set_mctrl     // 被跳过 (RS485模式下软件不控RTS)
        -> uart_rs485_config(port)             [serial_core.c:1405]
            -> port->rs485_config() = dw8250_rs485_config()  [8250_dwlib.c:191]
              -> // 写 TCR 寄存器: RS485_EN | AUTO_DE
```

- linux,rs485-enabled-at-boot-time 属性的意义

```
// 有 linux,rs485-enabled-at-boot-time

   uart_get_rs485_mode(): flags |= SER_RS485_ENABLED

   uart_rs485_config():  // 继续执行

   dw8250_rs485_config():  // 写 TCR 寄存器
   TCR = RS485_EN | AUTO_DE  (BIT0 | BIT5)
   set_mctrl() → return;  // 直接返回，不碰 RTS

// RS485 半双工模式，RTSN 由硬件自动控制
```

核心结论：`linux,rs485-enabled-at-boot-time` 导致驱动在 probe 阶段就通过 `dw8250_rs485_config()` 写 TCR 寄存器启用硬件 AUTO_DE，同时封堵所有软件对 RTS 的控制路径（`set_mctrl`），确保 RTSN 引脚完全由硬件自动管理收发方向。

- 编译内核更新设备树与内核到开发板内。

5. RS485功能测试

- 查看串口设备

```
root@rk3506-buildroot:/# ls /dev/ttyS*
/dev/ttyS1  /dev/ttyS2  /dev/ttyS3  /dev/ttyS4
root@rk3506-buildroot:/# ls -l /dev/ttyS*
crw-rw----    1 root     dialout     4,  65 Jun 28 21:35 /dev/ttyS1
crw-rw----    1 root     dialout     4,  66 Jun 28 21:35 /dev/ttyS2
crw-rw----    1 root     dialout     4,  67 Jun 28 21:35 /dev/ttyS3
crw-rw----    1 root     dialout     4,  68 Jun 28 21:35 /dev/ttyS4
root@rk3506-buildroot:/#
```

microcom 是 BusyBox 内置的极简串口工具，支持波特率设置、文件发送与十六进制模式，适合嵌入式调试。

- 启动 microcom

```
microcom -s 115200 /dev/ttyS3
```

说明：波特率 115200，8N1 是默认帧格式，通常不用改。

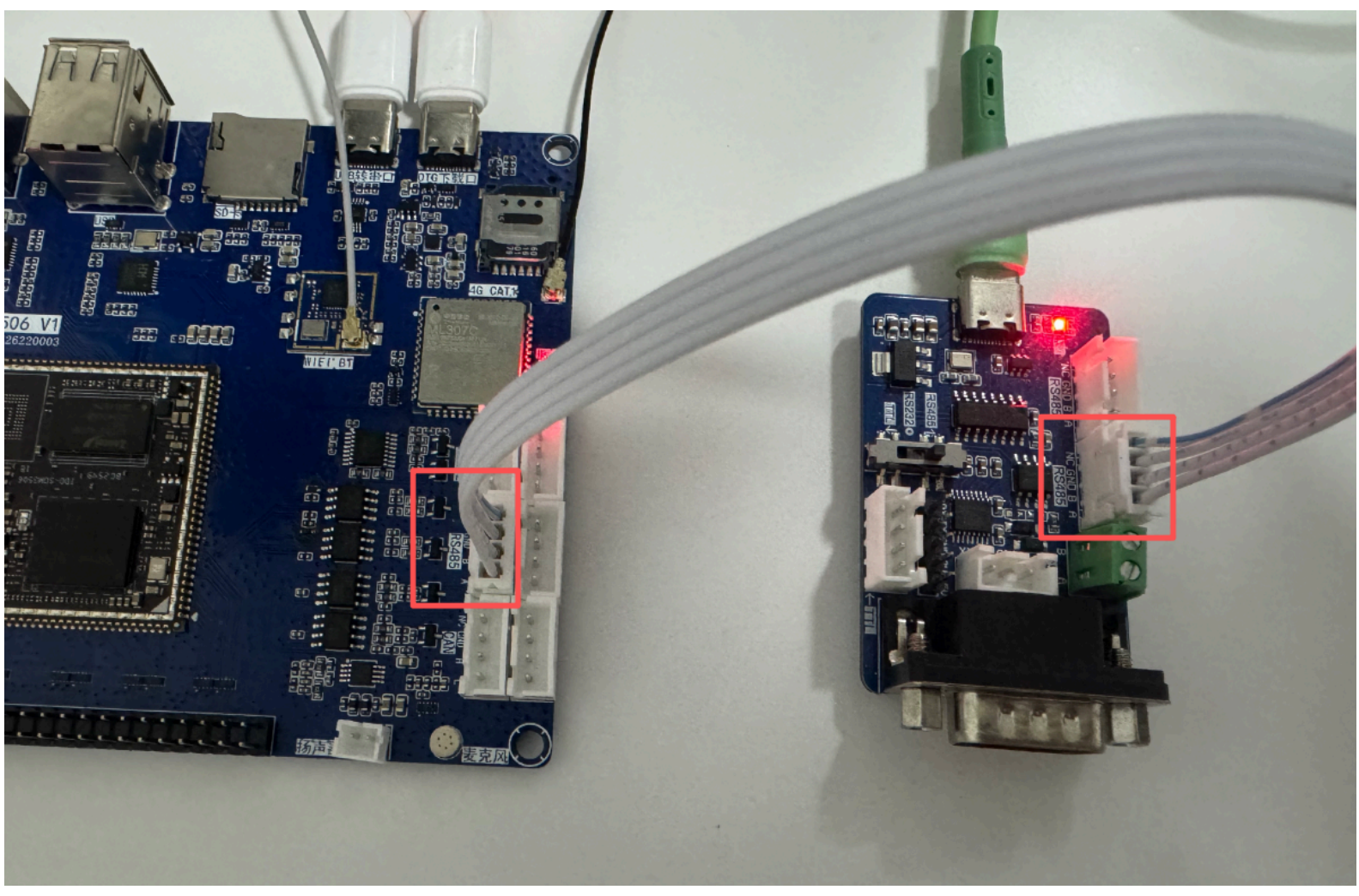
- 执行自环测试

启动后，键盘输入 `abc123`。

运行结果：

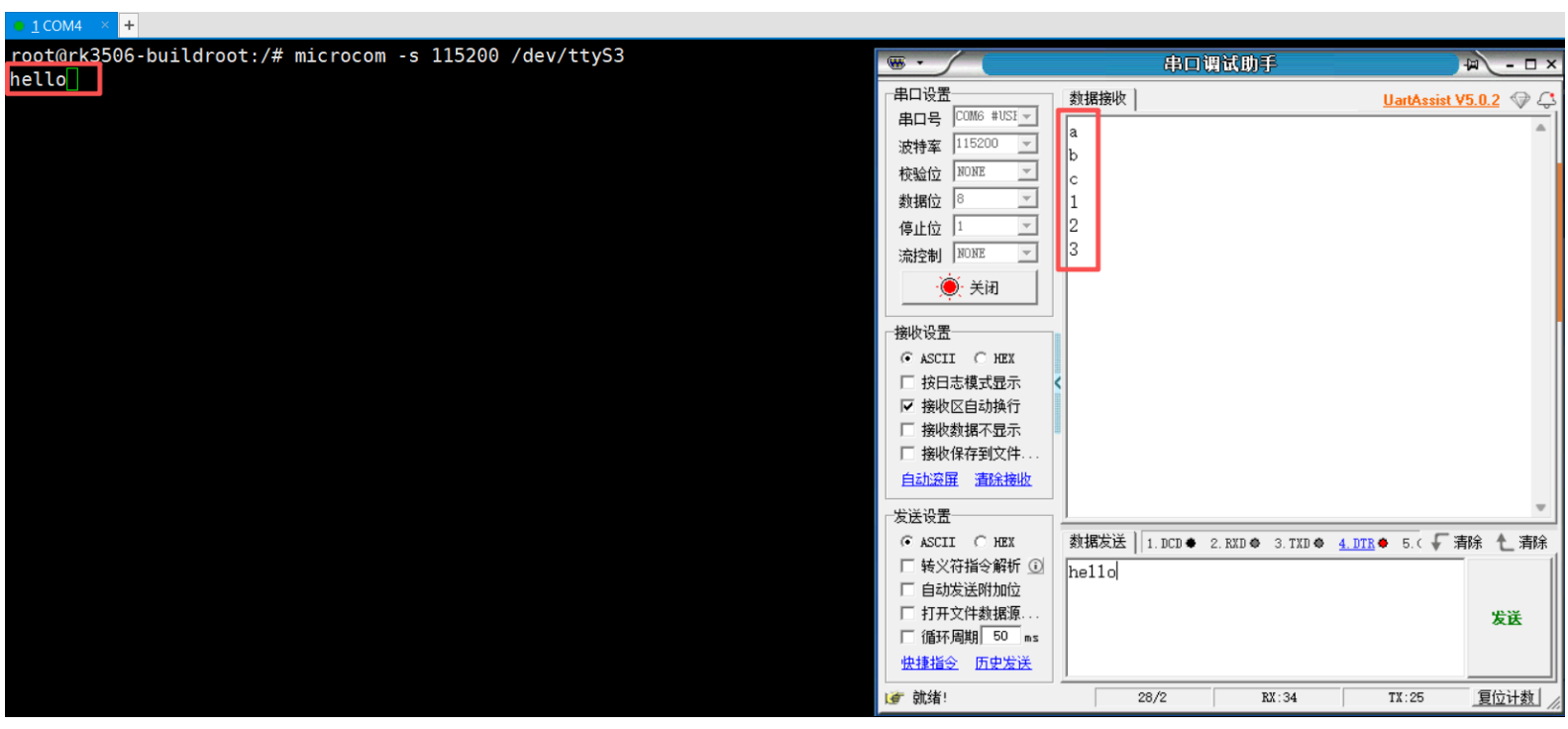
```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS1
```

接线如下图：



- 接线后再次输入adc123

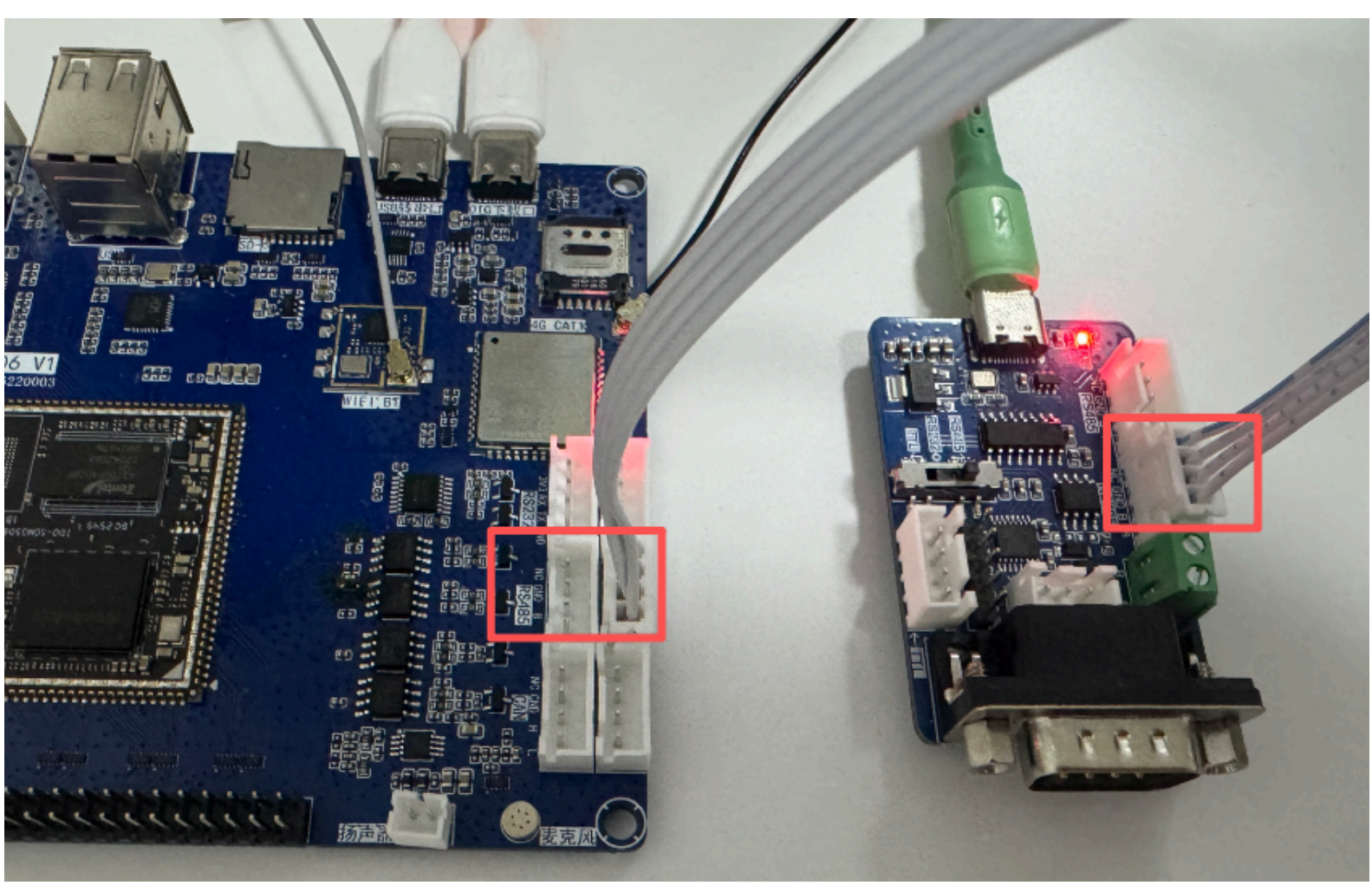
```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS3
[  739.802657] of_dma_request_slave_channel: dma-names property of node '/serial@ff0c0000' missing or empty
[  739.802714] dw-apb-uart ff0c0000.serial: failed to request DMA, use interrupt mode
hello
root@rk3506-buildroot:/#   # Ctrl + X 退出命令
```



- 测试UART4功能

```
root@rk3506-buildroot:/# microcom -s 115200 /dev/ttyS4
[  739.802657] of_dma_request_slave_channel: dma-names property of node '/serial@ff0c0000' missing or empty
[  739.802714] dw-apb-uart ff0c0000.serial: failed to request DMA, use interrupt mode
abc123
root@rk3506-buildroot:/#   # Ctrl + X 退出命令
```

- 硬件接线如下：

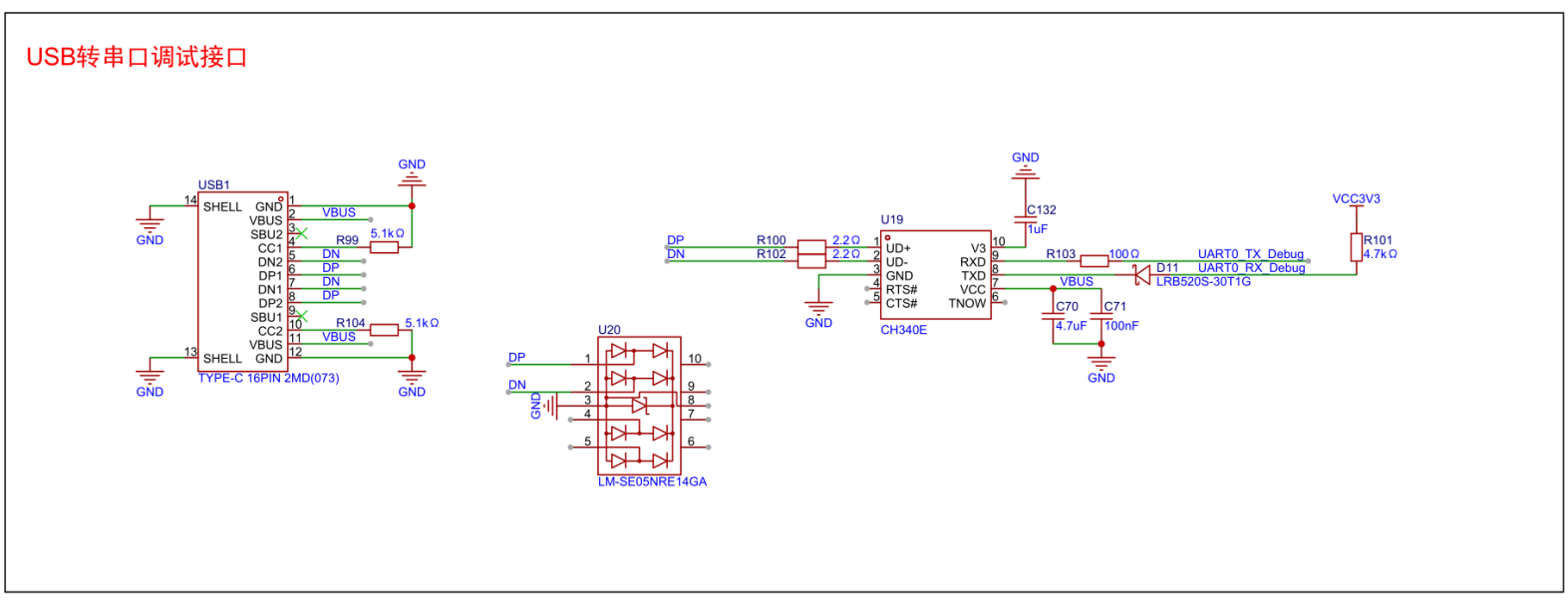


- 退出 microcom

默认快捷键是 `Ctrl + X`（按住 Ctrl 再按 X）。终端会直接回到命令行。

13.7 CH340(UART0)驱动

1. CH340原理图



RK3506 总共集成了 **6 个 UART 控制器。**但这不代表你最终能同时用满 6 个，因为芯片引脚大多为功能复用（IOMUX）：

串口编号	常用默认功能	说明
UART0	**调试串口（Debug UART）**	**系统默认控制台**，使用CH340做的串口，用来调试日志
UART1	通用串口	RS232第1路
UART2	通用串口	RS232第2路
UART3	通用串口	RS485第1路
UART4	通用串口	RS485第2路
UART5	未被启用	和网卡引脚冲突没有使用

2. CH340(UART0)设备树

- kernel/arch/arm/boot/dts/rk3502.dtsi

```
    uart0: serial@ff0a0000 {
        compatible = "rockchip,rk3506-uart", "snps,dw-apb-uart";
        reg = <0xff0a0000 0x100>;
        interrupts = <GIC_SPI 34 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        dmas = <&dmac0 4 0xff2880a8 0x03000100 0x0 0x0>,
               <&dmac0 5 0xff2880a8 0x0c000400 0x0 0x0>;
        clocks = <&cru SCLK_UART0>, <&cru PCLK_UART0>;
        clock-names = "baudclk", "apb_pclk";
        pinctrl-names = "default";
        pinctrl-0 = <&uart0_xfer_pins>;
        status = "disabled";
    };
```

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    fiq_debugger: fiq-debugger {
        compatible = "rockchip,fiq-debugger";
        rockchip,serial-id = <0>;
        rockchip,wake-irq = <0>;
```

```
        rockchip,irq-mode-enable = <1>;
        rockchip,baudrate = <1500000>;  /* Only 115200 and 1500000 */
        interrupts = <GIC_SPI 115 IRQ_TYPE_LEVEL_HIGH>;
    };

/*
fiq_debugger: fiq-debugger {
    compatible = "rockchip,fiq-debugger";
    // 没有 status → 默认 "okay" → 驱动正常 probe
};
只有需要禁用时才写 status = "disabled"。
*/
```

3. UART驱动分析

RK3506 使用 **Synopsys DesignWare APB UART (DW_apb_uart)** IP 核，由标准 `8250_dw` 驱动管理，没有独立的 Rockchip 串口驱动文件。

- kernel/drivers/tty/serial/8250

```
kernel/drivers/tty/serial/8250/
8250_dw.c     ← 主驱动 (probe/irq/termios)
8250_dwlib.c  ← DW 扩展库 (分频/RS485/FIFO)
8250_dma.c    ← DMA 支持 (PL330 循环DMA)
8250_port.c   ← 8250 核心 (startup/shutdown)
linux@ubuntu:~/rk3506$ tree kernel/drivers/tty/serial/8250/
kernel/drivers/tty/serial/8250/
├── 8250_accent.c
├── 8250_acorn.c
├── 8250_alpha.c
├── 8250_aspeed_vuart.c
├── 8250_bcm2835aux.c
├── 8250_bcm7271.c
├── 8250_boca.c
├── 8250_core.c
├── 8250_core.o
├── 8250_dma.c                  #  DMA 支持 (PL330 循环DMA)
├── 8250_dma.o
├── 8250_dw.c                   # 主驱动 (probe/irq/termios)
├── 8250_dwlib.c
├── 8250_dwlib.h
├── 8250_dwlib.o
├── 8250_dw.o
├── 8250_pnp.c
├── 8250_port.c                 #  8250 核心 (startup/shutdown)
├── 8250_port.o
linux@ubuntu:~/rk3506$
```

- kernel/drivers/tty/serial/8250/8250_dw.c

```
// SPDX-License-Identifier: GPL-2.0+
/*
 * Synopsys DesignWare 8250 driver.
 *
 * Copyright 2011 Picochip, Jamie Iles.
 * Copyright 2013 Intel Corporation
 *
 * The Synopsys DesignWare 8250 has an extra feature whereby it detects if the
 * LCR is written whilst busy.  If it is, then a busy detect interrupt is
 * raised, the LCR needs to be rewritten and the uart status register read.
 */
#include <linux/acpi.h>
#include <linux/clk.h>
#include <linux/delay.h>
#include <linux/device.h>
#include <linux/io.h>
#include <linux/mod_devicetable.h>
#include <linux/module.h>
#include <linux/notifier.h>
#include <linux/of.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/property.h>
#include <linux/reset.h>
#include <linux/slab.h>
#include <linux/workqueue.h>

#include <asm/byteorder.h>

#include <linux/serial_8250.h>
#include <linux/serial_reg.h>

#ifdef MODULE
#include "8250_dwlib.c"
#else
#include "8250_dwlib.h"
#endif

/* Offsets for the DesignWare specific registers */
#define DW_UART_USR     0x1f /* UART Status Register */
#define DW_UART_DMASA   0xa8 /* DMA Software Ack */
#define DW_UART_RFL     0x21 /* UART Receive Fifo Level Register */

#define OCTEON_UART_USR 0x27 /* UART Status Register */

#define RZN1_UART_TDMACR 0x10c /* DMA Control Register Transmit Mode */
#define RZN1_UART_RDMACR 0x110 /* DMA Control Register Receive Mode */

/* DesignWare specific register fields */
#define DW_UART_MCR_SIRE                BIT(6)

/* Renesas specific register fields */
#define RZN1_UART_xDMACR_DMA_EN         BIT(0)
#define RZN1_UART_xDMACR_1_WORD_BURST   (0 << 1)
#define RZN1_UART_xDMACR_4_WORD_BURST   (1 << 1)
#define RZN1_UART_xDMACR_8_WORD_BURST   (2 << 1)
#define RZN1_UART_xDMACR_BLK_SZ(x)      ((x) << 3)
static const struct of_device_id dw8250_of_match[] = {
        { .compatible = "snps,dw-apb-uart", .data = &dw8250_dw_apb },
#ifndef CONFIG_ROCKCHIP_MINI_KERNEL
        { .compatible = "cavium,octeon-3860-uart", .data = &dw8250_octeon_3860_data },
        { .compatible = "marvell,armada-38x-uart", .data = &dw8250_armada_38x_data },
        { .compatible = "renesas,rzn1-uart", .data = &dw8250_renesas_rzn1_data },
        { .compatible = "starfive,jh7100-uart", .data = &dw8250_starfive_jh7100_data },
#endif
        { /* Sentinel */ }
};
MODULE_DEVICE_TABLE(of, dw8250_of_match);

static const struct acpi_device_id dw8250_acpi_match[] = {
        { "80860F0A", (kernel_ulong_t)&dw8250_dw_apb },
        { "8086228A", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMD0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0020", (kernel_ulong_t)&dw8250_dw_apb },
        { "AMDI0022", (kernel_ulong_t)&dw8250_dw_apb },
        { "APMC0D08", (kernel_ulong_t)&dw8250_dw_apb},
        { "BRCM2032", (kernel_ulong_t)&dw8250_dw_apb },
        { "HISI0031", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C4", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT33C5", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3434", (kernel_ulong_t)&dw8250_dw_apb },
        { "INT3435", (kernel_ulong_t)&dw8250_dw_apb },
        { "INTC10EE", (kernel_ulong_t)&dw8250_dw_apb },
        { },
};
MODULE_DEVICE_TABLE(acpi, dw8250_acpi_match);

static struct platform_driver dw8250_platform_driver = {
        .driver = {
                .name           = "dw-apb-uart",
                .pm             = pm_ptr(&dw8250_pm_ops),
                .of_match_table = dw8250_of_match,
                .acpi_match_table = dw8250_acpi_match,
        },
        .probe                  = dw8250_probe,
        .remove                 = dw8250_remove,
};

module_platform_driver(dw8250_platform_driver);

MODULE_AUTHOR("Jamie Iles");
MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("Synopsys DesignWare 8250 serial port driver");
MODULE_ALIAS("platform:dw-apb-uart");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    Character devices  --->
        Serial drivers  --->
              <*> 8250/16550 and compatible serial support
              [*]   Support for variants of the 16550A serial port
              [*]   Console on 8250/16550 and compatible serial port
              (6)   Maximum number of 8250/16550 serial ports
              (6)   Number of 8250/16550 serial ports to register at runtime
              <*> Support for Synopsys DesignWare 8250 quirks
linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

5. CH340(UART0)测试

- 终端有打印信息表示UART0驱动正常。

```
[   15.778814] [HCC] BLE Mac Addr: 14:0a:02:9a:**:**
[   15.778844] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:4,device_msg:0
[   15.778867] [HCC] Get bsle device action msg, type=[0]
[   15.778878] [HCC] ble_enable_reply_cb is NULL, return!
[   15.778880] [HCC] begin to register hci device
[   15.778884] [HCC] bt_register_hci_dev enter
[   15.779658] [HCC] bt_register_hci_dev exit
[   15.779666] [HCC] hcc bt rx proc ble closed
[   15.784188] [HCC] enter hci_bt_open
[   15.784232] plat_soc:W]bsle_open_close_cmd, service[1], type[1].
[   15.784596] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:2,device_msg:1
[   15.784649] [HCC] start hcc_adapt_bsle_msg_rx_proc,type:4,device_msg:0
[   15.784708] [HCC] Get bsle device action msg, type=[0]
[   15.784722] [HCC]  ble btc open finish
[   15.800566] [HCC]  ble open time:16
[   15.800617] [HCC] hci_bt_setup
[   15.813432] Bluetooth: MGMT ver 1.22
```

第14章 Linux CAN驱动

14.1 CAN总线介绍

1. CAN总线介绍

什么是CAN

Controller Area Network，控制器局域网，是国际上应用最广泛的现场总线之一。

20世纪80年代初德国Bosch公司为解决现代汽车中众多控制单元、测试仪器之间的实时数据交换而开发的一种串行通信协议。

CAN的历史

- 1983年，Bosch开始研究车上网络技术。
- 1986年，Bosch在SAE(美国汽车工程师学会)大会公布CAN协议。
- 1987年，Intel和Philips先后推出CAN控制器芯片。
- 1991年，Bosch颁布CAN 2.0技术规范，CAN2.0包括A和B两个部分。
- 1991年，CAN总线最先在Benz S系列轿车上实现。
- 1993年，ISO颁布CAN国际标准ISO-11898。
- 1994年，SAE颁布基于CAN的J1939标准。
- 2003年，Maybach发布带76个ECU的新车型（CAN，LIN，MOST）。
- 2003年，VW发布带35个ECU的新型Golf。

CiA组织

1992年，一些公司创建了非盈利性组织CAN in Automation（CiA），以提供与CAN相关的技术、产品和市场营销方面的信息。此举旨在提升CAN的形象，并且为CAN协议未来的发展铺平道路。到2020年年初，已有670家公司加入CiA组织。

CAN的OSI模型

七层模型，亦称OSI（Open System Interconnection）。参考模型是国际标准化组织（ISO）制定的一个用于计算机或通信系统间互联的标准体系，一般称为OSI参考模型或七层模型。

从OSI网络模型的角度来看同，现场总线网络一般只实现了第1层（物理层）、第2层（数据链路层）、第7层（应用层）。因为现场总线通常只包括一个网段，因此不需要第3层（传输层）和第4层（网络层），也不需要第5层（会话层）第6层（描述层）的作用。

CAN只定义物理层和数据链路层，没有规定应用层。

CAN总线的特点

CAN总线的特点和工作原理如下：

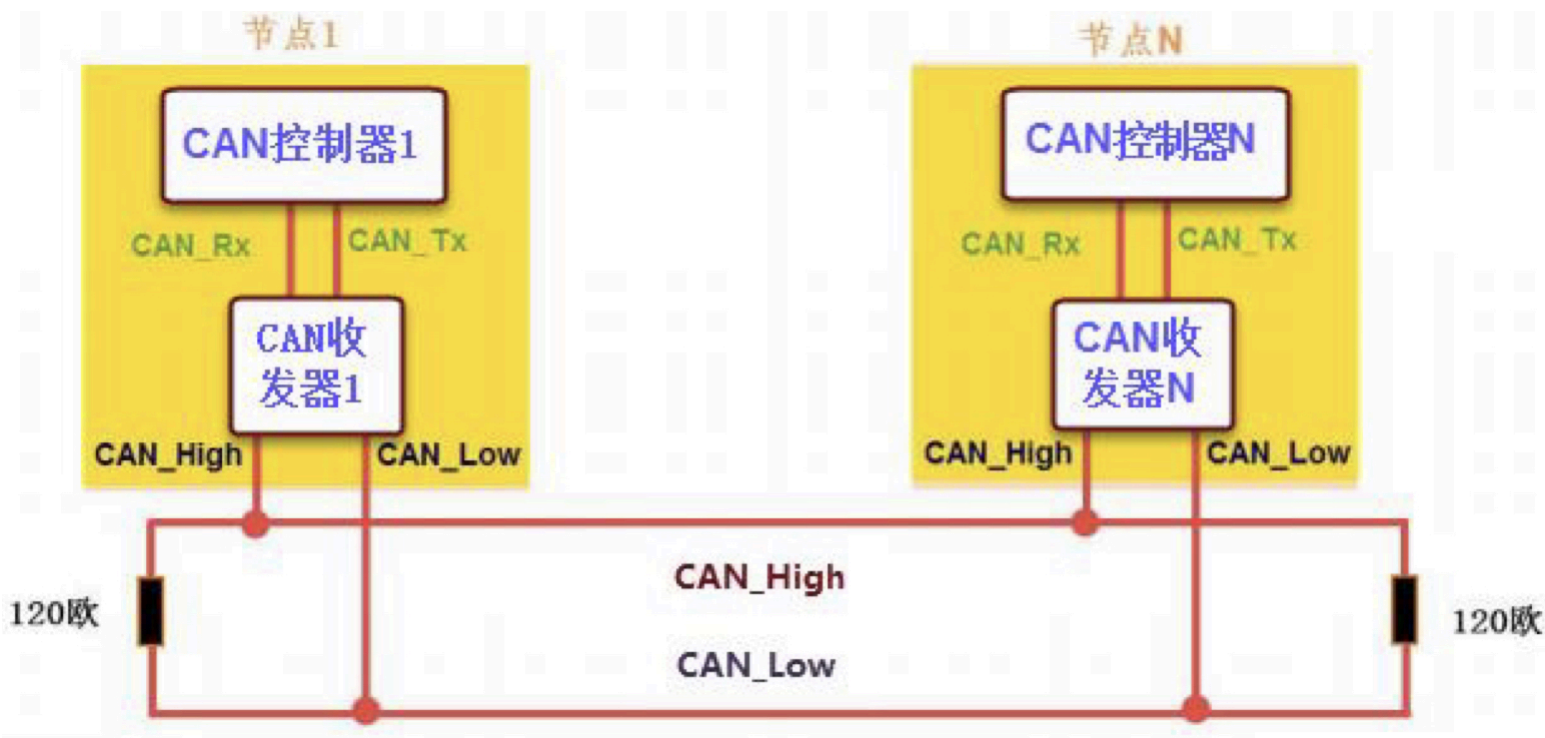
1. 多主从结构：CAN总线支持多个ECU连接到同一条总线上，这些ECU可以充当主设备或从设备。主设备负责发起数据传输，而从设备则接收主设备发送的数据。
2. 差分信号传输：CAN总线使用差分信号传输，其中一对线路（CAN_H和CAN_L）用于传输数据。这种差分传输方式可以提供较强的抗干扰能力，适应工业环境中的电磁干扰。
3. 优先级和仲裁：CAN总线采用基于标识符的仲裁机制，用于解决多个ECU同时发送数据的冲突。每个消息都有一个唯一的标识符，具有较低标识符的ECU在总线上具有更高的优先级，可以在其他ECU发送数据时中断并发送自己的数据。
4. 帧格式：CAN总线使用帧格式来组织和传输数据。一个CAN帧包含标识符、数据、控制位和CRC（循环冗余校验）等字段。数据可以是8字节的标准帧或者8到64字节的扩展帧。
5. 实时性和可靠性：CAN总线具有良好的实时性和可靠性，可以在繁忙的网络环境中传输数据。它支持快速的数据传输速率，通常可达到1Mbps。
6. 错误检测和纠正：CAN总线具有强大的错误检测和纠正机制，可以检测和纠正数据传输过程中的错误。它使用CRC校验和错误标志位来检测错误，并通过重传机制来纠正错误。

CAN总线广泛应用于汽车领域，用于车身控制、引擎管理、传感器数据传输等。它也在工业自动化、航空航天、医疗设备等领域得到广泛应用，为不同设备之间的可靠通信提供了解决方案。

2. CAN物理层协议

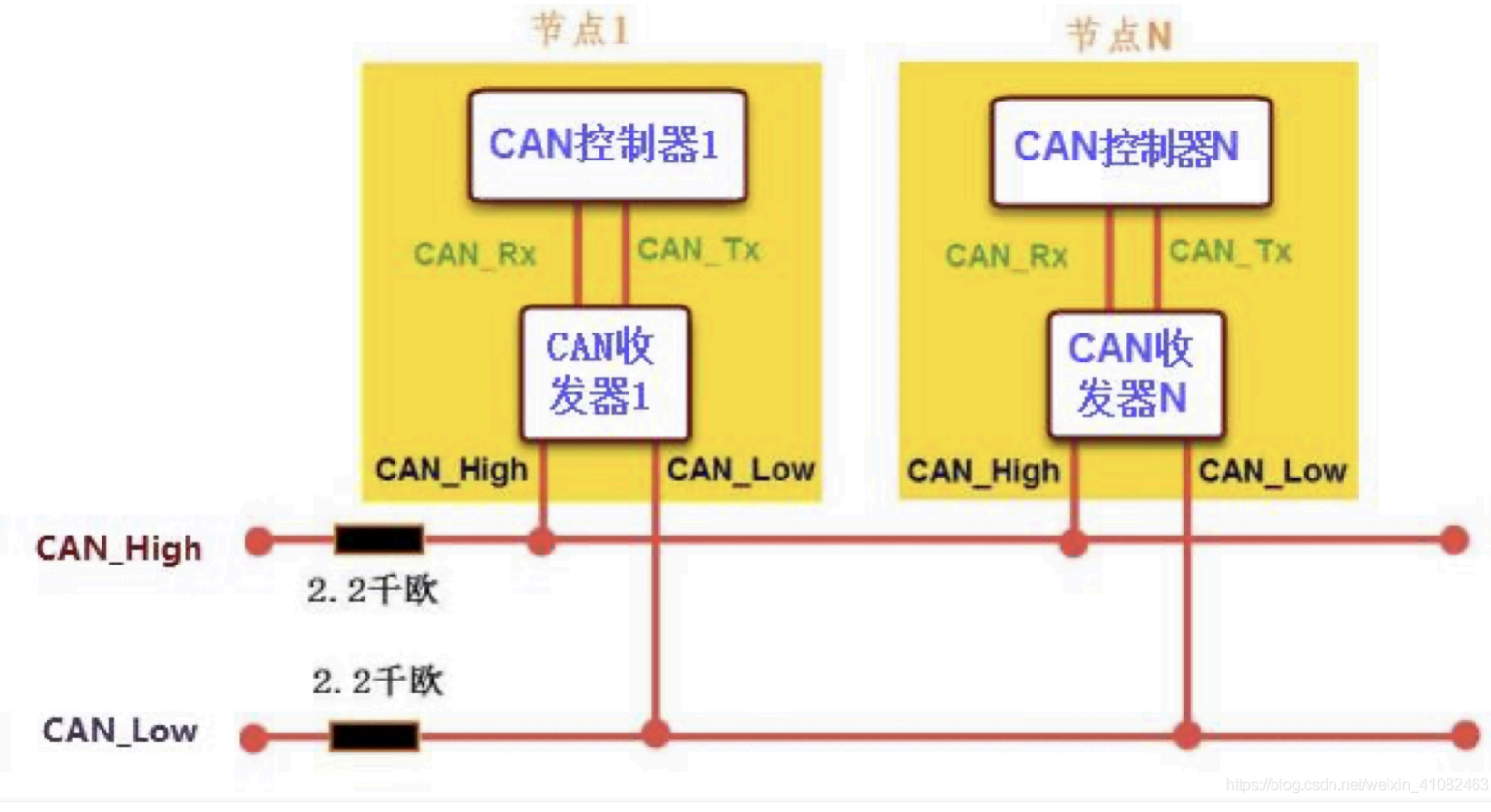
闭环网络：允许总线最长40m，最高速1Mbps

- 规定总线两端各有一个120Ω电阻。



开环网络：最大传输距离1Km，最高速125Kbps 。

- 规定每根线串联一个2.2kΩ的电阻

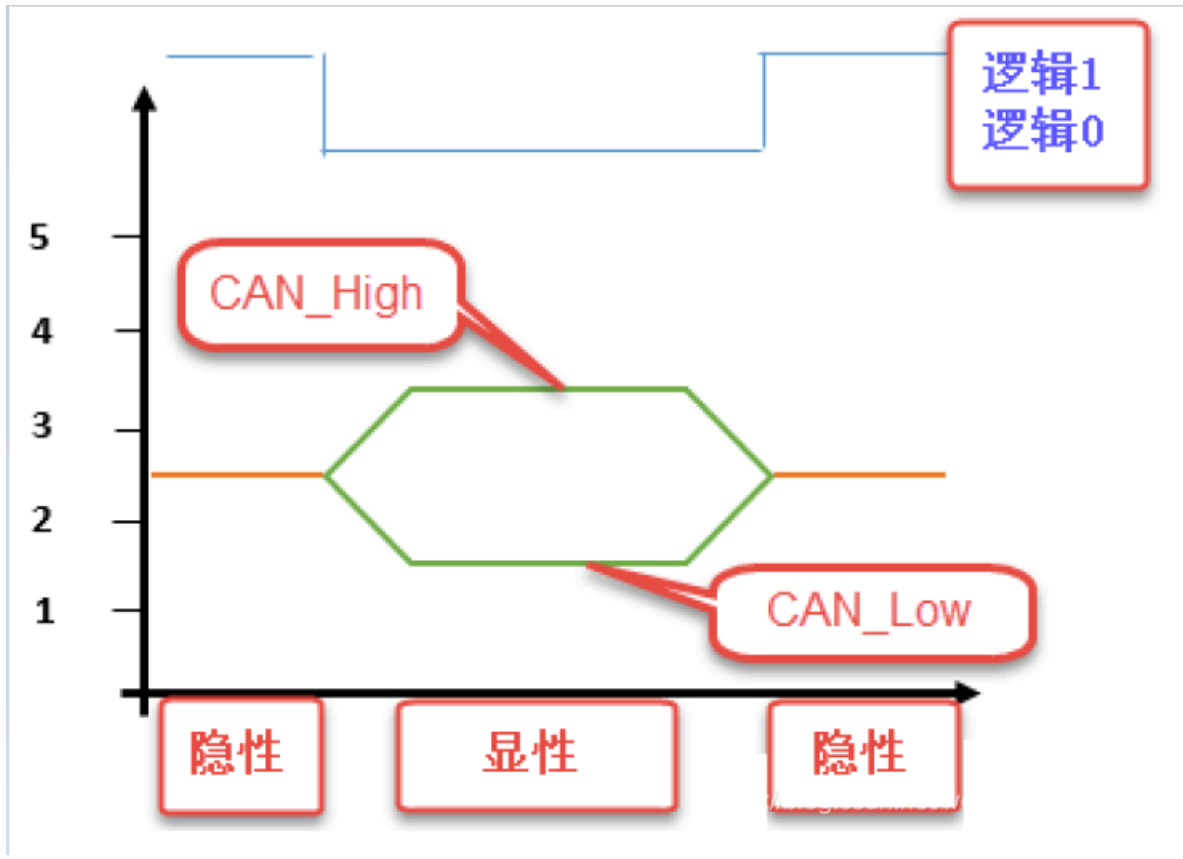


CAN协议差分信号。

- 显性电平对应“0”，隐性电平对应“1”。

  隐性电平（1）两条线电压都是2.5V，即压差为0；

  显性电平（0）CAN_High和CAN_Low分别为3.5V和1.5V，压差为2V。
- 总线上，只要有一个节点输出显性，则总线上为显性电平；只有所有节点都是隐性电平，总线才为隐性电平。
- 由于CAN是半双工的，收发分开进行，且是总线通讯，所以一个时刻只能有一个节点发送，其他节点在此时只能接收。



14.2 RK3506的CANFD接口

RK3506 是瑞芯微（Rockchip）面向工业物联网、智能显示与边缘计算等场景推出的入门级 SoC，其显著特点是**在极低功耗与低成本的前提下集成了丰富的工业接口，其中就包含双路 CAN FD 控制器。**以下是对该接口的详细介绍。

协议与速率

- **支持协议**：完整支持 **CAN 2.0B** 与 **CAN FD**（ISO 11898-1:2015 标准），可无缝兼容传统 CAN 网络。
- **通信速率**：
  - 仲裁段（Arbitration Phase）：最高 **1 Mbit/s。**
  - 数据段（Data Phase）：最高 **5 Mbit/s**（典型可达 5 Mbps，实际最大速率取决于外部收发器与总线拓扑）。
- **数据负载长度**：CAN FD 帧支持 **0 ~ 64 字节** 可变长度，大幅提升传输效率。

接口数量与信号

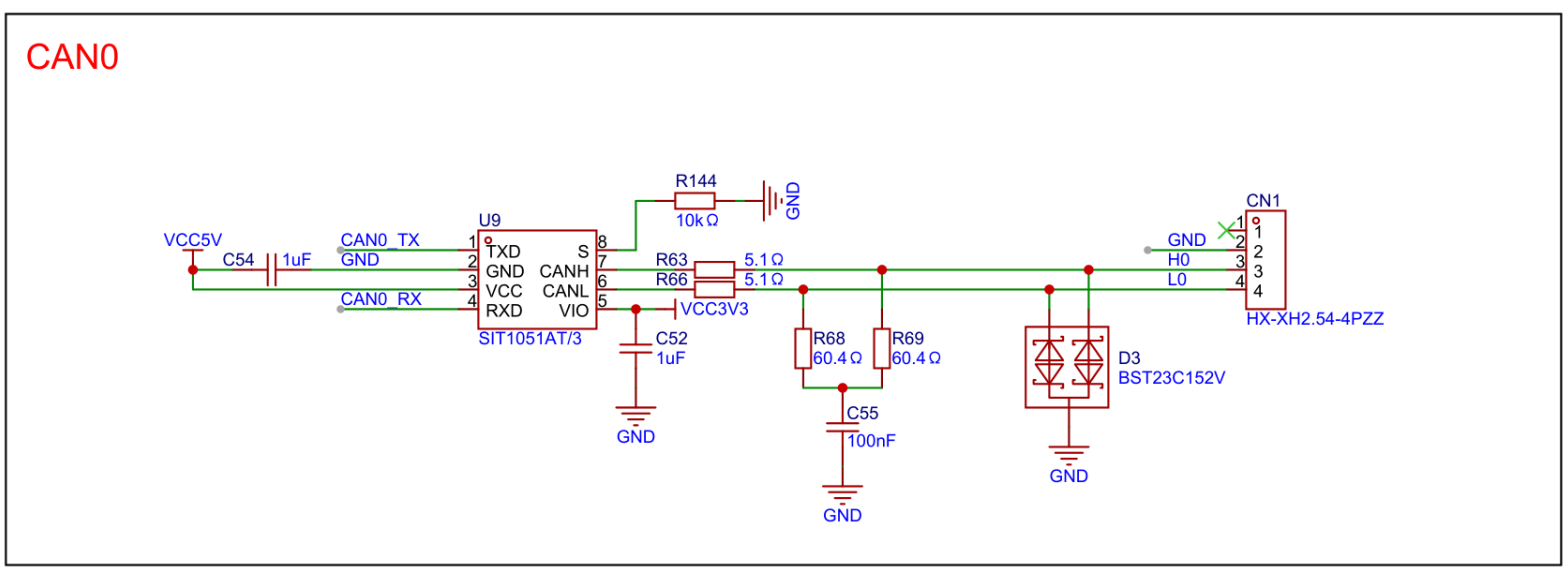
RK3506 提供 **两路独立 CAN FD 控制器**，对应引脚可复用为 `CAN0_TX/RX`、`CAN1_TX/RX`。在实际应用中，只需外加 CAN FD 收发器（如 TJA1043、MCP2562FD 等）即可接入总线。

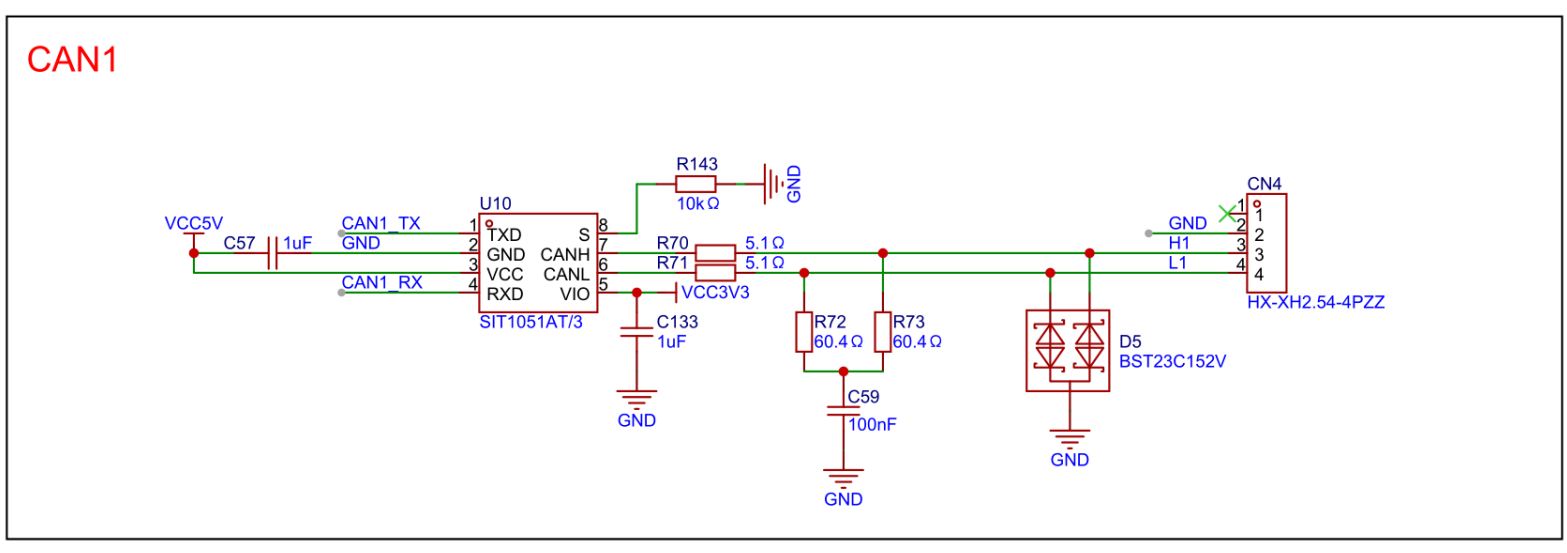
硬件特性与功能

- **帧格式**：同时支持标准帧（11-bit ID）和扩展帧（29-bit ID）。
- **发送缓冲区**：具有多个发送邮箱（通常为 3 级或以上 TX Buffer），支持硬件优先级仲裁，减少发送延迟。
- **接收缓冲区**：提供可配置的 **接收 FIFO**（通常为 16 级或以上深度），支持硬件过滤，可有效降低 CPU 负载。
- **过滤器**：支持多达 **28 组** 可编程的接收过滤器（根据各 IP 版本略有差异），可精确匹配 ID 或使用掩码过滤，减少无用帧处理。
- **工作模式**：
  - 正常模式
  - 静默模式（仅监听，不发送应答）
  - 回环模式（自测试用）
  - 回环+静默模式
- **错误管理**：完善的错误检测与处理机制（位错误、填充错误、CRC 错误、格式错误等），支持总线关闭（Bus-Off）自动恢复功能。
- **时间戳**：内置硬件时间戳单元，便于实现精确的报文时序分析与诊断。

14.3 CAN(CAN0~1)驱动

1. CAN原理图





2. CAN设备树

- kernel/arch/arm/boot/dts/rk3506.dtsi

```
    can0: can@ff320000 {
        compatible = "rockchip,rk3506-canfd", "rockchip,rk3576-canfd";
        reg = <0xff320000 0x1000>;
        interrupts = <GIC_SPI 45 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_CAN0>, <&cru HCLK_CAN0>;
        clock-names = "baudclk", "apb_pclk";
        resets = <&cru SRST_CAN0>, <&cru SRST_H_CAN0>;
        reset-names = "can", "can-apb";
        assigned-clocks = <&cru CLK_CAN0>;
        assigned-clock-rates = <300000000>;
        status = "disabled";
    };

    can1: can@ff330000 {
        compatible = "rockchip,rk3506-canfd", "rockchip,rk3576-canfd";
        reg = <0xff330000 0x1000>;
        interrupts = <GIC_SPI 46 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru CLK_CAN1>, <&cru HCLK_CAN1>;
        clock-names = "baudclk", "apb_pclk";
        resets = <&cru SRST_CAN1>, <&cru SRST_H_CAN1>;
        reset-names = "can", "can-apb";
        assigned-clocks = <&cru CLK_CAN1>;
        assigned-clock-rates = <300000000>;
        status = "disabled";
    };
```

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
//和rgb冲突
&can0 {
    assigned-clocks = <&cru CLK_CAN0>;
    assigned-clock-rates = <200000000>;
    pinctrl-0 = <&rm_io25_can0_tx &rm_io26_can0_rx>;
    pinctrl-names = "default";
    status = "disabled";
};

&can1 {
    assigned-clocks = <&cru CLK_CAN1>;
    assigned-clock-rates = <200000000>;
    pinctrl-0 = <&rm_io17_can1_tx &rm_io18_can1_rx>;
    pinctrl-names = "default";
    status = "okay";
};
```

修改设备树，启用can0的功能

```
//和rgb冲突 (MIPI板可用, RGB板需禁用)
```

```
&can0 {
    assigned-clocks = <&cru CLK_CAN0>;
    assigned-clock-rates = <200000000>;
    pinctrl-0 = <&rm_io25_can0_tx &rm_io26_can0_rx>;
    pinctrl-names = "default";
    status = "okay";
};

&can1 {
    assigned-clocks = <&cru CLK_CAN1>;
    assigned-clock-rates = <200000000>;
    pinctrl-0 = <&rm_io17_can1_tx &rm_io18_can1_rx>;
    pinctrl-names = "default";
    status = "okay";
};
```

3. CAN驱动分析

RK3506 没有独立的 CAN 驱动，直接复用了 RK3576 的。

- kernel/drivers/net/can/rockchip/rk3576_canfd.c

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2023 Rockchip Electronics Co., Ltd.
 * Rk3576 CANFD driver
 */

#include <linux/delay.h>
#include <linux/iopoll.h>
#include <linux/pinctrl/consumer.h>
#include <linux/clk.h>
#include <linux/errno.h>
#include <linux/init.h>
#include <linux/interrupt.h>
#include <linux/io.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/netdevice.h>
#include <linux/of.h>
#include <linux/of_device.h>
#include <linux/platform_device.h>
#include <linux/skbuff.h>
#include <linux/spinlock.h>
#include <linux/string.h>
#include <linux/types.h>
#include <linux/can/dev.h>
#include <linux/can/error.h>
#include <linux/reset.h>
#include <linux/pm_runtime.h>
#include <linux/dma-mapping.h>
#include <linux/dmaengine.h>

/* registers definition */
enum rk3576_canfd_reg {
    CANFD_MODE = 0x00,
    CANFD_CMD = 0x04,
    CANFD_STATE = 0x08,
    CANFD_INT = 0x0c,
    CANFD_INT_MASK = 0x10,
    CANFD_NBTP = 0x100,
    CANFD_DBTP = 0x104,
    CANFD_TDCR = 0x108,
    CANFD_BRS_CFG = 0x10c,
    CANFD_DMA_CTRL = 0x11c,
    CANFD_TXFIC = 0x200,
    CANFD_TXID = 0x204,
    CANFD_TXDAT0 = 0x208,
    CANFD_TXDAT1 = 0x20c,
    CANFD_TXDAT2 = 0x210,
    CANFD_TXDAT3 = 0x214,
    CANFD_TXDAT4 = 0x218,
    CANFD_TXDAT5 = 0x21c,
    CANFD_TXDAT6 = 0x220,
    CANFD_TXDAT7 = 0x224,
    CANFD_TXDAT8 = 0x228,
    CANFD_TXDAT9 = 0x22c,
    CANFD_TXDAT10 = 0x230,
    CANFD_TXDAT11 = 0x234,
    CANFD_TXDAT12 = 0x238,
    CANFD_TXDAT13 = 0x23c,
    CANFD_TXDAT14 = 0x240,
    CANFD_TXDAT15 = 0x244,
    CANFD_RXFIC = 0x300,
    CANFD_RXID = 0x304,
    CANFD_RXTS = 0x308,
    CANFD_RXDAT0 = 0x30c,
    CANFD_RXDAT1 = 0x310,
    CANFD_RXDAT2 = 0x314,
    CANFD_RXDAT3 = 0x318,
    CANFD_RXDAT4 = 0x31c,
    CANFD_RXDAT5 = 0x320,
    CANFD_RXDAT6 = 0x324,
    CANFD_RXDAT7 = 0x328,
    CANFD_RXDAT8 = 0x32c,
    CANFD_RXDAT9 = 0x330,
    CANFD_RXDAT10 = 0x334,
    CANFD_RXDAT11 = 0x338,
    CANFD_RXDAT12 = 0x33c,
    CANFD_RXDAT13 = 0x340,
    CANFD_RXDAT14 = 0x344,
    CANFD_RXDAT15 = 0x348,
    CANFD_RXFRD = 0x400,
    CANFD_STR_CTL = 0x600,
    CANFD_STR_STATE = 0x604,
    CANFD_STR_TIMEOUT = 0x608,
    CANFD_STR_WTM = 0x60c,
    CANFD_EXTM_START_ADDR = 0x610,
    CANFD_EXTM_SIZE = 0x614,
    CANFD_EXTM_WADDR = 0x618,
    CANFD_EXTM_RADDR = 0x61c,
    CANFD_EXTM_AHB_TXTHR = 0x620,
    CANFD_EXTM_LEFT_CNT = 0x624,
    CANFD_ATF0 = 0x700,
    CANFD_ATF1 = 0x704,
    CANFD_ATF2 = 0x708,
    CANFD_ATF3 = 0x70c,
    CANFD_ATF4 = 0x710,
    CANFD_ATFM0 = 0x714,
    CANFD_ATFM1 = 0x718,
    CANFD_ATFM2 = 0x71c,
    CANFD_ATFM3 = 0x720,
    CANFD_ATFM4 = 0x724,
    CANFD_ATF_DLC = 0x728,
    CANFD_ATF_CTL = 0x72c,
    CANFD_SPACE_CTRL = 0x800,
    CANFD_AUTO_RETX_CFG = 0x808,
    CANFD_AUTO_RETX_STATE0 = 0x80c,
    CANFD_AUTO_RETX_STATE1 = 0x810,
    CANFD_OLF_CFG = 0x814,
    CANFD_RXINT_CTRL = 0x818,
    CANFD_RXINT_TIMEOUT = 0x81c,
    CANFD_OTHER_CFG = 0x820,
    CANFD_WAVE_FILTER_CFG = 0x824,
    CANFD_RBC_CFG = 0x828,
    CANFD_TXCRC_CFG = 0x82c,
    CANFD_BUSOFFRCY_CFG = 0x830,
    CANFD_BUSOFF_RCY_THR = 0x834,
    CANFD_ERROR_CODE = 0x900,
    CANFD_ERROR_MASK = 0x904,
    CANFD_RXERRORCNT = 0x910,
    CANFD_TXERRORCNT = 0x914,
    CANFD_RX_RXSRAM_RDATA = 0xc00,
    CANFD_RTL_VERSION = 0xf0c,
};


static const struct of_device_id rk3576_canfd_of_match[] = {
    {
        .compatible = "rockchip,rk3576-canfd",
    },
    {},
};
MODULE_DEVICE_TABLE(of, rk3576_canfd_of_match);

static struct platform_driver rk3576_canfd_driver = {
    .driver = {
        .name = DRV_NAME,
```

```
        .pm = &rk3576_canfd_dev_pm_ops,
        .of_match_table = rk3576_canfd_of_match,
    },
    .probe = rk3576_canfd_probe,
    .remove = rk3576_canfd_remove,
};
module_platform_driver(rk3576_canfd_driver);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Elaine Zhang <zhangqing@rock-chips.com>");
MODULE_DESCRIPTION("RK3576 CANFD Drivers");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    [*] Network device support  --->
        <*>   CAN Device Drivers  --->
            <*>     RK3576 CANFD controller

linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

- 编译内核更新设备树与内核到开发板内。

5. CAN功能测试

- 查看系统CAN设备

```
root@rk3506-buildroot:/# ifconfig -a
Featureid0 Link encap:Ethernet  HWaddr 00:00:00:00:00:00
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

can0      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:64

can1      Link encap:UNSPEC  HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
          NOARP  MTU:16  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:10
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:65

eth0      Link encap:Ethernet  HWaddr 26:64:07:3A:AB:BD
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:76 Base address:0xa000

eth1      Link encap:Ethernet  HWaddr DE:3A:35:9A:26:91
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:78 Base address:0xe000

eth2      Link encap:Ethernet  HWaddr AC:0C:29:A3:9B:6D
          inet addr:192.168.0.100  Bcast:192.168.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:8 errors:0 dropped:0 overruns:0 frame:0
          TX packets:17 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1084 (1.0 KiB)  TX bytes:2234 (2.1 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:128 errors:0 dropped:0 overruns:0 frame:0
          TX packets:128 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9472 (9.2 KiB)  TX bytes:9472 (9.2 KiB)

p2p0      Link encap:Ethernet  HWaddr 14:0A:02:9A:B3:D6
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

wlan0     Link encap:Ethernet  HWaddr 14:0A:02:9A:B0:D6
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@rk3506-buildroot:/#
```

- CAN测试命令

```
ip link set can0 down
# 1. CAN FD 模式 (500k 标称, 2M 数据, BRS 使能)
ip link set can0 type can bitrate 500000 dbitrate 2000000 fd on
ip link set can0 up

# 2. classic CAN 模式
ip link set can1 type can bitrate 500000
ip link set can1 up

# 3. 发送 (classic CAN)
cansend can0 123#AABBCCDD

# 4. 发送 (CAN FD + BRS)
cansend can0 123##1AABBCCDDEEFF

# 5. 接收
candump can0

# 6. 环回测试 (无需外部总线)
ip link set can0 type can bitrate 500000 loopback on
ip link set can0 up
cansend can0 123#DEADBEEF
candump can0    # 应收到相同帧

# 7. 查看状态和错误计数
ip -details -statistics link show can0
```

- 回环测试CAN0通信

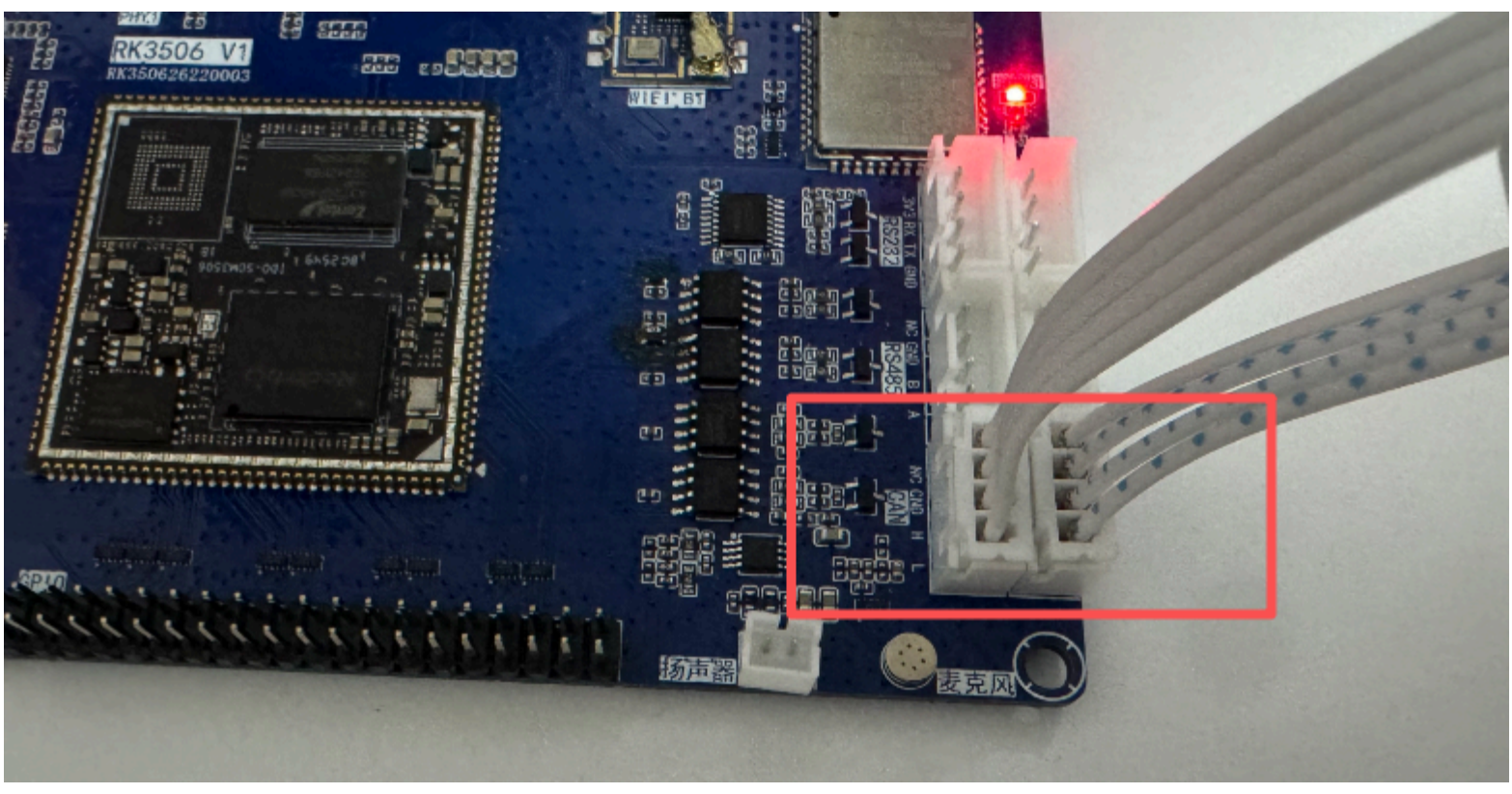
```
root@rk3506-buildroot:/# ip link show can0    # 检查设备是否存在
2: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can

# 回环模式 loopback on
# 驱动默认是 FD 模式，loopback on 不会清除 FD 标志，必须显式 fd off 关掉。
root@rk3506-buildroot:/# ip link set can0 type can bitrate 500000 loopback on fd off
[  838.507700] rk3576_canfd ff320000.can can0: bitrate error 0.1%
root@rk3506-buildroot:/# ip link set can0 up
root@rk3506-buildroot:/# candump can0 &
[1] 2000
root@rk3506-buildroot:/# cansend can0 123#AABBCCDD
root@rk3506-buildroot:/#   can0  123   [4]  AA BB CC DD

root@rk3506-buildroot:/#
```

- 回环测试CAN1通信

```
root@rk3506-buildroot:/# ip link show can1    # 检查设备是否存在
3: can1: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default qlen 10
    link/can

# 回环模式 loopback on
# 驱动默认是 FD 模式，loopback on 不会清除 FD 标志，必须显式 fd off 关掉。
```

```
root@rk3506-buildroot:/# ip link set can1 type can bitrate 500000 loopback on fd off
[ 1305.547526] rk3576_canfd ff330000.can can1: bitrate error 0.1%
root@rk3506-buildroot:/# ip link set can1 up
root@rk3506-buildroot:/# candump can1 &
[2] 2014
root@rk3506-buildroot:/# cansend can1 123#AABBCCDD
  can1  123   [4]  AA BB CC DD
root@rk3506-buildroot:/#
```

- CAN0和CAN1正常模式测试
- 硬件接线



- CAN0发送， CAN1接收

```
root@rk3506-buildroot:/# ip link set can0 down
root@rk3506-buildroot:/# ip link set can1 down
root@rk3506-buildroot:/# ip link set can0 type can bitrate 1000000 dbitrate 1000000 fd on
[   96.071287] rk3576_canfd ff320000.can can0: bitrate error 0.3%
[   96.232985] rk3576_canfd ff320000.can can0: bitrate error 0.3%
root@rk3506-buildroot:/# ip link set can1 type can bitrate 1000000 dbitrate 1000000 fd on
[  103.268165] rk3576_canfd ff330000.can can1: bitrate error 0.3%
[  103.268221] rk3576_canfd ff330000.can can1: bitrate error 0.3%
root@rk3506-buildroot:/# ip link set can0 up
root@rk3506-buildroot:/# ip link set can1 up
root@rk3506-buildroot:/# candump can1 &
[1] 1987
root@rk3506-buildroot:/# cansend can0 123#AABBCCDD
root@rk3506-buildroot:/#   can1  123   [4]  AA BB CC DD

root@rk3506-buildroot:/#
```

- CAN0接收， CAN1发送

```
root@rk3506-buildroot:/# ip link set can0 down
root@rk3506-buildroot:/# ip link set can1 down
root@rk3506-buildroot:/# ip link set can0 type can bitrate 1000000 dbitrate 1000000 fd on
[   68.463458] rk3576_canfd ff320000.can can0: bitrate error 0.3%
[   68.463581] rk3576_canfd ff320000.can can0: bitrate error 0.3%
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ip link set can1 type can bitrate 1000000 dbitrate 1000000 fd on
[  101.169696] rk3576_canfd ff330000.can can1: bitrate error 0.3%
[  101.169866] rk3576_canfd ff330000.can can1: bitrate error 0.3%
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ip link set can0 up
root@rk3506-buildroot:/# ip link set can1 up
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# candump can0 &
[1] 1990
root@rk3506-buildroot:/# cansend can1 123#AABBCCDD
root@rk3506-buildroot:/#   can0  123   [4]  AA BB CC DD

root@rk3506-buildroot:/#
```

第15章 Linux SD卡驱动

15.1. SDIO 安全数字输入输出

1. 什么是SDIO

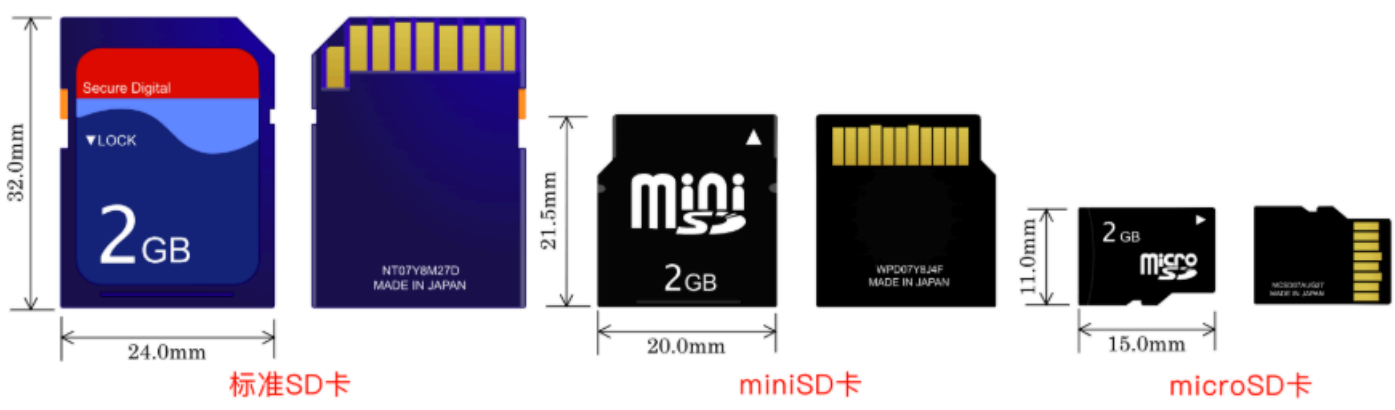
SDIO 的全称是 **Secure Digital Input/Output**（安全数字输入/输出）。

简单来说，**它是一种允许SD卡插槽不仅仅用于存储卡，还能连接其他外部设备的扩展技术。**

你可以把它理解为一个“多功能SD卡槽”。普通的SD卡槽只能插存储卡来扩展空间，而一个支持SDIO的SD卡槽，除了能插存储卡，还能插各种不同功能的“卡”，比如Wi-Fi卡、蓝牙卡、GPS卡等，来为设备添加新的硬件功能。

2. SD卡

安全数码卡（Secure Digital Memory Card），简称SD卡，是嵌入式设备上常用的一种存储介质，通常可以将SD卡分为标准SD卡、miniSD卡和microSD卡（TF卡）三种类型，每种卡形状大小不一，除标准SD卡卡身上拥有一个写保护开关外，其他的功能三张卡一致，如今miniSD卡正逐渐被microSD卡所取代，如下图所示为三种不同类型SD卡形状

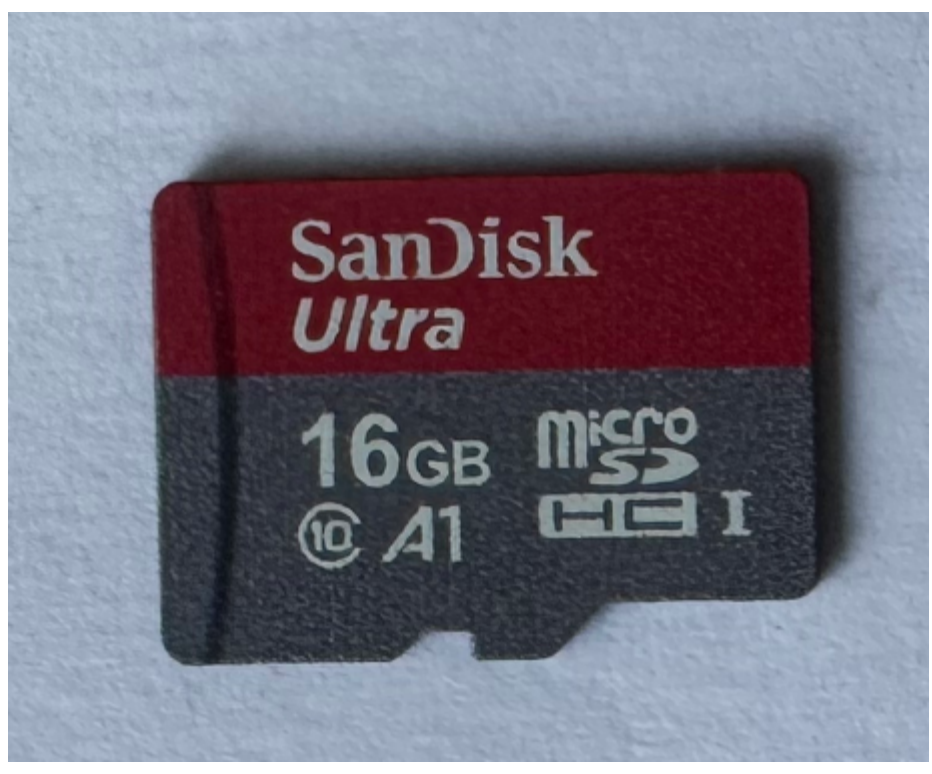


按照SD卡容量大小的不同可以将其分为SD、SDHC、SDXC等型号，按照SD卡读写机制速度的不同又可以将其分为Standard、High-speed、UHS-I等型号，具体如下表所示

SD卡按照不同容量和速度分类

读写速度 / 内存大小	Standard	High-speed	UHS-I	UHS-II	UHS-III	Express
SD	≤2GB ≤12.5MB/s					
SDHC		> 2GB - ≤32GB ≤25MB/s				
SDXC			> 32GB - ≤2TB ≤104MB/s			
SDUC				> 2TB - ≤128TB ≤312MB/s	> 2TB - ≤128TB ≤624MB/s	> 2TB - ≤128TB ≤985MB/s

- 识别SD卡的规格



3. RK3506的SD卡接口

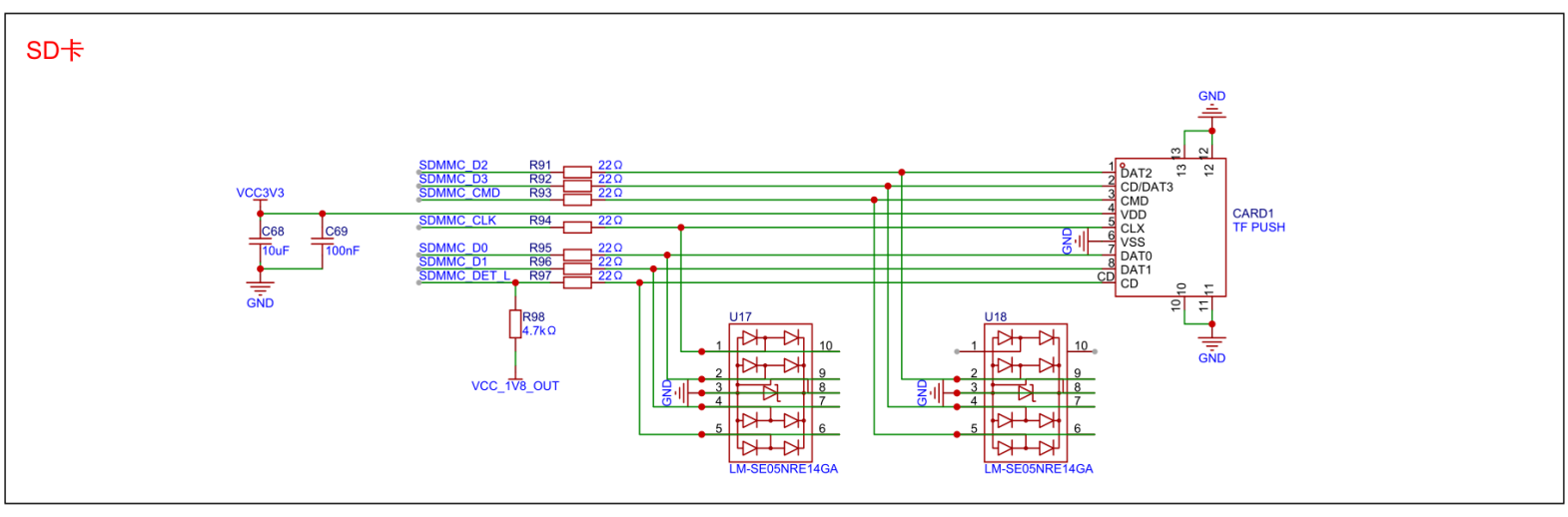
瑞芯微 RK3506 是一款面向 AIoT 领域的低功耗处理器，其 SD 卡接口通常集成在芯片的 **SDMMC（SD/MMC Host Controller）** 模块中。以下基于该平台的公开资料和同类芯片设计惯例，给出其 SD 卡接口的技术介绍。

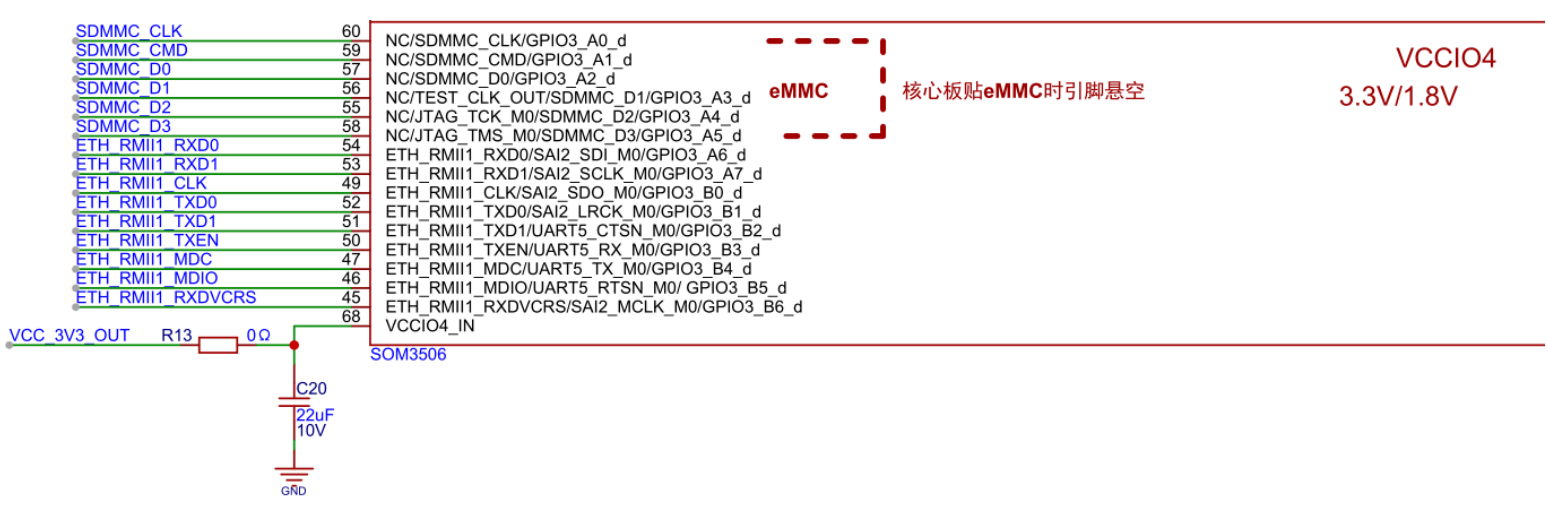
- 接口基本特性

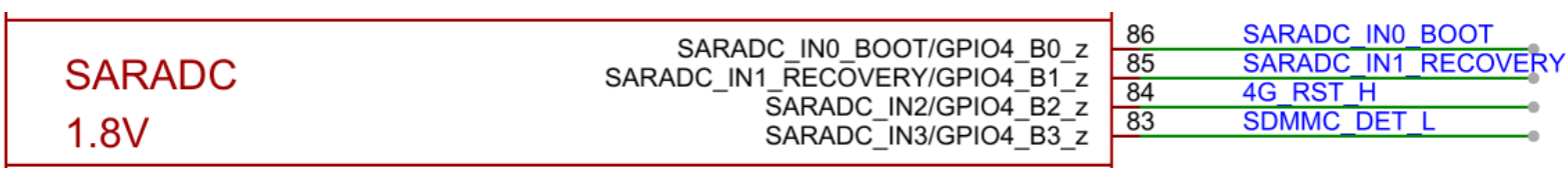
项目	说明
控制器	1 路 **SDMMC0**，专用于 SD 卡 / SDIO 设备
兼容协议	SD 3.0，向下兼容 SD 2.0、MMC 4.51

项目	说明
数据宽度	4-bit 模式 (CLK、CMD、DATA0~3)
UHS-I 速度模式	SDR12、SDR25、SDR50、**SDR104**、DDR50
最高理论速率	SDR104 下可达 **104 MB/s**
启动支持	**支持从 SD 卡启动**（与 eMMC、SPI NAND/NOR 等一起构成多启动源）
I/O 电压	3.3 V（默认模式）/ 1.8 V（UHS-I 高速模式），由 VCCIO_SD 域供电
卡检测 / 写保护	通常由普通 GPIO 模拟，非控制器内置硬件信号

4. SD卡接口原理图







5. SD卡电路分析

- 这个电路兼容3.3V的普通低速卡，也兼容 SD 3.0 UHS-I 高速模式（1.8V 信号）设计
- VCCIO4_IN 的 **SDMMC 接口的 IO 供电引脚。**RK3506 的 SDMMC/SDIO 接口电源域是 3.3V
- SD 卡内部的 IO 驱动电路支持 **3.3V** 和 **1.8V** 两种物理电平
- 驱动程序在识别到卡是高速卡后，主动发送 `CMD11` 指令给 SD 卡，要求 SD 卡将其内部信号电平从 3.3V 切换为 **1.8V**
- SD卡检测原理
  - **未插卡**：主控引脚 83 接收到的电压是外部 1.8V 上拉电压，内部逻辑判定为“高电平”。
  - **插卡瞬间**：TF 卡座的 10、11 脚机械短接到地，将外部的 1.8V 上拉电压瞬间拉低至 0V。

15.2 SD卡驱动

1. SD卡设备树

- MMC 控制器基节点定义
- kernel/arch/arm/boot/dts/rk3502.dtsi

```
mmc: mmc@ff480000 {
    compatible = "rockchip,rk3506-dw-mshc", "rockchip,rk3288-dw-mshc";
    reg = <0xff480000 0x4000>;
    interrupts = <GIC_SPI 86 IRQ_TYPE_LEVEL_HIGH>;
    max-frequency = <150000000>;
    bus-width = <4>;
    clocks = <&cru HCLK_SDMMC>, <&cru CCLK_SRC_SDMMC>;
    clock-names = "biu", "ciu";
    fifo-depth = <0x100>;
    resets = <&cru SRST_H_SDMMC>;
    reset-names = "reset";
    status = "disabled";
};
```

- **硬件**: DesignWare MMC 控制器 (`dw-mshc`)，兼容 RK3288 的 IP
- **寄存器基址**: `0xff480000`，长度 16KB
- **中断号**: SPI 86
- **总线宽度**: 最大 4 线 (D0~D3)
- **最大频率**: 150MHz
- **FIFO 深度**: 256 字节
- **默认状态**: `disabled`，由板级 dts 启用

- SDMMC pinmux 配置
- kernel/arch/arm/boot/dts/rk3506-pinctrl.dtsi

```
sdmmc {
    /omit-if-no-ref/
    sdmmc_bus4_pins: sdmmc-bus4-pins {
        rockchip,pins =
            /* sdmmc_d0 */
            <3 RK_PA2 1 &pcfg_pull_up>,
            /* sdmmc_d1 */
            <3 RK_PA3 1 &pcfg_pull_up>,
            /* sdmmc_d2 */
            <3 RK_PA4 1 &pcfg_pull_up>,
            /* sdmmc_d3 */
            <3 RK_PA5 1 &pcfg_pull_up>;
    };

    /omit-if-no-ref/
    sdmmc_clk_pins: sdmmc-clk-pins {
        rockchip,pins =
            /* sdmmc_clk */
            <3 RK_PA0 1 &pcfg_pull_none>;
    };
    /omit-if-no-ref/
    sdmmc_cmd_pins: sdmmc-cmd-pins {
        rockchip,pins =
            /* sdmmc_cmd */
            <3 RK_PA1 1 &pcfg_pull_up>;
    };
};
```

- SD 卡电源 regulator
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
vcc3v3_sd: vcc3v3-sd {
    compatible = "regulator-fixed";
    //gpio = <&gpio3 RK_PA6 GPIO_ACTIVE_LOW>;
    regulator-name = "vcc3v3_sd";
    regulator-min-microvolt = <3300000>;
    regulator-max-microvolt = <3300000>;
    //pinctrl-names = "default";
    //pinctrl-0 = <&sdmmc_pwren>;
};

vccio_sd: vccio-sd {
    compatible = "regulator-gpio";
    regulator-name = "vccio_sd";
    regulator-min-microvolt = <1800000>;
    regulator-max-microvolt = <3300000>;
    //gpios = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>;
    states = <3300000 1
              1800000 0>;
    //pinctrl-names = "default";
    //pinctrl-0 = <&sdmmc_volt>;
};
```

- `&mmc` 板级配置
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
&mmc {
    max-frequency = <150000000>;
```

```
    no-sdio;
    no-mmc;
    cap-mmc-highspeed;
    cap-sd-highspeed;
    cd-gpios = <&gpio4 RK_PB3 GPIO_ACTIVE_LOW>;
    disable-wp;
    //sd-uhs-sdr104;
    pinctrl-names = "default";
    pinctrl-0 = <&sdmmc_clk_pins &sdmmc_cmd_pins &sdmmc_bus4_pins &sdmmc_det>;
    //vqmmc-supply = <&vccio_sd>;
    //vmmc-supply = <&vcc3v3_sd>;
    status = "okay";
};
```

- 电源分析：
  - 固定 3.3V 供电，无 GPIO 控制（GPIO 和 pinctrl 均被注释），说明卡电源是**常供电**的，开机即上电
  - 该电源在 `&mmc` 节点中对应的 `vmmc-supply` 也被注释了，内核无法通过 regulator 框架管理 SD 卡电源开关
- CD (Card Detect) pinctrl
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    sdcard {
        sdmmc_det: sdmmc-det {
                rockchip,pins = <4 RK_PB3 RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
```

2. SD卡驱动分析

- kernel/drivers/mmc/host/dw_mmc-rockchip.c

```
// SPDX-License-Identifier: GPL-2.0-or-later
/*
 * Copyright (c) 2014, Fuzhou Rockchip Electronics Co., Ltd
 */

#include <linux/module.h>
#include <linux/platform_device.h>
#include <linux/clk.h>
#include <linux/mmc/host.h>
#include <linux/of_address.h>
#include <linux/mmc/slot-gpio.h>
#include <linux/pm_runtime.h>
#include <linux/rockchip/cpu.h>
#include <linux/slab.h>

#include "dw_mmc.h"
#include "dw_mmc-pltfm.h"

#define RK3288_CLKGEN_DIV   2
#define USRID_INTER_PHASE   0x20230001
#define SDMMC_TIMING_CON0   0x130
#define SDMMC_TIMING_CON1   0x134
#define SDMMC_MISC_CON      0x138
#define MEM_CLK_AUTOGATE_ENABLE BIT(5)
#define ROCKCHIP_MMC_DELAY_SEL BIT(10)
#define ROCKCHIP_MMC_DEGREE_MASK 0x3
#define ROCKCHIP_MMC_DELAYNUM_OFFSET 2
#define ROCKCHIP_MMC_DELAYNUM_MASK (0xff << ROCKCHIP_MMC_DELAYNUM_OFFSET)
#define PSECS_PER_SEC 1000000000000LL
#define ROCKCHIP_MMC_DELAY_ELEMENT_PSEC 60
#define HIWORD_UPDATE(val, mask, shift) \
        ((val) << (shift) | (mask) << ((shift) + 16))



static const struct of_device_id dw_mci_rockchip_match[] = {
    { .compatible = "rockchip,rk2928-dw-mshc",
        .data = &rk2928_drv_data },
    { .compatible = "rockchip,rk3288-dw-mshc",
        .data = &rk3288_drv_data },
    {},
};
MODULE_DEVICE_TABLE(of, dw_mci_rockchip_match);

static int dw_mci_rockchip_probe(struct platform_device *pdev)
{
    const struct dw_mci_drv_data *drv_data;
    const struct of_device_id *match;
    int ret;
    bool use_rpm = true;

    if (!pdev->dev.of_node)
        return -ENODEV;

    if ((!device_property_read_bool(&pdev->dev, "non-removable") &&
         !device_property_read_bool(&pdev->dev, "cd-gpios")) ||
        (device_property_read_bool(&pdev->dev, "no-sd") &&
         device_property_read_bool(&pdev->dev, "no-mmc")))
        use_rpm = false;

    match = of_match_node(dw_mci_rockchip_match, pdev->dev.of_node);
    drv_data = match->data;

    /*
     * increase rpm usage count in order to make
     * pm_runtime_force_resume calls rpm resume callback
     */
    pm_runtime_get_noresume(&pdev->dev);
    pm_runtime_set_active(&pdev->dev);

    if (use_rpm) {
        pm_runtime_enable(&pdev->dev);
        pm_runtime_set_autosuspend_delay(&pdev->dev, 50);
        pm_runtime_use_autosuspend(&pdev->dev);
    }

    ret = dw_mci_pltfm_register(pdev, drv_data);
    if (ret) {
        if (use_rpm) {
            pm_runtime_disable(&pdev->dev);
            pm_runtime_set_suspended(&pdev->dev);
        }
        pm_runtime_put_noidle(&pdev->dev);
        return ret;
    }

    if (use_rpm)
        pm_runtime_put_autosuspend(&pdev->dev);

    return 0;
}

static int dw_mci_rockchip_remove(struct platform_device *pdev)
{
    pm_runtime_get_sync(&pdev->dev);
    pm_runtime_disable(&pdev->dev);
    pm_runtime_put_noidle(&pdev->dev);

    dw_mci_pltfm_remove(pdev);

    return 0;
}

static const struct dev_pm_ops dw_mci_rockchip_dev_pm_ops = {
    SET_SYSTEM_SLEEP_PM_OPS(pm_runtime_force_suspend,
                pm_runtime_force_resume)
    SET_RUNTIME_PM_OPS(dw_mci_runtime_suspend,
                dw_mci_runtime_resume,
                NULL)
};

static struct platform_driver dw_mci_rockchip_pltfm_driver = {
    .probe      = dw_mci_rockchip_probe,
    .remove     = dw_mci_rockchip_remove,
    .driver     = {
        .name       = "dwmmc_rockchip",
        .probe_type = PROBE_PREFER_ASYNCHRONOUS,
        .of_match_table = dw_mci_rockchip_match,
        .pm     = &dw_mci_rockchip_dev_pm_ops,
    },
};

module_platform_driver(dw_mci_rockchip_pltfm_driver);
```

```
MODULE_AUTHOR("Addy Ke <addy.ke@rock-chips.com>");
MODULE_DESCRIPTION("Rockchip Specific DW-MSHC Driver Extension");
MODULE_ALIAS("platform:dwmmc_rockchip");
MODULE_LICENSE("GPL v2");
```

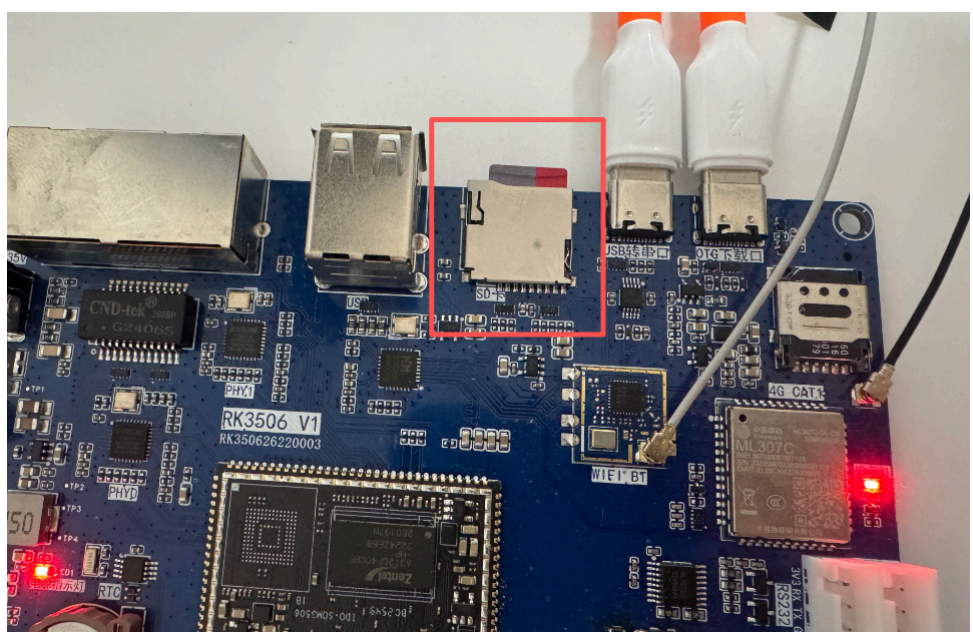
- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    <*> MMC/SD/SDIO card support  --->
        <*>   MMC block device driver
        (8)   Number of minors per block device
        <*>   Synopsys DesignWare Memory Card Interface
        -*-     Synopsys Designware MCI Support as platform device
        <*>     Rockchip specific extensions for Synopsys DW Memory Card Interface

linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

- 编译内核更新设备树与内核到开发板内。

3. SD卡功能测试

- 插入SD卡




```
root@rk3506-buildroot:/#  插入SD卡
root@rk3506-buildroot:/# [ 2064.699962] mmc_host mmc0: Bus speed (slot 0) = 50000000Hz (slot req 50000000Hz,
actual 50000000HZ div = 0)
[ 2064.700123] mmc0: new high speed SDHC card at address aaaa
[ 2064.702061] mmcblk0: mmc0:aaaa SC16G 14.8 GiB
[ 2064.709975]  mmcblk0: p1
[ 2065.143667] FAT-fs (mmcblk0p1): Volume was not properly unmounted. Some data may be corrupt. Please run
fsck.

root@rk3506-buildroot:/# ls /mnt/
external_sd  sdcard       storage      udisk        usb_storage
root@rk3506-buildroot:/# ls /mnt/sdcard/
???SD??.txt                System Volume Information
Illuminati                 TEST.TXT
MUSIC                      hello.c
root@rk3506-buildroot:/# echo "goodbye" > /mnt/sdcard/hello.txt
root@rk3506-buildroot:/# cat /mnt/sdcard/hello.txt
goodbye
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# df
Filesystem           1K-blocks      Used Available Use% Mounted on
ubi0:rootfs             115488     95008     20480  82% /
devtmpfs                115156         0    115156   0% /dev
tmpfs                   115248       200    115048   0% /var/log
tmpfs                   115248         8    115240   0% /tmp
tmpfs                   115248       236    115012   0% /run
tmpfs                   115248         0    115248   0% /dev/shm
/dev/ubi6_0               7796      4296      3500  55% /oem
/dev/ubi7_0              29700       380     29320   1% /userdata
/dev/mmcblk0p1        15540224     46728  15493496   0% /mnt/sdcard
root@rk3506-buildroot:/#   # 拔掉SD卡
root@rk3506-buildroot:/# [ 2349.907796] mmc0: card aaaa removed            # 拔掉SD卡

root@rk3506-buildroot:/#
```

第16章 Linux USB驱动

16.1. USB 通用串行总线

1. 什么是USB

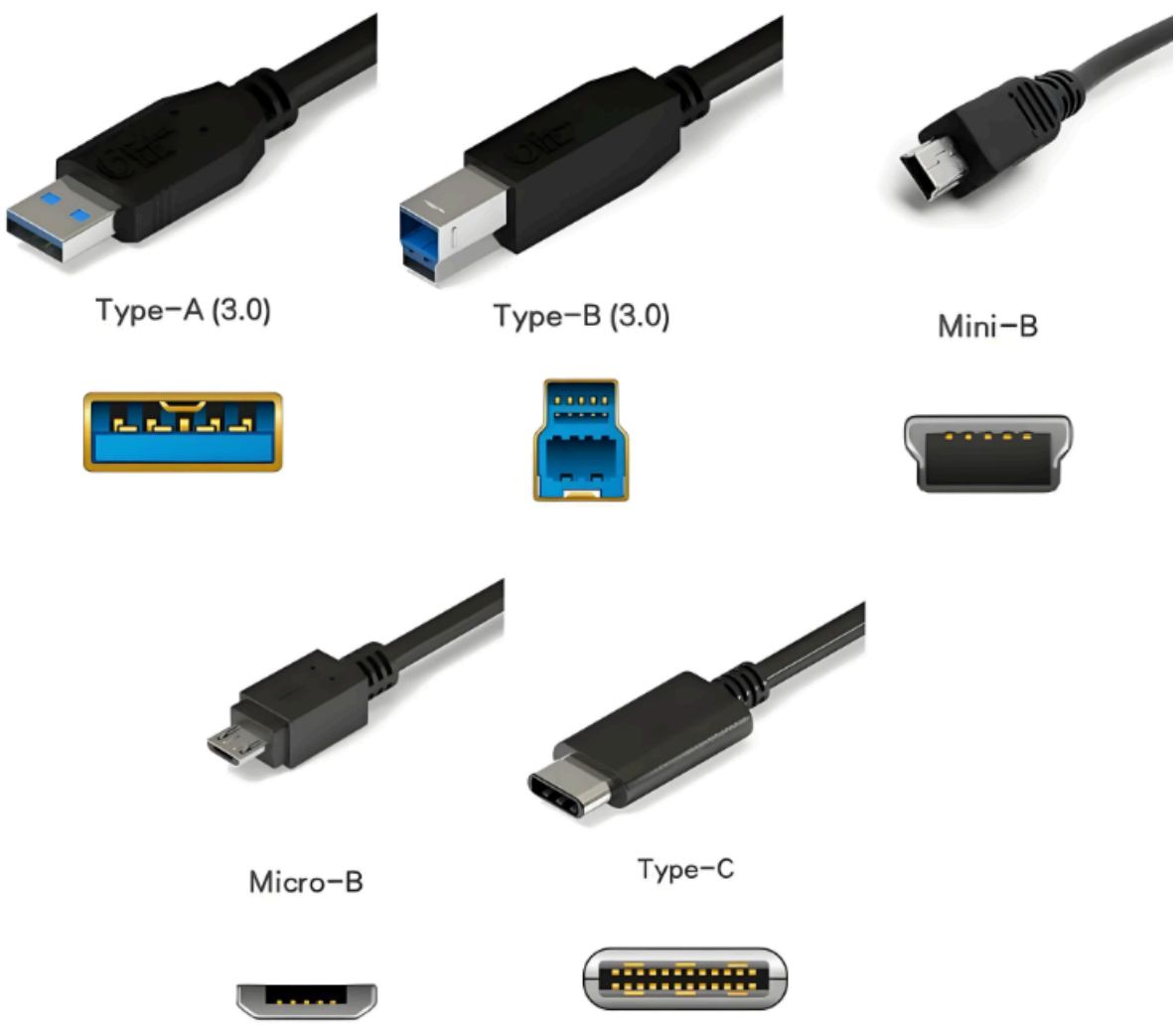
USB 的全称是 **U**niversal **S**erial **B**us。全称为 **通用串行总线**，是一种行业标准。

它最主要的功能是**定义电脑与外部设备（如U盘、键盘、手机等）之间的连接、通信和供电方式**

通用串行总线（Universal Serial Bus，简称USB）是1994年提出的一种支持热插拔的外部传输总线，现在不管是在什么设备上都可以非常容易看到它。

2. USB接口类型

我们来详细梳理一下常见的USB接口类型。需要注意的是，USB接口有两个维度：**接口形状（Type）** 和 **支持的标准（版本，如USB 2.0, 3.0）**。形状决定了物理连接方式，而版本决定了速度和功能。



1. USB Type-A（标准口）

这是**最常见、最经典**的USB接口。

- **特点**：矩形设计，有方向性，必须正面朝上才能插入。
- **用途**：绝大多数电脑主机、笔记本电脑、U盘、充电器上都使用这种接口。
- **提示**：蓝色接口的Type-A通常代表支持USB 3.0或更高标准，传输速度更快。

2. USB Type-B（方口）

这种接口通常用于**外部设备**端。

- **特点**：近似方形，边缘有斜面。
- **用途**：常见于打印机、扫描仪、部分外置硬盘盒和台式显示器。

3. Mini-USB

这是**Micro-USB出现之前**的便携设备接口。

- **特点**：比Type-A和B小，有一个凹槽。
- **用途**：在2000年代末期非常流行，用于旧款的数码相机、MP3播放器、移动硬盘等。**现在已基本被淘汰。**

4. Micro-USB

这是**2010年代安卓设备的绝对霸主。**

- **特点**：比Mini-USB更扁更小，更耐用。
- **用途**：曾是几乎所有安卓手机、平板、蓝牙耳机、充电宝的标准接口。
- **现状**：正被Type-C快速取代，但目前仍大量存在于很多低端手机和各类小型电子产品中。

5. USB Type-C（简称Type-C或C口）

这是**现在和未来的统一标准。**

- **特点**：
  - **最核心优势：正反可插。**没有方向性，使用时非常方便。
  - **更小巧**：适合超薄设备。
  - **功能强大**：不仅支持更快的数据传输（如USB 3.2 Gen 2、USB4），还支持强大的电力传输（USB PD，最高可达240W），并能兼容视频输出（如DisplayPort、HDMI），实现"一线连"显示器。
- **用途**：现代安卓手机、苹果iPad Pro/Air、MacBook、Windows轻薄本、旗舰平板、 Switch游戏机等新设备已全面采用。
- **重要提示**：**不是所有Type-C接口都一样！** 它的功能取决于设备支持的USB标准。一个Type-C口可能只支持USB 2.0速度（如一些廉价TWS耳机充电仓），也可能支持满速的USB4（如高端笔记本）。

3. USB 版本（速度和协议）

发布年份	USB版本				标志	传输速度 (bit)	理论速度 (Byte)
	目前官方版本名		官方市场代号	原名			
1996	Low-Speed 低速		Basic-Speed USB	USB 1.0		1.5Mbps	0.1875MB/s
1998	Full-Speed 全速			USB 1.1		12Mbps	1.5MB/s
2000	High-Speed 高速		高速 Hi-Speed USB	USB 2.0		480Mbps	60MB/s
2008	USB 3.2	Gen 1×1	超高速 USB5Gbps SuperSpeed USB 5Gbps	USB 3.0（曾用名USB 3.1 Gen 1）		5Gbps	500MB/s
2013		Gen 2×1	超高速USB 10Gbps SuperSpeed USB 10Gbps	USB 3.1（曾用名 USB 3.1 Gen 2）		10Gbps	1212.12MB/s
2017		Gen 2x2	超高速USB 20Gbps SuperSpeed USB 20Gbps	N/A		20Gbps	2424.24MB/s
2019	USB4					40Gbps	5GB/s
2000	USB On-The-Go	USB On-The-Go Supplement 0.7–1.3					
	USB Power Delivery						
	Wireless USB						

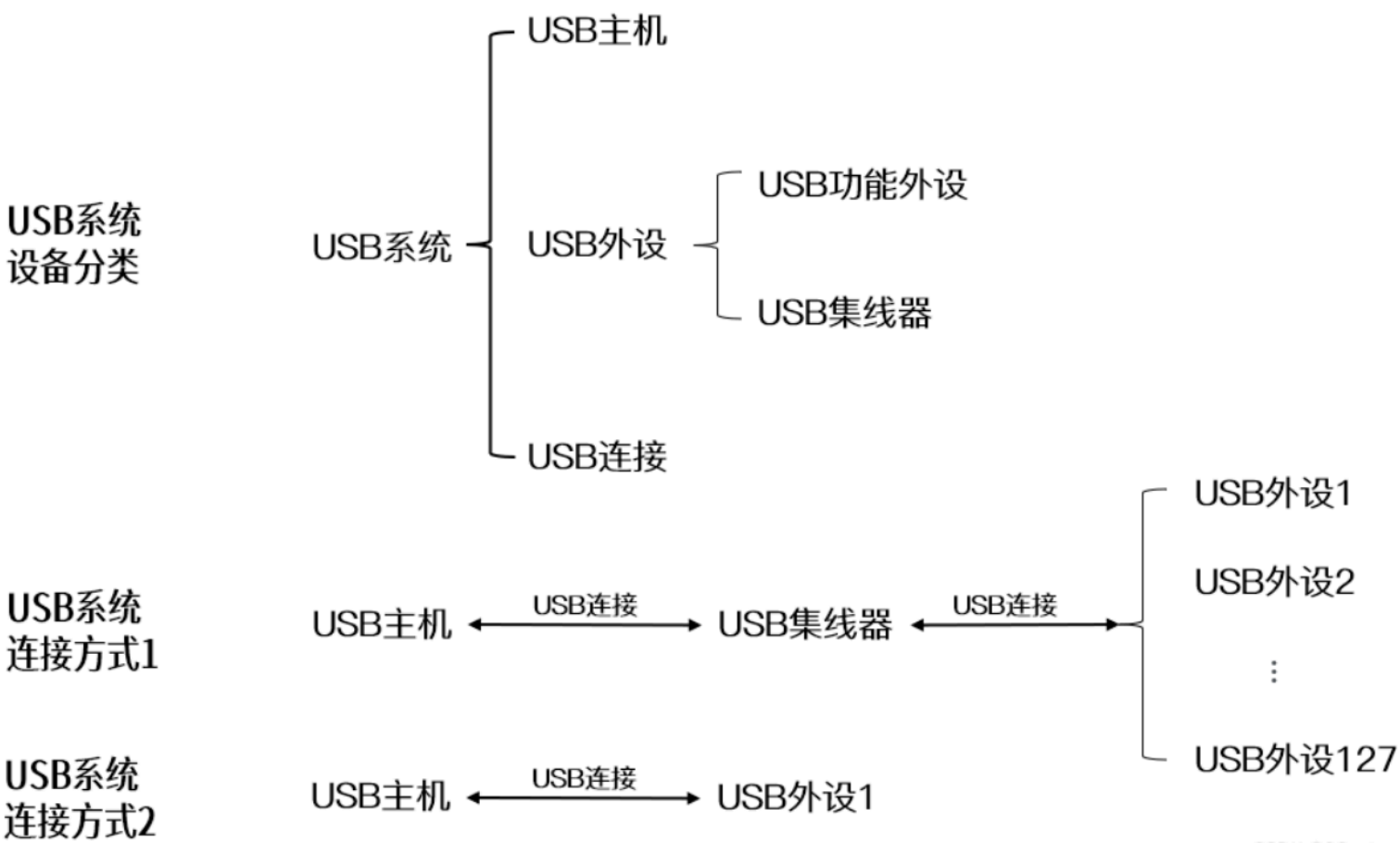
CSDN @OSnotes

4. USB系统

一个USB系统包括USB主机（USB HOST）、USB外设（USB DEVICE）和USB连接三个部分。

USB外设又可以分为USB功能外设（USB FUNCTION）和USB集线器（USB HUB）。

一个USB系统只能有一个USB主机，但USB主机通过连接USB集线器可以扩展USB端口数量，最多可以有127个USB从机，USB主机负责控制所有的USB外设，而USB外设接受USB主机的控制来实现某个具体的功能，USB设备分类及连接方式具体如下图所示



5. RK3506的USB

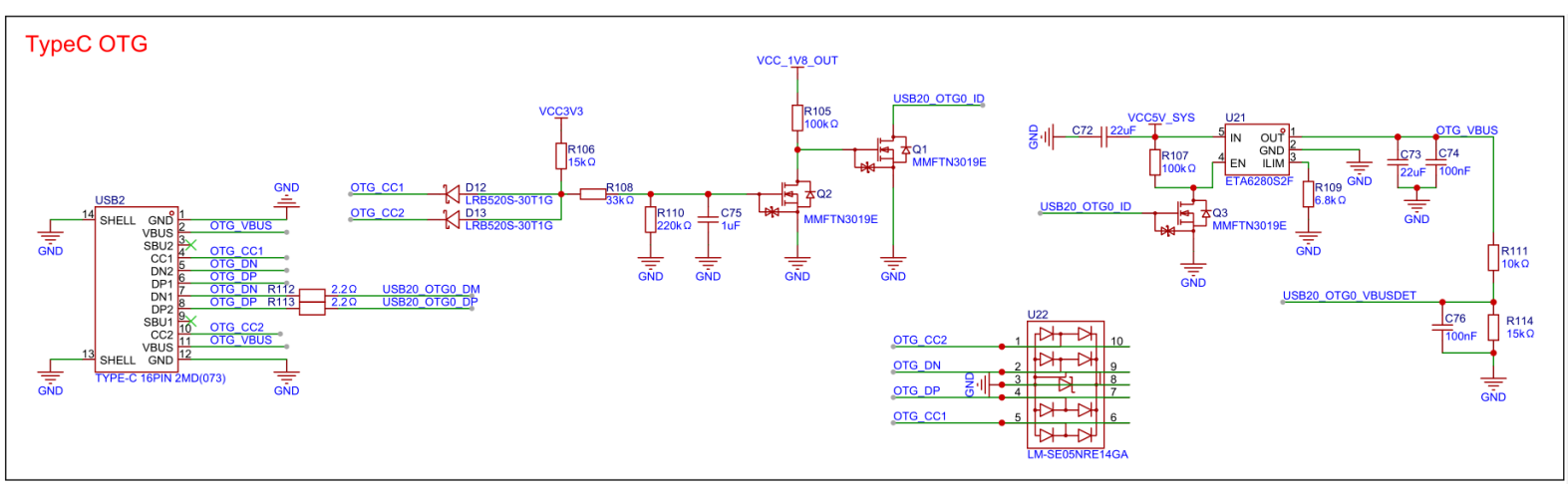
RK3506 内部集成了 **2 路 USB 2.0 控制器**，并且都内置了物理层收发器，外部设计时不需要额外添加 USB PHY 芯片，非常方便。这两路接口通常配置为：

- **两路 USB 2.0 OTG**（均向下兼容 USB 1.1/1.0）
- **最高速率**：均为 480Mbps（High Speed）
- **PHY 集成**：均内置 PHY，无需外部芯片
- **角色切换**：每路 OTG 都配有独立的 `USB_ID` 和 `USB_VBUSDET` 引脚，既可以由硬件 `ID` 电平自动切换，也可以通过软件强制设为 Host 或 Device 模式。
- **功能完全对等**：这两路 OTG 没有主从之分，功能是对称的，都可以独立地做主机接 U 盘、摄像头，也可以做设备连接电脑。

16.2 OTG接口原理图

- OGT0与OTG1接口

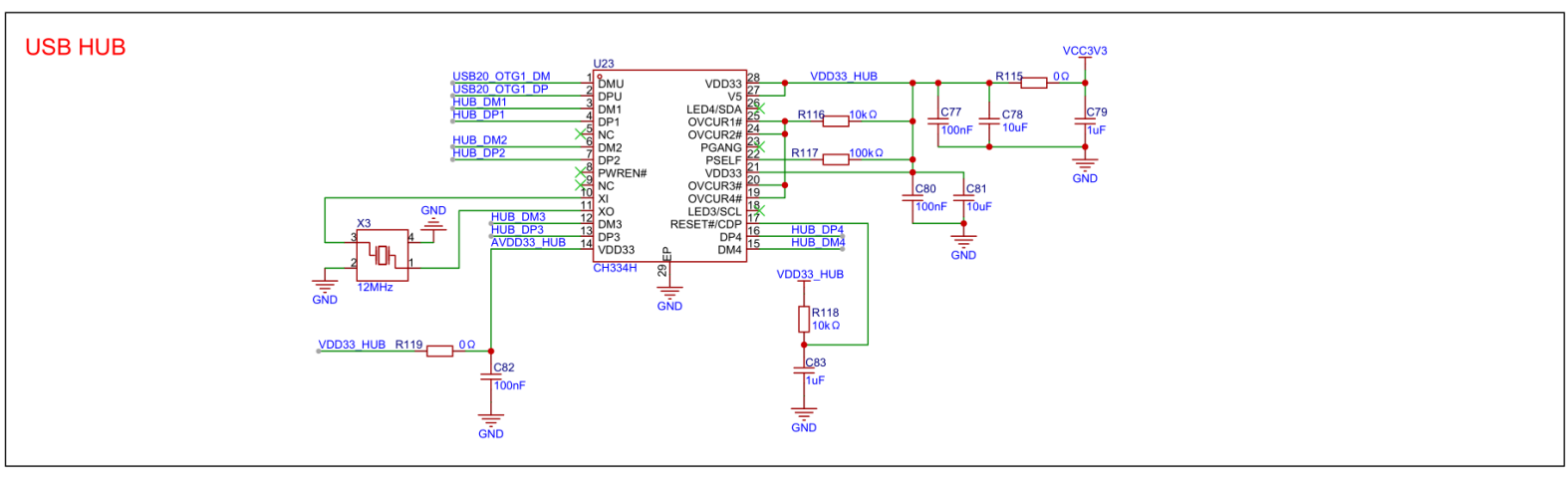
- OTG0



OTG0 接口原理（Type-C OTG 模式）：

- **模式识别**：Type-C 接口的 CC1/CC2 引脚经二极管 (D12/D13) 接入分压网络，通过控制 Q2 产生 `OTG0_ID` 信号，用于判断设备插入方向及主/从模式配置。
- **电源管理**：`OTG0_ID` 信号驱动 Q3 控制电源芯片 U21 (ETA6280S2F) 的 EN 端。U21 将系统 5V 转换为 `OTG_VBUS` 输出，以对连接的外设进行供电（支持高达 5V/1A 等配置）。
- **状态反馈**：通过 R111 与 R114 构成的分压电路，生成 `OTG0_VBUSDET` 检测信号，反馈给 SOC 以实现外设拔插状态的实时监控。
- **数据链路**：DM/DP 差分信号线串联 2.2Ω 电阻 (R112/R113) 后直接连接至 SOC，负责 HS/FS/LS 数据的高速交互。

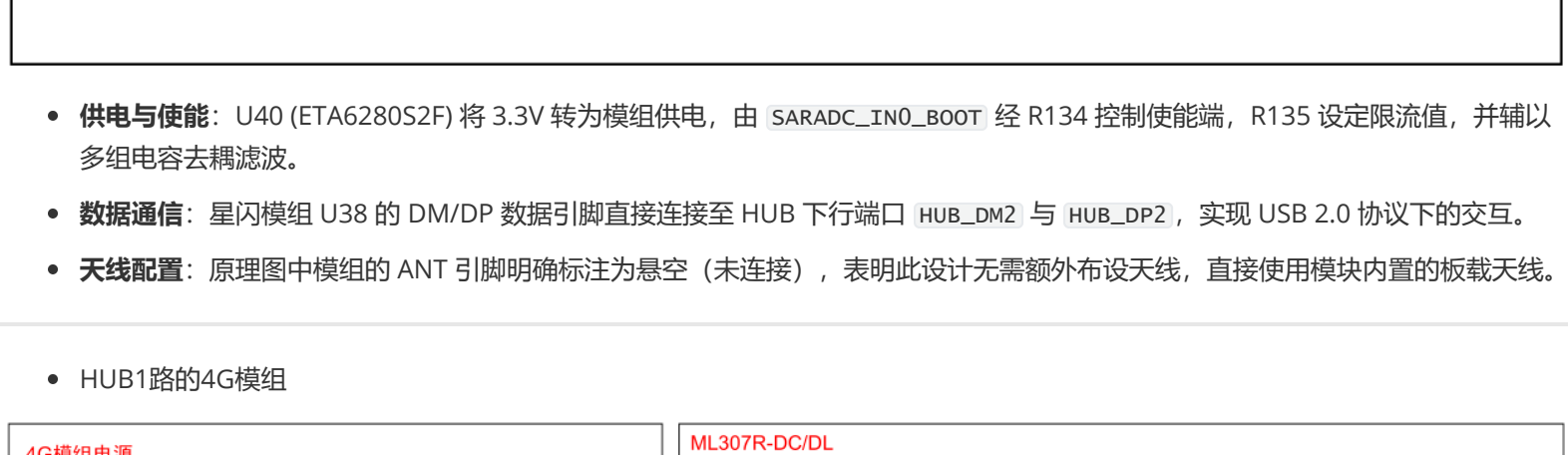
- OTG1



OTG1 接口原理（HUB 扩展模式）：

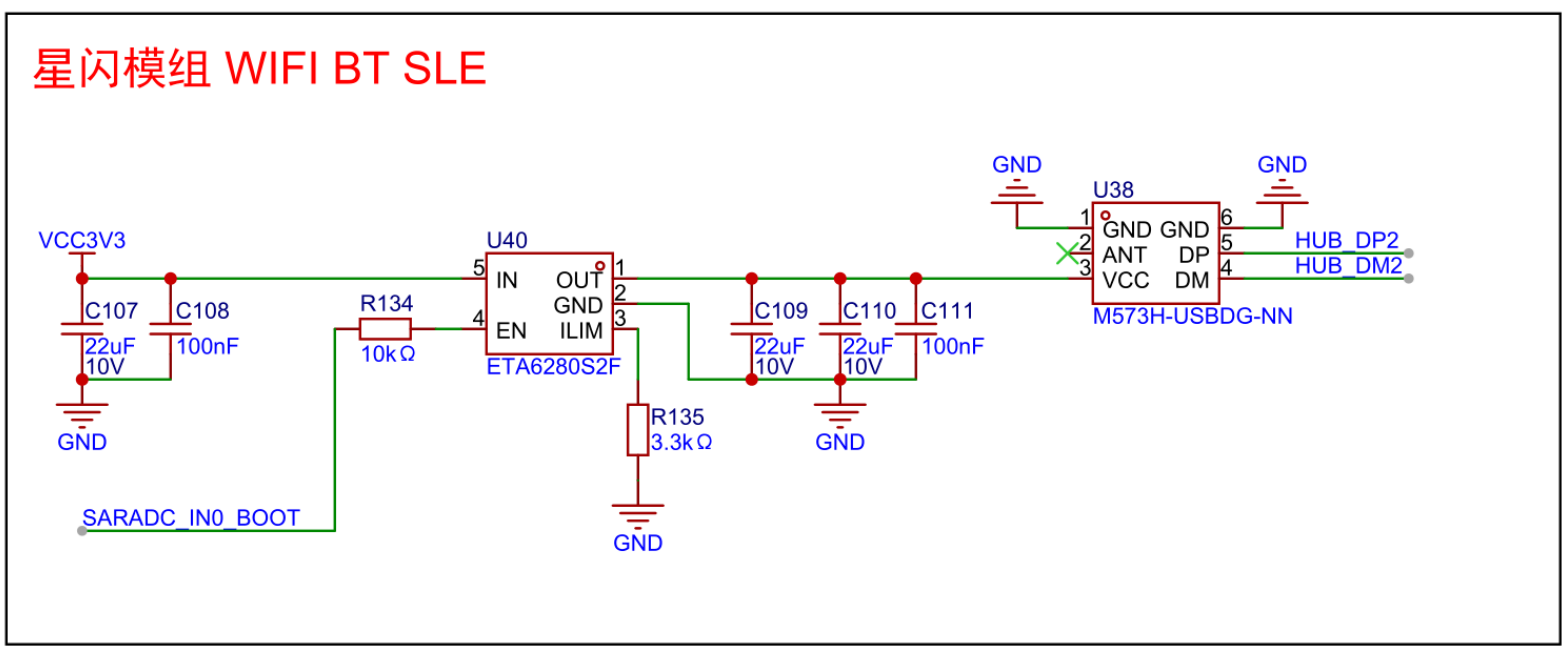
- **上行接入**：SOC 输出的 `USB20_OTG1_DM/DP` 信号作为主机端的上行接口，直接接入 USB HUB 芯片 U23 (CH334H) 的第 1、2 脚。
- **时钟与电源**：HUB 芯片由外部 12MHz 晶振 (X3) 提供基准时钟，并由独立的 3.3V 稳压电源 (VDD33_HUB) 供电，确保内部 PHY 和逻辑电路稳定运行。
- **功能扩展**：U23 将单路 USB 信号扩展为 4 路下行端口（通过 `HUB_DM/DP1~4` 引脚输出）。该链路在系统软件层通常被强制配置为 HOST 模式，实现单 SOC 接口支持多个外设同时接入的功能。

- HUB3~4路的双层USB口



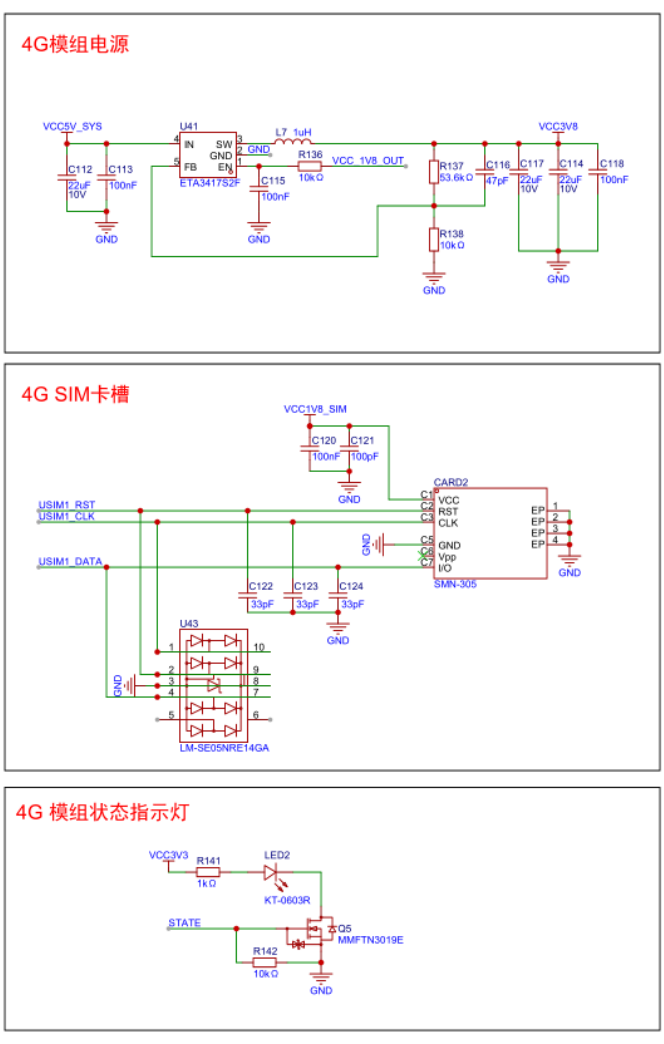
- **电源供应**：U24 (ETA6280S2F) 将系统5V转换为 `VCC5V_USB_HOST`，为接口 VBUS 供电，并由 R123 设定输出限流。
- **信号与防护**：两路 HUB 下行差分线（DM3/DP3、DM4/DP4）均串联 2.2Ω 匹配电阻，并配置 U25 滤波阵列，提供完善的 ESD 静电保护。
- **接口输出**：最终通过两个标准 USB Type-A 母口对外输出，实现两路独立的 USB 2.0 下行扩展主机端口。

- HUB2路的WIFIBT模组



- **供电与使能**：U40 (ETA6280S2F) 将 3.3V 转为模组供电，由 `SARADC_IN0_BOOT` 经 R134 控制使能端，R135 设定限流值，并辅以多组电容去耦滤波。
- **数据通信**：星闪模组 U38 的 DM/DP 数据引脚直接连接至 HUB 下行端口 `HUB_DM2` 与 `HUB_DP2`，实现 USB 2.0 协议下的交互。
- **天线配置**：原理图中模组的 ANT 引脚明确标注为悬空（未连接），表明此设计无需额外布设天线，直接使用模块内置的板载天线。

- HUB1路的4G模组



- **电源转换**：采用 ETA3475D2F 降压芯片将 5V 系统电源转换为 3.8V，并配合多组电容进行去耦滤波，为模组提供稳定可靠的供电。
- **SIM卡接口**：配备标准 6 引脚 SIM 卡槽，接口线上集成了专用的 ESD 静电防护器件，保障 USIM 数据通信的安全与稳定。
- **状态指示**：使用 GPIO 信号（`STATE`）控制三极管（Q5）驱动红色状态指示灯，用于实时反馈 4G 模组的运行与联网状态。

16.3 USB Host驱动

1. USB设备树

- USB控制器节点定义
- kernel/arch/arm/boot/dts/rk3502.dtsi

```
    usb2phy: usb2-phy@ff2b0000 {
        compatible = "rockchip,rk3506-usb2phy";
        reg = <0xff2b0000 0x8000>;
        clocks = <&cru CLK_REF_USBPHY_TOP>, <&cru PCLK_USBPHY>;
        clock-names = "phyclk", "apb_pclk";
        #clock-cells = <0>;
        rockchip,usbgrf = <&grf>;
        status = "disabled";

        u2phy_otg0: otg-port {
            #phy-cells = <0>;
            interrupts = <GIC_SPI 75 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 76 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 77 IRQ_TYPE_LEVEL_HIGH>;
            interrupt-names = "otg-bvalid",
                      "otg-id",
                      "linestate";
            rockchip,low-power-mode;
            status = "disabled";
        };

        u2phy_otg1: host-port {
            #phy-cells = <0>;
            interrupts = <GIC_SPI 80 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 81 IRQ_TYPE_LEVEL_HIGH>,
                     <GIC_SPI 82 IRQ_TYPE_LEVEL_HIGH>;
            interrupt-names = "otg-bvalid",
                      "otg-id",
                      "linestate";
            status = "disabled";
        };
    };



    usb20_otg0: usb@ff740000 {
        compatible = "rockchip,rk3506-usb", "rockchip,rk3066-usb",
                 "snps,dwc2";
        reg = <0xff740000 0x40000>;
        interrupts = <GIC_SPI 74 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru HCLK_USBOTG0>, <&cru HCLK_USBOTG0_PMU>,
             <&cru CLK_USBOTG0_ADP>;
```

```
        clock-names = "otg", "pmu", "adp";
        dr_mode = "otg";
        phys = <&u2phy_otg0>;
        phy-names = "usb2-phy";
        g-np-tx-fifo-size = <16>;
        g-rx-fifo-size = <280>;
        g-tx-fifo-size = <256 128 128 64 32 16>;
        status = "disabled";
    };

    usb20_otg1: usb@ff780000 {
        compatible = "rockchip,rk3506-usb", "rockchip,rk3066-usb",
                 "snps,dwc2";
        reg = <0xff780000 0x40000>;
        interrupts = <GIC_SPI 79 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru HCLK_USBOTG1>, <&cru HCLK_USBOTG1_PMU>,
             <&cru CLK_USBOTG1_ADP>;
        clock-names = "otg", "pmu", "adp";
        dr_mode = "otg";
        phys = <&u2phy_otg1>;
        phy-names = "usb2-phy";
        g-np-tx-fifo-size = <16>;
        g-rx-fifo-size = <280>;
        g-tx-fifo-size = <256 128 128 64 32 16>;
        status = "disabled";
    };
```

- `u2phy_otg0` 多了 `rockchip,low-power-mode` 属性，支持低功耗模式
- **OTG 是功能**，定义了设备的软件协议，**PHY 是硬件**，负责发送数据的物理层，类似网卡与网卡PHY。
- usb2phy： 是usb的phy的物理层。
- usb20_otg0： 是usb的软件协议。

- USB电源配置
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    vcc5v0_otg0: vcc5v0-otg0-regulator {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_otg0";
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        //gpio = <&gpio1 RK_PC4 GPIO_ACTIVE_HIGH>;
        //pinctrl-names = "default";
        //pinctrl-0 = <&vcc5v0_otg0_en>;
    };

    vcc5v0_otg1: vcc5v0-otg1-regulator {
        compatible = "regulator-fixed";
        regulator-name = "vcc5v0_otg1";
        regulator-boot-on;
        regulator-always-on;
        regulator-min-microvolt = <5000000>;
        regulator-max-microvolt = <5000000>;
        enable-active-high;
        //gpio = <&gpio1 RK_PD0 GPIO_ACTIVE_HIGH>;
        //pinctrl-names = "default";
        //pinctrl-0 = <&vcc5v0_otg1_en>;
    };


    &u2phy_otg0 {
        vbus-supply = <&vcc5v0_otg0>;    // VBUS 由 vcc5v0_otg0 提供
        status = "okay";
    };

    &u2phy_otg1 {
        phy-supply = <&vcc5v0_otg1>;     // PHY 电源由 vcc5v0_otg1 提供
        status = "okay";
    };

    &usb2phy {
        status = "okay";                  // 使能 USB 2.0 PHY 顶层
    };

    &usb20_otg0 {
        status = "okay";                  // 使能 OTG0 (默认 dr_mode = "otg")
    };

    &usb20_otg1 {
        dr_mode = "host";                 // 强制为主机模式
        status = "okay";
    };


// 两个 VBUS 使能引脚的 pinctrl 均已注释，与电源节点中的 GPIO 注释一致。
usb {
    //vcc5v0_otg0_en: vcc5v0-otg0-en {
    //      rockchip,pins = <1 RK_PC4 RK_FUNC_GPIO &pcfg_pull_none>;
    //};
    //vcc5v0_otg1_en: vcc5v0-otg1-en {
    //      rockchip,pins = <1 RK_PD0 RK_FUNC_GPIO &pcfg_pull_none>;
    //};
};
```

- 两个 OTG 的 5V 电源 GPIO 控制都被注释掉了，这意味着 VBUS 供电要么由硬件固定使能。

---

2. USB驱动分析

- 关键驱动文件

文件	作用
drivers/usb/dwc2/platform.c	DWC2 平台驱动：probe/remove、PHY 获取、时钟/regulator
drivers/usb/dwc2/params.c	参数匹配：`rk3066-usb` → `dwc2_set_rk_params()`
drivers/usb/dwc2/core.c	DWC2 核心：控制器复位、初始化、中断分发
drivers/usb/dwc2/hcd.c	Host Controller Driver — 主机模式
drivers/usb/dwc2/gadget.c	Peripheral Controller — 设备模式
drivers/usb/dwc2/drd.c	Dual-Role 切换逻辑
drivers/phy/rockchip/phy-rockchip-inno-usb2.c	RK3506 USB2 PHY 驱动 (Innosilicon IP)

- kernel/drivers/phy/rockchip/phy-rockchip-inno-usb2.c

```
// SPDX-License-Identifier: GPL-2.0-or-later
/*
 * Rockchip USB2.0 PHY with Innosilicon IP block driver
 *
 * Copyright (C) 2016 Fuzhou Rockchip Electronics Co., Ltd
 */

#include <linux/clk.h>
#include <linux/clk-provider.h>
#include <linux/delay.h>
#include <linux/extcon-provider.h>
#include <linux/interrupt.h>
#include <linux/io.h>
#include <linux/gpio/consumer.h>
#include <linux/jiffies.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_irq.h>
#include <linux/of_platform.h>
#include <linux/phy/phy.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/power_supply.h>
#include <linux/regmap.h>
#include <linux/reset.h>
#include <linux/rockchip/cpu.h>
#include <linux/mfd/syscon.h>
#include <linux/usb/of.h>
#include <linux/usb/otg.h>
#include <linux/usb/role.h>
#include <linux/usb/typec_mux.h>
#include <linux/wakelock.h>
```

```
#define BIT_WRITEABLE_SHIFT 16
#define SCHEDULE_DELAY      (60 * HZ)
#define OTG_SCHEDULE_DELAY  (1 * HZ)
#define BYPASS_SCHEDULE_DELAY   (2 * HZ)
#define FILTER_COUNTER      0xF4240


static const struct of_device_id rockchip_usb2phy_dt_match[] = {
#ifdef CONFIG_CPU_PX30
        { .compatible = "rockchip,px30-usb2phy", .data = &px30_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK1808
        { .compatible = "rockchip,rk1808-usb2phy", .data = &rk1808_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK312X
        { .compatible = "rockchip,rk3128-usb2phy", .data = &rk312x_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK322X
        { .compatible = "rockchip,rk3228-usb2phy", .data = &rk3228_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3308
        { .compatible = "rockchip,rk3308-usb2phy", .data = &rk3308_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3328
        { .compatible = "rockchip,rk3328-usb2phy", .data = &rk3328_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3366
        { .compatible = "rockchip,rk3366-usb2phy", .data = &rk3366_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3368
        { .compatible = "rockchip,rk3368-usb2phy", .data = &rk3368_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3399
        { .compatible = "rockchip,rk3399-usb2phy", .data = &rk3399_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3506
        { .compatible = "rockchip,rk3506-usb2phy", .data = &rk3506_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3528
        { .compatible = "rockchip,rk3528-usb2phy", .data = &rk3528_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3562
        { .compatible = "rockchip,rk3562-usb2phy", .data = &rk3562_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3568
        { .compatible = "rockchip,rk3568-usb2phy", .data = &rk3568_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3576
        { .compatible = "rockchip,rk3576-usb2phy", .data = &rk3576_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RK3588
        { .compatible = "rockchip,rk3588-usb2phy", .data = &rk3588_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RV1106
        { .compatible = "rockchip,rv1106-usb2phy", .data = &rv1106_phy_cfgs },
#endif
#ifdef CONFIG_CPU_RV1108
        { .compatible = "rockchip,rv1108-usb2phy", .data = &rv1108_phy_cfgs },
#endif
        {}
};
MODULE_DEVICE_TABLE(of, rockchip_usb2phy_dt_match);

static struct platform_driver rockchip_usb2phy_driver = {
        .probe          = rockchip_usb2phy_probe,
        .driver         = {
                .name   = "rockchip-usb2phy",
                .pm     = ROCKCHIP_USB2PHY_DEV_PM,
                .of_match_table = rockchip_usb2phy_dt_match,
        },
};
module_platform_driver(rockchip_usb2phy_driver);

MODULE_AUTHOR("Frank Wang <frank.wang@rock-chips.com>");
MODULE_DESCRIPTION("Rockchip USB2.0 PHY driver");
MODULE_LICENSE("GPL v2");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    [*] USB support  --->
        <M> Support for Host-side USB       # 是usbcore提供的 USB 核心协议栈，编译成模块由eudev自动加载。
        <M>   USB Gadget Support  --->      # USB 设备侧（Peripheral）协议栈核心，就是OTG可以作为设备被使用。
        <M>   DesignWare USB2 DRD Core Support
        #DesignWare USB 2.0 OTG 控制器的平台驱动。是 RK3506 的实际 USB 控制器硬件驱动。

              DWC2 Mode Selection (Dual Role mode)  --->
        {*}   USB Role Switch Support        #提供USB角色切换的通用API，DWC2 用它来在 Host 和 Device 之间动态切换
        <M>   OHCI HCD (USB 1.1) support     # 多余选项， 可以去可以不去
        <M>   EHCI HCD (USB 2.0) support     # 多余选项， 可以去可以不去
        <M>   xHCI HCD (USB 3.0) support     # 多余选项， 可以去可以不去
        <M>   USB Mass Storage support       # 大容量存储类驱动。OTG1 作为 Host 插 U 盘读写文件。

        <M>   USB Serial Converter support  --->
            [*]   USB Generic Serial Driver                          #USB 串口核心框架 (usbserial.ko)
            <M>   USB Winchiphead CH341 Single Port Serial Driver    #CH341 USB 转串口
            <M>   USB Winchiphead CH9344 Single Port Serial Driver   #CH9344 USB 转 4 路串口
            <M>   USB CP210x family of UART Bridge Controllers       #Silicon Labs CP210x USB 转串口
            <M>   USB FTDI Single Port Serial Driver                 #FTDI FT232 系列 USB 转串口
            <M>   USB driver for GSM and CDMA modems                 #华为/中兴等 4G 模块 (CDC-WDM)

        [*]   OTG support            #提供 USB OTG 协议相关的辅助功能（主要是 /sys 接口和 OTG 状态字符串定义）。

        <M>   USB Printer support    # USB 打印机类驱动（Host 侧）。极少使用。

    PHY Subsystem  --->
        <M> Rockchip INNO USB2PHY Driver   #这就是RK3506 USB 2.0 PHY 驱动（Innosilicon IP）

    <*> External Connector Class (extcon) support  --->
        <*>   USB GPIO extcon support

    #USB Human Interface Device 类驱动。用于 Host 模式下连接 USB 键盘、鼠标、触摸屏等 HID 设备。
    HID support  --->
        USB HID support  --->
            <M> USB HID transport layer

    [*] Network device support  --->
        #OTG1 Host 模式插 USB 网卡，或 OTG0 Device 模式向 PC 暴露 RNDIS 网卡功能。
          <M>   USB Network Adapters  --->
              <M> Multi-purpose USB Networking Framework
              各种 USB 网卡驱动...

linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

- DWC2 Mode Selection (Dual Role mode)

模式	你的配置	含义
`USB_DWC2_HOST`	未选中	纯主机模式 — 只编译 `hcd.c`，不支持 device
`USB_DWC2_PERIPHERAL`	未选中	纯设备模式 — 只编译 `gadget.c`，不支持 host
`USB_DWC2_DUAL_ROLE`	`=y`	双角色模式 — 同时编译 `hcd.c` + `gadget.c` + `drd.c`

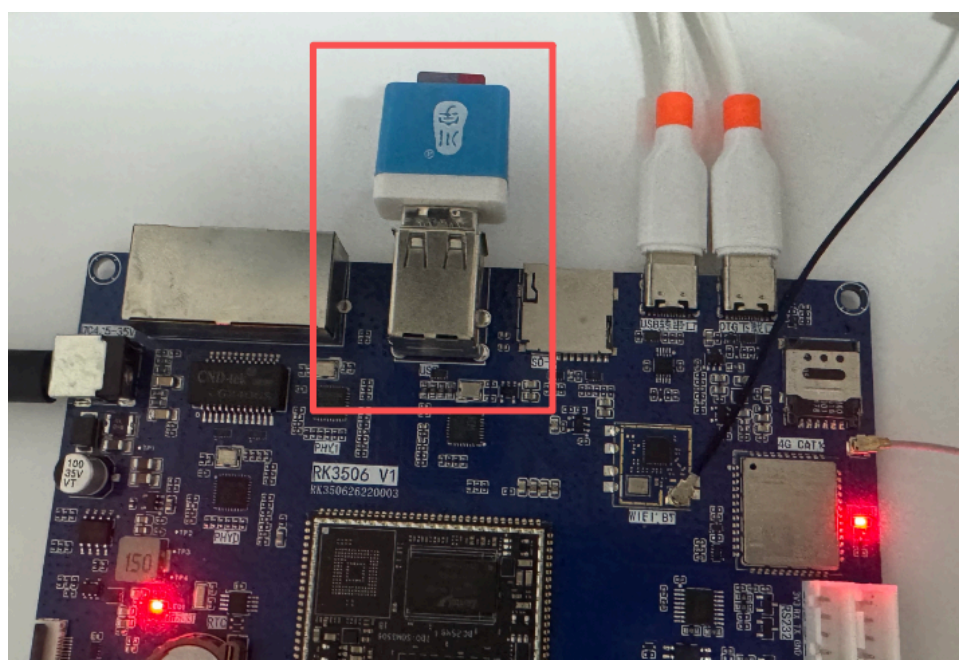
- Dual Role 在 RK3506 上的实际作用

控制器	DT `dr_mode`	实际运行角色
`usb20_otg0`	`"otg"` (DT 默认)	可切换 — ID 引脚决定是 Host 还是 Device
`usb20_otg1`	`"host"` (板级覆盖)	固定 Host — Dual Role 框架将其锁定为主机模式

- OHCI，EHCI，xHCI： RK3506 的 USB 控制器是 DWC2，不是 EHCI/OHCI。`ehci-hcd.ko` 和 `ohci-hcd.ko` 对 RK3506 没有任何作用，可以安全关闭。它们是为带标准 EHCI/OHCI 控制器的主控芯片（如 Allwinner、某些 x86）准备的。
- External Connector Class (extcon) support：EXTCON（外部连接器检测），PHY 驱动用 extcon 来通知 DWC2
  - `EXTCON_USB` — VBUS 已连接（设备插入/上电）
  - `EXTCON_USB_HOST` — ID 引脚接地（OTG 应切为 Host）
  - `EXTCON_USB_VBUS_EN` — VBUS 供电使能状态
- 编译内核更新设备树与内核到开发板内。

3. USB功能测试

- 插入U盘




```
root@rk3506-buildroot:/#  插入U盘
root@rk3506-buildroot:/# [ 6476.239691] usb 2-1.4: new high-speed USB device number 6 using dwc2
[ 6476.335738] usb-storage 2-1.4:1.0: USB Mass Storage device detected
[ 6476.337225] scsi host0: usb-storage 2-1.4:1.0
[ 6477.360530] scsi 0:0:0:0: Direct-Access     Mass     Storage Device   1.00 PQ: 0 ANSI: 0 CCS
[ 6477.592000] sd 0:0:0:0: [sda] 31116288 512-byte logical blocks: (15.9 GB/14.8 GiB)
[ 6477.592490] sd 0:0:0:0: [sda] Write Protect is off
[ 6477.593011] sd 0:0:0:0: [sda] No Caching mode page found
[ 6477.593123] sd 0:0:0:0: [sda] Assuming drive cache: write through
[ 6477.601996]  sda: sda1
[ 6477.602657] sd 0:0:0:0: [sda] Attached SCSI removable disk
[ 6477.990737] FAT-fs (sda1): Volume was not properly unmounted. Some data may be corrupt. Please run fsck.

root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ls /mnt/
external_sd  sdcard       storage      udisk        usb_storage
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ls /mnt/udisk/
???SD??.txt                System Volume Information
Illuminati                 hello.c
MUSIC                      hello.txt
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# echo "goodbye" > /mnt/udisk/test.txt
root@rk3506-buildroot:/# ls /mnt/udisk/
???SD??.txt                hello.c
Illuminati                 hello.txt
MUSIC                      test.txt
System Volume Information
root@rk3506-buildroot:/# cat /mnt/udisk/test.txt
goodbye
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# df
Filesystem           1K-blocks      Used Available Use% Mounted on
ubi0:rootfs             115488     95008     20480  82% /
devtmpfs                115156         0    115156   0% /dev
tmpfs                   115248       128    115120   0% /var/log
tmpfs                   115248         8    115240   0% /tmp
tmpfs                   115248       240    115008   0% /run
tmpfs                   115248         0    115248   0% /dev/shm
/dev/ubi7_0              29700       380     29320   1% /userdata
/dev/ubi6_0               7796      4296      3500  55% /oem
/dev/sda1             15540224     46720  15493504   0% /mnt/udisk
root@rk3506-buildroot:/#
root@rk3506-buildroot:/#   # 拔掉U盘
root@rk3506-buildroot:/# [ 6968.321598] usb 2-1.4: USB disconnect, device number 6

root@rk3506-buildroot:/#
```

16.4 USB 4G 驱动

1. ML307介绍

ML307C/R是中国移动物联网公司（中移物联）推出的一款小尺寸、国产化的4G LTE Cat.1无线通信模组。它专为物联网应用设计，旨在提供高性价比、低功耗且可靠的通信解决方案。

- ML307C的核心技术规格如下：

特性	详细规格
产品定位	小尺寸、国产化、高性价比的LTE Cat.1无线通信模组
芯片平台	翱捷科技（ASR）**ASR1605**，主频**614MHz**
网络制式	**4G Cat.1**（FDD-LTE/TDD-LTE），部分型号支持**2G GSM**网络回落
频段支持	**FDD**: B1/B3/B5/B8 **TDD**: B34/B38/B39/B40/B41
传输速率	**FDD**: 下行10Mbps / 上行5Mbps **TDD**: 下行9Mbps / 上行3Mbps
物理封装	**LCC+LGA** 封装，**109个引脚**（LCC 48个，LGA 61个）
物理尺寸	**17.7mm × 15.8mm × 2.2mm**
工作电压	**3.4V ~ 4.5V**（典型值3.8V）
工作温度	**-40℃ ~ +85℃**（工业级宽温）
接口资源	**UART**、**USB 2.0**、**I2C**、**SPI**、**ADC**、**PCM**、GPIO等
网络协议	支持IPv4/IPv6、TCP/UDP、HTTP/HTTPS、MQTT/MQTTS、FTP、LWM2M等

ML307C已在多个领域得到应用与验证：

- **智能出行**：**两轮电动车**的智能网联化、**车载智能机器人**的数据传输与交互、车辆追踪与车载T-BOX。
- **智能表计**：电表、水表、燃气表等设备的远程抄表。
- **共享经济**：共享充电宝、共享单车、自动售货机等设备的通信。
- **安防与追踪**：视频监控、定位追踪器、烟感、门禁等设备。
- **工业物联网**：工业传感器数据采集、环境监测、智慧农业等。

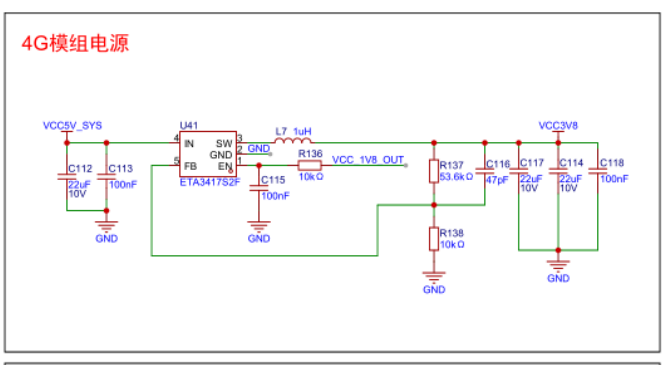
ML307C和ML307R的核心区别在于**产品定位**：**ML307R是“定位型”，内置了GNSS（北斗+GPS）功能；而ML307C是“紧凑型”**，主打小体积优化，没有内置定位功能。

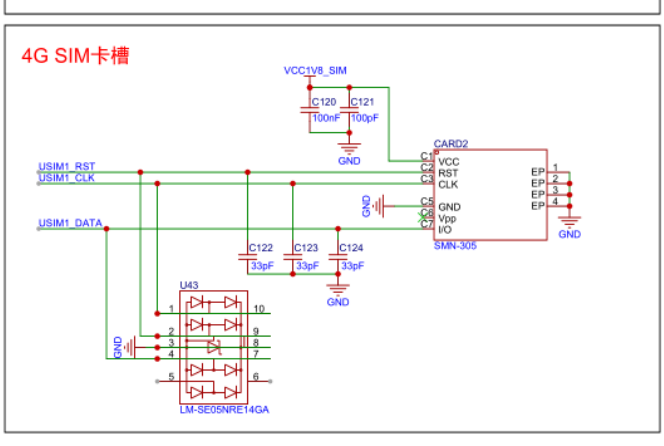
对比维度	ML307C (紧凑型)	ML307R (定位型)
核心定位	**紧凑型**，小体积优化，适合空间敏感场景	**定位型**，内置GNSS，用于追踪器等需要定位的设备
GNSS定位	**不支持**	**支持**（北斗+GPS）
物理封装	LCC+LGA，109引脚 (48 LCC + 61 LGA)	LCC+LGA，109引脚 (48 LCC + 61 LGA)
物理尺寸	17.7mm × 15.8mm × 2.2mm	17.7mm × 15.8mm × 2.2mm
网络制式	4G LTE Cat.1	4G LTE Cat.1
频段支持	LTE-FDD: B1/B3/B5/B8 LTE-TDD: B34/B38/B39/B40/B41	LTE-FDD: B1/B3/B5/B8 LTE-TDD: B34/B38/B39/B40/B41
传输速率	FDD: 下行10Mbps / 上行5Mbps TDD: 下行9Mbps / 上行3Mbps	FDD: 下行10Mbps / 上行5Mbps TDD: 下行9.010Mbps / 上行3.096Mbps
工作电压	3.4V ~ 4.5V	3.4V ~ 4.5V
OpenCPU	部分型号支持（如`ML307C-DC-CN`）	`ML307R-DC`型号支持
主要接口	UART, USB 2.0, I2C, SPI, ADC, PCM, GPIO等	UART, USB, GPIO, ADC, PWM等

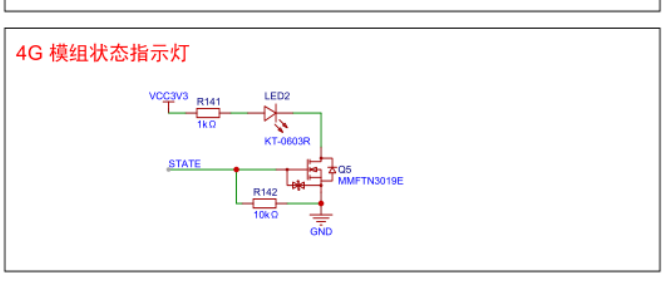
- **选择 ML307C**：如果你的**产品不需要定位功能**，但对**设备体积**有严格要求，希望在设计上更紧凑，那么ML307C是更合适的选择。
- **选择 ML307R**：如果你的**产品需要定位功能**，例如用于车辆追踪、人员定位、电子围栏等场景，那么内置GNSS的ML307R是首选。

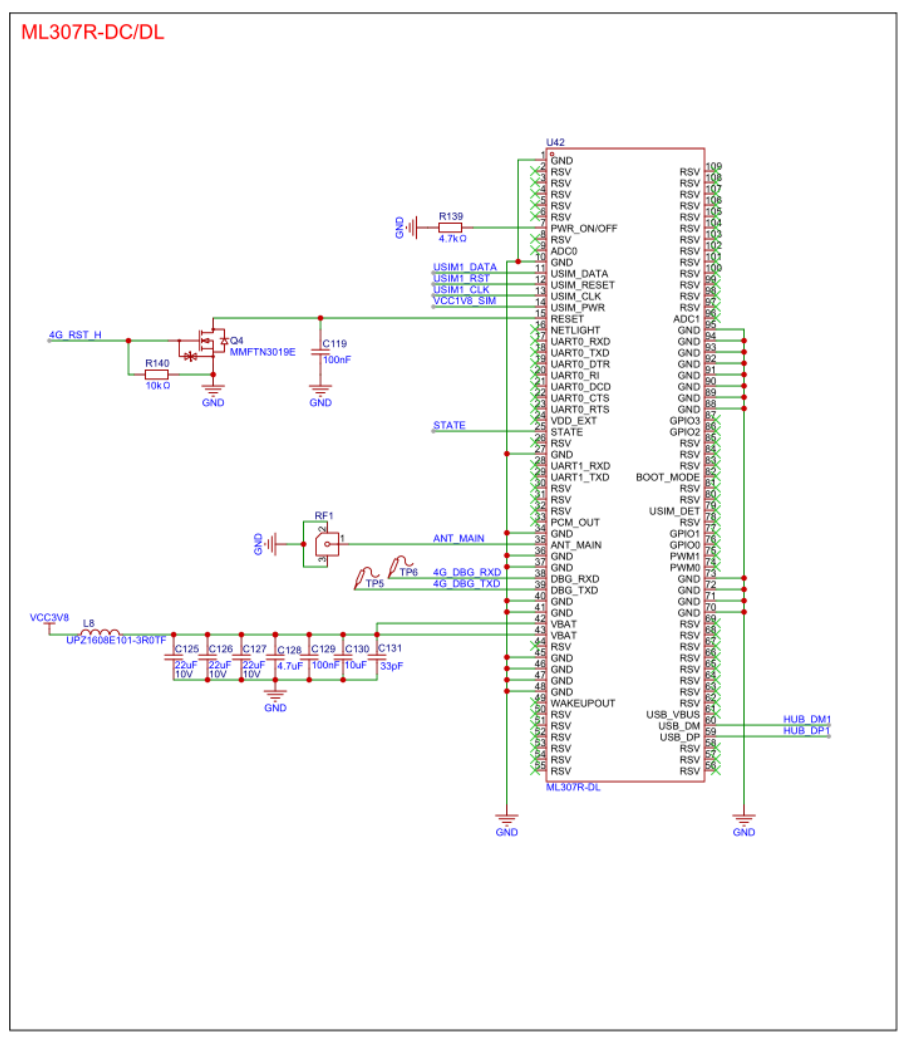
2. ML307原理图

- HUB1路的4G模组









- **电源转换**：采用 ETA3475D2F 降压芯片将 5V 系统电源转换为 3.8V，并配合多组电容进行去耦滤波，为模组提供稳定可靠的供电。
- **SIM卡接口**：配备标准 6 引脚 SIM 卡槽，接口线上集成了专用的 ESD 静电防护器件，保障 USIM 数据通信的安全与稳定。
- **状态指示**：使用 GPIO 信号（`STATE`）控制三极管（Q5）驱动红色状态指示灯，用于实时反馈 4G 模组的运行与联网状态。

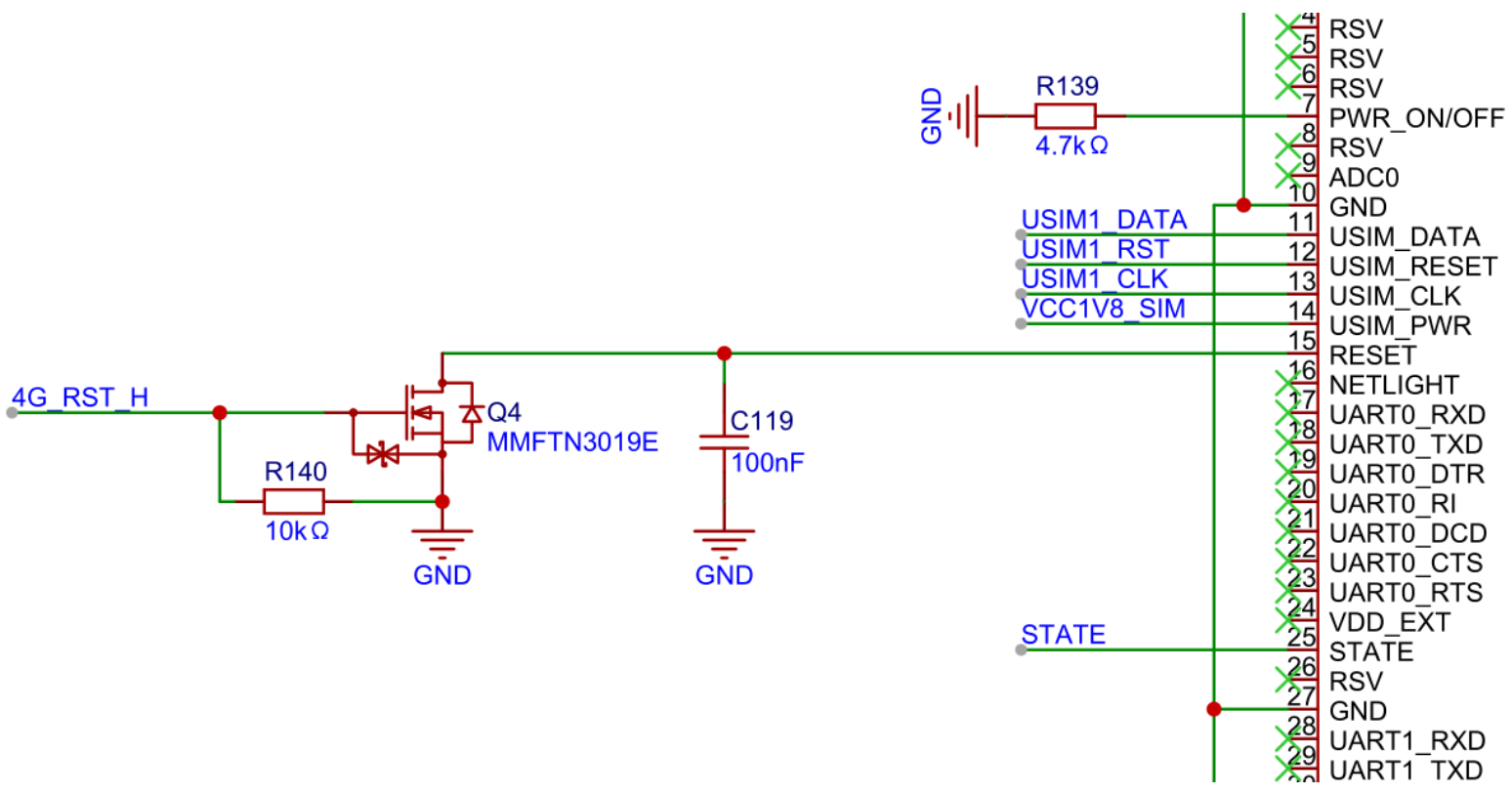
3. ML307设备树

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    leds: leds {
        status = "okay";
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&leds_gpio>;
        work_4g_rst: work-4g-rst {
            gpios = <&gpio4 RK_PB2 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

        wifi_en: wifi-en {
            gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
            default-state = "off";
        };

        gpio1_c7: gpio1-c7 {
            gpios = <&gpio1 RK_PC7 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };

    };


    led {
        leds_gpio: leds-gpio {
            rockchip,pins =
                <4 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>,
                <4 RK_PB0 RK_FUNC_GPIO &pcfg_pull_none>,
                <1 RK_PC7 RK_FUNC_GPIO &pcfg_pull_none>;

        };
    };
```

项目	值	说明
GPIO	`GPIO4_PB2`	4G 模组复位引脚
极性	`ACTIVE_HIGH`	高电平复位 / 低电平不复位
默认状态	`on` (高电平)	默认一直复位
sysfs 路径	`/sys/class/leds/work-4g-rst/brightness`	通过 LED 子系统控制

- 复位电路



4. ML307内核驱动

- kernel/drivers/usb/serial/option.c : 252-253

```
/*OneMo*/
#define ONEMO_VENDOR_ID                         0x2ecc
#define ONEMO_PRODUCT_ML307R                    0x3012
```

- kernel/drivers/usb/serial/option.c : 1279

```
{ USB_DEVICE_AND_INTERFACE_INFO(ONEMO_VENDOR_ID, ONEMO_PRODUCT_ML307R, 0xff, 0, 0) },
```

- 参数含义

项目	值	说明
USB VID	`0x2ecc`	OneMO (中移物联)
USB PID	`0x3012`	ML307R
驱动文件	`drivers/usb/serial/option.c`	USB 串口驱动
匹配方式	`0xff, 0, 0`	Vendor Specific 类接口

- 当前内核的 `option.c` **已包含 ML307R 支持**，插上模组后 USB 枚举即可生成 `/dev/ttyUSB0` ~ `/dev/ttyUSB2` 等串口设备。

5. 文件系统启用脚本

- 启用脚本 buildroot/board/rockchip/rk3506/fs-overlay/usr/bin/ml307.sh

```
#!/bin/sh
#ML307 init
sleep 1
echo 0 > /sys/class/leds/work-4g-rst/brightness
```

这个脚本配合你的 DTS 和硬件电路设计。它的目的就是**在开机 1 秒后，释放 4G 模块的复位状态，让模块正常上电启动。**

- 系统启用脚本
- buildroot/board/rockchip/rk3506/fs-overlay/etc/init.d/S89ML37init

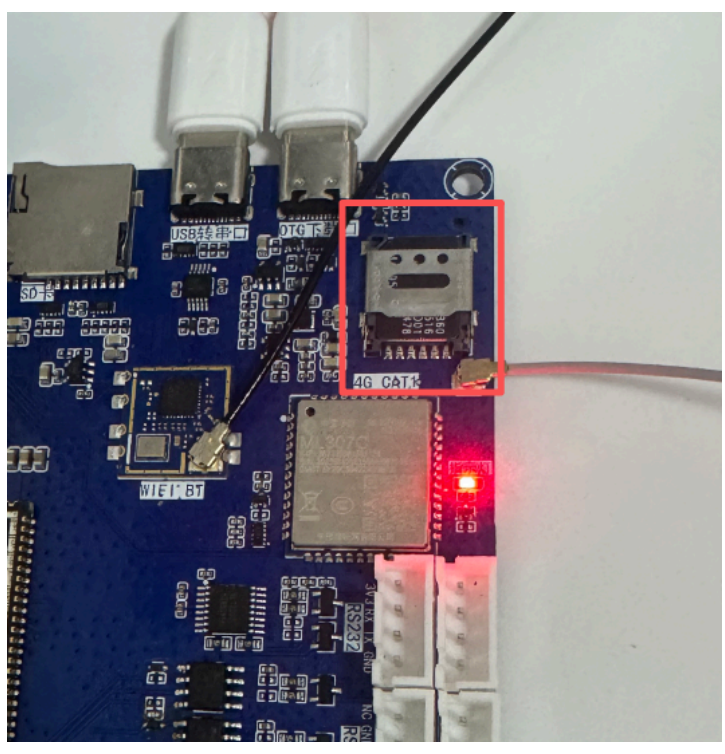
```
#! /bin/sh
case $1 in
    start)
        /usr/bin/ml307.sh
        ;;
    stop)
        echo "skip"
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        ;;
esac
```

系统启动日志:

```
[   16.144762] usb 2-1.1: new high-speed USB device number 5 using dwc2
[   16.239415] cdc_ether 2-1.1:1.0 eth2: register 'cdc_ether' at usb-ff780000.usb-1.1, CDC Ethernet Device,
ac:0c:29:a3:9b:6d
[   16.242505] option 2-1.1:1.4: GSM modem (1-port) converter detected
[   16.243062] usb 2-1.1: GSM modem (1-port) converter now attached to ttyUSB0
[   16.244323] option 2-1.1:1.3: GSM modem (1-port) converter detected
[   16.245159] usb 2-1.1: GSM modem (1-port) converter now attached to ttyUSB1
[   16.246376] option 2-1.1:1.2: GSM modem (1-port) converter detected
[   16.248156] usb 2-1.1: GSM modem (1-port) converter now attached to ttyUSB2
```

6. ML307测试命令

- 插上SIM卡



- 测试命令

```
cat /dev/ttyUSB2 &

#设置拨号协议为RNDIS
echo -e "AT+MDIALUPCFG="mode",0" > /dev/ttyUSB2

#设置拨号协议为ECM
echo -e "AT+MDIALUPCFG="mode",1" > /dev/ttyUSB2

#关闭回显
echo -e 'ATE0\r\n' > /dev/ttyUSB2

#查询PDP上下文配置
echo -e 'AT+CGDCONT?\r\n'> /dev/ttyUSB2

#查询驻网状态
echo -e 'AT+COPS?\r\n' > /dev/ttyUSB2

#设置自动拨号命令
echo -e 'AT+MDIALUP=1,1\r\n' > /dev/ttyUSB2

#查询拨号状态
echo -e 'AT+MDIALUP?\r\n' > /dev/ttyUSB2
```

- ML307R4 G模块支持RNDIS和ECM模块，两种模式选择其一即可。
- ML307R4G模块拨号上网后默认获取局域网内IP地址。

- 运行结果

```
root@rk3506-buildroot:/# cat /dev/ttyUSB2 &
[1] 1591
root@rk3506-buildroot:/# echo -e "AT+MDIALUPCFG="mode",0" > /dev/ttyUSB2
root@rk3506-buildroot:/#

OK


root@rk3506-buildroot:/# echo -e 'ATE0\r\n' > /dev/ttyUSB2
root@rk3506-buildroot:/#

OK


root@rk3506-buildroot:/# echo -e 'AT+CGDCONT?\r\n'> /dev/ttyUSB2
root@rk3506-buildroot:/#

+CGDCONT: 1,"IP","cmiottzx.ha.mnc008.mcc460.gprs",,0,0,,,,



OK


root@rk3506-buildroot:/# echo -e 'AT+COPS?\r\n' > /dev/ttyUSB2
root@rk3506-buildroot:/#

+COPS: 0,0,"CHINA MOBILE",7



OK


root@rk3506-buildroot:/# echo -e 'AT+MDIALUP=1,1\r\n' > /dev/ttyUSB2
root@rk3506-buildroot:/#

OK

+MDIALUP: 1,1,"10.77.7.184","","114.114.114.114","8.8.8.8"


root@rk3506-buildroot:/# echo -e 'AT+MDIALUP?\r\n' > /dev/ttyUSB2
root@rk3506-buildroot:/#

+MDIALUP: 1,1,"10.77.7.184","","114.114.114.114","8.8.8.8"



OK


root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ifconfig
eth0      Link encap:Ethernet  HWaddr 26:64:07:3A:AB:BD
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:76 Base address:0xe000

eth1      Link encap:Ethernet  HWaddr DE:3A:35:9A:26:91
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:78 Base address:0x2000

eth2      Link encap:Ethernet  HWaddr AC:0C:29:A3:9B:6D
          inet addr:192.168.0.100  Bcast:192.168.0.255  Mask:255.255.255.0
```

```
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:150 errors:0 dropped:0 overruns:0 frame:0
          TX packets:322 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5867 (5.7 KiB)  TX bytes:37455 (36.5 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:128 errors:0 dropped:0 overruns:0 frame:0
          TX packets:128 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:9472 (9.2 KiB)  TX bytes:9472 (9.2 KiB)

wlan0     Link encap:Ethernet  HWaddr 14:0A:02:9A:B0:D6
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

root@rk3506-buildroot:/# ping -I eth2 www.baidu.com
PING www.baidu.com (103.235.46.102): 56 data bytes
64 bytes from 103.235.46.102: seq=1 ttl=43 time=237.340 ms
64 bytes from 103.235.46.102: seq=3 ttl=43 time=235.891 ms
^C
--- www.baidu.com ping statistics ---
5 packets transmitted, 2 packets received, 60% packet loss
round-trip min/avg/max = 235.891/236.615/237.340 ms
root@rk3506-buildroot:/#
```

7. 自动联网脚本

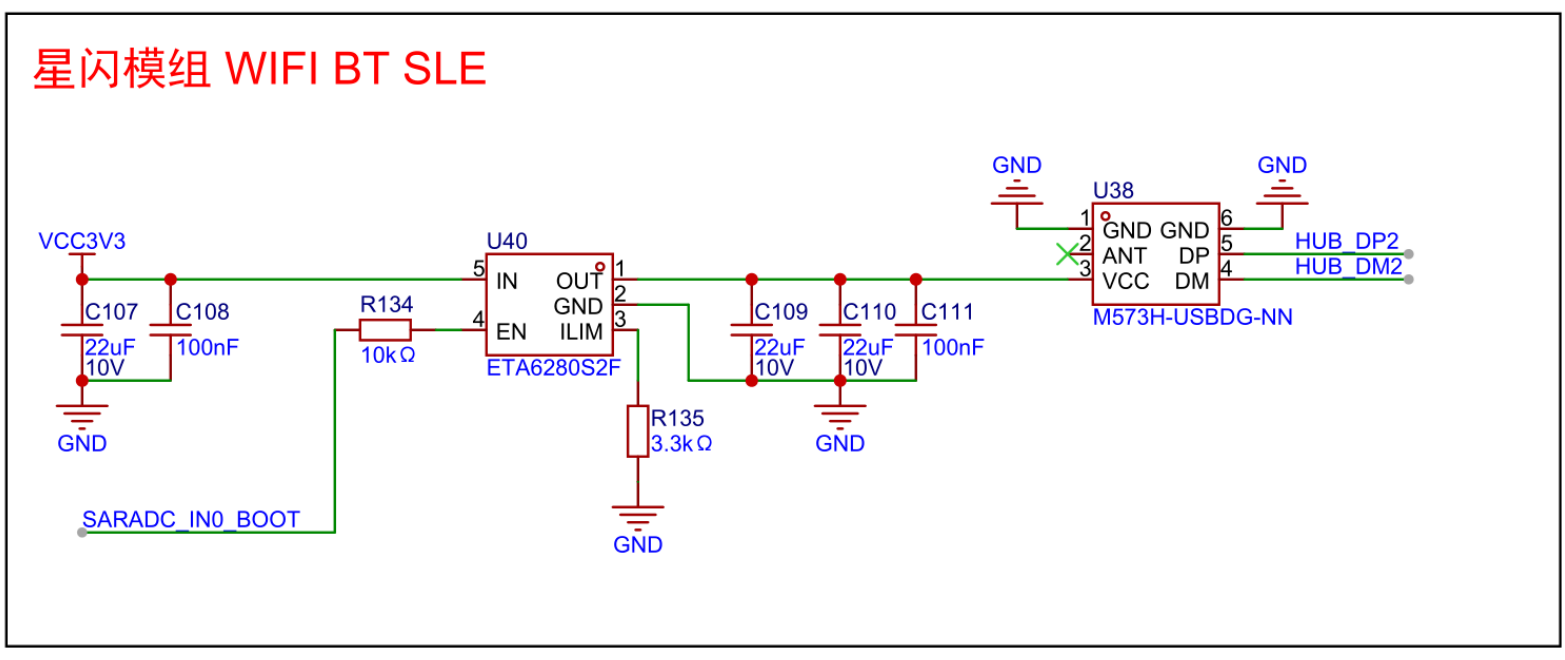
- buildroot/board/rockchip/rk3506/fs-overlay/usr/bin/ml307.sh

```
#!/bin/sh
#ML307 init
sleep 1
echo 0 > /sys/class/leds/work-4g-rst/brightness
sleep 1
echo -e 'AT+MDIALUP=1,1\r\n' > /dev/ttyUSB2
cat /dev/ttyUSB2 &
echo -e 'AT+MDIALUP?\r\n' > /dev/ttyUSB2
```

16.5 星闪WS73驱动

1. WS73 WIFI BT 原理图

- HUB2路的WIFIBT模组



- **供电与使能**：U40 (ETA6280S2F) 将 3.3V 转为模组供电，由 `SARADC_IN0_BOOT` 经 R134 控制使能端，R135 设定限流值，并辅以多组电容去耦滤波。
- **数据通信**：星闪模组 U38 的 DM/DP 数据引脚直接连接至 HUB 下行端口 `HUB_DM2` 与 `HUB_DP2`，实现 USB 2.0 协议下的交互。
- **天线配置**：原理图中模组的 ANT 引脚明确标注为悬空（未连接），表明此设计无需额外布设天线，直接使用模块内置的板载天线。

2. WS73 设备树

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
    leds: leds {
        status = "okay";
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&leds_gpio>;

        wifi_en: wifi-en {
            gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;       // GPIO 控制器 4，端口 B 引脚 0，高电平有效
            default-state = "off";                          // 内核启动时 GPIO 初始状态为低电平，WIFI不工作
        };

    };
```

- buildroot/board/rockchip/rk3506/fs-overlay/usr/bin/wifi_en.sh

```
#!/bin/sh
#wifi en
sleep 1
echo 1 > /sys/class/leds/wifi-en/brightness
```

- buildroot/board/rockchip/rk3506/fs-overlay/etc/init.d/S89WIFI_EN

```
#! /bin/sh

case $1 in
    start)
        /usr/bin/wifi_en.sh
        ;;
    stop)
        echo "skip"
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        ;;
esac
```

- 整个过程是星闪模上电过程

3. WS73 驱动分析

- 预编译内核模块
- buildroot/board/rockchip/rk3506/fs-overlay/usr/lib/modules/ws73/

```
buildroot/board/rockchip/rk3506/fs-overlay/usr/lib/modules/ws73/
├── ble_soc.ko
├── bp_test.ko
├── plat_soc.ko
├── sle_soc.ko
└── wifi_soc.ko

0 directories, 5 files
linux@ubuntu:~/rk3506$
```

- 文件解释

文件	大小	依赖	功能
plat_soc.ko	288K	`usbcore`	USB 平台传输层，加载时读取 `/etc/ws73_cfg.ini`
wifi_soc.ko	1139K	`cfg80211`	WiFi MAC/PHY 驱动，创建 `wlan0` / `wlan1` / `p2p0`
ble_soc.ko	12K	`bluetooth`	BLE 低功耗蓝牙
sle_soc.ko	17K	-	SLE 星闪协议
bp_test.ko	7K	-	诊断/测试 (不常加载)

- 固件文件
- buildroot/board/rockchip/rk3506/fs-overlay/etc/ws73/

```
linux@ubuntu:~/rk3506$ tree  buildroot/board/rockchip/rk3506/fs-overlay/etc/ws73/
buildroot/board/rockchip/rk3506/fs-overlay/etc/ws73/
├── btc_cali.bin
├── wifi_cali.bin
├── wow.bin
└── ws73.bin

0 directories, 4 files
linux@ubuntu:~/rk3506$
```

文件	用途	大小
ws73.bin	主固件	145K
wifi_cali.bin	WiFi 校准数据	21K
btc_cali.bin	BT 共存校准	8K
wow.bin	Wake-on-Wireless 固件	23K

- 配置文件
- buildroot/board/rockchip/rk3506/fs-overlay/etc/ws73_cfg.ini

```
#注:
#1、var=value, 等号左右没有空格
[HOST_WIFI_NORMAL]
#私有定制化--begin
#最大带宽能力 FPGA 40M ASIC 160M (0:20M,1:40M,2:80M,3:160M)
bw_max_width=1
#txbf cap
su_bfee=1
ldpc=1
ba_32bitmap=0
mtid_aggr_rx=0
front_switch=0
country_code=CN
region_table_default=CN
region_table_jp=JP
region_table_fcc=US,CA,KH
region_table_ce=RU,AU,MY,ID,TR,PL,FR,PT,IT,DE,ES,AR,ZA,MA,PH,TH,GB,CO,MX,EC,PE,CL,SA,EG,AE
#数采开关
data_sample=0
#对外交互操作日志记录文件路径(路径长度范围1~103)
external_record_file_path=/opt/wifi_external_record.txt
#对外交互操作日志记录文件大小(默认10k)
external_record_file_size=10

#支持ini配置WiFi接口名, start
#接口名长度需小于16个字符，超出的字符将被截断；建议用字母、数字、下划线，不支持中文字符及特殊字符；不可配置为相同名字，可能触发不符合预期的行为。
#当WSCFG_WLAN_INIT_VAP_COUNT=1时，station和softap只能起一个，p2p无法启动，wlan_ifname0表示station或softap接口名，wlan_ifname1和wlan_ifname2将被忽略。
#当WSCFG_WLAN_INIT_VAP_COUNT=2(默认值)时，station和softap只能起一个，p2p也可以启动，station和p2p可以并存，softap和p2p不能并存，wlan_ifname0表示station或softap接口名，wlan_ifname2表示p2p接口名，wlan_ifname1将被忽略。
#当WSCFG_WLAN_INIT_VAP_COUNT=3时，station和softap可以并存，station和p2p可以并存，softap和p2p不能并存，wlan_ifname0表示station接口名，wlan_ifname1表示softap接口名，wlan_ifname2表示p2p接口名。
#wlan_ifname0 如果删除或者配置为空，系统会使用默认值 "wlan0"
wlan_ifname0=wlan0
#wlan_ifname1 表示softap接口名，是可选配置，默认值是 "wlan1"
wlan_ifname1=wlan1
#wlan_ifname2 表示p2p接口名，是可选配置，默认值是 "p2p0"
wlan_ifname2=p2p0
#支持ini配置WiFi接口名, end
```

`plat_soc.ko` 通过参数 `g_ini_file_name_etc=/etc/ws73_cfg.ini` 在 insmod 时读取该文件。

---

4. 用户空间脚本

- buildroot/board/rockchip/rk3506/fs-overlay/usr/bin/wifi_en.sh

```
#!/bin/sh
#wifi en
sleep 1
echo 1 > /sys/class/leds/wifi-en/brightness
```

- buildroot/board/rockchip/rk3506/fs-overlay/etc/init.d/S89WIFI_EN

```
#! /bin/sh

case $1 in
    start)
        /usr/bin/wifi_en.sh
        ;;
    stop)
        echo "skip"
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        ;;
esac
```

- 以上是给wifi模组供电的脚本

- buildroot/board/rockchip/rk3506/fs-overlay/usr/bin/rkwifibt.sh

```
#!/bin/sh
#init ws73
ws73_plat=plat_soc
ws73_wifi=wifi_soc
ws73_sle=sle_soc
ws73_ble=ble_soc

if ! lsmod | grep -q "${ws73_plat}"; then
        insmod /usr/lib/modules/ws73/${ws73_plat}.ko
fi
sleep 0.5

if ! lsmod | grep -q "${ws73_wifi}"; then
        insmod /usr/lib/modules/ws73/${ws73_wifi}.ko
fi
sleep 0.5

if ! lsmod | grep -q "${ws73_sle}"; then
        insmod /usr/lib/modules/ws73/${ws73_sle}.ko
fi
sleep 0.5

if ! lsmod | grep -q "${ws73_ble}"; then
        insmod /usr/lib/modules/ws73/${ws73_ble}.ko
fi

sleep 1
ifconfig Featureid0 down
```

- buildroot/board/rockchip/rk3506/fs-overlay/etc/init.d/S90rkwifibt

```
#! /bin/sh

case $1 in
    start)
        /usr/bin/rkwifibt.sh
        ;;
    stop)
        echo "skip rmmod ws73 all modules"
        ;;
    *)
        echo "Usage: $0 {start|stop}"
        ;;
esac
```

5. WIFI测试

STA连接方法:

- 星闪wifi.so加载起来后，会打开Featureid0和wlanO两个网络节点，Featureid0会影响wlan0的ip获取，需要关闭，命令如下。

```
ifconfig Featureid0 down
```

- 我们在启动脚本时已经关闭。
- /etc/wpa_supplicant.conf

```
ctrl_interface=/var/run/wpa_supplicant
ap_scan=1
update_config=1
network={
    ssid="lvluo"
    psk="ab123456.."
    key_mgmt=WPA-PSK
}
```

- 链接网络命令

```
wpa_supplicant -Dnl80211 -c /etc/wpa_supplicant.conf -i wlan0 &

udhcpc -i wlan0
```

- 运行结果

```
root@rk3506-buildroot:/# vi /etc/wpa_supplicant.conf
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# wpa_supplicant -Dnl80211 -c /etc/wpa_supplicant.conf -i wlan0 &
[2] 1698
root@rk3506-buildroot:/# Successfully initialized wpa_supplicant
[12127.400269] hmac_fsm_change_state_etc state 6, vap_id 1
[12128.399003] hmac_single_hal_device_scan_complete:vap[1] time[974] chan_cnt[13] chan_0[1] back[0] event[6]
mode[0]
[12128.402971] hmac_fsm_change_state_etc state 7, vap_id 1
[12128.404113] scan: inform bss 12,other 0,vap 1
wlan0: Trying to associate with SSID 'lvluo'
[12128.420465] connect:bw[0] chan[1] ssid[lvluo]
[12128.457687] hmac_fsm_change_state_etc state 8, vap_id 1
[12128.483962] wifi:tx auth seq 1 alg 0 code 0
[12128.484008] hmac_fsm_change_state_etc state 9, vap_id 1
[12128.494820] wifi:rx auth seq 2 alg 0 code 0
[12128.495607] hmac_fsm_change_state_etc state 11, vap_id 1
[12128.496036] FPGA DO NOT support turbo QAM
[12128.496227] wifi:tx assoc req
[12128.496258] hmac_fsm_change_state_etc state 12, vap_id 1
[12128.512345] wifi:rx assoc rsp code 0
[12128.524196] hmac_fsm_change_state_etc state 1, vap_id 1
wlan0: Associated with 08:10:7c:42:01:3b
wlan0: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
[12128.631938] wifi:rx eapol 1/4
[12128.635117] wifi:tx eapol 2/4
[12128.640570] wifi:rx eapol 3/4
[12128.642351] wifi:tx eapol 4/4
wlan0: WPA: Key negotiation completed with 08:10:7c:42:01:3b [PTK=CCMP GTK=CCMP]
wlan0: CTRL-EVENT-CONNECTED - Connection to 08:10:7c:42:01:3b completed [id=0 id_str=]

root@rk3506-buildroot:/#
root@rk3506-buildroot:/# udhcpc -i wlan0
udhcpc: started, v1.36.1
udhcpc: broadcasting discover
udhcpc: broadcasting select for 192.168.2.4, server 192.168.2.1
udhcpc: lease of 192.168.2.4 obtained from 192.168.2.1, lease time 86400
deleting routers
adding dns 192.168.2.1
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ifconfig
eth0      Link encap:Ethernet  HWaddr 26:64:07:3A:AB:BD
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:76 Base address:0xe000

eth1      Link encap:Ethernet  HWaddr DE:3A:35:9A:26:91
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:78 Base address:0x2000

eth2      Link encap:Ethernet  HWaddr AC:0C:29:A3:9B:6D
          inet addr:192.168.0.100  Bcast:192.168.0.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:455 errors:0 dropped:0 overruns:0 frame:0
          TX packets:535 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:24919 (24.3 KiB)  TX bytes:61869 (60.4 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:96 errors:0 dropped:0 overruns:0 frame:0
          TX packets:96 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7104 (6.9 KiB)  TX bytes:7104 (6.9 KiB)

wlan0     Link encap:Ethernet  HWaddr 14:0A:02:9A:B0:D6
          inet addr:192.168.2.4  Bcast:192.168.2.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:63 errors:0 dropped:1 overruns:0 frame:0
          TX packets:24 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:4767 (4.6 KiB)  TX bytes:3950 (3.8 KiB)

root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ping -I wlan0 www.baidu.com
PING www.baidu.com (103.235.46.115): 56 data bytes
[12218.903204] [AMPDU RX] {hmac_mgmt_tx_addba_rsp_send::TID[0] STATUS[0] LUT INDEX[1] BAWSIZE[32].}
[12267.392212] [AMPDU TX] {hmac_mgmt_tx_addba_req_send::USER ID[1]TID NO[0] BAWSIZE[32] TXAMSDU[1].}
[12267.394285] [AMPDU TX] hmac_mgmt_rx_addba_rsp tidno[0]         status[0] dialog_token[2] ba_policy[1]
ba_timeout[0], baw_size[32] amsdu_supp[1] assoc_id[1]         max_rx_ampdu_factor[3] max_ampdu_len_exp[0]
protocol_mode[10] min_mpdu_start_spacing[5]
64 bytes from 103.235.46.115: seq=0 ttl=47 time=200.855 ms
64 bytes from 103.235.46.115: seq=2 ttl=47 time=202.157 ms
64 bytes from 103.235.46.115: seq=3 ttl=47 time=198.205 ms
64 bytes from 103.235.46.115: seq=4 ttl=47 time=207.332 ms
64 bytes from 103.235.46.115: seq=5 ttl=47 time=201.543 ms
^C
--- www.baidu.com ping statistics ---
6 packets transmitted, 5 packets received, 16% packet loss
round-trip min/avg/max = 198.205/202.018/207.332 ms
root@rk3506-buildroot:/#
```

第17章 Linux 网卡驱动

17.1 双以太网介绍

1. 以太网控制器

RK3506 是瑞芯微针对 AIoT、智能家居网关和中控屏等场景推出的一款高性价比处理器。它在网络连接方面最大的特点是**内置了一路百兆以太网 PHY + 一路独立的千兆以太网 MAC**，可以用极少的外部元件实现双网口方案。

RK3506 内部集成了两个以太网控制器，分别是：

- **10/100M 百兆以太网（内置 PHY）**
  - 支持 10/100Mbps 自适应
  - **MAC 与 PHY 均已集成在芯片内部**，直接输出 MDI 差分信号（TX/RX）
  - 外部只需连接网络变压器和 RJ45 座子，无需额外的以太网 PHY 芯片，成本极低
  - 接口通常通过 RMII 与内部 MAC 对接，但对外体现为可直接接网口
- **10/100/1000M 千兆以太网（仅 MAC，需外挂 PHY）**
  - 支持 10/100/1000Mbps 自适应
  - 芯片仅提供 **RGMII / RMII 接口的 MAC 控制器**，必须外接一颗千兆 PHY 芯片（如 RTL8211F 等）
  - 可灵活配置为千兆 WAN 口或 LAN 口

2. 控制器特性

- 以太网控制器基于 **Synopsys DesignWare MAC (DW-MAC)**，驱动成熟，在 Linux 内核中使用 `stmmac` 驱动
- 支持 **TSO (TCP Segmentation Offload)、LRO (Large Receive Offload)、VLAN 过滤、Wake-on-LAN (WOL)** 等硬件加速与低功耗特性
- 支持 **IEEE 1588v2** 高精度时间同步（取决于具体硬件配置和驱动实现，通常千兆 MAC 支持）

3. 典型应用

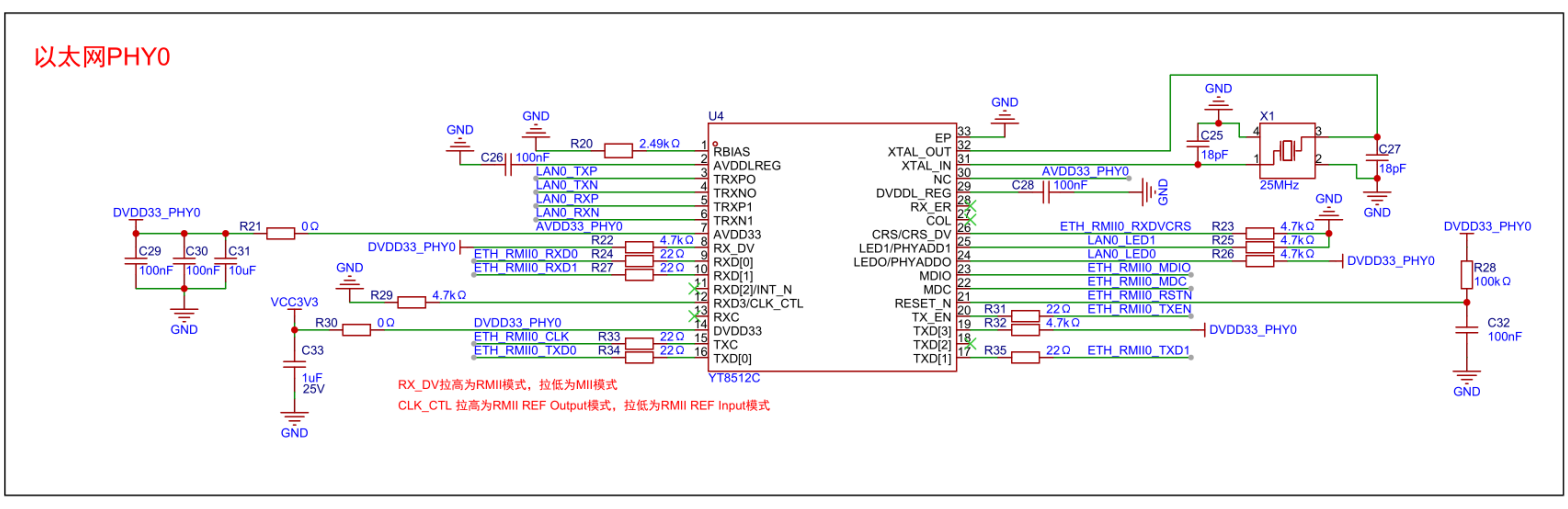
利用以上特性，RK3506 可以轻松实现 **“千兆 WAN + 百兆 LAN”** 或 **“双百兆”** 的网关方案：

- **WAN 口**使用千兆 MAC 外接千兆 PHY，用作上行口
- **LAN 口**直接使用内置百兆 PHY，驱动内部局域网设备

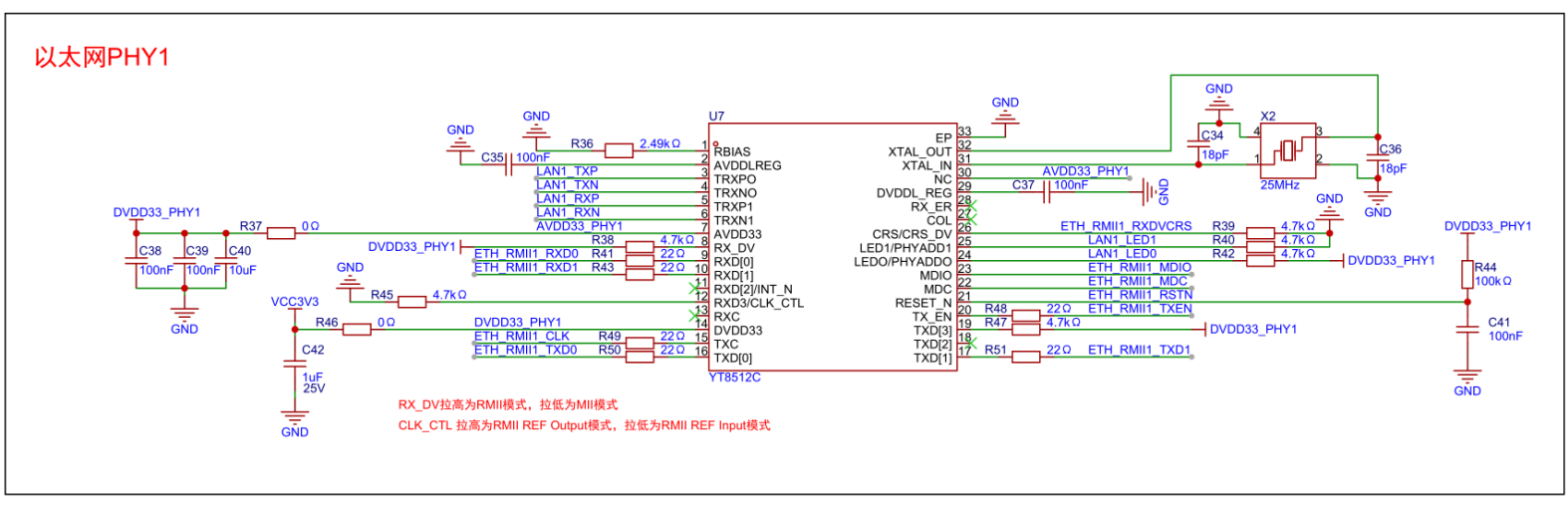
- 这种搭配在智能家居网关、4G/5G CPE、中控屏底座等应用中兼顾了成本和上行带宽

2. 以太网原理图

- 以太网0

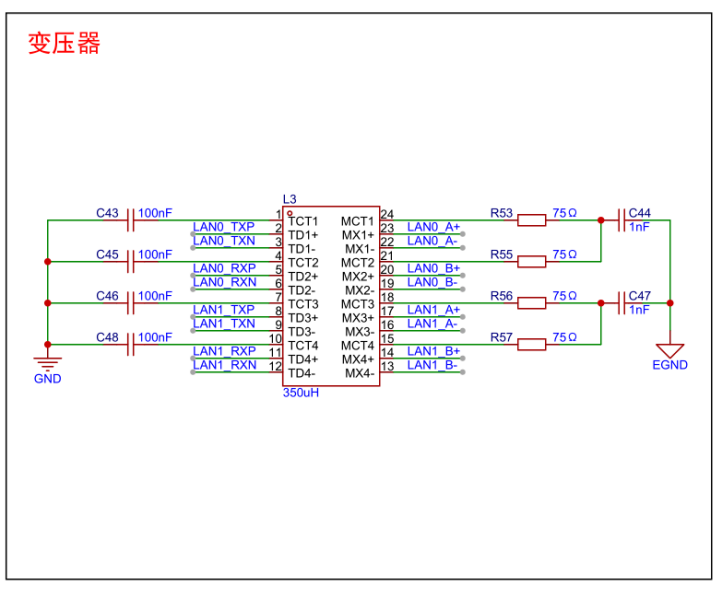


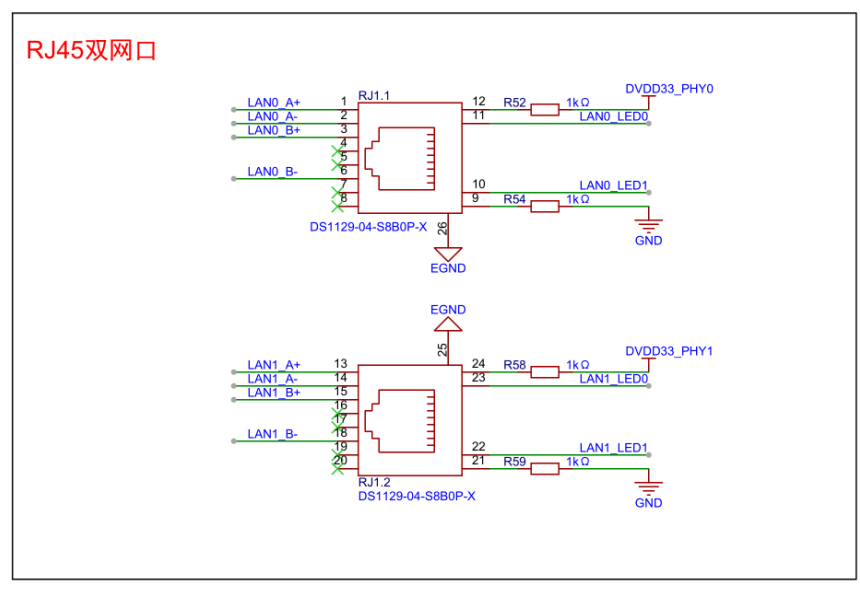
- 以太网1



- YT8512C芯片经上拉引脚设RMII模式，并启用内部50MHz参考时钟输出。
- 电源经多级电容滤波，配合25MHz晶振，复位与MDIO接口均配4.7kΩ上拉。

- 变压器与RJ45网口

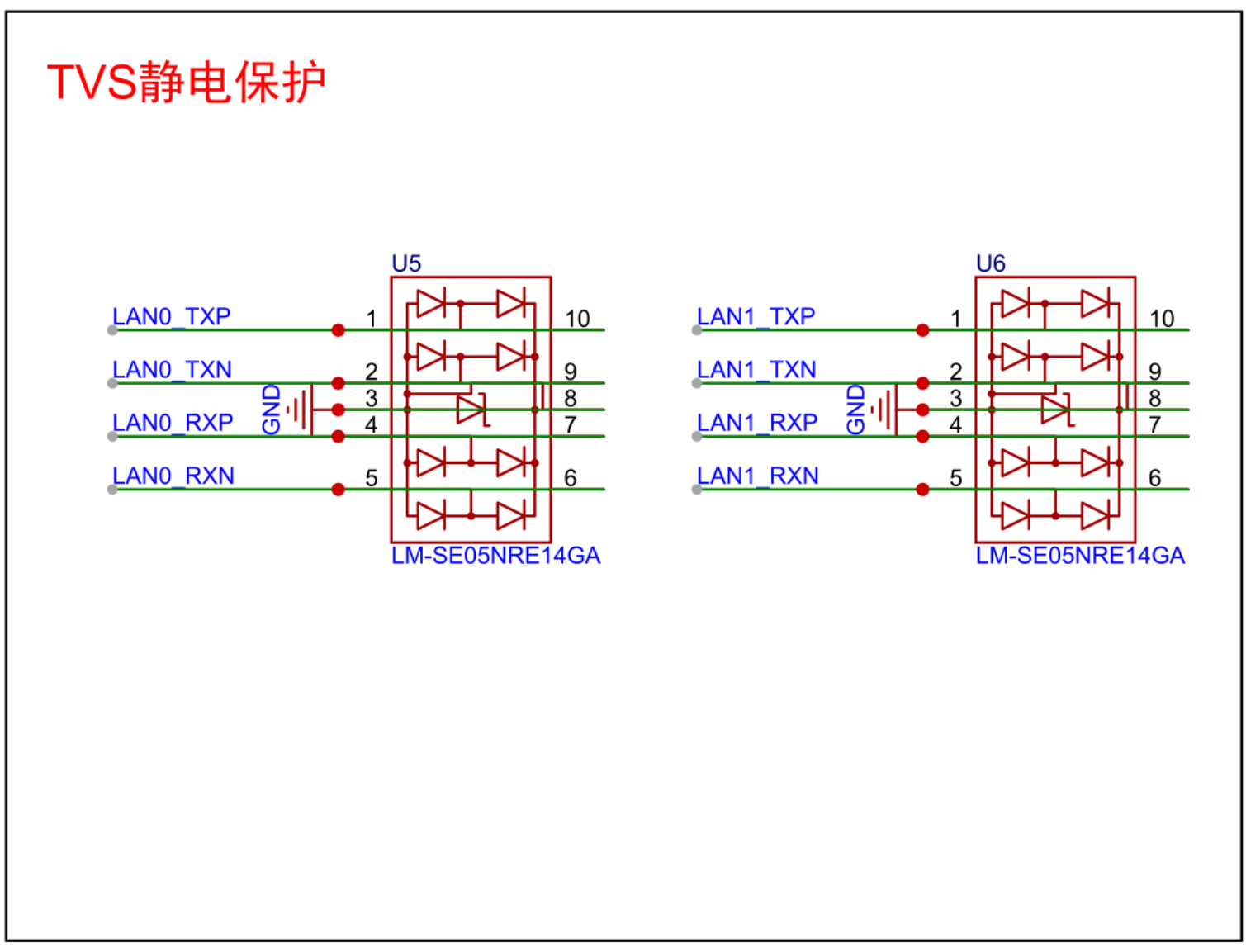




- 变压器集成共模扼流圈及隔离，配合75Ω匹配电阻与1nF电容作接地滤波。

2. 双RJ45接口含状态指示灯，通过1kΩ电阻上拉电源，并采用专用地（EGND）隔离。

- 静电防护



17.2 以太网驱动

1. 以太网设备树

RK3506 平台拥有两个独立的以太网 MAC 控制器，基于 Synopsys DesignWare MAC 4.20a IP 核。

- kernel/arch/arm/boot/dts/rk3506.dtsi

```
    aliases {
        ethernet0 = &gmac0;
        ethernet1 = &gmac1;
    };


    gmac0: ethernet@ff4c8000 {
        compatible = "rockchip,rk3506-gmac", "snps,dwmac-4.20a";
        reg = <0xff4c8000 0x2000>;
        interrupts = <GIC_SPI 66 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 69 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "macirq", "eth_wake_irq";
        rockchip,grf = <&grf>;
        clocks = <&cru CLK_MAC0>, <&cru CLK_MAC0_PTP>,
             <&cru PCLK_MAC0>, <&cru ACLK_MAC0>;
        clock-names = "stmmaceth", "ptp_ref",
                  "pclk_mac", "aclk_mac";
        resets = <&cru SRST_A_MAC0>;
        reset-names = "stmmaceth";

        assigned-clocks = <&cru CLK_MAC0_PTP>;
        assigned-clock-rates = <62500000>;

        snps,mixed-burst;
        snps,tso;

        snps,axi-config = <&gmac0_stmmac_axi_setup>;
        snps,mtl-rx-config = <&gmac0_mtl_rx_setup>;
        snps,mtl-tx-config = <&gmac0_mtl_tx_setup>;

        phy-mode = "rmii";
        status = "disabled";

        mdio0: mdio {
            compatible = "snps,dwmac-mdio";
            #address-cells = <0x1>;
            #size-cells = <0x0>;
        };

        gmac0_stmmac_axi_setup: stmmac-axi-config {
            snps,wr_osr_lmt = <4>;
            snps,rd_osr_lmt = <8>;
            snps,blen = <0 0 0 0 16 8 4>;
        };

        gmac0_mtl_rx_setup: rx-queues-config {
            snps,rx-queues-to-use = <1>;
            queue0 {
                status = "okay";
            };
        };

        gmac0_mtl_tx_setup: tx-queues-config {
            snps,tx-queues-to-use = <1>;
```

```
            queue0 {
                status = "okay";
            };
        };
    };

    gmac1: ethernet@ff4d0000 {
        compatible = "rockchip,rk3506-gmac", "snps,dwmac-4.20a";
        reg = <0xff4d0000 0x2000>;
        interrupts = <GIC_SPI 70 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 73 IRQ_TYPE_LEVEL_HIGH>;
        interrupt-names = "macirq", "eth_wake_irq";
        rockchip,grf = <&grf>;
        clocks = <&cru CLK_MAC1>, <&cru CLK_MAC1_PTP>,
             <&cru PCLK_MAC1>, <&cru ACLK_MAC1>;
        clock-names = "stmmaceth", "ptp_ref",
                  "pclk_mac", "aclk_mac";
        resets = <&cru SRST_A_MAC1>;
        reset-names = "stmmaceth";

        assigned-clocks = <&cru CLK_MAC1_PTP>;
        assigned-clock-rates = <62500000>;

        snps,mixed-burst;
        snps,tso;

        snps,axi-config = <&gmac1_stmmac_axi_setup>;
        snps,mtl-rx-config = <&gmac1_mtl_rx_setup>;
        snps,mtl-tx-config = <&gmac1_mtl_tx_setup>;

        phy-mode = "rmii";
        status = "disabled";

        mdio1: mdio {
            compatible = "snps,dwmac-mdio";
            #address-cells = <0x1>;
            #size-cells = <0x0>;
        };

        gmac1_stmmac_axi_setup: stmmac-axi-config {
            snps,wr_osr_lmt = <4>;
            snps,rd_osr_lmt = <8>;
            snps,blen = <0 0 0 0 16 8 4>;
        };

        gmac1_mtl_rx_setup: rx-queues-config {
            snps,rx-queues-to-use = <1>;
            queue0 {
                status = "okay";
            };
        };

        gmac1_mtl_tx_setup: tx-queues-config {
            snps,tx-queues-to-use = <1>;
            queue0 {
                status = "okay";
            };
        };
    };
```

将 `gmac0`/`gmac1` 映射为 `ethernet0`/`ethernet1`，Linux 内核通过别名确定网卡编号（eth0/eth1）。

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
&gmac0 {
    phy-mode = "rmii";
    clock_in_out = "input";                    // 时钟方向：PHY 提供时钟

    snps,reset-gpio = <&gpio0 RK_PC3 GPIO_ACTIVE_LOW>;  // GPIO0_PC3 复位
    snps,reset-active-low;                     // 低电平有效
    snps,reset-delays-us = <0 20000 100000>;   // pre(0) / pulse(20ms) / post(100ms)

    pinctrl-0 = <&eth_rmii0_miim_pins          // MDC/MDIO
                 &eth_rmii0_tx_bus2_pins       // TXD0, TXD1, TXEN
                 &eth_rmii0_rx_bus2_pins       // RXD0, RXD1, CRS_DV
                 &eth_rmii0_clk_pins>;         // REF_CLK

    phy-handle = <&rmii_phy0>;
    status = "okay";
};


&gmac1 {
    phy-mode = "rmii";
    clock_in_out = "input";

    snps,reset-gpio = <&gpio0 RK_PC4 GPIO_ACTIVE_LOW>;
    snps,reset-active-low;
    snps,reset-delays-us = <0 20000 100000>;

    pinctrl-names = "default";
    pinctrl-0 = <&eth_rmii1_miim_pins
             &eth_rmii1_tx_bus2_pins
             &eth_rmii1_rx_bus2_pins
             &eth_rmii1_clk_pins>;

    phy-handle = <&rmii_phy1>;
    status = "okay";
};
```

- PHY 配置

```
&mdio0 {
    rmii_phy0: phy@1 {
        compatible = "ethernet-phy-ieee802.3-c22";  // 标准 802.3 Clause 22 PHY
        reg = <0x01>;                               // MDIO 地址 1
    };
};

&mdio1 {
    rmii_phy1: phy@1 {
        compatible = "ethernet-phy-ieee802.3-c22";
        reg = <0x01>;                               // MDIO 地址 1
    };
};
```

- 两个 PHY 的 MDIO 地址相同（`0x01`），因为它们位于独立的 MDIO 总线上（`mdio0`/`mdio1`），不会冲突。

- Pinmux 管脚映射
- kernel/arch/arm/boot/dts/rk3506-pinctrl.dtsi

```
    eth_clk0_25m {
        /omit-if-no-ref/
        eth_clk0_25m_out_pins: eth-clk0-25m-out-pins {
            rockchip,pins =
                /* eth_clk0_25m_out */
                <0 RK_PC4 1 &pcfg_pull_none>;
        };
    };

    eth_clk1_25m {
        /omit-if-no-ref/
        eth_clk1_25m_out_pins: eth-clk1-25m-out-pins {
            rockchip,pins =
                /* eth_clk1_25m_out */
                <0 RK_PC3 1 &pcfg_pull_none>;
        };
    };

    eth_rmii0 {
        /omit-if-no-ref/
        eth_rmii0_miim_pins: eth-rmii0-miim-pins {
            rockchip,pins =
                /* eth_rmii0_mdc */
                <2 RK_PB6 1 &pcfg_pull_none>,
                /* eth_rmii0_mdio */
                <2 RK_PB7 1 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii0_rx_bus2_pins: eth-rmii0-rx-bus2-pins {
```

```
            rockchip,pins =
                /* eth_rmii0_rxd0 */
                <2 RK_PB0 1 &pcfg_pull_none>,
                /* eth_rmii0_rxd1 */
                <2 RK_PB1 1 &pcfg_pull_none>,
                /* eth_rmii0_rxdvcrs */
                <2 RK_PC0 1 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii0_tx_bus2_pins: eth-rmii0-tx-bus2-pins {
            rockchip,pins =
                /* eth_rmii0_txd0 */
                <2 RK_PB3 1 &pcfg_pull_none>,
                /* eth_rmii0_txd1 */
                <2 RK_PB4 1 &pcfg_pull_none>,
                /* eth_rmii0_txen */
                <2 RK_PB5 1 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii0_clk_pins: eth-rmii0-clk-pins {
            rockchip,pins =
                /* eth_rmii0_clk */
                <2 RK_PB2 1 &pcfg_pull_none>;
        };
    };

    eth_rmii1 {
        /omit-if-no-ref/
        eth_rmii1_miim_pins: eth-rmii1-miim-pins {
            rockchip,pins =
                /* eth_rmii1_mdc */
                <3 RK_PB4 2 &pcfg_pull_none>,
                /* eth_rmii1_mdio */
                <3 RK_PB5 2 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii1_rx_bus2_pins: eth-rmii1-rx-bus2-pins {
            rockchip,pins =
                /* eth_rmii1_rxd0 */
                <3 RK_PA6 2 &pcfg_pull_none>,
                /* eth_rmii1_rxd1 */
                <3 RK_PA7 2 &pcfg_pull_none>,
                /* eth_rmii1_rxdvcrs */
                <3 RK_PB6 2 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii1_tx_bus2_pins: eth-rmii1-tx-bus2-pins {
            rockchip,pins =
                /* eth_rmii1_txd0 */
                <3 RK_PB1 2 &pcfg_pull_none>,
                /* eth_rmii1_txd1 */
                <3 RK_PB2 2 &pcfg_pull_none>,
                /* eth_rmii1_txen */
                <3 RK_PB3 2 &pcfg_pull_none>;
        };

        /omit-if-no-ref/
        eth_rmii1_clk_pins: eth-rmii1-clk-pins {
            rockchip,pins =
                /* eth_rmii1_clk */
                <3 RK_PB0 2 &pcfg_pull_none>;
        };
    };
```

2. 以太网内核驱动

以太网驱动采用 **Synopsys DesignWare MAC (STMMAC) + Rockchip 平台胶水层** 的分层架构：

- kernel/drivers/net/ethernet/stmicro/stmmac/dwmac-rk.c

```
// SPDX-License-Identifier: GPL-2.0-or-later
/**
 * DOC: dwmac-rk.c - Rockchip RK3288 DWMAC specific glue layer
 *
 * Copyright (C) 2014 Chen-Zhi (Roger Chen)
 *
 * Chen-Zhi (Roger Chen)  <roger.chen@rock-chips.com>
 */

#include <linux/stmmac.h>
#include <linux/bitops.h>
#include <linux/clk.h>
#include <linux/phy.h>
#include <linux/phy/phy.h>
#include <linux/of_net.h>
#include <linux/gpio.h>
#include <linux/module.h>
#include <linux/nvmem-consumer.h>
#include <linux/of_gpio.h>
#include <linux/of_device.h>
#include <linux/platform_device.h>
#include <linux/regulator/consumer.h>
#include <linux/delay.h>
#include <linux/mfd/syscon.h>
#include <linux/regmap.h>
#include <linux/pm_runtime.h>
#include <linux/soc/rockchip/rk_vendor_storage.h>
#include <soc/rockchip/rockchip_csu.h>
#include "stmmac_platform.h"
#include "dwmac-rk-tool.h"

。。。

static SIMPLE_DEV_PM_OPS(rk_gmac_pm_ops, rk_gmac_suspend, rk_gmac_resume);

static const struct of_device_id rk_gmac_dwmac_match[] = {
#ifdef CONFIG_CPU_PX30
    { .compatible = "rockchip,px30-gmac",    .data = &px30_ops   },
#endif
#ifdef CONFIG_CPU_RK1808
    { .compatible = "rockchip,rk1808-gmac", .data = &rk1808_ops },
#endif
#ifdef CONFIG_CPU_RK312X
    { .compatible = "rockchip,rk3128-gmac", .data = &rk3128_ops },
#endif
#ifdef CONFIG_CPU_RK322X
    { .compatible = "rockchip,rk3228-gmac", .data = &rk3228_ops },
#endif
#ifdef CONFIG_CPU_RK3288
    { .compatible = "rockchip,rk3288-gmac", .data = &rk3288_ops },
#endif
#ifdef CONFIG_CPU_RK3308
    { .compatible = "rockchip,rk3308-mac",  .data = &rk3308_ops },
#endif
#ifdef CONFIG_CPU_RK3328
    { .compatible = "rockchip,rk3328-gmac", .data = &rk3328_ops },
#endif
#ifdef CONFIG_CPU_RK3366
    { .compatible = "rockchip,rk3366-gmac", .data = &rk3366_ops },
#endif
#ifdef CONFIG_CPU_RK3368
    { .compatible = "rockchip,rk3368-gmac", .data = &rk3368_ops },
#endif
#ifdef CONFIG_CPU_RK3399
    { .compatible = "rockchip,rk3399-gmac", .data = &rk3399_ops },
#endif
#ifdef CONFIG_CPU_RK3506
    { .compatible = "rockchip,rk3506-gmac", .data = &rk3506_ops },
#endif
#ifdef CONFIG_CPU_RK3528
    { .compatible = "rockchip,rk3528-gmac", .data = &rk3528_ops },
#endif
#ifdef CONFIG_CPU_RK3562
    { .compatible = "rockchip,rk3562-gmac", .data = &rk3562_ops },
#endif
#ifdef CONFIG_CPU_RK3568
    { .compatible = "rockchip,rk3568-gmac", .data = &rk3568_ops },
#endif
#ifdef CONFIG_CPU_RK3576
    { .compatible = "rockchip,rk3576-gmac", .data = &rk3576_ops },
#endif
#ifdef CONFIG_CPU_RK3588
    { .compatible = "rockchip,rk3588-gmac", .data = &rk3588_ops },
#endif
```

```
#ifdef CONFIG_CPU_RV1106
    { .compatible = "rockchip,rv1106-gmac", .data = &rv1106_ops },
#endif
#ifdef CONFIG_CPU_RV1108
    { .compatible = "rockchip,rv1108-gmac", .data = &rv1108_ops },
#endif
#ifdef CONFIG_CPU_RV1126
    { .compatible = "rockchip,rv1126-gmac", .data = &rv1126_ops },
#endif
    { }
};
MODULE_DEVICE_TABLE(of, rk_gmac_dwmac_match);

static struct platform_driver rk_gmac_dwmac_driver = {
    .probe  = rk_gmac_probe,
    .remove = rk_gmac_remove,
    .driver = {
        .name           = "rk_gmac-dwmac",
        .pm     = &rk_gmac_pm_ops,
        .of_match_table = rk_gmac_dwmac_match,
    },
};
module_platform_driver(rk_gmac_dwmac_driver);

MODULE_AUTHOR("Chen-Zhi (Roger Chen) <roger.chen@rock-chips.com>");
MODULE_DESCRIPTION("Rockchip RK3288 DWMAC specific glue layer");
MODULE_LICENSE("GPL");
```

- 驱动配置如下

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    [*] Network device support  --->
        [*]   Network core driver support
        [*]   Ethernet driver support  --->
                [*]    STMicroelectronics devices
                <M>    STMicroelectronics Multi-Gigabit Ethernet driver
                [*]    Ethtool feature for STMMAC
                [*]    Support full driver for STMMAC
                <M>    STMMAC Platform bus support
                <M>    Rockchip dwmac support
                [*]    Rockchip dwmac tool support
        {M}   PHY Device support and infrastructure  --->
linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

- 编译内核更新设备树与内核到开发板内。

3. 文件系统启用脚本

- /etc/init.d/S03modules_init.sh

```
root@rk3506-buildroot:/# cat /etc/init.d/S03modules_init.sh
#!/bin/sh
#
# Modules init
#


start() {
    find /lib/modules/$(uname -r)/kernel/ -name "*.ko" | xargs -I {} sh -c 'modprobe $(basename {})'
}

stop() {
#   find /lib/modules/$(uname -r)/kernel/ -name "*.ko" | xargs -I {} sh -c 'modprobe -r $(basename {})'
    echo "skip rmmmod *.ko"
}

case "$1" in
    start)
        start
        ;;
    stop)
        stop
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        exit 1
esac

exit $?
root@rk3506-buildroot:/#
```

- 遍历 `/lib/modules/6.1.118/kernel/` 下所有 `.ko`，逐个 `modprobe`，**加载的以太网相关模块驱动** 。

- /etc/init.d/S40network

```
root@rk3506-buildroot:/# cat /etc/init.d/S40network
#!/bin/sh
#
# Start the network....
#

# Debian ifupdown needs the /run/network lock directory
mkdir -p /run/network

case "$1" in
  start)
    printf "Starting network: "
    /sbin/ifup -a
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  stop)
    printf "Stopping network: "
    /sbin/ifdown -a
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  restart|reload)
    "$0" stop
    "$0" start
    ;;
  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
esac

exit $?


root@rk3506-buildroot:/#
```

- /etc/init.d/S41dhcpcd

```
DAEMON=/sbin/dhcpcd
CONFIG=/etc/dhcpcd.conf
DHCPCD_ARGS="-AL"

start-stop-daemon -S -x "$DAEMON" -p "$PIDFILE" \
    -- -f "$CONFIG" $DHCPCD_ARGS

# 等价于: /sbin/dhcpcd -f /etc/dhcpcd.conf -AL
```

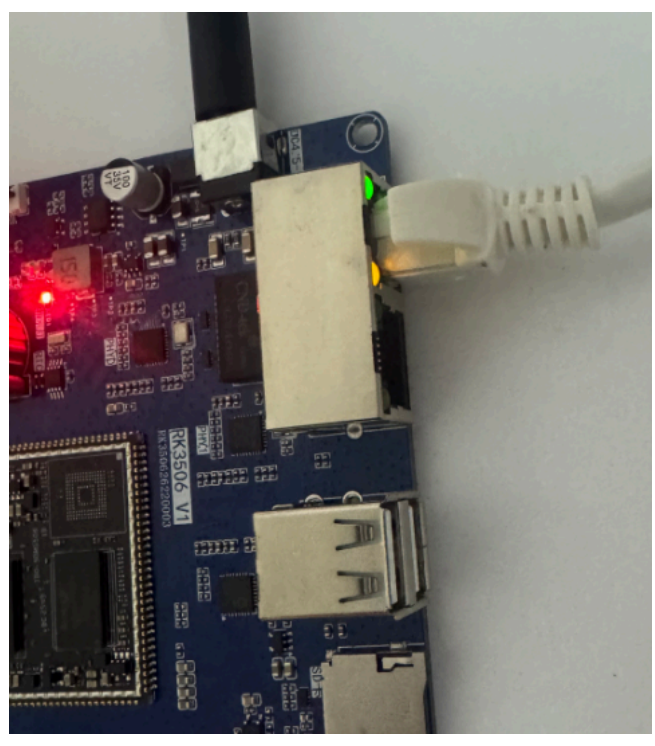
S40network 与 S41dhcpcd 的关系:

	S40network (ifup -a)	**S41dhcpcd (守护进程)**
二进制	busybox (92KB 内置)	dhcpcd (342KB 独立)
对 eth0 的行为	不处理（无 auto eth0）	监听 netlink，被动等待 carrier
对 wlan0 的行为	跳过（manual）	收到 carrier 后自动 DHCP
触发方式	一次性，启动时执行	常驻守护，持续监听
DHCP 方式	`udhcpc` (busybox 内置)	`dhcpcd` 自带完整客户端

S03 创建设备，S40 只配 lo，S41 启动 dhcpcd 守护进程。eth0 只有在被 `ip link set up` 之后，dhcpcd 才能在检测到 carrier 时自动 DHCP。这套机制的核心是 **netlink + epoll + carrier detection**，而非定时轮询。

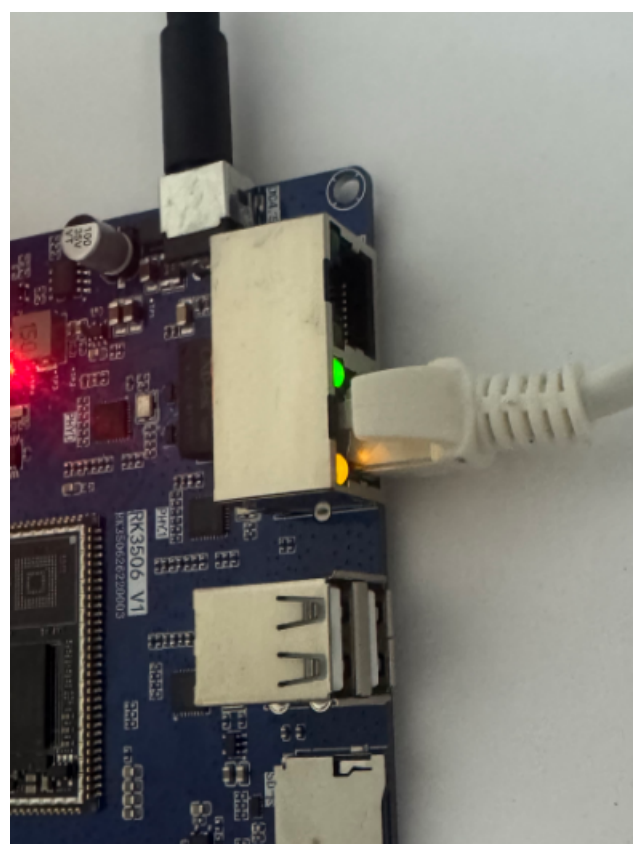
6. 以太网测试命令

- 插上网线 eth0



```
root@rk3506-buildroot:/# [ 1816.054732] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off

root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ping -I eth0 www.baidu.com
PING www.baidu.com (124.237.178.212): 56 data bytes
64 bytes from 124.237.178.212: seq=0 ttl=53 time=19.172 ms
64 bytes from 124.237.178.212: seq=1 ttl=53 time=18.851 ms
64 bytes from 124.237.178.212: seq=2 ttl=53 time=18.724 ms
64 bytes from 124.237.178.212: seq=3 ttl=53 time=19.317 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 18.724/19.016/19.317 ms
root@rk3506-buildroot:/#  # 拔掉网线
root@rk3506-buildroot:/# [ 2038.974595] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Down

root@rk3506-buildroot:/#
```

- 插上网线 eth1



```
root@rk3506-buildroot:/# [ 2130.170769] rk_gmac-dwmac ff4d0000.ethernet eth1: Link is Up - 100Mbps/Full - flow control off

root@rk3506-buildroot:/# ping -I eth1 www.baidu.com
PING www.baidu.com (124.237.178.212): 56 data bytes
64 bytes from 124.237.178.212: seq=0 ttl=53 time=19.453 ms
64 bytes from 124.237.178.212: seq=1 ttl=53 time=19.140 ms
64 bytes from 124.237.178.212: seq=2 ttl=53 time=67.293 ms
^C
--- www.baidu.com ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 19.140/35.295/67.293 ms
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# [ 2135.240576] rk_gmac-dwmac ff4d0000.ethernet eth1: Link is Down

root@rk3506-buildroot:/#
```

第18章 Linux LCD驱动

18.1 LCD接口

RK3506 是瑞芯微（Rockchip）面向 AIoT 和 HMI 等场景推出的通用型处理器，针对显示应用做了较好的性价比优化。它的 LCD 接口主要提供**两大类高速显示通道**，外加一些可用于低速屏的辅助接口，具体如下。

---

1. MIPI DSI 接口

这是 RK3506 的主力显示接口，适合驱动分辨率较高的 MIPI 屏。

- **通道规格**：**4 个数据通道**（4-lane），最高支持每通道 1.5 Gbps 左右的速率。
- **最大分辨率**：典型的可支持 **1920×1080 @ 60 fps**（全高清）。
- **工作模式**：支持 command 模式和 video 模式，可适配不同类型的小尺寸/中尺寸 DSI 面板。
- **色彩格式**：RGB565、RGB666、RGB888 等常见格式。
- **主要用途**：
  - 智能手机 / 平板类竖屏
  - 商业显示、人脸识别面板、智能家居中控屏
  - 常规 5 寸–10 寸左右高分辨率屏

注意：DSI 通道与 SoC 内部视频输出通路直连，不占用 GPIO 模拟，性能很好，是建议优先使用的接口。

---

2. 并行 RGB 接口

也就是传统的数字 LCD 接口，外接 RGB 屏时需要较多引脚，但普及度高。

- **接口宽度**：支持 **24-bit（RGB888）** 接口，也可向下兼容 18-bit（RGB666）、16-bit（RGB565）。
- **最大分辨率**：一般可以稳定达到 **1280×800 @ 60 fps** 左右，部分场景可达 1366×768 @ 60 fps。
- **同步信号**：标准 HSYNC、VSYNC、DE 模式（行/场同步或数据使能方式）。
- **时钟频率**：随屏分辨率不同，像素时钟通常在 30–80 MHz 范围。
- **主要用途**：
  - 7–10 寸左右的工业屏、低成本 TFT 屏
  - 显示同时需要与其它外设共享引脚时，要注意 IO 资源分配
  - 对 EMI 不敏感的设备，如固定安装的 HMI、工业面板

优点：屏资源丰富、驱动成熟，很多通用 RGB 屏直接可用。

缺点：占用 IO 多（24 bit 至少需 24 根数据线 + 控制线），对布线长度要求较高。

---

3. 辅助低速接口

如果系统需要驱动小尺寸的 MCU 屏或墨水屏，RK3506 还可以通过其它复用接口实现：

- **SPI 接口** 通过 SoC 的 SPI 控制器外接 SPI 协议的小屏（如 1.x–3.5 寸 TFT、电子纸），占用引脚很少，适合简单显示或低刷新场景。
- **8080 并行接口（灵活模拟）** RK3506 没有专用的 8080 总线，但可以利用其 **Flexible I/O**（灵活 GPIO）配合驱动的时序模拟 8080 接口，用来驱动一些 MCU 总线屏，不过刷新率较有限，一般用作静态信息显示。

---

4. 双屏异显能力

RK3506 支持 **MIPI DSI + RGB 同时输出**（双屏异显或同显），即：

- 主屏走 DSI，副屏走 RGB；
- 或者一个屏显示交互 UI，另一个屏显示固定图像/广告；
- 两个显示通道独立控制，可配置不同分辨率、不同刷新率。

这在需要**双屏收银机、电梯屏、自助终端**等应用中非常有价值。

5. MIPI DSI 结构

MIPI DSI（Display Serial Interface）是一种专为移动和嵌入式显示设计的串行总线，其协议栈从上到下可分为四层：

层次	功能	与 RK3506 的关系
应用层(VOP)	定义像素格式、视频/命令模式的时序行为	由显示控制器 (VOP) 和驱动配置

层次	功能	与 RK3506 的关系
协议层(DSI Host)	定义数据包格式，如何打包像素、命令和事件	DSI Host 控制器硬件自动完成
通道管理层(Lane)	把数据包分发到 1–4 条 Data Lane 上	根据屏要求配置 Lane 数量 (1/2/4)
物理层 (D-PHY)	负责电气传输，HS/LP 两种模式切换	RK3506 内部集成 D-PHY，支持 4 Lane

RK3506 内部集成了 DSI Host Controller 和 D-PHY，只要在驱动中配好参数，就能自动完成从像素流到串行信号的转换。

18.2 MIPI DSI 驱动

1. MIPI DSI 设备树

- kernel/arch/arm/boot/dts/rk3506.dtsi
- 显示数据通路 (VOP → DSI Host → DSI D-PHY → Panel)

```
    // 显示子系统
    display_subsystem: display-subsystem {
        compatible = "rockchip,display-subsystem";
        ports = <&vop_out>;
        status = "disabled";

        route {
            route_dsi: route-dsi {
                status = "disabled";
                logo,uboot = "logo.bmp";
                logo,kernel = "logo_kernel.bmp";
                logo,mode = "center";
                charge_logo,mode = "center";
                connect = <&vop_out_dsi>;
            };

            route_rgb: route-rgb {
                status = "disabled";
                logo,uboot = "logo.bmp";
                logo,kernel = "logo_kernel.bmp";
                logo,mode = "center";
                charge_logo,mode = "center";
                connect = <&vop_out_rgb>;
            };
        };
    };

    // VOP 使能
    vop: vop@ff600000 {
        compatible = "rockchip,rk3506-vop";
        reg = <0xff600000 0x200>, <0xff600a00 0x400>;
        reg-names = "regs", "gamma_lut";
        rockchip,grf = <&grf>;
        interrupts = <GIC_SPI 59 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru ACLK_VOP>, <&cru DCLK_VOP>, <&cru HCLK_VOP>;
        clock-names = "aclk_vop", "dclk_vop", "hclk_vop";
        status = "disabled";

        vop_out: port {
            #address-cells = <1>;
            #size-cells = <0>;

            vop_out_rgb: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&rgb_in_vop>;
            };

            vop_out_dsi: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&dsi_in_vop>;
            };
        };
    };


    // 连接 VOP 输出到 DSI 输入
    dsi: dsi@ff640000 {
        compatible = "rockchip,rk3506-mipi-dsi";
        reg = <0xff640000 0x10000>;
        interrupts = <GIC_SPI 60 IRQ_TYPE_LEVEL_HIGH>;
        clocks = <&cru PCLK_DSI_HOST>;
        clock-names = "pclk";
        resets = <&cru SRST_P_DSI_HOST>;
        reset-names = "apb";
        phys = <&dsi_dphy>;
        phy-names = "dphy";
        rockchip,grf = <&grf>;
        #address-cells = <1>;
        #size-cells = <0>;
        status = "disabled";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            dsi_in: port@0 {
                reg = <0>;
                #address-cells = <1>;
                #size-cells = <0>;
                dsi_in_vop: endpoint@0 {
                    reg = <0>;
                    remote-endpoint = <&vop_out_dsi>;
                    status = "disabled";
                };
            };
        };


    dsi_dphy: phy@ff670000 {
        compatible = "rockchip,rk3506-dsi-dphy";
        reg = <0xff670000 0x10000>,
              <0xff640000 0x10000>;
        reg-names = "phy", "host";
        clocks = <&cru CLK_REF_DPHY_TOP>,
             <&cru PCLK_DPHY>, <&cru PCLK_DSI_HOST>;
        clock-names = "ref", "pclk", "pclk_host";
        #clock-cells = <0>;
        resets = <&cru SRST_P_DPHY>;
        reset-names = "apb";
        #phy-cells = <0>;
        status = "disabled";
    };
```

- 背光设置
- kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts

```
backlight: backlight {
    compatible = "pwm-backlight";
    pwms = <&pwm0_4ch_2 0 25000 0>;  // PWM0_CH2, period=25000ns (40kHz)
    brightness-levels = <0 20 21 ... 255>;  // 256级非线性伽马校正表
    default-brightness-level = <200>;
};
```

- PWM 由 `pwm0_4ch_2` 提供，引脚 `rm_io4_pwm0_ch2`
- 亮度表不是线性的——低亮度区 (0~39) 每级加 1，保证暗光下的精细调节；这是典型的**人眼感知伽马补偿**做法
- 默认亮度 200/255 ≈ 78%

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts

```
// SPDX-License-Identifier: (GPL-2.0+ OR MIT)
/*
 * Copyright (c) 2024 Rockchip Electronics Co., Ltd.
 */


#include <dt-bindings/display/drm_mipi_dsi.h>
#include <dt-bindings/input/rk-input.h>
#include <dt-bindings/suspend/rockchip-rk3506.h>
#include "ido-evb3506-v1a-nand.dtsi"

/ {
    backlight: backlight {
        compatible = "pwm-backlight";
        pwms = <&pwm0_4ch_2 0 25000 0>;
```

```
        brightness-levels = <
              0  20  20  21  21  22  22  23
             23  24  24  25  25  26  26  27
             27  28  28  29  29  30  30  31
             31  32  32  33  33  34  34  35
             35  36  36  37  37  38  38  39
             40  41  42  43  44  45  46  47
             48  49  50  51  52  53  54  55
             56  57  58  59  60  61  62  63
             64  65  66  67  68  69  70  71
             72  73  74  75  76  77  78  79
             80  81  82  83  84  85  86  87
             88  89  90  91  92  93  94  95
             96  97  98  99 100 101 102 103
            104 105 106 107 108 109 110 111
            112 113 114 115 116 117 118 119
            120 121 122 123 124 125 126 127
            128 129 130 131 132 133 134 135
            136 137 138 139 140 141 142 143
            144 145 146 147 148 149 150 151
            152 153 154 155 156 157 158 159
            160 161 162 163 164 165 166 167
            168 169 170 171 172 173 174 175
            176 177 178 179 180 181 182 183
            184 185 186 187 188 189 190 191
            192 193 194 195 196 197 198 199
            200 201 202 203 204 205 206 207
            208 209 210 211 212 213 214 215
            216 217 218 219 220 221 222 223
            224 225 226 227 228 229 230 231
            232 233 234 235 236 237 238 239
            240 241 242 243 244 245 246 247
            248 249 250 251 252 253 254 255
        >;
        default-brightness-level = <200>;
        status = "okay";
    };
};

&display_subsystem {
    logo-memory-region = <&drm_logo>;
    status = "okay";
};

&pwm0_4ch_2 {
        pinctrl-names = "active";
        pinctrl-0 = <&rm_io4_pwm0_ch2>;
        status = "okay";
};

&dsi {
    status = "okay";
    //rockchip,lane-rate = <850>;
    dsi_panel: panel@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        reg = <0>;
        backlight = <&backlight>;
        reset-gpios  = <&gpio0 RK_PB5 GPIO_ACTIVE_LOW>;
        prepare-delay-ms = <5>;
        reset-delay-ms = <1>;
        init-delay-ms = <80>;
        disable-delay-ms = <10>;
        unprepare-delay-ms = <5>;

        width-mm = <68>;
        height-mm = <121>;

        dsi,flags = <(MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_BURST |
                  MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_NO_EOT_PACKET)>;
        dsi,format = <MIPI_DSI_FMT_RGB888>;
        dsi,lanes  = <2>;
        panel-init-sequence = [
            39 00 04 B9 F1 12 87
            39 00 04 B2 B4 03 70
            39 00 0B B3 10 10 28 28 03 FF 00 00 00 00
            15 00 02 B4 80
            39 00 03 B5 0A 0A
            39 00 03 B6 8D 8D
            39 00 05 B8 26 22 F0 13
            39 00 1C BA 31 81 05 F9 0E 0E 20 00 00 00 00 00 00 00 44 25 00 91 0A 00 00 01 4F 01 00 00 37
            15 00 02 BC 47
            39 00 06 BF 02 10 00 80 04
            39 00 0A C0 73 73 50 50 00 00 12 73 00
            39 00 12 C1 36 00 32 32 77 E1 77 77 CC CC FF FF 11 11 00 00 32
            39 00 0D C7 10 00 0A 00 00 00 00 00 ED C5 00 A5
            39 00 05 C8 10 40 1E 03
            15 00 02 CC 0B
            39 00 23 E0 00 0A 0F 2A 33 3F 44 39 06 0C 0E 14 15 13 15 10 18 00 0A 0F 2A 33 3F 44 39 06 0C 0E
14 15 13 15 10 18
            39 00 08 E1 11 11 91 00 00 00 00
            39 00 0F E3 07 07 0B 0B 0B 0B 00 00 00 00 FF 04 C0 10
            39 00 40 E9 C8 10 0A 00 00 80 81 12 31 23 4F 86 A0 00 47 08 00 00 0C 00 00 00 00 00 0C 00 00 00
98 02 8B AF 46 02 88 88 88 88 88 98 13 8B AF 57 13 88 88 88 88 88 00 00 00 00 00 00 00 00 00 00 00 00
            39 00 3E EA 97 0C 09 09 09 78 00 00 00 00 00 00 9F 31 8B A8 31 75 88 88 88 88 88 9F 20 8B A8 20
64 88 88 88 88 88 23 00 00 02 71 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 40 80 81 00 00 00 00
            39 00 04 EF FF FF 01
            05 96 01 11
            05 14 01 29
        ];

        disp_timings0: display-timings {
            native-mode = <&dsi_timing0>;
            dsi_timing0: timing0 {
                clock-frequency = <48000000>;
                hactive = <720>;
                vactive = <720>;
                hfront-porch = <106>;
                hsync-len = <8>;
                hback-porch = <120>;
                vfront-porch = <20>;
                vsync-len = <4>;
                vback-porch = <20>;
                hsync-active = <0>;
                vsync-active = <0>;
                de-active = <0>;
                pixelclk-active = <0>;
            };
        };

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                panel_in_dsi: endpoint {
                    remote-endpoint = <&dsi_out_panel>;
                };
            };
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@1 {
            reg = <1>;
            dsi_out_panel: endpoint {
                remote-endpoint = <&panel_in_dsi>;
            };
        };
    };

};

&dsi_dphy {
    status = "okay";
};

&dsi_in_vop {
    status = "okay";
};

&dsi_panel {
    power-supply = <&vcc3v3_lcd_n>;
```

```
};

&route_dsi{
        status = "okay";
        connect = <&vop_out_dsi>;
};

&i2c0 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&rm_io0_i2c0_scl
             &rm_io1_i2c0_sda>;

    gt911@14 {
        compatible = "goodix,gt9xx";
        reg = <0x14>;
        pinctrl-0 = <&touch_gpio>;
        touch-gpio = <&gpio0 RK_PA2 IRQ_TYPE_LEVEL_LOW>;
        reset-gpio = <&gpio0 RK_PA3 GPIO_ACTIVE_HIGH>;
        max-x = <720>;
        max-y = <720>;
        tp-size = <9110>;
        status = "okay";
    };
};

&vop {
    status = "okay";
};

&pinctrl {
    touch {
        touch_gpio: touch-gpio {
            rockchip,pins = <0 RK_PA2 RK_FUNC_GPIO &pcfg_pull_none>;
        };

        reset_gpio: reset-gpio {
            rockchip,pins = <0 RK_PA3 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};
```

2. MIPI DSI 内核驱动

- 主驱动：Synopsys DesignWare MIPI DSI + Rockchip 适配层
- kernel/drivers/gpu/drm/rockchip/dw-mipi-dsi-rockchip.c

```
// SPDX-License-Identifier: GPL-2.0+
/*
 * Copyright (C) Rockchip Electronics Co., Ltd.
 * Author:
 *      Chris Zhong <zyw@rock-chips.com>
 *      Nickey Yang <nickey.yang@rock-chips.com>
 */

#include <linux/clk.h>
#include <linux/iopoll.h>
#include <linux/math64.h>
#include <linux/mfd/syscon.h>
#include <linux/module.h>
#include <linux/of_device.h>
#include <linux/phy/phy.h>
#include <linux/pm_runtime.h>
#include <linux/regmap.h>

#include <video/mipi_display.h>
#include <uapi/linux/videodev2.h>
#include <drm/bridge/dw_mipi_dsi.h>
#include <drm/drm_mipi_dsi.h>
#include <drm/drm_of.h>
#include <drm/drm_simple_kms_helper.h>

#include "rockchip_drm_drv.h"
#include "rockchip_drm_vop.h"

。。。

static const struct of_device_id dw_mipi_dsi_rockchip_dt_ids[] = {
#if IS_ENABLED(CONFIG_CPU_PX30)
    {
     .compatible = "rockchip,px30-mipi-dsi",
     .data = &px30_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK312X)
    {
     .compatible = "rockchip,rk3128-mipi-dsi",
     .data = &rk3128_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK3288)
    {
     .compatible = "rockchip,rk3288-mipi-dsi",
     .data = &rk3288_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK3399)
    {
     .compatible = "rockchip,rk3399-mipi-dsi",
     .data = &rk3399_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK3506)
    {
     .compatible = "rockchip,rk3506-mipi-dsi",
     .data = &rk3506_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK3562)
    {
     .compatible = "rockchip,rk3562-mipi-dsi",
     .data = &rk3562_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RK3568)
    {
     .compatible = "rockchip,rk3568-mipi-dsi",
     .data = &rk3568_chip_data,
    },
#endif
#if IS_ENABLED(CONFIG_CPU_RV1126)
    {
     .compatible = "rockchip,rv1126-mipi-dsi",
     .data = &rv1126_chip_data,
    },
#endif
    { /* sentinel */ }
};
MODULE_DEVICE_TABLE(of, dw_mipi_dsi_rockchip_dt_ids);

struct platform_driver dw_mipi_dsi_rockchip_driver = {
    .probe      = dw_mipi_dsi_rockchip_probe,
    .remove     = dw_mipi_dsi_rockchip_remove,
    .driver     = {
        .of_match_table = dw_mipi_dsi_rockchip_dt_ids,
        .pm = &dw_mipi_dsi_rockchip_pm_ops,
        .name   = "dw-mipi-dsi-rockchip",
        /*
         * For dual-DSI display, one DSI pokes at the other DSI's
         * drvdata in dw_mipi_dsi_rockchip_find_second(). This is not
         * safe for asynchronous probe.
         */
        .probe_type = PROBE_FORCE_SYNCHRONOUS,
    },
};
```

- DSI 物理层（D-PHY）

```
// SPDX-License-Identifier: GPL-2.0
/*
 * Copyright (c) 2018 Rockchip Electronics Co., Ltd.
 *
 * Author: Wyon Bi <bivvy.bi@rock-chips.com>
 */
```

```
#include <linux/bits.h>
#include <linux/kernel.h>
#include <linux/clk.h>
#include <linux/iopoll.h>
#include <linux/clk-provider.h>
#include <linux/delay.h>
#include <linux/init.h>
#include <linux/mfd/syscon.h>
#include <linux/module.h>
#include <linux/of_device.h>
#include <linux/platform_device.h>
#include <linux/pm_runtime.h>
#include <linux/reset.h>
#include <linux/time64.h>

#include <linux/phy/phy.h>
#include <linux/phy/phy-mipi-dphy.h>
#include <linux/rockchip/cpu.h>

。。。

static const struct of_device_id inno_dsidphy_of_match[] = {
    {
        .compatible = "rockchip,px30-dsi-dphy",
        .data = &px30_video_phy_plat_data,
    }, {
        .compatible = "rockchip,px30s-dsi-dphy",
        .data = &px30s_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rk3128-dsi-dphy",
        .data = &rk3128_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rk3368-dsi-dphy",
        .data = &rk3368_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rk3506-dsi-dphy",
        .data = &rk3506_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rk3562-dsi-dphy",
        .data = &rk3562_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rk3568-dsi-dphy",
        .data = &rk3568_video_phy_plat_data,
    }, {
        .compatible = "rockchip,rv1126-dsi-dphy",
        .data = &rv1126_video_phy_plat_data,
    },
    {}
};
MODULE_DEVICE_TABLE(of, inno_dsidphy_of_match);

static struct platform_driver inno_dsidphy_driver = {
    .driver = {
        .name = "inno-dsidphy",
        .of_match_table = of_match_ptr(inno_dsidphy_of_match),
    },
    .probe = inno_dsidphy_probe,
    .remove = inno_dsidphy_remove,
};
module_platform_driver(inno_dsidphy_driver);

MODULE_AUTHOR("Wyon Bi <bivvy.bi@rock-chips.com>");
MODULE_DESCRIPTION("Innosilicon MIPI/LVDS/TTL Video Combo PHY driver");
MODULE_LICENSE("GPL v2");
```

- VOP 显示控制器
- kernel/drivers/gpu/drm/rockchip/rockchip_drm_vop.c
- Rockchip DRM 主驱动
- kernel/drivers/gpu/drm/rockchip/rockchip_drm_drv.c

- 驱动配置

```
linux@ubuntu:~/rk3506$ ./build.sh kernel-config
Device Drivers  --->
    Graphics support  --->
        # 启用 DRM (Direct Rendering Manager)
        <*> Direct Rendering Manager (XFree86 4.1.0 and higher DRI support)  --->
        <*> DRM Support for Rockchip   # 启用 Rockchip DRM 支持
        [*]   Rockchip VOP driver
        # 注意：RK3506 使用 VOP（不是 VOP2），DSI2 不需要启用。
        [*]   Rockchip specific extensions for Synopsys DW MIPI DSI
        [*]   Rockchip RGB support
        Display Panels  --->    #启用 Simple Panel 驱动（simple-panel-dsi）
            <*>   support for simple panels (other than eDP ones)
            [*]   compatible is simple-panel or panel-dpi or simple-panel-dsi only

        #启用 PWM Backlight 支持
        Backlight & LCD device support  --->
            <*>   Lowlevel Backlight controls
            <*>   Generic PWM based Backlight Driver

    PHY Subsystem  --->
        <*> Rockchip Innosilicon MIPI/LVDS/TTL PHY driver
        #对于 RK3506 / RV1106 这类芯片，DSI 物理层使用的是 Innosilicon IP
linux@ubuntu:~/rk3506$ ./build.sh kernel   # 编译内核
```

- 编译内核更新设备树与内核到开发板内。

3. buildroot的LVGL例子功能

- /app/lvgl_demo/

```
app/lvgl_demo/
├── CMakeLists.txt              # 顶层 CMake（选择 rk_demo 或 lv_demo）
├── lvgl8/                      # LVGL v8 适配层
│   ├── lv_port_init.c          # ★ LVGL + 显示 + 输入初始化
│   ├── lv_port_init.h
│   ├── lv_port_disp.c          # ★ 显示后端选择 (#if USE_RKADK / USE_DRM)
│   └── lv_port_disp.h
├── lvgl9/                      # LVGL v9 适配层（RK3506 不使用）
├── rk_demo/                    # ★ rk_demo 应用（RK3506 使用）
│   ├── main.c                  # ★ 主入口：lvgl_init() → rk_demo_init() → event loop
│   ├── main.h
│   ├── S10lv_demo              # ★ init 启动脚本
│   ├── home/                   #   主页面 UI
│   ├── smart_home/             #   智能家居 UI
│   ├── furniture_control/      #   家具控制 UI
│   ├── setting/                #   设置 UI
│   ├── resource/               #   图片/字体资源
│   └── rockit/                 #   多媒体集成
├── lv_demo/                    # 通用 lv_demo 应用（widgets/benchmark/music）
│   ├── main.c
│   └── S10lv_demo
├── common/                     # 通用工具函数
├── sys/                        # 系统工具
├── cJSON/                      # JSON 解析库
└── tools/                      # 构建工具
```

4. LVGL例子启动分析

- 整体启动链路

```
Linux 内核启动
    ↓
BusyBox init 启动，读取 /etc/inittab
    ↓
执行 S00async-commit.sh（异步提交优化）
    ↓
执行 S10lv_demo → 启动 /usr/bin/rk_demo（LVGL GUI）
    ↓
返回 rcS 继续启动其他系统服务（网络、WiFi/BT、USB 等）
```

- buildroot/output/rockchip_rk3506/target/etc/inittab

```
# /etc/inittab
#
# Copyright (C) 2001 Erik Andersen <andersen@codepoet.org>
#
# Note: BusyBox init doesn't support runlevels.  The runlevels field is
```

```
# completely ignored by BusyBox init. If you want runlevels, use
# sysvinit.
#
# Format for each entry: <id>:<runlevels>:<action>:<process>
#
# id        == tty to run on, or empty for /dev/console
# runlevels == ignored
# action    == one of sysinit, respawn, askfirst, wait, and once
# process   == program to run

# Startup the system
::sysinit:/bin/mount -t sysfs sysfs /sys
::sysinit:/etc/init.d/rcS.early
::sysinit:/bin/mkdir -p /dev/pts /dev/shm
::sysinit:/bin/mount -t tmpfs tmpfs /var/log
::sysinit:/bin/mount -t tmpfs tmpfs /tmp
::sysinit:/bin/mount -t tmpfs tmpfs /run
::sysinit:/bin/mkdir -p /run/lock/subsys
::sysinit:/sbin/swapon -a
null::sysinit:/bin/ln -sf /proc/self/fd /dev/fd
null::sysinit:/bin/ln -sf /proc/self/fd/0 /dev/stdin
null::sysinit:/bin/ln -sf /proc/self/fd/1 /dev/stdout
null::sysinit:/bin/ln -sf /proc/self/fd/2 /dev/stderr
::sysinit:/bin/hostname -F /etc/hostname
# now run any rc scripts
::sysinit:/etc/init.d/rcS

# Put a getty on the serial port
::respawn:-/bin/sh # ttyS0::respawn:/sbin/getty -L ttyS0 115200 vt100 # GENERIC_SERIAL

# Stuff to do for the 3-finger salute
#::ctrlaltdel:/sbin/reboot

# Stuff to do before rebooting
::shutdown:/etc/init.d/rcK
::shutdown:/sbin/swapoff -a
::shutdown:/bin/umount -a -r
```

优先执行 `::sysinit:/etc/init.d/rcS.early`

- buildroot/output/rockchip_rk3506/target/etc/init.d/rcS.early

```
linux@ubuntu:~/rk3506$ cat
#!/bin/sh

# Start all init scripts in /etc/init.d/pre_init/
# executing them in numerical order.
#
for i in /etc/init.d/pre_init/S??* ;do

     # Ignore dangling symlinks (if any).
     [ ! -f "$i" ] && continue

     case "$i" in
        *.sh)
            # Source shell script for speed.
            (
                trap - INT QUIT TSTP
                set start
                . $i
            )
            ;;
        *)
            # No sh extension, so fork subprocess.
            $i start
            ;;
    esac
done
```

搜索目录下的脚本 `/etc/init.d/pre_init/S??`

```
linux@ubuntu:~/rk3506$ ls buildroot/output/rockchip_rk3506/target/etc/init.d/pre_init/
S00async-commit.sh  S10lv_demo
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ cat buildroot/output/rockchip_rk3506/target/etc/init.d/pre_init/S00async-commit.sh
#!/bin/sh
### BEGIN INIT INFO
# Provides:       async-commit
# Required-Start:
# Required-Stop:
# Default-Start:  S
# Default-Stop:
# Description:    Enable ASYNC_COMMIT for Rockchip BSP kernel > 4.4
### END INIT INFO

case "$1" in
        start)
                /usr/bin/async-commit ;;
        stop)
                ;;
        restart|reload)
                ;;
        *)
                echo "Usage: $0 {start|stop|restart}"
                exit 1
esac

exit $?
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ cat buildroot/output/rockchip_rk3506/target/etc/init.d/pre_init/S10lv_demo
#! /bin/sh

start() {
        set -a
        export LV_DRIVERS_SET_PLANE=CURSOR
        ulimit -n 1024
        set +a
        rk_demo &
        sleep 1 # All processer help rk_demo output first UI.
}

case "$1" in
        start)
        start
        ;;
        *)
        echo "Usage: $0 {start}"
        exit 1
esac

exit $?
linux@ubuntu:~/rk3506$
```

- /usr/bin/async-commit 作用

一句话：启用 DRM 的 `ASYNC_COMMIT` 属性，加速显示提交。

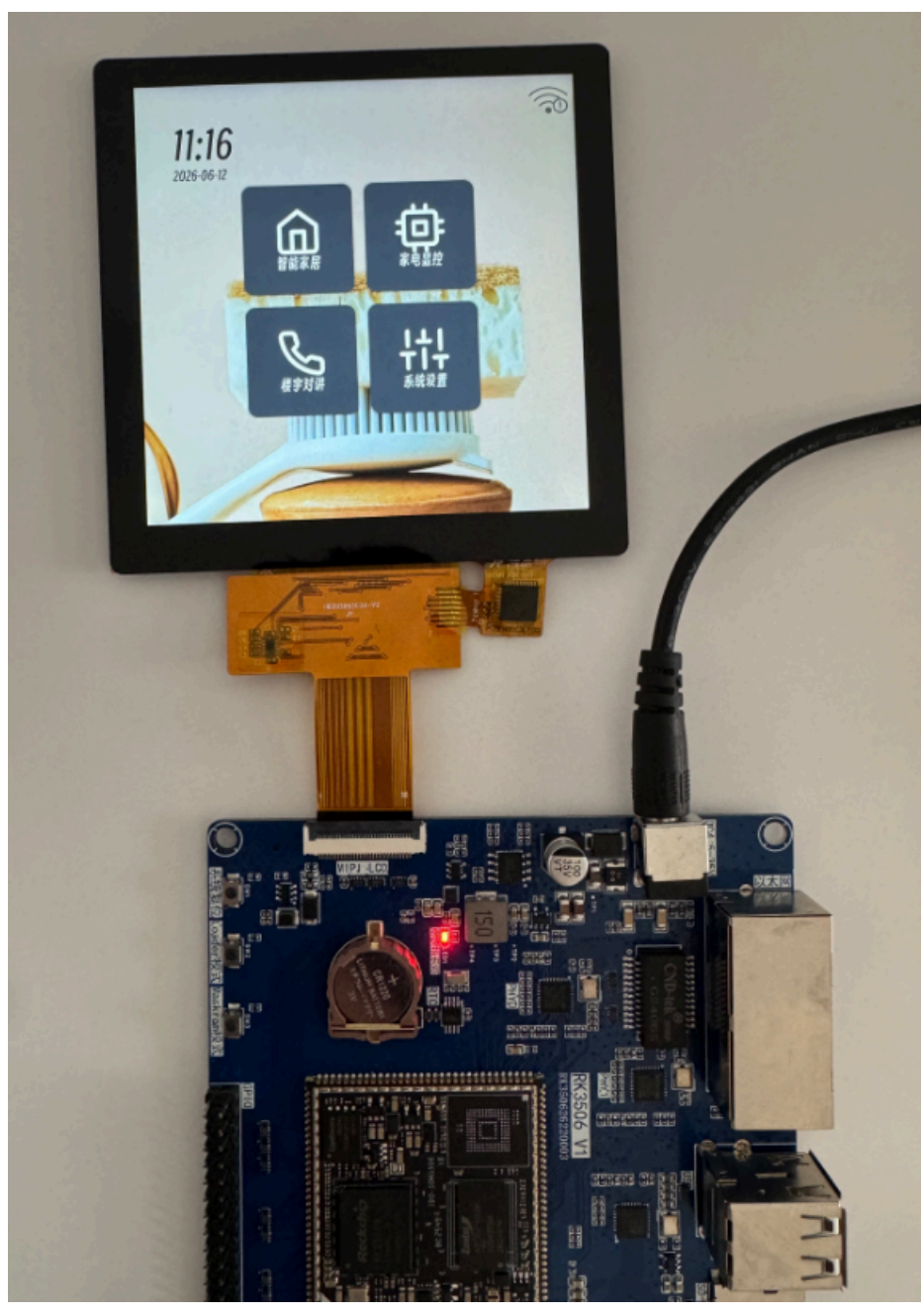
具体做法：脚本遍历所有 DRM plane，将 `ASYNC_COMMIT` 属性设为 1。这会通知 Rockchip BSP 内核（≥4.4）使用异步提交模式——显示提交不再等待 VBlank 同步，直接更新 framebuffer，从而**显著降低首帧渲染延迟**，配合 `S10lv_demo` 实现快速启动显示效果。

- 启动时序图总结

```
[内核]                              [用户空间]
  |                                    |
  ├─ DSI/MIPI 面板初始化               |
  ├─ VOP 初始化                        |
  ├─ 背光 PWM 初始化                   |
  ├─ 触摸控制器初始化                  |
  ├─ DRM Logo 显示                     |
  |                                    |
  ├─ 挂载 rootfs ──────────────────────┤
  |                                    ├─ init 读取 inittab
  |                                    ├─ rcS.early 执行
  |                                    |   ├─ S00async-commit (DRM 优化)
  |                                    |   └─ S10lv_demo → rk_demo &
  |                                    |       ├─ lv_port_init (DRM + 触摸)
  |                                    |       ├─ 字体/样式初始化
  |                                    |       ├─ rk_demo_init (构建 UI)
  |                                    |       └─ lv_task_handler 主循环
  |                                    |
  |                                    ├─ rcS 执行
  |                                    |   ├─ S00mountall (挂载文件系统)
  |                                    |   ├─ S03modules_init (加载模块)
  |                                    |   └─ S50/S89/S99 (USB/WiFi/BT/应用)
```

5. MIPI DSI测试

- 插上MIPI 屏幕后， 显示只能家具的实例



第四部分 EtherCAT主站实战

第19章 EtherCAT协议

19.1 EtherCAT概述

1. EtherCAT定义

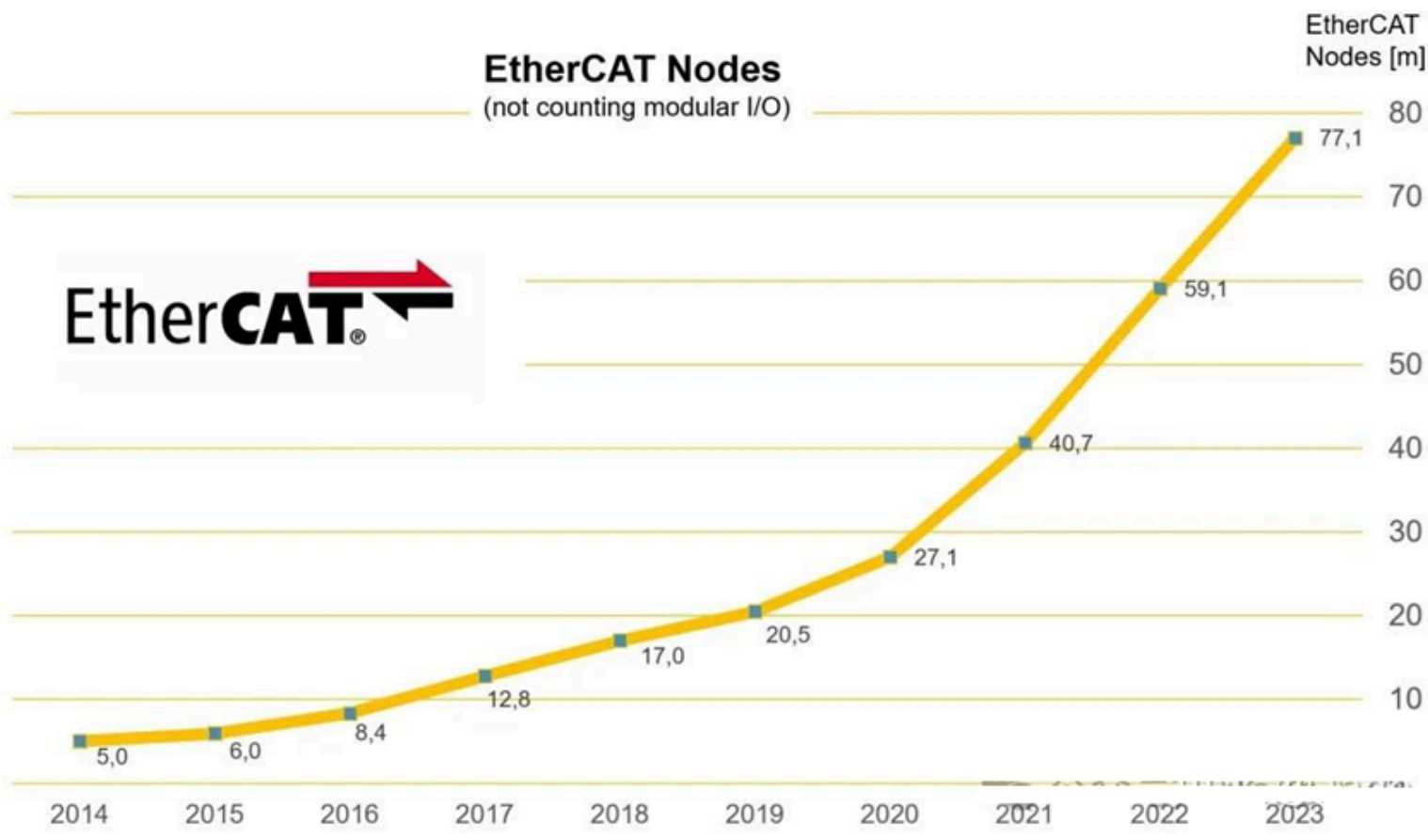
EtherCAT（以太网控制自动化技术，Ethernet for Control Automation Technology的缩写）是一个开放架构，以以太网为基础的现场总线系统，EtherCAT是确定性的**工业以太网**，最早是由德国倍福Beckhoff公司于2003研发推出，于2007年成为国际标准，并于2014年成为中国国家标准。

自动化对通讯一般会要求较短的资料更新时间（或称为周期时间）、资料同步时的通讯抖动量低，而且硬件的成本要低，EtherCAT开发的目的就是让以太网可以运用在自动化应用中。

EtherCAT的出现为系统的实时性能和拓扑的灵活性树立了新的标准。

2. EtherCAT全球节点数

根据ETG官网数据，全球EtherCAT总计7700万个节点数，含2023年新增的1800万个 。



上图可看出，从2020年开始，每年增加的节点数急速上升且趋势一直保持。

EtherCAT技术协会的会员数量也在持续增长。ETG拥有来自74个国家的7600多家会员单位，仍然是全球最大的现场总线用户组织。

尽管目前的经济形势充满挑战，但在过去12个月里，会员甚至在加速增长中，ETG报告称，2023年新增会员超过500家。

3. EtherNet以太网

20世纪80年代，以太网是一种相对专业的计算机连接技术，由经验丰富的大师在高度受控的办公环境下管理。

从那时起，技术和性能取得了令人难以置信的进步，世界上许多人在日常生活中使用有线和无线（Wi-Fi）以太网。

当然，联网对于工业自动化目的也同样至关重要，但用于该服务的技术始终需要满足以下几个要求：

- **适用于现场环境的外形因素和安装方法（易于安装，能承受环境温度/污垢/湿气/物理条件）**
- **对来自电源、电机、接触器和其他来源的电噪声具有强大的抵抗力**
- **足够的带宽和响应能力**

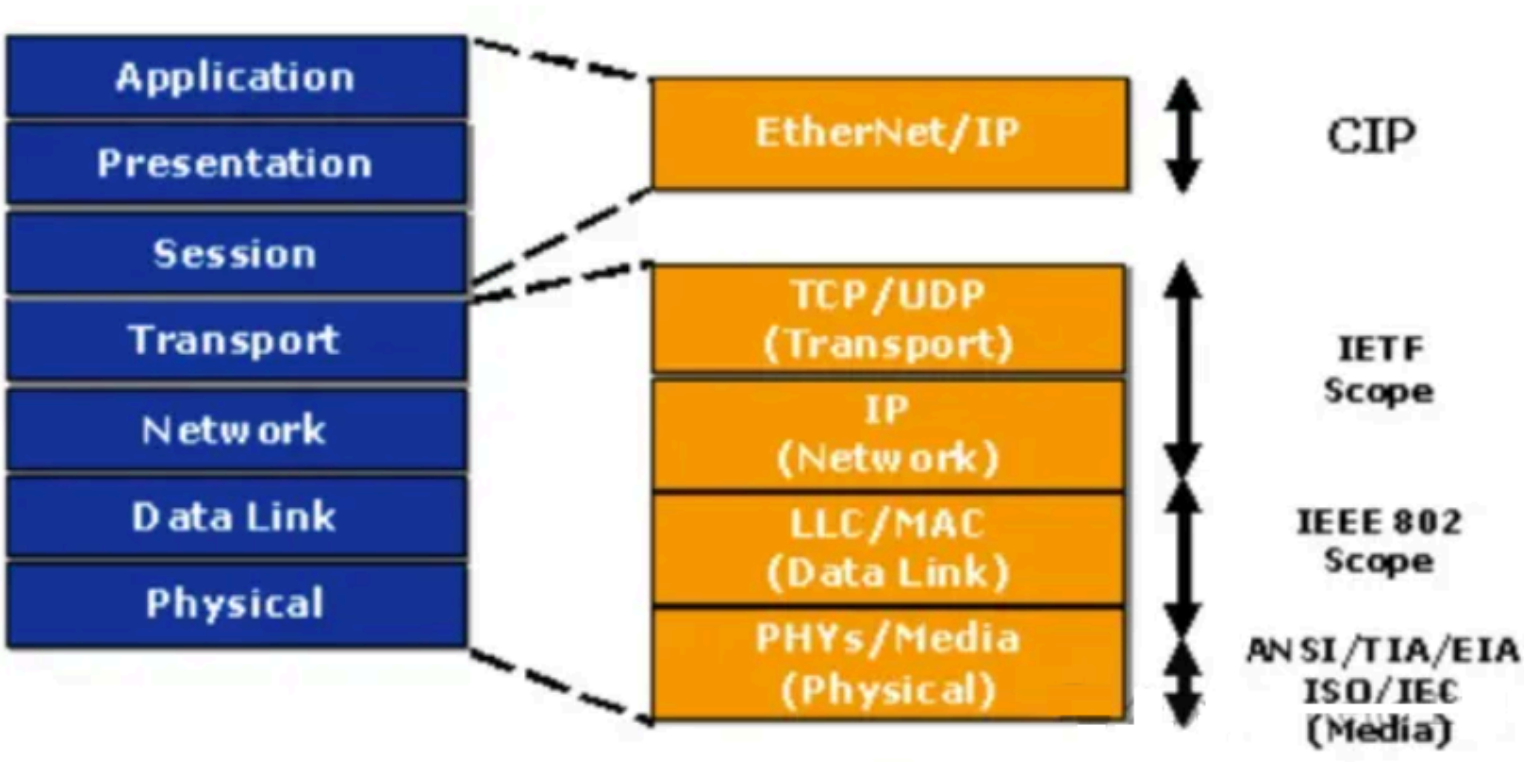
最早的数字通信和网络操作技术（OT）方法是专门的，甚至有时是专有的，而且是有限的（有几个版本是可行）。然而，最终的规模经济使以太网信息技术（IT）有可能在所有级别的工业自动化通信中占据主导地位。

4. EtherNet vs EtherCAT

基于以太网的总线解决方案已成为运动和工业控制通信的主导方法。TCP、UDP、ModbusTCP、EtherNet/IP、ProfiNet和EtherCAT均是实现高速、稳健通信的几个以太网协议。

由于EtherCAT和以太网使用相同的物理硬件，并且经常一起使用，因此很难区分它们。然而，控制系统设计者可以通过理解几个关键因素（如应用场合、同步性和成本等）来做出明智的选择。

一般地，EtherNet遵循标准的OSI 7层网络协议如下图所示：



相比较而言，EtherCAT基于EtherNet进行了创新和改进，没有那么多层，仅第1、2层用来传递数据。

所以基于此在传递数据上，速度更快，且减少了各层之间的数据传递风险和不确定性。

5. 通讯速度对比

我们通过下面的例子来说明EtherCAT总线通讯的速度：

Ethernet 802.3 Frame

Preamble	Dest Address	Source Address	Ethernet Type	Payload	Frame Check
8 bytes	6 bytes	6 bytes	2 bytes	Min 46 bytes	4 bytes

EtherCAT Frame

Header	Data Payload	More Data Payloads...
2 bytes	12 bytes + data bytes	12 bytes + data bytes

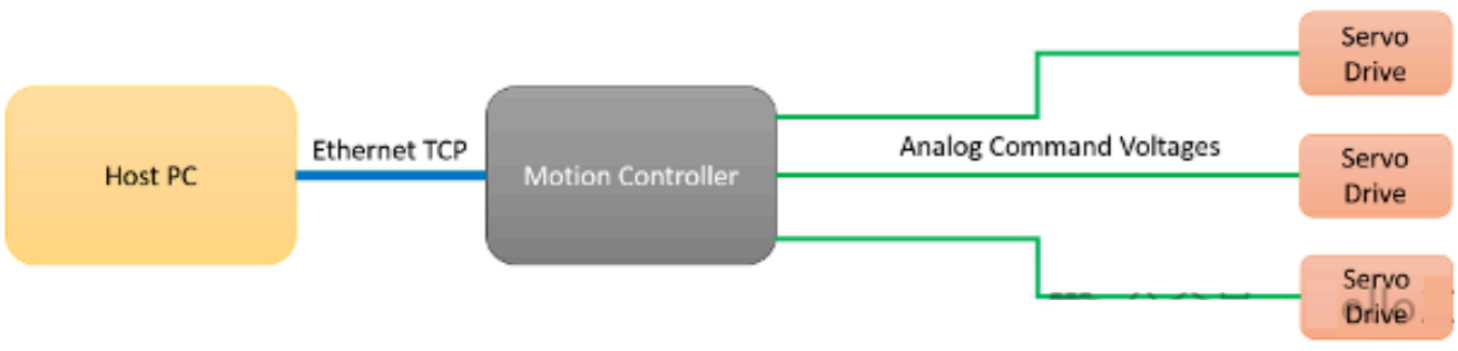
根据上述通讯帧，如果传递一个DINT类型的（4个字节）的数据，那么：

- EtherNet需要传输总计：8+6+6+2+46+4 = 72 byte 的数据长度
- EtherCAT需要传输总计：2+12+4 = 18byte 的数据长度

所以，同样的通讯介质下，EtherCAT通讯所需要的时间仅仅是EtherNet通讯的四分之一。

6. 运控对比

如果使用以太网的方式做运动控制，一般指的是主机和Motion运动控制器之间通过TCP通讯的方式进行指令和数据收发，通过控制器来实现运动控制，那么本身以太网的实时性不高，所以无法控制同步性能很强的运控解决方案。



相对于上述方案而言，EtherCAT总线控制方案里面多了EtherCAT Master主站，所有的从站（IO或者伺服）通过EtherCAT进行实时通讯，最低可达微秒的级别。

当然，EtherCAT需要更多的成本，首先需要对主从站进行软件配置，其次每个从站需要特定的协议数据包，且无法与传统的路由器、交换机进行网络兼容，最后每个从站还需要专用的EtherCAT总线ASIC芯片。

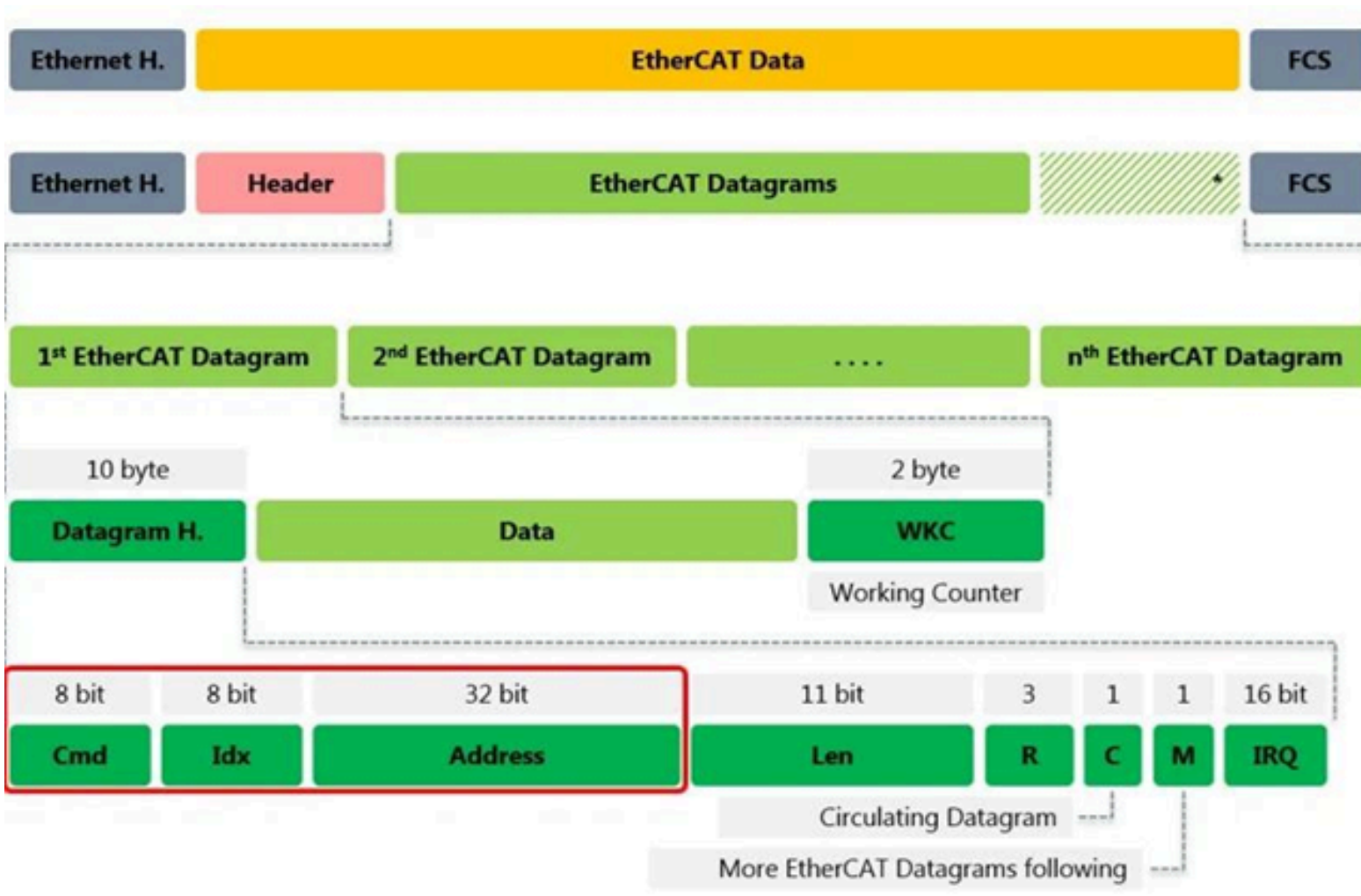
对于更低成本但实时性要求不高的场合，可以考虑通用的EtherNet通讯，毕竟这种技术已经发展近50年，相对成熟，实施也更快。

而对于运动控制场合，特别是1ms内的同步控制，那首选是EtherCAT。

19.2 EtherCAT的运行机制

1. EtherCAT帧

与标准以太网通信一样，EtherCAT利用以太网帧在整个网络中传输数据。EtherCAT帧基于IEEE 802.3以太网标准；然而，它们是以一种特殊的方式构建的，可以针对增加的带宽和短周期的过程数据对它们进行优化。

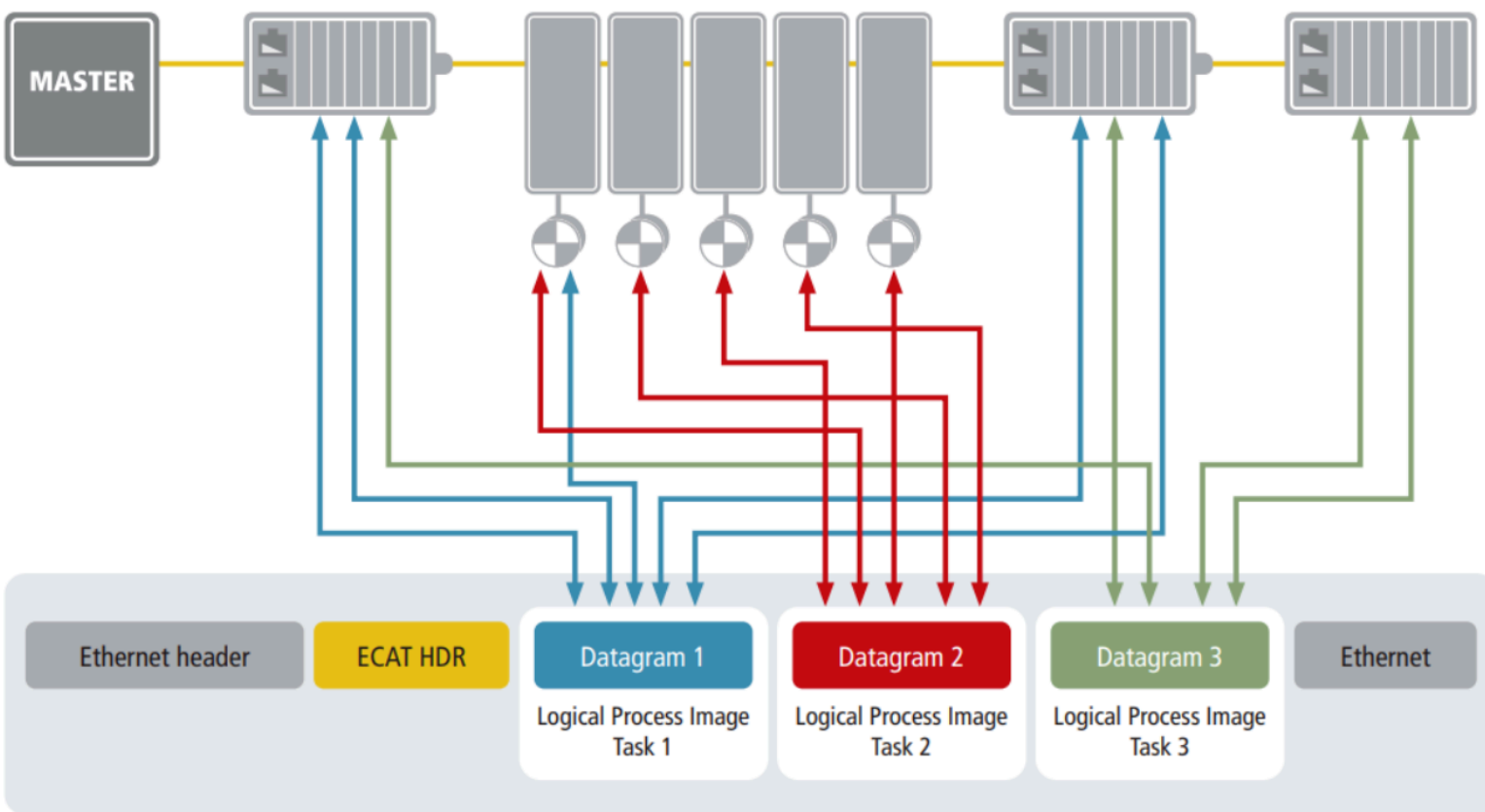


- EtherCAT帧还消除了更大的协议栈，如UDP/IP或TCP/IP，这意味着EtherCAT不是一个基于IP的协议，更类似于第2层或数据链路层协议。
- EtherCAT帧或电报由以太网标头组成，后跟EtherCAT数据，并以帧检查序列（FCS）结束。
- EtherCAT协议通过使用以太网头中EtherType字段中的0x88A4标识符进行识别。
- EtherCAT数据包含一个EtherCAT特定的头，后面跟着EtherCAT　Datagram。
- EtherCAT标头指定后续EtherCAT数据报文的总长度和类型。
- EtherCAT头之后是EtherCAT数据报文，它包含将在网络中读取或写入的实际数据。这些数据包括地址规范、主机想要执行的命令类型（即读取、写入或读写）以及循环过程数据（PDO）。

- EtherCAT帧最大1498字节，超量则主设备多发帧并带标识符指示后续帧。
- 主机发送的每帧经所有节点，无需交换机，降低了定时延迟和硬件成本。

2. 机制On the fly

从字面上意思来看，意味着EtherCAT处理数据是“飞一般的快速”。



EtherCAT利用On-the-fly机制，每周期只发一帧。主站组装帧后，顺序经过各从站。帧含寻址、读写命令及过程数据。

从站动态提取发往自己的数据，并插入新数据。帧到达最后一个节点后，通过以太网全双工发回主站。

形象的比喻：

3. On-the-fly优势

EtherCAT的动态处理机制（On-the-fly）提供了许多性能和成本节约优势。

- 大网络可能用多帧，但单帧通常足矣收发所有数据。
- 主站是唯一发送设备；其余设备仅接收、处理、转发，杜绝意外延迟，适合实时应用。

19.3 EtherCAT同步性

1. 同步性重要性

在工业自动化设备中，之所以选择总线，主要考虑的是通讯速度、带宽还有就是稳定性和同步性。

特别是在运控领域，微秒级别的同步，高速运动过程中的位置锁存（飞拍、追剪等应用），那同步性是相当重要。

2. 分布式时钟（DC）原理

在EtherCAT总线网络中，术语“**分布式时钟**”是指分布式时钟的逻辑网络。

EtherCAT通过分布式时钟，能在非常窄的容差内同步所有本地总线设备的时间。支持分布式时钟的从站，内含本地时钟，上电后最初本地运行，基于内部独立时钟生成器（石英振荡器）。

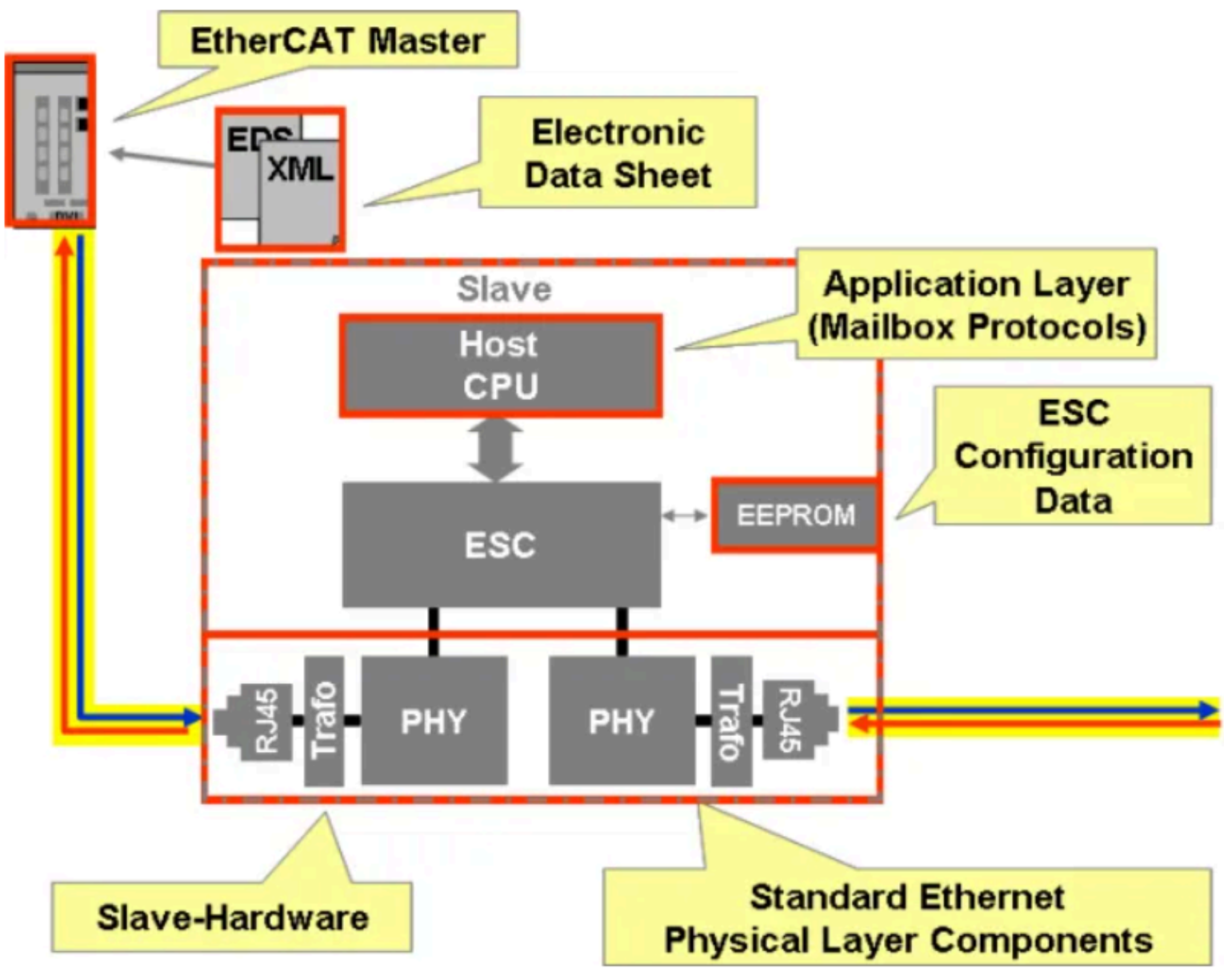
EtherCAT链中选定一从站为参考时钟（M），其他从时钟（S）与之同步，参考时钟即系统时间。若主机支持分布式时钟（如BeckhoffTwinCAT），可自动连续同步。

由于EtherCAT的环形结构，如果参考时钟在拓扑上位于所有其他从站时钟之前，**EtherCAT主设备选择第一个具有分布式时钟功能的EtherCAT从站作为参考时钟。**

EtherCAT配置的特点是EtherCAT主机操作和管理与连接的EtherCAT从站的总线。其中一个EtherCAT从站包含参考时钟(M)，所有其他EtherCAT设备（即包括EtherCAT主站）代表从站时钟（S)。

EtherCAT从站控制器(ESC)处理通信和分布式时钟，是ASIC或可重编程FPGA等芯片。每个EtherCAT从设备都有这样一个ESC，确保通过现场总线在主从间交换循环与非循环数据。它可直接处理数字I/O，或通过串/并口连接处理器处理驱动任务，并管理本地分布式时钟。

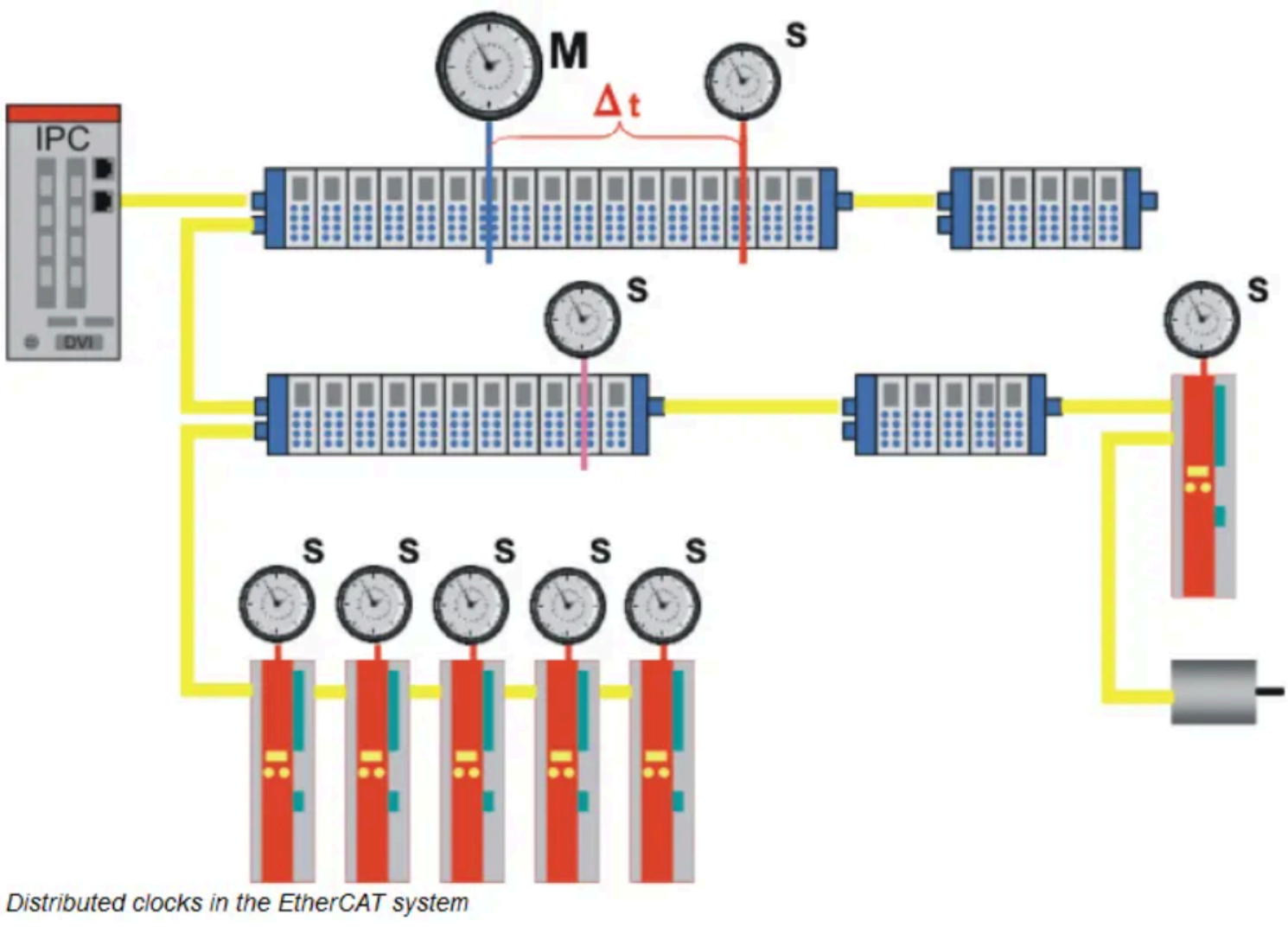
EtherCAT从站的示意图如图所示。ESC功能和嵌入典型EtherCAT从属机的示例。此外，在后者中嵌入ESC及其基本功能的选择。



- 信号经转变器从RJ45至PHY，提取数据送ESC处理。
- 报文以最小延迟经PHY和插座动态处理中继至下从机；
- 启动时ESC从EEPROM加载配置；
- 若有CPU，从站可经接口与其通信。

3. 同步性能

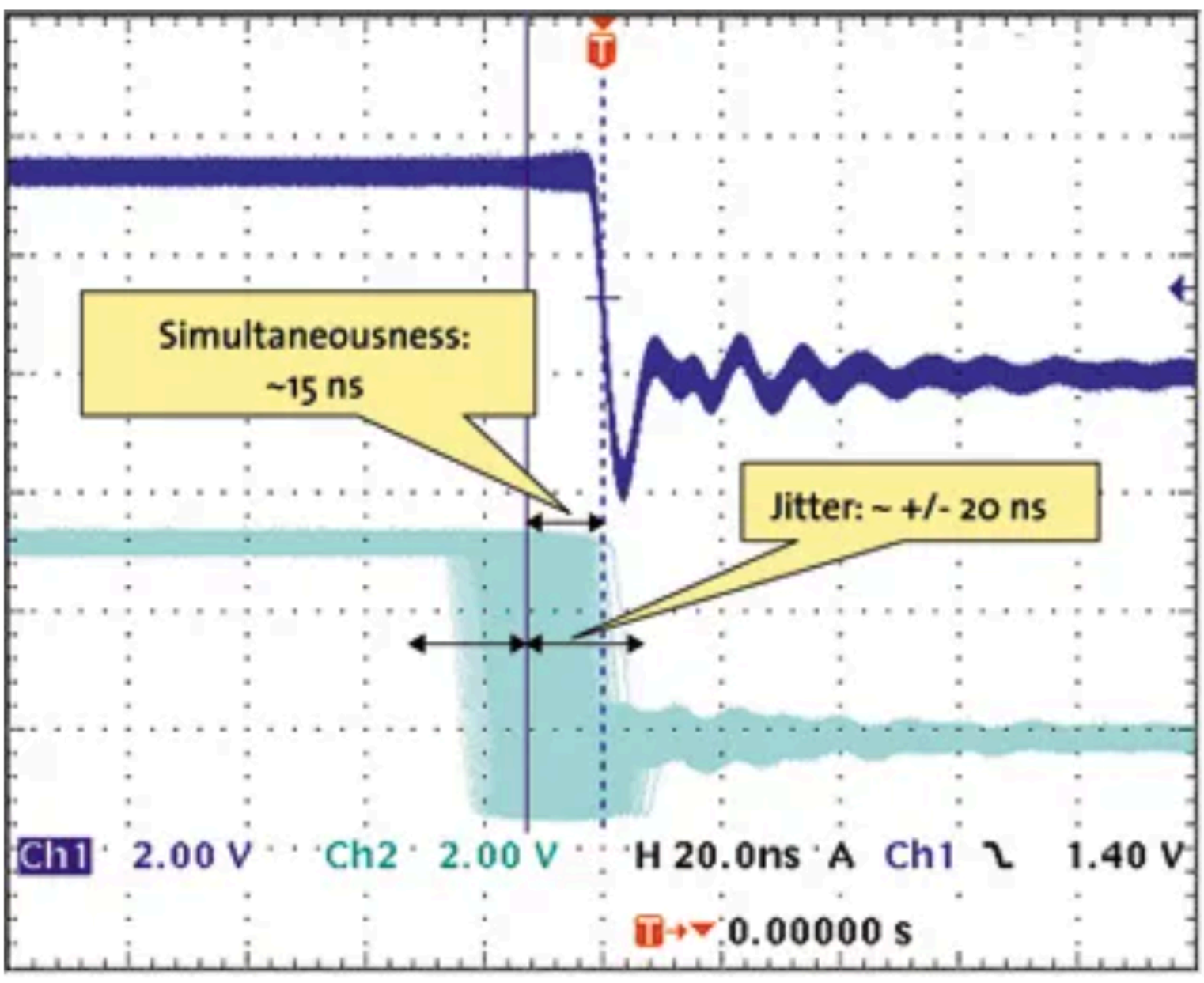
与立即受到通信误差影响的完全同步通信相比，分布式同步时钟对系统抖动有高容忍度。因此，EtherCAT的节点同步方案基于此分布式时钟（DC）。



Distributed clocks in the EtherCAT system

节点中时钟的校准完全基于硬件。从第一个DC从设备开始的时间被循环地分配给系统中的所有其他设备。通过这种机制，从设备时钟可以精确地调整到该参考时钟。

从参考时钟发送到从设备的时间略有延迟，因此必须测量并补偿每个从设备的传播延迟，以确保同步性和同时性。这种延迟是在网络启动期间测量的，如果需要，甚至是在操作期间连续测量的，以确保时钟在彼此小于1μs的范围内同步。



- *上图：同步性与一致性：相距电缆长度为有120米的两个分布系统，带有300个节点的示波器比较*
- 有图可以看出300个示波器的波形合并成1个波形， 波形误差在20ns之间， 相当的低了。

此外，高分辨率分布时钟还可提供数据采集的本地精确时间。采样极短时，微小同步偏差即致速度计算大阶跃。例如，运动控制器通过顺序检测的位置计算速度便是如此。

EtherCAT引入时间戳，链接系统时间与测量值，速度计算摆脱同步误差，其精度要高于基于自由同步误差的通讯测量技术。

4. 分布式时钟优势

EtherCAT的分布式时钟提供了几个优势，使其成为工业自动化的首选。

EtherCAT分布式时钟实现的紧密同步，提高了控制精度、准确性和整体系统性能。由于所有设备的时间都是相同的，运动控制系统可以以最小的误差执行协调的运动。

EtherCAT的分布式时钟提供了一种经济高效的解决方案。与其他需要额外硬件进行时间同步的协议不同，EtherCAT使用低成本的EtherCAT ESC芯片实现时间同步。

这些芯片集成了分布式时钟所需的多种功能，无需专门的卡或复杂的软件堆栈。

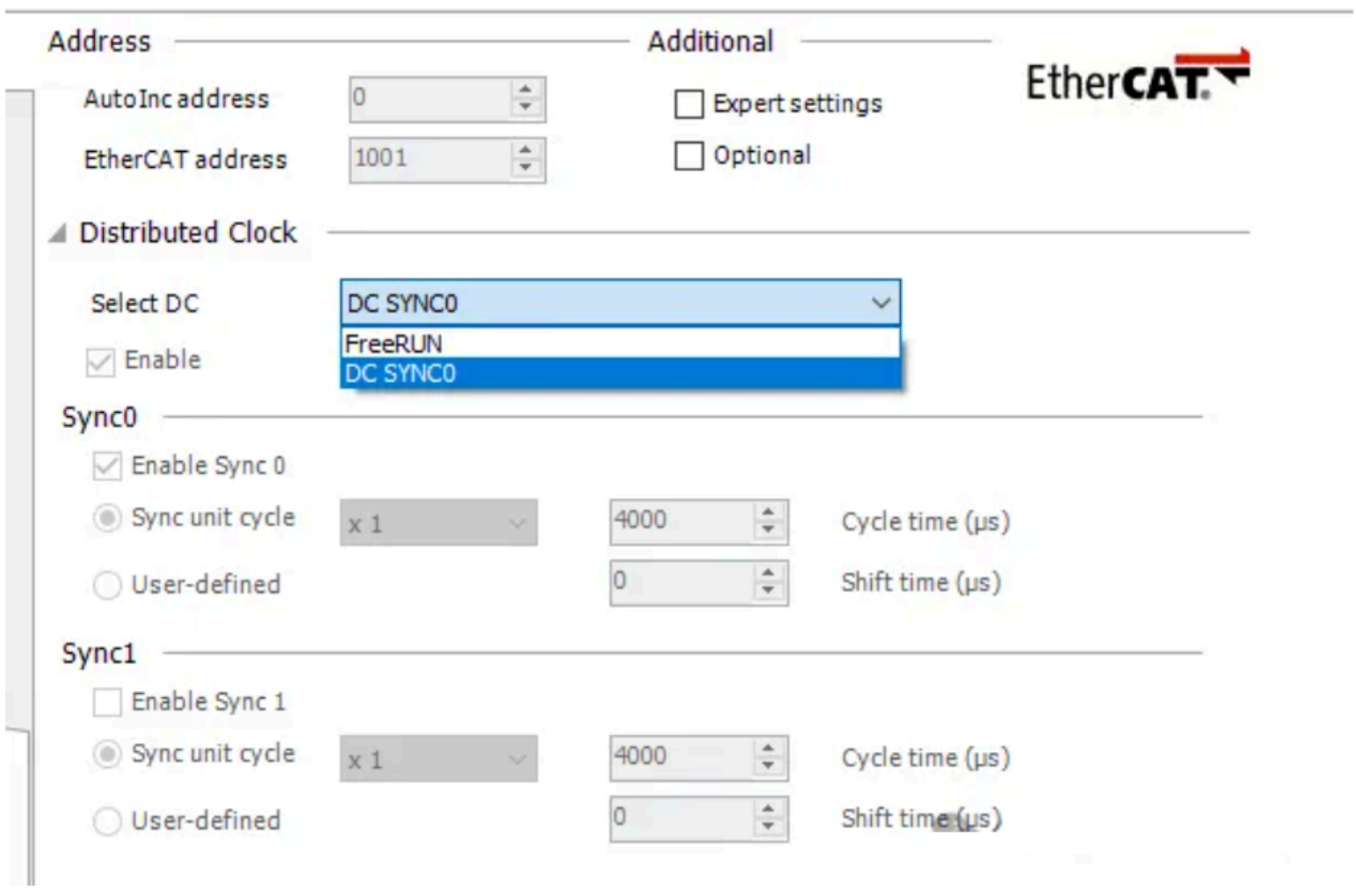
5. 从站同步模式

Ether CAT从站主要有以下同步模式：

- **（Free Run）：自由运行**
  - EtherCAT从站与EtherCAT不同步。从设备根据自己的周期自主运行，不与EtherCAT周期同步。
- **（Synchronous with SM event ）：与SM事件同步**
  - EtherCAT从设备与SyncManager 2（SM2）事件（如果传输周期性输出）或SM3事件（如果仅传输周期性输入）同步。处理经过的帧时，SyncManager会触发SM2/SM3事件。
- **（Synchronous with SYNC event (distributed clocks)）：与SYNC事件（分布式时钟）同步**
  - EtherCAT从机与分布式时钟系统的SYNC0或SYNC1事件同步。

同步模式所有参数都列为EtherCAT从站设备CoE列表中的对象。可由从设备在线读取，或通过描述从设备的XML文件离线确定。

例如，在CODESYS平台控制器中，设定伺服的同步模式，有些伺服默认是不会选择DC模式，这个时候需要自己在图示处选择：



实际测试发现，部分设备伺服运行有异常抖动，无论怎么设定其他参数都无效，而将DC项目设为该模式DC Sync0即可解决。

19.4 EtherCAT性能

1. 基于以太网的通讯协议

以太网无处不在，并且具有成本效益，它采用公共物理链路且速度更快。正因如此，多种工业通信协议正转移到基于以太网的解决方案上。支持 TCP/IP 的以太网通信通常具有不确定性，反应时间通常约为 100ms。

工业以太网协议使用经过修改的介质访问控制 (MAC) 层来实现非常低的延迟和确定性响应。

我们来详细了解一些流行的工业以太网通信协议。

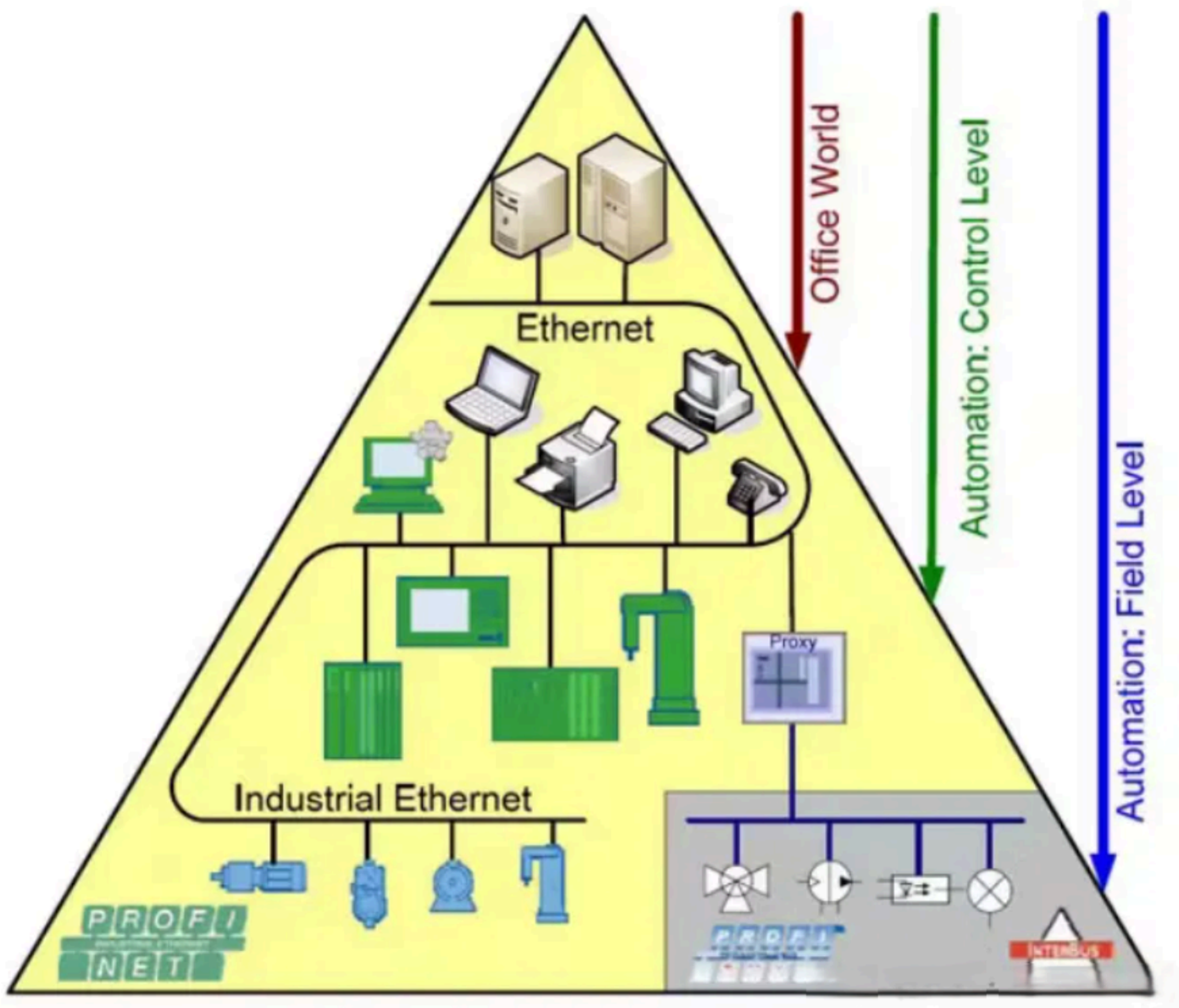
主要有以下几种：

- EtherCAT

- Powerlink
- Ethernet/Internet Protocol (IP)
- Modbus/Transmission Control Protocol (TCP)
- ProfiNet

2. 以太网不同类型对比

以太网在工厂的不同层级都有使用，例如（自动化现场层、控制层、办公应用层），如下图所示



3. EtherCAT性能

借助于从站硬件集成和网络控制器主站的直接内存存取，整个协议的处理过程都在硬件中得以实现，因此，完全独立于协议堆栈的实时运行系统、CPU 性能或软件实现方式。1000个I/O的更新时间只需30 μs，其中还包括I/O周期时间（参见下图）

Process Data	Update Time
256 distributed digital I/O	11 μs = 0,01 ms
1000 distributed digital I/O	30 μs
200 analog I/O (16 bit)	50μs ↔ 20 kHz
100 Servo Axis, with 8 Bytes input and output data each	100 μs
1 Fieldbus Master-Gateway (1486 Bytes Input and 1486 Bytes Output Data)	150 μs

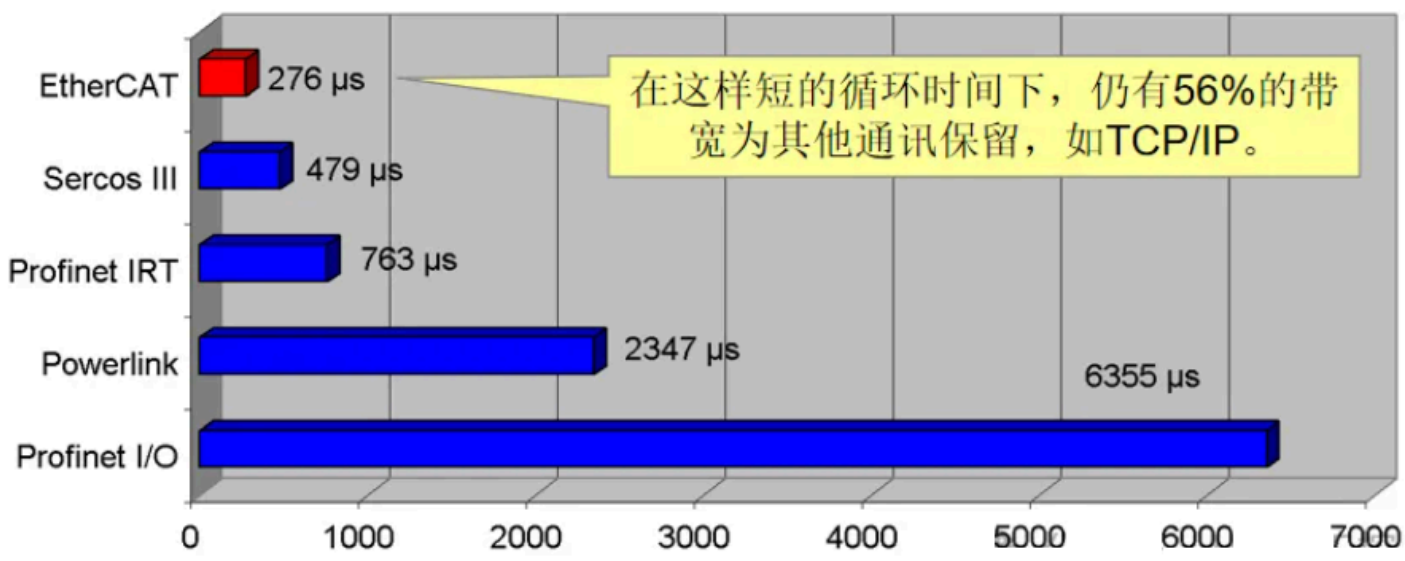
100个伺服轴的通讯也非常快速：可在每100μs中更新带有命令值和控制数据的所有轴的实际位置及状态，分布时钟技术使轴的同步偏差小于1微秒。而即使是在保证这种性能的情况下，带宽仍足以实现异步通讯，如TCP/IP、下载参数或上载诊断数据。

超高性能的EtherCAT技术可以实现传统的现场总线系统无法迄及的控制理念。EtherCAT使通讯技术和现代工业PC所具有的超强计算能力相适应，总线系统不再是控制理念的瓶颈，分布式I/O可能比大多数本地I/O接口运行速度更快。

EtherCAT技术原理具有可塑性，并不束缚于100Mbps的通讯速率，甚至有可能扩展为1000Mbps的以太网。

得益于EtherCAT的On-the-fly运行机制和独特的EtherCAT通讯帧结构，他的性能相比较而言，目前是最优的：

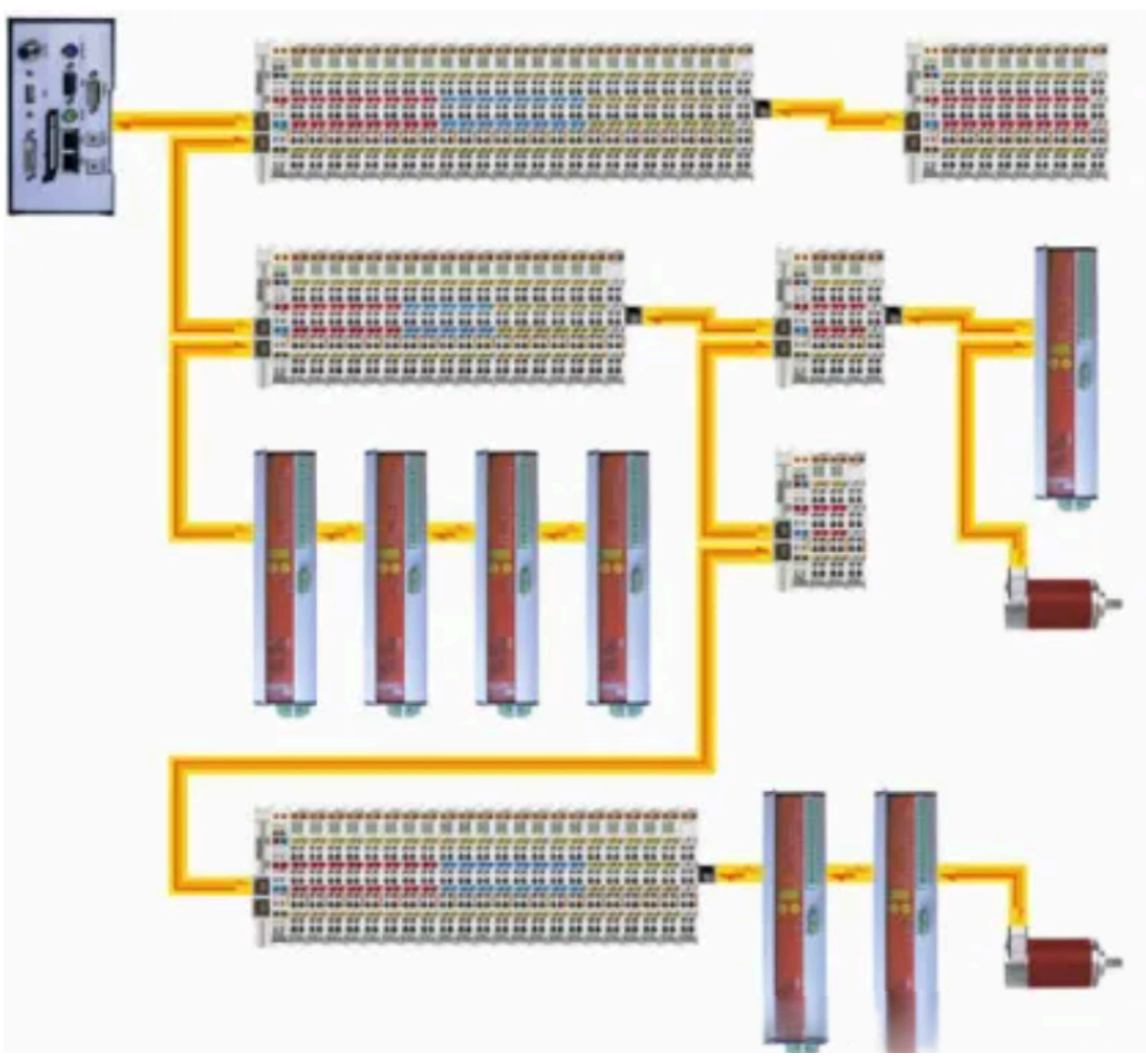
- 40 轴 (每轴 20 Byte 输入和输出数据)
- 50 I/O 站，共计560个EtherCAT总线端子模块
- 2000 个数字量 + 200个模拟量, 总线长度500米
- EtherCAT：循环时间 = 276 μs
  总线负载44%，报文长度 = 122 μs



19.5 EtherCAT拓扑结构

1. 拓扑结构

EtherCAT几乎支持所有的拓扑结构：线型、树型、星型、菊花链型。EtherCAT 使得带有成百上千个节点的纯总线型或线性拓扑结构成为可能，而不受限于级联交换机或集线器。

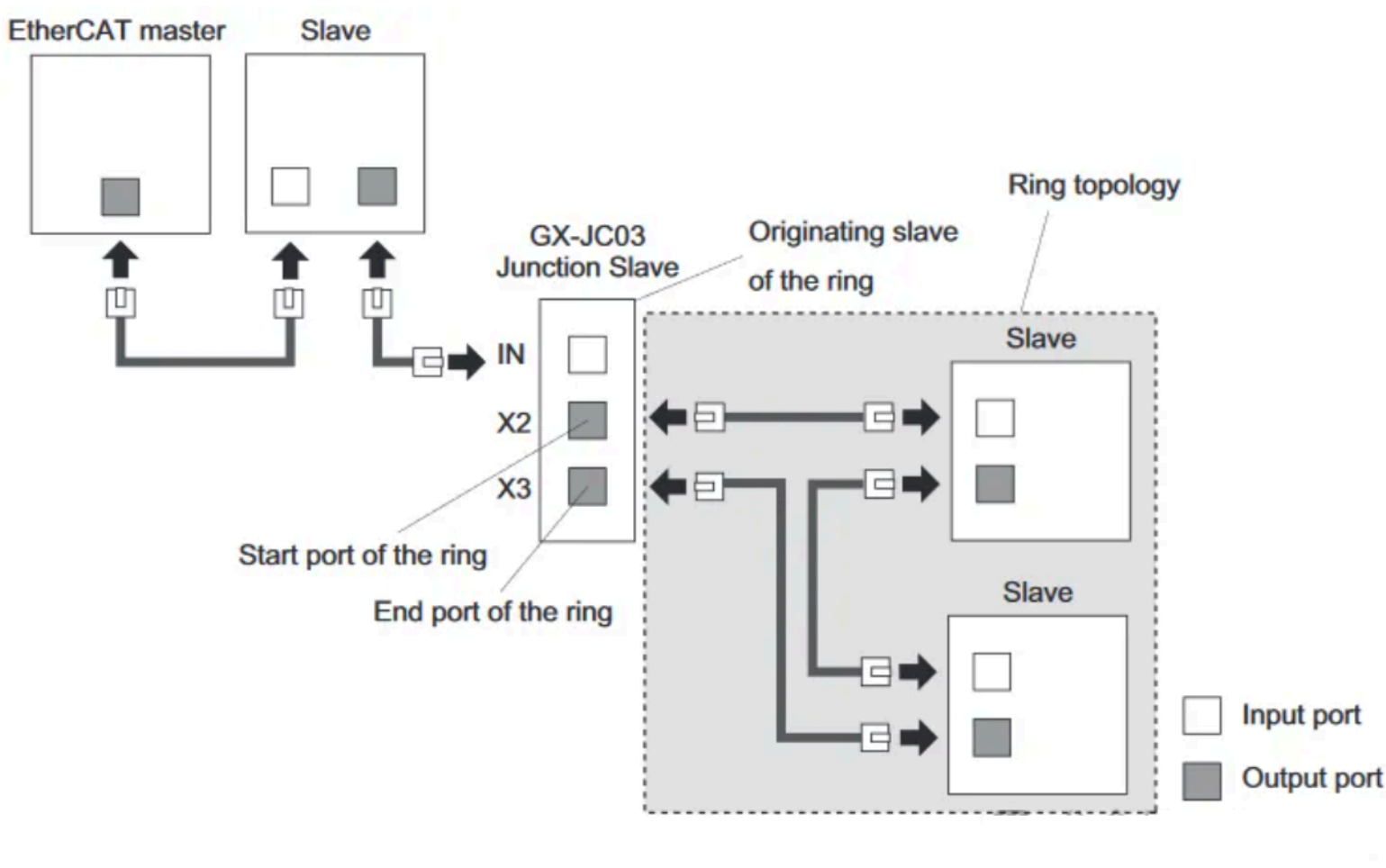


EtherCAT 网络可连接多达65535个设备，网络容量几乎没有限制。由于实际上节点数量没有限制，可以将模块化的 I/O 设备设计为每个 I/O 片都是一个独立的EtherCAT从站。因此无需本地扩展总线，高性能的EtherCAT 能直达每个模块。因为在总线耦合器上无需网关，所以没有任何延时。

2. 硬件支持

常规的线性拓扑结构都容易实现，但是星形的拓扑没有合适的Junction支持是比较难实现的。

比如倍福，提供类似的接口模块，这些型号主要包括：EK1122,EK1521,EP1122还有CU112x.等。

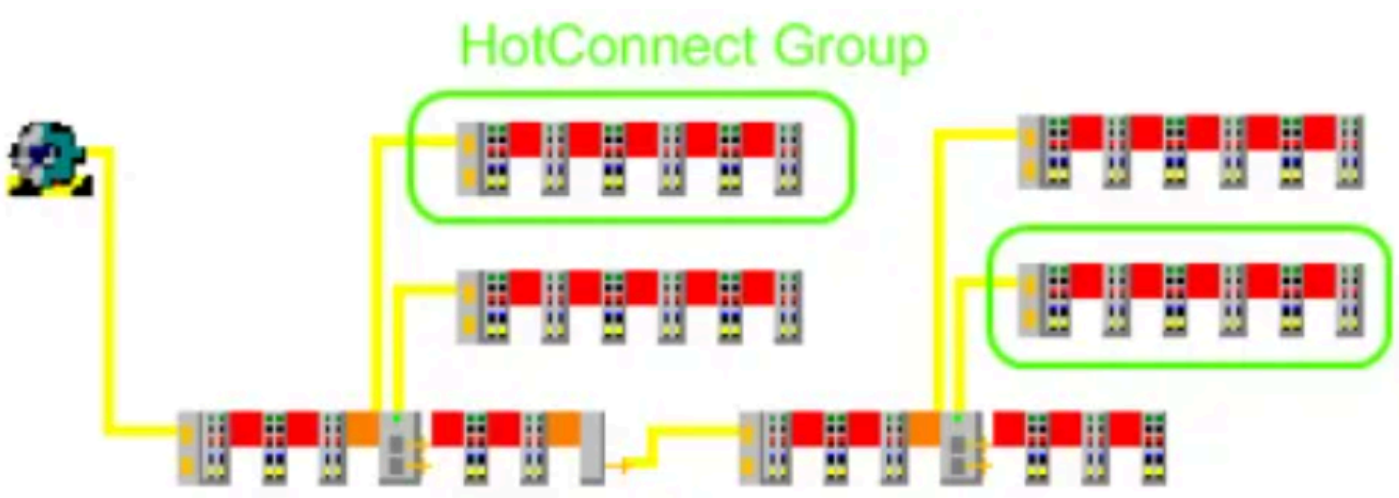


例如上图中的**环形结构**需要GX-JC03这样的从站（IN、X2、X3），才可以方便实现环形拓扑结构。需要注意的是三个接口已经预定义，不能随便接入，即使都是常规的RJ45接口。

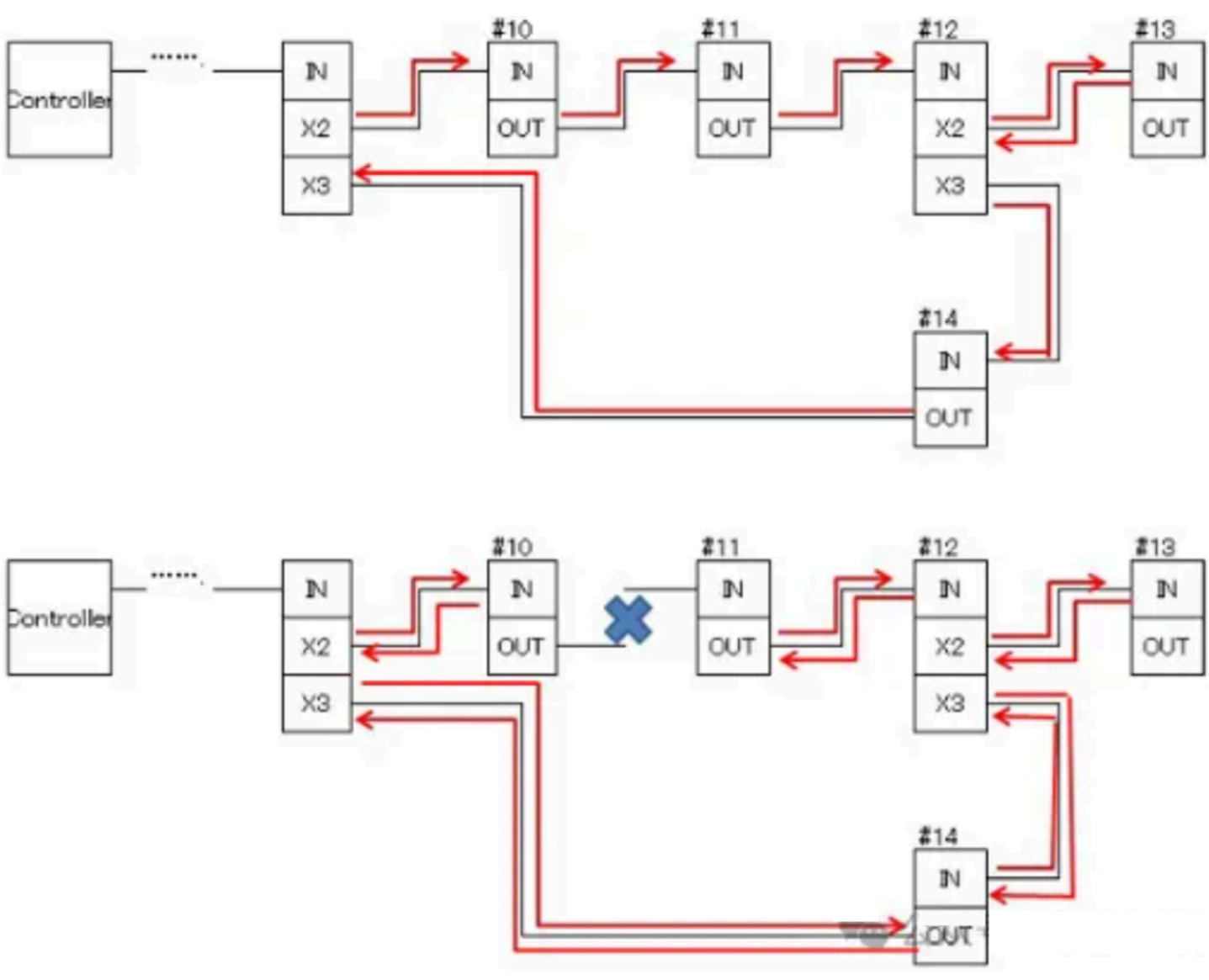
3. 热连接(Hot Connect)和冗余

EtherCAT热连接技术使得拓扑更加灵活，对于热连接的组，就算移除掉（在启动前或系统运行中），不会导致整体进入异常。这样方便我们对设备进行模块化设计，按照实际需要进行从站的配置。

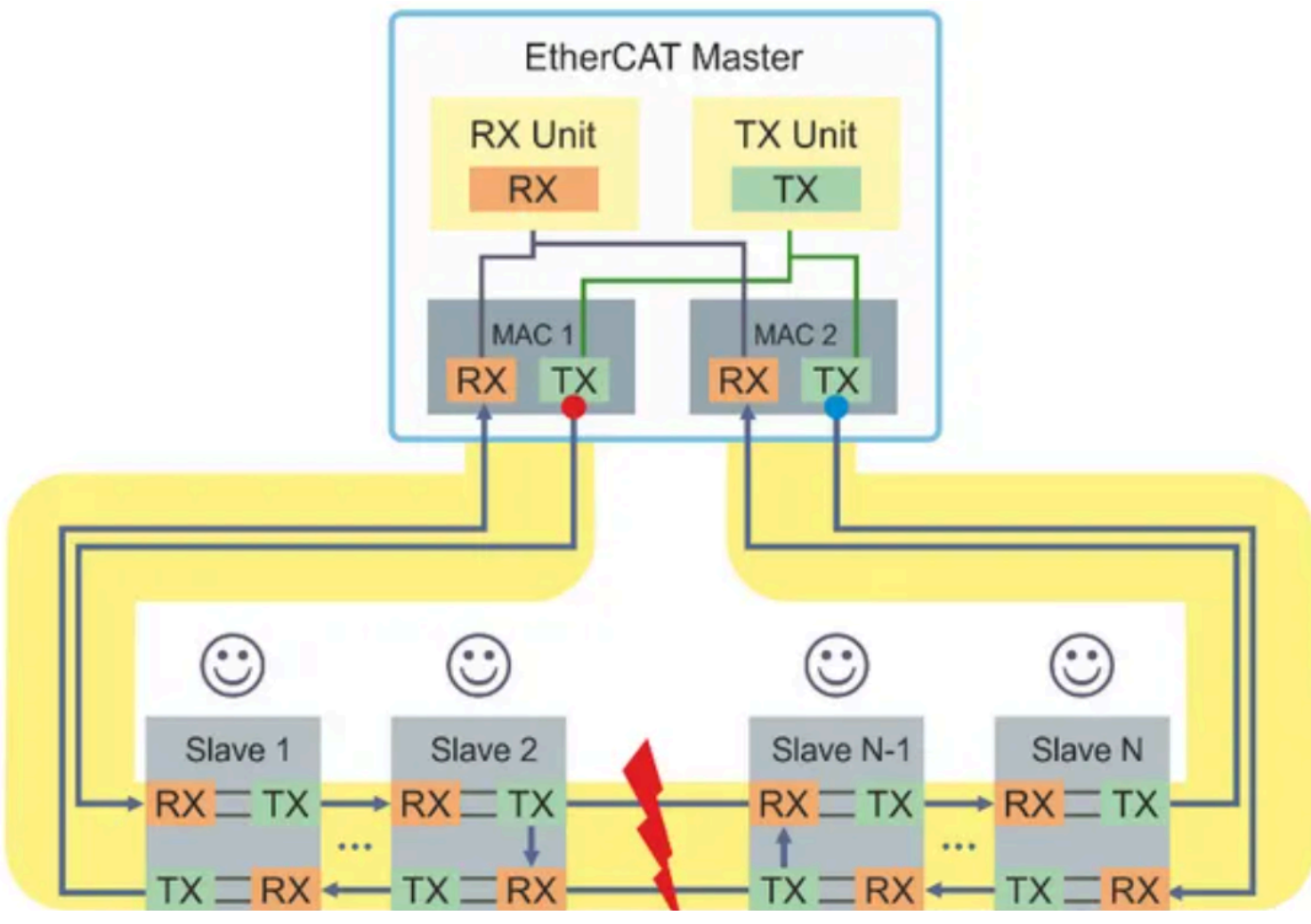
例如，我们某款设备常规6个伺服从站，但是也有可能5个，把其中另一个设为热连接后，就可以直接移除，设备整体运行不受影响。



在线形结构中，如果两个EtherCAT从站之间线缆断开或者送掉后，有可能导致EtherCAT网络故障，设备无法进入正常运行模式。



但是，如果有环形拓扑结构及线缆冗余技术，设备运行过程中，突然断线或者失去连接，完全不会影响整体的设备运行，这对于一些稳定要求性高的设备和产线而言，此结构非常有必要。

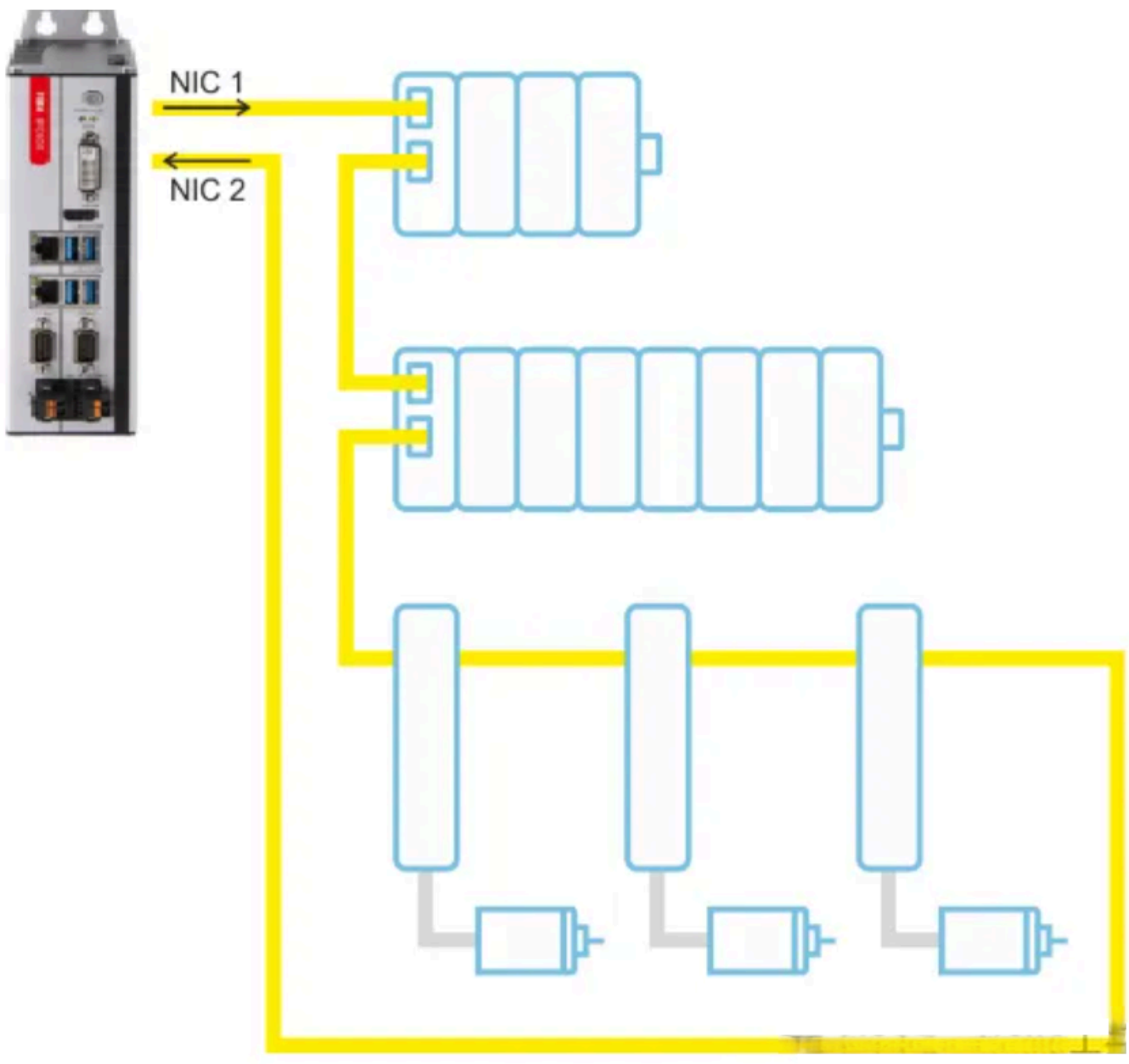


当然，这种是从站设备之间通过EtherCAT Junction 实现的环形连接，主站也可以设置冗余。

19.6 EtherCAT总线冗余

1. 线缆冗余 Cable Redundancy

线缆冗余模式下的EtherCAT®总线系统可以在电缆中断或节点故障的情况下保持通信。基于上期说的环形拓扑，如果环在某个点上中断，两个分支仍然可以到达。

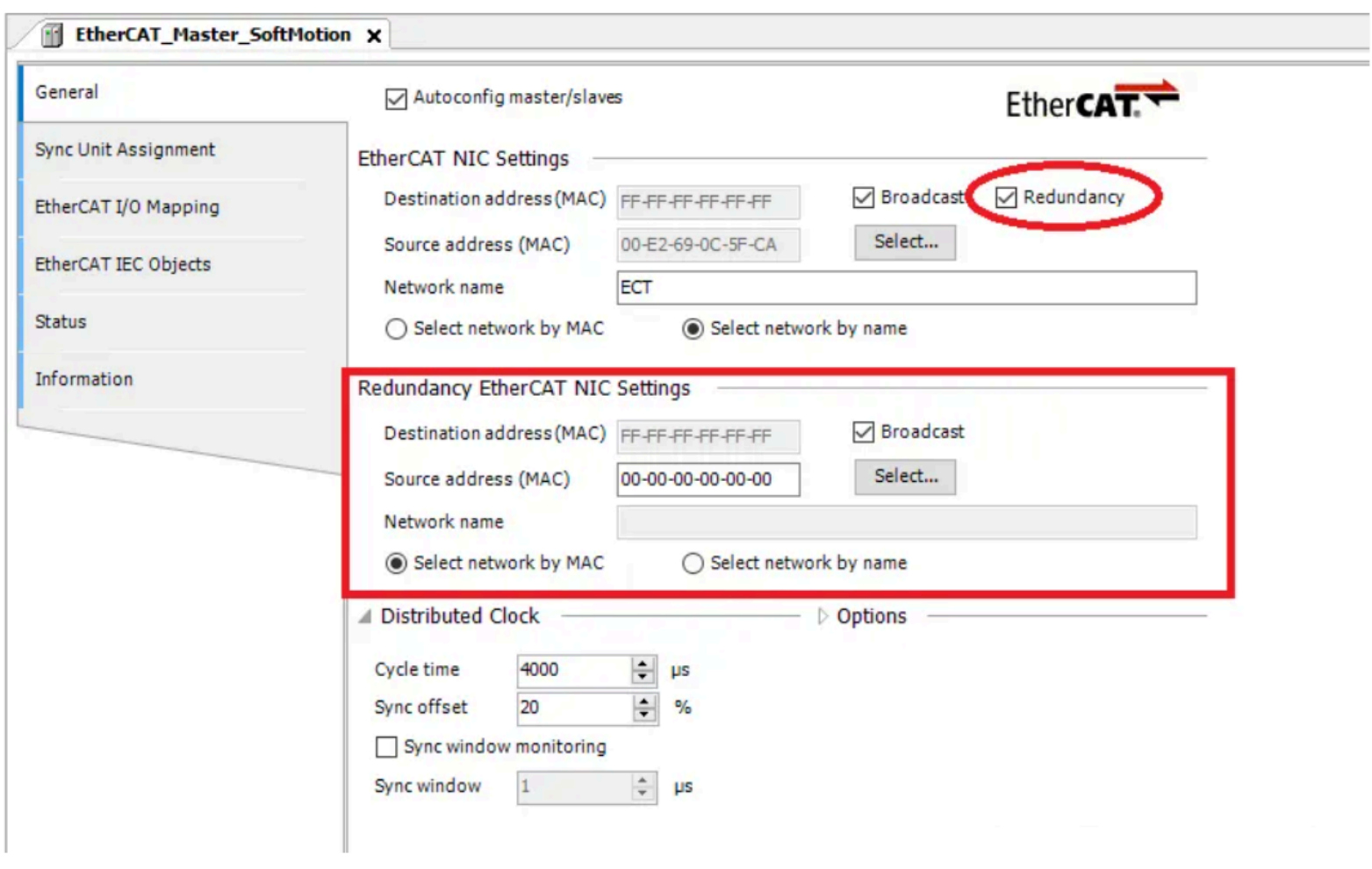


所以我们的主站至少需要两个EtherCAT通讯接口。如上图控制器，我们基于CODESYS平台开发的控制器，可以实现多个EtherCAT通讯，且可以配置成冗余模式。

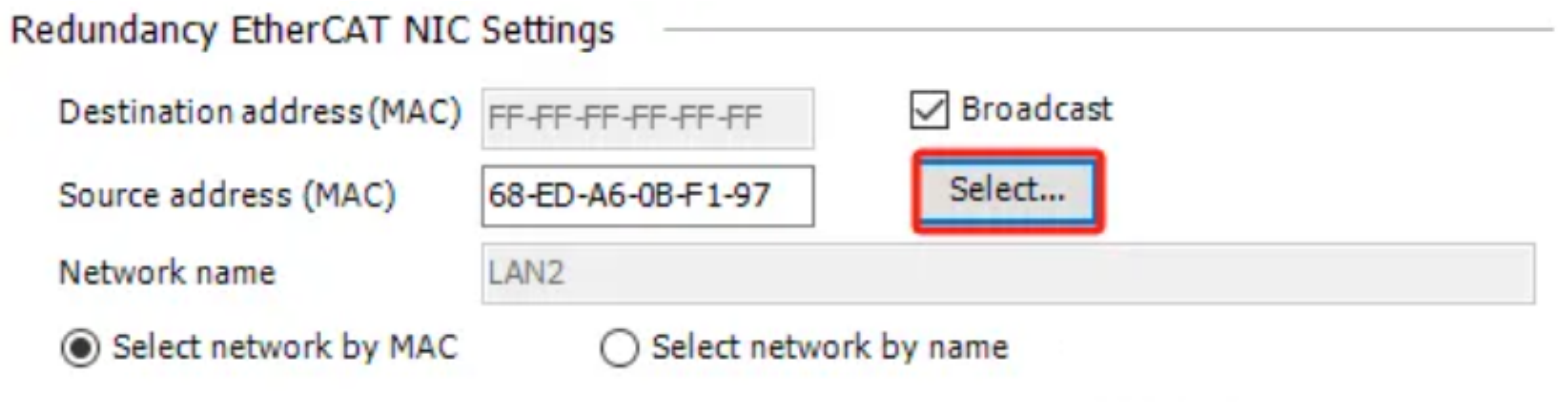
2. 冗余配置 Redundancy

首先，控制器至少需要两个EtherCAT通讯口，当然我们可以通过普通的网口进行驱动配置。

我们打开CODESYS IDE编程软件进行配置：



打开后，先**添加EtherCAT_Master_Softmotion**这种类型，然后在上图的**Recundancy冗余这个选项进行勾选。**这个时候，下方就会多出红色方框的Redundancy EtherCAT NIC Settings选项。



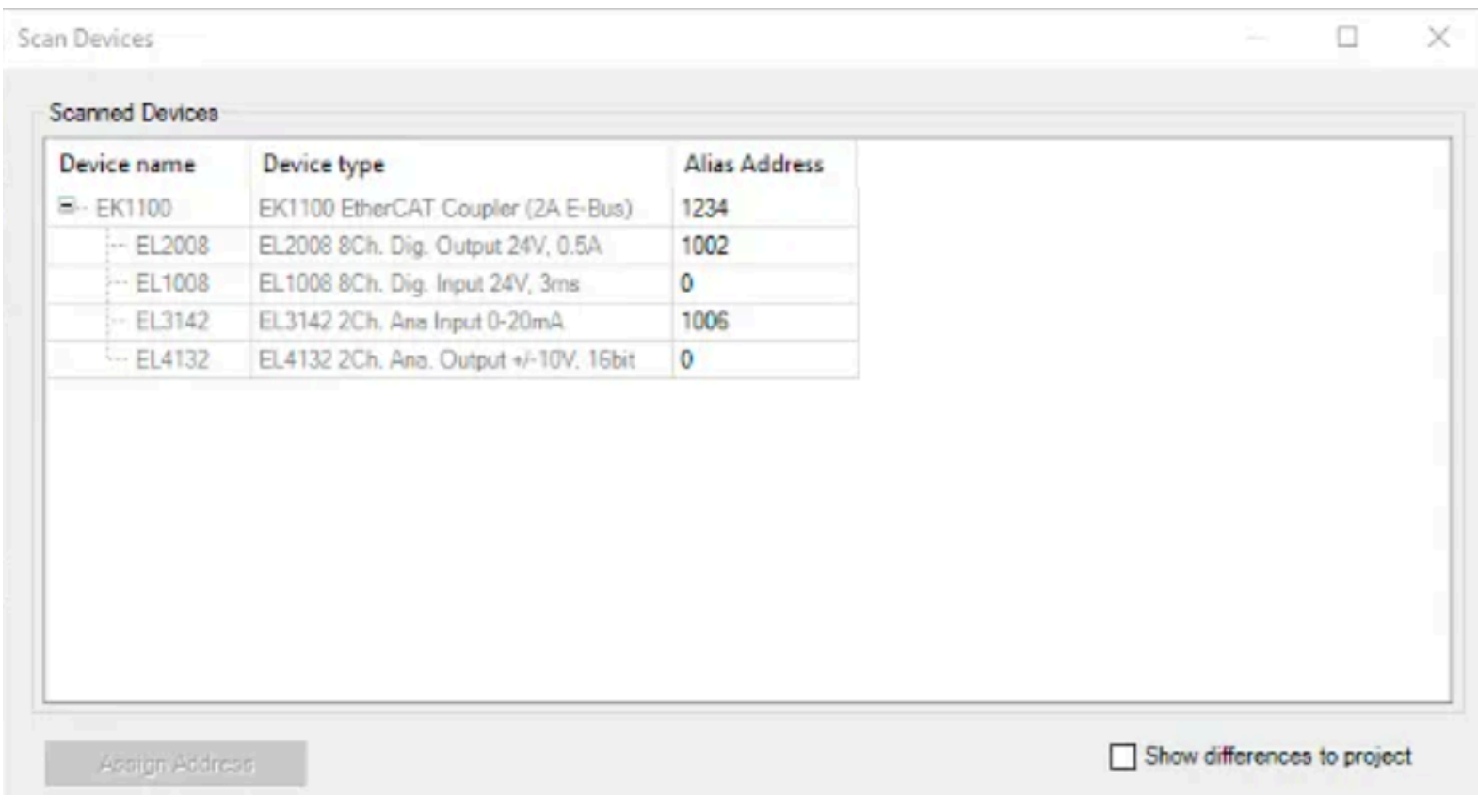
再次选择Select..在弹出的对话框里**选中第二个EtherCAT网卡**即可，最后**按照环形拓扑结构进行EtherCAT网络连接**即可。

19.7 EtherCAT配置简单

1. 拓扑结构

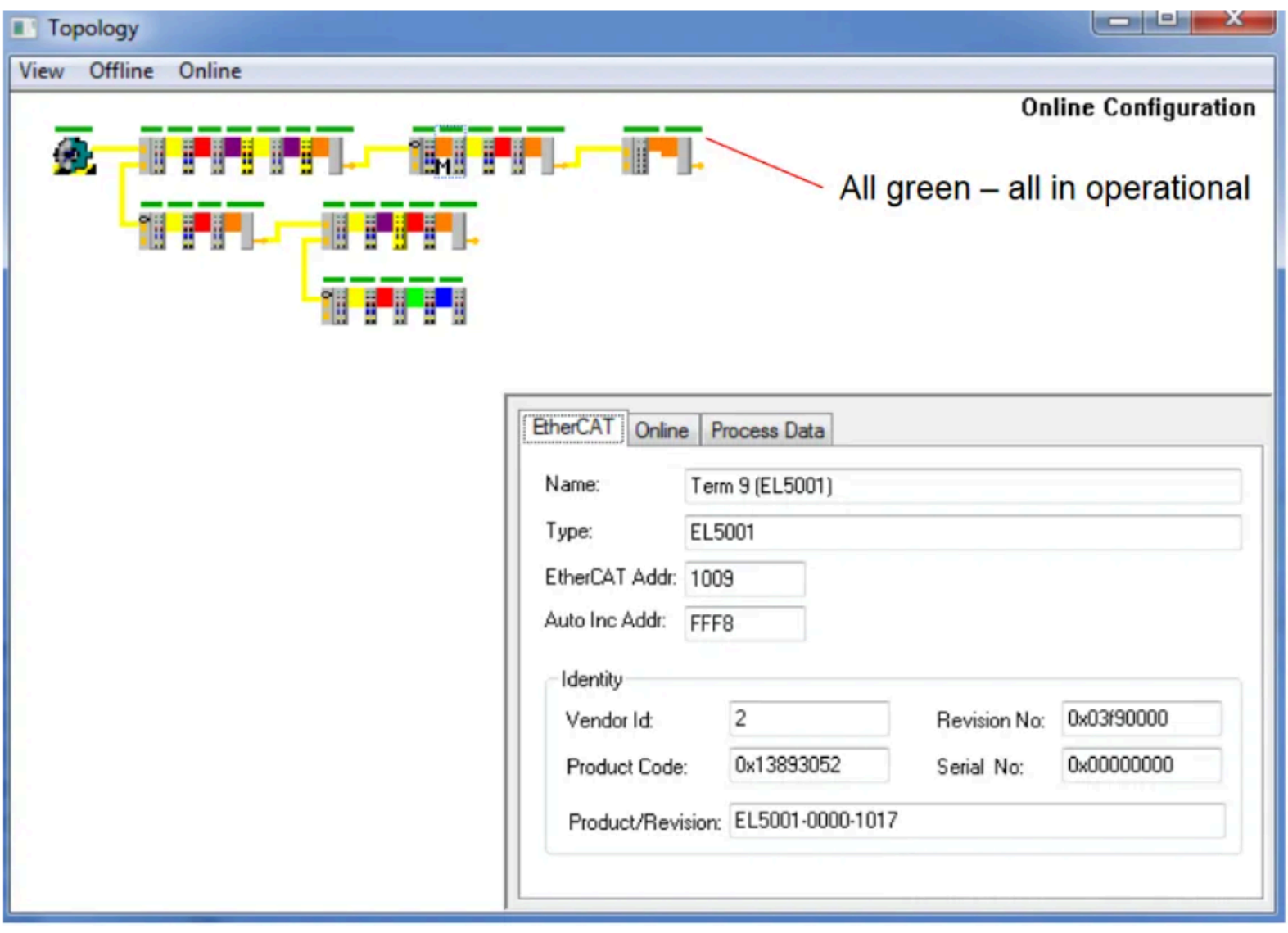
我们已经在前期介绍过EtherCAT总线的拓扑形式，几乎包含所有的拓扑类型。所以对于工业上的多么复杂的网络结构都能轻松应对。

而EtherCAT网络可以自动进行扫描，无需手动一个个的添加，方便快捷，例如在CODESYS平台添加设备，直接scan device：



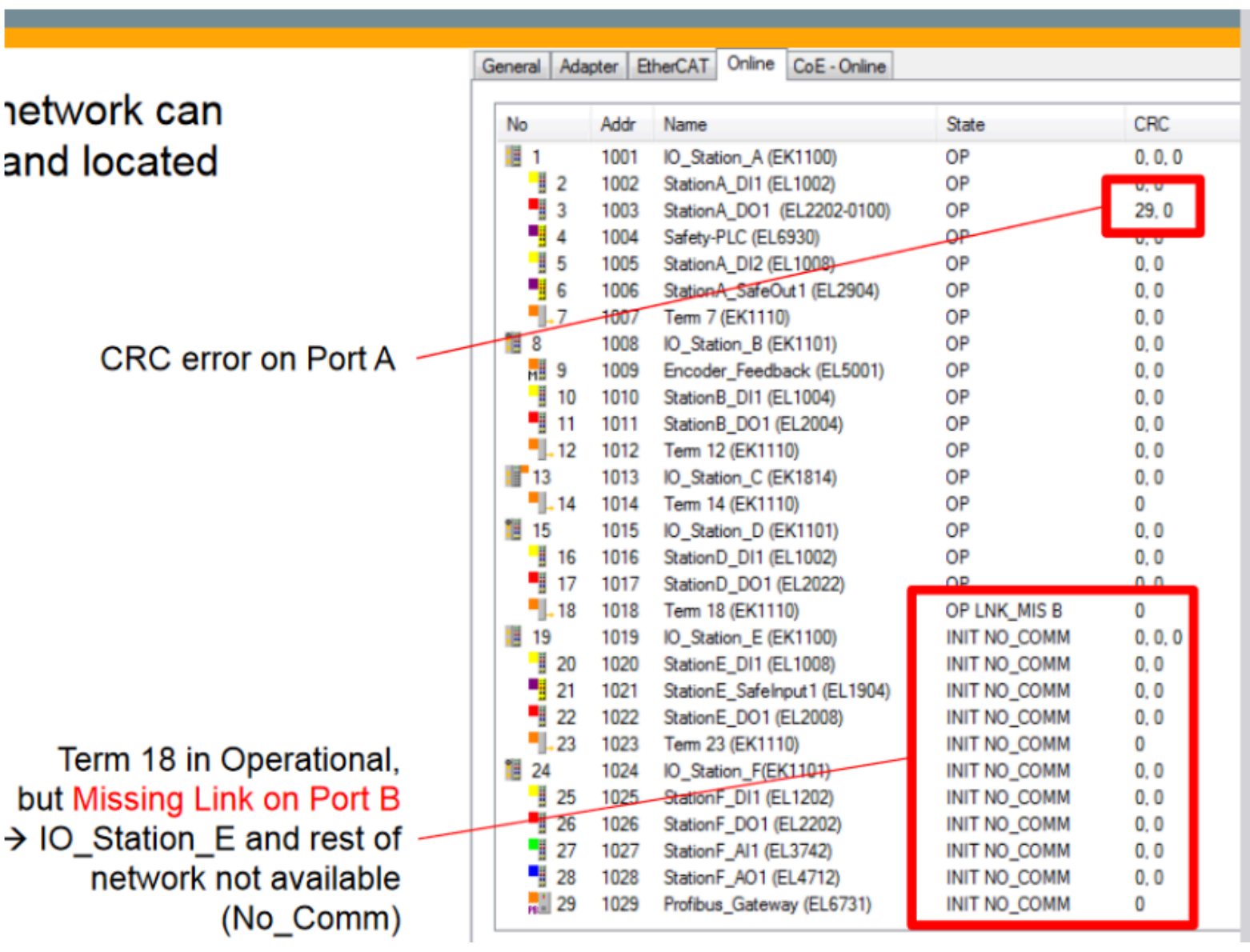
上面确定无误后，直接Copy All Devices to Project即可。需要注意的是，已经有安装好设备的XML描述文件，这样才能识别成功。

也可以对扫描的网络和实际的设备进行自动对比：

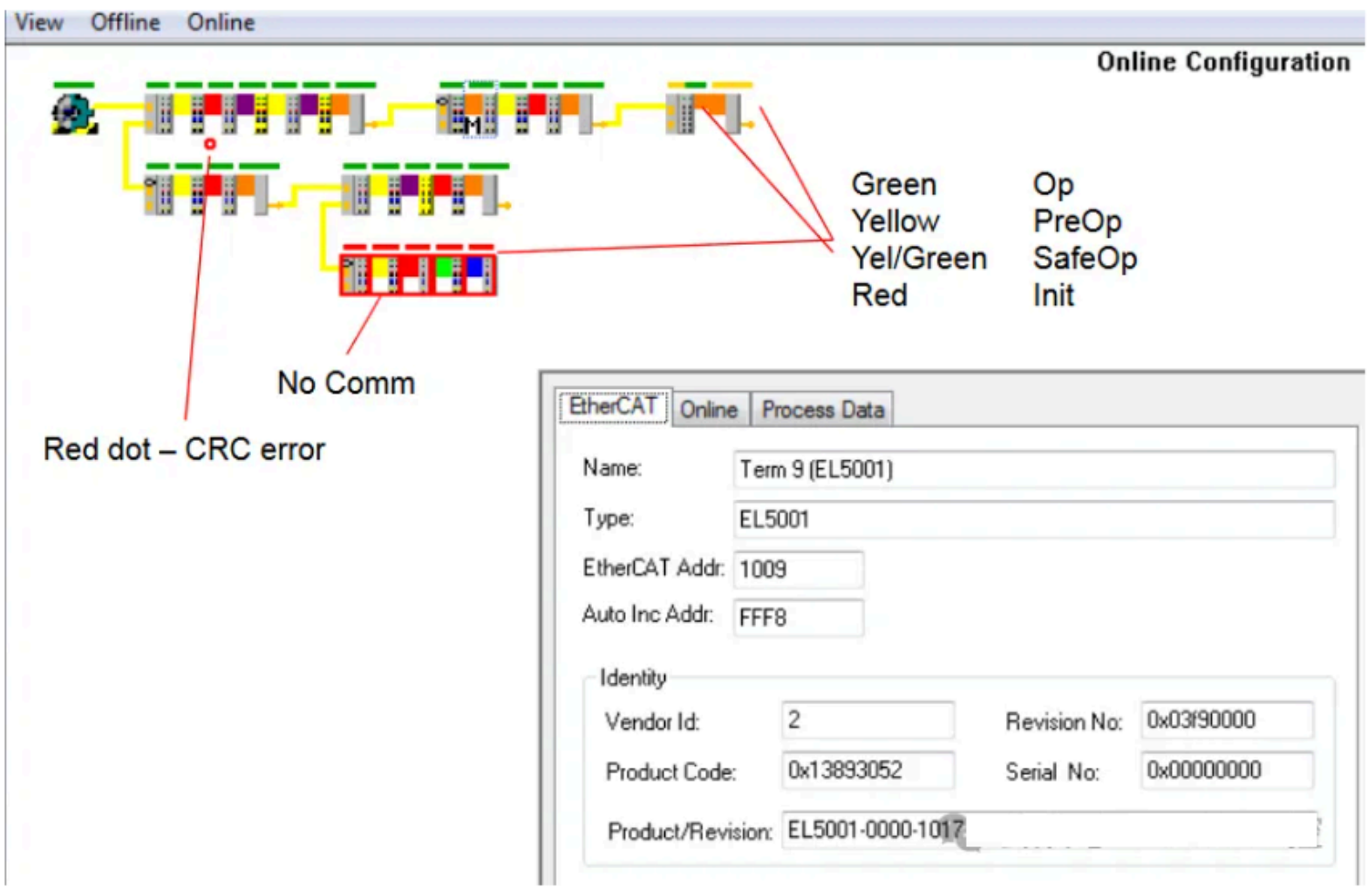


2. 精确诊断

当某个设备发生故障时，可以精确定位到具体节点。



当然，通过网络结构Online在线查看，可图示化定位具体的位置：

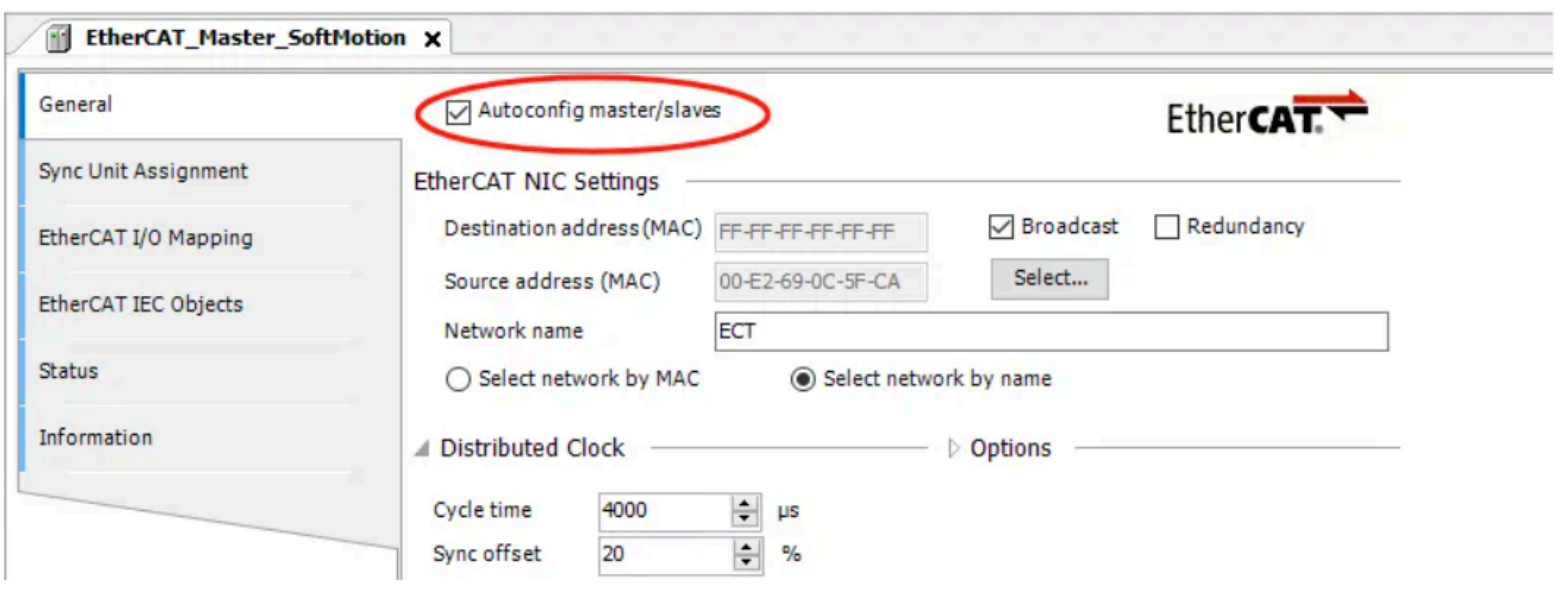


基于这种方式就可以精确定位到故障节点位置，还有可能发生故障的具体信息，方便我们进行故障的排除和解决。

3. 无需寻址

在其他一些网络配置过程中，老需要对设备进行拨码或者设定地址，而在EtherCAT总线中无需手动寻址，自动完成，方便快捷。

例如，在CODESYS平台添加EtherCAT主站，寻址的选项默认都是自动的方式：



所以，在使用EtherCAT总线技术，寻址是无需考虑的事项。

19.8 EtherCAT线缆标准

1. 线缆标准

我们已经在前期介绍过EtherCAT总线的线缆标准，这里我们再介绍下：

Ethernet/EtherCAT		Plug connector			Cable		Standard
Signal	Description	M8	M12	RJ45[1]	ZB9010, ZB9020, ZB9030, ZB9032, ZK1090-6292, ZK1090-3xxx-xxxx	ZB9031 and old versions of ZB9030, ZB9032, ZK1090-3xxx-xxxx	TIA-568B
Tx +	Transmit Data+	Pin 1	Pin 1	Pin 1	yellow[2]	orange/white[3]	white/orange
Tx -	Transmit Data-	Pin 4	Pin 3	Pin 2	orange[2]	orange[3]	orange
Rx +	Receive Data+	Pin 2	Pin 2	Pin 3	white[2]	blue/white[3]	white/green
Rx -	Receive Data-	Pin 3	Pin 4	Pin 6	blue[2]	blue[3]	green
Shield	Shield	Housing		Shroud	Screen	Screen	Screen

- 总结和普通网线一样

2. 平行线还是交叉线

一般在EtherNet以太网接线有两种形式平行（直连）和交叉：

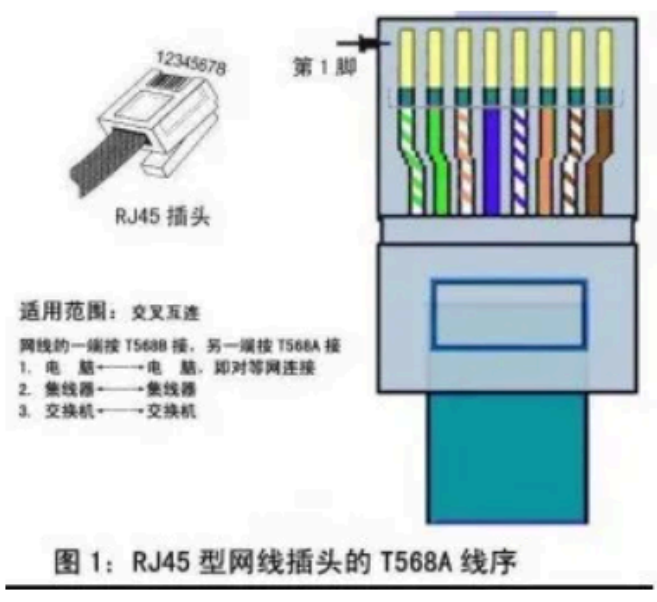


图 1：RJ45 型网线插头的 T568A 线序

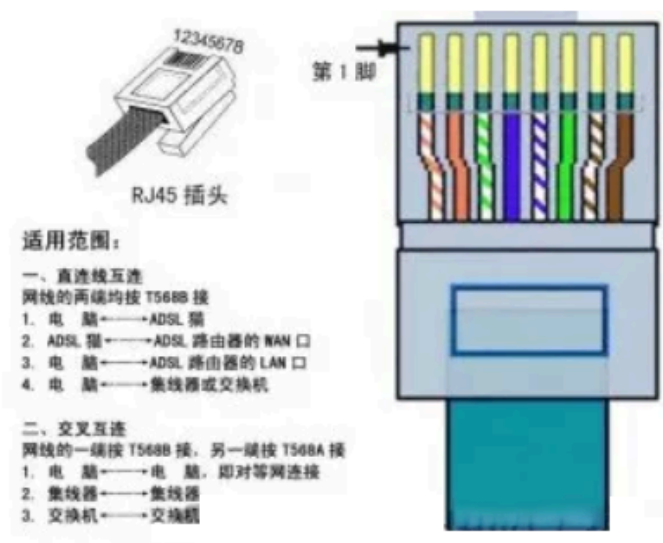
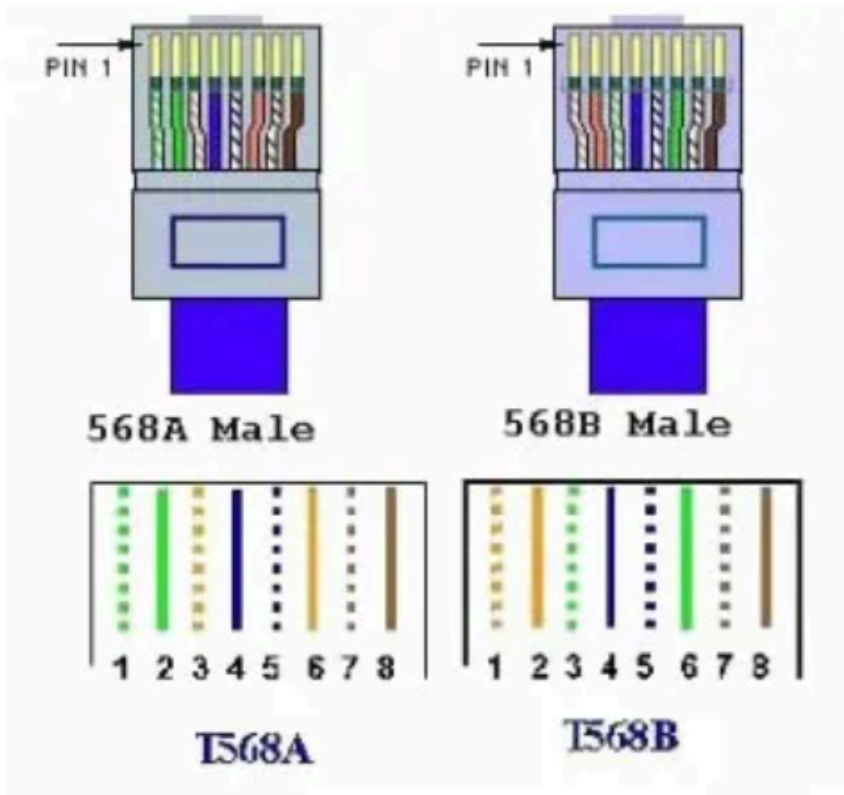


图 2：RJ45 型网线插头的 T568B 线序

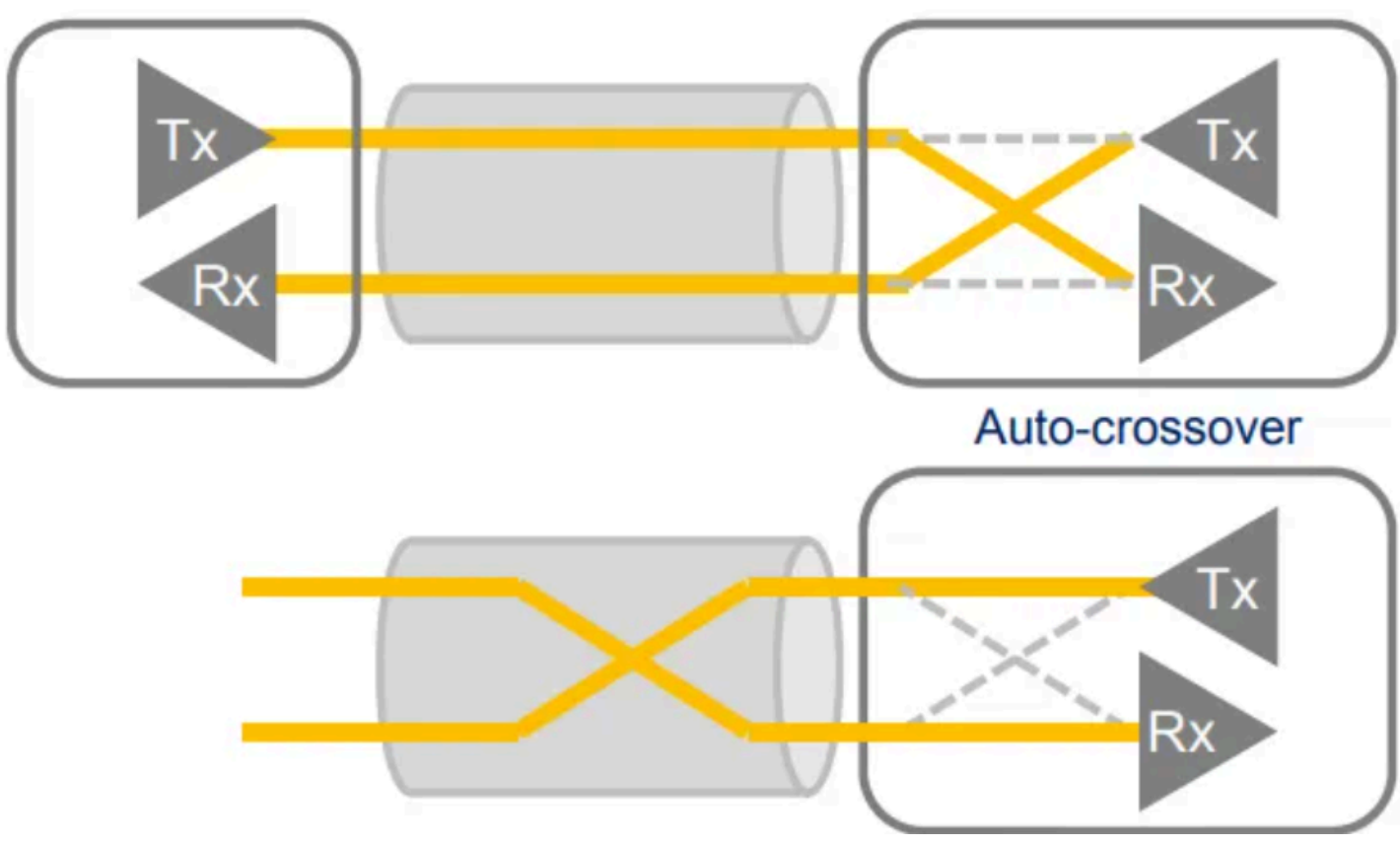
如果是自己制作电缆，可以按照下面方式进行：



标准568A：绿白–1，绿–2，橙白–3，蓝–4，蓝白–5，橙–6，棕白–7，棕–8

标准568B：橙白–1，橙–2，绿白–3，蓝–4，蓝白–5，绿–6，棕白–7，棕–8

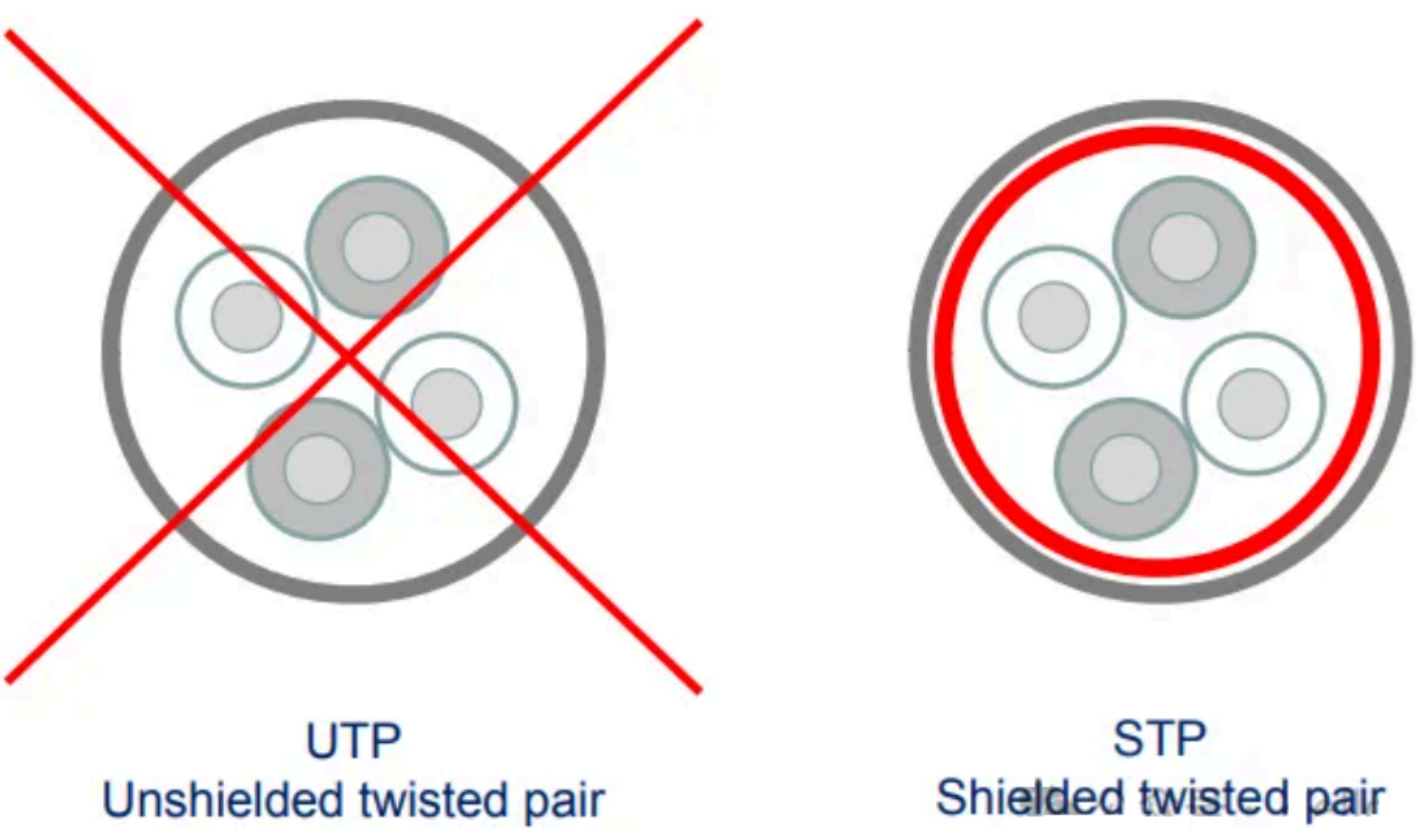
一般地，同种设备用交叉线，异种设备用直连线。比方说，都是网卡或交换机，就用交叉线，如果是网卡连交换机，就用直连线。



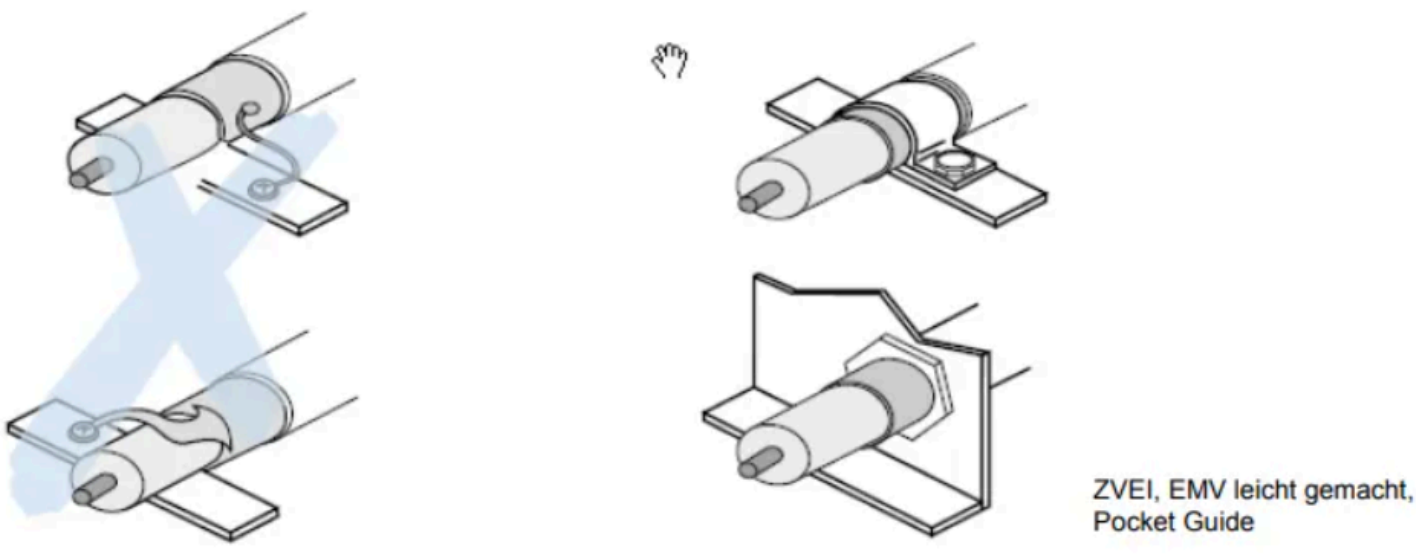
而EtherCAT总线不管是平行还是交叉，都可以，任由你选择。

需要重点说明的是屏蔽和接地的问题，网络的稳定性很关键：

第一：屏蔽双绞线（STP）



第二：可靠接地（保证电缆两端屏蔽层大面积接触且低阻抗导体连接接地）



3. 硬件接口

按照常规的硬件接口来说，推荐下图几种：

RJ45 connector, 4-pin,
IP 20,
for field-assembly

M12 connector, 4-pin,
d-coded, IP67,
for field-assembly

M12 socket on RJ45
socket, straight outlet

19.9 EtherCAT取代PCI

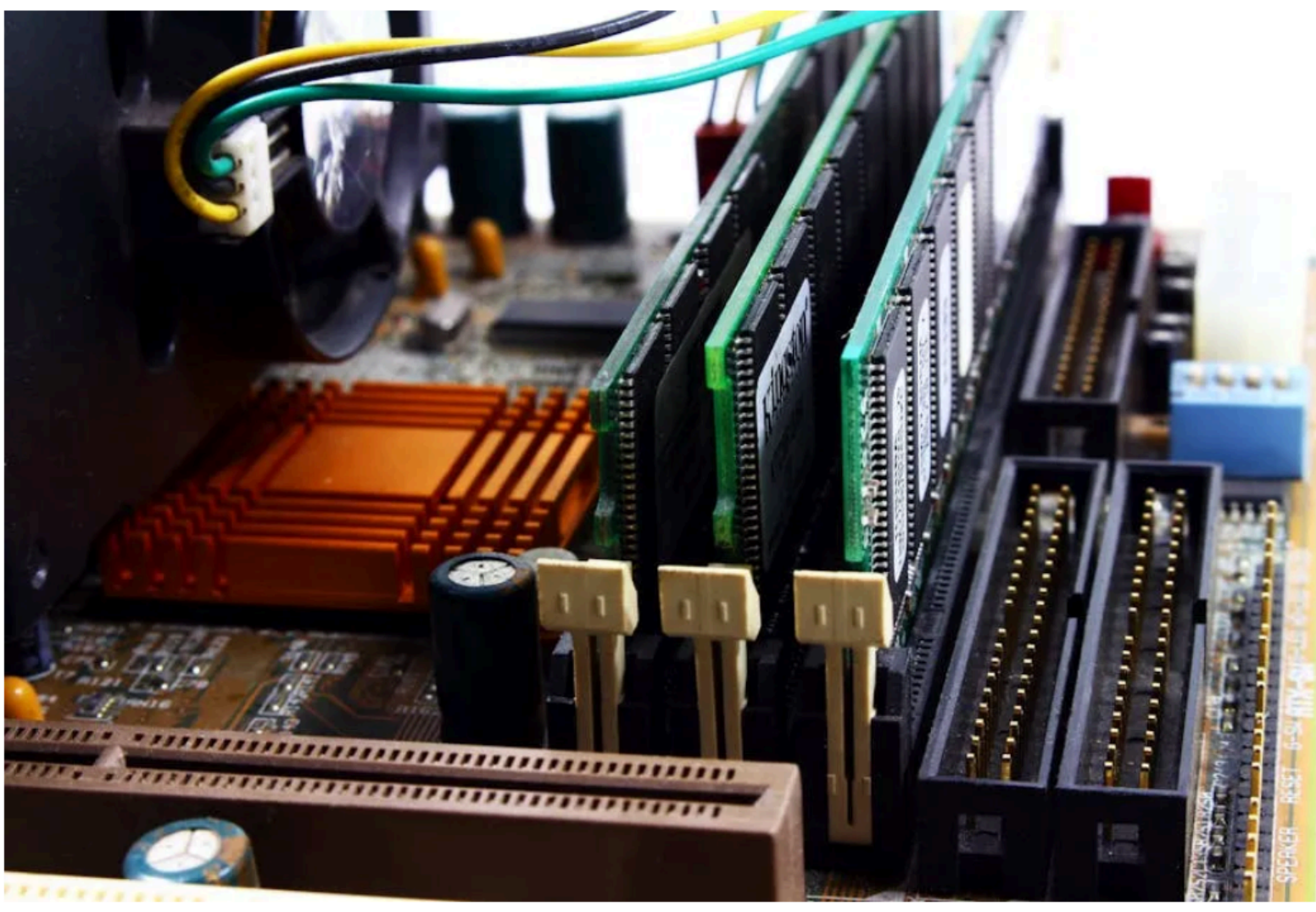
1. PCI总线

PCI(Peripheral Component Interconnect)是 一种由英特尔（Intel）公司1991年推出的用于定义局部总线的标准。此标准允许在计算机内安装多达10个遵从PCI标准的扩展卡。

PCI插槽是基于PCI局部总线的扩展插槽，其颜色一般为乳白色，位于主板上AGP插槽的下方，ISA插槽的上方。其位宽为32位或64位，工作频率为33MHz,最大数据传输率为133MB/sec(32位)和266MB/sec(64位)。

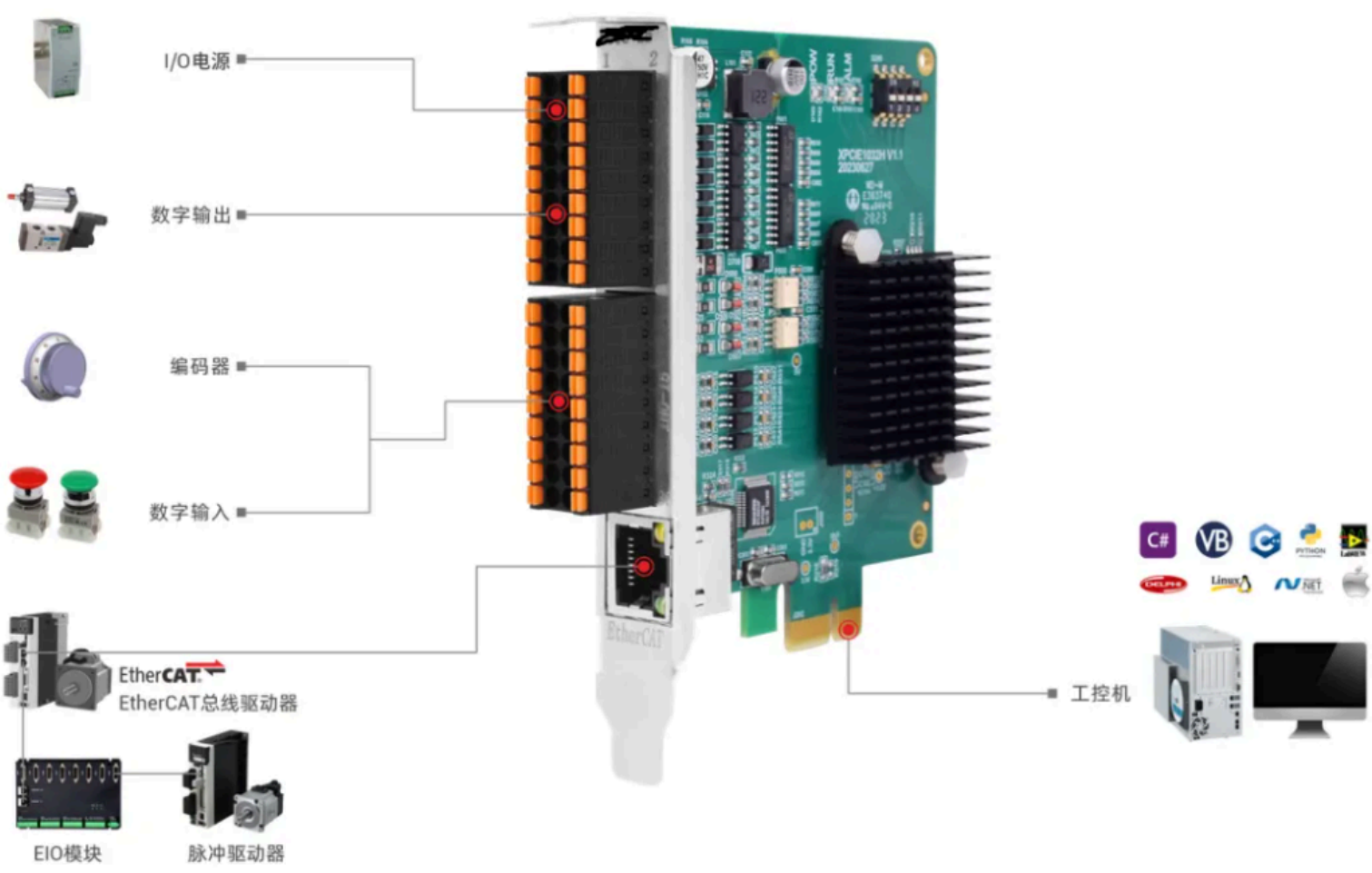
可插接显卡、声卡、网卡、内置Modem、内置ADSLModem、USB2.0卡、IEEE1394卡、IDE接口卡、RAID卡、电视卡、视频采集卡以及其它种类繁多的扩展卡。

PCI插槽是主板的主要扩展插槽，通过插接不同的扩展卡可以获得目前电脑能实现的几乎所有功能，是名副其实的“万用”扩展插槽。



2. 取代PCI

在这种PCI卡的应用中，基本上用卡就可以行了。如下图一般的连接和系统框架：



但是这里面有几点值得考虑：

- 接线和布线。这种方式必须要考虑到PCI的接口稳固性，然后还需要接很大的线缆。
- 多轴。如果轴数，IO数量多时，这种方式要插太多卡，每个卡上面需要接密密麻麻的线缆。
- 体积。带卡槽的工控机一般的体积都很大，美观和安装空间还有散热等都要考虑。

通过EtherCAT总线方案，控制器就能把上述所有的功能放到类似上图的高性能控制器中，不要考虑扩展卡槽等结构可能出现的接触不良等问题。

19.10 EtherCAT设备行规

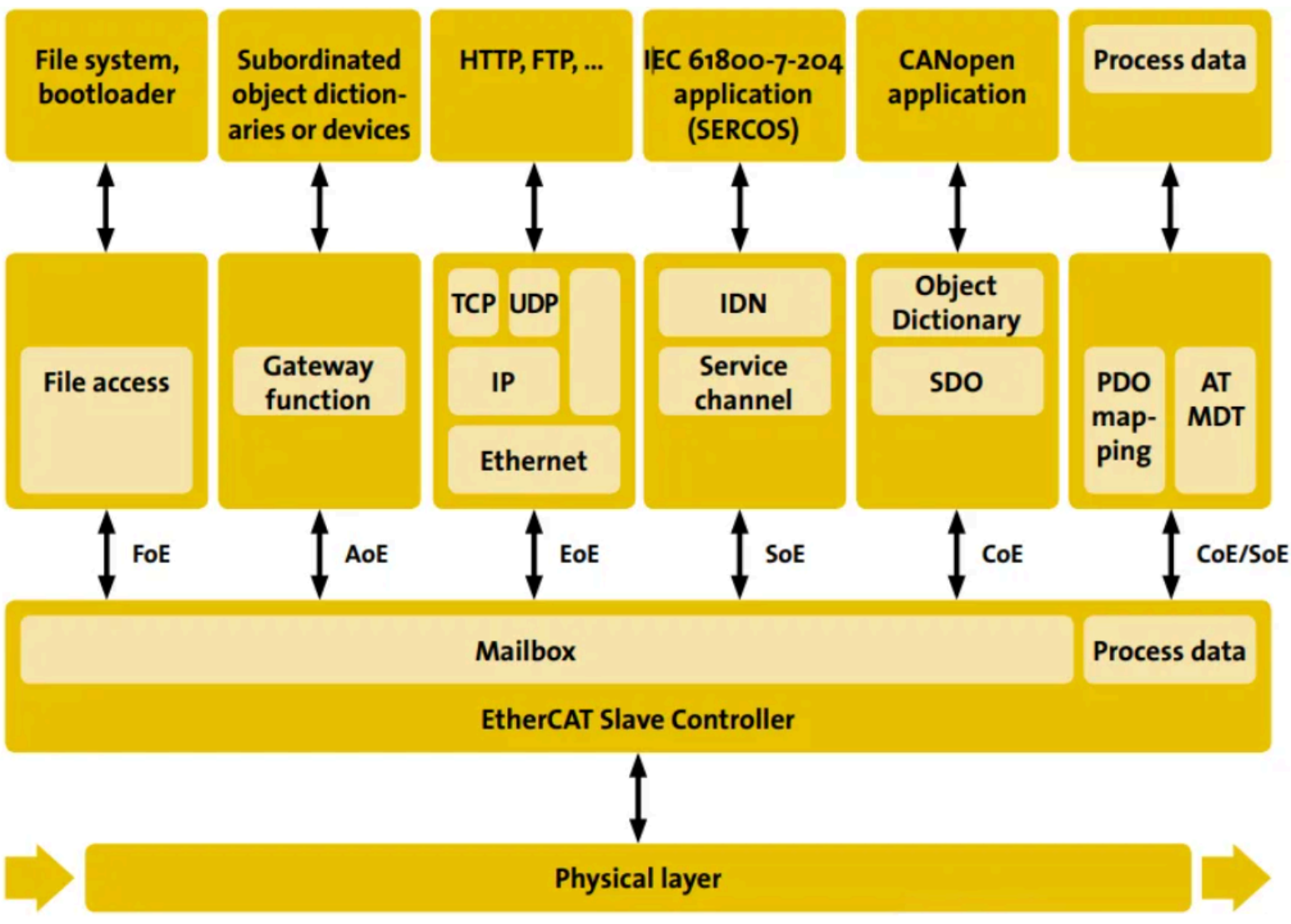
1. 设备行规（Device Profile）

设备通信行规（**Device Profile**）描述了应用参数和设备的功能行为，包括设备类特有的状态机。

许多设备类别（如I/O、驱动器、阀门）已具备成熟的现场总线设备配置文件，用户熟悉相关参数与工具。因此，未开发专门的EtherCAT配置文件，而是提供简单接口，从而方便从现场总线向EtherCAT的迁移。

为了支持更多种类的设备以及更广泛的应用层，主要建立了以下 EtherCAT 通信行规：

- **基于EtherCAT的CAN应用协议（CoE）**
- **符合IEC 61800-7-204标准的伺服驱动设备行规（SoE）**
- **Ethernet over EtherCAT（EoE）**
- **File access over EtherCAT（FoE）**
- **ADS over EtherCAT（AoE）**



在同一系统中应用层的不同设备行规可共存

2. CoE协议

CANopen over EtherCAT (CoE) 使用 CoE 协议，EtherCAT 可提供与 CANopen® 标准 EN 50325-4 相同的通信机制：包括对象字典、过程数据对象映像（PDO）以及服务数据对象（SDO），甚至相似的网络管理。

因此，在已经实施了 CANopen 的设备中，仅需稍加变动即可轻松实现EtherCAT，而且绝大部分的CANopen固件甚至都能得以重复利用。可以突破 8 字节的 PDO 限制，并可使用 EtherCAT 增强的带宽资源实现整个对象字典的上传。

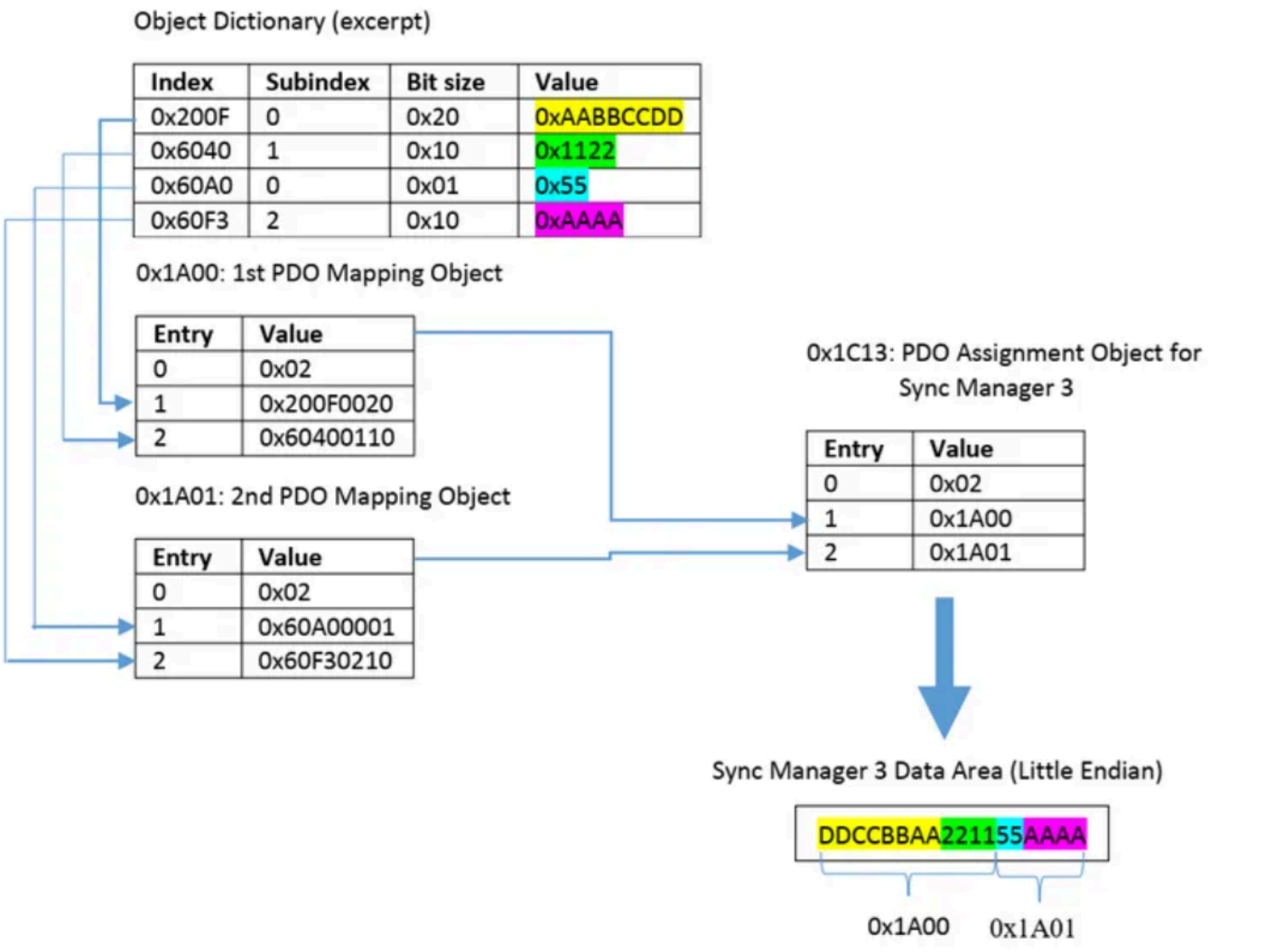
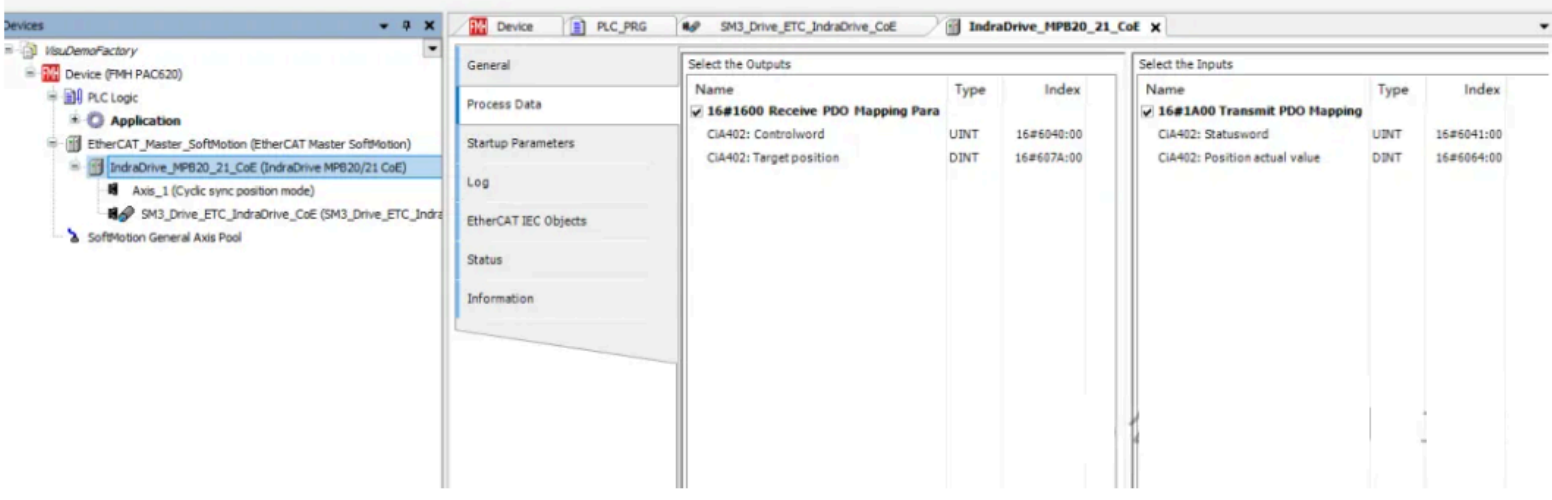


Figure: PDO Mapping in EtherCAT

另外，设备行规，如驱动器行规CiA402，也可以被重复应用EtherCAT。例如下方在CODESYS编辑器中添加IndraDrive_MPB20_21_CoE的伺服：



基于这种方式就可以精确定位到故障节点位置，还有可能发生故障的具体信息，方便我们进行故障的排除和解决。

3. EoE协议

Ethernet over EtherCAT（EoE）EtherCAT 仍使用以太网物理层和以太网数据帧。

使用 EoE 协议，可以在 EtherCAT 网段实现以太网数据的传输。EoE让EtherCAT网段传输以太网数据，设备经交换机端口接入。帧通过EtherCAT传送，类似IP协议，使网络透明；带有交换机端口的设备负责将 TCP/IP 数据片段插入到 EtherCAT 数据流中， 避免了对网络实时性的影响。

另外，EtherCAT 设备也可以支持以太网协议（例如 HTTP 协议），因此在 EtherCAT 网段 外，这些 EtherCAT 设备看上去就像一个标准的以太网节点。主站设备如同一个链路层交换机，根据各个节点的 MAC地址通过 EoE 发送数据帧给对应的节点。诸如**集成 web服务器、 电子邮件、FTP传输等所有的互联网技术**都可以在 EtherCAT 环境中得以应用。

4. 其他协议

Servo Profile over EtherCAT（SOE） SERCOS™ 被认为是一种实时通信接口，尤其适用于运动控制。用于伺服驱动的 SERCOS™行规属于国际标准 IEC 61800-7 的范畴。标准还包含了该行规对应于 EtherCAT 行规的映射。包含访问驱动内部参数和功能的服务通道被映射到 EtherCAT 邮箱。

File access over EtherCAT（FoE）FoE 协议与 TFTP（Trivial File Transfer Protocol）协议类似，可通过网络访问设备中的文件 以及将统一的固件跨网络上传到设备。FoE 协议有意识地用一种精简模式定义，因此可由 BootLoader 程序支持，而无需 TCP/IP 协议堆栈。

ADS over EtherCAT (AoE)作为基于邮箱的客户端-服务器协议，ADS over EtherCAT (AoE) 由 EtherCAT 规范定义。CoE 提供语义概念，AoE 通过路由和并行服务补充功能，且开销远小于 IP。

AoE 就是 EtherCAT 这个高速自动化王国里，那套灵活、轻便、不会干扰主线生产的内部通信专线。远程诊断工具，就像工程师从总部拨入分机，直接查看设备状态。

19.11 EtherCAT安全性

1. 概述

国内目前对设备安全的认证实际上基本上没有，仅有的是加一些急停、安全门等那已经算不错的了。

而我们一些出口到欧美的设备，如果没有这些安全性的标准认证，那是不予验收的。

诚然，如果要设备符合安全标准，那增加的成本是成倍数的，当然客户也能接受这样的成本，售价自然也会高点。但是，真正掌握设备的安全设计实际上理论性的和实践性的经验都是非常宝贵，掌握这些的工程师相对较少。

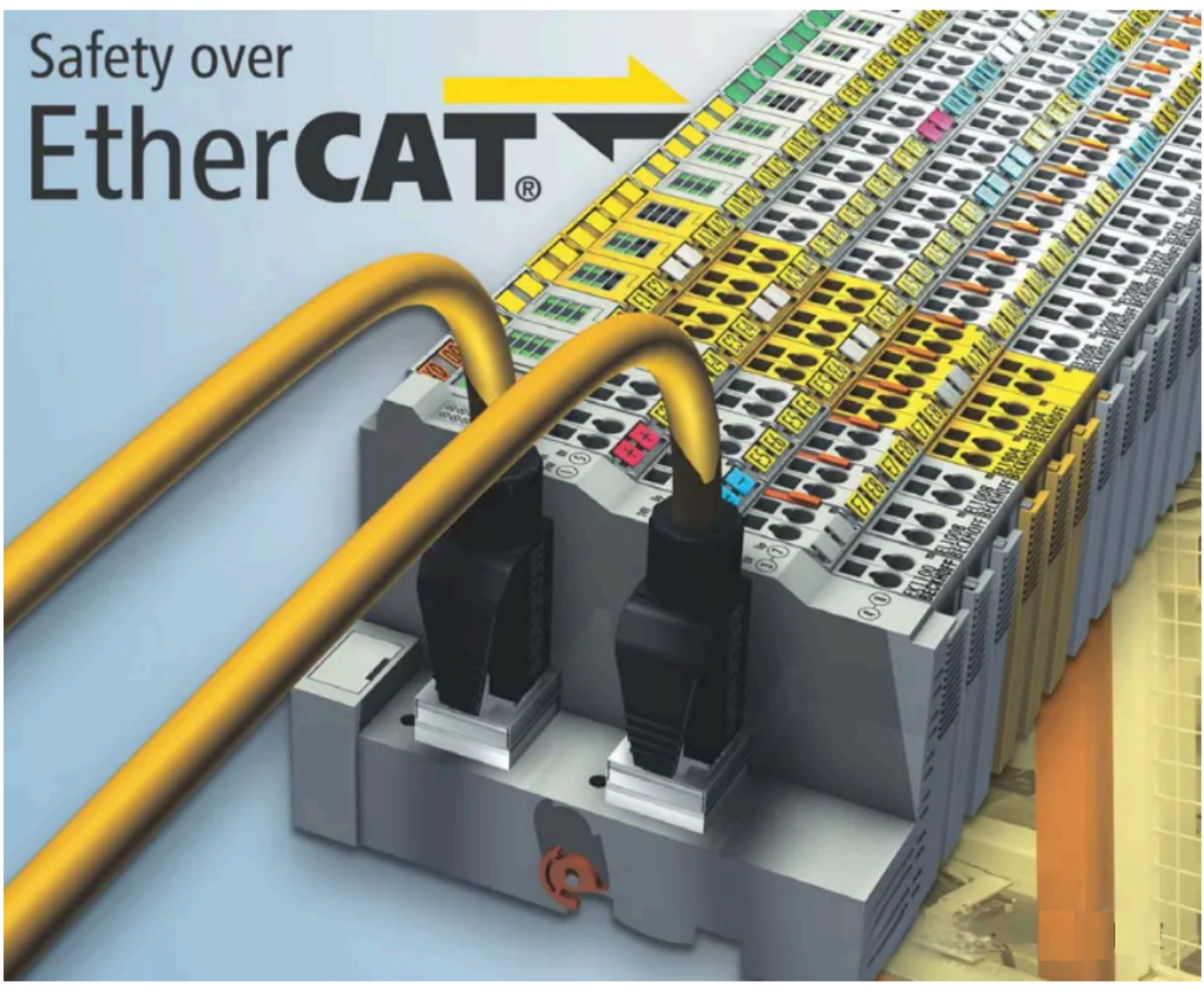
如果您是EtherCAT总线伺服、IO模块等研发企业或组织，可以深入研究这一块，开发和定制FSOE功能的控制器和IO等产品，国内是个空白市场。

2. FSOE（Safety Over EtherCAT）

除了采用安全传感器（如光幕、安全门监控设备或双手控制单元）通常通过评估设备进行监控，并通过硬接线逻辑影响安全输出，目前也出现了一种新趋势：自动化组件和通信系统中的智能安全解决方案允许将安全技术集成到机器设计中。

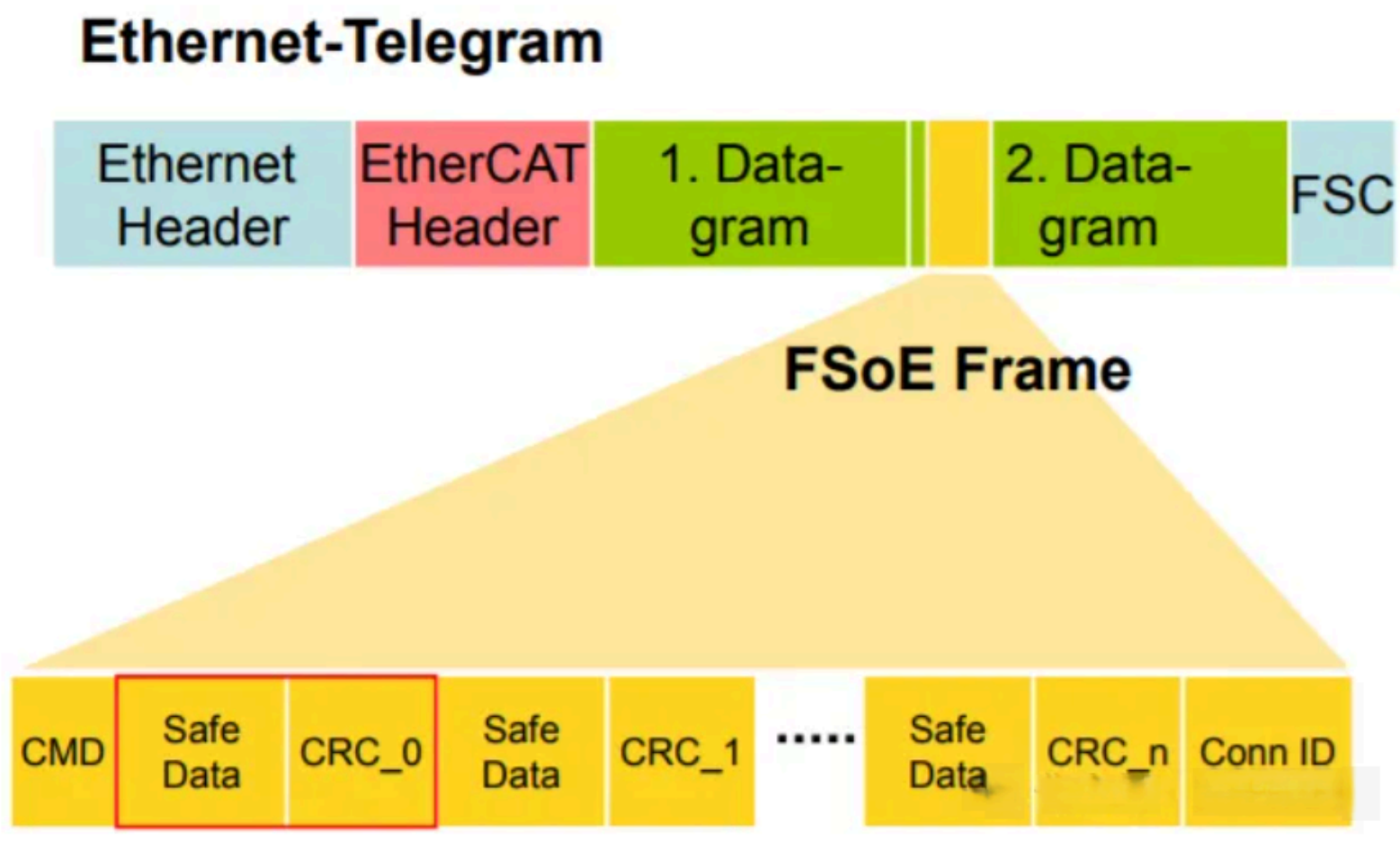
高级通信系统不仅提供控制信息的确定性传输，还允许在同一介质上传输安全相关数据。

而EtherCAT协议的安全性解决方案基于Safety over EtherCAT协议。



FSOE协议在EtherCAT技术组(ETG)内披露，旨在实现EtherCAT上的安全数据通信。该协议的特点包括：

- **符合IEC61508的SIL3标准。**
- **同一通信系统上传输安全和非安全信息。**
- **协议独立于传输系统和介质。**
- **安全过程数据的长度不受协议限制。**
- **可能实现非常短的帧长度。**
- **传输速度和周期时间不受限制**



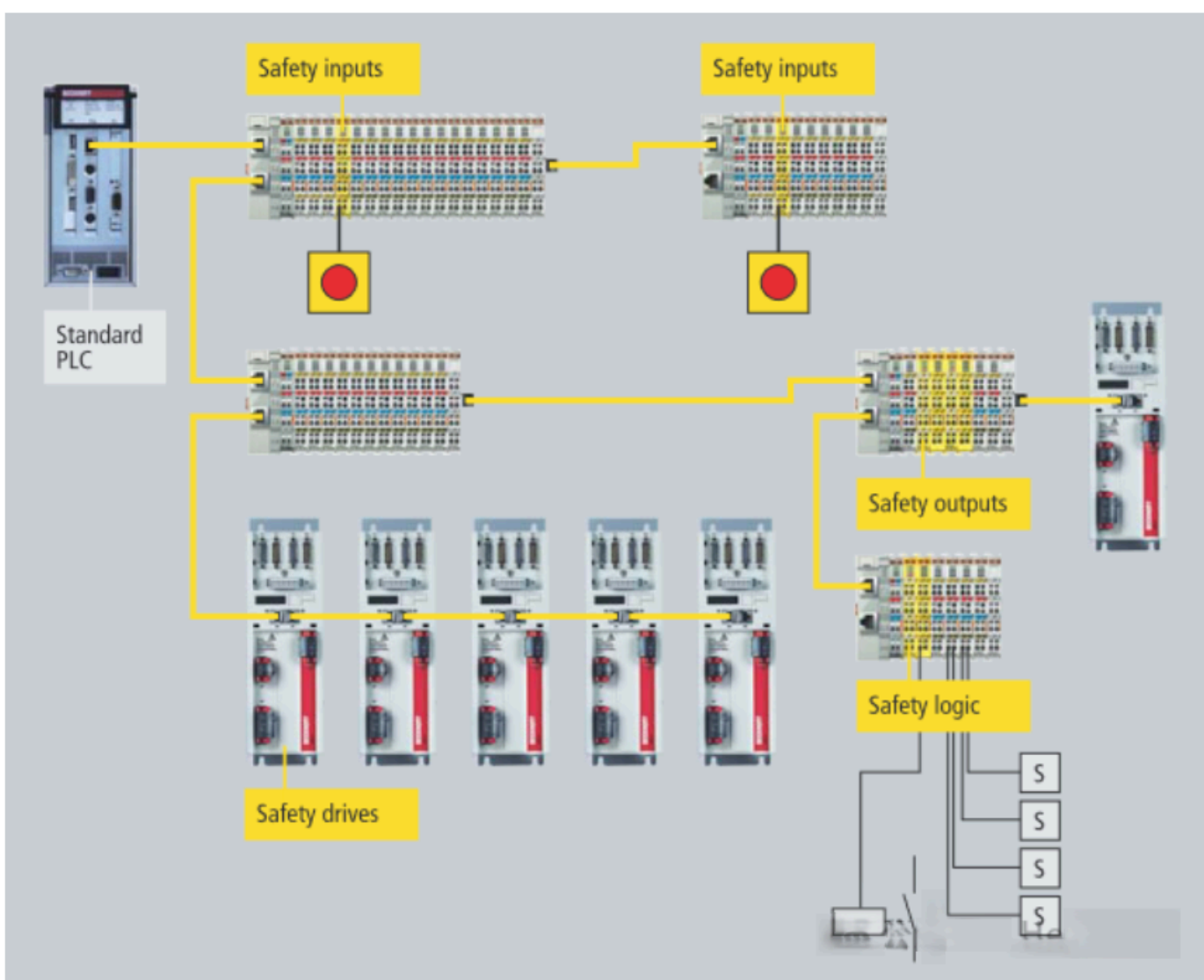
Safety over EtherCAT技术包括软件架构和硬件架构。该协议通过德国技术检验协会(TÜV)的评估，并被认证为符合IEC 61508 SIL 3标准的用于在Safety over EtherCAT设备之间传输过程数据的协议。

3. 好处

采用这种协议的好处有：

- **将安全概念无缝集成到机器设计中。**
- **无需为标准和安全应用开发单独的工具。**
- **简单处理和透明的安全功能。**
- **安全功能提供出色的诊断选项。**
- **控制和安全信息使用单一通信系统。**
- **实时和确定性方面没有性能限制。**
- **灵活的扩展选项。**

例如，在倍福系统里面可以采用SafetyInput输入模块及SafetyOutput输出模块接入安全性的传感器信号和安全输出信号，而且也支持伺服驱动器的Safety Drive；同时可以在同一系统（Automatic Logic）里面直接添加安全控制的逻辑（SafetyLogic），非常的便捷。



19.12 EtherCAT主站实施

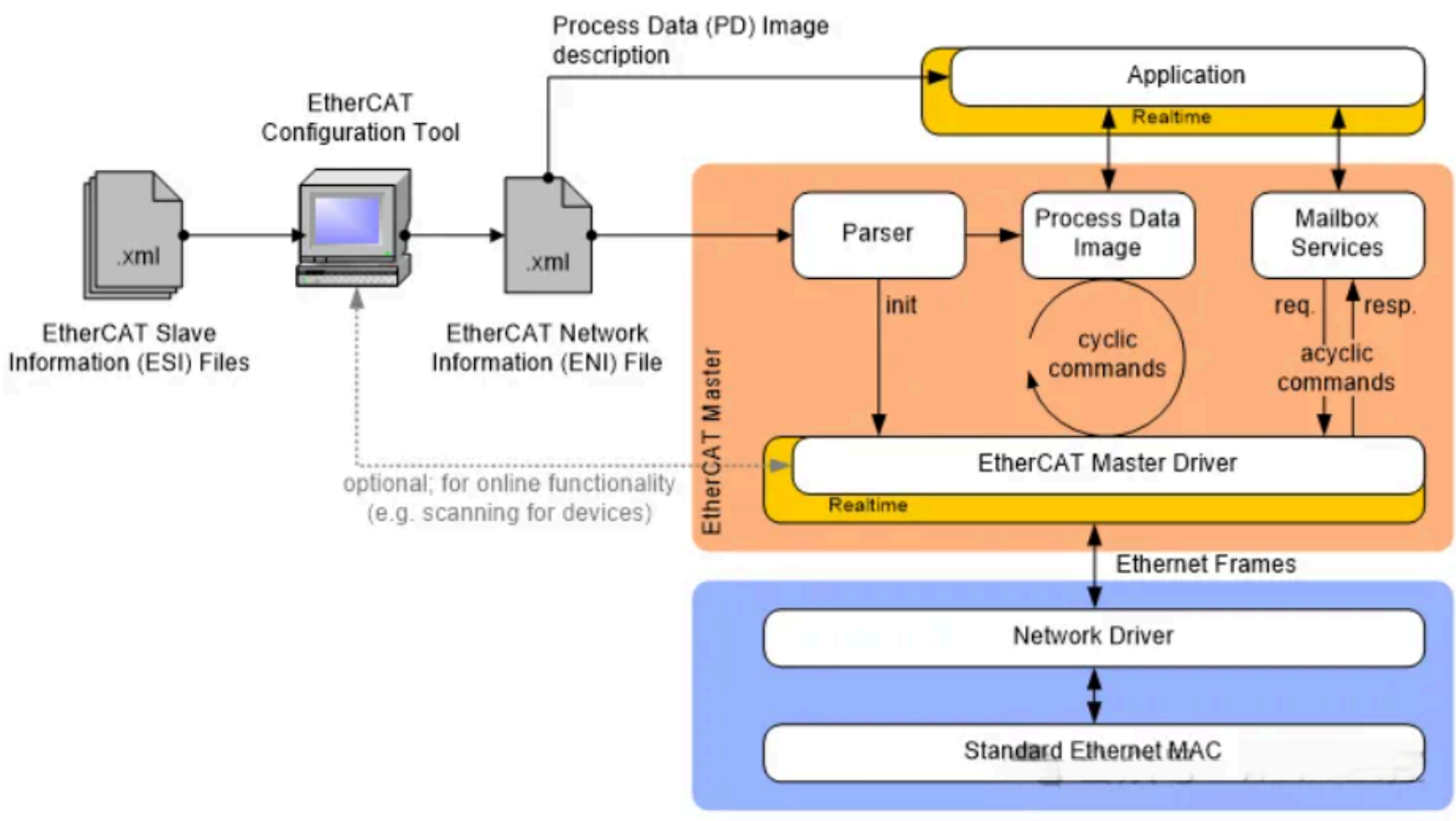
1. EtherCAT Master

对于开发EtherCAT Master，并不需要特殊的硬件，唯一要求简单的难以置信：**一个以太网端口。**

实现 EtherCAT主站接口既可以使用板载以太网控制器，也可以使用成本低廉的标准网卡，无需专用的接口卡。

这意味着，EtherCAT 主站设备仅使用标准的网卡就可以实现硬实时网络解决方案。

一般的控制器平台，提供EtherCAT主站的驱动和代码，只需要配置下就可以了：



2. 系统平台

已经在各种实时操作系统上实现了EtherCAT主站，包括但并不限于：eCos, INtime, MICROWARE OS-9, MQX，On Time RTOS-32, Proconos OS, Real-Time Java, RT Kernel, RT-Linux, RTX，RTXC, RTAI Linux, PikeOS, **Linux with RT-Preempt**, QNX, VxWin + CeWin, VxWorks, Windows CE, Windows XP/XPE with CoDeSys SP RTE, Windows NT/NTE/2000/XP/XPE/Vista with TwinCAT RTE, Windows 7 and **XENOMAI Linux**

主要两种方式开发主站：

1. **可以获得开源主站协议栈，作为示例代码或商业软件。**也有各种公司提供各种硬件平台上的实施服务，例如CODESYS平台直接支持EtherCAT主站，只需购买相应授权即可。可以在EtherCAT网站上的产品区找到快速增长的供应商信息。
2. **另一种EtherCAT主站的实现方式是使用样本代码**，花费不高。软件以源代码形式提供，包括所有的EtherCAT主站功能，甚至还包括EoE（EtherCAT实现以太网）功能。

开发人员只要把这些应用于Windows环境的代码与目标硬件及所使用的RTOS加以匹配就可以了。该软件代码已经成功应用于多个系统。

3. IgH EtherCAT Master

EtherCAT IGH，即 IgH EtherCAT Master，是由德国 IgH GmbH 开发的开源 EtherCAT 主站协议栈，采用 GPLv2 许可证。它以内核模块形式运行在 Linux 上，结合用户空间的 libethercat 库，为应用程序提供完整的 EtherCAT 总线服务。

IGH 的一大优势是兼容标准以太网硬件，如 Intel e1000e、r8169 和 igb 等常见网卡，无需专用主站卡；通过整合 Xenomai、RTAI 或 PREEMPT_RT 实时补丁，可实现低至数十微秒的硬实时控制周期，满足运动控制等苛刻场景。

在协议支持上，IGH 实现了完整的从站状态机、分布式时钟（DC）同步，并支持 CoE、SoE 及 VoE 等应用层协议。其内置的 `ethercat` 命令行工具可轻松扫描总线拓扑、显示从站信息、配置同步管理器和 PDO 映射，甚至直接读写 SDO，极大简化了开发与现场调试。

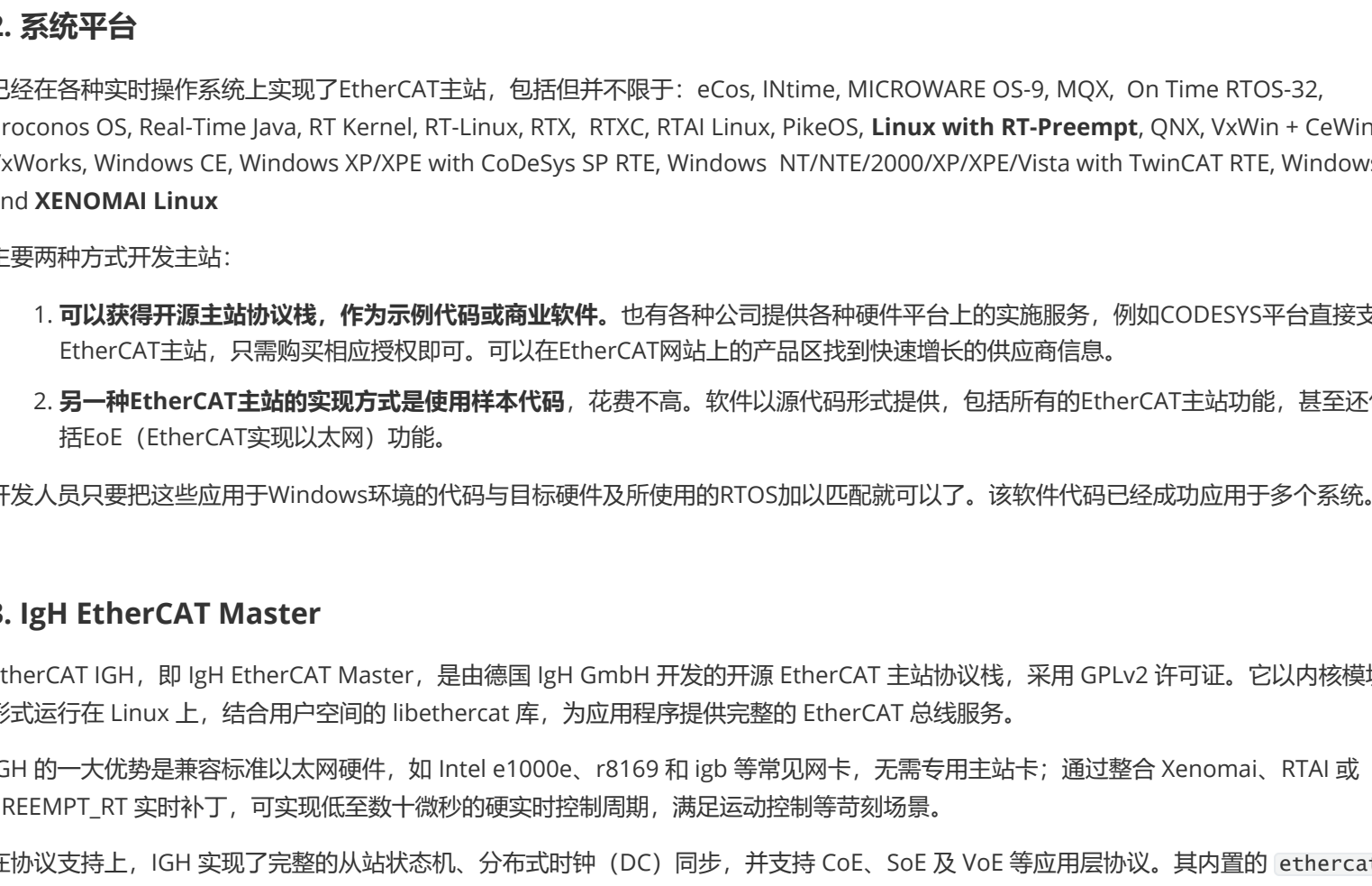
得益于此，IGH 已被广泛集成到 LinuxCNC、开源机器人系统以及众多定制自动化装备中，成为工业 EtherCAT 开发和学术研究的事实标准开源主站。

19.13 EtherCAT从站实施

1. ESC

EtherCAT从站设备使用成本低廉的EtherCAT从站控制器（ESC），ESC 可以是ASIC、FPGA或集成到有该协议标准的微控制器。

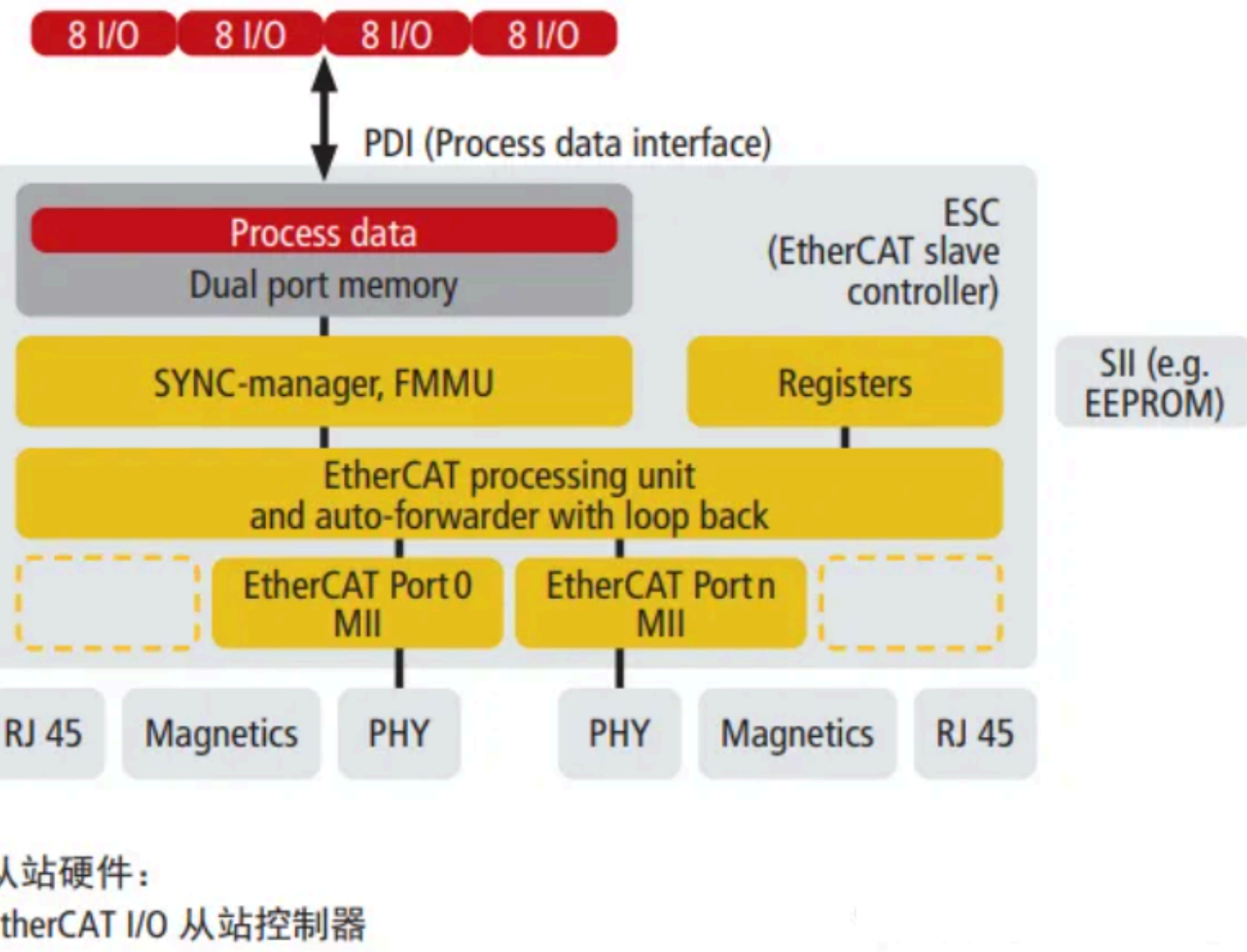
对于简单的EtherCAT从站设备甚至无需额外的微控制器，其输入/输出可直接与ESC连接。对于比较复杂的从站设备，微控制器性能会略微影响其通信性能，多数情况下，一个8位的微控制器就足够了。



从站硬件：
EtherCAT I/O 从站控制器

ESC 内部 DPRAM 大小及 FMMU 数量多样，并提供多种过程数据接口（PDI）供外部应用控制器访问应用存储器。

- 32位并行 I/O：适用于最多32位数字I/O，无需额外应用控制器，可用于简单传感器或执行器。
- 串行外设接口（SPI）：用于小数据量传输，如模拟量I/O、编码器或简单驱动。
- 并行8/16位微控制器接口：相当于集成 DPRAM 的现场总线通用接口，适合大数据量复杂节点。
- FPGA 及 OCV：用于不同微控制器的同步总线实施。



从站硬件：
带主CPU的EtherCAT从站控制器

- 硬件配置信息存储在 EEPROM 中，其从站接口信息（SII）包含从站基本特性，即使无设备描述文件，主站也可在启动时通过读取 SII 操作设备。
- EtherCAT 从站信息（ESI）文件为 XML 格式，完整描述了过程数据及映射选项、邮箱协议与可选特性、同步模式等网络访问属性。
- 网络配置工具利用 ESI 信息完成在线和离线配置。

2. 从站芯片

- **ASIC类型**

ASIC是从站控制器（ESC）所采用的专用集成电路芯片。

常见的 ASIC 类型（芯片型号）包括：

- **Beckhoff 系列**：如 ET1100、ET1200 等
- **Microchip 系列**：如 LAN9252、LAN9253 等
- **Hilscher 系列**：如 netX 系列芯片
- **其他厂商**：如 Infineon、Renesas 提供的相关 EtherCAT 从站芯片

- 常见的 ESC FPGA IP 核方案

这些 IP 核直接决定了兼容性和功能，是实际选型中更关键的“类型”：

- **Beckhoff 官方 IP 核**
  - **ET1810 / ET1811**：最经典、兼容性最好的 EtherCAT 从站 IP，授权给各大 FPGA 厂商。
- **厂商集成 IP**
  - 很多 FPGA 厂商与 Beckhoff 合作，在开发环境中直接提供经过验证的 ESC IP，如 Intel 的 “EtherCAT Slave Controller IP”、Xilinx 的 “EtherCAT Slave IP” 等。
- **第三方 IP**
  - 如 **Hilscher netX** 系列虽然本身就是芯片，但也提供用于 Altera/Xilinx 的 IP 许可。
  - 小规模、资源优化的第三方 IP 核也不少。
- **开源 IP 核**
  - 如 **EtherCAT Slave Core (ESC) on FPGA** 等开源项目，可用于学习和原型验证，但需仔细评估其完整性和实时性。

19.14 EtherCAT开放性

1. 开放性

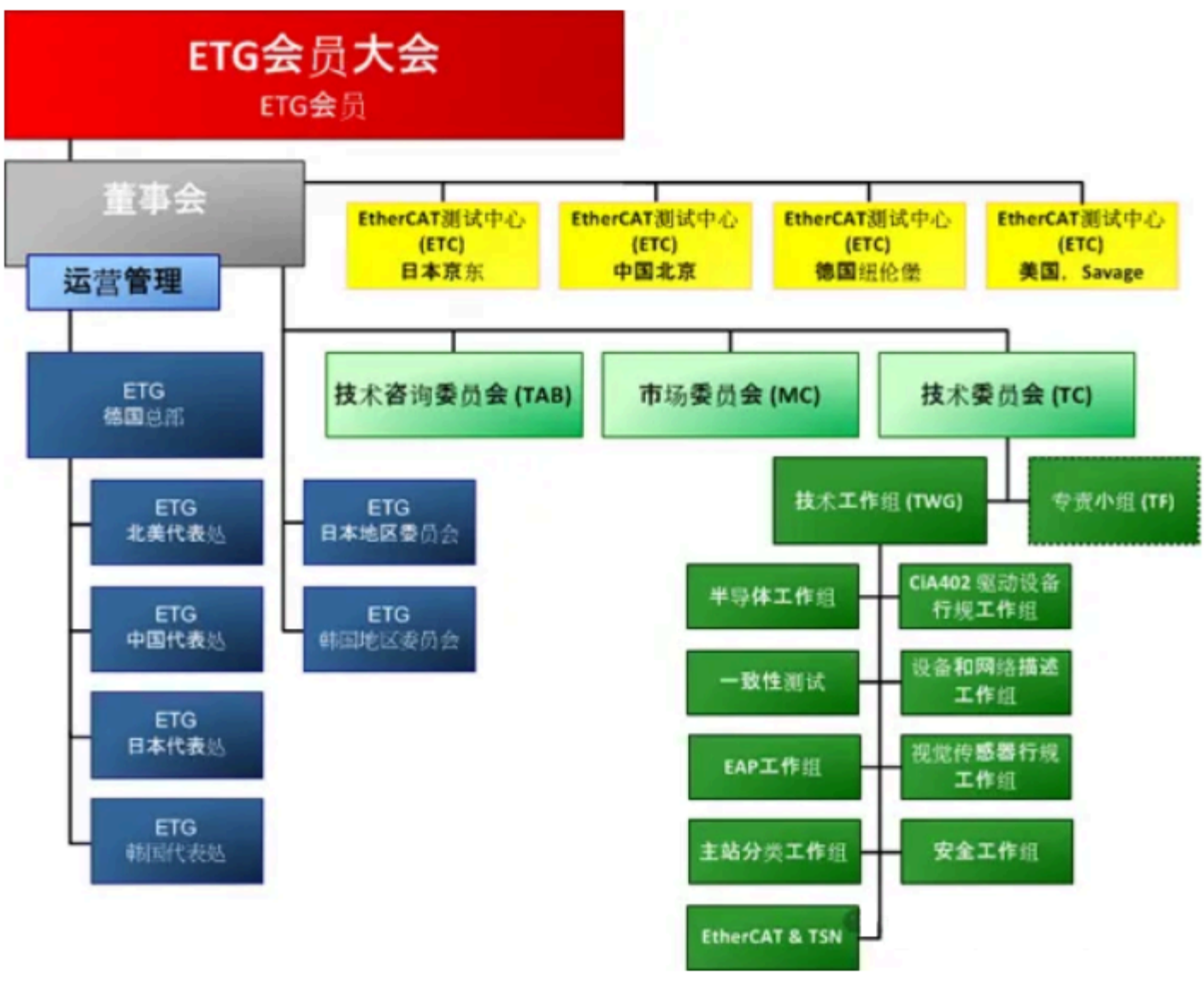
EtherCAT总线之所以在这短短30年，国内也就将近10年，快速成为工业通信的主流协议，且被国内厂家首选的总线协议，那EtherCAT的开放性功不可没。

从EtherCAT总线的创建（德国BECKHOFF），再到ETG组织的开放推广，可以提供给全球有需要的个人或组织机构。

2. ETG

EtherCAT技术协会（ETG）确保EtherCAT技术对所有潜在用户开放，由设备制造商、技术提供商和用户共同推动发展。

- 多个技术工作组致力于保持 EtherCAT 的稳定性与互操作性，因此只维护单一版本，不每年更新。
- 在全球举办研讨会、workshop 和展会，并提供产品指南、联合展台及展示，帮助会员推广 EtherCAT 产品。
- ETG 是全球最大的现场总线组织，更看重会员活跃度而非数量。
- EtherCAT 设备数量与种类无可比拟，在欧洲、亚洲和美国的采用率均超过其他工业以太网技术。



从EtherCAT总线的推广方式及管理模式，对于我们技术路线的企业组织来说，非常值得借鉴。

19.15 EtherCAT广泛应用

1. EtherCAT优势

主要以下几点：

- **卓越的性能**
- **灵活的拓扑**
- **简单且耐用**
- **集成安全**
- **低成本易实现**

2. 应用广泛

基于这些原因，EtherCAT广泛适用于：

- 机器人， 机床， 包装机械，印刷机，塑料制造机器，冲压机，半导体制造机器，试验台，测试系统，抓取机器
- 电厂，变电站，材料处理应用，行李运送系统，舞台控制系统，自动化装配系统，纸浆和造纸机，隧道控制系统
- 焊接机，起重机和升降机，农场机械，海岸应用，锯木厂，窗户生产设备，楼宇控制系统，钢铁厂，风机，家具生产设备
- 铣床，自动引导车，娱乐自动化，制药设备，木材加工机器，平板玻璃生产设备，称重系统

第20章 IgH主站源码

20.1 IgH协议栈介绍

1. IgH 介绍

IgH 全称 Ingenieurgemeinschaft Holzmann，是一个专为 **Linux 系统（含实时扩展）** 设计的开源 EtherCAT 主站协议栈。

它的设计目标是在标准 PC 硬件上实现微秒级实时性，并提供完整的主站功能。

2. 核心架构

IgH 采用**内核与用户空间分离**的模块化结构，主要包含三层：

1. **通用主站核心（Kernel Master Core）**
   - 作为内核模块运行，负责 EtherCAT 状态机、过程数据对象（PDO）交换、总线扫描、配置存储等。
   - 不依赖特定硬件，保持代码通用性。
2. **实时以太网驱动（Realtime Ethernet Drivers）**
   - 这是 IgH 实现高性能的关键。它**绕过 Linux 标准网络栈**，直接操作网卡硬件。
   - 为多种常见网卡提供了内核驱动（如 LAN9252、Intel e1000e、Realtek r8169等），保证数据帧以最高优先级发送和接收，典型周期抖动可小于 10 微秒。
3. **用户空间应用接口（Userspace Library）**
   - 提供 `libethercat` 库，应用在用户空间链接此库即可访问主站。
   - 通过字符设备 `/dev/EtherCAT` 与内核主站交互，使应用开发就像写普通程序一样简单，无需处理复杂的内核编程。

3. 主要特点

- **高实时性**：专有网卡驱动 + Linux 实时内核补丁（如 Xenomai、PREEMPT_RT），可满足严苛的运动控制需求。
- **功能全面**：完整支持分布式时钟（DC）、SDO 邮箱协议、CoE/EoE/FoE 应用层协议、线缆冗余等。
- **配套工具**：提供命令行工具 `ethercat`，可在线查看从站信息、调试参数、更新固件、测试总线负载等。
- **成熟稳定**：社区维护多年，大量工业现场验证，文档和案例丰富。
- **局限**：体系强依赖 Linux，不易移植到其他操作系统或裸机平台。

4. 同类工程横向对比

我将 IgH 与以下四种典型的 EtherCAT 主站方案进行对比：

特性	IgH EtherCAT Master	SOEM	Acontis EC-Master	TwinCAT	LinuxCNC EtherCAT
类型	开源，Linux 内核模块	开源，用户空间 C 库	商业闭源，跨平台 SDK	商业闭源，与 Windows 深度绑定	开源，Linux 用户空间组件
核心定位	高性能、功能完备的通用主站	极度轻量、易移植的简单主站	工业级、跨平台、高可靠性的商业主站	自动化软件平台的核心部件	专为数控软件 LinuxCNC 设计的驱动

特性	IgH EtherCAT Master	SOEM	Acontis EC-Master	TwinCAT	LinuxCNC EtherCAT
运行平台	**仅 Linux**（需实时内核）	几乎**任何平台**（Linux, Windows, VxWorks, MCU 裸机等）	**极广**：Linux, Windows, VxWorks, QNX, RTOS, 裸机	**仅 Windows**	**仅 Linux**（需实时内核）
实时机制	**内核实时驱动**，直接操作网卡	**用户空间**，依赖 OS 的定时与网络 API	多种模式可选：内核驱动、用户空间、硬件卸载	**内核级实时**，与 Windows 底层深度整合	**用户空间**，依赖 Linux 实时内核调度
开发集成	链接 `libethercat` 库，独立开发	**源码级集成**，直接将 .c/.h 文件加入项目	提供 C/C++/C# API 和完整 SDK	集成在 TwinCAT 工程环境中，通过 ADS 通信	LinuxCNC 的一个模块，需在 HAL 框架下配置
优势	实时性极高，功能成熟，社区强大，硬件驱动丰富	代码量极小，**移植极其方便**，学习成本低，资源占用少	性能极致，跨平台，功能最全，提供 SIL3 安全认证与商业技术支持	开发效率最高，硬件生态完善，配置自动化，全球标准	对数控应用**开箱即用**，与 LinuxCNC 无缝集成
劣势	环境搭建与移植较复杂，无法脱离 Linux	实时性能受 OS 限制，高级同步（如 DC）调优较难	**价格昂贵**，需商业授权，闭源	闭源，平台锁定，成本高，灵活性低	应用面极窄，无通用性
典型场景	基于 PC+Linux 的高性能运动控制器、PLC、视觉系统	MCU 或嵌入式处理器上的低成本主站、快速原型验证	高端工业产品（机器人、高端 PLC），需跨平台或安全认证的场合	基于 Windows PC 的工厂自动化、高端设备控制	数控机床、机器人等使用 LinuxCNC 的改造项目

- **SOEM (Simple Open EtherCAT Master)**：设计哲学是"简单"。如果你需要在裸机 MCU 或 RTOS 上做一个 EtherCAT 主站，或者想快速上手学习，SOEM 几乎是唯一的高效选择。它用极少的代码实现了主站基本功能，但对动态性能敏感的任务需要开发者自行优化时钟同步。
- **Acontis EC-Master**：商业领域的事实标准之一。其性能、兼容性和认证程度最高，支持主站冗余、热插拔、环形拓扑恢复等高级特性，但单许可费用可达数千欧元。
- **TwinCAT**：它不是单纯的主站协议栈，而是包含 PLC 运行时、NC 运动控制、HMI 的全集成平台。使用它意味着选择了倍福的 Windows 实时控制技术体系。
- **LinuxCNC EtherCAT**：它是在 IgH 或 SOEM 基础上为数控系统定制的驱动，用户无需编写主站代码，只需通过配置文件即可驱动 EtherCAT 伺服和 I/O。

---

5. 选型建议

1. **做高性能 Linux 工业控制器**（如高精度运控、视觉对位） → **IgH** 是首选。
2. **嵌入式低成本主站、快速出原型、非 Linux 平台** → **SOEM** 最合适。
3. **需要商业支持、安全认证、跨平台、顶级性能的正式产品** → 选择 **Acontis EC-Master**。
4. **已用倍福生态，Windows 环境** → **TwinCAT** 效率无人能及。
5. **改造或开发数控机床，使用 LinuxCNC** → 直接用内置的 **LinuxCNC EtherCAT** 驱动。

20.2 IgH 协议栈详解

1. 源码框架

IgH是开源的EtherCAT主站协议栈，虽然功能上和从站兼容性比不上商业协议栈，但完全能满足学习和普通场景的使用，公司也可以基于IgH作产品开发，基本能满足大部分使用场景。

- IgH源码地址：

```
https://gitlab.com/etherlab.org/ethercat
```

- 源码框架

```
linux@ubuntu:~/ethercat$ tree -L 1
.
├── AUTHORS
├── bootstrap
├── CodingStyle.md
├── configure.ac
├── CONTRIBUTING.md
├── COPYING
├── COPYING.LESSER
├── devices
├── documentation
├── Doxyfile.in
├── doxygen-layout
├── ethercat.spec.in
├── examples
├── fake_lib
├── FEATURES.md
├── globals.h
├── include
├── INSTALL.md
├── Kbuild.in
├── lib
├── Makefile.am
├── Makefile.kbuild
├── master
├── NEWS.md
├── README.EoE
├── README.md
├── script
├── TODO
├── tool
└── tty

11 directories, 19 files
linux@ubuntu:~/ethercat$
```

- devices 目录下存放的是经过优化的实时网卡驱动
- examples 是上层应用程序示例程序
- include 是程序依赖的头文件
- lib 是主站接口封装的库文件，供应用程序调用
- master EtherCAT主站程序
- tool 是IgH命令工具，可以获取和设置主、从站信息

2. 主栈启动流程 ORPHANED

主栈代码位于master/ 目录下，IgH主栈编出来的是ec_master.ko , 所以是以KO的形式加载的，驱动的入口函数在module.c中，声明了驱动入口函数ec_init_module

- master/module.c

```
/*****************************************************************************/

/** \cond */

MODULE_AUTHOR("Florian Pose <fp@igh.de>");
MODULE_DESCRIPTION("EtherCAT master driver module");
MODULE_LICENSE("GPL");
MODULE_VERSION(EC_MASTER_VERSION);

module_param_array(main_devices, charp, &master_count, S_IRUGO);
MODULE_PARM_DESC(main_devices, "MAC addresses of main devices");
module_param_array(backup_devices, charp, &backup_count, S_IRUGO);
MODULE_PARM_DESC(backup_devices, "MAC addresses of backup devices");
module_param_named(debug_level, debug_level, uint, S_IRUGO);
MODULE_PARM_DESC(debug_level, "Debug level");
module_param_named(run_on_cpu, run_on_cpu, uint, S_IRUGO);
MODULE_PARM_DESC(run_on_cpu, "Bind kthreads to a specific cpu");

/** \endcond */

/*****************************************************************************/
/*****************************************************************************/

/** \cond */

module_init(ec_init_module);
module_exit(ec_cleanup_module);

EXPORT_SYMBOL(ecdev_offer);

EXPORT_SYMBOL(ecrt_request_master);
EXPORT_SYMBOL(ecrt_release_master);
EXPORT_SYMBOL(ecrt_version_magic);

/** \endcond */
```

```
/*****************************************************************************/
```

IgH支持多主站和双网口设备冗余，目前我们只讨论单主站和单网卡情况

- master/module.c

```
/*****************************************************************************/

/** Module initialization.
 *
 * Initializes \a master_count masters.
 * \return 0 on success, else < 0
 */
int __init ec_init_module(void)
{
    int i, ret = 0;

    EC_INFO("Master driver %s\n", EC_MASTER_VERSION);

    sema_init(&master_sem, 1);

    if (master_count) {
        if (alloc_chrdev_region(&device_number,
                    0, master_count, "EtherCAT")) {
            EC_ERR("Failed to obtain device number(s)!\n");
            ret = -EBUSY;
            goto out_return;
        }
    }

#if LINUX_VERSION_CODE < KERNEL_VERSION(6, 4, 0)
    class = class_create(THIS_MODULE, "EtherCAT");
#else
    class = class_create("EtherCAT");
#endif
    if (IS_ERR(class)) {
        EC_ERR("Failed to create device class.\n");
        ret = PTR_ERR(class);
        goto out_cdev;
    }

    // zero MAC addresses
    memset(macs, 0x00, sizeof(uint8_t) * EC_MAX_MASTERS * 2 * ETH_ALEN);

    // process MAC parameters
    for (i = 0; i < master_count; i++) {
        ret = ec_mac_parse(macs[i][0], main_devices[i], 0);
        if (ret)
            goto out_class;

        if (i < backup_count) {
            ret = ec_mac_parse(macs[i][1], backup_devices[i], 1);
            if (ret)
                goto out_class;
        }
    }

    // initialize static master variables
    ec_master_init_static();

    if (master_count) {
        if (!(masters = kmalloc(sizeof(ec_master_t) * master_count,
                        GFP_KERNEL))) {
            EC_ERR("Failed to allocate memory"
                    " for EtherCAT masters.\n");
            ret = -ENOMEM;
            goto out_class;
        }
    }

    for (i = 0; i < master_count; i++) {
        ret = ec_master_init(&masters[i], i, macs[i][0], macs[i][1],
                    device_number, class, debug_level, run_on_cpu);
        if (ret)
            goto out_free_masters;
    }

    EC_INFO("%u master%s waiting for devices.\n",
            master_count, (master_count == 1 ? "" : "s"));
    return ret;

out_free_masters:
    for (i--; i >= 0; i--)
        ec_master_clear(&masters[i]);
    kfree(masters);
out_class:
    class_destroy(class);
out_cdev:
    if (master_count)
        unregister_chrdev_region(device_number, master_count);
out_return:
    return ret;
}
```

主栈初始化首先创建了一个字符设备，作用是IgH上层应用程序是通过ioctl接口来调用主栈驱动程序接口的，创建`/dev/EtherCAT0`设备，上层应用程序可以通过这个设备调用驱动接口，然后是调用主栈初始化功能接口。

- master/module.c:155

```
    for (i = 0; i < master_count; i++) {
        ret = ec_master_init(&masters[i], i, macs[i][0], macs[i][1],
                    device_number, class, debug_level, run_on_cpu);
        if (ret)
            goto out_free_masters;
    }
```

- master/master.c:212

```
/*****************************************************************************/

/**
   Master constructor.
   \return 0 in case of success, else < 0
*/

int ec_master_init(ec_master_t *master, /**< EtherCAT master */
        unsigned int index, /**< master index */
        const uint8_t *main_mac, /**< MAC address of main device */
        const uint8_t *backup_mac, /**< MAC address of backup device */
        dev_t device_number, /**< Character device number. */
        struct class *class, /**< Device class. */
        unsigned int debug_level, /**< Debug level (module parameter). */
        unsigned int run_on_cpu /**< bind created kernel threads to a cpu */
        )
{
    ...

    master->phase = EC_ORPHANED;  // 主站初始化后的状态
    master->active = 0;
    master->config_changed = 0;

    master->injection_seq_fsm = 0;
    master->injection_seq_rt = 0;

    master->slaves = NULL;
    master->slave_count = 0;

    INIT_LIST_HEAD(&master->configs);
    INIT_LIST_HEAD(&master->domains);

    master->app_time = 0ULL;
    master->dc_ref_time = 0ULL;
    ...

    return ret;
}

/*****************************************************************************/
```

- master/master.h

```
/*****************************************************************************/

/** EtherCAT master phase.
 */
```

```
typedef enum {
    EC_ORPHANED, /**< Orphaned phase. The master has no Ethernet device
                    attached. */
    EC_IDLE, /**< Idle phase. An Ethernet device is attached, but the master
                is not in use, yet. */
    EC_OPERATION /**< Operation phase. The master was requested by a realtime
                    application. */
} ec_master_phase_t;

/*****************************************************************************/
```

这个枚举描述的是 **IgH EtherCAT 主站自身的运行生命周期**，而非从站状态：

- **ORPHANED（孤儿）**：主站未绑定物理网卡，无物理通信能力。通常因网卡驱动未释放或配置导致。
- **IDLE（空闲）**：网卡已绑定，可响应 `ethercat` 命令行扫描总线，但上层实时应用未加载（未调用 `ecrt_master_activate`），**无**周期性过程数据（PDO）交互。
- **OPERATION（运行）**：已被实时应用激活，执行周期性的数据帧收发，控制主站进入 OP 态，进行实时数据交换。

流转顺序：`ORPHANED → IDLE → OPERATION`（应用退出后回退至 `IDLE`）。

3. 主栈启动流程 IDLE

ec_master.ko加载后只是初始化了主栈的一些数据和状态，并未运行主栈，因为此时网络并未启动，主栈无法和从站通信。

当我们加载实时网卡驱动时，此时建立了网络连接，就可以启动主栈来根据连接的从站做信息的同步，

这里以STMMAC网卡为例，这是因为RK3506用的就是STMMAC的IP核：

- 打开网卡
- devices/stmmac/stmmac_main-6.1-ethercat.c:7509-7516

```
。。。
    if (get_ecdev(priv)) {
        init_irq_work(&priv->ec_watchdog_kicker, ec_kick_watchdog);
        ret = ecdev_open(get_ecdev(priv));      // ★
        if (ret) {
            ecdev_withdraw(get_ecdev(priv));
            goto error_netdev_register;
        }
    }

int ecdev_open(ec_device_t *device)
{
    // ① 打开 net_device 硬件
    ret = ec_device_open(device);
    //   └─ device->dev->netdev_ops->ndo_open(dev)  → stmmac_open()

    // ② 检查所有设备都已打开 → 进入 IDLE 阶段
    if (all_open) {
        ec_master_enter_idle_phase(master);
        // └─ master->phase = EC_IDLE
        // └─ ec_master_thread_start(ec_master_idle_thread, "EtherCAT-IDLE")
        //      └─ kthread_create + wake_up_process  → idle 线程开始循环
    }
}
```

- Idle 线程主循环
- master/master.c

```
/*****************************************************************************/

/** Master kernel thread function for IDLE phase.
 */
static int ec_master_idle_thread(void *priv_data)
{
    ec_master_t *master = (ec_master_t *) priv_data;
    int fsm_exec;
#ifdef EC_USE_HRTIMER
    size_t sent_bytes;
#endif

    // send interval in IDLE phase
    ec_master_set_send_interval(master, 1000000 / HZ);

    EC_MASTER_DBG(master, 1, "Idle thread running with send interval = %u us,"
            " max data size=%zu\n", master->send_interval,
            master->max_queue_size);

    while (!kthread_should_stop()) {
        ec_datagram_output_stats(&master->fsm_datagram);

        // receive
        if (ec_rt_lock_interruptible(&master->io_mutex))
            break;
        ecrt_master_receive(master);
        rt_mutex_unlock(&master->io_mutex);

        // execute master & slave state machines
        if (down_interruptible(&master->master_sem)) {
            break;
        }

        fsm_exec = ec_fsm_master_exec(&master->fsm);

        ec_master_exec_slave_fsms(master);

        up(&master->master_sem);

        // queue and send
        if (ec_rt_lock_interruptible(&master->io_mutex))
            break;
        if (fsm_exec) {
            ec_master_queue_datagram(master, &master->fsm_datagram);
        }
        ecrt_master_send(master);
#ifdef EC_USE_HRTIMER
        sent_bytes = master->devices[EC_DEVICE_MAIN].tx_skb[
            master->devices[EC_DEVICE_MAIN].tx_ring_index]->len;
#endif
        rt_mutex_unlock(&master->io_mutex);

        if (ec_fsm_master_idle(&master->fsm)) {
#ifdef EC_USE_HRTIMER
            ec_master_nanosleep(master->send_interval * 1000);
#else
            set_current_state(TASK_INTERRUPTIBLE);
            schedule_timeout(1);
#endif
        } else {
#ifdef EC_USE_HRTIMER
            ec_master_nanosleep(sent_bytes * EC_BYTE_TRANSMISSION_TIME_NS);
#else
            schedule();
#endif
        }
    }

    EC_MASTER_DBG(master, 1, "Master IDLE thread exiting...\n");

    return 0;
}
```

- 分析源码

```
while (!kthread_should_stop()) {
    ┌──────────────────────────────────────────┐
    | ① RECEIVE: ecrt_master_receive(master)   |
    |    → ec_device_poll()                    |
    |      → device->poll(dev) 即 ec_poll()    |  ← 驱动层
    └──────────────────────────────────────────┘

    ┌──────────────────────────────────────────┐
    | ② FSM:   ec_fsm_master_exec()            |
    |          ec_master_exec_slave_fsms()     |  ← 从站扫描+配置
    └──────────────────────────────────────────┘

    ┌──────────────────────────────────────────┐
    | ③ SEND:  ec_master_queue_datagram()      |
    |          ecrt_master_send(master)        |  ← 发送数据报
    └──────────────────────────────────────────┘

    ┌──────────────────────────────────────────┐
    | ④ SLEEP: schedule_timeout(1) / schedule()|
    └──────────────────────────────────────────┘
}
```

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

4. 主栈启动流程 OPERATION

IgH EtherCAT Master 有三个阶段：

```
ORPHANED  ──(ecdev_open)──▶  IDLE  ──(ecrt_master_activate)──▶  OPERATION
                              |                                    |
                              |  ec_master_idle_thread()           |  ec_master_operation_thread()
                              |  主栈自己轮询收发                    |  用户应用程序接管收发
                              |  扫描从站 + EoE 可用                 |  实时控制循环
```

IDLE 阶段主栈自己轮询执行扫描/配置 FSM，OPERATION 阶段则**把收发控制权交给用户应用程序**，主栈只保留 FSM 数据报的注入能力。

- master/master.c
- ecrt_master_activate()

```c
int ecrt_master_activate(ec_master_t *master)
{
    // ① 完成所有域的 FMMU 配置
    list_for_each_entry(domain, &master->domains, list) {
        ec_domain_finish(domain, domain_offset);
    }

    // ② 停止 IDLE 线程
    ec_master_thread_stop(master);          // 停掉 "EtherCAT-IDLE"
    ec_master_eoe_stop(master);             // 停掉 EoE 线程

    // ③ 注入计数器归零
    master->injection_seq_fsm = 0;
    master->injection_seq_rt = 0;

    // ④ ★ 切换收发回调：内部 → 用户应用
    master->send_cb = master->app_send_cb;     // 应用注册的 send 回调
    master->receive_cb = master->app_receive_cb; // 应用注册的 receive 回调
    master->cb_data = master->app_cb_data;

    // ⑤ ★ 启动 OPERATION 线程
    ec_master_thread_start(master, ec_master_operation_thread, "EtherCAT-OP");

    // ⑥ 重新允许扫描
    master->allow_scan = 1;
    master->active = 1;
    master->config_changed = 1; // 触发状态机应用配置
}
```

- OPERATION 线程
- master/master.c:1614-1668
- ec_master_operation_thread()

```c
static int ec_master_operation_thread(void *priv_data)
{
    ec_master_t *master = (ec_master_t *) priv_data;
    unsigned int seq_rt;

    while (!kthread_should_stop()) {
        // ① 检查是否有新的 FSM 数据报需要注入
        seq_rt = smp_load_acquire(&master->injection_seq_rt);
        if (seq_rt == master->injection_seq_fsm) {
            // RT 线程已消费上一个 FSM 数据报，可以注入新的

            // ② 执行主站 FSM
            if (ec_fsm_master_exec(&master->fsm)) {
                // FSM 产生了新数据报 → 递增加入注入队列
                smp_store_release(&master->injection_seq_fsm,
                        master->injection_seq_fsm + 1);
            }

            // ③ 执行从站 FSM
            ec_master_exec_slave_fsms(master);
        }

        // ④ 休眠（与 RT 发送周期同步）
        ec_master_nanosleep(master->send_interval * 1000);
        // 或 schedule_timeout(1) / schedule()
    }
}
```

- IDLE vs OPERATION 线程的关键差异

	IDLE 线程	OPERATION 线程
线程名	`EtherCAT-IDLE`	`EtherCAT-OP`
收发职责	主栈自己调用 `ecrt_master_receive/send`	**不调用**！由用户应用接管
数据报注入	直接把 FSM 数据报入队 `ec_master_queue_datagram()`	通过 `injection_seq_fsm/rt` 无锁注入
休眠机制	空闲 `schedule_timeout(1)`，忙 `schedule()`	`ec_master_nanosleep(send_interval)` 与 RT 同步
用途	拓扑扫描、从站配置、EoE	应用实时控制期间的 FSM 维护

5. 发送报文

```
用户程序 (RT 线程)
  |
  ▼
ecrt_master_send(master)                                  [master.c:2460]
  |
  ├── ① FSM 数据报注入 (无锁)
  |     injection_seq_rt != seq_fsm ?
  |     → ec_master_queue_datagram(master, &master->fsm_datagram)
  |
  ├── ② 外部数据报注入
  |     → ec_master_inject_external_datagrams(master)
  |
  └── ③ 逐设备发送
        for (dev_idx) {
          if (link_state == UP)
            ec_master_send_datagrams(master, dev_idx) [master.c:1010]
              |
              | 循环: 将队列中的数据报拼接到 Eth 帧
              |
              ├── ec_device_tx_data(device)          [device.c:301]
              |     → 预分配的 skb ring buffer (EC_TX_RING_SIZE)
              |
              ├── 拼接 EtherCAT 帧:
              |     [Length] [Datagram1] [Datagram2] ...
              |     Header   Header+Data+Ftr  ...
              |
              ├── ec_device_send(device, frame_size)  [device.c:320]
              |     → ndo_start_xmit(skb, dev)
              |
              └── 标记 datagram->state = EC_DATAGRAM_SENT
        }
```

- 数据报队列管理
- master/master.c

```c
void ec_master_queue_datagram(ec_master_t *master, ec_datagram_t *datagram)
{
    // 防重复入队检查
    list_for_each_entry(queued_datagram, &master->datagram_queue, queue) {
        if (queued_datagram == datagram) {
            datagram->skip_count++;
            datagram->state = EC_DATAGRAM_QUEUED;
            return;
        }
    }
    list_add_tail(&datagram->queue, &master->datagram_queue);
    datagram->state = EC_DATAGRAM_QUEUED;
}
```

- 出队 + 拼接 ec_master_send_datagrams()
- master/master.c

```c
void ec_master_send_datagrams(ec_master_t *master, ec_device_index_t dev_idx)
{
    do {
        frame_data = NULL;
        follows_word = NULL;

        // 遍历队列中所有 QUEUED 状态的数据报
        list_for_each_entry(datagram, &master->datagram_queue, queue) {
            if (datagram->state != EC_DATAGRAM_QUEUED) continue;
```

```
        if (datagram->device_index != device_index) continue;

        if (!frame_data) {
            // ★ 获取预分配的 TX skb
            frame_data = ec_device_tx_data(&master->devices[dev_idx]);
            cur_data = frame_data + EC_FRAME_HEADER_SIZE;  // 跳过帧头
        }

        // 检查数据报是否能放入当前帧
        datagram_size = EC_DATAGRAM_HEADER_SIZE + datagram->data_size
            + EC_DATAGRAM_FOOTER_SIZE;
        if (cur_data - frame_data + datagram_size > ETH_DATA_LEN) {
            more_datagrams_waiting = 1;   // 帧满，需要再发一帧
            break;
        }

        // ★ 拼接 EtherCAT 数据报到帧中
        EC_WRITE_U8(cur_data, datagram->type);           // 命令字节
        EC_WRITE_U8(cur_data + 1, datagram->index);      // 索引
        memcpy(cur_data + 2, datagram->address, EC_ADDR_LEN); // 地址
        EC_WRITE_U16(cur_data + 6, datagram->data_size & 0x7FF); // 长度
        // 设置 "more follows" bit
        if (follows_word)
            EC_WRITE_U16(follows_word, EC_READ_U16(follows_word) | 0x8000);
        follows_word = cur_data + 6;
        cur_data += EC_DATAGRAM_HEADER_SIZE;

        memcpy(cur_data, datagram->data, datagram->data_size);   // 数据体
        cur_data += datagram->data_size;

        EC_WRITE_U16(cur_data, 0x0000);  // WKC 清零
        cur_data += EC_DATAGRAM_FOOTER_SIZE;
    }

    // ★ 写入 EtherCAT 帧头
    EC_WRITE_U16(frame_data, ((cur_data - frame_data
                    - EC_FRAME_HEADER_SIZE) & 0x7FF) | 0x1000);  // Type=1
    // 填充到最小帧长
    while (cur_data - frame_data < ETH_ZLEN - ETH_HLEN)
        EC_WRITE_U8(cur_data++, 0x00);

        // ★ 发送帧
        ec_device_send(&master->devices[device_index], cur_data - frame_data);

        // 标记数据报为已发送
        list_for_each_entry_safe(...) {
            datagram->state = EC_DATAGRAM_SENT;
            datagram->jiffies_sent = jiffies_sent;
        }
    } while (more_datagrams_waiting);  // 如果帧满，再发新帧
}
```

- 拼接示意图

```
以太网帧 (ETH_DATA_LEN = 1500)
┌──────┬──────────────────────────────────────────────────────────────┐
│ 14B  │                      帧数据体 (最多1486B)                    │
├──────┼────┬────┬────┬────┬──────┬────┬────┬────┬──────┬────┬────┤
│ Eth  │Len │ D1 │ D1 │WKC│      │ D2 │ D2 │WKC│...│ pad  │ Dn │WKC │
│Header│+Flag│Hdr │Data│   │      │Hdr │Data│   │   │      │Hdr+Data│  │
│      │ 2B │10B │  XB│2B │      │10B │ YB │2B │   │      │10B+ZB│2B │
└──────┴────┴────┴────┴────┴──────┴────┴────┴────┴──────┴────┴────┘
       ↑
  0x1000 = EtherCAT 帧类型
  更多从站位: bit 15 of 长度字段
  工作计数器 WKC: 每数据报尾部 2B
```

- 调用 ndo_start_xmit 发送
- master/device.c

```
void ec_device_send(ec_device_t *device, size_t size)
{
    struct sk_buff *skb = device->tx_skb[device->tx_ring_index];

    skb->len = ETH_HLEN + size;   // 设置真实帧长度

    // ★ 调用 stmmac 驱动的 ndo_start_xmit
    if (device->dev->netdev_ops->ndo_start_xmit(skb, device->dev) == NETDEV_TX_OK) {
        device->tx_count++;
        // ...
    }
}
```

6. 接收报文

IgH通过调用ecrt_master_receive接口接收数据报文

```
用户程序 (RT 线程)
  |
  ▼
ecrt_master_receive(master)                              [master.c:2508]
  |
  ├── ① 逐设备轮询网卡
  |     for (dev_idx) {
  |       ec_device_poll(&master->devices[dev_idx])      [device.c:463]
  |         └ device->poll(device->dev)                   ← 注册的回调
  |              └ ec_poll(netdev)                       [main.c:6902]
  |                   └ for each DMA channel:
  |                       stmmac_rx(priv, budget, chan)   [main.c:5255]
  |                         |
  |                         |  检查 RX 描述符 OWN bit
  |                         |  OWN=0 → DMA 已完成, 帧已就绪
  |                         |
  |                         |  if (get_ecdev(priv)):
  |                         |    dma_sync_single_for_cpu()
  |                         |    ecdev_receive(va, buf1_len)  [device.c:621]
  |                         |      └ ec_master_receive_datagrams()  [master.c:1147]
  |                         |           |
  |                         |           |  解析 EtherCAT 帧头
  |                         |           |  逐数据报匹配 → memcpy 数据
  |                         |           |  读取 WKC → 标记 RECEIVED
  |                         |           |
  |                         |    dma_sync_single_for_device()  // 归还 buffer
  |                         |    continue  // 跳过 skb/XDP 处理
  |                         |
  |                         |  else:  标准内核 skb/XDP 路径
  |     }
  |
  └── ② 超时检测
        for each datagram in queue:
          if (SENT && jiffies_poll - jiffies_sent > timeout)
            → EC_DATAGRAM_TIMED_OUT
```

- 入口：ecrt_master_receive()
- master/master.c:2508-2557

```
int ecrt_master_receive(ec_master_t *master)
{
    unsigned int dev_idx;

    // ——— 阶段 1: 轮询收包 ———
    for (dev_idx = EC_DEVICE_MAIN; dev_idx < ec_master_num_devices(master);
            dev_idx++) {
        ec_device_poll(&master->devices[dev_idx]);    // ★ 触发硬件轮询
    }
    ec_master_update_device_stats(master);

    // ——— 阶段 2: 超时检测 ———
    list_for_each_entry_safe(datagram, next, &master->datagram_queue, queue) {
        if (datagram->state != EC_DATAGRAM_SENT) continue;

        if (master->devices[EC_DEVICE_MAIN].jiffies_poll -
                datagram->jiffies_sent > timeout_jiffies) {
            list_del_init(&datagram->queue);
            datagram->state = EC_DATAGRAM_TIMED_OUT;
            master->stats.timeouts++;
        }
    }
    return 0;
}
```

- ec_device_poll() → ec_poll()

- master/device.c:463-475

```
void ec_device_poll(ec_device_t *device)
{
    device->cycles_poll = get_cycles();   // 记录轮询时间戳 (cycles)
    device->jiffies_poll = jiffies;       // 记录轮询时间戳 (jiffies)
    device->poll(device->dev);             // ★ 调用驱动注册的 ec_poll
}
```

- stmmac/stmmac_main-6.1-ethercat.c:6902-6927

```
void ec_poll(struct net_device *netdev)
{
    struct stmmac_priv *priv = netdev_priv(netdev);
    int budget = 128;
    u32 maxq;

    if (!get_ecdev(priv)) return;
    maxq = max(priv->plat->rx_queues_to_use, priv->plat->tx_queues_to_use);

    // Watchdog: 每 2 秒 kick 一次
    if (jiffies - priv->ec_watchdog_jiffies >= 2 * HZ) {
        priv->ec_watchdog_jiffies = jiffies;
        irq_work_queue(&priv->ec_watchdog_kicker);
    }

    // 遍历所有 DMA 通道
    for (i = 0; i < maxq; i++) {
        stmmac_rx(priv, budget, chan);        // ★ 收包
        stmmac_tx_clean(priv, budget, chan);  // ★ 清理 TX
    }
}
```

- ecdev_receive() → ec_master_receive_datagrams()
- master/device.c:621-654

```
void ecdev_receive(ec_device_t *device, const void *data, size_t size)
{
    const void *ec_data = data + ETH_HLEN;  // 跳过以太网头
    size_t ec_size = size - ETH_HLEN;

    // 统计
    device->rx_count++;
    device->rx_bytes += size;

    // ★ 解析 EtherCAT 帧，匹配数据报
    ec_master_receive_datagrams(device->master, device, ec_data, ec_size);
}
```

- ec_master_receive_datagrams() — 帧解析与数据报匹配
- master/master.c:1147-1276

```
void ec_master_receive_datagrams(
        ec_master_t *master, ec_device_t *device,
        const uint8_t *frame_data, size_t size)
{
    // ——— ① 校验帧头 ———
    if (unlikely(size < EC_FRAME_HEADER_SIZE)) {
        master->stats.corrupted++;
        return;
    }

    // ——— ② 读取 EtherCAT 帧长度 ———
    frame_size = EC_READ_U16(cur_data) & 0x07FF;
    cur_data += EC_FRAME_HEADER_SIZE;

    if (unlikely(frame_size > size)) {
        master->stats.corrupted++;
        return;
    }

    // ——— ③ 逐数据报解析 ———
    cmd_follows = 1;
    while (cmd_follows) {
        // 读取数据报头
        datagram_type  = EC_READ_U8 (cur_data);       // 命令类型
        datagram_index = EC_READ_U8 (cur_data + 1);   // 索引号
        data_size      = EC_READ_U16(cur_data + 6) & 0x07FF;  // 数据长度
        cmd_follows    = EC_READ_U16(cur_data + 6) & 0x8000;   // "more follows" bit
        cur_data += EC_DATAGRAM_HEADER_SIZE;

        // ——— ④ 匹配队列中的数据报 ———
        matched = 0;
        list_for_each_entry(datagram, &master->datagram_queue, queue) {
            if (datagram->index == datagram_index
                && datagram->state == EC_DATAGRAM_SENT    // ← 必须是已发送
                && datagram->type == datagram_type
                && datagram->data_size == data_size) {
                matched = 1;
                break;
            }
        }

        if (!matched) {
            master->stats.unmatched++;
            cur_data += data_size + EC_DATAGRAM_FOOTER_SIZE;
            continue;             // 跳过未匹配的
        }

        // ——— ⑤ 拷贝数据 (写命令除外) ———
        if (datagram->type != EC_DATAGRAM_APWR &&
            datagram->type != EC_DATAGRAM_FPWR &&
            datagram->type != EC_DATAGRAM_BWR &&
            datagram->type != EC_DATAGRAM_LWR) {
            memcpy(datagram->data, cur_data, data_size);
        }
        cur_data += data_size;

        // ——— ⑥ 读取 Working Counter ———
        datagram->working_counter = EC_READ_U16(cur_data);
        cur_data += EC_DATAGRAM_FOOTER_SIZE;

        // ——— ⑦ 标记完成 ———
        datagram->state = EC_DATAGRAM_RECEIVED;
        datagram->cycles_received = device->cycles_poll;
        datagram->jiffies_received = device->jiffies_poll;
        list_del_init(&datagram->queue);  // 从队列移除
    }
}
```

7. 从站扫描流程

当我们启动主栈后可使用ethercat命令行工具查看从站的的相关信息，比如： ethercat slave

```
0  0:0  PREOP  +  CoolDrive RC
1  0:1  PREOP  +  CoolDrive RC
2  0:2  PREOP  +  CoolDrive RC
3  0:3  PREOP  +  CoolDrive RC
4  0:4  PREOP  +  CoolDrive RC
5  0:5  PREOP  +  CoolDrive RC
```

可以查看到当前总线上连接的从站的基本信息，ethercat命令行还有更多的功能我们后面介绍。

- 整体扫描机制概述

扫描过程由 **主站 FSM**（`fsm_master.c`）驱动，通过 idle 线程每周期调用 `ec_fsm_master_exec()` 推进状态。核心分为三个阶段：

```
阶段 1: 广播探测      → 发现总线上有多少从站响应
阶段 2: 清除地址      → 清除所有从站地址 + DC 延迟测量
阶段 3: 逐站扫描      → 对每个从站分配地址、读状态、读基础数据、读 SII
```

- 总览状态机流转图

```
ec_fsm_master_state_start                    [fsm_master.c:220]
  |
  |  发送广播读命令 (BRD) 到 0x0130 (AL Status)
  |  → 获取 slaves_responding = WKC
  |
  ▼
ec_fsm_master_state_broadcast                [fsm_master.c:278]
  |
  |  if (slaves_responding 变化)
  |    → rescan_required = 1
  |    → kmalloc(slaves, count)
```

```
   |    → ec_slave_init() × N
   |    → ec_fsm_master_enter_clear_addresses()
   |
   ├──▶ ec_fsm_master_state_clear_addresses   [fsm_master.c:844]
   |       |
   |       | 广播写命令 (BWR) 到 0x0010 → 清零所有站地址
   |       |
   |       ▼
   |    ec_fsm_master_state_dc_measure_delays  [fsm_master.c:889]
   |       |
   |       | 广播写命令 (BWR) 到 0x0900 → 使能从站 DC 延迟测量
   |       |
   |       ▼
   |    ec_fsm_master_state_scan_slave         [fsm_master.c:944]
   |       |
   |       |  逐从站扫描:
   |       |  ┌── ec_fsm_slave_scan_state_start    分配站地址 (APWR 0x0010)
   |       |  ├── ec_fsm_slave_scan_state_address  确认地址写入
   |       |  ├── ec_fsm_slave_scan_state_state    读 AL 状态 (FPRD 0x0130)
   |       |  ├── ec_fsm_slave_scan_state_base     读基础数据 (FPRD 0x0000)
   |       |  ├── ec_fsm_slave_scan_state_dc_cap   读 DC 能力 (FPRD 0x0910)
   |       |  ├── ec_fsm_slave_scan_state_datalink 检测端口连接
   |       |  ├── ec_fsm_slave_scan_state_sii_size 读 SII 大小
   |       |  ├── ec_fsm_slave_scan_state_sii_data 逐字读 SII 内容
   |       |  ├── ec_fsm_slave_scan_state_preop    请求 PREOP 状态
   |       |  ├── ec_fsm_slave_scan_state_sync     配置 Sync Manager
   |       |  ├── ec_fsm_slave_scan_state_pdos     配置 PDO
   |       |  └── ec_fsm_slave_scan_state_end      扫描完成
   |       |
   |       |  slave++ → 扫描下一个从站
   |       |
   |       ▼ (所有从站扫描完毕)
   |    ec_master_attach_slave_configs()
   |    ec_master_calc_dc()
   |    ec_master_eoe_start()
   |
  ▼
ec_fsm_master_state_read_state              [fsm_master.c:752]
  |
  |  轮询各从站 AL 状态, 处理错误标志
  |
  ▼
ec_fsm_master_state_acknowledge             [fsm_master.c:806]
  |
  |  确认从站状态变更 (如有 ACK_ERR)
  |
  ▼
ec_fsm_master_state_configure_slave         (action_configure)
  |
  |  将从站推进到目标状态 (PREOP → SAFEOP → OP)
  |
  ▼
(循环回 broadcast, 周期性监测 topology 变化)
```

- STATE_START — 触发扫描
- master/fsm_master.c:220

```
void ec_fsm_master_state_start(ec_fsm_master_t *fsm)
{
    // 发送广播读命令: 读 0x0130 (AL Status Register), 2 字节
    ec_datagram_brd(fsm->datagram, 0x0130, 2);
    ec_datagram_zero(fsm->datagram);
    fsm->datagram->device_index = fsm->dev_idx;
    fsm->retries = EC_FSM_RETRIES;
    fsm->state = ec_fsm_master_state_broadcast;
}
```

EtherCAT 原理： 广播读命令 (BRD) 会遍历所有从站，每个从站将自己的 AL 状态寄存器内容做 `OR` 操作。WKC 返回响应的从站数量（每个从站 WKC++）。

- STATE_BROADCAST — 发现从站
- master/fsm_master.c:278

```
void ec_fsm_master_state_broadcast(ec_fsm_master_t *fsm)
{
    // ★ 关键: WKC = 响应从站数
    if (datagram->working_counter != fsm->slaves_responding[fsm->dev_idx]) {
        fsm->rescan_required = 1;                          // 标记需要扫描
        fsm->slaves_responding[fsm->dev_idx] = datagram->working_counter;
    }

    // 所有设备都广播完成后...
    if (fsm->rescan_required) {
        // kmalloc 分配 slave 数组
        master->slaves = kmalloc(sizeof(ec_slave_t) * count, GFP_KERNEL);

        // 初始化每个从站
        for (i = 0; i < count; i++) {
            ec_slave_init(slave, master, dev_idx, ring_position, i+1);
        }
        master->slave_count = count;

        // ★ 进入清除地址阶段
        ec_fsm_master_enter_clear_addresses(fsm);
    }
}
```

- STATE_CLEAR_ADDRESSES — 清除站地址
- master/fsm_master.c:844

```
// 进入前: 广播写命令 (BWR) 到 0x0010 (Station Address)
ec_datagram_bwr(fsm->datagram, 0x0010, 2);
EC_WRITE_U16(fsm->datagram->data, 0x0000);  // 写 0 → 清零所有站地址

// 完成后: DC 延迟测量
ec_datagram_bwr(datagram, 0x0900, 1);       // 使能从站 DC 延迟测量
fsm->state = ec_fsm_master_state_dc_measure_delays;
```

扫描前必须将所有从站地址归零。后续按 `ring_position` 顺序逐站分配新地址。

- STATE_SCAN_SLAVE — 逐站扫描（核心）
- master/fsm_master.c:944

```
void ec_fsm_master_state_scan_slave(ec_fsm_master_t *fsm)
{
    // 执行从站扫描子状态机 (每周期推进一步)
    if (ec_fsm_slave_scan_exec(&fsm->fsm_slave_scan)) {
        return;  // 子状态机仍在运行, 等待下一周期
    }

    // 当前从站扫描完成 → 扫描下一个
    fsm->slave++;
    master->scan_index++;

    if (fsm->slave < master->slaves + master->slave_count) {
        ec_fsm_slave_scan_start(&fsm->fsm_slave_scan, fsm->slave);
        ec_fsm_slave_scan_exec(&fsm->fsm_slave_scan);
        return;  // 继续扫描下一个
    }

    // ★ 所有从站扫描完成
    EC_MASTER_INFO(master, "Bus scanning completed in %lu ms.\n", ...);
    master->scan_busy = 0;
    wake_up_interruptible(&master->scan_queue);

    ec_master_attach_slave_configs(master);  // 绑定应用配置
    ec_master_calc_dc(master);               // 计算 DC 时钟拓扑
    ec_master_eoe_start(master);             // 启动 EoE 处理
}
```

20.3 IgH 命令行工具使用

1. EtherCAT命令格式

IgH EtherCAT 命令行工具完整参考

> 工具名称: `ethercat`
>
> 调用格式: `ethercat <COMMAND> [OPTIONS] [ARGUMENTS]`
>
> 命令名支持缩写，只要不产生歧义即可。

> 例如：ethercat 可以缩写 为 ether ， 只要和系统别的命令不冲突就可以

- 全局选项

选项	含义
`--master -m <master>`	选择主站。逗号分隔，支持范围。例: `1,3`, `5-7,9`, `-3`。默认: `-`（全部）
`--force -f`	强制执行命令
`--quiet -q`	减少输出信息
`--verbose -v`	输出更多信息
`--help -h`	显示帮助信息

- 从站选择（适用大多数涉及从站的命令）

选项	含义
`--alias -a <alias>`	按 alias 地址选择从站
`--position -p <pos>`	按位置选择从站

选择规则：

1. 两者都不指定 → 选择所有从站
2. 只指定 `-p` → 按绝对环位位置匹配
3. 只指定 `-a` → 匹配该 alias 及其后续从站（直到遇到不同的 alias）
4. 两者都指定 → `-p` 解释为相对于 alias 从站的偏移

- 数据类型（`--type -t <type>`）

`bool`, `int8`, `int16`, `int32`, `int64`, `uint8`, `uint16`, `uint32`, `uint64`, `float`, `double`, `string`, `octet_string`, `unicode_string`, `sm8`, `sm16`, `sm32`, `sm64`

---

2. alias 设置从站地址

命令格式:

```
ethercat alias [OPTIONS] <ALIAS>
```

命令解释:

向从站的 SII（Slave Information Interface）写入 alias 地址。alias 地址是一个 16 位的无符号整数，用于在总线上唯一标识从站，方便后续按名称引用。设置为 0 表示移除 alias。

由于该操作直接修改 SII 中的非易失性存储，选择多个从站时需要 `--force` 确认，防止误操作。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--force -f`

命令示例:

```
# 将从站 0 的 alias 设置为 0x1000
$ ethercat alias -p 0 0x1000

# 移除从站 5 的 alias（设为 0）
$ ethercat alias -p 5 0

# 将所有从站的 alias 设为 0x2000
$ ethercat alias -f 0x2000
```

---

3. config 显示配置

命令格式:

```
ethercat config [OPTIONS]
```

命令解释:

显示主站的从站配置信息，用于验证应用层配置是否与物理总线一致。

- 非 verbose 模式每行显示一个配置: alias:相对位置、期望的 VendorID/ProductCode、已连接从站的位置、当前应用层状态。
- verbose 模式额外显示已配置的 PDO、SDO、IDN、特征标志、EoE IP 参数和 DC 配置。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--verbose -v`

命令示例:

```
# 列出所有配置（简洁模式）
$ ethercat config
1001:0  0x00000002/0x044d2c52  0  OP
1002:0  0x00000002/0x07d23052  1  OP

# 查看某个配置的完整信息
$ ethercat config -v -a 1001 -p 0
```

---

4. crc 诊断错误

命令格式:

```
ethercat crc
ethercat crc reset
```

命令解释:

诊断 EtherCAT 总线的 CRC 错误，显示每个从站每个端口的三个关键寄存器：

- **CRC**: CRC 错误计数器（寄存器 0x300, 0x302, 0x304, 0x306）——传输数据校验错误
- **PHY**: 物理接口错误计数器（寄存器 0x301, 0x303, 0x305, 0x307）——物理层错误
- **FWD**: 转发接收错误计数器（寄存器 0x308, 0x309, 0x30a, 0x30b）——转发帧错误
- **NXT**: 用箭头（↑↓）指示下一个从站方向

`reset` 子命令将全部从站的所有 CRC 错误计数器清零，用于开始新一轮监测。

命令示例:

```
# 查看 CRC 状态（排查通信质量问题）
$ ethercat crc
   |Port 0         |Port 1         |Port 2         |Port 3         |
   |CRC PHY FWD NXT|CRC PHY FWD NXT|CRC PHY FWD NXT|CRC PHY FWD NXT|
  0|  0   0   0    |  0   0   0   ↓|               |               |EK1100
  1|  0   0   0   ↑|  0   0   0   ↓|               |               |EL3162
  2|  0   0   0   ↑|               |               |               |EL2008


# 清零所有从站的 CRC 计数器
$ ethercat crc reset
```

---

5. cstruct 生成C结构体

命令格式:

```
ethercat cstruct [OPTIONS]
```

命令解释:

根据所选从站的 PDO 信息，自动生成 C 语言结构体定义。输出包含：

- `ec_pdo_entry_info_t` 数组 —— 每个 PDO 条目的 index、subindex、bit_length
- `ec_pdo_info_t` 数组 —— 每个 PDO 的 index、条目数量、指向条目数组的指针
- `ec_sync_info_t` 数组 —— Sync Manager 配置（方向、看门狗等）

生成的代码可直接用于应用层的 `ecrt_slave_config_pdos()` 函数调用，避免手工编写冗长的 PDO 配置代码。

选项: `--alias -a <alias>`, `--position -p <pos>`

命令示例:

```
# 为从站 0 生成 C 结构体
$ ethercat cstruct -p 0
/* Master 0, Slave 0, "EL3162"
 * Vendor ID:       0x00000002
 * Product code:    0x044d2c52
 * Revision number: 0x00140000
 */

ec_pdo_entry_info_t slave_0_pdo_entries[] = {
    {0x3101, 0x01, 16}, /* Status */
    {0x3101, 0x02, 16}, /* Value */
```

```
    {0x3101, 0x03, 16}, /* Value */
    {0x3101, 0x04, 16}, /* Value */
};

ec_pdo_info_t slave_0_pdos[] = {
    {0x1a00, 4, slave_0_pdo_entries + 0}, /* Channel1 */
};

ec_sync_info_t slave_0_syncs[] = {
    {0, EC_DIR_OUTPUT, 0, NULL, EC_WD_DISABLE},
    {1, EC_DIR_INPUT, 0, NULL, EC_WD_DISABLE},
    {2, EC_DIR_OUTPUT, 1, slave_0_pdos + 0, EC_WD_DISABLE},
    {3, EC_DIR_INPUT, 1, slave_0_pdos + 0, EC_WD_DISABLE},
    {0xff}
};
```

---

6. data 输出域的过程数据

命令格式:

```
ethercat data [OPTIONS]
```

命令解释:

以二进制原始格式输出域的过程数据。

数据直接写入标准输出，多个域的数据按顺序拼接。该命令主要用于将过程数据管道传输给其他工具（如 `xxd` 查看十六进制），或在脚本中捕获实时过程数据。

数据处理由应用层负责，此命令仅做透明的数据转发。

选项: `--domain -d <index>` （省略则输出所有域）

命令示例:

```
# 输出所有域的过程数据并用 xxd 查看
$ ethercat data | xxd
00000000: 0000 0000 0000 0000 0000 0000            ............

# 仅输出 Domain0 的数据到文件
$ ethercat data -d 0 > process_data.bin
```

---

7. debug 调试级别

命令格式:

```
ethercat debug <LEVEL>
```

命令解释:

设置主站的运行时调试级别，调试信息输出到 syslog（`/var/log/syslog` 或 `dmesg`）。

- **0**: 关闭调试输出
- **1**: 基本调试信息（状态变化、错误等）
- **2**: 打印所有帧的完整内容（极大量输出，会严重影响性能，仅在开发调试时使用且需立即关闭）

此设置在模块卸载后丢失。

命令示例:

```
# 开启基本调试
$ ethercat debug 1

# 关闭调试
$ ethercat debug 0

# 打印完整帧内容（谨慎！仅在排查底层协议问题时短暂开启）
$ ethercat debug 2
```

---

8. domains 显示过程数据域

命令格式:

```
ethercat domains [OPTIONS]
```

命令解释:

显示已配置的过程数据域（domain）的运行时信息。每个域包含：

- **LogBaseAddr**: 逻辑数据报的基地址（LRD/LWR/LRW 寻址用）
- **Size**: 域的过程数据总字节数
- **WorkingCounter**: `当前值/期望值`。两者相等说明上一周期所有 PDO 交换成功；不等则说明有从站未响应。

verbose 模式额外显示参与该域的每个从站 FMMU 配置以及当前过程数据的十六进制内容。

选项: `--domain -d <index>`, `--verbose -v`

命令示例:

```
# 列出所有域及其 WorkingCounter（检查通信健康状态）
$ ethercat domains
Domain0: LogBaseAddr 0x00000000, Size   6, WorkingCounter 3/3
Domain1: LogBaseAddr 0x00000006, Size  12, WorkingCounter 3/3

# 详细查看域（含 FMMU 和当前过程数据）
$ ethercat domains -v
Domain0: LogBaseAddr 0x00000000, Size   6, WorkingCounter 3/3
  SlaveConfig 1001:0, SM2 (Output), LogAddr 0x00000000, Size 6
    00 00 00 00 00 00
Domain1: LogBaseAddr 0x00000006, Size  12, WorkingCounter 3/3
  SlaveConfig 1001:0, SM3 ( Input), LogAddr 0x00000006, Size 12
    00 00 00 00 00 00 00 00 00 00 00 00

# 仅查看 Domain0 的详细信息
$ ethercat domains -d 0 -v
```

---

9. download 向从站写入数据

命令格式:

```
ethercat download [OPTIONS] <INDEX> <SUBINDEX> <VALUE>
ethercat download [OPTIONS] <INDEX> <VALUE>
```

命令解释:

通过 SDO（Service Data Object）协议向从站写入数据。这是 CoE（CAN over EtherCAT）协议的核心操作，用于配置从站参数。

- **第一种形式（3 个参数）**：普通 SDO 访问，指定 Index + Subindex + 数值
- **第二种形式（2 个参数）**：Complete Access 方式，一次性写入整个 SDO 对象

数据类型默认为 SDO 字典中定义的类型；若从站不支持 SDO 信息服务，则 `--type` 为强制参数。`VALUE` 为 `-` 时从标准输入读取数据。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`

命令示例:

```
# 写入 Modes of Operation = 8 (CSP 模式)
$ ethercat download -p 0 0x6060 0 8 --type uint8

# 写入十六进制值
$ ethercat download -p 0 0x1018 1 0x00000002 --type uint32

# 写入字符串（设备名称）
$ ethercat download -p 0 0x1008 0 "MyEtherCATDevice" --type string

# 从标准输入写入数据
$ echo -n "hello" | ethercat download -p 0 0x2000 0 - --type octet_string
```

---

10. eoe 统计信息

命令格式:

```
ethercat eoe [OPTIONS]
```

命令解释:

显示 Ethernet over EtherCAT (EoE) 虚拟网络接口的运行时统计信息。EoE 允许在 EtherCAT 总线上传输标准以太网帧，使从站支持 TCP/IP 等高层协议。输出每行一个 EoE 接口，包含：接口名称、关联从站位置、链路状态、收发字节数/速率、发送队列使用率。需编译时启用 `EC_EOE` 支持。

命令示例:

```
# 查看所有 EoE 接口的流量统计
$ ethercat eoe
Interface  Slave  State  RxBytes  RxRate  TxBytes  TxRate  TxQueue
eoe0           0     up     1234     100     5678      50    0/256
eoe1           3     up     2345     200     9876     100    0/256
```

---

11. foe_read 读文件

命令格式:

```
ethercat foe_read [OPTIONS] <SOURCEFILE>
```

命令解释:

通过 FoE（File access over EtherCAT）协议从从站读取文件。FoE 类似于 TFTP 协议，用于固件更新、配置文件读取等场景。`SOURCEFILE` 是在从站上存储的文件名。默认输出到标准输出，可通过 `--output-file` 指定本地保存路径。需要选择单个从站。

选项: `--output-file -o <file>`, `--alias -a <alias>`, `--position -p <pos>`

命令示例:

```
# 从从站读取文件并输出到标准输出
$ ethercat foe_read -p 0 firmware.bin

# 读取并保存到本地文件
$ ethercat foe_read -p 0 -o local_firmware.bin firmware.bin

# 按 alias 读取
$ ethercat foe_read -a 1001 -o config.xml device_config.xml
```

---

12. foe_write 写文件

命令格式:

```
ethercat foe_write [OPTIONS] <FILENAME>
```

命令解释:

通过 FoE（File access over EtherCAT）协议将本地文件写入从站。常用于固件更新、配置文件下发等。`FILENAME` 为本地文件路径，`-` 表示从标准输入读取。若 `FILENAME` 为 `-`，则必须用 `--output-file` 指定从站端的目标文件名；否则默认使用本地文件的 basename。需要选择单个从站。

选项: `--output-file -o <file>`, `--alias -a <alias>`, `--position -p <pos>`

命令示例:

```
# 将本地固件文件写入从站（目标文件名用本地文件名的 basename）
$ ethercat foe_write -p 0 new_firmware.bin

# 指定从站端目标文件名
$ ethercat foe_write -p 0 -o /firmware/app.bin my_firmware.bin

# 从标准输入写入
$ cat firmware.bin | ethercat foe_write -p 0 -o firmware.bin -
```

---

13. graph 总线拓扑图

命令格式:

```
ethercat graph [OPTIONS] [INFO]
```

命令解释:

以 DOT 语言（Graphviz 工具的标准输入格式）输出 EtherCAT 总线的拓扑图。可选的 `INFO` 参数在图中附加信息：

- **（无参数）**: 仅基础拓扑，含节点位置/alias 和端口类型
- **DC**: 附加分布式时钟延迟信息和 DC 类型
- **CRC**: 附加各端口的 CRC/FWD 错误计数值

输出可用 Graphviz 工具链（`dot`, `neato` 等）渲染为 SVG、PNG、PDF 等格式。

命令示例:

```
# 生成基础拓扑图
$ ethercat graph
strict graph bus {
    rankdir="LR"
    ...
    master [label="EtherCAT\nMaster"]
    master -- slave0[label="EBUS"]
    slave0 [shape="box",label="0 / 0:0\nEK1100"]
    slave1 [shape="box",label="1 / 0:1\nEL3162"]
    ...
}

# 生成带 DC 信息的拓扑图并渲染为 SVG
$ ethercat graph DC | dot -Tsvg > bus_dc.svg

# 生成带 CRC 错误信息的拓扑图并渲染为 PNG
$ ethercat graph CRC | dot -Tpng > bus_crc.png

# 直接生成基础拓扑的 SVG
$ ethercat graph | dot -Tsvg > bus.svg
```

---

14. ip 网络参数

命令格式:

```
ethercat ip [OPTIONS] <ARGS>
```

命令解释:

为从站的 EoE（Ethernet over EtherCAT）功能设置 IP 网络参数。参数以键值对形式出现（必须成对），支持的键为：

键	别名	值格式	说明
`ip_address`	`addr`	`<IPv4>[/prefix]`	IP 地址，可选子网前缀
`mac_address`	`link`	`<MAC>`	MAC 地址（冒号或连字符分隔）
`default_gateway`	`default`	`<IPv4>`	默认网关
`dns_address`	`dns`	`<IPv4>`	DNS 服务器
`hostname`	`name`	`<hostname>`	主机名（最长 32 字节）

IPv4 地址支持点分十进制或主机名（自动 DNS 解析）。需编译时启用 `EC_EOE`。需要选择单个从站。

命令示例:

```
# 完整 IP 配置
$ ethercat ip -p 0 \
    ip_address 192.168.1.100/24 \
    default_gateway 192.168.1.1 \
    dns_address 8.8.8.8 \
    hostname my_ethercat_device

# 使用缩写参数名
$ ethercat ip -p 3 addr 172.16.0.10/24 default 172.16.0.1

# 仅设置 MAC 和 IP
$ ethercat ip -p 0 link 00:11:22:33:44:55 addr 10.0.0.50/16
```

---

15. master 主站信息

命令格式:

```
ethercat master [OPTIONS]
```

命令解释:

显示 EtherCAT 主站的综合状态信息，包括：

- **Phase**: 主站运行阶段 —— Waiting for devices / Idle / Operation
- **Active**: 主站是否激活
- **Slaves**: 已扫描到的从站数量
- **Ethernet devices**: 每个网络设备的 MAC 地址、连接状态、链路状态、收发帧数/字节数/速率/错误数
- **Common**: 所有设备的综合统计 —— 收发帧/字节/速率、丢失帧数、帧丢失率
- **Distributed Clocks**: 参考时钟从站、DC 参考时间、应用时间

用 `-m` 选择具体主站。在多主站系统中默认显示全部。

选项: `--master -m <indices>`

命令示例:

```
# 查看主站状态
$ ethercat master
Master0
  Phase: Operation
  Active: yes
  Slaves: 3
  Ethernet devices:
    Main: 00:0e:8c:a3:77:9e (attached)
      Link: UP
      Tx frames:   271828
      Tx bytes:    47382910
      Rx frames:   271827
      Rx bytes:    47382908
      Tx errors:   0
      Tx frame rate [1/s]: 1000 1000 1000
      Tx rate [KByte/s]:   156.2 156.2 156.2
      Rx frame rate [1/s]: 1000 1000 1000
      Rx rate [KByte/s]:   156.2 156.2 156.2
    Common:
      Tx frames:   271828
      Tx bytes:    47382910
      Rx frames:   271827
      Rx bytes:    47382908
      Lost frames: 0
      Frame loss [%]: 0.0 0.0 0.0
  Distributed clocks:
    Reference clock:   Slave 0
    DC reference time: 0
    Application time:  1692345678000000000
                       2023-08-18 09:21:18.000000000

# 指定查看主站 1
$ ethercat master -m 1
```

16. pdos 列出pdo关系

命令格式:

```
ethercat pdos [OPTIONS]
```

命令解释:

列出所选从站的 Sync Manager 配置、PDO 分配和 PDO 映射关系。输出为三层缩进结构：

1. **Sync Manager 层**: SM 编号、物理起始地址、默认大小、控制寄存器值、是否启用
2. **PDO 分配层**: PDO 方向（Tx/Rx，以从站视角）、十六进制索引、PDO 名称
3. **PDO 映射层**: 每个条目的 index:subindex、位长度、描述

通过 `--skin etherlab` 可生成 EtherLab 的 Matlab/Octave 配置模板。注意：PDO 信息可能来自 SII 或 CoE 通信区，取决于从站的配置方式。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--skin -s <skin>` (`default` / `etherlab`)

命令示例:

```
# 查看从站 0 的 PDO 配置
$ ethercat pdos -p 0
SM0: PhysAddr 0x1000, DefaultSize    0, ControlRegister 0x26, Enable 1
SM1: PhysAddr 0x1080, DefaultSize    0, ControlRegister 0x22, Enable 1
SM2: PhysAddr 0x1100, DefaultSize    0, ControlRegister 0x24, Enable 1
  RxPDO 0x1600 "Channel1"
  PDO entry 0x7000:01, 16 bit, "Output"
SM3: PhysAddr 0x1140, DefaultSize    0, ControlRegister 0x20, Enable 1
  TxPDO 0x1a00 "Channel1"
  PDO entry 0x6000:01, 16 bit, "Status"
  PDO entry 0x6000:02, 16 bit, "Value"

# 生成 EtherLab 风格模板
$ ethercat pdos -p 0 -s etherlab

# 查看所有从站的 PDO
$ ethercat pdos
```

17. reg_read 读寄存器

命令格式:

```
ethercat reg_read [OPTIONS] <ADDRESS> [SIZE]
```

命令解释:

直接读取从站的物理寄存器（ESC 寄存器空间）。

`ADDRESS` 为 16 位寄存器偏移地址，`SIZE` 是要读取的字节数。若指定了有固定字节大小的数据类型（如 `uint16`），则 `SIZE` 可省略，工具自动使用类型对应的大小。

ADDRESS + SIZE 不能超过 0xFFFF（64KB）。输出格式取决于 `--type` 参数（十六进制+十进制、字符串等）。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`

命令示例:

```
# 读取从站 0 的 AL 状态寄存器 (0x0130, 2 字节)
$ ethercat reg_read -p 0 0x0130 2 --type uint16
0x0008 8

# 读取 ESC 类型寄存器 (0x0000)
$ ethercat reg_read -p 0 0x0000 2 --type uint16
0x0011 17

# 读取原始十六进制字节（无数据类型）
$ ethercat reg_read -p 0 0x0110 2
0x01 0x02

# 读取 64 字节并用 xxd 分析
$ ethercat reg_read -p 0 0x0000 64 | xxd
```

18. reg_write 写寄存器

命令格式:

```
ethercat reg_write [OPTIONS] <ADDRESS> <DATA>
```

命令解释:

向从站的物理寄存器（ESC 寄存器空间）写入数据。

- **有 `--type` 时**: `DATA` 按指定的数据类型解析（如 `100` 表示十进制数值、`0x64` 表示十六进制）
- **无 `--type` 时**: `DATA` 为文件路径；`-` 表示从标准输入读取

ADDRESS + SIZE 不能超过 0xFFFF。`--emergency` 标志使写入以紧急请求方式发送（用于处理某些从站在非 OP 状态时的通信限制）。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`, `--emergency -e`

命令示例:

```
# 以 uint8 类型写入寄存器
$ ethercat reg_write -p 0 0x0300 0 --type uint8

# 从文件写入寄存器数据
$ ethercat reg_write -p 0 0x0400 register_data.bin

# 从标准输入写入（例如写入 16 字节的 0x00）
$ dd if=/dev/zero bs=16 count=1 2>/dev/null | ethercat reg_write -p 0 0x0300 -

# 以紧急请求方式写入（用于无法通过正常方式通信的从站）
$ ethercat reg_write -p 0 -e 0x0130 0x01 --type uint8
```

19. rescan 重新扫描

命令格式:

```
ethercat rescan
```

命令解释:

触发 EtherCAT 总线重新扫描。主站将遗忘此前收集的所有从站信息，重新扫描总线并重新读取各从站的 SII 数据。常用于：

- 物理添加/移除从站后刷新拓扑
- 从站固件更新后重新识别
- 总线故障恢复后重新建立通信

不需要任何参数，作用于所有主站。

命令示例:

```
# 重新扫描总线
$ ethercat rescan

# 扫描后立即查看更新后的从站列表
$ ethercat rescan && ethercat slaves
0  0:0  PREOP  +  EK1100 EtherCAT Coupler (2A E-Bus)
1  0:1  PREOP  +  EL3162 2C. Ana. Input 0-10V
2  0:2  PREOP  +  EL2008 8C. Dig. Output 24V, 0.5A
```

---

20. sdos 列出SDO字典

命令格式:

```
ethercat sdos [OPTIONS]
```

命令解释:

列出所选从站的 SDO（Service Data Object）字典。两层缩进显示：

1. **SDO 层**: SDO 的十六进制 index 和名称
2. **SDO 条目层**: index:subindex、访问权限（按 PREOP、SAFEOP、OP 顺序，`r`=可读/`w`=可写/`-`=不可访问）、数据类型、位长度、描述

`--quiet` 模式仅输出 SDO 列表，跳过所有条目，适合快速浏览。SDO 字典信息来自从站的 CoE 对象字典，仅当从站支持 SDO Info 服务时可用。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--quiet -q`

命令示例:

```
# 列出从站 0 的完整 SDO 字典
$ ethercat sdos -p 0
SDO 0x1000, "Device type"
  0x1000:00, r-r-r-, uint32, 32 bit, "Device type"
SDO 0x1008, "Device name"
  0x1008:00, r-r-r-, string, 144 bit, "Device name"
SDO 0x1018, "Identity object"
  0x1018:01, r-r-r-, uint32, 32 bit, "Vendor id"
  0x1018:02, r-r-r-, uint32, 32 bit, "Product code"
  0x1018:03, r-r-r-, uint32, 32 bit, "Revision number"
  0x1018:04, r-r-r-, uint32, 32 bit, "Serial number"
SDO 0x6060, "Modes of operation"
  0x6060:00, rwrwrw, int8, 8 bit, "Modes of operation"
...

# 仅列出 SDO 对象（不展开条目，快速浏览）
$ ethercat sdos -p 0 -q
SDO 0x1000, "Device type"
SDO 0x1008, "Device name"
SDO 0x1018, "Identity object"
SDO 0x6060, "Modes of operation"
...
```

---

21. sii_read 读从站

命令格式:

```
ethercat sii_read [OPTIONS]
```

命令解释:

读取从站 SII（Slave Information Interface，即 EEPROM）的全部内容。SII 存储了从站的标识信息、Sync Manager 配置、PDO 映射、DC 配置等关键参数。

- **非 verbose**: 输出原始二进制数据（可直接管道到文件做备份）
- **verbose**: 输出分类组织的文本格式，每类有名称和十六进制内容。SII 类别包括 STRINGS、DataTypes、General、FMMU、SyncM、TXPDO、RXPDO、DC

需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--verbose -v`

命令示例:

```
# 输出原始 SII 二进制数据并保存
$ ethercat sii_read -p 0 > slave0_sii_backup.bin

# 以详细分类格式查看 SII 内容
$ ethercat sii_read -p 0 -v
SII Area:
  03 00 00 00 02 00 00 00 00 00 00 00 00 00 00 00
  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  ...
SII Category 0x000a (STRINGS), 10 words
  45 4c 33 31 36 32 00 00 00 00 00 00 00 00 00 00
  00 00 00 00
SII Category 0x001e (General), 48 words
  00 00 00 00 4d 2c 00 00 14 00 00 00 00 00 00 00
  ...
```

---

22. sii_write 写从站

命令格式:

```
ethercat sii_write [OPTIONS] <FILENAME>
```

命令解释:

向从站的 SII EEPROM 写入完整的 SII 映像数据。此操作为非易失性写入（修改从站 EEPROM 内容），通常在以下场景使用：

- 恢复从站的 SII 备份
- 更新从站设备描述信息
- 修复损坏的 SII 数据

写入前自动进行完整性验证（CRC 校验、类别结构遍历），若不通过会拒绝写入；可用 `--force` 跳过验证（慎用）。`FILENAME` 为 `-` 时从标准输入读取数据。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--force -f`

命令示例:

```
# 将备份的 SII 数据恢复到从站
$ ethercat sii_write -p 0 slave0_sii_backup.bin

# 从标准输入写入
$ cat sii_repaired.bin | ethercat sii_write -p 0 -

# 强制写入（跳过验证，慎用）
$ ethercat sii_write -p 0 -f modified_sii.bin
```

---

23. slaves 查看从站

命令格式:

```
ethercat slaves [OPTIONS]
```

命令解释:

显示当前总线上检测到的 EtherCAT 从站列表。这是最常用的诊断命令之一。

非 verbose 输出（每行一个从站）：

```
0  0:0  PREOP  +  EK1100 EtherCAT Coupler (2A E-Bus)
|  | |  |      |  |
|  | |  |      |  \- 设备名称（来自 SII）；若无则为 VendorID:ProductCode
|  | |  |      \- 错误标志: + = 正常, E = 扫描/配置失败
|  | |  \- 当前应用层状态: INIT / PREOP / SAFEOP / OP / BOOT
|  | \- 相对于上一个 alias 从站的偏移位置
|  \- alias 地址（若设置），或上一个 alias 从站的地址，或 0
\- 绝对环位位置
```

verbose 输出: 每个从站打印完整信息块，包括 Identity（Vendor ID、Product Code、Revision、Serial）、DL 信息（FMMU、DC 支持、端口状态）、Mailbox 协议支持情况、General 信息（设备名、CoE 详情、电流消耗等）。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--verbose -v`

命令示例:

```
# 列出所有从站（简洁模式）
$ ethercat slaves
0  0:0  PREOP  +  EK1100 EtherCAT Coupler (2A E-Bus)
1  0:1  PREOP  +  EL3162 2C. Ana. Input 0-10V
2  0:2  PREOP  +  EL2008 8C. Dig. Output 24V, 0.5A

# 仅查看从站 1 的详细信息
$ ethercat slaves -p 1 -v
=== Master 0, Slave 1 ===
Device: Main
State: PREOP
Flag: +
Identity:
  Vendor Id:       0x00000002
  Product code:    0x044d2c52
  Revision number: 0x00140000
  Serial number:   0x00000000
DL information:
  FMMU bit operation: no
  Distributed clocks: yes, 64 bit
  DC system time transmission delay: 100 ns
Port  Type  Link  Loop    Signal  NextSlave  RxTime [ns]  Diff [ns]  NextDc [ns]
   0  EBUS  up    open    yes     0          -            -          -
   1  EBUS  up    open    yes     2          12345600000  100        200
   2  N/A   down  open    no      -          -            -          -
   3  N/A   down  open    no      -          -            -          -
Mailboxes:
  Bootstrap RX: 0x0000/0, TX: 0x0000/0
  Standard  RX: 0x1000/128, TX: 0x1080/128
  Supported protocols: CoE, FoE
General:
  Group: Slice
  Image name: EL3162
  Order number: EL3162
  Device name: EL3162 2C. Ana. Input 0-10V
  CoE details:
    Enable SDO: yes
    Enable SDO Info: yes
    Enable PDO Assign: yes
    Enable PDO Configuration: yes
    Enable Upload at startup: no
    Enable SDO complete access: no
  Flags:
    Enable SafeOp: no
    Enable notLRW: no
  Current consumption: 0 mA

# 按 alias 过滤从站
$ ethercat slaves -a 1001
```

---

24. soe_read 读命令

命令格式:

```
ethercat soe_read [OPTIONS] <IDN>
ethercat soe_read [OPTIONS] <DRIVE> <IDN>
```

命令解释:

通过 SoE（Servo drive profile over EtherCAT）协议读取伺服驱动器的 IDN（Identification Number）。SoE 用于访问符合 IEC 61800-7-204 (SERCOS) 标准的伺服驱动器参数。

- **DRIVE**: 驱动号 0-7，省略时默认 0
- **IDN**: 支持两种格式：
  - **字符串格式**: `S-0-150`（标准数据）或 `P-0-150`（产品数据）
  - **数字格式**: 16 位无符号整数，Bit15=P/S 标志，Bit14-12=参数集，Bit11-0=数据块号

需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`

命令示例:

```
# 读取标准 IDN（默认 drive 0）
$ ethercat soe_read -p 0 S-0-150
0x0000 0

# 读取产品 IDN（P 前缀）
$ ethercat soe_read -p 0 0 P-0-150 --type uint16
0x0001 1

# 使用数字形式的 IDN（0x0096 = S-0-150）
$ ethercat soe_read -p 0 0 0x0096 --type uint32
0x00000000 0

# 读取字符串类型的 IDN（如驱动器名称）
$ ethercat soe_read -p 0 S-0-140 --type string
MyServoDrive
```

---

25. soe_write 写命令

命令格式:

```
ethercat soe_write [OPTIONS] <IDN> <VALUE>
ethercat soe_write [OPTIONS] <DRIVE> <IDN> <VALUE>
```

命令解释:

通过 SoE（Servo drive profile over EtherCAT）协议向伺服驱动器参数写入数据。

- **DRIVE**: 驱动号 0-7，省略时默认 0
- **IDN**: 字符串格式（`S-0-150` / `P-0-150`）或数字格式（16 位）
- **VALUE**: 要写入的值，按 `--type` 指定的类型解析

与 SDO 的 download 不同，此处的 `--type` 是强制参数（驱动器的 SoE 字典不会自动提供类型信息）。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`

命令示例:

```
# 写入标准 IDN（drive 0，参数值 100）
$ ethercat soe_write -p 0 0 S-0-150 100 --type uint16

# 写入产品 IDN
$ ethercat soe_write -p 0 0 P-0-150 200 --type uint32

# 写入字符串参数（如驱动器名称）
$ ethercat soe_write -p 0 0 S-0-140 "NewDriveName" --type string

# 使用数字形式 IDN
$ ethercat soe_write -p 0 0 0x0096 50 --type uint16
```

---

26. states 从站状态

命令格式:

```
ethercat states [OPTIONS] <STATE>
```

命令解释:

请求从站切换到指定的应用层（AL）状态。EtherCAT 定义了 5 种状态：

STATE	含义	说明
`INIT`	初始化	仅基础通信可用，无邮箱和过程数据
`PREOP`	预运行	邮箱通信可用（SDO），无过程数据
`BOOT`	引导	用于固件更新的特殊状态
`SAFEOP`	安全运行	过程数据输入可用，输出保持安全值
`OP`	运行	完全运行，输入输出过程数据均有效

从站必须按顺序（或跳过某些状态）完成状态迁移。实际的状态切换由主站协调完成。典型启动路径：INIT → PREOP → SAFEOP → OP。

选项: `--alias -a <alias>`, `--position -p <pos>`

命令示例:

```
# 先将所有从站初始化为 INIT
$ ethercat states INIT

# 再切换到 PREOP（此时可通信 SDO）
$ ethercat states PREOP

# 切换到 SAFEOP（过程数据输入开始更新）
$ ethercat states SAFEOP

# 切换到 OP（完全运行状态）
$ ethercat states OP

# 仅对某个从站操作
$ ethercat states -p 0 OP

# 进入 BOOT 状态（为固件更新做准备）
$ ethercat states -p 0 BOOT
```

---

27. upload 读从站

命令格式:

```
ethercat upload [OPTIONS] <INDEX> <SUBINDEX>
```

命令解释:

通过 SDO 协议从从站读取数据。这是 CoE 协议中的 SDO Upload 操作。

- `INDEX`: SDO 索引，16 位无符号整数（通常为十六进制）
- `SUBINDEX`: SDO 条目子索引，8 位无符号整数

数据类型默认从 SDO 字典中获取；若从站不支持 SDO Info 服务或 SDO 不在字典中，则 `--type` 为强制参数。输出格式取决于类型（数值类型显示十六进制+十进制，字符串直接输出等）。需要选择单个从站。

选项: `--alias -a <alias>`, `--position -p <pos>`, `--type -t <type>`

命令示例:

```
# 读取设备类型 (0x1000:00)
$ ethercat upload -p 0 0x1000 0 --type uint32
0x00020192 131474

# 读取设备名称 (0x1008:00)
$ ethercat upload -p 0 0x1008 0 --type string
EL3162

# 读取 Vendor ID (0x1018:01)
$ ethercat upload -p 0 0x1018 1 --type uint32
0x00000002 2

# 读取 Product Code (0x1018:02)
$ ethercat upload -p 0 0x1018 2 --type uint32
0x044d2c52 72100946

# 读取当前操作模式 (0x6061:00)
$ ethercat upload -p 0 0x6061 0 --type uint8
0x08 8

# 读取实际位置 (0x6064:00)
$ ethercat upload -p 0 0x6064 0 --type int32
0x00002710 10000
```

---

28. version 查看igh版本

命令格式:

```
ethercat version [OPTIONS]
```

命令解释:

显示 IgH EtherCAT Master 的版本号。版本号来自编译时定义的 `EC_MASTER_VERSION` 宏。

命令示例:

```
$ ethercat version
IgH EtherCAT master 1.6.0-rc1
```

---

29. xml文件

命令格式:

```
ethercat xml [OPTIONS]
```

命令解释:

根据所选从站的 ESI（EtherCAT Slave Information）数据，生成符合 **ETG.2000** 规范的 `EtherCATInfo` XML 描述文件。XML 包含从站的 Vendor ID、Product Code、SM 配置、PDO 分配与映射等完整信息。

- 选择**单个从站**时输出单个 `<EtherCATInfo>` 元素
- 选择**多个从站**时所有元素包裹在 `<EtherCATInfoList>` 中

生成的 XML 可用于 EtherCAT 配置工具（如 Beckhoff TwinCAT、EtherLab）的从站描述文件导入。注意：PDO 信息可能来自 SII 或 CoE 通信区，取决于从站当前配置状态。

选项: `--alias -a <alias>`, `--position -p <pos>`

命令示例:

```
# 生成单个从站的设备描述 XML
$ ethercat xml -p 0
<?xml version="1.0" ?>
<EtherCATInfo>
  <!-- Slave 0 -->
  <Vendor>
    <Id>2</Id>
  </Vendor>
  <Descriptions>
    <Devices>
      <Device>
        <Type ProductCode="#x044d2c52" RevisionNo="#x00140000">EL3162</Type>
        <Name><![CDATA[EL3162 2C. Ana. Input 0-10V]]></Name>
        <Sm Enable="1" StartAddress="#x1000" ControlByte="#x26" DefaultSize="0" />
        <Sm Enable="1" StartAddress="#x1080" ControlByte="#x22" DefaultSize="0" />
        <Sm Enable="1" StartAddress="#x1100" ControlByte="#x24" DefaultSize="0" />
        <Sm Enable="1" StartAddress="#x1140" ControlByte="#x20" DefaultSize="0" />
        <TxPdo Sm="3" Fixed="1" Mandatory="1">
          <Index>#x1a00</Index>
          <Name>Channel1</Name>
          <Entry>
            <Index>#x6000</Index>
            <SubIndex>1</SubIndex>
            <BitLen>16</BitLen>
            <Name>Status</Name>
            <DataType>UINT16</DataType>
          </Entry>
          ...
        </TxPdo>
      </Device>
    </Devices>
  </Descriptions>
</EtherCATInfo>

# 生成所有从站的 XML
$ ethercat xml > all_slaves.xml

# 按 alias 生成
$ ethercat xml -a 1001 > slave_1001.xml
```

---

30. 典型工作流程汇总

```
# 第一步：系统上电后检查总线
ethercat master                # 查看主站状态和网卡信息
ethercat slaves                # 查看所有在线从站

# 第二步：检查从站配置和 I/O 映射
ethercat slaves -v -p 0        # 查看从站 0 的完整设备信息
ethercat pdos -p 0             # 查看从站 0 的 PDO 分配与映射
ethercat sdos -p 0             # 查看从站 0 的 SDO 字典

# 第三步：生成应用代码参考
ethercat cstruct -p 0          # 生成从站 0 的 C 结构体配置
```

```
# 第四步：启动总线运行
ethercat states INIT           # 初始化所有从站
ethercat states PREOP          # 进入预运行（SDO 可用）
ethercat states SAFEOP         # 进入安全运行（输入数据可用）
ethercat states OP             # 进入完全运行

# 第五步：运行时诊断
ethercat domains -v            # 查看域数据和 WorkingCounter（验证通信）
ethercat crc                   # 排查 CRC 通信错误
ethercat data | xxd            # 捕获实时过程数据

# 第六步：在线参数调整
ethercat download -p 0 0x6060 0 8 --type uint8   # 设置操作模式
ethercat upload   -p 0 0x6061 0 --type uint8     # 确认设置生效

# 固件更新流程
ethercat states -p 0 BOOT                        # 进入 Boot 状态
ethercat foe_write -p 0 new_firmware.bin         # 写入新固件
ethercat states -p 0 INIT                        # 回到 Init
ethercat rescan && ethercat slaves               # 重新扫描验证

# 拓扑可视化
ethercat graph | dot -Tsvg > bus.svg             # 生成总线拓扑图
ethercat graph CRC | dot -Tpng > bus_crc.png      # 含 CRC 信息的拓扑图
```

20.4 IgH SDO原理

1. SDO 概述

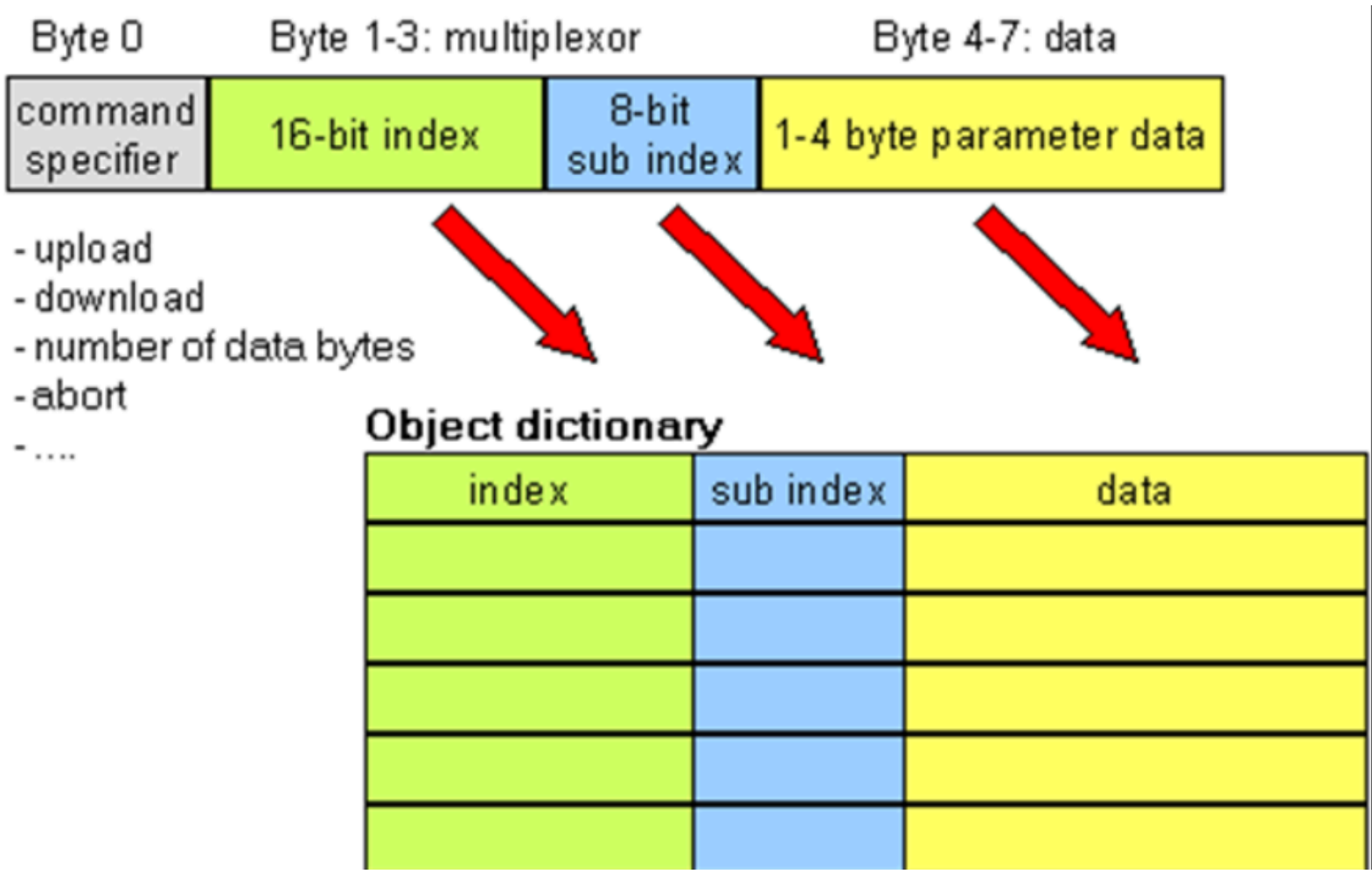
就是Modbus 的样式的通信协议， 发送一包数据回复一包数据， 问答式通信。

在 IgH EtherCAT Master 中，**SDO（服务数据对象）** 是通过邮箱协议（Mailbox）进行非周期性数据读写的关键通道，通常用于参数配置、状态查询、故障诊断等场景（与之相对，过程数据 PDO 用于循环实时通信）。

- **功能**：读写从站对象字典（Object Dictionary）中的任意条目（索引+子索引）。
- **特性**：基于邮箱协议，速度较慢，不适合高速循环传输；可以访问 PDO 映射之外的参数。
- **常见用途**：修改伺服驱动器控制参数、读取诊断信息、配置电机参数、更新 PDO 映射等。

2. SDO数据格式

主站通过使用 SDO对象字典进行读取/写入操作，完成对从站设备进行参数设置、状态监视等。



3. SDO非实时

EtherCAT 的过程数据（PDO）是主站周期性地、在确定的时间窗口内直接读写从站存储区，从站**必须在极短时间内处理**，所以能够保证每一次操作都在固定时间内完成，成功与否有明确的“硬实时”保障。

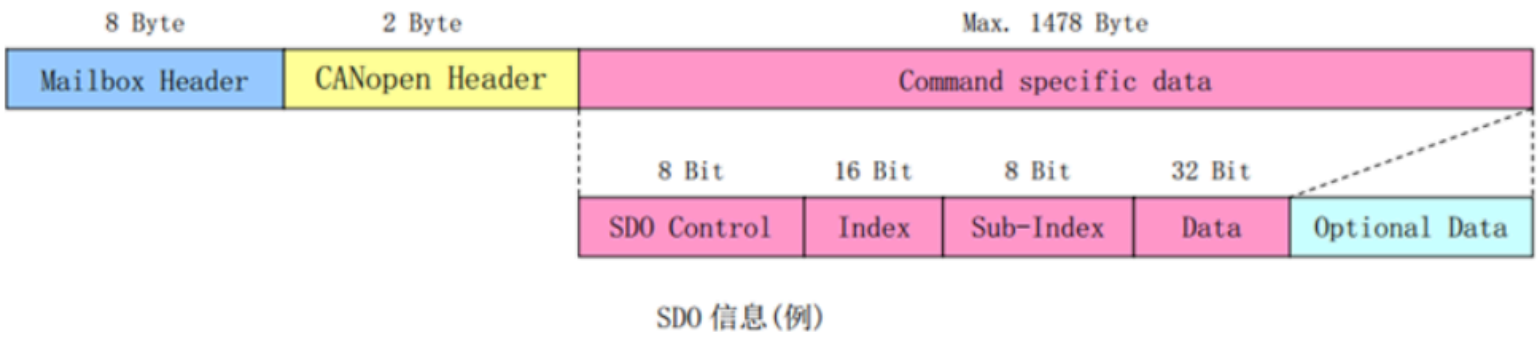
而 SDO 走的是邮箱（Mailbox）协议，属于非周期性数据。它的传输是：

- **异步的**：插入在实时帧的空隙中，没有固定的发送时间点；
- **可被分段、重组**：大数据需要拆包、从站也需要时间处理；
- **容易受总线负载和从站忙碌状态影响**：如果从站正忙于处理其他任务，可能延迟响应甚至丢弃 SDO 请求；
- **没有超时强制成功保证**：主站发送 SDO 请求后，可能因超时、从站忙、校验错误等导致失败，且不会像 PDO 那样自动重发。

因此，单次 SDO 操作并**不一定能成功**，或者**虽然发送了命令，但从站未必执行完成**，需要通过多次访问来确认和补偿。

从站读取数据，主站同样需要发起多次访问，以确保数据正确读取

4. SDO邮箱协议



SDO 信息(例)

从站的**SM0，SM1**同步管理做为邮箱通信使用，当有邮箱数据时会触发SM0或SM1信号，进行邮箱数据的处理（SM0=mailbox output SM1=mailbox input）

邮箱头占用6个字节，CANopen头占用2个字节。

5. SDO 命令行工具

IgH 主要有三个与 SDO 相关的子命令：

命令	用途
`ethercat sdos`	列出从站的所有 SDO 字典条目（索引、数据类型、访问权限等）
`ethercat upload`	从从站读取一个 SDO 的值
`ethercat download`	向从站写入一个 SDO 值

20.5 IgH PDO原理

在 IgH EtherCAT Master 中，**过程数据对象（PDO, Process Data Object）** 是实现高性能实时控制的核心机制。它与上一篇提到的 SDO 不同，PDO 不经过邮箱协议，而是直接映射到每个通信周期的 EtherCAT 帧中，以**硬实时**方式在主站与从站之间交换控制指令和反馈数据。

下面从 IgH 的角度详解 PDO 原理。

1. PDO 是什么？为什么需要它?

- **功能**：周期性地传输实时控制数据（如伺服的目标位置、速度、实际位置、状态字等）。
- **与 SDO 的区别**：
  - SDO：非周期、慢速、可访问任意对象字典条目，用于参数配置和诊断。
  - PDO：周期、高速、只传输“映射”到的少量关键数据，用于实时控制环。
- **物理形态**：PDO 数据直接嵌入 EtherCAT 帧的数据区（逻辑过程数据映像），不经过邮箱，传输延迟极小、抖动低。

2. PDO 映射原理

PDO 的实质是**将对象字典中分散的参数“拼接”到一个连续的数据块中**，通过一个映射表告诉主站/从站如何解释这段数据。

从站侧的 PDO 映射对象

每个从站可以定义多个 **RxPDO**（主站发送给从站）和 **TxPDO**（从站发送给主站）。对应的对象字典条目为：

- **RxPDO 映射**：`0x1600` ~ `0x1603`（最多 4 个，具体数量因从站而异）
- **TxPDO 映射**：`0x1A00` ~ `0x1A03`
- **PDO 分配**：`0x1C12`（SM2 的 PDO 分配）和 `0x1C13`（SM3）

例如，一个伺服驱动器的默认 RxPDO 映射可能为：

```
0x1600.01 → 0x6040:00 (控制字，16 位)
0x1600.02 → 0x607A:00 (目标位置，32 位)
```

这样，RxPDO 整体就是一个 6 字节的连续数据块（2 + 4），主站只需每周期写入这 6 字节即可。

3. 同步管理器（Sync Manager, SM）

过程数据通过特定的 SM 通道传输，标准分配为：

- **SM2**：用于 RxPDO（输出数据，主站→从站）
- **SM3**：用于 TxPDO（输入数据，从站→主站）

SM 设置了缓冲模式、长度、方向等参数，确保数据在中断和循环之间的一致性。

---

4. PDO 传输类型

PDO 的传输策略（通信行为）由对象字典 `0x1C32`（SM2）和 `0x1C33`（SM3）的"同步类型"决定，常见类型：

传输类型值	含义	说明
0	自由运行	从站内部定时器驱动，与主站周期无关
1-240	同步周期倍乘	每 N 个 SM 事件产生一次数据更新（N 即数值）
254	异步（由供应商指定）	通常映射到状态改变事件
255	同步于 SM 事件	最常见，随 DC 同步时钟或 SM 事件立即更新

在工业应用中，绝大多数场景使用 **同步（255 或 1 倍周期）** 配合分布式时钟（DC）来保证多轴同步。

---

5. IgH 命令行下的 PDO 查看

命令	作用
`ethercat pdos -p 0`	显示从站 0 的 PDO 映射详情（索引、子索引、条目名称、位宽）
`ethercat pdos -p 0 -s`	同时显示同步管理器的 PDO 分配（0x1C12/0x1C13）
`ethercat cstruct`	生成可直接用于 C 代码的结构体，方便访问域数据
`ethercat data`	实时打印过程数据映像的十六进制和 ASCII 内容，用于在线调试

示例输出（`ethercat pdos -p 0`）：

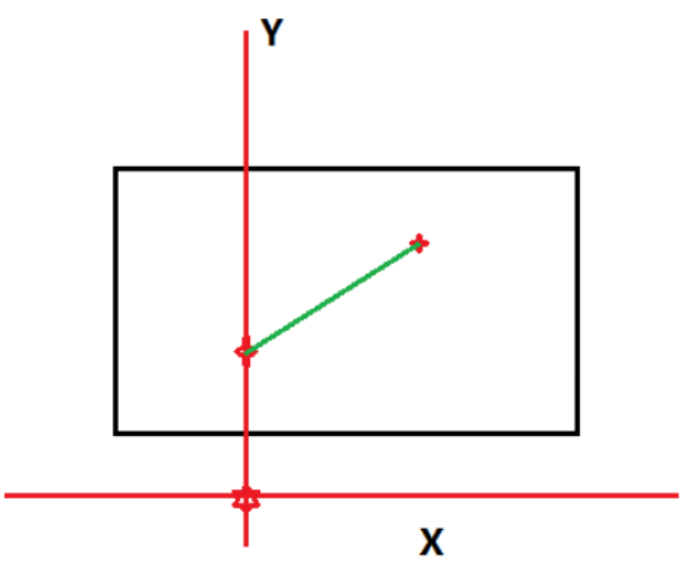
```
SM2: PhysAddr 0x1000, DefaultSize   6, ControlByte 0x24, Enable 1
  RxPDO 0x1600 "Control"
    PDO entry 0x6040:00, 16 bit, "Controlword"
    PDO entry 0x607A:00, 32 bit, "Target position"
```

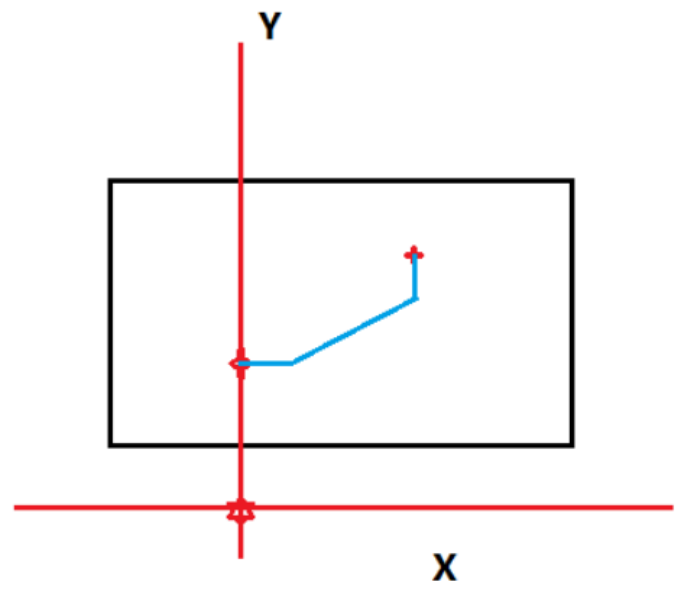
20.6 IgH DC原理

1. DC 理解

(Distributed Clocks) 分布式时钟

分布式时钟可以这样理解：每个节点都拥有自己的本地时钟，系统要求在某一特定时刻，所有节点能同时执行某个预定动作。要想实现"同时触发"，各节点的时钟必须保持一致，这就涉及时钟同步。





假设有两台打印机：一台价值上万元，另一台仅9.9元包邮。现在要求它们在纸上绘制一条斜线。

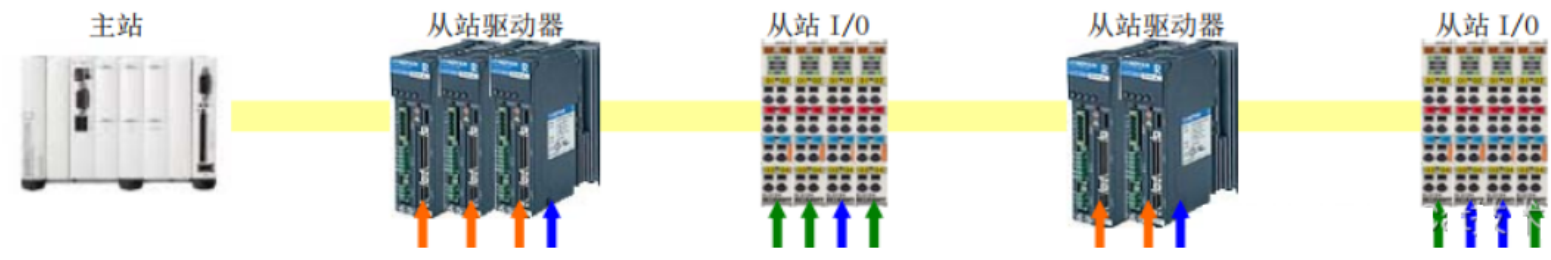
- 高端打印机实现了完全同步：X轴和Y轴同时收到移动信号，同步开始运动并同时到达终点，画出的斜线笔直、精准。
- 廉价打印机则缺乏同步：X轴先动，约1秒后Y轴才响应。X轴到达目标坐标后停止，Y轴随后才到达，虽然最终也停在同一个终点，但运动路径发生了明显错位，画出的斜线歪曲变形。

这个例子说明，尽管最终位置一致，但中间轨迹却大相径庭。对于机器人加工、精密作业等场景，同步性直接决定了执行过程是否符合设计预期——同步性越好，成品越接近理想规划。

2. 时钟不同步因素

EtherCAT主站上接了如下设备，一般会选择第一个带DC功能的从站作为参考时钟，利于参考时钟的往后传播（后面介绍）。

引起时钟不同步的有：



您提供的三点内容，是关于分布式时钟同步中常见的三大误差来源。以下是对其语义的清晰整理：

---

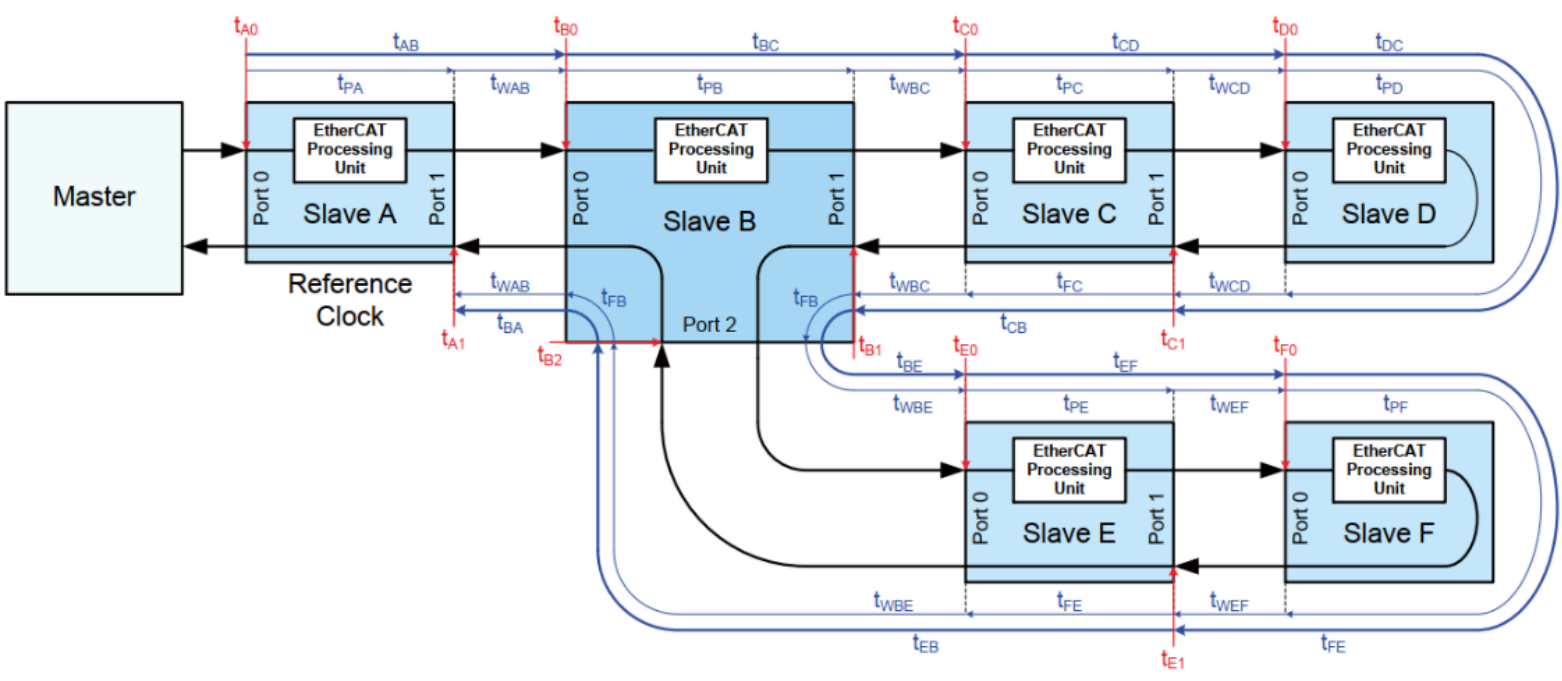
1. 传播延迟（Propagation Delay）
指报文在总线上从主站发送到各从站，或从站之间转发所耗费的传输时间。例如，参考时钟（第一个从站）在 `9:00:00.000000000` 收到同步报文，而第二个从站则在 `9:00:00.000000500` 才收到，两者相差 500 纳秒。这种延迟可能由信号传播和节点处理转发共同引起，当总线上串联的设备增多时，累积的传播延迟会显著增大。

2. 偏移（Offset Compensation）
指各从站因启动时间不同而导致的时钟基准不一致。例如，第一个从站启动一小时后，第二个从站才上电，那么两个时钟的初始计时起点就相差一个小时，形成固定的绝对时间偏移，需要在同步时进行补偿。

3. 漂移（Drift Compensation）
指每个从站内部晶振的实际振荡频率存在微小差异，导致时钟走时速率不同。例如，第一个从站每秒振荡 `1,000,000,010` 次，第二个从站每秒振荡 `999,999,990` 次，两者每秒就会产生 20 纳秒的计时误差。这种差异随时间累积，会逐渐扩大，即使初始同步了，后期也会出现明显偏差，因此需要定期校准。

DC同步就是要解决上述存在的问题。

3. DC时钟同步计算方法



EtherCAT 的分布式时钟（DC）同步过程可以简单总结为以下四个步骤：

1. **确立参考时钟**：系统选定网络中第一个具有 DC 功能的从站（如图中的 **Slave A**）作为**参考时钟**（Reference Clock），作为整个网络的时间基准。
2. **测量传输延迟**：通过数据帧在网络中的往返传输，记录报文经过各个从站端口时的接收和发送时刻（即打时间戳）。利用这些时间戳，精确计算出相邻从站之间的**链路传播延迟**（如图中的 t_{AB}, t_{BC} 等）。
3. **计算时钟偏移**：每个从站根据测量到的链路延迟，计算出自身本地时钟与参考时钟（Slave A）之间的**时间偏差**（Offset）。
4. **实时补偿调整**：每个从站根据计算出的偏移量和延迟，在本地对时钟进行动态补偿，不断调整本地时钟的速度和相位，使其与参考时钟保持精确同步，从而实现整个系统的高精度时钟对齐。

通过硬件级同步，让所有 DC 从站共享同一个全局时钟（以 2000-01-01 为起点的纳秒计数器），实现 <100ns 的多从站同步精度。

20.7 IgH（FSM）有限状态机

1. 什么是状态机

状态机（State Machine）可以理解为一种描述“事物在不同模式之间如何切换”的模型。它的核心思想是：**一个系统只能处于某个确定的状态，当发生特定事件或满足条件时，它会跳转到另一个状态。**

最简单的类比：交通信号灯

想象一个十字路口的红绿灯，它就是一个典型的状态机：

- **状态**：`红灯`、`绿灯`、`黄灯`
- **切换条件**：计时器到时
- **转换规则**：`红灯(60秒)` → `绿灯(30秒)` → `黄灯(3秒)` → `红灯...`

交通灯**不能**直接从红灯跳到黄灯，也不能同时亮两种颜色——这就是状态机“同一时刻只有唯一状态”的特性。

IgH EtherCAT 主站中，每个从站都遵循 EtherCAT 协议的状态机：

- **状态**：`INIT`、`PREOP`、`SAFEOP`、`OP`
- **事件**：主站发送的特定命令（例如写 AL 控制寄存器）
- **转换路径**：只能按 `INIT → PREOP → SAFEOP → OP` 一步步升级，不能跳跃，也可以降级（如 `OP → INIT` 直接返回）。

这个硬性规定就是一个状态机在起作用，确保了配置过程的安全有序。

为什么状态机这么有用?

- **把复杂逻辑清晰化**：避免写一堆纠缠的 `if-else`，用状态图一眼看清系统可能处于哪种模式。
- **防止非法操作**：比如你可以定义“只能在待机状态才能开始充电”，其他状态下收到充电命令会被忽略。
- **易于调试和扩展**：问题发生时，你可以问“当时在哪个状态？收到了什么事件？”而且增加新功能往往就是增加一两个新状态。

生活中的状态机

- 洗衣机：`浸泡 → 洗涤 → 漂洗 → 脱水`
- 电梯：`静止 → 开门 → 关门 → 上行/下行`
- 用户登录流程：`未登录 → 输入密码 → 验证中 → 登录成功/失败`

---

2. 有限状态机概念

FSM 的英文全称是 **Finite State Machine**，中文通常翻译为 **有限状态机。**

FSM（有限状态机）是整个主站协议栈的核心运行框架。EtherCAT 的配置与数据交互大多是**异步、分步完成**的，无法用单一函数调用瞬间搞定，所以 IgH 内部用大量相互协作的有限状态机，把复杂操作拆成一个个状态，在每个周期里推进一小步，从而实现非阻塞的实时控制。

---

1. 为什么 IgH 要用有限状态机?

- EtherCAT 从站的状态切换（INIT → PREOP → SAFEOP → OP）需要主站多次发送命令并等待响应。
- 从站扫描（读取 Vendor ID、Product ID、PDO 映射、SDO 对象字典）同样要反复读写 ESC 寄存器/内存。
- 配置过程（写 SDO、设置分布式时钟同步）可能很耗时，如果设计成阻塞调用会卡死整个应用。

因此 IgH 把所有这些“需要多个 EtherCAT 帧才能完成的工作”都实现成 **FSM。**每个 FSM 记录当前执行到哪一步，由主循环（用户层调用 `ecrt_master_exec()` ）周期性地喂数据、推进状态，直到最终完成或报错。

---

3. IgH 里主要的有限状态机

在 IgH 源码中，你会看到大量 `ec_fsm_xxx` 开头的函数和结构体，每个就是一个独立的状态机。核心的有：

FSM	作用	何时运行
Master FSM	主站自身的生命周期管理（空闲、扫描总线、配置、运行、故障恢复）	内核线程或用户层 `ecrt_master_exec()` 不断推进
Slave Scan FSM	扫描总线上的从站：读取 AL 状态、身份信息、物理地址等	主站状态机在空闲后进入“扫描总线”阶段时启动
SII FSM	从 EEPROM 中读取从站信息接口（SII）：包括字符串信息、PDO 映射、默认标识等	扫描到新从站后，读取其 EEPROM 内容
Slave Config FSM	把单个从站配置成应用要求的状态：切换状态、写 SDO、配置 DC 时钟等	用户调用 `ecrt_slave_config_*` 函数后，由主站调度执行
Domain FSM	负责域（过程数据区）的配置：设置 FMMU、同步管理器，激活过程数据交换	用户调用 `ecrt_master_activate()` 时触发
DC FSM	处理分布式时钟同步：漂移补偿、主站时钟同步等	在 SAFEOP 到 OP 转换阶段，以及持续运行时的时钟同步循环中
Request FSM	处理应用层触发的异步请求（例如主站自定义请求、邮箱通信）	用户调用 `ecrt_master_*` 请求时创建，由主站执行

另外还有一个 **EoE（Ethernet over EtherCAT）FSM**、**FoE 固件更新 FSM** 等，按需运行。

---

4. 这些 FSM 是如何串联工作的?

以主站从“加载”到“进入 OP 并开始交换过程数据”为例，整体流程就像一个大型状态机，内部嵌套许多小 FSM：

1. **主站启动** → Master FSM 进入 `Orphan` 阶段，等待以太网设备。
2. 设备就绪后 → Master FSM 进入 `Idle` 阶段，随后被触发开始 **扫描总线。**
3. **总线扫描** 阶段启动 **Slave Scan FSM**，顺序询问每个从站，发现新从站后，为该从站创建 **SII FSM** 去读取 EEPROM。所有从站地址和基础信息读取完毕后，扫描结束，Master FSM 回到 Idle。
4. 用户程序请求配置从站（例如写 SDO、设置 PDO 映射）→ 主站为对应从站生成 **Slave Config FSM。**它会负责：
    - 向从站发送状态切换请求（INIT → PREOP）。
    - 通过邮箱（CoE 协议）写 SDO。
    - 可能需要启动 **DC FSM** 来配置分布式时钟。
    - Slave Config FSM 执行完所有配置项后，标记从站配置完成。
5. 用户调用 `ecrt_master_activate()` → 主站生成 **Domain FSM**：
    - 计算所有过程数据域的内存布局。
    - 为每个从站设置 FMMU 和 Sync Manager。
    - 最后请求所有已配置从站切换到 SAFEOP → OP。
6. 所有从站成功进入 OP 后，Master FSM 进入 `Operation` 状态，实时循环开始通过 `ecrt_domain_queue` 和 `ecrt_master_send/ecrt_master_receive` 交换过程数据。
7. 在运行中，**DC FSM** 可能持续运行，以补偿时钟漂移。

关键点：这些阶段不是阻塞的，每一步都由 `ecrt_master_exec()` 周期驱动，直到最终状态。

6. FSM总结

IgH 中的 FSM 是一组精密协作的非阻塞执行引擎，把复杂的 EtherCAT 协议交互拆解为可逐步推进的状态。作为应用开发者，最重要的就是理解：所有配置和激活动作不会瞬间完成，必须通过 `ecrt_master_exec` 这个统一的“驱动器”给 FSM 执行，直到它们到达终态。

第21章 IgH主站移植

21.1 切换RT-Linux

1. 切换RT内核

- 切换到rt 实时性分支上再去编译sdk

```
git branch    --查看分支
git checkout ido-evb3506-preempt-rt  --切换分支
```

- 运行结果

```
linux@ubuntu:~/rk3506$ git branch
* ido-evb3506
  ido-evb3506-preempt-rt
  ido-evb3506-xenomai-rt
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ git checkout ido-evb3506-preempt-rt     # 直接切换报错
error: 您对下列文件的本地修改将被检出操作覆盖：
        kernel-6.1/arch/arm/configs/rk3506_defconfig
请在切换分支前提交或贮藏您的修改。
error: 您对下列文件的本地修改将被检出操作覆盖：
        buildroot/configs/rockchip_rk3506_defconfig
请在切换分支前提交或贮藏您的修改。
正在终止
linux@ubuntu:~/rk3506$ git stash                    #暂存修改
保存工作目录和索引状态 WIP on ido-evb3506: 43f0c25a8 Release ido-evb3506 v1.0
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ git checkout ido-evb3506-preempt-rt
正在更新文件: 100% (213/213), 完成.
```

```
切换到分支 'ido-evb3506-preempt-rt'
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ git branch
  ido-evb3506
* ido-evb3506-preempt-rt
  ido-evb3506-xenomai-rt
linux@ubuntu:~/rk3506$
```

2. 选择项目

```
linux@ubuntu:~/rk3506$ git branch
  ido-evb3506
* ido-evb3506-preempt-rt
  ido-evb3506-xenomai-rt
linux@ubuntu:~/rk3506$ ./build.sh lunch
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-07-09_14-01-21
Pick a defconfig:

1. rockchip_ido_evb3506_v1_emmc_mipi_1024x600_defconfig
2. rockchip_ido_evb3506_v1_emmc_mipi_1280x800_defconfig
3. rockchip_ido_evb3506_v1_emmc_mipi_720x720_defconfig
4. rockchip_ido_evb3506_v1_emmc_rgb_1024x600_defconfig
5. rockchip_ido_evb3506_v1_flash_mipi_1024x600_defconfig
6. rockchip_ido_evb3506_v1_flash_mipi_1280x800_defconfig
7. rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
8. rockchip_ido_evb3506_v1_flash_rgb_1024x600_defconfig
Which would you like? [1]: 7
Switching to defconfig:
/home/linux/rk3506/device/rockchip/.chip/rockchip_ido_evb3506_v1_flash_mipi_720x720_defconfig
#
# configuration written to /home/linux/rk3506/output/.config
#
Using last kernel version(6.1)
linux@ubuntu:~/rk3506$
```

3. 编译内核

编译内核前先清除一个切换版本的过程文件

```
linux@ubuntu:~/rk3506$ ./build.sh clean-kernel      # 清理内核
linux@ubuntu:~/rk3506$ ./build.sh clean-rootfs      # 清理文件系统
linux@ubuntu:~/rk3506$ ./build.sh all
Log colors: message notice warning error fatal

Log saved at /home/linux/rk3506/output/sessions/2026-07-09_14-05-47
Using last kernel version(6.1)

============================================
          Final configs
============================================
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS=/home/linux/rk3506/device/rockchip/.chip/thunderboot.its
RK_BOOT_IMG=zboot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3506
。。。
------ OK ------
********rkImageMaker ver 2.29********
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/linux/rk3506/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$
```

错误解决

```
Your network is not able to access sources.buildroot.net!
Please retry later (it could be down for a while) or setup a VPN to bypass the GFW.
Unset RK_NETWORK_CHECK in the SDK config to continue...

ERROR: Running /home/linux/rk3506/device/rockchip/common/scripts/mk-rootfs.sh - build_buildroot failed!
ERROR: exit code 1 from line 16:
    "$RK_SCRIPTS_DIR/mk-buildroot.sh" $RK_BUILDROOT_CFG "$IMAGE_DIR"
ERROR: call stack:
    mk-rootfs.sh: build_buildroot(16)
    mk-rootfs.sh: build_hook(365)
    mk-rootfs.sh: main(421)
ERROR: Running /home/linux/rk3506/device/rockchip/common/build-hooks/99-all.sh - build_all failed!
ERROR: exit code 1 from line 21:
    "$RK_SCRIPTS_DIR/mk-rootfs.sh"
ERROR: call stack:
    99-all.sh: build_all(21)
    99-all.sh: build_hook(136)
    build-helper: try_func(63)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
ERROR: Running /home/linux/rk3506/device/rockchip/common/build-hooks/99-all.sh - try_func build_hook all
failed!
ERROR: exit code 1 from line 67:
    build_hook
ERROR: call stack:
    build-helper: try_func(67)
    build-helper: try_hook(96)
    build-helper: source(165)
    99-all.sh: main(147)
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ ping www.baidu.com              # 网络故障
ping: www.baidu.com: 域名解析出现暂时性错误
linux@ubuntu:~/rk3506$
```

- 网络问题解决后，继续编译，后续问题以此类推。

- 添加自动复制文件的脚本

增加自动复制脚本

执行命令：

```
linux@ubuntu:~/rk3506$ vi build.sh
linux@ubuntu:~/rk3506$
```

- build.sh 在文件的末尾追加如下内容

```
if [ -e rockdev ] ;then
    cp -v rockdev/* /mnt/hgfs/linuxshare/
fi
```

设置共享目录， 这个时候固件会自动复制到你指定的目录内。

- 执行结果

```
linux@ubuntu:~/rk3506$ ./build.sh all
。。。
make: Leaving directory '/home/linux/rk3506/kernel-6.1'
+ /home/linux/rk3506/device/rockchip/common/scripts/mk-fitimage.sh kernel/zboot.img
/home/linux/rk3506/device/rockchip/.chip/thunderboot.its kernel/arch/arm/boot/zImage
kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dtb kernel/resource.img
FIT description: U-Boot FIT source file for arm
Created:         Fri Jul 10 19:52:57 2026
 Image 0 (fdt)
  Description:  unavailable
  Created:      Fri Jul 10 19:52:57 2026
  Type:         Flat Device Tree
  Compression:  uncompressed
  Data Size:    44621 Bytes = 43.58 KiB = 0.04 MiB
  Architecture: ARM
  Load Address: 0x00063000
  Hash algo:    sha256
  Hash value:   ce3b800071136cd2bf8866e5bcf668619a52d1026d2c820f6a1fb00512ea0410
 Image 1 (kernel)
  Description:  unavailable
```

```
  Created:      Fri Jul 10 19:52:57 2026
  Type:         Kernel Image
  Compression:  uncompressed
  Data Size:    4248816 Bytes = 4149.23 KiB = 4.05 MiB
  Architecture: ARM
  OS:           Linux
  Load Address: 0x01100000
  Entry Point:  0x01100000
  Hash algo:    sha256
  Hash value:   09abce3100172bca79032218cb961feadb89d04984aabc9e96d88e197bb71761
 Default Configuration: 'conf'
 Configuration 0 (conf)
  Description:  Boot Linux kernel with FDT blob and no ramdisk
  Kernel:       kernel
  FDT:          fdt
+ ln -rsf kernel/zboot.img /home/linux/rk3506/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm/boot/dts/ido-evb3506-v1a-flash-mipi-720x720.dts
Running 10-kernel.sh - build_kernel succeeded.
'rockdev/MiniLoaderAll.bin' -> '/mnt/hgfs/linuxshare/MiniLoaderAll.bin'
'rockdev/boot.img' -> '/mnt/hgfs/linuxshare/boot.img'
'rockdev/misc.img' -> '/mnt/hgfs/linuxshare/misc.img'
'rockdev/rootfs.img' -> '/mnt/hgfs/linuxshare/rootfs.img'
'rockdev/uboot.img' -> '/mnt/hgfs/linuxshare/uboot.img'
linux@ubuntu:~/rk3506$
```

4. 普通Linux内核实时性

- 测试RT实时性之前需要关闭UI显示demo，否则会影响实时性测试。

```
root@rk3506-buildroot:/# killall rk_demo
```

- 空载测试

```
# 在实时性测试中的主要用意是：防止内核因内存过度分配触发 OOM Killer 而误杀实时进程，从而保障实时任务持续运行不中断。
echo 2 > /proc/sys/vm/overcommit_memory

# 以最高实时优先级运行 3 个内存锁定的测试线程，全部绑定在 CPU 0 核上，持续测量定时器的实际唤醒时间与预期时间的差异（延迟）
cyclictest -c 0 -m -t 3 -p 99
```

- 参数详解

参数	含义
`-c 0`	选择时钟源为 `CLOCK_MONOTONIC`（单调递增时钟）。这是默认的时钟，不受系统时间跳变影响，适合测量纯内核调度延迟。
`-m`	锁定内存（`mlockall`），将所有当前及未来的内存页锁定在物理内存中
`-t 3`	创建 **3 个测量线程**。如果不指定 CPU 亲和性，这 3 个线程会在所有可用 CPU 上由内核自由调度。
`-p 99`	设置线程的实时优先级为 **99**（`SCHED_FIFO` 调度策略）。这是 Linux 允许的最高用户态实时优先级，确保测试线程几乎不会被其他普通任务抢占。

- 运行结果

```
root@rk3506-buildroot:/# cat /proc/version
Linux version 6.1.118 (mhkwork@dell-PowerEdge-R430) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #4 SMP PREEMPT Thu Jan 15 10:38:30 CST 2026
root@rk3506-buildroot:/#
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/# echo 2 > /proc/sys/vm/overcommit_memory
root@rk3506-buildroot:/# cyclictest -c 0 -m -t 3 -p 99
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 0.53 0.23 0.09 1/110 1642

T: 0 ( 1640) P:99 I:1000 C:   2793 Min:      6 Act:   10 Avg:   10 Max:     107
T: 1 ( 1641) P:99 I:1500 C:   1857 Min:      6 Act:   10 Avg:   10 Max:      61
T: 2 ( 1642) P:99 I:2000 C:   1388 Min:      6 Act:    9 Avg:   10 Max:      24
^Croot@rk3506-buildroot:/#
```

- **内核类型**：`6.1.118 SMP PREEMPT`（普通抢占内核，非 PREEMPT_RT）
- **平均延迟**：三个线程均为 **10 µs**，一致性很好，说明多数周期内调度响应稳定。
- **最大延迟**：T0 达 **107 µs**，T1 为 61 µs，T2 仅 24 µs。T0 偶发百微秒级抖动，T2 表现更好。
- **实时性评估**：
  在普通 PREEMPT 内核下，平均 10 µs 属较好水平；但最大延迟 107 µs 表明仍存在非实时干扰（如硬件中断、内核临界区等），无法满足硬实时要求（通常需 ≤20~30 µs）。
- **overcommit_memory=2 的影响**：测试能成功运行（得益于 `killall rk_demo` 释放内存），但该设置本身不会降低延迟，仅限制内存分配。
- **负载 0.53**：轻度负载，未造成显著干扰。

总结：当前普通内核可满足一般软实时任务，若需将最大延迟压缩至 30 µs 以内，需切换到 PREEMPT_RT/Xenomai 内核并配合中断隔离等优化。

压力测试

```
stress -c 3 --io 1 --vm 1 --vm-bytes 1M --timeout 1m &
```

- `stress`：一个施加系统负载的工具，可以模拟 CPU、内存、I/O、磁盘等压力。
- `-c 3`：创建 3 个进程，每个进程循环执行 `sqrt()` 计算，以占用 **CPU** 资源（即产生 3 个 CPU 密集型负载）。
- `--io 1`：创建 1 个进程，循环调用 `sync()` 系统调用，产生 **I/O 压力**（刷写缓冲区到磁盘）。
- `--vm 1`：创建 1 个进程来分配并反复操作内存，模拟 **内存压力**。
- `--vm-bytes 1M`：每个内存压力进程使用的内存大小为 **1 MB**。这里只有一个 `--vm` 进程，所以总占用约 1 MB 内存（用于不断的分配和释放/重写操作）。
- `--timeout 1m`：设置整个 `stress` 运行的超时时间为 **1分钟**，超时后所有子进程自动终止。加上这个防止一直运行。
- **末尾 `&`**：将整个命令放到 **后台执行**，不阻塞终端。

- 运行结果

```
root@rk3506-buildroot:/# cat /proc/version
Linux version 6.1.118 (mhkwork@dell-PowerEdge-R430) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #4 SMP PREEMPT Thu Jan 15 10:38:30 CST 2026
root@rk3506-buildroot:/
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/#
root@rk3506-buildroot:/#
                        # 运行 1 分钟后自动停止
root@rk3506-buildroot:/# stress -c 3 --io 1 --vm 1 --vm-bytes 1M --timeout 1m &
[2] 1634
root@rk3506-buildroot:/# stress: info: [1634] dispatching hogs: 3 cpu, 1 io, 1 vm, 0 hdd

root@rk3506-buildroot:/# cyclictest -c 0 -m -t 3 -p 99
policy: fifo: loadavg: 1.90 0.48 0.16 6/123 1666
T: 0 ( 1664) P:99 I:1000 C:  14500 Min:      7 Act:   39 Avg:   31 Max:     100
T: 1 ( 1665) P:99 I:1500 C:   9647 Min:      8 Act:   54 Avg:   30 Max:      73
T: 2 ( 1666) P:99 I:2000 C:   7219 Min:      9 Act:   42 Avg:   29 Max:      62
^Croot@rk3506-buildroot:/#
```

1. **实时性表现良好**
   在非 RT 内核、多线程混合压力下，最大延迟仅 100 µs，表明内核的 PREEMPT 机制已有效降低了关中断/关抢占的时间，响应边界控制得不错。
2. **可满足多数嵌入式场景**
   对于 1ms 周期的实时任务，100 µs 的最大延迟意味着最差响应时间为周期的 10%，仍可满足工业控制、音频处理、电机驱动等常见需求（一般要求 < 几百 µs）。
3. **提升空间**
   如果应用要求更严苛（如 <50 µs 甚至 <20 µs），可考虑：
   - 切换到 `PREEMPT_RT` 全实时内核（通常能降低最大延迟一个数量级）
   - 设置 CPU 隔离（`isolcpus`）并将实时线程绑定到专用核心
   - 优化驱动，减少长时间禁止中断的代码段
4. **负载与核心数推测**
   1 分钟负载 1.90，而 stress 只占用了 3 个 CPU worker，推测 CPU 至少为 4 核（因为还有 cyclictest 的 3 个实时线程+系统任务）。若 CPU 数量更多，压力更分散，本次测试的竞争不算极端；若 CPU 较少，则此延迟数据更具参考价值

5. RT-Linux内核实时性

编译PREEMPT_RT内核并下载系统到开发板内。

- 运行结果

```
root@rk3506-buildroot:/# cat /proc/version
Linux version 6.1.118-rt36 (linux@ubuntu) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile
Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture
10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #1 SMP PREEMPT_RT Thu Jul  9 15:35:27 CST 2026
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/# echo 2 > /proc/sys/vm/overcommit_memory
root@rk3506-buildroot:/# cyclictest -c 0 -m -t 3 -p 99
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 1.21 0.98 0.49 1/147 1706

T: 0 ( 1704) P:99 I:1000 C:   6052 Min:      1 Act:    1 Avg:    2 Max:      10
T: 1 ( 1705) P:99 I:1500 C:   4023 Min:      0 Act:    1 Avg:    2 Max:      14
T: 2 ( 1706) P:99 I:2000 C:   3008 Min:      5 Act:    6 Avg:   12 Max:      32
^Croot@rk3506-buildroot:/#
```

内核类型：`6.1.118-rt36 SMP PREEMPT_RT`（实时内核）

- **平均延迟**：T0/T1 平均仅 **2 µs**，T2 平均 **12 µs**，调度响应极快，一致性好。
- **最大延迟**：T0 最大 **10 µs**，T1 最大 **14 µs**，T2 最大 **32 µs。**T0/T1 达到硬实时顶尖水平，T2 稍高但远优于普通内核。
- **实时性评估**：
  在 PREEMPT_RT 内核下，平均延迟低至 2 µs，最大延迟控制在 32 µs 以内，已满足硬实时要求（通常 ≤20~30 µs）。T2 的 32 µs 略高，可能受未完全隔离的硬件中断或微量内核临界区影响，整体实时性卓越。
- **overcommit_memory=2 的影响**：测试能成功运行，该设置仅限制内存分配，不影响延迟。
- **负载 1.21**：存在轻度负载，但未对实时线程造成明显干扰。

总结：PREEMPT_RT 内核实时性能优异，相比普通内核最大延迟降低约 70%，已具备生产级硬实时能力，可直接用于高精度实时控制场景。

压力测试

```
stress -c 3 --io 1 --vm 1 --vm-bytes 1M --timeout 1m &
```

- `stress`：一个施加系统负载的工具，可以模拟 CPU、内存、I/O、磁盘等压力。
- `-c 3`：创建 3 个进程，每个进程循环执行 `sqrt()` 计算，以占用 **CPU** 资源（即产生 3 个 CPU 密集型负载）。
- `--io 1`：创建 1 个进程，循环调用 `sync()` 系统调用，产生 **I/O 压力**（刷写缓冲区到磁盘）。
- `--vm 1`：创建 1 个进程来分配并反复操作内存，模拟 **内存压力。**
- `--vm-bytes 1M`：每个内存压力进程使用的内存大小为 **1 MB。**这里只有一个 `--vm` 进程，所以总占用约 1 MB 内存（用于不断的分配和释放/重写操作）。
- `--timeout 1m`：设置整个 `stress` 运行的超时时间为 **1分钟**，超时后所有子进程自动终止。加上这个防止一直运行。
- **末尾 `&`**：将整个命令放到 **后台执行**，不阻塞终端。

- 运行结果

```
root@rk3506-buildroot:/# cat /proc/version
Linux version 6.1.118-rt36 (linux@ubuntu) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile
Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture
10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #1 SMP PREEMPT_RT Thu Jul  9 15:35:27 CST 2026
root@rk3506-buildroot:/#
root@rk3506-buildroot:/
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# echo 2 > /proc/sys/vm/overcommit_memory
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# stress -c 3 --io 1 --vm 1 --vm-bytes 1M --timeout 1m &
[2] 1634
root@rk3506-buildroot:/# stress: info: [1634] dispatching hogs: 3 cpu, 1 io, 1 vm, 0 hdd

root@rk3506-buildroot:/# cyclictest -c 0 -m -t 3 -p 99
# /dev/cpu_dma_latency set to 0us
popolicy: fifo: loadavg: 4.96 1.59 0.57 7/158 1688
T: 0 ( 1686) P:99 I:1000 C:  33169 Min:      1 Act:    8 Avg:    4 Max:      36
T: 1 ( 1687) P:99 I:1500 C:  22086 Min:      1 Act:    4 Avg:    5 Max:      31
T: 2 ( 1688) P:99 I:2000 C:  16544 Min:      1 Act:    5 Avg:    5 Max:      20
^Croot@rk3506-buildroot:/#
```

- **PREEMPT_RT 内核带来的革命性改变**
  未打 RT 补丁的 PREEMPT 内核虽然在大部分时间表现良好，但仍存在少量长延迟路径（如持有自旋锁时禁止抢占），导致最大延迟达到 100µs。RT 补丁将这些路径全部转化为可抢占上下文，将最差情况控制在 36µs 以内，非常适合 **硬实时场景**（如运动控制、高速数据采集、实时音频处理）。
- **当前平台的实时能力评级**
  在 1ms 周期任务下，最大延迟仅占周期的 3.6%（36µs），**确定性极佳。**即使是 500µs 周期的任务也完全可行。这证明 RK3506 配合 PREEMPT_RT 内核已具备 **工业级微秒级实时响应** 能力。
- **进一步优化空间**
  如果目标延迟需低至 10µs 以下，可尝试：
  - CPU 隔离（`isolcpus=`）+ 实时线程亲和性（`taskset`），将 cyclictest 绑定到独立核心，彻底避开 stress 进程的干扰。
  - 优化设备树与驱动，关闭非必要的电源管理功能，减少额外的时钟域切换开销

6. 隔离CPU的RT内核

隔离cpu2，系统启动后，让ethercat demo运行在cpu2中，从而提高ethercat实时性。

修改kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi，bootargs增加isolcpus=2参数：

- kernel/arch/arm/boot/dts/ido-evb3506-v1a-nand.dtsi

```
        chosen {
                bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0 ubi.mtd=5 root=ubi0:rootfs
rw rootfstype=ubifs rootwait snd_aloop.index=7 snd_aloop.use_raw_jiffies=1 isolcpus=2 rcu_nocbs=2
irqaffinity=0-1 rcu_nocb_poll";
        };


# 参数解释
isolcpus=2              # 隔离 CPU2，普通进程不会自动调度到该核。
rcu_nocbs=2             #将 RCU 回调处理移出 CPU2，这样可以防止 RCU 软中断在 CPU2 上执行
irqaffinity=0-1         #默认让硬件中断只落在 CPU0-1 上
rcu_nocb_poll           #：让 `rcuog` 线程持续轮询，以功耗换确定性，消除唤醒抖动。
# 四者协同，把 CPU2 变成低噪声实时专核。
```

编译内核并下载内核到板子内。

查看隔离内核

```
root@rk3506-buildroot:/# cat /sys/devices/system/cpu/isolated
2
root@rk3506-buildroot:/#
```

绑定任务到第3颗核心上

```
taskset -c 2 cyclictest -t 1 -p 99 -m -S -D 1m
```

- `taskset -c 2`：将 cyclictest 绑定到 CPU 核心 2，防止迁移
- `-t 1`：只创建 1 个测量线程
- `-p 99`：SCHED_FIFO 优先级 99
- `-m`：锁定内存（`mlockall`）
- `-S`：SMP 模式（单线程时会自动适应）
- `-D 1m`：**运行总时长 1 分钟**，时间到后自动终止并打印统计报告

压力隔离内核测试

```
root@rk3506-buildroot:/# cat /proc/version
Linux version 6.1.118-rt36 (linux@ubuntu) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for the A-profile
Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture
10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #1 SMP PREEMPT_RT Thu Jul  9 15:35:27 CST 2026
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# cat /sys/devices/system/cpu/isolated
2
root@rk3506-buildroot:/#
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/# taskset -c 2 cyclictest -t 1 -p 99 -m -S -D 1m
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 1.31 1.16 0.59 1/143 1663

T: 0 ( 1662) P:99 I:1000 C:  43467 Min:      0 Act:    1 Avg:    2 Max:       7
^Croot@rk3506-buildroot:/#
```

- **最大延迟：7 µs**
- **平均延迟：2 µs**
- **最小延迟：0 µs**

- **周期：1000 µs (1 ms)**

在 `PREEMPT_RT` 内核 + `isolcpus=2` + 绑核 + 内存锁定 的配合下，系统负载仅 1.31，CPU2 几乎没有干扰源。
7 µs 的最大延迟仅占 1 ms 周期的 **0.7%**，确定性极高，已达微秒级硬实时水准。

21.2 IgH 交叉编译

1. 安装编译依赖环境

```
sudo apt-get install autoconf autogen libtool pkg-config -y
```

2. 下载开源ethercat源码

直接从github下载：

```
git clone -b stable-1.6 https://gitlab.com/etherlab.org/ethercat.git
```

- 运行结果

```
linux@ubuntu:~/rk3506$ sudo apt-get install autoconf autogen libtool pkg-config -y
[sudo] linux 的密码：
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
autoconf 已经是最新版 (2.71-2)。
libtool 已经是最新版 (2.4.6-15build2)。
pkg-config 已经是最新版 (0.29.2-1ubuntu3)。
pkg-config 已设置为手动安装。
下列【新】软件包将被安装：
  autogen autogen-doc libopts25 libopts25-dev
升级了 0 个软件包，新安装了 4 个软件包，要卸载 0 个软件包，有 39 个软件包未被升级。
需要下载 1,713 kB 的归档。
解压缩后会消耗 4,866 kB 的额外空间。
获取:1 http://mirrors.aliyun.com/ubuntu jammy/universe amd64 libopts25 amd64 1:5.18.16-4 [59.5 kB]
获取:2 http://mirrors.aliyun.com/ubuntu jammy/universe amd64 libopts25-dev amd64 1:5.18.16-4 [95.9 kB]
获取:3 http://mirrors.aliyun.com/ubuntu jammy/universe amd64 autogen amd64 1:5.18.16-4 [558 kB]
获取:4 http://mirrors.aliyun.com/ubuntu jammy/universe amd64 autogen-doc all 1:5.18.16-4 [1,000 kB]
已下载 1,713 kB，耗时 1秒 (1,834 kB/s)
正在选中未选择的软件包 libopts25:amd64。
(正在读取数据库 ... 系统当前共安装有 246746 个文件和目录。)
准备解压 .../libopts25_1%3a5.18.16-4_amd64.deb  ...
正在解压 libopts25:amd64 (1:5.18.16-4) ...
正在选中未选择的软件包 libopts25-dev:amd64。
准备解压 .../libopts25-dev_1%3a5.18.16-4_amd64.deb  ...
正在解压 libopts25-dev:amd64 (1:5.18.16-4) ...
正在选中未选择的软件包 autogen。
准备解压 .../autogen_1%3a5.18.16-4_amd64.deb  ...
正在解压 autogen (1:5.18.16-4) ...
被已安装的软件包 libopts25-dev:amd64 (1:5.18.16-4) 中的文件替换了...
正在选中未选择的软件包 autogen-doc。
准备解压 .../autogen-doc_1%3a5.18.16-4_all.deb  ...
正在解压 autogen-doc (1:5.18.16-4) ...
正在设置 libopts25:amd64 (1:5.18.16-4) ...
正在设置 autogen-doc (1:5.18.16-4) ...
正在设置 libopts25-dev:amd64 (1:5.18.16-4) ...
正在设置 autogen (1:5.18.16-4) ...
正在处理用于 libc-bin (2.35-0ubuntu3.13) 的触发器 ...
正在处理用于 man-db (2.10.2-1) 的触发器 ...
正在处理用于 install-info (6.8-4build1) 的触发器 ...
linux@ubuntu:~/rk3506$ git clone -b stable-1.6 https://gitlab.com/etherlab.org/ethercat.git
正克隆到 'ethercat'...
remote: Enumerating objects: 23757, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 23757 (delta 0), reused 0 (delta 0), pack-reused 23754 (from 1)
接收对象中: 100% (23757/23757), 47.57 MiB | 1.98 MiB/s, 完成.
处理 delta 中: 100% (18165/18165), 完成.
linux@ubuntu:~/rk3506$ ls
app        common  ethercat  kernel      oebuild.log  README.md   rockdev  tools
buildroot  device  external  kernel-6.1  output       rkbin       rootfs   u-boot
build.sh   docs    hal       Makefile    prebuilts    rkflash.sh  rtos     yocto
linux@ubuntu:~/rk3506$
linux@ubuntu:~/rk3506$ ls ethercat/
AUTHORS         CONTRIBUTING.md  Doxyfile.in       FEATURES.md  lib              README.EoE  tty
bootstrap       COPYING          doxygen-layout    globals.h    Makefile.am      README.md
build.sh        COPYING.LESSER   ethercat.spec.in  include      Makefile.kbuild  script
CodingStyle.md  devices          examples          INSTALL.md   master           TODO
configure.ac    documentation    fake_lib          Kbuild.in    NEWS.md          tool
linux@ubuntu:~/rk3506$
```

3. 编译ethercat源码

编译EtherCAT IgH , 瑞芯微 SDK 里面默认有 EtherCAT IgH 的主站源码，我们直接用这个主站的源码就可以。

- 创建编译配置文件
- external/ethercat_igh/build.sh

```
#!/bin/bash
set -euo pipefail

ETHERCAT_DIR="/home/linux/rk3506/external/ethercat_igh"
cd "$ETHERCAT_DIR"

BUILD_DIR="${ETHERCAT_DIR}/out"
DEST="/home/linux/rk3506/buildroot/output/rockchip_rk3506/target"
KERNEL_DIR="/home/linux/rk3506/kernel"
KVER="6.1.118-rt36"
TOOLCHAIN2="/home/linux/rk3506/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-"

# ==================== 前置检查 ====================

if [ ! -d "$DEST" ]; then
    echo "Error: 目标目录不存在: $DEST"
    echo "请先编译 buildroot"
    exit 1
fi

echo "目标目录: $DEST"
echo "内核版本: $KVER"

# ==================== 编译 ====================

echo "=== 1/5 配置 ==="
if ! ./bootstrap 2>/dev/null; then
    echo "Warning: bootstrap 失败，尝试继续（可能 configure 已存在）"
fi

TC_PREFIX="${DEST}/../host/bin/arm-buildroot-linux-gnueabihf-"

./configure --prefix="${BUILD_DIR}" \
            --with-linux-dir="${KERNEL_DIR}" \
            --enable-generic=yes \
            --enable-8139too=no \
            --enable-e100=no \
            --enable-e1000=no \
            --enable-e1000e=no \
            --enable-r8169=no \
            --enable-ccat=no \
            --enable-igb=no \
            --enable-igc=no \
            --with-systemdsystemunitdir=no \
            --build=x86_64-pc-linux-gnu \
            --host=arm-buildroot-linux-gnueabihf \
            CC="${TC_PREFIX}gcc" \
            CXX="${TC_PREFIX}g++" \
            AR="${TC_PREFIX}ar" \
            LD="${TC_PREFIX}ld" \
            RANLIB="${TC_PREFIX}ranlib" \
            STRIP="${TC_PREFIX}strip"

echo "=== 2/5 编译用户空间 ==="
make -j"$(nproc)"

echo "=== 3/5 编译内核模块 ==="
make ARCH=arm CROSS_COMPILE="${TOOLCHAIN2}" modules

echo "=== 4/5 安装 ==="
make install

# ==================== 复制到目标 ====================

echo "=== 5/5 复制到目标 ==="
```

```
MODULES_DIR="${DEST}/lib/modules/${KVER}"
mkdir -p \
    "${MODULES_DIR}" \
    "${DEST}/usr/bin" \
    "${DEST}/usr/sbin" \
    "${DEST}/usr/lib" \
    "${DEST}/etc/sysconfig" \
    "${DEST}/etc/init.d" \
    "${DEST}/usr/share/bash-completion/completions"

# 内核驱动模块
cp -vf master/ec_master.ko      "${MODULES_DIR}/"
cp -vf devices/ec_generic.ko    "${MODULES_DIR}/"
cp -vf examples/mini/ec_mini.ko "${MODULES_DIR}/"

# 生成模块依赖（depmod -b 专为交叉编译场景设计，直接使用宿主机 depmod 即可）
depmod -b "${DEST}" "${KVER}"

# 应用程序
cp -vf "${BUILD_DIR}/bin/ethercat"     "${DEST}/usr/bin/"
cp -vf "${BUILD_DIR}/sbin/ethercatctl" "${DEST}/usr/sbin/"

relocate_paths() {
    sed -i \
        -e "s|${BUILD_DIR}/bin/|/usr/bin/|g" \
        -e "s|${BUILD_DIR}/sbin/|/usr/sbin/|g" \
        -e "s|${BUILD_DIR}/etc/|/etc/|g" \
        "$1"
}

relocate_paths "${DEST}/usr/sbin/ethercatctl"

# 共享库
cp -vf "${BUILD_DIR}/lib/libethercat.so.1.1.0" "${DEST}/usr/lib/"
ln -sf libethercat.so.1.1.0 "${DEST}/usr/lib/libethercat.so.1"
ln -sf libethercat.so.1.1.0 "${DEST}/usr/lib/libethercat.so"

# 配置文件
cp -vf "${BUILD_DIR}/etc/ethercat.conf"       "${DEST}/etc/"
cp -vf "${BUILD_DIR}/etc/init.d/ethercat"     "${DEST}/etc/init.d/"
cp -vf "${BUILD_DIR}/etc/sysconfig/ethercat" "${DEST}/etc/sysconfig/"

configure_ethercat_config() {
    local cfg="$1"

    [ -f "${cfg}" ] || return 0

    sed -i 's/^MASTER0_DEVICE=""/MASTER0_DEVICE="eth0"/' "${cfg}"
    sed -i 's/^DEVICE_MODULES=""/DEVICE_MODULES="generic"/' "${cfg}"
    sed -i 's/^UPDOWN_INTERFACES=""/UPDOWN_INTERFACES="eth0"/' "${cfg}"

    if grep -q '^MASTER_MODULE_OPTIONS=' "${cfg}"; then
        sed -i 's/^MASTER_MODULE_OPTIONS=.*/MASTER_MODULE_OPTIONS="thread_cpu=2 idle_thread_rt_prio=15
op_thread_rt_prio=42"/' "${cfg}"
    else
        printf '\nMASTER_MODULE_OPTIONS="thread_cpu=2 idle_thread_rt_prio=15 op_thread_rt_prio=42"\n' >>
"${cfg}"
    fi
}

# 设置默认 EtherCAT 配置：
# - MASTER0_DEVICE=eth0: 主站绑定到 eth0
# - DEVICE_MODULES=generic: 使用 ec_generic 适配层
# - UPDOWN_INTERFACES=eth0: 启停主站时自动拉起/拉低 eth0
# - MASTER_MODULE_OPTIONS=...: 将主站内核线程固定到 CPU2，且优先级低于周期线程
configure_ethercat_config "${DEST}/etc/ethercat.conf"
configure_ethercat_config "${DEST}/etc/sysconfig/ethercat"

# 修正 init 脚本中写死的路径
relocate_paths "${DEST}/etc/init.d/ethercat"

# bash 命令补全
cp -vf "${BUILD_DIR}/share/bash-completion/completions/ethercat" \
    "${DEST}/usr/share/bash-completion/completions/"

echo "=== 完成 ==="
```

- 运行结果

```
linux@ubuntu:~/rk3506$ cd external/ethercat_igh/
linux@ubuntu:~/rk3506/external/ethercat_igh$ ls
AUTHORS         configure.ac    devices           examples     INSTALL     Makefile.am      README.EoE  tool
bootstrap       COPYING         documentation     FEATURES.md  INSTALL.md  Makefile.kbuild  README.md   tty
build.sh        COPYING.LESSER  Doxyfile.in       globals.h    Kbuild.in   master           script
CodingStyle.md  demo            ethercat.spec.in  include      lib         NEWS             TODO
linux@ubuntu:~/rk3506/external/ethercat_igh$
linux@ubuntu:~/rk3506/external/ethercat_igh$ touch build.sh
linux@ubuntu:~/rk3506/external/ethercat_igh$ chmod a+x build.sh
linux@ubuntu:~/rk3506/external/ethercat_igh$
linux@ubuntu:~/rk3506/external/ethercat_igh$ ./build.sh
目标目录: /home/linux/rk3506/buildroot/output/rockchip_rk3506/target
内核版本: 6.1.118-rt36
=== 1/5 配置 ===
configure: WARNING: unrecognized options: --enable-igc
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for arm-buildroot-linux-gnueabihf-strip... no
checking for strip... strip
checking for a race-free mkdir -p... /usr/bin/mkdir -p
.......
config.status: config.h is unchanged
config.status: executing depfiles commands
config.status: executing libtool commands
configure: WARNING: unrecognized options: --enable-igc
=== 2/5 编译用户空间 ===
make  all-recursive
make[1]: 进入目录“/home/linux/rk3506/external/ethercat_igh”
Making all in include
.......
make[3]: 离开目录“/home/linux/rk3506/external/ethercat_igh/examples”
make[2]: 离开目录“/home/linux/rk3506/external/ethercat_igh/examples”
make[2]: 进入目录“/home/linux/rk3506/external/ethercat_igh”
make[2]: 离开目录“/home/linux/rk3506/external/ethercat_igh”
make[1]: 离开目录“/home/linux/rk3506/external/ethercat_igh”
=== 3/5 编译内核模块 ===
make -C "/home/linux/rk3506/kernel" M="/home/linux/rk3506/external/ethercat_igh" INSTALL_MOD_DIR="ethercat"
modules
make[1]: 进入目录“/home/linux/rk3506/kernel-6.1”
make[1]: 离开目录“/home/linux/rk3506/kernel-6.1”
=== 4/5 安装 ===
Making install in include
make[1]: 进入目录“/home/linux/rk3506/external/ethercat_igh/include”
make[2]: 进入目录“/home/linux/rk3506/external/ethercat_igh/include”
make[2]: 对“install-exec-am”无需做任何事。
........
make[2]: 离开目录“/home/linux/rk3506/external/ethercat_igh”
make[1]: 离开目录“/home/linux/rk3506/external/ethercat_igh”
=== 5/5 复制到目标 ===
'master/ec_master.ko' -> '/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/lib/modules/6.1.118-
rt36/ec_master.ko'
'devices/ec_generic.ko' -> '/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/lib/modules/6.1.118-
rt36/ec_generic.ko'
'examples/mini/ec_mini.ko' ->
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/lib/modules/6.1.118-rt36/ec_mini.ko'
'out/bin/ethercat' -> '/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/bin/ethercat'
'out/sbin/ethercatctl' -> '/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/sbin/ethercatctl'
'out/lib/libethercat.so.1.1.0' ->
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/lib/libethercat.so.1.1.0'
'out/etc/ethercat.conf' -> '/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/etc/ethercat.conf'
'out/etc/init.d/ethercat' ->
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/etc/init.d/ethercat'
'out/etc/sysconfig/ethercat' ->
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/etc/sysconfig/ethercat'
'out/share/bash-completion/completions/ethercat' ->
'/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/share/bash-completion/completions/ethercat'
=== 完成 ===
linux@ubuntu:~/rk3506/external/ethercat_igh$
```

编译文件系统并下载到开发板内。

4. IgH主站测试

- 启用busybox中的modinfo

```
root@rk3506-buildroot:/# ./build.sh bmake:busybox-menuconfig
Linux System Utilities
    Linux Module Utilities  --->
       -> [*] modinfo
```

保存编译文件系统，并下载文件系统到开发板内。

- 开发板上执行

```
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/# /etc/init.d/ethercat start
Starting EtherCAT master 1.6.0-rc1 [   29.929926] EtherCAT: Master driver 1.6.0-rc1 unknown
[   29.932957] EtherCAT: 1 master waiting for devices.
[   29.970156] ec_generic: EtherCAT master generic Ethernet device module 1.6.0-rc1 unknown
[   29.970246] EtherCAT: Accepting 26:64:07:3A:AB:BD as main device for master 0.
[   29.970283] ec_generic: Binding socket to interface 3 (eth0).
[   29.974607] EtherCAT 0: Starting EtherCAT-IDLE thread.
[   29.974807] EtherCAT 0: Link state of ecm0 changed to UP.
[   29.979212] EtherCAT WARNING 0: 1 datagram TIMED OUT!
 done
root@rk3506-buildroot:/# [   30.979586] EtherCAT WARNING 0: 465 datagrams TIMED OUT!
[   31.980570] EtherCAT WARNING 0: 469 datagrams TIMED OUT!
[   32.980566] EtherCAT WARNING 0: 477 datagrams TIMED OUT!
[   33.980564] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   34.981567] EtherCAT WARNING 0: 487 datagrams TIMED OUT!
[   35.982566] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   36.983568] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   37.983563] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   38.985014] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   39.452585] temp 45 state 1 th:90 105 125 opt:0 0 0
[   39.984566] EtherCAT WARNING 0: 496 datagrams TIMED OUT!
[   40.985566] EtherCAT WARNING 0: 489 datagrams TIMED OUT!
[   41.985563] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   42.985604] EtherCAT WARNING 0: 495 datagrams TIMED OUT!
[   43.985631] EtherCAT WARNING 0: 500 datagrams TIMED OUT!
[   44.985563] EtherCAT WARNING 0: 496 datagrams TIMED OUT!
[   45.985565] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   46.985606] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   47.985798] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   48.986563] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   49.987564] EtherCAT WARNING 0: 494 datagrams TIMED OUT!
[   50.988563] EtherCAT WARNING 0: 498 datagrams TIMED OUT!
[   51.988749] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   52.989566] EtherCAT WARNING 0: 497 datagrams TIMED OUT!
[   53.622090] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Down
[   53.622789] EtherCAT 0: Link state of ecm0 changed to DOWN.
[   58.742475] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off
[   58.743758] EtherCAT 0: Link state of ecm0 changed to UP.
[   58.746729] EtherCAT 0: 1 slave(s) responding on main device.
[   58.746749] EtherCAT 0: Slave states on main device: INIT.
[   58.748416] EtherCAT 0: Scanning bus.
[   59.004437] EtherCAT 0: Bus scanning completed in 257 ms.
[   59.004488] EtherCAT 0: Using slave 0 as DC reference clock.
[   59.005775] EtherCAT 0: Slave states on main device: PREOP.
[   59.094620] EtherCAT WARNING 0: 316 datagrams TIMED OUT!
[   60.093626] EtherCAT ERROR 0-0: Failed to receive AL state datagram: Datagram timed out.
[   60.095931] EtherCAT WARNING 0: 5 datagrams TIMED OUT!
[   60.095948] EtherCAT WARNING 0: 3 datagrams UNMATCHED!
[   61.096612] EtherCAT WARNING 0: 3 datagrams TIMED OUT!
[   61.096630] EtherCAT WARNING 0: 4 datagrams UNMATCHED!
[   62.096618] EtherCAT WARNING 0: 2 datagrams TIMED OUT!
[   62.096637] EtherCAT WARNING 0: 2 datagrams UNMATCHED!
[   73.086629] EtherCAT WARNING 0: 1 datagram TIMED OUT!
[   73.086648] EtherCAT WARNING 0: 2 datagrams UNMATCHED!
[   73.092761] EtherCAT ERROR 0-0: Failed to receive AL state datagram: Datagram timed out.

root@rk3506-buildroot:/#
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# [   75.086819] EtherCAT WARNING 0: 6 datagrams TIMED OUT!
[   75.086877] EtherCAT WARNING 0: 6 datagrams UNMATCHED!
[   75.092752] EtherCAT 0: 0 slave(s) responding on main device.
[   75.095014] EtherCAT 0: 1 slave(s) responding on main device.
[   75.095031] EtherCAT 0: Slave states on main device: PREOP.
[   75.100193] EtherCAT 0: Scanning bus.
[   75.348278] EtherCAT 0: Bus scanning completed in 253 ms.
[   75.348369] EtherCAT 0: Using slave 0 as DC reference clock.
```

这是正常现象，说明 EtherCAT 组件都工作正常：

状态	说明
`Master driver 1.6.0-rc1 unknown`	主站驱动加载成功 ✅
`1 master waiting for devices`	主站就绪，等待从站 ✅
`ec_generic: ... Binding socket to interface 3 (eth0)`	通用网卡驱动绑定 eth0 成功 ✅
`Link state of ecm0 changed to UP`	物理链路已接通 ✅
`datagrams TIMED OUT`	没有 EtherCAT 从站设备应答 ⚠️

`TIMED OUT` 是因为 **eth0 上没有实际连接 EtherCAT 从站。**

主站在循环发送数据帧但收不到响应，这是预期行为——接上从站设备就会消失。

21.3 电机从站命令解析

1. 硬件接线

- 硬件介绍

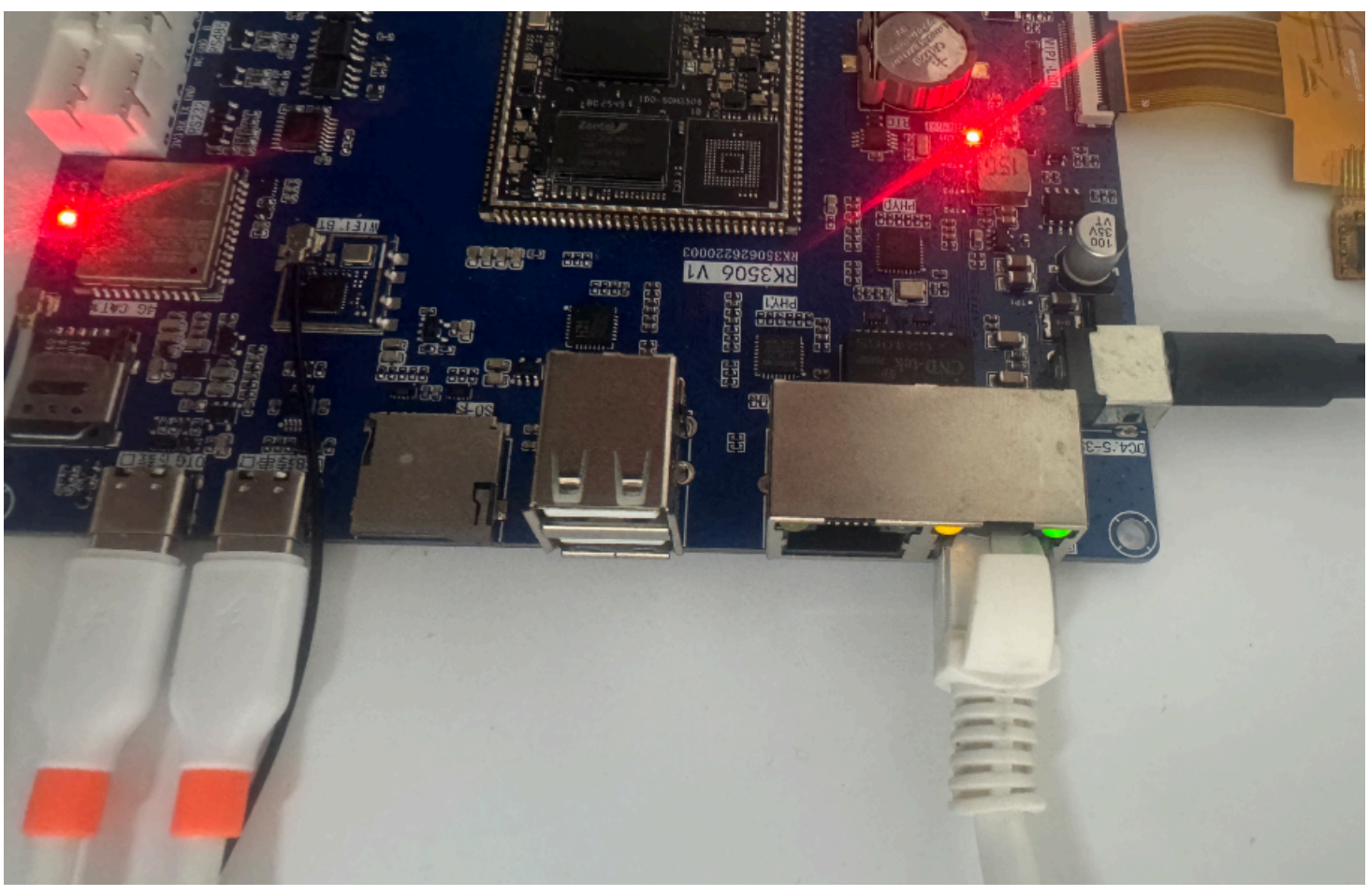
深圳新川电机 CL3-E57H (CiA 402 伺服驱动)，默认 CSP 模式，Xinchuan DS2-E (以及其他标准 CiA402 驱动器)。

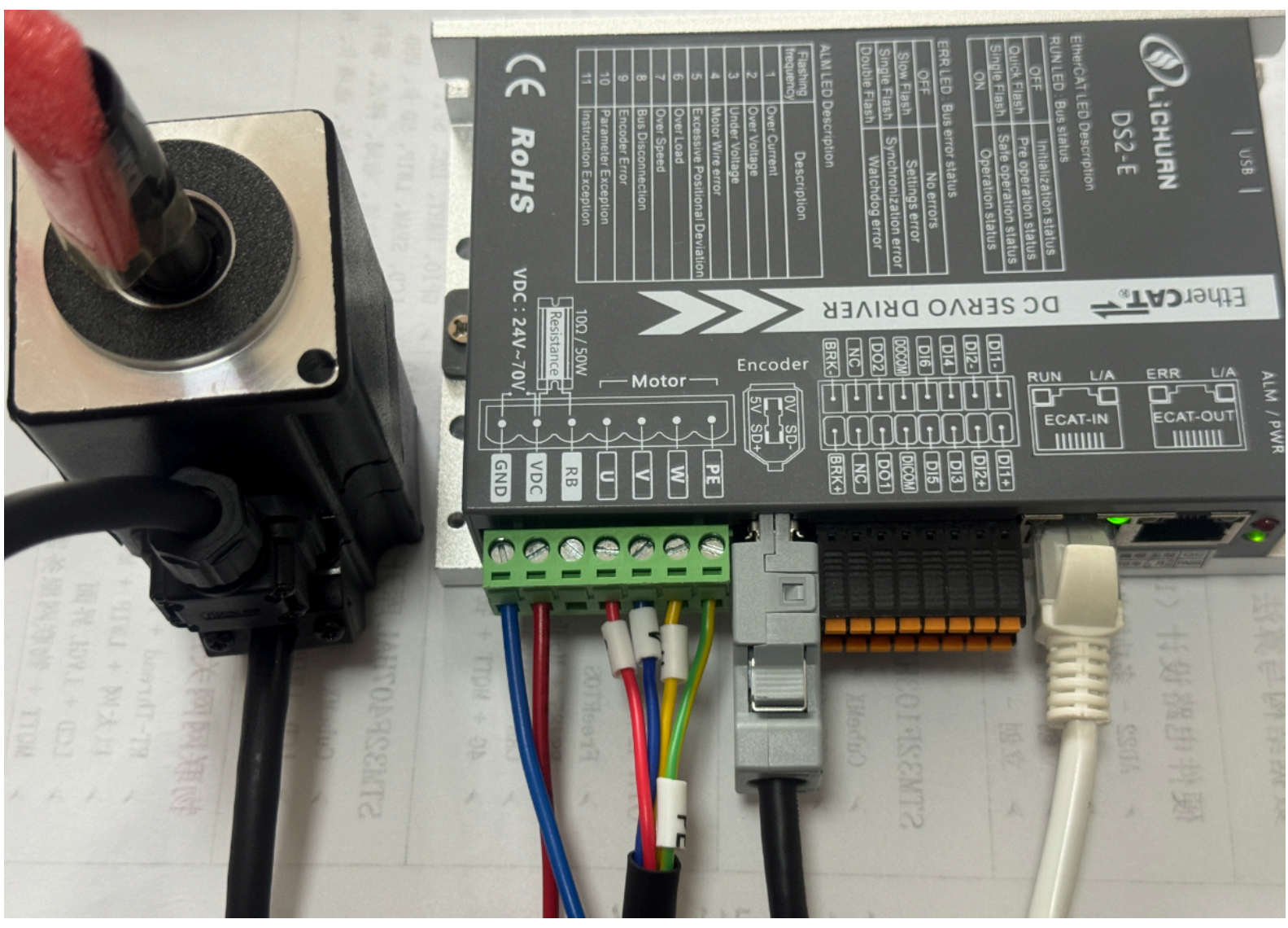
- 力川伺服驱动器淘宝链接

```
https://detail.tmall.com/item.htm?
from=cart&id=684535074515&mi_id=0000ncg7I7Zh6cEiTNtXoXsr88_jpnbXBvG7V9i3QLVL-
uI&skuId=6039033385671&spm=a1z0d.6639537%2F202410.item.d684535074515.350b7484Bplilm&upStreamPrice=70000
```

- 电机接线

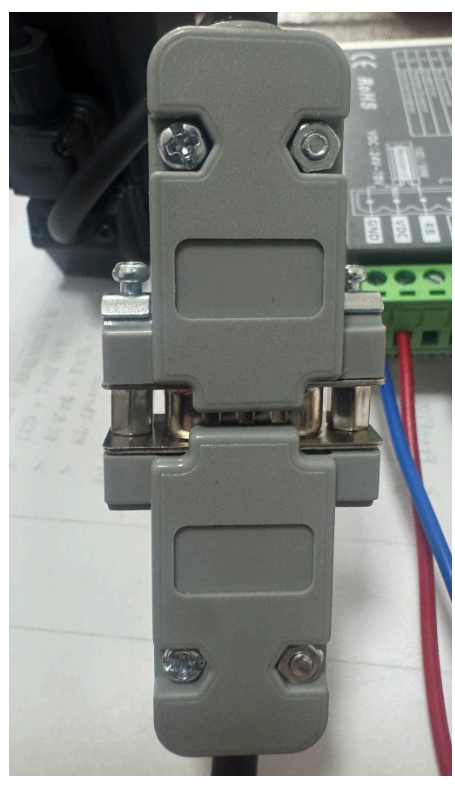
RK3506开发板eth0网口与伺服驱动器Ethercat-IN网口处于同一局域网内。





- 电源供电24V
  - 红色：VCC
  - 黑色：GND

- 接头部分



2. 电机测试脚本

深圳新川电机 CL3-E57H (CiA 402 伺服驱动)，默认 CSP 模式。以下是完整的测试命令过程：

- external/ethercat_igh/motor_test.sh

```
#!/bin/bash
# ============================================================================
#  EtherCAT 入门脚本  --  一步步看懂 IgH 主站和 CiA402 伺服驱动器
#
#  适配:  Xinchuan CL3-E57H (以及其他标准 CiA402 驱动器)
#  接口:  ethercat 命令行工具 (IgH EtherCAT Master 自带)
#  原则:  只读不写  -  所有命令都是 upload (SDO 读取), 不会让电机转动
#
#  EtherCAT 基本概念 (边看边对照):
#  ┌──────────────┐          ┌──────────────┐          ┌──────────────┐
#  |  主站 Master  |==网线==|  从站 Slave 0 |==网线==|  从站 Slave 1 | ...
#  └──────────────┘          └──────────────┘          └──────────────┘
#
#  主站 = 你的 Linux + IgH 驱动, 从站 = 伺服驱动器/IO 模块等
#  EtherCAT 帧由主站发出, 经过每个从站时 "飞读飞写", 一圈回来数据就到了
#
#  两种数据通道 (关键区别):
#    SDO (Service Data Object)     = 邮箱通信, 慢, 类似 "打电话问一下", 用于配置
#    PDO (Process Data Object)     = 过程数据, 快, 类似 "流水线传送带", 用于实时控制
#                                                                  - 只读 SDO
#  从站状态机:  INIT → PREOP → SAFEOP → OP  (SDO 需要 ≥ PREOP, PDO 需要 OP)
#
#   用法:
#     ./motor_test.sh master   看看主站和几个从站
#     ./motor_test.sh slave    读从站的身份证 (厂家/型号/版本)
#     ./motor_test.sh sdo      浏览从站有多少个 SDO 对象
#     ./motor_test.sh pdo      看 PDO 映射: 实时通信时传哪些数据
#     ./motor_test.sh status   读当前运行状态 (位置/速度/状态字)
#     ./motor_test.sh cstruct  把 CiA402 对象字典打印成 C 结构体参考
#
#  环境变量 (可跳过, 默认就对单从站):
#    MOTOR_POS=1 ./motor_test.sh status    # 如果从站不是位置0
# ============================================================================

# ============================================================================
# 准备: 确定从站位置和 ethercat 命令路径
# ============================================================================
POS=${MOTOR_POS:-0}                       # 从站 0 (第一个从站), 可用环境变量改
```

```
EC=${ETHERCAT_BIN:-ethercat}            # 默认用 PATH 里的 ethercat
# 下面两行只是想让你看到: ethercat 命令后面总是带 -p位置  表示"对哪个从站说"
echo "== 目标从站: -p$POS  (ethercat 路径: $EC)"
echo ""

# ============================================================================
# 命令 1: master  -  主站状态 + 从站列表
# ============================================================================
show_master() {
    echo "============================================================================"
    echo "  1.1  ethercat master  -  主站在不在?"
    echo "============================================================================"
    echo ""
    echo "  \$ ethercat master"
    echo ""
    # ----- 实际命令 -----
    $EC master
    echo ""
    echo "  关注: Phase 是否 = Operation  (主站必须 Operation 从站才能 OP)"
    echo "        Slaves 数目是否正确"
    echo ""

    echo "============================================================================"
    echo "  1.2  ethercat slaves  -  几个从站, 各自什么状态?"
    echo "============================================================================"
    echo ""
    echo "  \$ ethercat slaves"
    echo ""
    # ----- 实际命令 -----
    $EC slaves
    echo ""
    echo "  输出解读:"
    echo "    第一列 = 从站位置 (0,1,2...)"
    echo "    第二列 = 从站状态:"
    echo "      INIT   = 刚上电, 只识别身份"
    echo "      PREOP  = 可以 SDO 通信 (我们能读参数)"
    echo "      SAFEOP = PDO 输入生效, 输出安全值"
    echo "      OP     = 正常运行, PDO 双向通信"
    echo "    第三列 + Alias + 设备名"
    echo ""
}

# ============================================================================
# 命令 2: slave  -  从站身份信息
# ============================================================================
show_slave() {
    echo "============================================================================"
    echo "  2.1  设备名 + 版本号  (SDO 读对象字典 0x1008~0x100A)"
    echo "============================================================================"
    echo ""
    echo "  SDO 读 = ethercat upload -p<从站> --type <数据类型> <索引> <子索引>"
    echo ""

    echo "  [对象 0x1008] 设备名:"
    echo "  \$ ethercat upload -p$POS --type string 0x1008 0"
    $EC upload -p$POS --type string 0x1008 0
    echo ""

    echo "  [对象 0x1009] 硬件版本:"
    echo "  \$ ethercat upload -p$POS --type string 0x1009 0"
    $EC upload -p$POS --type string 0x1009 0
    echo ""

    echo "  [对象 0x100A] 固件版本:"
    echo "  \$ ethercat upload -p$POS --type string 0x100A 0"
    $EC upload -p$POS --type string 0x100A 0
    echo ""

    echo "============================================================================"
    echo "  2.2  身份标识  (0x1018 Identity Object)"
    echo "============================================================================"
    echo ""

    echo "  [0x1018:01] 厂商 ID:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1018 1"
    $EC upload -p$POS --type uint32 0x1018 1
    echo "  (0xA79 = 深圳新川 Xinchuan)"
    echo ""

    echo "  [0x1018:02] 产品型号:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1018 2"
    $EC upload -p$POS --type uint32 0x1018 2
    echo ""

    echo "  [0x1018:03] 硬件版本号:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1018 3"
    $EC upload -p$POS --type uint32 0x1018 3
    echo ""

    echo "  [0x1018:04] 序列号:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1018 4"
    $EC upload -p$POS --type uint32 0x1018 4
    echo ""
}

# ============================================================================
# 命令 3: sdo  -  浏览从站的 SDO 字典 (配置参数清单)
# ============================================================================
show_sdo() {
    echo "============================================================================"
    echo "  3.1  ethercat sdos  -  这个从站有多少可读写的 SDO?"
    echo "============================================================================"
    echo ""
    echo "  SDO 字典 = 从站的 "参数清单", 每个对象有一个 16-bit 索引 + 8-bit 子索引"
    echo "  0x1000~0x1FFF = 通信对象     (Communication Profile)"
    echo "  0x2000~0x5FFF = 厂商自定义   (Manufacturer Specific)"
    echo "  0x6000~0x9FFF = 设备行规对象  (如 CiA402 电机行规)"
    echo ""
    echo "  \$ ethercat sdos -p$POS | head -60"
    echo ""
    $EC sdos -p$POS 2>/dev/null | head -60
    echo "  ..."
    echo ""

    echo "============================================================================"
    echo "  3.2  手工读几个关键 SDO 看看值"
    echo "============================================================================"
    echo ""
    echo "  -- 通信参数 (0x1000~0x1FFF) --"
    echo ""

    echo "  [0x1000:00] 设备类型 (CiA402 电机 = 0x00020192),"
    echo "              告诉主站"我是干什么的":"
    $EC upload -p$POS --type uint32 0x1000 0  2>/dev/null
    echo ""

    echo "  [0x1C00:00] 同步管理器类型, 决定 SM2/SM3 怎么映射 PDO:"
    $EC upload -p$POS --type uint8 0x1C00 0  2>/dev/null
    echo ""

    echo "  [0x1C12:00] SM2 分配的 RxPDO 个数:"
    $EC upload -p$POS --type uint8 0x1C12 0  2>/dev/null
    echo ""

    echo "  [0x1C13:00] SM3 分配的 TxPDO 个数:"
    $EC upload -p$POS --type uint8 0x1C13 0  2>/dev/null
    echo ""

    echo "  -- CiA402 电机行规 (0x6040~) --"
    echo ""

    echo "  [0x6040:00] 控制字 (ControlWord) - 上位机写这个来启停电机:"
    $EC upload -p$POS --type uint16 0x6040 0  2>/dev/null
    echo ""

    echo "  [0x6041:00] 状态字 (StatusWord) - 驱动器回报当前状态:"
    $EC upload -p$POS --type uint16 0x6041 0  2>/dev/null
    echo ""

    echo "  [0x6060:00] 运行模式 (8=CSP 9=CSV 3=PV ...):"
    $EC upload -p$POS --type int8 0x6060 0  2>/dev/null
    echo ""

    echo "  [0x6061:00] 当前运行模式 (驱动器回报):"
    $EC upload -p$POS --type int8 0x6061 0  2>/dev/null
    echo ""
}
```

```
# ================================================================================
# 命令 4: pdo  -  看 PDO 映射 (快通道传什么数据)
# ================================================================================
show_pdo() {
    echo "================================================================================"
    echo "  4.1  ethercat pdos  -  PDO 映射一览"
    echo "================================================================================"
    echo ""
    echo "  PDO = 每个周期自动交换的数据包, 不用每次发 SDO 读写命令"
    echo "  RxPDO = 主站发给从站 (比如: 控制字 + 目标位置)"
    echo "  TxPDO = 从站发给主站 (比如: 状态字 + 实际位置)"
    echo "  SM2 承载 RxPDO,  SM3 承载 TxPDO"
    echo ""
    echo "  \$ ethercat pdos -p$POS"
    echo ""
    $EC pdos -p$POS
    echo ""

    echo "================================================================================"
    echo "  4.2  PDO 映射寄存器原始值  (RxPDO=0x1600+, TxPDO=0x1A00+)"
    echo "================================================================================"
    echo ""
    echo "  映射条目是一个 32-bit 数, 格式: 0x 索引(16bit) 子索引(8bit) 位宽(8bit)"
    echo "  例如 0x60400010 表示: 对象 0x6040, 子索引 0, 位宽 16 bits"
    echo ""

    echo "  [0x1600] RxPDO 1 的映射条目:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1600 1  2>/dev/null"
    $EC upload -p$POS --type uint32 0x1600 1 2>/dev/null
    echo "  \$ ethercat upload -p$POS --type uint32 0x1600 2  2>/dev/null"
    $EC upload -p$POS --type uint32 0x1600 2 2>/dev/null
    echo ""

    echo "  [0x1A00] TxPDO 1 的映射条目:"
    echo "  \$ ethercat upload -p$POS --type uint32 0x1A00 1  2>/dev/null"
    $EC upload -p$POS --type uint32 0x1A00 1 2>/dev/null
    echo "  \$ ethercat upload -p$POS --type uint32 0x1A00 2  2>/dev/null"
    $EC upload -p$POS --type uint32 0x1A00 2 2>/dev/null
    echo ""
}

# ================================================================================
# 命令 5: status  -  读电机当前运行数据
# ================================================================================
show_status() {
    echo "================================================================================"
    echo "  5.1  ethercat states  -  从站现在什么状态?"
    echo "================================================================================"
    echo ""
    $EC states -p$POS
    echo ""

    echo "================================================================================"
    echo "  5.2  状态字  (0x6041)  -  解码驱动器状态机"
    echo "================================================================================"
    echo ""

    # 读原始值
    RAW=$($EC upload -p$POS --type uint16 0x6041 0 2>/dev/null)
    echo "  [0x6041:00] 状态字原始值: $RAW"

    # 对新手: 直接展示 hex → dec, 然后逐 bit 解释
    HEX=$(echo "$RAW" | awk '{print $1}')
    DEC=$(printf '%d' "$HEX" 2>/dev/null)

    echo ""
    echo "  hex=$HEX  dec=$DEC"
    echo ""

    # 掩码 0x6F = bits 0,1,2,3,5,6  = CiA402 状态机核心位
    STATE=$(( DEC & 0x6F ))
    echo "  核心状态 (sw & 0x6F) = $STATE"
    echo ""

    # 逐 bit 解读 (这是教学重点)
    echo "  逐位解读:"
    echo "  ----------------------------------------------------------------"
    echo "  bit 0  Ready to switch on:  $(( (DEC >> 0) & 1 ))  1=驱动器就绪待命"
    echo "  bit 1  Switched on:         $(( (DEC >> 1) & 1 ))  1=主电已接通"
    echo "  bit 2  Operation enabled:   $(( (DEC >> 2) & 1 ))  1=使能,可以转了!"
    echo "  bit 3  Fault:               $(( (DEC >> 3) & 1 ))  1=故障!看 ErrorCode"
    echo "  bit 4  Voltage enabled:     $(( (DEC >> 4) & 1 ))  1=主电电压正常"
    echo "  bit 5  Quick stop:          $(( (DEC >> 5) & 1 ))  0=急停中/1=正常"
    echo "  bit 6  Switch on disabled:  $(( (DEC >> 6) & 1 ))  1=只能接受 shutdown"
    echo "  bit 7  Warning:             $(( (DEC >> 7) & 1 ))  1=有警告"
    echo "  bit 10 Target reached:      $(( (DEC >> 10) & 1 )) 1=到达目标"
    echo "  bit 11 Internal limit:      $(( (DEC >> 11) & 1 )) 1=碰到内部限位"
    echo "  bit 12 Set-point ack:       $(( (DEC >> 12) & 1 )) 1=已确认设定值"
    echo "  ----------------------------------------------------------------"

    # 故障时读错误码
    if [ $((DEC & 0x08)) -ne 0 ]; then
        echo ""
        echo "  ⚠  bit3=1 有故障! 读错误码:"
        echo "  \$ ethercat upload -p$POS --type uint16 0x603F 0"
        $EC upload -p$POS --type uint16 0x603F 0
    fi

    echo ""
    echo "  状态机流转路径 (手动使能的逻辑):"
    echo "    上电 -> Not Ready (0) -> SwitchOnDisabled (64=0x40)"
    echo "         -> ReadyToSwitchOn (33=0x21) -> SwitchedOn (35=0x23)"
    echo "         -> OperationEnabled (39=0x27) ✅ 电机能转了!"
    echo ""

    # 运行模式
    echo "================================================================================"
    echo "  5.3  运行模式"
    echo "================================================================================"
    echo ""
    echo "  [0x6061:00] 当前运行模式 (驱动器回报):"
    MODE=$($EC upload -p$POS --type int8 0x6061 0 2>/dev/null | awk '{print $2}')
    echo "  原始值: $MODE"
    case $MODE in
        0)  MDESC="未配置" ;;
        1)  MDESC="PP (Profile Position 位置模式)" ;;
        3)  MDESC="PV (Profile Velocity 速度模式)" ;;
        6)  MDESC="HM (Homing 回零)" ;;
        8)  MDESC="CSP (Cyclic Sync Position 周期同步位置)" ;;
        9)  MDESC="CSV (Cyclic Sync Velocity 周期同步速度)" ;;
        10) MDESC="CST (Cyclic Sync Torque 周期同步转矩)" ;;
        *)  MDESC="未知" ;;
    esac
    echo "  含义: $MDESC"
    echo ""
    echo "  常见模式对照:"
    echo "    8 CSP = 主站每个周期发目标位置, 适合多轴插补"
    echo "    9 CSV = 主站发目标速度"
    echo "    1 PP  = 主站发目标位置+速度, 驱动器自己走梯形加减速"
    echo "    3 PV  = 主站发目标速度, 驱动器自己加减速"
    echo ""

    # 实际值
    echo "================================================================================"
    echo "  5.4  实际运行值"
    echo "================================================================================"
    echo ""

    echo "  [0x6064:00] 实际位置 (编码器值, 单位取决于驱动器配置):"
    echo "  \$ ethercat upload -p$POS --type int32 0x6064 0"
    $EC upload -p$POS --type int32 0x6064 0
    echo ""

    echo "  [0x606C:00] 实际速度 (单位取决于驱动器配置):"
    echo "  \$ ethercat upload -p$POS --type int32 0x606C 0"
    $EC upload -p$POS --type int32 0x606C 0
    echo ""

    echo "  [0x6077:00] 实际转矩 (千分之额定转矩):"
    echo "  \$ ethercat upload -p$POS --type int16 0x6077 0"
    $EC upload -p$POS --type int16 0x6077 0
    echo ""

    echo "  [0x60F4:00] 跟随误差 (指令位置 - 实际位置):"
    echo "  \$ ethercat upload -p$POS --type int32 0x60F4 0"
    $EC upload -p$POS --type int32 0x60F4 0
    echo "  (这个值越小越好, 大了说明电机跟不上指令)"
    echo ""
```

```
    echo "  [0x60B9:00] 数字输入 (限位开关/原点信号等):"
    echo "  \$ ethercat upload -p$POS --type uint32 0x60B9 0"
    $EC upload -p$POS --type uint32 0x60B9 0
    echo ""
}

# ============================================================================
# 命令 6: cstruct  —  把 CiA402 对象字典打印成 C 结构体
# ============================================================================
show_cstruct() {
    echo "============================================================================="
    echo "  CiA402 常用对象字典  C 结构体参考"
    echo "============================================================================="
    echo ""
    echo "  把下面内容另存为 ecat_motor.h, 写 IgH 应用时直接 include"
    echo ""

    cat << 'CSTRUCT'
/* ===== 运行指令  —  写 PDO (RxPDO, 主站→从站) ===== */

typedef struct __attribute__((packed)) {
    /* 对象 0x6040 - CiA402 控制字 (使能/停机/故障复位等) */
    uint16_t    CtrlWord;
    /* 对象 0x607A - 目标位置 (CSP 模式), 单位 inc (编码器脉冲) */
    int32_t     TargetPos;
    /* 对象 0x60FF - 目标速度 (CSV/PV 模式) */
    int32_t     TargetVel;
    /* 对象 0x60B2 - 转矩偏移, 单位 0.1% 额定转矩 */
    int16_t     TorqueOffset;
    /* 对象 0x6060 - 运行模式 (1=PP 3=PV 6=HM 8=CSP 9=CSV 10=CST) */
    int8_t      OpMode;
    /* 预留 3 字节对齐 */
    uint8_t     _pad[3];
} rx_pdo_t;

/* ===== 运行反馈  —  读 PDO (TxPDO, 从站→主站) ===== */

typedef struct __attribute__((packed)) {
    /* 对象 0x6041 - 状态字 (bit0-6=状态机, bit3=fault, bit10=到位) */
    uint16_t    StatusWord;
    /* 对象 0x6064 - 实际位置 (编码器读数) */
    int32_t     ActPos;
    /* 对象 0x606C - 实际速度 */
    int32_t     ActVel;
    /* 对象 0x6077 - 实际转矩 (千分之额定) */
    int16_t     ActTorque;
    /* 对象 0x60F4 - 跟随误差 = 目标位置-实际位置 */
    int32_t     FollowErr;
    /* 对象 0x6061 - 当前运行模式 (驱动器回报) */
    int8_t      ModeDisplay;
    /* 对象 0x603F - 故障码 */
    uint16_t    ErrorCode;
    /* 对象 0x60B9 - 数字输入 (bit0=正限位 bit1=负限位 bit2=原点 …) */
    uint32_t    DigitalIn;
} tx_pdo_t;

/* ===== 控制字常用值 (CiA402 状态机) ===== */
#define CW_SHUTDOWN          0x0006  /* 关断 */
#define CW_SWITCH_ON         0x0007  /* 接通主电 */
#define CW_ENABLE            0x000F  /* 使能! */
#define CW_DISABLE_VOLTAGE   0x0000  /* 断开主电 */
#define CW_QUICKSTOP         0x0002  /* 急停 */
#define CW_FAULT_RESET       0x0080  /* 故障复位 (bit7=0→1) */

/* ===== 状态字掩码 ===== */
#define SW_READY_MASK        0x006F  /* 掩掉厂商位, 只看 CiA402 核心位 */
#define SW_FAULT             (1<<3)  /* bit3 = 故障 */
#define SW_TARGET_REACHED    (1<<10) /* bit10 = 到达目标 */
#define SW_WARNING           (1<<7)  /* bit7 = 警告 */

/* ===== CiA402 核心状态值 (sw & 0x6F) ===== */
enum {
    ST_NOT_READY        = 0x00,  /* 0   上电中 */
    ST_SWITCH_ON_DIS    = 0x40,  /* 64  只能发 shutdown */
    ST_READY_SWITCH_ON  = 0x21,  /* 33  准备好接通主电 */
    ST_SWITCHED_ON      = 0x23,  /* 35  主电已通, 等待使能 */
    ST_OP_ENABLED       = 0x27,  /* 39  运转中! */
    ST_QUICK_STOP       = 0x07,  /* 7   急停激活 */
    ST_FAULT_REACT      = 0x0F,  /* 15  故障处理中 */
    ST_FAULT            = 0x08,  /* 8   故障停机 */
};

/* ===== 运行模式 (0x6060/0x6061) ===== */
enum {
    MODE_PP  = 1,   /* Profile Position   位置模式 */
    MODE_PV  = 3,   /* Profile Velocity   速度模式 */
    MODE_HM  = 6,   /* Homing             回零 */
    MODE_CSP = 8,   /* Cyclic Sync Pos    周期同步位置 */
    MODE_CSV = 9,   /* Cyclic Sync Vel    周期同步速度 */
    MODE_CST = 10,  /* Cyclic Sync Trq    周期同步转矩 */
};
CSTRUCT

    echo ""
    echo "============================================================================="
    echo "  对应实际从站的 PDO 映射, 确认结构体与映射一致:"
    echo "============================================================================="
    echo ""
    echo "  \$ ethercat pdos -p$POS"
    echo ""
    $EC pdos -p$POS
    echo ""
    echo "  写代码前务必确认上面的 PDO 条目与结构体字段一一对应!"
    echo "  不同驱动器/不同固件的映射可能不同 (比如有的多了 digital outputs)"
    echo ""
}

# ============================================================================
# 入口: 根据参数选命令
# ============================================================================
case "${1:-help}" in
    master)  show_master  ;;
    slave)   show_slave   ;;
    sdo)     show_sdo     ;;
    pdo)     show_pdo     ;;
    status)  show_status  ;;
    cstruct) show_cstruct ;;

    help|--help|-h)
        echo "用法:  $0 <命令>"
        echo ""
        echo "  master   主站信息 + 从站列表  (第一步从这里开始)"
        echo "  slave    从站身份信息         (厂商/型号/版本)"
        echo "  sdo      从站 SDO 字典        (有哪些对象可以读)"
        echo "  pdo      PDO 映射             (快通道传什么数据)"
        echo "  status   运行状态             (位置/速度/状态字解码)"
        echo "  cstruct  CiA402 C 结构体      (写代码时直接参考)"
        echo ""
        echo "所有命令只读不写, 不会让电机转动。"
        echo "环境变量:  MOTOR_POS=从站位置  ETHERCAT_BIN=ethercat路径"
        ;;

    *)  echo "未知命令: $1  试试: $0 help" >&2; exit 1 ;;
esac
```

3. 主站分析运行结果

```
root@rk3506-buildroot:/# /etc/init.d/ethercat start
Starting EtherCAT master 1.6.0-rc1 [  371.754333] EtherCAT: Master driver 1.6.0-rc1 unknown
[  371.754737] EtherCAT: 1 master waiting for devices.
[  371.765351] ec_generic: EtherCAT master generic Ethernet device module 1.6.0-rc1 unknown
[  371.765408] EtherCAT: Accepting 26:64:07:3A:AB:BD as main device for master 0.
[  371.765450] ec_generic: Binding socket to interface 3 (eth0).
 done
[  371.774150] EtherCAT 0: Starting EtherCAT-IDLE thread.
[  371.774290] EtherCAT 0: Link state of ecm0 changed to UP.
[  371.776293] EtherCAT 0: 1 slave(s) responding on main device.
root@rk3506-buildroot:/# [  371.776311] EtherCAT 0: Slave states on main device: PREOP.
[  371.777584] EtherCAT 0: Scanning bus.
[  371.950300] EtherCAT 0: Bus scanning completed in 174 ms.
[  371.950320] EtherCAT 0: Using slave 0 as DC reference clock.

root@rk3506-buildroot:/#
root@rk3506-buildroot:/# motor_test.sh  master
== 目标从站: -p0  (ethercat 路径: ethercat)
```

```
========================================================================
  1.1  ethercat master  -  主站在不在?
========================================================================

  $ ethercat master

Master0
  Phase: Idle
  Active: no
  Slaves: 1
  Ethernet devices:
    Main: 26:64:07:3a:ab:bd (attached)
      Link: UP
      Tx frames:   36051
      Tx bytes:    2166356
      Rx frames:   36050
      Rx bytes:    2166296
      Tx errors:   0
      Tx frame rate [1/s]:    492    493    349
      Tx rate [KByte/s]:     28.8   28.9   20.4
      Rx frame rate [1/s]:    492    493    349
      Rx rate [KByte/s]:     28.8   28.9   20.4
    Common:
      Tx frames:   36051
      Tx bytes:    2166356
      Rx frames:   36050
      Rx bytes:    2166296
      Lost frames: 0
      Tx frame rate [1/s]:    492    493    349
      Tx rate [KByte/s]:     28.8   28.9   20.4
      Rx frame rate [1/s]:    492    493    349
      Rx rate [KByte/s]:     28.8   28.9   20.4
      Loss rate [1/s]:          0      0      0
      Frame loss [%]:         0.0    0.0    0.0
  Distributed clocks:
    Reference clock:   Slave 0
    DC reference time: 0
    Application time:  0
                       2000-01-01 00:00:00.000000000

  关注: Phase 是否 = Operation  (主站必须 Operation 从站才能 OP)
        Slaves 数目是否正确

========================================================================
  1.2  ethercat slaves  -  几个从站, 各自什么状态?
========================================================================

  $ ethercat slaves

0  0:0  PREOP  +  DS2-EC

  输出解读:
    第一列 = 从站位置 (0,1,2...)
    第二列 = 从站状态:
      INIT   = 刚上电, 只识别身份
      PREOP  = 可以 SDO 通信 (我们能读参数)
      SAFEOP = PDO 输入生效, 输出安全值
      OP     = 正常运行, PDO 双向通信
    第三列 + Alias + 设备名

root@rk3506-buildroot:/#
```

4. 从站SDO对象字典

- 从站SDO执行结果分析

```
root@rk3506-buildroot:/# motor_test.sh slave
== 目标从站: -p0  (ethercat 路径: ethercat)

========================================================================
  2.1  设备名 + 版本号  (SDO 读对象字典 0x1008~0x100A)
========================================================================

  SDO 读 = ethercat upload -p<从站> --type <数据类型> <索引> <子索引>

  [对象 0x1008] 设备名:
  $ ethercat upload -p0 --type string 0x1008 0
ECAT-DR

  [对象 0x1009] 硬件版本:
  $ ethercat upload -p0 --type string 0x1009 0
1.0

  [对象 0x100A] 固件版本:
  $ ethercat upload -p0 --type string 0x100A 0
5.11

========================================================================
  2.2  身份标识  (0x1018 Identity Object)
========================================================================

  [0x1018:01] 厂商 ID:
  $ ethercat upload -p0 --type uint32 0x1018 1
0x00000a79 2681
  (0xA79 = 深圳新川 Xinchuan)

  [0x1018:02] 产品型号:
  $ ethercat upload -p0 --type uint32 0x1018 2
0x00005000 20480

  [0x1018:03] 硬件版本号:
  $ ethercat upload -p0 --type uint32 0x1018 3
0x00000001 1

  [0x1018:04] 序列号:
  $ ethercat upload -p0 --type uint32 0x1018 4
0x00000001 1

root@rk3506-buildroot:/#
```

- 运行结果

```
root@rk3506-buildroot:/# motor_test.sh sdo
== 目标从站: -p0  (ethercat 路径: ethercat)

========================================================================
  3.1  ethercat sdos  -  这个从站有多少可读写的 SDO?
========================================================================

  SDO 字典 = 从站的 参数清单, 每个对象有一个 16-bit 索引 + 8-bit 子索引
  0x1000~0x1FFF = 通信对象      (Communication Profile)
  0x2000~0x5FFF = 厂商自定义    (Manufacturer Specific)
  0x6000~0x9FFF = 设备行规对象  (如 CiA402 电机行规)

  $ ethercat sdos -p0 | head -60

  ...

========================================================================
  3.2  手工读几个关键 SDO 看看值
========================================================================

  -- 通信参数 (0x1000~0x1FFF) --

  [0x1000:00] 设备类型 (CiA402 电机 = 0x00020192),
              告诉主站我是干什么的:
0x00040192 262546

  [0x1C00:00] 同步管理器类型, 决定 SM2/SM3 怎么映射 PDO:
0x04 4

  [0x1C12:00] SM2 分配的 RxPDO 个数:
0x01 1

  [0x1C13:00] SM3 分配的 TxPDO 个数:
0x01 1

  -- CiA402 电机行规 (0x6040~) --

  [0x6040:00] 控制字 (ControlWord) - 上位机写这个来启停电机:
0x0000 0

  [0x6041:00] 状态字 (StatusWord) - 驱动器回报当前状态:
0x0000 0

#   PV  = 你告诉驱动器 "跑多快" → 驱动器自己慢慢加速过去
```

```
#  CSV = 你每个周期告诉驱动器 "现在跑多快" → 你自己算速度
#  CSP = 你每个周期告诉驱动器 "现在到哪" → 你自己算位置轨迹
  [0x6060:00] 运行模式 (8=CSP 9=CSV 3=PV ...):
0x00 0

  [0x6061:00] 当前运行模式 (驱动器回报):
0x00 0

root@rk3506-buildroot:/#
```

CSP、CSV、PV 是 CiA402 中最常用的三种运行模式，区别在于 "谁来算加减速曲线":

一句话区分

- PV = 你告诉驱动器 "跑多快" → 驱动器自己慢慢加速过去
- CSV = 你每个周期告诉驱动器 "现在跑多快" → 你自己算速度
- CSP = 你每个周期告诉驱动器 "现在到哪" → 你自己算位置轨迹

本质区别：轨迹规划（加减速）在谁手里?

详细对比

- PV — Profile Velocity（速度模式，3），主站发一次: "以 1000 rpm 跑"
  - 驱动器自己做: 0 → 加速 → 匀速 1000 → 直到你发新速度
    - 通信量最小：发一次目标速度就行，不需要每个周期发
    - 驱动器内置梯形加减速：驱动器根据 0x6083（加速度）和 0x6084（减速度）自己算
    - 适用场景：传送带、搅拌机、主轴——速度恒定或偶尔调速
    - 不适用：多轴插补——因为几根轴各跑各的，位置不同步
- CSV — Cyclic Sync Velocity（周期同步速度，9）
  - 周期 1: 主站发 "速度=100" → 驱动器执行
  - 周期 2: 主站发 "速度=200" → 驱动器执行
  - 周期 3: 主站发 "速度=300" → 驱动器执行
  - 每个周期(如 1ms)都要发一次
    - 主站自己算速度曲线：你把速度指令拆成每个周期的点
    - 驱动器不做加减速：你发多少跑多少，跳变就硬切
    - 适用场景：需要自己控制速度曲线的场合，比如电子齿轮、飞剪
    - 比 PV 灵活：速度可以每个周期变，能做出任意速度曲线
- CSP — Cyclic Sync Position（周期同步位置，8）
  - 周期 1: 主站发 "目标位置=1000" → 驱动器走到 1000
  - 周期 2: 主站发 "目标位置=1010" → 驱动器走到 1010
  - 周期 3: 主站发 "目标位置=1020" → 驱动器走到 1020
  - ...位置一步步累加, 走出你要的轨迹
    - 主站规划完整轨迹：你在应用层算出每个周期的位置点
    - 驱动器只做位置闭环：PID 追踪你给的目标位置
    - 多轴联动的基础：几根轴的轨迹在同一个周期同步下发，走出圆弧/直线
    - 适用场景：机械臂、CNC、多轴插补——需要精确位置同步
    - 这是最常用的：90% 的实时 EtherCAT 应用都用 CSP

---

5. 从站PDO对象字典

- 运行结果

```
root@rk3506-buildroot:/# motor_test.sh pdo
== 目标从站: -p0  (ethercat 路径: ethercat)

================================================================================
  4.1  ethercat pdos  -  PDO 映射一览
================================================================================

  PDO = 每个周期自动交换的数据包, 不用每次发 SDO 读写命令
  RxPDO = 主站发给从站 (比如: 控制字 + 目标位置)
  TxPDO = 从站发给主站 (比如: 状态字 + 实际位置)
  SM2 承载 RxPDO,  SM3 承载 TxPDO

  $ ethercat pdos -p0

SM0: PhysAddr 0x1000, DefaultSize  128, ControlRegister 0x26, Enable 1
SM1: PhysAddr 0x1100, DefaultSize  128, ControlRegister 0x22, Enable 1
SM2: PhysAddr 0x1200, DefaultSize   16, ControlRegister 0x64, Enable 1
  RxPDO 0x1600 ""
    PDO entry 0x6040:00, 16 bit, ""
    PDO entry 0x607a:00, 32 bit, ""
    PDO entry 0x60ff:00, 32 bit, ""
    PDO entry 0x6060:00,  8 bit, ""
    PDO entry 0x0000:00,  8 bit, "Gap"
SM3: PhysAddr 0x1300, DefaultSize   39, ControlRegister 0x20, Enable 1
  TxPDO 0x1a00 ""
    PDO entry 0x6041:00, 16 bit, ""
    PDO entry 0x6064:00, 32 bit, ""
    PDO entry 0x606c:00, 32 bit, ""
    PDO entry 0x6061:00,  8 bit, ""
    PDO entry 0x0000:00,  8 bit, "Gap"

================================================================================
  4.2  PDO 映射寄存器原始值  (RxPDO=0x1600+, TxPDO=0x1A00+)
================================================================================

  映射条目是一个 32-bit 数, 格式: 0x 索引(16bit) 子索引(8bit) 位宽(8bit)
  例如 0x60400010 表示: 对象 0x6040, 子索引 0, 位宽 16 bits

  [0x1600] RxPDO 1 的映射条目:
  $ ethercat upload -p0 --type uint32 0x1600 1  2>/dev/null
0x60400010 1614807056
  $ ethercat upload -p0 --type uint32 0x1600 2  2>/dev/null
0x607a0020 1618608160

  [0x1A00] TxPDO 1 的映射条目:
  $ ethercat upload -p0 --type uint32 0x1A00 1  2>/dev/null
0x60410010 1614872592
  $ ethercat upload -p0 --type uint32 0x1A00 2  2>/dev/null
0x60640020 1617166368

root@rk3506-buildroot:/#
```

- SM0: 邮箱输入 (主站→从站) — SDO 走这里, 不管它
- SM1: 邮箱输出 (从站→主站) — SDO 走这里, 不管它
- SM2: 过程数据输入 (主站→从站) — 即 RxPDO, 16 字节
- SM3: 过程数据输出 (从站→主站) — 即 TxPDO, 39 字节

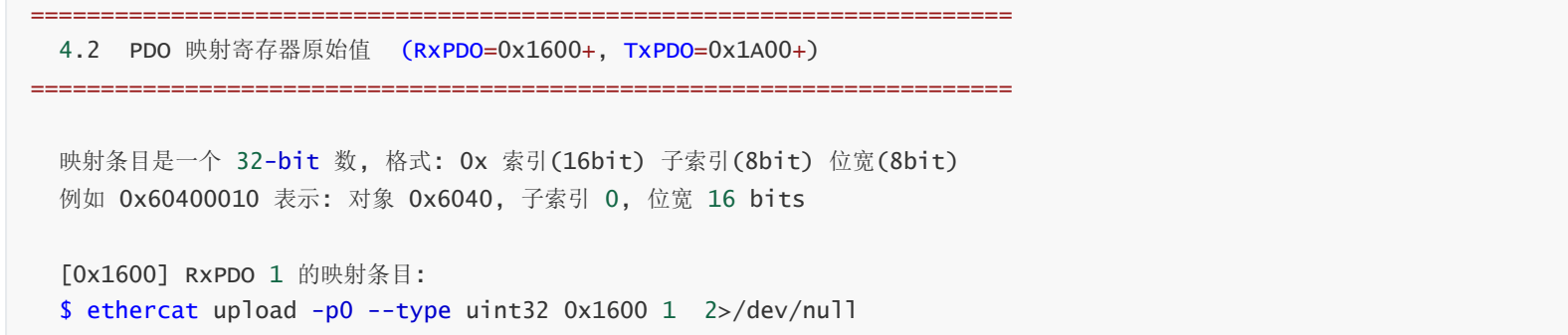
SM2 和 SM3 是实时通道，我们关注这两个。

SM2: RxPDO 0x1600 — 主站发给从站，共 10 字节

```
PDO entry 0x6040:00, 16 bit, ""      ← 控制字       2字节,  偏移 0
PDO entry 0x607a:00, 32 bit, ""      ← 目标位置     4字节,  偏移 2
PDO entry 0x60ff:00, 32 bit, ""      ← 目标速度     4字节,  偏移 6
PDO entry 0x6060:00,  8 bit, ""      ← 运行模式     1字节,  偏移 10
PDO entry 0x0000:00,  8 bit, "Gap"   ← 填充对齐     1字节,  偏移 11
```

总计 12 字节(但 SM2 DefaultSize=16, 有4字节预留)

- 内存布局:



这个映射同时包含了 0x607A(CSP 目标位置) 和 0x60FF(CSV 目标速度)，意味着驱动器的一套 PDO 映射兼容 CSP 和 CSV 两种模式。

你通过改 0x6060来切换模式，切到 CSP 时 0x60FF 字段被忽略，切到 CSV 时 0x607A 被忽略。

- TXPDO

```
SM3: TxPDO 0x1A00 — 从站回给主站，共 10 字节

  PDO entry 0x6041:00, 16 bit, ""      ← 状态字      2 字节,  偏移 0
  PDO entry 0x6064:00, 32 bit, ""      ← 实际位置    4 字节,  偏移 2
  PDO entry 0x606c:00, 32 bit, ""      ← 实际速度    4 字节,  偏移 6
  PDO entry 0x6061:00,  8 bit, ""      ← 当前模式    1 字节,  偏移 10
  PDO entry 0x0000:00,  8 bit, "Gap"   ← 填充对齐    1 字节,  偏移 11
                                                    总计 12 字节
                                        (SM3 DefaultSize=39, 说明还有别的映射)

内存布局：

字节:  0  1    2  3  4  5    6  7  8  9    10       11       12..38
       ┌──────┬──────────┬──────────┬──────┬──────┬──────────────┐
       │6041  │   6064   │   606C   │6061  │ pad  │ (其他 PDO?)  │
       │StatWd│  ActPos  │  ActVel  │ Mode │      │              │
       │u16   │   s32    │   s32    │  s8  │  s8  │              │
       └──────┴──────────┴──────────┴──────┴──────┴──────────────┘
```

对应 C 结构体

```c
// 主站 → 从站 (SM2, 12 字节有效)
typedef struct __attribute__((packed)) {
    uint16_t  CtrlWord;     // 0x6040  偏移 0
    int32_t   TargetPos;    // 0x607A  偏移 2  (CSP 模式用)
    int32_t   TargetVel;    // 0x60FF  偏移 6  (CSV/PV 模式用)
    int8_t    OpMode;       // 0x6060  偏移 10 (1=PP 3=PV 8=CSP 9=CSV)
    int8_t    _pad;         // Gap     偏移 11
} rx_pdo_t;

// 从站 → 主站 (SM3, 12 字节有效)
typedef struct __attribute__((packed)) {
    uint16_t  StatusWord;   // 0x6041  偏移 0
    int32_t   ActPos;       // 0x6064  偏移 2
    int32_t   ActVel;       // 0x606C  偏移 6
    int8_t    ModeDisplay;  // 0x6061  偏移 10
    int8_t    _pad;         // Gap     偏移 11
} tx_pdo_t;

//Gap 的作用： 0x6060 是 8 位，后面跟一个 8 位 Gap 是为了保持 16 位对齐——EtherCAT 从站硬件通常要求 2 字节对齐。
```

- 系统功能图

```
主站 (你的 Linux)                          从站 (CL3-E57H 驱动器)
═══════════                                ═══════════════════

                          SM2 (RxPDO)
每周期发 12 字节 ──────────────────────→  收到控制字+目标位置+目标速度+模式
  CtrlWord  = 0x000F                      → 解读: 使能 + CSP 模式
  TargetPos = 递增的位置                    → 电机跟踪这个位置
  OpMode    = 8                           → 跑 CSP

                          SM3 (TxPDO)
收到反馈 ←──────────────────────────────  每周期回 12 字节
  StatusWord  = 0xX027                    ← 状态: Operation Enabled
  ActPos      = 编码器读数                  ← 电机现在在哪
  ActVel      = 实际速度                    ← 电机现在多快

这个驱动器的 PDO 设计很标准：一套映射同时支持 CSP(用 0x607A) 和 CSV(用 0x60FF)，改 0x6060 就能切模式，不用重新配 PDO。
```

6. 从机状态

- 运行结果

```
root@rk3506-buildroot:/# motor_test.sh status
== 目标从站: -p0  (ethercat 路径: ethercat)

================================================================================
  5.1  ethercat states  ?  从站现在什么状态?
================================================================================

'states' takes exactly one argument!

ethercat states [OPTIONS] <STATE>

Request application-layer states.

Arguments:
  STATE can be 'INIT', 'PREOP', 'BOOT', 'SAFEOP', or 'OP'.

Command-specific options:
  --alias    -a <alias>
  --position -p <pos>    Slave selection. See the help of
                         the 'slaves' command.

Numerical values can be specified either with decimal (no
prefix), octal (prefix '0') or hexadecimal (prefix '0x') base.

================================================================================
  5.2  状态字  (0x6041)  ?  解码驱动器状态机
================================================================================

  [0x6041:00] 状态字原始值: 0x0000 0

  hex=0x0000  dec=0

  核心状态 (sw & 0x6F) = 0

  逐位解读:
  ------------------------------------------------------------------------
  bit 0  Ready to switch on:  0  1=驱动器就绪待命
  bit 1  Switched on:         0  1=主电已接通
  bit 2  Operation enabled:   0  1=使能,可以转了!
  bit 3  Fault:               0  1=故障!看 ErrorCode
  bit 4  Voltage enabled:     0  1=主电电压正常
  bit 5  Quick stop:          0  0=急停中/1=正常
  bit 6  Switch on disabled:  0  1=只能接受 shutdown
  bit 7  Warning:             0  1=有警告
  bit 10 Target reached:      0 1=到达目标
  bit 11 Internal limit:      0 1=碰到内部限位
  bit 12 Set-point ack:       0 1=已确认设定值
  ------------------------------------------------------------------------

  状态机流转路径 (手动使能的逻辑):
    上电 -> Not Ready (0) -> SwitchOnDisabled (64=0x40)
         -> ReadyToSwitchOn (33=0x21) -> SwitchedOn (35=0x23)
         -> OperationEnabled (39=0x27) ? 电机能转了!

================================================================================
  5.3  运行模式
================================================================================

  [0x6061:00] 当前运行模式 (驱动器回报):
  原始值: 0
  含义: 未配置

  常见模式对照:
    8 CSP = 主站每个周期发目标位置, 适合多轴插补
    9 CSV = 主站发目标速度
    1 PP  = 主站发目标位置+速度, 驱动器自己走梯形加减速
    3 PV  = 主站发目标速度, 驱动器自己加减速

================================================================================
  5.4  实际运行值
================================================================================

  [0x6064:00] 实际位置 (编码器值, 单位取决于驱动器配置):
  $ ethercat upload -p0 --type int32 0x6064 0
0x00000000 0

  [0x606C:00] 实际速度 (单位取决于驱动器配置):
  $ ethercat upload -p0 --type int32 0x606C 0
0xfffffff0 -16

  [0x6077:00] 实际转矩 (千分之额定转矩):
  $ ethercat upload -p0 --type int16 0x6077 0
0x0000 0

  [0x60F4:00] 跟随误差 (指令位置 - 实际位置):
  $ ethercat upload -p0 --type int32 0x60F4 0
0x00000000 0
  (这个值越小越好, 大了说明电机跟不上指令)

  [0x60B9:00] 数字输入 (限位开关/原点信号等):
  $ ethercat upload -p0 --type uint32 0x60B9 0
Data type mismatch. Expected uint32 with 4 byte, but got 2 byte.
```

```
root@rk3506-buildroot:/#
```

7. 把对象字典映射成结构体

```
root@rk3506-buildroot:/# motor_test.sh cstruct
== 目标从站: -p0  (ethercat 路径: ethercat)

================================================================================
  CiA402 常用对象字典  C 结构体参考
================================================================================

  把下面内容另存为 ecat_motor.h, 写 IgH 应用时直接 include

/* ===== 运行指令  -  写 PDO (RxPDO, 主站→从站) ===== */

typedef struct __attribute__((packed)) {
    /* 对象 0x6040 - CiA402 控制字 (使能/停机/故障复位等) */
    uint16_t    CtrlWord;
    /* 对象 0x607A - 目标位置 (CSP 模式), 单位 inc (编码器脉冲) */
    int32_t     TargetPos;
    /* 对象 0x60FF - 目标速度 (CSV/PV 模式) */
    int32_t     TargetVel;
    /* 对象 0x60B2 - 转矩偏移, 单位 0.1% 额定转矩 */
    int16_t     TorqueOffset;
    /* 对象 0x6060 - 运行模式 (1=PP 3=PV 6=HM 8=CSP 9=CSV 10=CST) */
    int8_t      OpMode;
    /* 预留 3 字节对齐 */
    uint8_t     _pad[3];
} rx_pdo_t;

/* ===== 运行反馈  -  读 PDO (TxPDO, 从站→主站) ===== */

typedef struct __attribute__((packed)) {
    /* 对象 0x6041 - 状态字 (bit0-6=状态机, bit3=fault, bit10=到位) */
    uint16_t    StatusWord;
    /* 对象 0x6064 - 实际位置 (编码器读数) */
    int32_t     ActPos;
    /* 对象 0x606C - 实际速度 */
    int32_t     ActVel;
    /* 对象 0x6077 - 实际转矩 (千分之额定) */
    int16_t     ActTorque;
    /* 对象 0x60F4 - 跟随误差 = 目标位置-实际位置 */
    int32_t     FollowErr;
    /* 对象 0x6061 - 当前运行模式 (驱动器回报) */
    int8_t      ModeDisplay;
    /* 对象 0x603F - 故障码 */
    uint16_t    ErrorCode;
    /* 对象 0x60B9 - 数字输入 (bit0=正限位 bit1=负限位 bit2=原点 …) */
    uint32_t    DigitalIn;
} tx_pdo_t;

/* ===== 控制字常用值 (CiA402 状态机) ===== */
#define CW_SHUTDOWN          0x0006  /* 关断 */
#define CW_SWITCH_ON         0x0007  /* 接通主电 */
#define CW_ENABLE            0x000F  /* 使能! */
#define CW_DISABLE_VOLTAGE   0x0000  /* 断开主电 */
#define CW_QUICKSTOP         0x0002  /* 急停 */
#define CW_FAULT_RESET       0x0080  /* 故障复位 (bit7=0→1) */

/* ===== 状态字掩码 ===== */
#define SW_READY_MASK        0x006F  /* 掩掉厂商位, 只看 CiA402 核心位 */
#define SW_FAULT             (1<<3)  /* bit3 = 故障 */
#define SW_TARGET_REACHED    (1<<10) /* bit10 = 到达目标 */
#define SW_WARNING           (1<<7)  /* bit7 = 警告 */

/* ===== CiA402 核心状态值 (sw & 0x6F) ===== */
enum {
    ST_NOT_READY        = 0x00,  /* 0  上电中 */
    ST_SWITCH_ON_DIS    = 0x40,  /* 64  只能发 shutdown */
    ST_READY_SWITCH_ON  = 0x21,  /* 33  准备好接通主电 */
    ST_SWITCHED_ON      = 0x23,  /* 35  主电已通, 等待使能 */
    ST_OP_ENABLED       = 0x27,  /* 39  运转中! */
    ST_QUICK_STOP       = 0x07,  /* 7   急停激活 */
    ST_FAULT_REACT      = 0x0F,  /* 15  故障处理中 */
    ST_FAULT            = 0x08,  /* 8   故障停机 */
};

/* ===== 运行模式 (0x6060/0x6061) ===== */
enum {
    MODE_PP  = 1,   /* Profile Position   位置模式 */
    MODE_PV  = 3,   /* Profile Velocity   速度模式 */
    MODE_HM  = 6,   /* Homing             回零 */
    MODE_CSP = 8,   /* Cyclic Sync Pos    周期同步位置 */
    MODE_CSV = 9,   /* Cyclic Sync Vel    周期同步速度 */
    MODE_CST = 10,  /* Cyclic Sync Trq    周期同步转矩 */
};

================================================================================
  对应实际从站的 PDO 映射, 确认结构体与映射一致:
================================================================================

  $ ethercat pdos -p0

SM0: PhysAddr 0x1000, DefaultSize  128, ControlRegister 0x26, Enable 1
SM1: PhysAddr 0x1100, DefaultSize  128, ControlRegister 0x22, Enable 1
SM2: PhysAddr 0x1200, DefaultSize   16, ControlRegister 0x64, Enable 1
  RxPDO 0x1600 ""
    PDO entry 0x6040:00, 16 bit, ""
    PDO entry 0x607a:00, 32 bit, ""
    PDO entry 0x60ff:00, 32 bit, ""
    PDO entry 0x6060:00,  8 bit, ""
    PDO entry 0x0000:00,  8 bit, "Gap"
SM3: PhysAddr 0x1300, DefaultSize   39, ControlRegister 0x20, Enable 1
  TxPDO 0x1a00 ""
    PDO entry 0x6041:00, 16 bit, ""
    PDO entry 0x6064:00, 32 bit, ""
    PDO entry 0x606c:00, 32 bit, ""
    PDO entry 0x6061:00,  8 bit, ""
    PDO entry 0x0000:00,  8 bit, "Gap"

 // 写代码前务必确认上面的 PDO 条目与结构体字段一一对应!
 // 不同驱动器/不同固件的映射可能不同 (比如有的多了 digital outputs)

root@rk3506-buildroot:/#
```

21.4 电机控制实例

1. 电机控制源码

把电机文件的xml文件让ai解析， 给生成电机驱动的例子， 我这个就是这么完成的。

- external/ethercat_igh/motor_app/ec_motor.c

```
/* ================================================================================
 * ec_motor.c  --  IgH EtherCAT 入门 RT 应用 (CSV 速度模式)
 *
 *   跑起来后, 每 1ms 做这些事:
 *     ① 收 EtherCAT 帧          (ecrt_master_receive)
 *     ② 读从站返回的状态字+位置+速度 (EC_READ_xxx)
 *     ③ 走 CiA402 状态机, 决定下一步控制字  (if/else 链)
 *     ④ 写目标速度              (EC_WRITE_xxx)
 *     ⑤ 写控制字                (EC_WRITE_xxx)
 *     ⑥ 发 EtherCAT 帧          (ecrt_master_send)
 *
 *   数据流向:
 *   ┌──────────────────┐                ┌──────────────────────┐
 *   |    你的 Linux     |    网线        |   CL3-E57H 驱动器     |
 *   |  (IgH 主站)       |◄═══════►|   (EtherCAT 从站)     |
 *   |                   |                |                      |
 *   |  RxPDO 0x1600:    |──写入──→ |  0x6040 控制字 u16   |
 *   |    CtrlWord  u16  |                |  0x60FF 目标速度 s32 |
 *   |    TargetVel s32  |                |                      |
 *   |                   |                |                      |
 *   |  TxPDO 0x1A00:    |←─读取─── |  0x6041 状态字 u16   |
 *   |    StatusWd  u16  |                |  0x6064 实际位置 s32 |
 *   |    ActPos    s32  |                |  0x606C 实际速度 s32 |
 *   |    ActVel    s32  |                |                      |
 *   └──────────────────┘                └──────────────────────┘
 *
 *   用法:
 *     sudo ./ec_motor 50000     以速度 50000 转
 *     sudo ./ec_motor           以默认速度 100000 转
 *     Ctrl+C 可随时停止
```

```
 *
 *   注意: 第一次跑先用 motor_test.sh 确认从站状态(需 PREOP 以上)
 * ============================================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sched.h>
#include <time.h>
#include <sys/mman.h>
#include "ecrt.h"          /* IgH EtherCAT 实时接口 */

/* ============================================================================
 * 第一部分: 硬编码配置  (改了驱动/从站就要改这里)
 * ============================================================================ */

#define VENDOR_ID       0x00000a79   /* 深圳新川 Xinchuan */
#define PRODUCT_CODE    0x00005000   /* CL3-E57H 产品码     */
#define SLAVE_POS       0            /* 第几个从站, 从 0 开始 */
#define SLAVE_ALIAS     0            /* 别名, 一般用 0 */

#define PERIOD_NS       1000000L     /* 周期 1ms = 1000000 纳秒 */
#define RT_PRIO         20           /* RT 线程优先级 20 (< 中断线程 50, 不能抢中断) */
#define DEFAULT_SPEED   100000       /* 默认目标速度, 单位 count/s (编码器脉冲/秒) */

/* ============================================================================
 * 第二部分: PDO 描述 — 告诉主站 "每个周期交换哪些数据"
 *
 *   PDO 在从站出厂时就定好了, 不是我们说了算。
 *   先用 'ethercat pdos -p0' 看从站的映射, 然后把条目抄过来。
 *   一定要和实际从站一致, 不然数据错位。
 *
 *   本从站 (CL3-E57H) 实际映射:
 *     RxPDO 0x1600 = 0x6040(u16) 0x607A(s32) 0x60FF(s32) 0x6060(s8)
 *     TxPDO 0x1A00 = 0x6041(u16) 0x6064(s32) 0x606C(s32) 0x6061(s8)
 *   下面结构只用了速度模式相关的, 0x607A 和 0x6060/0x6061 也注册了但未使用。
 * ============================================================================ */

/* -- 2.1: 数据条目 — 每个对象的三要素: {索引, 子索引, 位宽} -- */
static ec_pdo_entry_info_t rx_pdo_entries[] = {
    { 0x6040, 0x00, 16 },   /* 控制字 ControlWord           */
    { 0x607A, 0x00, 32 },   /* 目标位置 TargetPos (暂不用)   */
    { 0x60FF, 0x00, 32 },   /* 目标速度 TargetVel ★我们用这个 */
    { 0x6060, 0x00,  8 },   /* 运行模式 OpMode (暂不用)      */
};

static ec_pdo_entry_info_t tx_pdo_entries[] = {
    { 0x6041, 0x00, 16 },   /* 状态字 StatusWord      ★必读 */
    { 0x6064, 0x00, 32 },   /* 实际位置 ActPos         ★可读 */
    { 0x606C, 0x00, 32 },   /* 实际速度 ActVel         ★可读 */
    { 0x6061, 0x00,  8 },   /* 当前模式 ModeDisplay    ★可读 */
};

/* -- 2.2: PDO 容器 — 把条目装进 PDO 0x1600 和 0x1A00 -- */
/*         参数: { PDO 编号, 条目数, 条目数组 }                    */
static ec_pdo_info_t rx_pdos[] = { { 0x1600, 4, rx_pdo_entries } };
static ec_pdo_info_t tx_pdos[] = { { 0x1A00, 4, tx_pdo_entries } };

/* -- 2.3: SM 同步管理器 — SM0/1=邮箱(SDO), SM2=输出(RxPDO), SM3=输入(TxPDO) -- */
/*         这就是 EtherCAT ESC 硬件的四个通道                          */
static ec_sync_info_t syncs[] = {
    /*  {SM编号, 方向,            PDO数, PDO数组, 看门狗 }              */
    { 0, EC_DIR_OUTPUT,       0, NULL,     EC_WD_DISABLE },  /* SM0: 邮箱输出 */
    { 1, EC_DIR_INPUT,        0, NULL,     EC_WD_DISABLE },  /* SM1: 邮箱输入 */
    { 2, EC_DIR_OUTPUT,       1, rx_pdos,  EC_WD_ENABLE  },  /* SM2: RxPDO ★ */
    { 3, EC_DIR_INPUT,        1, tx_pdos,  EC_WD_DISABLE },  /* SM3: TxPDO ★ */
    { 0xFF, EC_DIR_OUTPUT, 0, NULL, EC_WD_DISABLE }          /* 结束标记 */
};

/* ============================================================================
 * 第三部分: PDO 注册表 — 告诉主站 "每个对象在报文里放第几个字节"
 *
 *   注册后主站会填写 off_xxx 变量, 代表该对象在报文中的字节偏移。
 *   写作 off_xxxx, 读作 "对象 0xXXXX 的偏移量"。
 *
 *   内存布局 (RxPDO):
 *     Byte:  0  1   2  3  4  5   6  7  8  9   10
 *           ┌────┬──────────┬──────────┬────┐
 *           |6040 |   607A     |   60FF     |6060 |
 *           | u16 |   s32      |   s32      | s8  |
 *           └────┴──────────┴──────────┴────┘
 *            off_   off_        off_         off_
 *            ctrl   tpos        tvel         mode
 *
 *   内存布局 (TxPDO):
 *     Byte:  0  1   2  3  4  5   6  7  8  9   10
 *           ┌────┬──────────┬──────────┬────┐
 *           |6041 |   6064     |   606C     |6061 |
 *           | u16 |   s32      |   s32      | s8  |
 *           └────┴──────────┴──────────┴────┘
 *            off_   off_        off_         off_
 *            stat   apos        avel         moded
 * ============================================================================ */

/* 声明偏移变量: 主站激活后会填入实际值 */
static unsigned int off_ctrl;    /* 控制字在 RxPDO 中的偏移 */
static unsigned int off_tvel;    /* 目标速度在 RxPDO 中的偏移 */
static unsigned int off_tpos;    /* 目标位置在 RxPDO 中的偏移 (不用) */
static unsigned int off_mode;    /* 运行模式在 RxPDO 中的偏移 (不用) */

static unsigned int off_stat;    /* 状态字在 TxPDO 中的偏移 */
static unsigned int off_apos;    /* 实际位置在 TxPDO 中的偏移 */
static unsigned int off_avel;    /* 实际速度在 TxPDO 中的偏移 */
static unsigned int off_moded;   /* 当前模式在 TxPDO 中的偏移 (不用) */

/* 注册表: "把 0x6040 放到 off_ctrl, 把 0x60FF 放到 off_tvel, ..." */
static ec_pdo_entry_reg_t domain_regs[] = {
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x6040, 0, &off_ctrl,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x607A, 0, &off_tpos,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x60FF, 0, &off_tvel,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x6060, 0, &off_mode,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x6041, 0, &off_stat,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x6064, 0, &off_apos,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x606C, 0, &off_avel,  NULL },
    { SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE, 0x6061, 0, &off_moded, NULL },
    {}   /* 空行结束 */
};

/* ============================================================================
 * 第四部分: 全局变量
 * ============================================================================ */

static volatile sig_atomic_t keep_running = 1;   /* Ctrl+C 设置为 0, 优雅退出 */

static void handle_signal(int sig)
{
    (void)sig;
    keep_running = 0;
}

/* ---- 辅助: 纳秒加法, 用于精确控制周期 ---- */
static void add_ns(struct timespec *t, long ns)
{
    t->tv_nsec += ns;
    while (t->tv_nsec >= 1000000000L) {
        t->tv_nsec -= 1000000000L;
        t->tv_sec++;
    }
}

/* ============================================================================
 * 第五部分: 主函数 — 初始化 → 循环 → 退出清洗
 * ============================================================================ */

int main(int argc, char **argv)
{
    /* -- 5.0: 解析命令行 -- */
    long target_speed = (argc >= 2) ? strtol(argv[1], NULL, 0) : DEFAULT_SPEED;
    printf("\n");
    printf("═══════════════════════════════════════════════════\n");
    printf("  IgH EtherCAT 电机测试  (CSV 速度模式)\n");
    printf("  目标速度 = %ld  count/s    周期 = %.1f ms\n", target_speed, PERIOD_NS / 1000000.0);
    printf("═══════════════════════════════════════════════════\n\n");

    /* ========================================================== */
     * 步骤 1: 获取主站  — 向 IgH 内核模块要主站 0 的控制权
```

```
     * ========================================================================= */
    printf("[步骤 1/6] 获取主站...\n");
    ec_master_t *master = ecrt_request_master(0);
    if (!master) {
        fprintf(stderr, "  ** 失败: ecrt_request_master 返回 NULL\n");
        fprintf(stderr, "  ** 可能原因: 没插网线 / 没加载 ec_master.ko / 不是 root\n");
        return 1;
    }
    printf("  ✓ 主站 0 获取成功\n\n");

    /* =========================================================================
     * 步骤 2: 创建数据域  — "域"是一块连续内存, 所有从站 PDO 数据都映射在里面
     *   单从站场景: 一个域就够了。多从站: 可以共用一个域, 也可以各用一个。
     * ========================================================================= */
    printf("[步骤 2/6] 创建数据域...\n");
    ec_domain_t *domain = ecrt_master_create_domain(master);
    if (!domain) {
        fprintf(stderr, "  ** 失败: 创建 domain 失败\n");
        return 1;
    }
    printf("  ✓ 数据域创建成功\n\n");

    /* =========================================================================
     * 步骤 3: 配置从站  — 告诉主站 "从站 0 长什么样, 用哪些 PDO"
     *   vendor_id + product_code 必须和从站 ESI 信息严格匹配。
     *   从站不在线 / 码不匹配 → 这里直接失败。
     * ========================================================================= */
    printf("[步骤 3/6] 配置从站...\n");
    ec_slave_config_t *sc = ecrt_master_slave_config(
        master, SLAVE_ALIAS, SLAVE_POS, VENDOR_ID, PRODUCT_CODE);
    if (!sc) {
        fprintf(stderr, "  ** 失败: 从站配置失败 (检查 vendor/product code 是否匹配)\n");
        return 1;
    }

    /* 把第二步定义的 syncs (SM+P DO) 装到从站配置里 */
    if (ecrt_slave_config_pdos(sc, EC_END, syncs)) {
        fprintf(stderr, "  ** 失败: PDO 配置失败\n");
        return 1;
    }

    /* 配置 DC 同步: 激活 SYNC0, 周期 = PERIOD_NS */
    ecrt_slave_config_dc(sc, 0x0300, PERIOD_NS, 0, 0, 0);

    printf("  ✓ 从站配置完成 (vendor=0x%X product=0x%X)\n\n",
           VENDOR_ID, PRODUCT_CODE);

    /* =========================================================================
     * 步骤 4: 注册 PDO 条目 — 把每个对象和数据域的字节偏移绑定
     *   填完之后, off_ctrl / off_tvel / off_stat / off_apos / off_avel 被赋值。
     *   周期循环里用这些偏移从报文里读写数据。
     * ========================================================================= */
    printf("[步骤 4/6] 注册 PDO 条目...\n");
    if (ecrt_domain_reg_pdo_entry_list(domain, domain_regs)) {
        fprintf(stderr, "  ** 失败: PDO 注册失败\n");
        return 1;
    }
    printf("  ✓ PDO 偏移:  CtrlWd=+%u  TargetVel=+%u  StatusWd=+%u  ActPos=+%u  ActVel=+%u\n\n",
           off_ctrl, off_tvel, off_stat, off_apos, off_avel);

    /* =========================================================================
     * 步骤 5: 激活主站  — 主站开始发报文, 把从站从 PREOP 拉到 SAFEOP → OP
     *   到这里之前不会发出任何过程数据; 一旦 activate, PDO 开始流动。
     * ========================================================================= */
    printf("[步骤 5/6] 激活主站 (进入 OP)...\n");
    if (ecrt_master_activate(master)) {
        fprintf(stderr, "  ** 失败: 激活失败\n");
        return 1;
    }

    /* 激活后拿到数据域的指针, 以后每周期从这个指针读写 */
    uint8_t *pd = ecrt_domain_data(domain);
    if (!pd) {
        fprintf(stderr, "  ** 失败: 获取数据域指针失败\n");
        return 1;
    }
    printf("  ✓ 主站已激活, 从站应进入 OP\n");
    {
        /* 快速验证一下从站是否在线 */
        ec_master_state_t ms;
        ecrt_master_state(master, &ms);
        printf("  ✓ 从站数=%u  状态=%s (期望 OP)\n\n",
               ms.slaves_responding,
               ms.link_up ? "link_up" : "link_down?");
    }

    /* =========================================================================
     * 步骤 6: 设置 RT 线程 — SCHED_FIFO + 锁内存
     *   为什么要锁内存? 普通进程内存可能被 swap 出去, swap 时延迟不可控。
     *   为什么优先级 < 50? 因为 PREEMPT-RT 下硬件中断线程是 FIFO 50,
     *   如果应用跑得比中断高, 会产生 RK3506 遇到过的 RCU stall 崩溃。
     * ========================================================================= */
    printf("[步骤 6/6] 设置 RT 线程 (FIFO %d)...\n", RT_PRIO);

    /* 锁住所有已分配和未来分配的内存, 防止 page fault */
    if (mlockall(MCL_CURRENT | MCL_FUTURE) != 0) {
        perror("  mlockall 警告 (非致命, 继续跑)");
    }

    /* 把当前线程设为 FIFO 实时调度, 优先级 20 */
    struct sched_param sp;
    sp.sched_priority = RT_PRIO;
    if (sched_setscheduler(0, SCHED_FIFO, &sp) != 0) {
        perror("  sched_setscheduler 警告 (可能需要 sudo)");
    }
    printf("  ✓ RT 线程就绪  (使用 sudo 运行避免权限问题)\n\n");

    /* 拦截 Ctrl+C */
    signal(SIGINT,  handle_signal);
    signal(SIGTERM, handle_signal);

    /* =========================================================================
     * ╔═══════════════════════════════════════════════════════════════╗
     * ║              ★ 周期循环 — EtherCAT 的核心 ★                  ║
     * ║   下面这几十行就是全部了。每 1ms 执行一圈。                   ║
     * ╚═══════════════════════════════════════════════════════════════╝
     * ========================================================================= */
    printf("──────────────────────────────────────────────────────────────\n");
    printf("  开始实时循环...  (Ctrl+C 停止)\n");
    printf("──────────────────────────────────────────────────────────────\n\n");

    struct timespec wakeup_time;
    clock_gettime(CLOCK_MONOTONIC, &wakeup_time);

    long     cycle_count   = 0;
    long     overrun_count = 0;     /* 周期过冲次数(迟到 >1ms) */
    uint64_t app_time      = 0;     /* 应用时间 [ns], DC 同步用 */

    while (keep_running) {

        /* ---- 6a: 等待下个周期开始 ---- */
        add_ns(&wakeup_time, PERIOD_NS);
        clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &wakeup_time, NULL);

        /* ---- 6a-2: 检测过冲 — 如果晚了就重新对齐, 避免连锁迟到 ---- */
        struct timespec now;
        clock_gettime(CLOCK_MONOTONIC, &now);
        if (now.tv_sec > wakeup_time.tv_sec ||
            (now.tv_sec == wakeup_time.tv_sec && now.tv_nsec > wakeup_time.tv_nsec)) {
            wakeup_time = now;   /* 用当前时间重置基准 */
            overrun_count++;
        }

        /* ---- 6a-3: 告诉主站当前应用时间 (DC 偏移计算用) ---- */
        app_time += PERIOD_NS;
        ecrt_master_application_time(master, app_time);

        /* ---- 6b: 收帧 — 把总线上的数据读进内存 ---- */
        ecrt_master_receive(master);
        ecrt_domain_process(domain);

        /* ---- 6c: ★ 读 — 从报文里提取从站返回的数据 ★ ---- */
        uint16_t status_word = EC_READ_U16(pd + off_stat);   /* 0x6041 状态字          */
        int32_t  act_pos     = EC_READ_S32(pd + off_apos);   /* 0x6064 实际位置         */
        int32_t  act_vel     = EC_READ_S32(pd + off_avel);   /* 0x606C 实际速度         */
        uint8_t  mode_disp   = EC_READ_U8 (pd + off_moded);  /* 0x6061 当前模式 (诊断) */

        /* ---- 6d: ★ 算 — CiA402 状态机: 根据状态字决定控制字 ★ ----
```

```
   CiA402 状态机的核心逻辑 (对照 0x6041 的 bit 位):

     上电 → Not Ready     (sw&0x6F = 0x00)   → 发 shutdown (0x06)
          ↓
      SwitchOnDisabled    (sw&0x6F = 0x40)   → 发 shutdown (0x06)
          ↓
      ReadyToSwitchOn     (sw&0x6F = 0x21)   → 发 switch on (0x07)
          ↓
      SwitchedOn          (sw&0x6F = 0x23)   → 发 enable (0x0F)
          ↓
      OperationEnabled    (sw&0x6F = 0x27)   → 保持 enable (0x0F) ★★★
          ↑                                                    |
          └── 这就是我们要的 "电机在转" 状态 ──────────────────┘

   如果 bit3 (Fault) = 1 → 发 fault reset (0x0080)  清除故障
   ---------------------------------------------------------------- */
uint16_t control_word;
int      enabled = 0;          /* 本周期是否使能 */

if (status_word & 0x0008) {                          /* bit3=1  故障 */
    control_word = 0x0080;   /* 发 fault reset      */
}
else if ((status_word & 0x004F) == 0x0040) {         /* SwitchOnDisabled */
    control_word = 0x0006;   /* shutdown: 准备接通  */
}
else if ((status_word & 0x006F) == 0x0021) {         /* ReadyToSwitchOn  */
    control_word = 0x0007;   /* switch on: 接通主电  */
}
else if ((status_word & 0x006F) == 0x0023) {         /* SwitchedOn       */
    control_word = 0x000F;   /* enable: 使能!       */
}
else if ((status_word & 0x006F) == 0x0027) {         /* OperationEnabled */
    control_word = 0x000F;   /* 保持 enable         */
    enabled = 1;             /* 可以写速度了!        */
}
else {
    control_word = 0x0006;   /* 未知状态, 发 shutdown 回到安全位置 */
}
```

```
    /* ---- 6e: ★ 写 - 把我们的指令填进报文 ★ ----

       三个关键字段:
         0x6060 Modes of Op    = 9  (CSV 速度模式, 必须写!)
         0x60FF Target Velocity = 目标速度 (count/s)
         0x6040 ControlWord     = 控制字 (告诉驱动器下一步干嘛)

       只有使能状态 (enabled=1) 才给速度, 否则写 0。                    */
    EC_WRITE_U8 (pd + off_mode, 9);                          /* 0x6060 = CSV 模式    */
    EC_WRITE_S32(pd + off_tvel,
                 enabled ? (int32_t)target_speed : 0);       /* 0x60FF 目标速度      */
    EC_WRITE_U16(pd + off_ctrl, control_word);               /* 0x6040 控制字        */

    /* ---- 6f: ★ 发帧 - 把报文推到网线上 ---- */
    ecrt_domain_queue(domain);          /* 把域数据排队, 准备发送 */
    ecrt_master_send(master);           /* 实际发出 EtherCAT 帧   */

    /* ---- 6g: 每秒打印一次状态 (含过冲计数) ---- */
    cycle_count++;
    if (cycle_count % 1000 == 0) {   /* 1ms × 1000 = 1 秒 */
        printf("  [t=%lds]  Status=0x%04X  %s  Mode=%d "
               "Vel=%d  Pos=%d  Ctrl=0x%04X  overrun=%ld\n",
               cycle_count / 1000,
               status_word,
               enabled ? "★ENABLED★" : "--------",
               mode_disp,
               act_vel, act_pos, control_word, overrun_count);
        fflush(stdout);
    }
} /* while (keep_running) - 周期循环结束 */

/* ============================================================
 * ╔══════════════════════════════════════════════════════════╗
 * ║                退出清洗 - 安全停机                       ║
 * ║ 不能直接退出, 要发 shutdown (0x06) 让驱动器断电。        ║
 * ║ 跑 200 个周期 (~200ms) 确保指令送达。                    ║
 * ╚══════════════════════════════════════════════════════════╝
 * ============================================================ */
printf("\n--------------------------------------------------------------\n");
printf("  收到退出信号, 安全停机中...\n");
printf("--------------------------------------------------------------\n");

    for (int i = 0; i < 200; i++) {
        add_ns(&wakeup_time, PERIOD_NS);
        clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &wakeup_time, NULL);

        /* 过冲检测 - 同主循环 */
        struct timespec now;
        clock_gettime(CLOCK_MONOTONIC, &now);
        if (now.tv_sec > wakeup_time.tv_sec ||
            (now.tv_sec == wakeup_time.tv_sec && now.tv_nsec > wakeup_time.tv_nsec)) {
            wakeup_time = now;
        }

        app_time += PERIOD_NS;
        ecrt_master_application_time(master, app_time);

        ecrt_master_receive(master);
        ecrt_domain_process(domain);

        EC_WRITE_U8 (pd + off_mode, 9);                  /* 保持 CSV 模式    */
        EC_WRITE_S32(pd + off_tvel, 0);            /* 速度归零         */
        EC_WRITE_U16(pd + off_ctrl, 0x0006);       /* 控制字=shutdown */

        ecrt_domain_queue(domain);
        ecrt_master_send(master);

    }

    /* ---- 释放主站 - 还给内核 ---- */
    ecrt_release_master(master);
    printf("  已安全退出。\n\n");
    return 0;
}

/* ============================================================================
 * 学完这个, 接下来可以:
 *   1. 改成 CSP 位置模式 - 每周期累加 target_pos, 让电机走到指定位置
 *   2. 加入 DC 时钟同步 - 多个从站的周期严格对齐
 *   3. 启用看门狗 - SM3 TxPDO 也设 EC_WD_ENABLE, 超时自动停机
 *   4. 加命令行参数 - 灵活切换 CSP/CSV/速度/周期
 *   5. 多从站 - 循环配置多个 sc, 在 domain 里注册多组 PDO
 * ============================================================================ */
```

- external/ethercat_igh/motor_app/Makefile

```
# ec_motor 交叉编译 Makefile
#
#   make            交叉编译出 ec_motor(ARM)
#   make clean
#   make deploy     拷到 buildroot target rootfs 的 /usr/bin
#
# 变量可在命令行覆盖,如: make CROSS=... EC_DIR=...

CROSS  ?= /home/linux/rk3506/buildroot/output/rockchip_rk3506/host/bin/arm-buildroot-linux-gnueabihf-
EC_DIR ?= /home/linux/rk3506/external/ethercat_igh/out
TARGET_ROOTFS ?= /home/linux/rk3506/buildroot/output/rockchip_rk3506/target

CC      := $(CROSS)gcc
CFLAGS  := -O2 -Wall -Wextra -I$(EC_DIR)/include
LDFLAGS := -L$(EC_DIR)/lib -lethercat -lpthread

BIN := ec_motor

$(BIN): ec_motor.c
    $(CC) $(CFLAGS) $< -o $@ $(LDFLAGS)
    $(CROSS)size $@

clean:
    rm -f $(BIN)

intall: $(BIN)
    install -m 0755 $(BIN) $(TARGET_ROOTFS)/usr/bin/$(BIN)
    @echo "已安装到 $(TARGET_ROOTFS)/usr/bin/$(BIN)"

.PHONY: clean deploy
```

- 编译过程

```
linux@ubuntu:~/rk3506$ cd external/ethercat_igh/motor_app/
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$ make
/home/linux/rk3506/buildroot/output/rockchip_rk3506/host/bin/arm-buildroot-linux-gnueabihf-gcc -O2 -Wall -
Wextra -I/home/linux/rk3506/external/ethercat_igh/out/include ec_motor.c -o ec_motor -
L/home/linux/rk3506/external/ethercat_igh/out/lib -lethercat -lpthread
/home/linux/rk3506/buildroot/output/rockchip_rk3506/host/bin/arm-buildroot-linux-gnueabihf-size ec_motor
   text    data     bss     dec     hex filename
   7483     828      36    8347    209b ec_motor
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$ make install
make: *** 没有规则可制作目标“install”。 停止。
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$ make install
install -m 0755 ec_motor /home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/bin/ec_motor
已安装到 /home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/bin/ec_motor
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$ ls -l
/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/bin/ec_motor
-rwxr-xr-x 1 linux linux 17948  7月 14 14:48
/home/linux/rk3506/buildroot/output/rockchip_rk3506/target/usr/bin/ec_motor
linux@ubuntu:~/rk3506/external/ethercat_igh/motor_app$
```

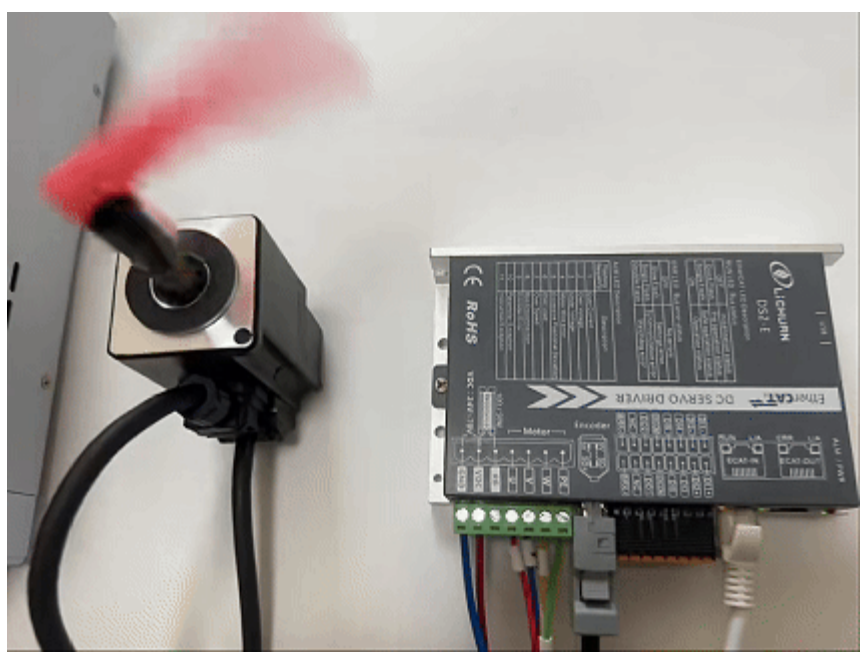
编译文件系统并下载到开发板内。

2. 电机运行过程

```
root@rk3506-buildroot:/# killall rk_demo
root@rk3506-buildroot:/# /etc/init.d/ethercat start
Starting EtherCAT master 1.6.0-rc1 [  536.373467] EtherCAT: Master driver 1.6.0-rc1 unknown
[  536.373851] EtherCAT: 1 master waiting for devices.
[  536.387357] ec_generic: EtherCAT master generic Ethernet device module 1.6.0-rc1 unknown
[  536.387414] EtherCAT: Accepting 26:64:07:3A:AB:BD as main device for master 0.
[  536.387459] ec_generic: Binding socket to interface 3 (eth0).
[  536.394212] EtherCAT 0: Starting EtherCAT-IDLE thread.
[  536.394346] EtherCAT 0: Link state of ecm0 changed to UP.
 done
[  536.396258] EtherCAT 0: 1 slave(s) responding on main device.
[  536.396279] EtherCAT 0: Slave states on main device: PREOP.
[  536.397138] EtherCAT 0: Scanning bus.
root@rk3506-buildroot:/# [  536.569300] EtherCAT 0: Bus scanning completed in 173 ms.
[  536.569319] EtherCAT 0: Using slave 0 as DC reference clock.
root@rk3506-buildroot:/#
root@rk3506-buildroot:/# ec_motor
═══════════════════════════════════════════════════
[   61.346244] EtherCAT: Requesting master 0...
  IgH EtherCAT 电机测试  (CSV 速度模式)
[   61.346265] EtherCAT: Successfully requested master 0.
  目标速度 = 100000  count/s     周期 = 1.0 ms
═══════════════════════════════════════════════════
[   61.348002] EtherCAT 0: Domain0: Logical address 0x00000000, 22 byte, expected working counter 3.

[   61.348019] EtherCAT 0:   Datagram domain0-0-main: Logical offset 0x00000000, 22 byte, type LRW at
99e52ed3.
[步骤 1/6] 获取主站...
[   61.348071] EtherCAT 0: Master thread exited.
  ✓ 主站 0 获取成功
[   61.348084] EtherCAT 0: Starting EtherCAT-OP thread.

[步骤 2/6] 创建数据域...
  ✓ 数据域创建成功

[步骤 3/6] 配置从站...
  ✓ 从站配置完成 (vendor=0xA79 product=0x5000)

[步骤 4/6] 注册 PDO 条目...
  ✓ PDO 偏移:  CtrlWd=+0  TargetVel=+6  StatusWd=+11  ActPos=+13  ActVel=+17

[步骤 5/6] 激活主站 (进入 OP)...
  ✓ 主站已激活, 从站应进入 OP
  ✓ 从站数=1  状态=link_up (期望 OP)

[步骤 6/6] 设置 RT 线程 (FIFO 20)...
  ✓ RT 线程就绪  (使用 sudo 运行避免权限问题)

──────────────────────────────────────────────────
  开始实时循环...  (Ctrl+C 停止)
──────────────────────────────────────────────────

[   61.497889] EtherCAT 0: Domain 0: Working counter changed to 3/3.
[   61.755820] EtherCAT 0: Slave states on main device: OP.
  [t=1s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=1933  Pos=4212800  Ctrl=0x000F  overrun=1000
  [t=2s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=11533  Pos=4222141  Ctrl=0x000F  overrun=2000
  [t=3s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=21483  Pos=4241578  Ctrl=0x000F  overrun=3000
  [t=4s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=31399  Pos=4271119  Ctrl=0x000F  overrun=4000
  [t=5s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=41366  Pos=4310761  Ctrl=0x000F  overrun=5000
  [t=6s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=51383  Pos=4360568  Ctrl=0x000F  overrun=6000
  [t=7s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=61383  Pos=4420447  Ctrl=0x000F  overrun=7000
  [t=8s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=71416  Pos=4490464  Ctrl=0x000F  overrun=8000
  [t=9s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=81266  Pos=4570667  Ctrl=0x000F  overrun=9000
  [t=10s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=91266  Pos=4660892  Ctrl=0x000F  overrun=10000
  [t=11s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=99466  Pos=4760363  Ctrl=0x000F  overrun=11000
  [t=12s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=100049  Pos=4861804  Ctrl=0x000F  overrun=12000
  [t=13s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=99999  Pos=4963320  Ctrl=0x000F  overrun=13000
  [t=14s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=100016  Pos=5064724  Ctrl=0x000F  overrun=14000
  [t=15s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=99999  Pos=5166088  Ctrl=0x000F  overrun=15000
  [t=16s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=99999  Pos=5267601  Ctrl=0x000F  overrun=16000
  [t=17s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=99983  Pos=5369001  Ctrl=0x000F  overrun=17000

──────────────────────────────────────────────────
  收到退出信号, 安全停机中...
──────────────────────────────────────────────────
^C[   79.613355] EtherCAT 0: Releasing master...
[   79.613418] EtherCAT 0: Master thread exited.
[   79.613443] EtherCAT 0: Starting EtherCAT-IDLE thread.
  已安全退出。
[   79.614174] EtherCAT 0: Released.

root@rk3506-buildroot:/# [   79.622854] EtherCAT 0: Slave states on main device: PREOP.
root@rk3506-buildroot:/#


root@rk3506-buildroot:/# ec_motor 50000

═══════════════════════════════════════════════════
[  250.838469] EtherCAT: Requesting master 0...
  IgH EtherCAT 电机测试  (CSV 速度模式)
[  250.838490] EtherCAT: Successfully requested master 0.
  目标速度 = 50000  count/s     周期 = 1.0 ms
[  250.838733] EtherCAT 0: Domain0: Logical address 0x00000000, 22 byte, expected working counter 3.
═══════════════════════════════════════════════════
[  250.838745] EtherCAT 0:   Datagram domain0-0-main: Logical offset 0x00000000, 22 byte, type LRW at
7ff68845.

[  250.838906] EtherCAT 0: Master thread exited.
[步骤 1/6] 获取主站...
[  250.838921] EtherCAT 0: Starting EtherCAT-OP thread.
  ✓ 主站 0 获取成功
[  250.839082] EtherCAT WARNING 0: 1 datagram UNMATCHED!

[步骤 2/6] 创建数据域...
  ✓ 数据域创建成功

[步骤 3/6] 配置从站...
  ✓ 从站配置完成 (vendor=0xA79 product=0x5000)

[步骤 4/6] 注册 PDO 条目...
  ✓ PDO 偏移:  CtrlWd=+0  TargetVel=+6  StatusWd=+11  ActPos=+13  ActVel=+17

[步骤 5/6] 激活主站 (进入 OP)...
  ✓ 主站已激活, 从站应进入 OP
  ✓ 从站数=1  状态=link_up (期望 OP)

[步骤 6/6] 设置 RT 线程 (FIFO 20)...
  ✓ RT 线程就绪  (使用 sudo 运行避免权限问题)

──────────────────────────────────────────────────
  开始实时循环...  (Ctrl+C 停止)
──────────────────────────────────────────────────

[  250.944322] EtherCAT 0: Domain 0: Working counter changed to 3/3.
[  251.202822] EtherCAT 0: Slave states on main device: OP.
  [t=1s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=2533  Pos=5447899  Ctrl=0x000F  overrun=1000
  [t=2s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=12366  Pos=5458004  Ctrl=0x000F  overrun=2000
  [t=3s]  Status=0x1237  ★ENABLED★  Mode=9 Vel=22266  Pos=5478219  Ctrl=0x000F  overrun=3000
  [t=4s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=32216  Pos=5508543  Ctrl=0x000F  overrun=4000
  [t=5s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=42133  Pos=5548931  Ctrl=0x000F  overrun=5000
  [t=6s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49616  Pos=5598352  Ctrl=0x000F  overrun=6000
```

```
  [t=7s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49933  Pos=5649039  Ctrl=0x000F  overrun=7000
  [t=8s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49999  Pos=5699812  Ctrl=0x000F  overrun=8000
  [t=9s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49966  Pos=5750506  Ctrl=0x000F  overrun=9000
  [t=10s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50016  Pos=5801219  Ctrl=0x000F  overrun=10000
  [t=11s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49983  Pos=5851950  Ctrl=0x000F  overrun=11000
  [t=12s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49966  Pos=5902643  Ctrl=0x000F  overrun=12000
  [t=13s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49983  Pos=5953403  Ctrl=0x000F  overrun=13000
  [t=14s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50083  Pos=6004136  Ctrl=0x000F  overrun=14000
  [t=15s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50033  Pos=6054865  Ctrl=0x000F  overrun=15000
  [t=16s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=49983  Pos=6105548  Ctrl=0x000F  overrun=16000
  [t=17s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50066  Pos=6156253  Ctrl=0x000F  overrun=17000
  [t=18s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50016  Pos=6207001  Ctrl=0x000F  overrun=18000
  [t=19s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50033  Pos=6257710  Ctrl=0x000F  overrun=19000
  [t=20s]  Status=0x1637  ★ENABLED★  Mode=9 Vel=50083  Pos=6308467  Ctrl=0x000F  overrun=20000

──────────────────────────────────────────────────────────────
  收到退出信号，安全停机中...
──────────────────────────────────────────────────────────────
^C  已安全退出。

[  271.376260] EtherCAT 0: Releasing master...
[  271.376311] EtherCAT 0: Master thread exited.
[  271.376337] EtherCAT 0: Starting EtherCAT-IDLE thread.
root@rk3506-buildroot:/# [  271.376512] EtherCAT 0: Released.
[  271.385863] EtherCAT 0: Slave states on main device: PREOP.

root@rk3506-buildroot:/#
```

3. 电机运行图



第22章 主站电机控制实现

22.1 Codex CLI安装

1. Codex CLI 介绍

Codex CLI 是 OpenAI 官方推出的、可在本地终端中运行的AI编程智能体（Coding Agent）。你可以把它想象成一个能在你的电脑上直接帮你写代码、改代码、运行命令的AI助手。

2. 核心能力

- **本地运行，安全可控**：Codex CLI 完全在你的本地计算机上运行，可以安全地读取、修改和运行你指定目录下的代码。它采用 Rust 编写，保证了运行速度和效率，代码也已在 GitHub 上开源。
- **"智能体循环"工作模式**：它的核心是一个"智能体循环"。简单来说，你给它一个任务（如"修复登录页面的bug"），它会自主地进行**思考 → 调用工具（如读写文件、执行命令）→ 根据结果再思考**的循环，直到任务完成为止。
- **强大的集成能力**：它不仅可以独立在终端使用，还能与你的开发环境深度集成。

3. 主要功能

根据官方文档，Codex CLI 提供了非常丰富的功能：

- **交互式编程**：在终端输入 `codex` 即可启动交互式会话，用自然语言描述你的需求。
- **代码审查**：在你提交代码前，可以调用一个独立的 Codex 智能体对代码进行审查。
- **子任务并行**：支持使用"子智能体"来并行处理复杂任务，提高效率。
- **联网搜索**：让 Codex 在需要时搜索网络，获取最新信息来完成任务。
- **图像处理**：可以在 CLI 中生成或编辑图像，或将设计稿作为参考提供给 Codex。
- **脚本自动化**：支持通过 `exec` 命令编写脚本，将重复的工作流自动化。
- **云端任务**：可以启动一个 Codex Cloud 任务，在云端环境中运行，并将结果应用回本地。

4. 安装Codex CLI

- **安装 Node.js**

CC Switch 管理的 CLI 工具（Claude Code、Codex、Gemini CLI）需要 Node.js 环境。

```
# 官网的链接
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs

# 或则使用这种方式
# 或使用 nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.0/install.sh | bash
nvm install --lts
```

- **安装 CLI 工具**

```
sudo npm install -g @openai/codex

# 国内用户如下载慢，使用镜像源
npm install -g @openai/codex --registry=https://registry.npmmirror.com
```

- **运行Codex CLI**

```
codex
```

- 执行结果

```
linux@ubuntu:~$ curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
[sudo] linux 的密码：
2026-07-17 15:26:41 - Installing pre-requisites
命中:1 http://mirrors.aliyun.com/ubuntu jammy InRelease
获取:2 http://mirrors.aliyun.com/ubuntu jammy-updates InRelease [128 kB]
命中:3 https://packages.microsoft.com/repos/code stable InRelease
命中:4 https://deb.nodesource.com/node_24.x nodistro InRelease
获取:5 http://mirrors.aliyun.com/ubuntu jammy-backports InRelease [127 kB]
获取:6 http://mirrors.aliyun.com/ubuntu jammy-security InRelease [129 kB]
获取:7 http://mirrors.aliyun.com/ubuntu jammy-updates/main amd64 Packages [3,645 kB]
获取:8 http://mirrors.aliyun.com/ubuntu jammy-updates/main i386 Packages [1,042 kB]
获取:9 http://mirrors.aliyun.com/ubuntu jammy-updates/main Translation-en [547 kB]
获取:10 http://mirrors.aliyun.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [117 kB]
获取:11 http://mirrors.aliyun.com/ubuntu jammy-updates/restricted amd64 Packages [6,245 kB]
获取:12 http://mirrors.aliyun.com/ubuntu jammy-updates/restricted Translation-en [1,195 kB]
获取:13 http://mirrors.aliyun.com/ubuntu jammy-updates/universe i386 Packages [815 kB]
获取:14 http://mirrors.aliyun.com/ubuntu jammy-updates/universe amd64 Packages [1,278 kB]
获取:15 http://mirrors.aliyun.com/ubuntu jammy-updates/universe Translation-en [322 kB]
获取:16 http://mirrors.aliyun.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [360 kB]
获取:17 http://mirrors.aliyun.com/ubuntu jammy-updates/multiverse amd64 DEP-11 Metadata [940 B]
获取:18 http://mirrors.aliyun.com/ubuntu jammy-backports/main amd64 DEP-11 Metadata [5,740 B]
获取:19 http://mirrors.aliyun.com/ubuntu jammy-backports/universe amd64 DEP-11 Metadata [12.4 kB]
获取:20 http://mirrors.aliyun.com/ubuntu jammy-security/main amd64 Packages [3,361 kB]
获取:21 http://mirrors.aliyun.com/ubuntu jammy-security/main i386 Packages [863 kB]
获取:22 http://mirrors.aliyun.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [74.1 kB]
获取:23 http://mirrors.aliyun.com/ubuntu jammy-security/universe amd64 Packages [1,044 kB]
获取:24 http://mirrors.aliyun.com/ubuntu jammy-security/universe i386 Packages [706 kB]
获取:25 http://mirrors.aliyun.com/ubuntu jammy-security/universe amd64 DEP-11 Metadata [127 kB]
已下载 22.1 MB，耗时 10分 55秒 (33.8 kB/s)
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 43 个软件包可以升级。请执行 'apt list --upgradable' 来查看它们。
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
ca-certificates 已经是最新版 (20260601~22.04.1)。
curl 已经是最新版 (7.81.0-1ubuntu1.25)。
gnupg 已经是最新版 (2.2.27-3ubuntu2.5)。
apt-transport-https 已经是最新版 (2.4.14)。
升级了 0 个软件包，新安装了 0 个软件包，要卸载 0 个软件包，有 43 个软件包未被升级。
gpg: 警告：家目录'/home/linux/.gnupg'的所有权不安全
```

```
命中:1 http://mirrors.aliyun.com/ubuntu jammy InRelease
命中:2 http://mirrors.aliyun.com/ubuntu jammy-updates InRelease
命中:3 http://mirrors.aliyun.com/ubuntu jammy-backports InRelease
命中:4 http://mirrors.aliyun.com/ubuntu jammy-security InRelease
命中:5 https://packages.microsoft.com/repos/code stable InRelease
命中:6 https://deb.nodesource.com/node_24.x nodistro InRelease
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
有 43 个软件包可以升级。请执行 ‘apt list --upgradable’ 来查看它们。
2026-07-17 15:37:47 - Repository configured successfully.
2026-07-17 15:37:47 - To install Node.js, run: apt install nodejs -y
2026-07-17 15:37:47 - You can use N|solid Runtime as a node.js alternative
2026-07-17 15:37:47 - To install N|solid Runtime, run: apt install nsolid -y
linux@ubuntu:~$
linux@ubuntu:~$ sudo apt-get install -y nodejs
[sudo] linux 的密码：
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
下列软件包将被升级：
  nodejs
升级了 1 个软件包，新安装了 0 个软件包，要卸载 0 个软件包，有 42 个软件包未被升级。
需要下载 38.7 MB 的归档。
解压缩后会消耗 847 kB 的额外空间。
获取:1 https://deb.nodesource.com/node_24.x nodistro/main amd64 nodejs amd64 24.18.0-1nodesource1 [38.7 MB]
已下载 38.7 MB，耗时 15秒 (2,534 kB/s)
(正在读取数据库 ... 系统当前共安装有 247342 个文件和目录。)
准备解压 .../nodejs_24.18.0-1nodesource1_amd64.deb  ...
Detected old npm client, removing...
正在解压 nodejs (24.18.0-1nodesource1) 并覆盖 (24.16.0-1nodesource1) ...
正在设置 nodejs (24.18.0-1nodesource1) ...
正在处理用于 man-db (2.10.2-1) 的触发器 ...
linux@ubuntu:~$
linux@ubuntu:~$ node --version
v24.18.0
linux@ubuntu:~$
linux@ubuntu:~$ sudo npm install -g @openai/codex

changed 2 packages in 22s
linux@ubuntu:~$
linux@ubuntu:~$ codex --version
codex-cli 0.144.5
```

5. 配置中转站

- 推荐一个我自己在用的

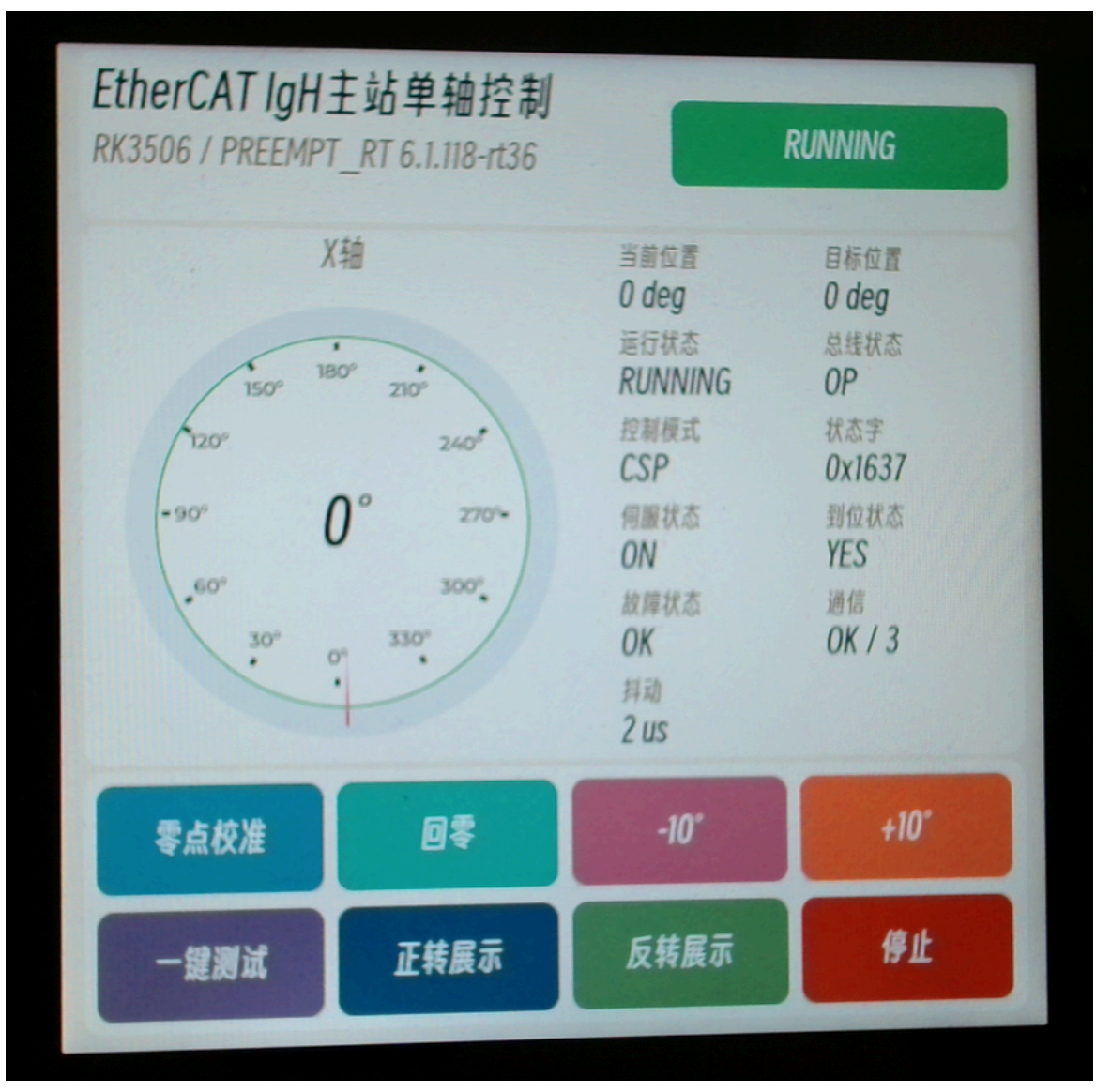
```
https://keygo.vip
```

- 理解1个概念，0.04是什么意思， 数字越小越便宜
- 运行结果

```
linux@ubuntu:~$ codex
╭──────────────────────────────────────────────╮
│ >_ OpenAI Codex (v0.144.5)                   │
│                                              │
│ model:     gpt-5.4 xhigh   /model to change  │
│ directory: ~                                 │
╰──────────────────────────────────────────────╯

  Tip: GPT-5.5 is now available in Codex. It's our strongest agentic coding model yet, built to reason
  through large codebases, check assumptions with tools, and keep going until the work is done.

  Learn more: https://openai.com/index/introducing-gpt-5-5/


› Improve documentation in @filename

  gpt-5.4 xhigh · ~
linux@ubuntu:~$
```

22.2 项目最终效果

1. 项目最终效果



- 输入日志

```
DDR d27ac532c4 typ 25/03/11-14:46:30,fwver: v1.06
tREFI:4x, sr_idle:93, pd_idle:13
perf_cpu_rt
PHY drv:clk:40,ca:48,DQ:40,odt:240
vrefinner:50%, vrefout:50%
dram drv:40,odt:120
sr_dq:0, sr_ca:0, sr_clk:0
rg:0x10-0x1-0x2, 0x6-0x1-0x2,status:a007
rdtrn:0x14-0x2d-0x47(0x33)
wrtrn:0x0-0x1f-0x3d(0x3d)
DDR3, 750MHz
BW=16 Col=10 Bk=8 CS0 Row=14 CS=1 Size=256MB
out
U-Boot SPL board init
U-Boot SPL 2017.09 (Jul 18 2026 - 03:13:49)
sfc cmd=03H(6BH-x4)
SPI Nand ID c8 92 c8
unrecognized JEDEC id bytes: ff, c8, 92
Trying to boot from MTD1
Trying fit image at 0x2000 sector
## Verified-boot: 0
## Checking optee 0x00001000 ... sha256(e2a712a533...) + OK
Trying kernel at 0xea00 sector from 'boot' part
## Checking fdt 0x00063000 ... sha256(ac6983c2d2...) + OK
## Checking kernel 0x01100000 ... sha256(a2a0b384c4...) + OK
Jumping to Kernel(0x01100000) via OP-TEE(0x00001000)
Total: 290.959/337.944 ms

I/TC:
I/TC: OP-TEE version: 3.13.0-1038-gb204648dc #derrick.huang (gcc version 10.2.1 20201103 (GNU Toolchain for
the A-profile Architecture 10.2-2020.11 (arm-10.16))) #2 2025年 12月 11日 星期四 11:51:18 CST arm, fwver:
v2.40
I/TC: Status: cluster=0xc00, core=0xe100, bootcpu=0
I/TC: Next entry point address: 0x01100000
I/TC: OP-TEE memory size: TEEOS 0x5e000 TA 0x1000 SHM 0x1000
I/TC: Primary CPU initializing
I/TC: Primary CPU switching to normal world boot
[    0.428761] Booting Linux on physical CPU 0xf00
[    0.428777] Linux version 6.1.118-rt36 (linux@ubuntu) (arm-none-linux-gnueabihf-gcc (GNU Toolchain for
the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-
profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1.20210621) #30 SMP PREEMPT_RT Sat Jul 18 12:47:22 CST
2026
[    0.428787] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=50c5387d
[    0.428797] CPU: div instructions available: patching division code
```

```
[    0.428801] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing instruction cache
[    0.429071] OF: fdt: Machine model: Rockchip RK3506 IDO EVB3506 V10 Board
[    0.430998] earlycon: uart8250 at MMIO32 0xff0a0000 (options '')
[    0.437041] printk: bootconsole [uart8250] enabled
[    0.437147] Memory policy: Data cache writealloc
[    0.437354] Reserved memory: created CMA memory pool at 0x0ea00000, size 22 MiB
[    0.437361] OF: reserved mem: initialized node linux,cma, compatible id shared-dma-pool
[    0.437764] Zone ranges:
[    0.437774]   Normal   [mem 0x0000000000000000-0x00000000ffffffff]
[    0.437787] Movable zone start for each node
[    0.437789] Early memory node ranges
[    0.437792]   node   0: [mem 0x0000000000000000-0x00000000ffffffff]
[    0.437798] Initmem setup node 0 [mem 0x0000000000000000-0x00000000ffffffff]
[    0.448082] psci: probing for conduit method from DT.
[    0.448095] psci: PSCIv1.0 detected in firmware.
[    0.448102] psci: Using standard PSCI v0.2 function IDs
[    0.448111] psci: SMC Calling Convention v1.1
[    0.448250] percpu: Embedded 12 pages/cpu s16800 r8192 d24160 u49152
[    0.448356] Built 1 zonelists, mobility grouping on.  Total pages: 65024
[    0.448369] Kernel command line: earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0 ubi.mtd=5
root=ubi0:rootfs ro rootfstype=ubifs rootwait snd_aloop.index=7 snd_aloop.use_raw_jiffies=1
isolcpus=domain,managed_irq,2 nohz_full=2 irqaffinity=0-1 rcu_nocbs=2 rcu_nocb_poll printk.time=1
mtdparts=spi-
nand0:0x600000@0x400000(uboot),0x100000@0xa00000(misc),0x40000@0xb00000(vnvm),0x1200000@0xb40000(recovery),0
xa00000@0x1d40000(boot),0xa000000@0x2740000(rootfs),0x1000000@0xc740000(oem),0x2820000@0xd740000(userdata)
user_debug=31
[    0.448852] Dentry cache hash table entries: 32768 (order: 5, 131072 bytes, linear)
[    0.448891] Inode-cache hash table entries: 16384 (order: 4, 65536 bytes, linear)
[    0.448899] mem auto-init: stack:off, heap alloc:off, heap free:off
[    0.454509] Memory: 230008K/262144K available (3596K kernel code, 393K rwdata, 1868K rodata, 184K init,
124K bss, 9608K reserved, 22528K cma-reserved)
[    0.454663] SLUB: HWalign=64, Order=0-3, MinObjects=0, CPUs=3, Nodes=1
[    0.454904] rcu: Preemptible hierarchical RCU implementation.
[    0.454911] rcu:     RCU restricting CPUs from NR_CPUS=4 to nr_cpu_ids=3.
[    0.454917] rcu:     RCU priority boosting: priority 1 delay 500 ms.
[    0.454922] rcu:     RCU_SOFTIRQ processing moved to rcuc kthreads.
[    0.454925]  No expedited grace period (rcu_normal_after_boot).
[    0.454929] rcu: RCU calculated value of scheduler-enlistment delay is 100 jiffies.
[    0.454932] rcu: Adjusting geometry for rcu_fanout_leaf=16, nr_cpu_ids=3
[    0.454967] NR_IRQS: 16, nr_irqs: 16, preallocated irqs: 16
[    0.455854] rcu:     Offload RCU callbacks from CPUs: 2.
[    0.455866] rcu:     Poll for callbacks from no-CBs CPUs.
[    0.455874] rcu: srcu_init: Setting srcu_struct sizes based on contention.
[    0.466424] arch_timer: cp15 timer(s) running at 24.00MHz (virt).
[    0.466450] clocksource: arch_sys_counter: mask: 0xffffffffffffff max_cycles: 0x588fe9dc0, max_idle_ns:
440795202592 ns
[    0.466466] sched_clock: 56 bits at 24MHz, resolution 41ns, wraps every 4398046511097ns
[    0.466476] Switching to timer-based delay loop, resolution 41ns
[    0.466890] Console: colour dummy device 80x30
[    0.492249] Calibrating delay loop (skipped), value calculated using timer frequency.. 48.00 BogoMIPS
(lpj=24000)
[    0.492261] CPU: Testing write buffer coherency: ok
[    0.492305] pid_max: default: 4096 minimum: 301
[    0.492466] Mount-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
[    0.492483] Mountpoint-cache hash table entries: 1024 (order: 0, 4096 bytes, linear)
[    0.493303] CPU0: update cpu_capacity 1024
[    0.496916] CPU0: thread -1, cpu 0, socket 15, mpidr 80000f00
[    0.498241] Setting up static identity map for 0x108200 - 0x108254
[    0.499223] rcu: Hierarchical SRCU implementation.
[    0.499230] rcu:     Max phase no-delay instances is 400.
[    0.499335] printk: bootconsole [uart8250] printing thread started
[    0.501903] smp: Bringing up secondary CPUs ...
I/TC: Secondary CPU 1 initializing
I/TC: Secondary CPU 1 switching to normal world boot
[    0.504255] CPU1: update cpu_capacity 1024
[    0.504269] CPU1: thread -1, cpu 1, socket 15, mpidr 80000f01
I/TC: Secondary CPU 2 initializing
I/TC: Secondary CPU 2 switching to normal world boot
[    0.507254] CPU2: update cpu_capacity 1024
[    0.507268] CPU2: thread -1, cpu 2, socket 15, mpidr 80000f02
[    0.507403] smp: Brought up 1 node, 3 CPUs
[    0.507417] SMP: Total of 3 processors activated (144.00 BogoMIPS).
[    0.507424] CPU: All CPU(s) started in SVC mode.
[    0.508501] devtmpfs: initialized
[    0.521010] VFP support v0.3: implementor 41 architecture 2 part 30 variant 7 rev 5
[    0.521322] clocksource: jiffies: mask: 0xffffffff max_cycles: 0xffff
                                                                      [    0.521345] futex hash table
entries: 16 (order: -2, 1024 bytes, linear)
[    0.521437] pinctrl core: initialized pinctrl subsystem
[    0.522643] NET: Registered PF_NETLINK/PF_ROUTE protocol family
[    0.523183] DMA: preallocated 256 KiB pool for atomic coherent allocations
[    0.524319] Registered FIQ tty driver
[    0.524744] thermal_sys: Registered thermal governor 'step_wise'
[    0.524752] thermal_sys: Registered thermal governor 'user_space'
[    0.525053] cpuidle: using governor menu
[    0.526680] ramoops: boot-log-0      0xd000@0x00083000
[    0.526919] ramoops: console 0x20000@0x00090000
[    0.527207] printk: console [ramoops-1] enabled
[    0.527211] printk: console [ramoops-1] printing thread started
[    0.527217] pstore: Registered ramoops as persistent store backend
[    0.527221] ramoops: using 0x2d000@0x83000, ecc: 0
[    0.540639] rockchip-gpio ff940000.gpio: probed /pinctrl/gpio@ff940000
[    0.541758] rockchip-gpio ff870000.gpio: probed /pinctrl/gpio@ff870000
[    0.542836] rockchip-gpio ff1c0000.gpio: probed /pinctrl/gpio@ff1c0000
[    0.543911] rockchip-gpio ff1d0000.gpio: probed /pinctrl/gpio@ff1d0000
[    0.545210] rockchip-gpio ff1e0000.gpio: probed /pinctrl/gpio@ff1e0000
[    0.545323] rockchip-pinctrl pinctrl: probed pinctrl
[    0.565334] printk: console [ttyFIQ0] enabled
[    0.565339] printk: console [ttyFIQ0] printing thread started
[    0.565345] printk: bootconsole [uart8250] disabled
[    0.565439] printk: bootconsole [uart8250] printing thread stopped
[    0.565887] Registered fiq debugger ttyFIQ0
[    0.570406] usbcore: registered new interface driver usbfs
[    0.570469] usbcore: registered new interface driver hub
[    0.570518] usbcore: registered new device driver usb
[    0.570912] mc: Linux media interface: v0.10
[    0.570992] videodev: Linux video capture interface: v2.00
[    0.571033] pps_core: LinuxPPS API ver. 1 registered
[    0.571038] pps_core: Software ver. 5.3.6 - Copyright 2005-2007 Rodolfo Giometti <giometti@linux.it>
[    0.571061] PTP clock support registered
[    0.572591] Advanced Linux Sound Architecture Driver Initialized.
[    0.573594] rockchip-cpuinfo cpuinfo: SoC            : 35060000
[    0.573610] rockchip-cpuinfo cpuinfo: Serial         : 1af40e62f3b29ca2
[    0.574583] clocksource: Switched to clocksource arch_sys_counter
[    0.586902] NET: Registered PF_INET protocol family
[    0.587120] IP idents hash table entries: 4096 (order: 3, 32768 bytes, linear)
[    0.588275] tcp_listen_portaddr_hash hash table entries: 256 (order: 0, 5120 bytes, linear)
[    0.588304] Table-perturb hash table entries: 65536 (order: 6, 262144 bytes, linear)
[    0.588320] TCP established hash table entries: 2048 (order: 1, 8192 bytes, linear)
[    0.588351] TCP bind hash table entries: 2048 (order: 4, 81920 bytes, linear)
[    0.588482] TCP: Hash tables configured (established 2048 bind 2048)
[    0.588700] UDP hash table entries: 128 (order: 0, 6144 bytes, linear)
[    0.588729] UDP-Lite hash table entries: 128 (order: 0, 6144 bytes, linear)
[    0.588951] NET: Registered PF_UNIX/PF_LOCAL protocol family
[    0.590672] rockchip-thermal ff650000.tsadc: tsadc is probed successfully!
[    0.592076] Initialise system trusted keyrings
[    0.592280] workingset: timestamp_bits=30 max_order=16 bucket_order=0
[    0.597166] squashfs: version 4.0 (2009/01/31) Phillip Lougher
[    0.597305] Key type asymmetric registered
[    0.597315] Asymmetric key parser 'x509' registered
[    0.597446] io scheduler mq-deadline registered
[    0.597453] io scheduler kyber registered
[    0.597759] io scheduler bfq registered
[    0.607519] dma-pl330 ff000000.dma-controller: Loaded driver for PL330 DMAC-241330
[    0.607535] dma-pl330 ff000000.dma-controller:       DBUFF-32x8bytes Num_Chans-6 Num_Peri-12 Num_Events-
12
[    0.610050] dma-pl330 ff008000.dma-controller: Loaded driver for PL330 DMAC-241330
[    0.610065] dma-pl330 ff008000.dma-controller:       DBUFF-128x8bytes Num_Chans-8 Num_Peri-20 Num_Events-
16
[    0.610465] rockchip-system-monitor rockchip-system-monitor: system monitor probe
[    0.611754] Serial: 8250/16550 driver, 6 ports, IRQ sharing disabled
[    0.612464] ff0b0000.serial: ttyS1 at MMIO 0xff0b0000 (irq = 58, base_baud = 1500000) is a 16550A
[    0.613420] ff0d0000.serial: ttyS3 at MMIO 0xff0d0000 (irq = 59, base_baud = 1500000) is a 16550A
[    0.614766] random: crng init done
[    0.620214] rockchip-drm display-subsystem: bound ff600000.vop (ops 0xb04bd704)
[    0.624256] spi-nand spi2.0: GigaDevice SPI NAND was found.
[    0.624271] spi-nand spi2.0: 256 MiB, block size: 128 KiB, page size: 2048, OOB size: 128
[    0.625756] 8 cmdlinepart partitions found on MTD device spi-nand0
[    0.625770] Creating 8 MTD partitions on "spi-nand0":
[    0.625781] 0x000000400000-0x000000a00000 : "uboot"
[    0.627104] 0x000000a00000-0x000000b00000 : "misc"
[    0.628689] 0x000000b00000-0x000000b40000 : "vnvm"
[    0.630021] 0x000000b40000-0x000001d40000 : "recovery"
[    0.631476] 0x000001d40000-0x000002740000 : "boot"
[    0.632900] 0x000002740000-0x00000c740000 : "rootfs"
[    0.634896] 0x00000c740000-0x00000d740000 : "oem"
[    0.636206] 0x00000d740000-0x00000ff60000 : "userdata"
[    0.638889] CAN device driver interface
```

```
[    0.640465] rk3576_canfd ff330000.can: CAN info: use_dma = 0, rx_max_data = 18, fifo_depth = 14
[    0.641777] rk_gmac-dwmac ff4c8000.ethernet: clock input or output? (input).
[    0.641802] rk_gmac-dwmac ff4c8000.ethernet: integrated PHY? (no).
[    0.641844] rk_gmac-dwmac ff4c8000.ethernet: clock input from PHY
[    0.642102] rk_gmac-dwmac ff4c8000.ethernet: init for RMII
[    0.642339] rk_gmac-dwmac ff4c8000.ethernet: User ID: 0x40, Synopsys ID: 0x51
[    0.642359] rk_gmac-dwmac ff4c8000.ethernet:         DWMAC4/5
[    0.642367] rk_gmac-dwmac ff4c8000.ethernet: DMA HW capability register supported
[    0.642373] rk_gmac-dwmac ff4c8000.ethernet: RX Checksum Offload Engine supported
[    0.642378] rk_gmac-dwmac ff4c8000.ethernet: TX Checksum insertion supported
[    0.642383] rk_gmac-dwmac ff4c8000.ethernet: Wake-Up On Lan supported
[    0.642473] rk_gmac-dwmac ff4c8000.ethernet: TSO supported
[    0.642480] rk_gmac-dwmac ff4c8000.ethernet: Enable RX Mitigation via HW Watchdog Timer
[    0.642488] rk_gmac-dwmac ff4c8000.ethernet: rk_get_eth_addr: vendor MAC unavailable (-1), use stable
fallback 02:a2:9c:b2:f3:6d
[    0.642498] rk_gmac-dwmac ff4c8000.ethernet: device MAC address 02:a2:9c:b2:f3:6d
[    0.642509] rk_gmac-dwmac ff4c8000.ethernet: Enabled RFS Flow TC (entries=10)
[    0.642517] rk_gmac-dwmac ff4c8000.ethernet: TSO feature enabled
[    0.642523] rk_gmac-dwmac ff4c8000.ethernet: Using 40/40 bits DMA host/device width
[    0.775160] rk_gmac-dwmac ff4d0000.ethernet: clock input or output? (input).
[    0.775184] rk_gmac-dwmac ff4d0000.ethernet: integrated PHY? (no).
[    0.775222] rk_gmac-dwmac ff4d0000.ethernet: clock input from PHY
[    0.775487] rk_gmac-dwmac ff4d0000.ethernet: init for RMII
[    0.775770] rk_gmac-dwmac ff4d0000.ethernet: User ID: 0x40, Synopsys ID: 0x51
[    0.775791] rk_gmac-dwmac ff4d0000.ethernet:         DWMAC4/5
[    0.775799] rk_gmac-dwmac ff4d0000.ethernet: DMA HW capability register supported
[    0.775805] rk_gmac-dwmac ff4d0000.ethernet: RX Checksum Offload Engine supported
[    0.775810] rk_gmac-dwmac ff4d0000.ethernet: TX Checksum insertion supported
[    0.775814] rk_gmac-dwmac ff4d0000.ethernet: Wake-Up On Lan supported
[    0.775906] rk_gmac-dwmac ff4d0000.ethernet: TSO supported
[    0.775913] rk_gmac-dwmac ff4d0000.ethernet: Enable RX Mitigation via HW Watchdog Timer
[    0.775921] rk_gmac-dwmac ff4d0000.ethernet: rk_get_eth_addr: vendor MAC unavailable (-1), use stable
fallback 02:a2:9c:b2:f3:6e
[    0.775932] rk_gmac-dwmac ff4d0000.ethernet: device MAC address 02:a2:9c:b2:f3:6e
[    0.775941] rk_gmac-dwmac ff4d0000.ethernet: Enabled RFS Flow TC (entries=10)
[    0.775950] rk_gmac-dwmac ff4d0000.ethernet: TSO feature enabled
[    0.775956] rk_gmac-dwmac ff4d0000.ethernet: Using 40/40 bits DMA host/device width
[    0.911834] i2c_dev: i2c /dev entries driver
[    0.915783] rtc-hym8563 0-0051: rtc information is valid
[    0.922896] rtc-hym8563 0-0051: registered as rtc0
[    0.923958] rtc-hym8563 0-0051: setting system clock to 2026-07-18T09:04:37 UTC (1784365477)
[    0.924757] goodix_ts_probe() start
[    1.082607] ==>Gt9XX invert_x:0, invert_y:0, swap_x_y:0
[    1.082635] ======= abs_y_max=720ax=720
[    1.082841] input: goodix-ts as /devices/platform/ff040000.i2c/i2c-0/0-0014/input/input0
[    1.087045] softdog: initialized. soft_noboot=0 soft_margin=60 sec soft_panic=0 (nowayout=0)
[    1.087059] softdog:              soft_reboot_cmd=<not set> soft_active_on_boot=0
[    1.087286] cpu cpu0: bin=0
[    1.087338] cpu cpu0: leakage=7
[    1.087375] cpu cpu0: pvtm = 1749, get from otp
[    1.087398] cpu cpu0: pvtm-volt-sel=4
[    1.087409] cpu cpu0: soc version=0, speed=4
[    1.088257] cpu cpu0: avs=0
[    1.088442] cpu cpu0: l=10000 h=2147483647 hyst=5000 l_limit=0 h_limit=0 h_table=1
[    1.089912] Synopsys Designware Multimedia Card Interface Driver
[    1.091764] dwmmc_rockchip ff480000.mmc: IDMAC supports 32-bit address mode.
[    1.091796] dwmmc_rockchip ff480000.mmc: Using internal DMA controller.
[    1.091805] dwmmc_rockchip ff480000.mmc: Version ID is 270a
[    1.092051] dwmmc_rockchip ff480000.mmc: DW MMC controller at irq 71,32 bit host data width,256 deep fifo
[    1.092220] usbcore: registered new interface driver usbhid
[    1.092229] usbhid: USB HID core driver
[    1.092279] dwmmc_rockchip ff480000.mmc: Got CD GPIO
[    1.100527] NET: Registered PF_PACKET protocol family
[    1.100541] can: controller area network core
[    1.100787] NET: Registered PF_CAN protocol family
[    1.100803] can: raw protocol
[    1.100814] can: broadcast manager protocol
[    1.100835] can: netlink gateway - max_hops=1
[    1.105010] mmc_host mmc0: Bus speed (slot 0) = 400000Hz (slot req 400000Hz, actual 400000HZ div = 0)
[    1.110866] mtd vendor storage:20200313 ret = 0
[    1.111186] Registering SWP/SWPB emulation handler
[    1.111444] Loading compiled-in X.509 certificates
[    1.116045] rga2 ff610000.rga: probe successfully, irq = 75, hw_version:3.a.07135
[    1.116228] rga: IOMMU binding successfully, default mapping core[0x4]
[    1.116411] rga: Module initialized. v1.3.8
[    1.118309] rockchip-drm display-subsystem: bound ff600000.vop (ops 0xb04bd704)
[    1.118424] rockchip-drm display-subsystem: bound ff640000.dsi (ops 0xb04be2d8)
[    1.119218] [drm] Initialized rockchip 4.0.0 20140818 for display-subsystem on minor 0
[    1.129311] Console: switching to colour frame buffer device 90x45
[    1.132809] rockchip-drm display-subsystem: [drm] fb0: rockchipdrmfb frame buffer device
[    1.133071] rockchip-drm display-subsystem: [drm] run display error_event monitor
[    1.133773] input: adc-keys as /devices/platform/adc-keys/input/input1
[    1.137169] ubi0: attaching mtd5
[    1.532328] ubi0: scanning is finished
[    1.538071] ubi0: attached mtd5 (name "rootfs", size 160 MiB)
[    1.538088] ubi0: PEB size: 131072 bytes (128 KiB), LEB size: 126976 bytes
[    1.538095] ubi0: min./max. I/O unit sizes: 2048/2048, sub-page size 2048
[    1.538101] ubi0: VID header offset: 2048 (aligned 2048), data offset: 4096
[    1.538106] ubi0: good PEBs: 1280, bad PEBs: 0, corrupted PEBs: 0
[    1.538111] ubi0: user volume: 1, internal volumes: 1, max. volumes count: 128
[    1.538117] ubi0: max/mean erase counter: 1/0, WL threshold: 4096, image sequence number: 2130849255
[    1.538124] ubi0: available PEBs: 0, total reserved PEBs: 1280, PEBs reserved for bad PEB handling: 40
[    1.538888] ubi0: background thread "ubi_bgt0d" started, PID 118
[    1.540985] UBIFS (ubi0:0): Mounting in unauthenticated mode
[    1.581198] UBIFS (ubi0:0): UBIFS: mounted UBI device 0, volume 0, name "rootfs", R/O mode
[    1.581216] UBIFS (ubi0:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes: 2048 bytes/2048
bytes
[    1.581225] UBIFS (ubi0:0): FS size: 128626688 bytes (122 MiB, 1013 LEBs), max 1024 LEBs, journal size
9023488 bytes (8 MiB, 72 LEBs)
[    1.581236] UBIFS (ubi0:0): reserved for root: 0 bytes (0 KiB)
[    1.581242] UBIFS (ubi0:0): media format: w4/r0 (latest is w5/r0), UUID 94F6DADB-0551-48F2-A00D-
95F3A5E58463, small LPT model
[    1.582667] VFS: Mounted root (ubifs filesystem) readonly on device 0:13.
[    1.583312] devtmpfs: mounted
[    1.583466] Freeing unused kernel image (initmem) memory: 184K
[    1.583475] Kernel memory protection not selected by kernel config.
[    1.583496] Run /sbin/init as init process
2026-07-18 09:04:38 motor_stack_ctl: stack start begin
2026-07-18 09:04:38 motor_stack_ctl: started ui pid=147
2026-07-18 09:04:38 motor_stack_ctl: started stack worker pid=157
2026-07-18 09:04:38 motor_stack_ctl: stack start detached, ui first and backend waits in background
[    2.395249] rk_gmac-dwmac ff4c8000.ethernet eth0: Register MEM_TYPE_PAGE_POOL RxQ-0
[    2.397830] rk_gmac-dwmac ff4c8000.ethernet eth0: PHY [stmmac-0:01] driver [YT8512B Ethernet] (irq=POLL)
[    2.398228] rockchip-vop ff600000.vop: [drm:vop_enable] Update mode to 720x720p60, type: 16
[    2.398953] dwmac4: Master AXI performs any burst length
[    2.398981] rk_gmac-dwmac ff4c8000.ethernet eth0: No Safety Features support found
[    2.399006] rk_gmac-dwmac ff4c8000.ethernet eth0: IEEE 1588-2008 Advanced Timestamp supported
[    2.399448] rk_gmac-dwmac ff4c8000.ethernet eth0: registered PTP clock
[    2.408716] rk_gmac-dwmac ff4c8000.ethernet eth0: configuring for phy/rmii link mode
[    2.410312] rk_gmac-dwmac ff4c8000.ethernet eth0: Link is Up - 100Mbps/Full - flow control off
[    2.419366] rk_gmac-dwmac ff4d0000.ethernet eth1: Register MEM_TYPE_PAGE_POOL RxQ-0
[    2.422082] rk_gmac-dwmac ff4d0000.ethernet eth1: PHY [stmmac-1:01] driver [YT8512B Ethernet] (irq=POLL)
[    2.423215] dwmac4: Master AXI performs any burst length
[    2.423247] rk_gmac-dwmac ff4d0000.ethernet eth1: No Safety Features support found
[    2.423276] rk_gmac-dwmac ff4d0000.ethernet eth1: IEEE 1588-2008 Advanced Timestamp supported
[    2.423711] rk_gmac-dwmac ff4d0000.ethernet eth1: registered PTP clock
[    2.432659] rk_gmac-dwmac ff4d0000.ethernet eth1: configuring for phy/rmii link mode
Start mounting all internal partitions in /etc/fstab
Log saved to /var/log/mount-all.log
[    2.701357] dw-mipi-dsi-rockchip ff640000.dsi: [drm:dw_mipi_dsi_bridge_atomic_enable] final DSI-Link
bandwidth: 588 x 2 Mbps
[    2.802956] ubi6: attaching mtd6
[    3.016159] ubi6: scanning is finished
[    3.022035] ubi6: attached mtd6 (name "oem", size 16 MiB)
[    3.022053] ubi6: PEB size: 131072 bytes (128 KiB), LEB size: 126976 bytes
[    3.022061] ubi6max. I/O unit sizes: 2048/2048, sub-page size 2048
[    3.022067] ubi6: VID header offset: 2048 (aligned 2048), data offset: 4096
[    3.022072] ubi6: good PEBs: 128, bad PEBs: 0, corrupted PEBs: 0
[    3.022077] ubi6: user volume: 1, internal volumes: 1, max. volumes count: 128
[    3.022083] ubi6: max/mean erase counter: 1/0, WL threshold: 4096, image sequence number: 2059247836
[    3.022090] ubi6: available PEBs: 0, total reserved PEBs: 128, PEBs reserved for bad PEB handling: 40
[    3.022294] ubi6: background thread "ubi_bgt6d" started, PID 282
[    3.123237] UBIFS (ubi6:0): Mounting in unauthenticated mode
[    3.172421] UBIFS (ubi6:0): UBIFS: mounted UBI device 6, volume 0, name "oem", R/O mode
[    3.172477] UBIFS (ubi6:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes: 2048 bytes/2048
bytes
[    3.172490] UBIFS (ubi6:0): FS size: 9396224 bytes (8 MiB, 74 LEBs), max 132 LEBs, journal size 2412544
bytes (2 MiB, 19 LEBs)
[    3.172737] UBIFS (ubi6:0): reserved for root: 0 bytes (0 KiB)
[    3.172751] UBIFS (ubi6:0): media format: w4/r0 (latest is w5/r0), UUID 7F10E537-7C3D-4953-ADEC-
E87930B072C6, small LPT model
[    3.496240] ubi7: attaching mtd7
[    3.807907] ubi7: scanning is finished
[    3.816945] ubi7: attached mtd7 (name "userdata", size 40 MiB)
[    3.817100] ubi7: PEB size: 131072 bytes (128 KiB), LEB size: 126976 bytes
[    3.817115] ubi7: min./max. I/O unit sizes: 2048/2048, sub-page size 2048
[    3.817122] ubi7: VID header offset: 2048 (aligned 2048), data offset: 4096
[    3.817127] ubi7: good PEBs: 321, bad PEBs: 0, corrupted PEBs: 0
```

```
[    3.817133] ubi7: user volume: 1, internal volumes: 1, max. volumes count: 128
[    3.817139] ubi7: max/mean erase counter: 3/1, WL threshold: 4096, image sequence number: 2146236446
[    3.817252] ec_master: loading out-of-tree module taints kernel.
[    3.820655] ubi7: available PEBs: 0, total reserved PEBs: 321, PEBs reserved for bad PEB handling: 40
[    3.825689] ubi7: background thread "ubi_bgt7d" started, PID 322
[    3.829522] EtherCAT: Master driver 1.6.0-rc1 unknown
[    3.830712] EtherCAT: 1 master waiting for devices.
[    3.871303] ec_generic: EtherCAT master generic Ethernet device module 1.6.0-rc1 unknown
[    3.871352] EtherCAT: Accepting 02:A2:9C:B2:F3:6D as main device for master 0.
[    3.871385] ec_generic: Binding socket to interface 3 (eth0).
[    3.881686] EtherCAT 0: Starting EtherCAT-IDLE thread.
[    3.881851] EtherCAT 0: Link state of ecm0 changed to UP.
[    3.883685] EtherCAT 0: 1 slave(s) responding on main device.
[    3.883708] EtherCAT 0: Slave states on main device: SAFEOP + ERROR.
[    3.884052] UBIFS (ubi7:0): Mounting in unauthenticated mode
[    3.884487] UBIFS (ubi7:0): background thread "ubifs_bgt7_0" started, PID 329
[    3.884604] EtherCAT 0: Scanning bus.
[    3.885428] EtherCAT WARNING 0-0: Slave has state error bit set (SAFEOP + ERROR)!
[    3.910731] UBIFS (ubi7:0): recovery needed
[    4.067018] UBIFS (ubi7:0): recovery completed
[    4.067043] UBIFS (ubi7:0): UBIFS: mounted UBI device 7, volume 0, name "userdata"
[    4.067051] UBIFS (ubi7:0): LEB size: 126976 bytes (124 KiB), min./max. I/O unit sizes: 2048 bytes/2048
bytes
[    4.067137] UBIFS (ubi7:0): FS size: 33648640 bytes (32 MiB, 265 LEBs), max 16912 LEBs, journal size
9023488 bytes (8 MiB, 72 LEBs)
[    4.067156] UBIFS (ubi7:0): reserved for root: 0 bytes (0 KiB)
[    4.067162] UBIFS (ubi7:0): media format: w4/r0 (latest is w5/r0), UUID 37OE9B62-F479-41BD-B5O6-
B7C420721F58, small LPT model
[    4.079876] EtherCAT 0: Bus scanning completed in 196 ms.
[    4.079900] EtherCAT 0: Using slave 0 as DC reference clock.
[    4.084124] EtherCAT ERROR 0-0: AL status message 0x001B: "Sync manager watchdog".
[    4.085053] EtherCAT 0-0: Acknowledged state SAFEOP.
[    4.090675] EtherCAT 0: Slave states on main device: PREOP.
Saving 256 bits of creditable seed for next boot
Starting syslogd: OK
Starting klogd: OK
Running sysctl: OK
Populating /dev using udev: [    4.444845] udevd[400]: starting version 3.2.14
[    4.560740] udevd[413]: starting eudev-3.2.14
done
[    8.045825] EtherCAT: Requesting master 0...
[    8.045848] EtherCAT: Successfully requested master 0.
[    8.046123] EtherCAT 0: Domain0: Logical address 0x00000000, 18 byte, expected working counter 3.
[    8.046135] EtherCAT 0:   Datagram domain0-0-main: Logical offset 0x00000000, 18 byte, type LRW at
ce31bfcd.
[    8.046240] EtherCAT 0: Master thread exited.
[    8.046286] EtherCAT 0: Starting EtherCAT-OP thread.
Seeding 256 bits and crediting
Saving 256 bits of creditable seed for next boot
root@rk3506-buildroot:/# [    9.209625] EtherCAT 0: Domain 0: Working counter changed to 3/3.
[    9.461815] EtherCAT 0: Slave states on main device: OP.
```

2. 项目指标

- 抖动控制在5u以内
- 一键测试： 旋转 +10° -> 0° -> -10°
- 正转展示：按照6 RPM速度正向转动
- 反转展示：按照6 RPM速度反向转动
- 停止： 在正转或反转时， 用于停止转动

3. 抖动计算原理

电机程序当前显示的"抖动"是周期线程的最大唤醒延迟，不是电机位置误差，也不是 EtherCAT DC 相位误差。

例如：1 kHz PDO 通信表示每 1 ms 完成一次 EtherCAT 控制周期。理想时间轴是：

t = 0 ms　接收上一帧 -> 处理反馈 -> 写入目标 PDO -> 发送
t = 1 ms　同样流程
t = 2 ms　同样流程
...

核心是使用绝对时间，而不是"本次做完后再 sleep 1 ms"。

wakeup_time += 1 ms;

这样某次循环即使晚了 20 us，下一次目标仍是固定的 2 ms、3 ms 时刻，不会把误差持续累积。

1kHz 的两类抖动

- **唤醒延迟**
  - 计划在 Tn 开始，实际在 An 开始：
  - latency[n] = An - Tn
  - 例如目标时刻为 10.000 ms，实际开始为 10.075 ms，则唤醒抖动为 75 us。

- **实际周期偏差**
  - 相邻两次实际开始时间的间隔：
  - period[n] = An - A(n-1)
- period_jitter[n] = period[n] - 1000 us
- 例如一次周期是 1.030 ms，下一次是 0.970 ms，平均仍可能为 1 ms，但瞬时周期存在 +30
- us/-30 us 偏差。

你当前代码显示的 jitter_us 实际取的是第一类，即统计窗口内的最大唤醒延迟：

jitter_us = ceil(max(An - Tn) / 1000)

- **为什么要"睡眠 + 忙等"**

普通 clock_nanosleep() 可能在目标时刻后几十微秒才被调度。当前实现会：

- 先睡到 Tn - 300 us
- 再循环读取单调时钟，忙等到 Tn

这能降低最后一段的调度不确定性。

但在 1 kHz 下，300 us 忙等占一个 CPU 核约 30%，可能挤占网卡 IRQ 或其他实时任务。通常可先从 50-150 us 忙等窗口开始测量，再决定是否增大。

1 kHz 的实际目标， 对于 1 ms 周期：

- max wakeup latency < 100 us：较好
- 100-200 us：多数单轴场景可用
- 300 us：应检查 CPU 绑定、IRQ 亲和性、实时优先级、后台负载
- = 1 ms：发生周期丢失，下一帧已经赶不上原定节拍

般 EtherCAT PDO 周期建议：

场景	常用频率	周期
状态采集、普通 IO	100-250 Hz	10-4 ms
普通位置控制	500 Hz	2 ms
伺服 CSP/同步运动	1 kHz	1 ms
高动态、多轴插补	2-4 kHz	0.5-0.25 ms

- 你当前是 250 Hz / 4 ms，用于界面演示、单轴低速运动基本够用。
- 若追求更平滑的 CSP 位置控制，建议先提升到 500 Hz，稳定后再测试 1 kHz。

4. 抖动4us以内方案

目标：在 1 ms EtherCAT 周期下，控制线程从理论唤醒点到实际开始执行的最大调度抖动不超过 4 us。

该指标与 DC 同步误差、报文延迟分开统计。

1. CPU 与系统隔离
   - CPU2 专供 EtherCAT 实时任务；CPU0 处理 IRQ、RCU、workqueue、存储及后台服务；CPU1 运行 UI 和守护进程。
   - 启用 PREEMPT_RT，隔离 CPU2 的 tick、RCU 回调和普通 IRQ。
   - CPU2 禁用 cpuidle，CPU/devfreq 固定 performance 与最高频率。
   - 禁用 RT 带宽限制、timer migration、调度组自动聚合，并降低内存回写和压缩干扰。
2. EtherCAT 网络路径
   - eth0 IRQ 固定 CPU2，IRQ 线程 SCHED_FIFO 60，ksoftirqd/2 为 FIFO 55。
   - EtherCAT OP/IDLE 分别保持 FIFO 42/15。
   - 关闭 EEE、GRO、GSO、TSO、LRO 与中断合并；关闭 RX RPS，TX XPS 固定 CPU2。
   - 确认 CPU2 无显示、Wi-Fi、存储等非必要 IRQ。

3. 周期控制线程
    - 控制线程绑定 CPU2，采用 SCHED_FIFO 70。
    - 使用 CLOCK_MONOTONIC + TIMER_ABSTIME 绝对时间唤醒，避免相对 sleep 累积误差。
    - 保留周期末 300 us 的受控忙等，降低 1 ms 周期尾部唤醒延迟。
    - 启动时执行 mlockall 并预触发实时内存；周期内禁止动态分配、文件 I/O、同步日志和阻塞锁。
4. DC 与保护
    - DC 校时使用理论唤醒时间，先完成预热和同步稳定后再使能伺服。
    - 记录唤醒延迟、周期偏差、执行时长、DC 差值、WKC 和丢帧。
    - 单次超过 4 us 告警；连续 3 次超过 5 us，或 WKC/DC 异常时冻结目标位置并停止新运动指令。
5. 验收
    - 空载连续运行 24 小时：控制线程 max <= 4 us，无丢帧和 WKC 异常。
    - 在 UI、网络、存储和 CPU 组合压力下：p99.99 <= 4us，不得出现超过 5 us 的保护事件。
    - 使用 CPU2 上的 cyclictest 建立基线，但以控制线程内部纳秒时间戳作为最终验收结果。

22.3 电机控制实战

1. 第一阶段： 关闭开机自动运行的智能家居程序

关闭智能家居开启电机界面自启动

- 提示词

```
电机程序设置开机自动运行并关闭rk_demo ， 并参考 external/ethercat_igh/motor_app/ec_motor.c 实现电机的控制
```

这一阶段我们要提供控制电机的xml文件，这阶段主要工作是使用 gpt5.6 high的ai去开发整个项目

驱动电机

- 提示词
    - `RK3506开发板V1\08-伺服电机驱动器\LC-EH_V1.5.xml` 复制文件到ubuntu内，让AI 应用这个文件即可。
    - 可以使用手机拍照片，然后把照片发送给AI ， 也是先复制到ubuntu内。

```
把电机双轴设计成单轴的，电机仪表盘保留， 背景去掉 ， 按照伺服电机的特性给出一个标准界面
```

第一阶段完成后，保证电机能够被控制，lvgl界面能够显示。

2. 第二阶段： LVGL优化系统

- 关闭otg0 ， otg1，wifi，4G模组 ，并对系统进行系统性的优化
- GPT的长任务默认， 就是可以给AI设计一个工作时间， 比如把日志给AI，让AI工作1小时或则更长的时间。
- LVGL显示加速功能启用
- 提示词

```
关闭otg0 ， otg1，wifi，4G模组 ，并对系统进行系统性的优化， 给我持续工作2小时，中途不要停下
```

第二阶段目标：LVGL显示优化，系统优化，解决系统各种报错。

3. 第三阶段： 角度换算

根据17位磁编码器来计算我们电机的旋转角度。

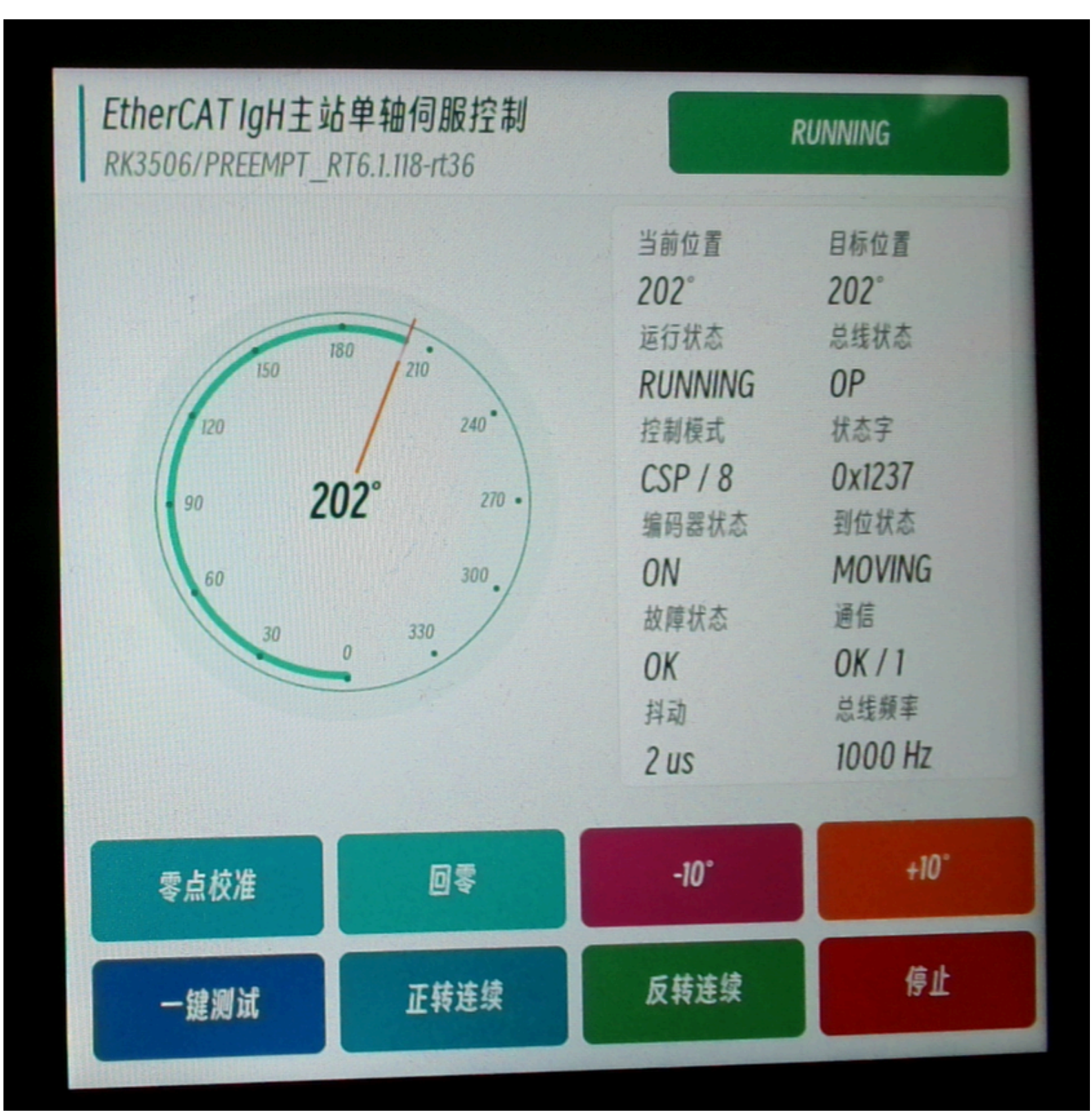
```
DC48V-750W.pdf
用户使用手册25-12-18.pdf
DS2-E伺服操作手册.pdf
LC-EH_V1.5.xml
```

- 根据编码器的输出位数来确定我们的旋转的角度。

4. 第四阶段： 操作系统系统的实时性提升

通过各种手段，提供操作系统的实时性，让EtherCAT总线的抖动降到最低，4us以内吧。

- **1KHZ/1ms方案**
- 软件效果
    - 抖动控制在4us以内

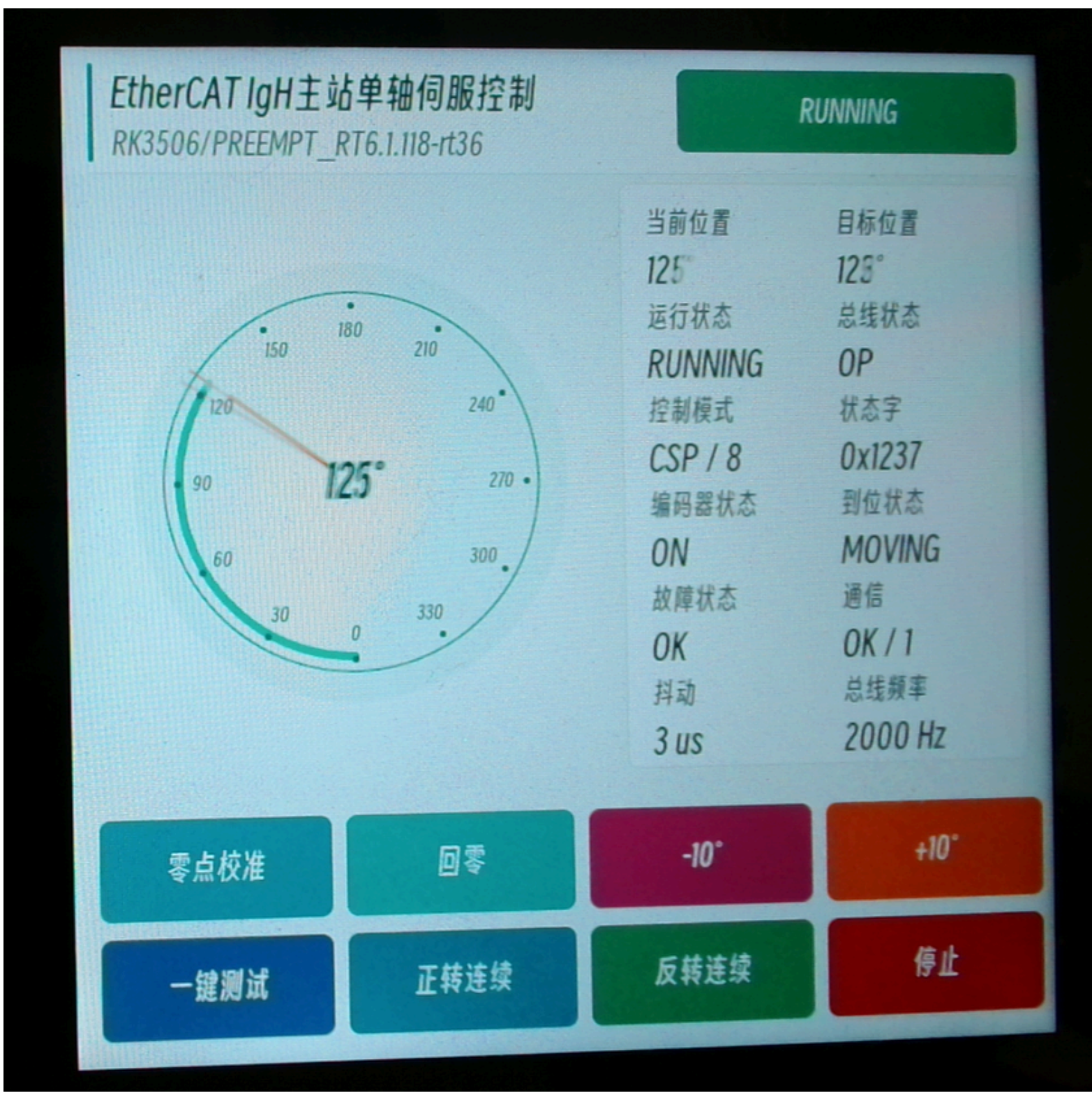


- **2KHZ 500us方案**

```
最终方案

1. EtherCAT 使用 ec_stmmac 原生驱动，不再依赖普通 eth0 netdev 状态。

DEVICE_MODULES="stmmac"
MASTER0_DEVICE="ff:ff:ff:ff:ff:ff"
MASTER_MODULE_OPTIONS="thread_cpu=2 idle_thread_rt_prio=15 op_thread_rt_prio=42"

2. GMAC0 由 EtherCAT 专用驱动绑定。

ff4c8000_driver = rk_gmac-dwmac-ethercat
ec_stmmac loaded
normal eth0 link state is not required

3. CPU 隔离和线程分配保持如下：

CPU2: EtherCAT + servo_rt 实时控制
servo_rt       SCHED_FIFO 70
EtherCAT-OP    SCHED_FIFO 42
EtherCAT-IDLE  SCHED_FIFO 15
CPU0: servo_backend
CPU1: motor_demo / UI

4. 启动流程调整：先完成 EtherCAT process data prime，再启动 servo_rt 实时线程。

关键日志：

Priming EtherCAT process data before starting servo_rt

这样避免 servo_rt 在 EtherCAT OP FSM 进入 OP 前抢占 CPU2。

5. 保留诊断能力，便于后续现场排查：
```

```
servo-op-activate
servo-op-request
servo-op-fsm ready
servo-op-fsm blocked
diag cycle=...

当前验收指标

正常状态应看到：

servo_rt_check 20260727-gmac-watchdog-v13
Link: UP
Lost frames: 0
al_state 0x08 (OP)
domain_wc 3
flag.bus_op yes
servo_rt FIFO 70 on CPU2
EtherCAT-OP FIFO 42 on CPU2
jitter / wakeup latency <= 5 us
```

- 运行效果
  - 抖动控制在5us内



- **4KHZ 250us方案**

可行性分析

```
- 4 kHz 方案 有条件可行，但不能直接用当前 2 kHz 参数硬切。

  可行点

  - ec_stmmac 原生 EtherCAT 路径已经稳定，Lost frames=0，GMAC watchdog 已解决。
  - 100 Mbps EtherCAT 带宽足够。当前 2 kHz 约 117 KB/s，4 kHz 约翻倍到 230 KB/s，带宽不是瓶颈。
  - 当前抖动能压到 5 us 内，说明 CPU 隔离、RT 内核、DC 同步基础是对的。

  主要风险

  - 当前周期是 500 us，4 kHz 周期只有 250 us。
  - 当前配置里 wakeup_spin=480 us，这个在 4 kHz 下必须改，建议先降到 200~220 us 以内。
  - 之前日志里 cycle_exec_max_us 出现过 300~400 us，如果这是稳态值，则 4 kHz 不可行；如果只是启动/诊断阶段峰值，则还有机
    会。

  - servo_rt FIFO70 和 EtherCAT-OP FIFO42 同在 CPU2，4 kHz 下更容易互相挤占，需要确认稳态执行时间。

  建议验证路径

  1. 先不要上电机运动，只切 cycle=4000 Hz / 250000 ns。
  2. sync0_shift 改为约 125000 ns。
  3. wakeup_spin 改为 200000~220000 ns。
  4. 关闭高频日志，只保留统计。
  5. 看这几个指标：

  deadline_misses = 0
  Lost frames = 0
  domain_wc = 3
  al_state = OP
  wakeup_latency_max_us <= 5
  cycle_exec_max_us < 150 us 更稳
  period_us 约 249..251

  结论

  4 kHz 可以试，但当前方案需要调参和压缩实时线程执行时间。判断标准很简单：如果稳态 cycle_exec_max_us 能压到 150 us 以
  下，4 kHz 可作为正式方案；如果仍接近或超过 250 us，就应维持 2 kHz。
```

- 运行结论

```
- 这次 4K 已经真正生效了，界面显示 4K 是对的：

  - 启动：cycle_hz=4000
  - 驱动：cycle=4000 Hz/250000 ns
  - socket：cycle_hz 4000
  - EtherCAT：Tx/Rx frame rate 4000
  - 丢包：Lost frames: 0

  原故障不是网卡，也不是 UI，而是驱动器拒绝 250us DC Sync0 周期。

  关键证据在 kernel log：

  EtherCAT ERROR 0-0: AL status message 0x001A: "Synchronization error"
  EtherCAT ERROR 0-0: Failed to set SAFEOP state, slave refused state change
  EtherCAT ERROR 0-0: AL status message 0x0036: "DC Sync0 Cycle Time"

  这说明主站按 4K 配了：

  cycle=4000 Hz/250000 ns
  sync0_shift=125000 ns

  但 DS2-EC 从站认为这个 SYNC0 cycle time = 250000ns 不合法或当前参数不匹配，所以无法从 PREOP 进入 SAFEOP/OP。因此：

  al_state 0x02 (PREOP)
  domain_wc 0
  status_word 0x0000
  bus_op no

  也就是说现在 4K 帧在链路上能跑，但驱动器没有进入 EtherCAT OP，PDO 控制没有真正闭环。

  另外还有一个次要问题：4K 当前实时余量偏紧。

  period_us 152..391
  wakeup_latency_max_us 142
  cycle_exec_max_us 355

  目标周期是 250us，但执行最大已经到 355us，周期抖动也很大。即使驱动器接受 250us Sync0，这个用户态 4K 周期也还需要继续压
  缩执行时间和调度抖动。

  结论：

  1. GMAC / EtherCAT native stmmac 方案没问题，4K 帧率跑起来了，0 丢包。
  2. 当前 4K 卡在从站 DC 配置，根因是 AL 0x0036 DC Sync0 Cycle Time。
  3. DS2-EC 很可能不支持当前 250us Sync0，或者需要额外配置驱动器内部插补/同步周期参数。
  4. 2K 仍是当前可控稳定方案；4K 要继续，需要先确认 DS2-EC 是否支持 250us DC Sync0，再做驱动器 SDO/厂商参数配置和实时余
     量优化。
```

航哥嵌入式 | 获取课程服务和硬件 | 微信/抖音店铺：yhcoder

5. 第五阶段： 工程源码打包

- 对整个项目进行瘦身， 裁剪内核，去除多余的模块，压缩文件系统的尺寸

名称	修改日期	类型	大小
boot.img	7/27 10:57	光盘映像文件	3,935 KB
MiniLoaderAll.bin	7/27 10:57	BIN 文件	275 KB
misc.img	7/27 10:57	光盘映像文件	48 KB
oem.img	7/27 10:57	光盘映像文件	6,528 KB
parameter.txt	7/27 10:57	文本文档	1 KB
recovery.img	7/27 10:57	光盘映像文件	15,384 KB
rootfs.img	7/27 10:57	光盘映像文件	9,600 KB
uboot.img	7/27 10:57	光盘映像文件	4,096 KB
update.img	7/27 10:57	光盘映像文件	42,581 KB
userdata.img	7/27 10:57	光盘映像文件	2,432 KB

- 还有清空过程文件，保留buildroot下的dl目录文件，git提交代码， 对源码进行打包。

```
linux@ubuntu:~/rk3506$ ls
app        common  external  kernel-6.1   prebuilts                    rkbin       rtos    yocto
buildroot  device  hal       Makefile     README.md                    rkflash.sh  tools
build.sh   docs    kernel    oebuild.log  rk3506-src-654c9f428.tar.gz  rootfs      u-boot
linux@ubuntu:~/rk3506$
```